



OpenShift Container Platform 4.22

Hosted control planes

Using hosted control planes with OpenShift Container Platform

OpenShift Container Platform 4.22 Hosted control planes

Using hosted control planes with OpenShift Container Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides instructions for managing hosted control planes for OpenShift Container Platform. With hosted control planes, you create control planes as pods on a management cluster without the need for dedicated physical or virtual machines for each control plane.

Table of Contents

CHAPTER 1. HOSTED CONTROL PLANES RELEASE NOTES	12
1.1. NEW FEATURES AND ENHANCEMENTS	12
1.2. FIXED ISSUES	12
1.3. TECHNOLOGY PREVIEW FEATURES STATUS	17
1.4. KNOWN ISSUES	18
CHAPTER 2. HOSTED CONTROL PLANES OVERVIEW	20
2.1. INTRODUCTION TO HOSTED CONTROL PLANES	20
2.1.1. Architecture of hosted control planes	20
2.1.2. Benefits of hosted control planes	21
2.2. DIFFERENCES BETWEEN HOSTED CONTROL PLANES AND OPENSIFT CONTAINER PLATFORM	22
2.2.1. Cluster creation and lifecycle	22
2.2.2. Cluster configuration	22
2.2.3. etcd encryption	22
2.2.4. Operators and control plane	22
2.2.5. Updates	23
2.2.6. Machine configuration and management	23
2.2.7. Networking	24
2.2.8. Web console	25
2.3. RELATIONSHIP BETWEEN HOSTED CONTROL PLANES, MULTICLUSTER ENGINE OPERATOR, AND RHACM	25
2.3.1. Hosted clusters in Red Hat Advanced Cluster Management	26
2.4. VERSIONING FOR HOSTED CONTROL PLANES	26
2.4.1. Management cluster	27
2.4.2. HyperShift Operator	27
2.4.3. hosted control planes CLI	27
2.4.4. hypershift.openshift.io API	28
2.4.5. Control Plane Operator	28
2.5. GLOSSARY OF COMMON CONCEPTS AND PERSONAS FOR HOSTED CONTROL PLANES	28
2.5.1. Concepts	28
2.5.2. Personas	29
CHAPTER 3. PREPARING TO DEPLOY HOSTED CONTROL PLANES	31
3.1. REQUIREMENTS FOR HOSTED CONTROL PLANES	31
3.1.1. Support matrix for hosted control planes	31
3.1.1.1. Management cluster support	31
3.1.1.2. Hosted cluster support	32
3.1.1.3. Hosted cluster platform support	33
3.1.1.4. Multi-architecture support	34
3.1.1.5. Updates of multicluster engine Operator	35
3.1.1.6. Technology Preview features	36
3.1.2. Additional resources	36
3.1.3. FIPS-enabled hosted clusters	37
3.1.4. CIDR ranges for hosted control planes	37
3.2. SIZING GUIDANCE FOR HOSTED CONTROL PLANES	37
3.2.1. Pod limits	38
3.2.2. Request-based resource limit	38
3.2.3. Load-based limit	38
3.2.4. Sizing calculation example	39
3.2.5. Shared infrastructure between hosted and standalone control planes	41
3.3. OVERRIDING RESOURCE UTILIZATION MEASUREMENTS	42
3.3.1. Overriding resource utilization measurements for a hosted cluster	42

3.3.2. Disabling the metric service monitoring	43
3.4. INSTALLING THE HOSTED CONTROL PLANES COMMAND-LINE INTERFACE	44
3.4.1. Installing the hosted control planes command-line interface from the terminal	44
3.4.2. Installing the hosted control planes command-line interface by using the web console	45
3.4.3. Installing the hosted control planes command-line interface by using the content gateway	46
3.5. DISTRIBUTING HOSTED CLUSTER WORKLOADS	47
3.5.1. Node labeling for hosted control planes	47
3.5.2. Labeling management cluster nodes	48
3.5.3. Priority classes	49
3.5.4. Custom taints and tolerations	49
3.5.5. Control plane isolation	50
3.5.5.1. Control plane pod isolation	50
3.6. ENABLING OR DISABLING THE HOSTED CONTROL PLANES FEATURE	51
3.6.1. Manually enabling the hosted control planes feature	51
3.6.2. Manually enabling the hypershift-addon managed cluster add-on for local-cluster	52
3.6.3. Uninstalling the HyperShift Operator	53
3.6.4. Disabling the hosted control planes feature	53
CHAPTER 4. DEPLOYING HOSTED CONTROL PLANES	55
4.1. DEPLOYING HOSTED CONTROL PLANES ON AWS	55
4.1.1. Preparing to deploy hosted control planes on AWS	55
4.1.1.1. Prerequisites to deploy hosted control planes on AWS	55
4.1.1.2. Creating the Amazon Web Services S3 bucket and S3 OIDC secret	56
4.1.1.3. Creating a routable public zone for hosted clusters	58
4.1.1.4. Creating an AWS IAM role and STS credentials	58
4.1.1.5. Enabling AWS PrivateLink for hosted control planes	62
4.1.2. Enabling external DNS for hosted control planes on AWS	63
4.1.2.1. Setting up external DNS for hosted control planes	64
4.1.2.2. Creating the public DNS hosted zone	65
4.1.2.3. Creating a hosted cluster by using the external DNS on AWS	67
4.1.2.4. Defining a custom DNS name	69
4.1.3. Creating a hosted cluster on AWS	70
4.1.3.1. Creating a hosted cluster on AWS by using the CLI	71
4.1.3.2. Creating a hosted cluster by providing AWS STS credentials	73
4.1.3.3. Creating a hosted cluster in multiple zones on AWS	74
4.1.4. Accessing a hosted cluster on AWS	75
4.1.5. Running hosted clusters on an ARM64 architecture	76
4.1.5.1. Creating a hosted cluster on an ARM64 OpenShift Container Platform cluster	77
4.1.5.2. Creating an ARM or AMD NodePool object on AWS hosted clusters	78
4.1.6. Creating a private hosted cluster on AWS	79
4.1.6.1. Accessing a private hosted cluster on AWS	80
4.2. DEPLOYING HOSTED CONTROL PLANES ON AZURE	81
4.2.1. Overview of hosted control planes on Azure	81
4.2.1.1. Architecture of hosted control planes on Azure	81
4.2.1.2. Deployment workflow for hosted control planes on Azure	82
4.2.1.3. Azure infrastructure and credentials	83
4.2.1.3.1. Summary of requirements	83
4.2.1.3.2. DNS management options	83
4.2.1.3.3. Resource group strategy	84
4.2.1.3.4. Security considerations for hosted control planes on Azure	84
4.2.2. Setting up Azure resources	84
4.2.2.1. Setting up an OIDC issuer	84
4.2.2.2. Creating Azure Workload Identities	86

4.2.2.3. Creating Azure infrastructure	88
4.2.3. Configuring an Azure management cluster for hosted control planes	90
4.2.4. Creating a hosted cluster on Azure	93
4.2.5. Private hosted clusters on Azure	95
4.2.5.1. Preparing a subnet for a private hosted cluster on Azure	95
4.2.5.2. Installing the HyperShift Operator with private platform support	97
4.2.5.3. Configuring IAM resources for a private hosted cluster	99
4.2.5.4. Creating infrastructure for a private hosted cluster	100
4.2.5.5. Creating a private Azure hosted cluster	101
4.2.5.6. Accessing a private Azure hosted cluster	104
4.2.5.7. Troubleshooting private hosted clusters on Azure	105
4.2.6. Example: Autoscaling a hosted cluster on Azure	106
4.2.7. Deleting an Azure hosted cluster and its resources	107
4.2.7.1. Deleting a hosted cluster on Azure	107
4.2.7.2. Deleting Azure infrastructure	108
4.2.7.3. Deleting Azure Workload Identities	109
4.2.7.4. Deleting a private hosted cluster on Azure	110
4.3. DEPLOYING HOSTED CONTROL PLANES ON BARE METAL WITH THE AGENT PLATFORM	110
4.3.1. About hosted control planes on bare metal with the Agent platform	110
4.3.2. Preparing to deploy hosted control planes on bare metal	111
4.3.2.1. Bare metal firewall, port, and service requirements	111
4.3.2.2. Bare metal infrastructure requirements	113
4.3.3. Configuring a management cluster for bare metal	113
4.3.4. DNS configurations on bare metal	113
4.3.4.1. Defining a custom DNS name	115
4.3.5. Creating an InfraEnv resource	116
4.3.5.1. Creating an InfraEnv resource and adding nodes	116
4.3.5.2. Creating an InfraEnv resource by using the console	122
4.3.6. Creating a hosted cluster on bare metal	122
4.3.6.1. Creating a hosted cluster by using the CLI	122
4.3.6.2. Creating a hosted cluster on bare metal by using the console	129
4.3.6.3. Creating a hosted cluster on bare metal by using a mirror registry	130
4.3.7. Verifying hosted cluster creation	132
4.4. DEPLOYING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION	133
4.4.1. Prerequisites to deploy hosted control planes on OpenShift Virtualization	133
4.4.1.1. OpenShift Virtualization firewall and port requirements	135
4.4.2. Live migration for compute nodes	136
4.4.3. Configuring MetalLB	137
4.4.4. Hosted clusters with the KubeVirt platform	138
4.4.4.1. Creating a hosted cluster with the KubeVirt platform by using the CLI	138
4.4.4.2. Creating a hosted cluster with the KubeVirt platform by using external infrastructure	140
4.4.4.3. Creating a hosted cluster by using the console	141
4.4.5. Configuring the default ingress and DNS for hosted control planes on OpenShift Virtualization	143
4.4.5.1. Defining a custom DNS name	143
4.4.6. Customized ingress and DNS behavior	145
4.4.6.1. Creating a hosted cluster that specifies the base domain	145
4.4.6.2. Setting up the load balancer	146
4.4.6.3. Setting up a wildcard DNS	148
4.4.7. Additional networks, guaranteed CPUs, and VM scheduling for node pools	148
4.4.7.1. Adding multiple networks to a node pool	149
4.4.7.1.1. Using an additional network as default	149
4.4.7.2. Requesting guaranteed CPU resources	150
4.4.7.3. Scheduling KubeVirt VMs on a set of nodes	151

4.4.8. Scaling a node pool	151
4.4.8.1. Adding node pools	152
4.4.9. Verifying hosted cluster creation on OpenShift Virtualization	154
4.5. DEPLOYING HOSTED CONTROL PLANES ON NON-BARE-METAL AGENT MACHINES	155
4.5.1. Preparing to deploy hosted control planes on non-bare-metal agent machines	156
4.5.1.1. Prerequisites for deploying hosted control planes on non-bare-metal agent machines	156
4.5.1.2. Firewall, port, and service requirements for non-bare-metal agent machines	157
4.5.1.3. Infrastructure requirements for non-bare-metal agent machines	158
4.5.2. DNS configuration on non-bare-metal agent machines	158
4.5.3. Defining a custom DNS name	159
4.5.4. Creating a hosted cluster on non-bare-metal agent machines by using the CLI	161
4.5.5. Creating a hosted cluster on non-bare-metal agent machines by using the web console	163
4.5.6. Creating a hosted cluster on non-bare-metal agent machines by using a mirror registry	164
4.5.7. Verifying hosted cluster creation on non-bare-metal agent machines	165
4.6. DEPLOYING HOSTED CONTROL PLANES ON IBM Z	166
4.6.1. Prerequisites to configure hosted control planes on IBM Z	166
4.6.2. IBM Z infrastructure requirements	167
4.6.3. DNS configuration for hosted control planes on IBM Z	167
4.6.3.1. Defining a custom DNS name	168
4.6.4. Creating a hosted cluster on bare metal for IBM Z	170
4.6.5. Creating an InfraEnv resource for hosted control planes on IBM Z	172
4.6.6. Adding IBM Z KVM as agents	172
4.6.7. Adding IBM Z LPAR as agents	174
4.6.7.1. Adding IBM z/VM as agents	175
4.6.8. Scaling the NodePool object for a hosted cluster on IBM Z	177
4.7. DEPLOYING HOSTED CONTROL PLANES ON IBM POWER	179
4.7.1. Prerequisites to configure hosted control planes on IBM Power	180
4.7.2. IBM Power infrastructure requirements	180
4.7.3. DNS configuration for hosted control planes on IBM Power	181
4.7.3.1. Defining a custom DNS name	181
4.7.4. Creating a hosted cluster by using the CLI	183
4.7.5. About creating heterogeneous node pools on agent hosted clusters	190
4.7.5.1. Creating the AgentServiceConfig custom resource	190
4.7.5.2. Create an agent cluster	191
4.7.5.3. Creating heterogeneous node pools	192
4.7.5.4. DNS configuration for hosted control planes	193
4.7.5.5. Creating infrastructure environment resources	193
4.7.5.6. Adding agents to the heterogeneous cluster	195
4.7.5.7. Scaling the node pool	196
4.8. DEPLOYING HOSTED CONTROL PLANES ON OPENSTACK	197
4.8.1. Prerequisites for OpenStack	198
4.8.2. Preparing the management cluster for etcd local storage	199
4.8.3. Creating a floating IP for ingress	201
4.8.4. Uploading the RHCOS image to OpenStack	201
4.8.5. Creating a hosted cluster on OpenStack	202
4.8.5.1. Options for creating a Hosted Control Planes cluster on OpenStack	203
CHAPTER 5. MANAGING HOSTED CONTROL PLANES	205
5.1. MANAGING HOSTED CONTROL PLANES ON AWS	205
5.1.1. Prerequisites to manage AWS infrastructure and IAM permissions	205
5.1.1.1. Infrastructure requirements for AWS	205
5.1.1.2. Unmanaged infrastructure for the HyperShift Operator in an AWS account	205
5.1.1.3. Unmanaged infrastructure requirements for a management AWS account	206

5.1.1.4. Infrastructure requirements for a management AWS account	206
5.1.1.5. Infrastructure requirements for an AWS account in a hosted cluster	207
5.1.1.6. Kubernetes-managed infrastructure in a hosted cluster AWS account	207
5.1.2. Identity and Access Management (IAM) permissions for hosted control planes on AWS	208
5.1.3. Separate creation of the hosted cluster and its resources	214
5.1.3.1. Creating the AWS infrastructure separately	214
5.1.3.2. Creating a hosted cluster separately	218
5.1.4. Transitioning a hosted cluster from single-architecture to multi-architecture	219
5.1.5. Creating node pools on the multi-architecture hosted cluster	222
5.1.6. Adding or updating AWS tags for a hosted cluster	223
5.1.7. Configuring node pool capacity blocks on AWS	225
5.1.7.1. Deleting a hosted cluster after configuring node pool capacity blocks	228
5.1.8. Amazon Spot Instance support for node pools	228
5.1.8.1. Configuring Amazon SQS and Amazon EventBridge to receive Amazon EC2 interruption events	228
5.1.8.2. Enabling Amazon Spot Instance support for node pools	230
5.2. MANAGING HOSTED CONTROL PLANES ON BARE METAL	232
5.2.1. Accessing the hosted cluster	232
5.2.2. Scaling the NodePool object for a hosted cluster	233
5.2.2.1. Adding node pools	235
5.2.2.2. Enabling node auto-scaling for the hosted cluster	236
5.2.2.3. Disabling node auto-scaling for the hosted cluster	238
5.2.3. Handling ingress in a hosted cluster on bare metal	238
5.2.4. Enabling machine health checks on bare metal	241
5.2.5. Disabling machine health checks on bare metal	242
5.3. MANAGING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION	243
5.3.1. Accessing the hosted cluster	243
5.3.2. Enabling node auto-scaling for the hosted cluster	243
5.3.3. Storage for hosted control planes on OpenShift Virtualization	245
5.3.3.1. Mapping KubeVirt CSI storage classes	246
5.3.3.2. Mapping a single KubeVirt CSI volume snapshot class	248
5.3.3.3. Mapping multiple KubeVirt CSI volume snapshot classes	248
5.3.3.4. Configuring KubeVirt VM root volume	249
5.3.3.5. Enabling KubeVirt VM image caching	250
5.3.3.6. KubeVirt CSI storage security and isolation	251
5.3.3.7. Configuring etcd storage	251
5.3.4. Attaching NVIDIA GPU devices by using the hcp CLI	252
5.3.5. Attaching NVIDIA GPU devices by using the NodePool resource	253
5.3.6. Evicting KubeVirt virtual machines	255
5.3.7. Spreading node pool VMs by using topologySpreadConstraint	256
5.4. MANAGING HOSTED CONTROL PLANES ON NON-BARE-METAL AGENT MACHINES	257
5.4.1. Accessing the hosted cluster	258
5.4.2. Scaling the NodePool object for a hosted cluster	258
5.4.2.1. Adding node pools	261
5.4.2.2. Enabling node auto-scaling for the hosted cluster	262
5.4.2.3. Disabling node auto-scaling for the hosted cluster	263
5.4.3. Handling ingress in a hosted cluster on non-bare-metal agent machines	264
5.4.4. Enabling machine health checks on non-bare-metal agent machines	267
5.4.5. Disabling machine health checks on non-bare-metal agent machines	268
5.5. MANAGING HOSTED CONTROL PLANES ON IBM POWER	268
5.5.1. Creating an InfraEnv resource for hosted control planes on IBM Power	268
5.5.2. Adding IBM Power agents to the InfraEnv resource	269
5.5.3. Scaling the NodePool object for a hosted cluster on IBM Power	270
5.6. MANAGING HOSTED CONTROL PLANES ON RHOSP	272

5.6.1. Accessing the hosted cluster	272
5.6.2. Enabling node auto-scaling for the hosted cluster	273
5.6.3. Configuring node pools for availability zones	274
5.6.4. Additional ports for node pools	276
5.6.4.1. Options for additional ports for node pools	276
5.6.4.2. Creating additional ports for node pools	277
5.6.5. Node tuning for hosted clusters on RHOSP	278
5.6.5.1. Tuning performance for hosted cluster nodes	278
5.6.5.2. Enabling the SR-IOV Network Operator in a hosted cluster	281
CHAPTER 6. DEPLOYING HOSTED CONTROL PLANES IN A DISCONNECTED ENVIRONMENT	282
6.1. INTRODUCTION TO HOSTED CONTROL PLANES IN A DISCONNECTED ENVIRONMENT	282
6.2. DEPLOYING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION IN A DISCONNECTED ENVIRONMENT	282
6.2.1. Configuring image mirroring for hosted control planes in a disconnected environment	283
6.2.2. Applying objects in the management cluster	285
6.2.3. Deploying multicluster engine Operator for a disconnected installation of hosted control planes	286
6.2.4. TLS certificates for a disconnected installation of hosted control planes	286
6.2.4.1. Adding the registry CA to the management cluster	287
6.2.4.2. Adding the registry CA to the compute nodes for the hosted cluster	287
6.2.5. Creating a hosted cluster on OpenShift Virtualization in a disconnected environment	288
6.2.5.1. Prerequisites to deploy hosted control planes on OpenShift Virtualization	288
6.2.5.2. Creating a hosted cluster with the KubeVirt platform by using the CLI	290
6.2.6. Configuring the default ingress and DNS for hosted control planes on OpenShift Virtualization	291
6.2.7. Customize ingress and DNS behavior	292
6.2.8. Creating a hosted cluster that specifies the base domain	292
6.2.9. Setting up the load balancer	294
6.2.10. Setting up a wildcard DNS	295
6.2.11. Deployment finalization	296
6.2.11.1. Monitoring the control plane	296
6.2.11.2. Monitoring the data plane	297
6.3. DEPLOYING HOSTED CONTROL PLANES ON BARE METAL IN A DISCONNECTED ENVIRONMENT	297
6.3.1. Disconnected environment architecture for bare metal	297
6.3.2. Requirements to deploy hosted control planes on bare metal in a disconnected environment	301
6.3.3. Extracting the release image digest	302
6.3.4. DNS configurations on bare metal	302
6.3.5. Deploying a registry for hosted control planes in a disconnected environment	303
6.3.6. Setting up a management cluster for hosted control planes in a disconnected environment	306
6.3.7. Configuring the web server for hosted control planes in a disconnected environment	307
6.3.8. Configuring image mirroring for hosted control planes in a disconnected environment	307
6.3.9. Applying objects in the management cluster	310
6.3.10. Deploying AgentServiceConfig resources	311
6.3.11. Adding the registry CA to the management cluster	314
6.3.12. Adding the registry CA to the compute nodes for the hosted cluster	314
6.3.13. Hosted clusters on bare metal in a disconnected environment	315
6.3.13.1. Deploying hosted cluster objects	315
6.3.13.2. Creating a NodePool object for the hosted cluster	320
6.3.13.3. Creating an InfraEnv resource for the hosted cluster	321
6.3.13.4. Creating bare-metal hosts for the hosted cluster	322
6.3.13.5. Scaling up the node pool	324
6.3.14. Additional resources	325
6.4. DEPLOYING HOSTED CONTROL PLANES ON IBM Z IN A DISCONNECTED ENVIRONMENT	325
6.4.1. Prerequisites to deploy hosted control planes on IBM Z in a disconnected environment	326

6.4.2. Adding credentials and the registry certificate authority to the management cluster	326
6.4.3. Update the registry certificate authority in the AgentServiceConfig resource with the mirror registry	327
6.4.4. Adding the registry certificate authority to the hosted cluster	329
6.5. MONITORING USER WORKLOAD IN A DISCONNECTED ENVIRONMENT	330
6.5.1. Resolving user workload monitoring issues	330
6.5.2. Verifying the status of the hosted control plane feature	330
6.5.3. Configuring the hypershift-addon managed cluster add-on to run on an infrastructure node	331
CHAPTER 7. CONFIGURING CERTIFICATES FOR HOSTED CONTROL PLANES	333
7.1. CONFIGURING A CUSTOM API SERVER CERTIFICATE IN A HOSTED CLUSTER	333
7.2. CONFIGURING THE KUBERNETES API SERVER FOR A HOSTED CLUSTER	334
7.3. OAUTH SERVER CERTIFICATES FOR HOSTED CONTROL PLANES	336
7.4. CONFIGURING OAUTH SERVER CERTIFICATES FOR A HOSTED CLUSTER	337
7.5. TROUBLESHOOTING ACCESSING A HOSTED CLUSTER BY USING A CUSTOM DNS	340
CHAPTER 8. UPDATING HOSTED CONTROL PLANES	341
8.1. REQUIREMENTS TO UPGRADE HOSTED CONTROL PLANES	341
8.2. HOSTED CLUSTER AND NODE POOL VERSION SKEW POLICY	341
8.2.1. Version compatibility reporting	342
8.3. SETTING UPDATE CHANNELS IN A HOSTED CLUSTER	342
8.4. UPDATES OF THE OPENSIFT CONTAINER PLATFORM VERSION IN A HOSTED CLUSTER	345
8.4.1. The multicloud engine Operator hub management cluster	345
8.4.2. Supported OpenShift Container Platform versions in a hosted cluster	346
8.5. UPDATES FOR THE HOSTED CLUSTER	347
8.6. UPDATES FOR NODE POOLS	348
8.6.1. Replace updates for node pools	348
8.6.2. In place updates for node pools	348
8.7. UPDATING NODE POOLS IN A HOSTED CLUSTER	348
8.8. UPDATING A CONTROL PLANE IN A HOSTED CLUSTER	349
8.9. UPDATING A HOSTED CLUSTER BY USING THE MULTICLOUD ENGINE OPERATOR CONSOLE	350
CHAPTER 9. HIGH AVAILABILITY FOR HOSTED CONTROL PLANES	352
9.1. ABOUT HIGH AVAILABILITY FOR HOSTED CONTROL PLANES	352
9.1.1. Impact of the failed management cluster component	352
9.2. RECOVERING AN UNHEALTHY ETCD CLUSTER FOR HOSTED CONTROL PLANES	352
9.2.1. Checking the status of an etcd cluster	352
9.2.2. Recovering a failing etcd pod	353
9.3. BACKING UP AND RESTORING ETCD IN AN ON-PREMISE ENVIRONMENT	354
9.3.1. Backing up and restoring etcd on a hosted cluster in an on-premise environment	354
9.4. BACKING UP AND RESTORING ETCD ON THE MANAGEMENT CLUSTER	359
9.4.1. Taking a snapshot of etcd for a hosted cluster	359
9.4.2. Restoring an etcd snapshot on a hosted cluster	361
9.5. BACKING UP AND RESTORING A HOSTED CLUSTER ON OPENSIFT VIRTUALIZATION	366
9.5.1. Backing up a hosted cluster on OpenShift Virtualization	366
9.5.2. Restoring a hosted cluster on OpenShift Virtualization	368
9.6. DISASTER RECOVERY FOR A HOSTED CLUSTER IN AWS	369
9.6.1. Overview of the backup and restore process for hosted control planes on AWS	370
9.6.2. Backing up a hosted cluster on AWS	374
9.6.3. Restoring a hosted cluster	380
9.6.4. Deleting a hosted cluster from your source management cluster	383
9.7. DISASTER RECOVERY FOR A HOSTED CLUSTER BY USING OADP	386
9.7.1. Preparing AWS to use OADP for disaster recovery	386
9.7.2. Preparing bare metal to use OADP for disaster recovery	387
9.7.3. Data plane workload backup	388

9.7.4. Control plane workload backup	388
9.7.4.1. Backing up the control plane workload on AWS	388
9.7.4.2. Backing up the control plane workload on a bare metal	391
9.7.5. Hosted cluster restoration by using OADP	393
9.7.5.1. Restoring a hosted cluster into the same management cluster by using OADP	393
9.7.5.2. Restoring a hosted cluster into a new management cluster by using OADP	396
9.7.6. Observing the backup and restore process	401
9.7.7. Using the Velero CLI to describe the Backup and Restore resources	401
9.8. AUTOMATED DISASTER RECOVERY FOR A HOSTED CLUSTER BY USING OADP	402
9.8.1. Prerequisites to automate disaster recovery by using OADP	402
9.8.2. Configuring OADP to automate disaster recovery for hosted control planes	403
9.8.3. Automation of the backup and restore process with a DPA	403
9.8.3.1. Creating a Data Protection Application for bare metal	403
9.8.3.2. Creating a Data Protection Application for AWS	405
9.8.4. Backing up the data plane workload by using the OADP Operator	406
9.8.5. Backing up the control plane workload	406
9.8.6. Restoring a hosted cluster by using OADP	408
9.8.7. Observing the backup and restore process	410
9.8.8. Using the Velero CLI to describe the Backup and Restore resources	410
9.9. BACKING UP ETCD DATA FOR HOSTED CONTROL PLANES BY USING THE ETCD SNAPSHOT METHOD	411
9.9.1. Overview of the etcd snapshot method	411
9.9.2. Configuring the etcd snapshot method	412
9.9.3. Backing up etcd data by using the etcd snapshot method	413
9.9.4. Restoring etcd data by using the etcd snapshot method	414
9.9.5. Condition codes for the etcd snapshot method	415
CHAPTER 10. AUTHENTICATION AND AUTHORIZATION FOR HOSTED CONTROL PLANES	417
10.1. CONFIGURING THE OAUTH SERVER FOR A HOSTED CLUSTER BY USING THE CLI	417
10.2. CONFIGURING THE OAUTH SERVER FOR A HOSTED CLUSTER BY USING THE WEB CONSOLE	419
10.3. IAM ROLES ASSIGNED BY USING THE CCO IN A HOSTED CLUSTER ON AWS	421
10.3.1. Enabling Operators to support CCO-based workflows with AWS STS	421
10.3.2. Verifying the CCO installation in a hosted cluster on AWS	427
10.4. ADDITIONAL RESOURCES	427
CHAPTER 11. HANDLING MACHINE CONFIGURATION FOR HOSTED CONTROL PLANES	428
11.1. CONFIGURING NODE POOLS FOR HOSTED CONTROL PLANES	428
11.2. REFERENCING THE KUBELET CONFIGURATION IN NODE POOLS	429
11.3. CONFIGURING NODE TUNING IN A HOSTED CLUSTER	430
11.4. DEPLOYING THE SR-IOV OPERATOR FOR HOSTED CONTROL PLANES	432
11.5. CONFIGURING THE NTP SERVER FOR HOSTED CLUSTERS	433
11.6. SCALING UP AND DOWN WORKLOADS IN A HOSTED CLUSTER	437
11.7. SCALING UP WORKLOADS IN A HOSTED CLUSTER	438
11.8. SETTING THE PRIORITY EXPANDER IN A HOSTED CLUSTER	438
11.9. BALANCING IGNORED LABELS IN A HOSTED CLUSTER	440
CHAPTER 12. FEATURE GATES IN A HOSTED CLUSTER	442
12.1. ENABLING FEATURE SETS BY USING FEATURE GATES	442
CHAPTER 13. OBSERVABILITY FOR HOSTED CONTROL PLANES	444
13.1. CONFIGURING METRICS SETS FOR HOSTED CONTROL PLANES	444
13.1.1. SRE metrics set example	444
13.2. CUSTOMIZED HOSTED CLUSTER IDENTIFIERS	447
13.2.1. Example: Setting a custom cluster identifier	447

13.2.1.1. Cluster identifier reuse after a reinstall	448
13.3. ENABLING MONITORING DASHBOARDS IN A HOSTED CLUSTER	448
13.3.1. Dashboard customization	449
13.4. CONNECTIVITY MONITORING FOR HOSTED CONTROL PLANES	449
13.4.1. Connectivity monitoring from the control plane to the data plane	450
13.4.2. Connectivity monitoring from the data plane to the control plane	451
CHAPTER 14. NETWORKING FOR HOSTED CONTROL PLANES	453
14.1. NETWORKING OVERVIEW FOR HOSTED CONTROL PLANES	453
14.1.1. Control plane and data plane networking considerations	453
14.1.2. DHCP requirements for hosted control planes	454
14.2. NETWORK ISOLATION FOR HOSTED CLUSTERS	454
14.2.1. Control plane isolation	454
14.2.1.1. Control plane pod isolation	455
14.3. FIREWALL, PORT, AND SERVICE REQUIREMENTS FOR HOSTED CONTROL PLANES	456
14.3.1. Bare metal firewall, port, and service requirements	456
14.3.2. OpenShift Virtualization firewall and port requirements	457
14.3.3. Firewall, port, and service requirements for non-bare-metal agent machines	458
14.3.4. Example firewall configuration	459
14.4. INGRESS AND EGRESS REQUIREMENTS FOR HOSTED CONTROL PLANES	459
14.4.1. Ingress requirements for hosted control planes	459
14.4.2. Egress requirements for hosted control planes	462
14.4.3. Handling ingress in a hosted cluster on bare metal	463
14.5. CONFIGURING INTERNAL OVN IPV4 SUBNETS FOR HOSTED CLUSTERS	466
14.6. PROXY SUPPORT FOR HOSTED CONTROL PLANES	471
14.6.1. Control plane workloads that need to access external services	472
14.6.2. Compute nodes that need to access an ignition endpoint	472
14.6.3. Compute nodes that need to access the API server	472
14.6.4. Management clusters that need external access	473
14.6.5. Management cluster with a proxy and a hosted cluster with a secondary network and no default pod network	473
CHAPTER 15. TROUBLESHOOTING HOSTED CONTROL PLANES	474
15.1. GATHERING INFORMATION TO TROUBLESHOOT HOSTED CONTROL PLANES	474
15.2. GATHERING OPENSIFT CONTAINER PLATFORM DATA FOR A HOSTED CLUSTER	475
15.2.1. Gathering data for a hosted cluster by using the CLI	475
15.2.2. Gathering data for a hosted cluster by using the web console	476
15.3. ENTERING THE MUST-GATHER COMMAND IN A DISCONNECTED ENVIRONMENT	476
15.4. TROUBLESHOOTING HOSTED CLUSTERS ON OPENSIFT VIRTUALIZATION	477
15.4.1. Troubleshooting HostedCluster resource stuck in a partial state	477
15.4.2. Identifying why no compute nodes are registered	477
15.4.3. Identifying why compute nodes are not ready	478
15.4.4. Identifying why ingress and console cluster Operators are not coming online	478
15.4.5. Identifying why load balancer services for the hosted cluster are unavailable	479
15.4.6. Identifying why hosted cluster PVCs are not available	479
15.4.7. Identifying why VM nodes are not joining the cluster	480
15.4.8. Resolving RHCOS image mirroring failures	480
15.4.9. Returning non-bare-metal clusters to the late binding pool	481
15.5. TROUBLESHOOTING HOSTED CLUSTERS ON BARE METAL	482
15.5.1. Determining why nodes are not added to a hosted cluster on bare metal	482
15.6. RESTARTING HOSTED CONTROL PLANE COMPONENTS	482
15.7. PAUSING THE RECONCILIATION OF A HOSTED CLUSTER AND HOSTED CONTROL PLANE	484
15.8. SCALING DOWN THE DATA PLANE TO ZERO	485

15.9. RESOLVING AGENT SERVICE FAILURES FOR HOSTED CONTROL PLANES ON IBM Z	487
15.10. KNOWN LIMITATIONS FOR INTERNAL SUBNETS FOR HOSTED CLUSTERS	488
15.10.1. Configuring the ovnKubernetesConfig object fails with an error	488
15.10.2. Setting CIDR values in internal subnet fields	488
15.10.3. Ensuring a valid IPv4 CIDR format	489
15.10.4. Avoiding an overlap between OVN subnets and CIDR values	489
15.10.5. Resolving a stuck OVN rollout	490
15.11. TROUBLESHOOTING CONNECTIVITY FOR HOSTED CONTROL PLANES	490
15.11.1. Troubleshooting connectivity from the control plane to the data plane	490
CHAPTER 16. DESTROYING A HOSTED CLUSTER	492
16.1. DESTROYING A HOSTED CLUSTER ON AWS	492
16.1.1. Destroying a hosted cluster on AWS by using the CLI	492
16.2. DESTROYING A HOSTED CLUSTER ON BARE METAL	493
16.2.1. Destroying a hosted cluster on bare metal by using the CLI	493
16.2.2. Destroying a hosted cluster on bare metal by using the web console	493
16.3. DESTROYING A HOSTED CLUSTER ON OPENSIFT VIRTUALIZATION	493
16.3.1. Destroying a hosted cluster on OpenShift Virtualization by using the CLI	493
16.4. DESTROYING A HOSTED CLUSTER ON IBM Z	494
16.4.1. Destroying a hosted cluster on x86 bare metal with IBM Z compute nodes	494
16.5. DESTROYING A HOSTED CLUSTER ON IBM POWER	495
16.5.1. Destroying a hosted cluster on IBM Power by using the CLI	495
16.6. DESTROYING A HOSTED CONTROL PLANE ON OPENSTACK	495
16.6.1. Destroying a hosted cluster by using the CLI	495
16.7. DESTROYING A HOSTED CLUSTER ON NON-BARE-METAL AGENT MACHINES	496
16.7.1. Destroying a hosted cluster on non-bare-metal agent machines	496
16.7.2. Destroying a hosted cluster on non-bare-metal agent machines by using the web console	496
CHAPTER 17. MANUALLY IMPORTING A HOSTED CLUSTER	497
17.1. LIMITATIONS OF MANAGING IMPORTED HOSTED CLUSTERS	497
17.2. MANUALLY IMPORTING HOSTED CLUSTERS	497
17.3. MANUALLY IMPORTING A HOSTED CLUSTER ON AWS	498
17.4. DISABLING THE AUTOMATIC IMPORT OF HOSTED CLUSTERS INTO MULTICLUSTER ENGINE OPERATOR	499

CHAPTER 1. HOSTED CONTROL PLANES RELEASE NOTES

With this release, hosted control planes for OpenShift Container Platform 4.22 is available. Hosted control planes for OpenShift Container Platform 4.22 supports multicluster engine for Kubernetes Operator version 4.22.

1.1. NEW FEATURES AND ENHANCEMENTS

This release adds improvements related to the following components and concepts:

Monitor connectivity from the data plane to the control plane

In this release, you can monitor connectivity from the data plane to the control plane by using the **ControlPlaneConnectionAvailable** condition. For more information, see [Connectivity monitoring from the data plane to the control plane](#).

Implement network segmentation for hosted clusters

In this release, you can configure network isolation for hosted clusters with container-based isolation, VM-based isolation, or physical isolation. For more information, see [Network isolation for hosted clusters](#).

Enable Amazon Spot Instance support

In this release, you can enable Amazon Spot Instance support for compute nodes to reduce cloud infrastructure costs. Amazon Spot Instances are suitable for hosted cluster workloads that are fault-tolerant, stateless, and flexible. For more information, see [Amazon Spot Instance support for node pools](#).

Back up etcd data for hosted control planes by using the etcd snapshot method (Technology Preview)

As an alternative for the default volume snapshot approach, you can use the etcd snapshot approach to back up and restore etcd data for hosted control planes. The etcd snapshot method is a Technology Preview feature. For more information, see [Backing up etcd data for hosted control planes by using the etcd snapshot method](#).

Deploy self-managed hosted control planes on Microsoft Azure (Technology Preview)

In this release, you can create public or private hosted clusters on Azure as a Technology Preview feature. For more information, see [Deploying hosted control planes on Azure](#).

1.2. FIXED ISSUES

The following issues are fixed for this release:

- Before this update, services in the hosted control plane namespace, such as the **aws-ebs-csi-driver-controller-metrics** service, used the **service-ca** annotation (**service.beta.openshift.io/serving-cert-secret-name**) to generate TLS certificates. As a consequence, control plane services incorrectly depended on the OpenShift Service CA Operator in the hosted cluster for certificate generation, which weakened the security boundary between the control plane and the hosted cluster. With this release, the Control Plane Operator creates and manages TLS certificates for the **aws-ebs-csi-driver-controller-metrics** service directly, signed by the hosted control plane root CA, eliminating the dependency on the OpenShift Service CA Operator. The implementation checks for **service-ca** annotations to ensure a smooth upgrade path from older deployments. As a result, control plane isolation and certificate lifecycle management are improved. ([OCPBUGS-34662](#))
- Before this update, the HyperShift Operator metrics collector validated the proxy CA bundle certificates on every metrics collection cycle. As a consequence, when a certificate in the CA

bundle expired, repeated **proxy ca bundle is invalid** messages were posted in the HyperShift Operator logs without identifying the hosted cluster, making it difficult to diagnose the cluster with the invalid proxy CA certificate. With this release, certificate validation is moved to the **HostedCluster** reconcile loop, and a new **ValidProxyConfiguration** condition is added to the **HostedCluster** API. The metrics collector now reads the validation result from the condition instead of directly performing validation. As a result, the metrics collector no longer posts repeated messages in the logs, and affected clusters can be identified. ([OCBUGS-55151](#))

- Before this update, KubeVirt virtual machines (VMs) used in node pools were not configured with an external eviction strategy. As a consequence, the Cluster API Provider for KubeVirt controller did not detect eviction requests during node drains on the underlying infrastructure cluster. Node drains were not coordinated properly, the hosted control plane function was disrupted, and pods failed when node pool VMs were shut down. With this release, KubeVirt VMs are configured with the external eviction strategy in the VM template specification. As a result, the Cluster API Provider for KubeVirt can detect eviction events and coordinate the draining of hosted cluster nodes during infrastructure operations. For VMs that support live migration, the Cluster API Provider for KubeVirt skips the drain process and allows the VMs to be migrated without disruption. ([OCBUGS-58397](#))
- Before this update, when you removed the **additionalTrustBundle** field from the **HostedCluster** specification, the **additionalTrustBundle** certificate was not removed from the **user-ca-bundle** config map. As a consequence, it appeared that the **additionalTrustBundle** certificate was not removed from the hosted clusters. With this release, the reconciliation logic ensures that the **user-ca-bundle** config map is deleted from the hosted cluster when you delete the **additionalTrustBundle** field. As a result, when you delete the **additionalTrustBundle** field from the **HostedCluster** specification, the certificate is removed, improving security and consistency. ([OCBUGS-60707](#))
- Before this update, the control plane deployments related to Cluster API (**cluster-api** and **capi-provider**) in the hosted control plane namespace lacked finalizers. As a consequence, if these deployments were deleted before the **HostedCluster** resource was deleted, the controller pods would stop running before they could process finalizers on their managed Cluster API resources (**Machine** objects, **MachineDeployment** objects, platform-specific infrastructure objects), leading to orphaned cloud resources such as EC2 instances, VMs, disks, and load balancers. With this update, the HyperShift Operator adds a finalizer, **hypershift.openshift.io/component-finalizer**, to the **cluster-api** and **capi-provider** deployments. The finalizer is only removed after the underlying infrastructure resources have been deleted during **HostedCluster** teardown. As a result, accidental deletion of these deployments is blocked until Cluster API resources are properly cleaned up, preventing orphaned cloud resources. ([OCBUGS-63452](#))
- Before this update, when the **ValidAWSIdentityProvider** condition was copied from the control plane to the hosted cluster, the logic preserved the earlier status if the new condition was **Unknown**. As a consequence, when the earlier condition was **True** and the new condition was **Unknown**, the update was skipped. With this release, the condition on the hosted cluster correctly reflects the current health of the AWS Identity Provider referenced in the cloud credentials. ([OCBUGS-66325](#))
- Before this update, the Cluster Network Operator failed to recognize KubeVirt as a supported platform for hosted control planes with dual-stack networking. As a consequence, on deployments of hosted control planes on OpenShift Virtualization with dual-stack networking, the Cluster Network Operator deployment failed. With this release, the Cluster Network Operator recognizes KubeVirt as a supported platform for hosted control planes with dual-stack networking. As a result, deploying hosted control planes on OpenShift Virtualization with IPv4/IPv6 dual-stack networking succeeds. ([OCBUGS-66417](#))

- Before this update, the cluster autoscaler did not include the **hypershift.openshift.io/nodepool-globalps-enabled** label in its **--balancing-ignore-label** list. As a consequence, when the autoscaler balanced node groups, it treated nodes with and without this label as belonging to different groups, causing uneven scaling across nodes in the same **NodePool** object. With this update, the **hypershift.openshift.io/nodepool-globalps-enabled** label is added to the balancing ignore list of the autoscaler. As a result, the autoscaler distributes new nodes evenly across node groups regardless of the Global Pull Secret eligibility label. ([OCPBUGS-73817](#))
- Before this update, when you created a hosted cluster that used a **NodePort** publishing strategy, specifying a port outside the Kubernetes service node port range, such as **10000**, was silently accepted during cluster creation. As a consequence, the cluster installation got stuck with only 3 pods in the hosted cluster namespace, because the Control Plane Operator rejected the port for being outside the acceptable range of **30000 - 32767**, causing a late failure after resources were already provisioned. With this release, early validation is added for the **NodePort.Port** value against the configured **ServiceNodePortRange** parameter of the cluster. Invalid values are rejected upfront with a clear message indicating the allowed range. As a result, you receive an immediate validation error when you specify a **NodePort** that is outside the acceptable range, and avoid stuck cluster installations. ([OCPBUGS-65842](#))
- Before this update, the **hypershift.openshift.io/nodepool-globalps-enabled** label was applied to nodes by the Hosted Cluster Config Operator **globalps** controller, which discovered eligible nodes by querying **MachineSet** objects and **Machine** objects during its periodic reconciliation. As a consequence, when a new **Replace** node joined the cluster, the **global-pull-secret-syncer** DaemonSet pod could not schedule on it until the next reconcile cycle of the **globalps** controller, causing a delay of up to 15 minutes. With this update, the label is set directly on Cluster API **Machine** objects by the HyperShift Operator during **MachineDeployment** reconciliation, so it propagates to nodes at creation time by using the Hosted Cluster Config Operator Node controller. As a result, new **Replace** nodes on AWS are immediately eligible for the **global-pull-secret-syncer** DaemonSet, eliminating the scheduling delay. ([OCPBUGS-77966](#))
- Before this update, the ignition server deployment computed registry overrides by performing live HTTP registry connectivity checks (**LookupMappedImage/GetMetadata**) during every Control Plane Operator reconciliation. As a consequence, network conditions caused the **--registry-overrides** argument and **MIRRORED_RELEASE_IMAGE** environment variable to return different values on each reconciliation, triggering constant deployment regenerations and pod restarts. With this update, the ignition server deployment uses the static registry overrides from the **HostedCluster** specification instead of performing live registry lookups at deploy time. The ignition server already resolves per-image mirrors at runtime by using its own override logic. As a result, ignition server deployments remain stable with consistent configuration, eliminating unnecessary pod restarts. ([OCPBUGS-60185](#))
- Before this update, when the **allowedCIDRBlocks** parameter was removed from the **HostedCluster** specification, the **LoadBalancerSourceRanges** field on the external router **LoadBalancer** service was not cleared. As a consequence, stale Classless Inter-Domain Routing (CIDR) restrictions remained on the router service after the administrator removed the access restrictions, continuing to block traffic that should have been allowed. With this update, the reconciliation logic always sets the **LoadBalancerSourceRanges** field on the external router service to match the current **allowedCIDRBlocks** value, including clearing it when the list is empty. As a result, removing the **allowedCIDRBlocks** parameter from the **HostedCluster** specification correctly removes the CIDR restrictions from the router service. ([OCPBUGS-69761](#))
- Before this update, the **HostedControlPlane** controller set the **HostedControlPlaneAvailable**

condition to **True** after the Kubernetes API server was reachable, without verifying that all control plane components had finished rolling out. As a consequence, customers could interact with the cluster before components such as the **kube-controller-manager**, **oauth-server**, or **kube-scheduler** were fully ready, which could lead to failures or unexpected behavior. With this update, the controller now lists all control plane component resources in the hosted control plane namespace and verifies that each has its **Available** condition set to **True** before setting the **HostedControlPlaneAvailable** condition to **True**. If any components are not yet available, the condition reports the **ComponentsNotAvailable** reason with a message listing the pending components. After the cluster reaches the available state, later component rollouts, such as during upgrades, do not flip the condition back to **False**. As a result, the hosted control plane now only reports **Available=True** after all control plane components have completed their initial rollout, ensuring a more reliable user experience. ([OCPBUGS-74648](#))

- Before this update, the Hosted Cluster Config Operator contained logic that modified the **openshift-controller-manager-config** config map to disable the **serviceaccount-pull-secrets** controller when the **managementState** parameter of the image registry was set to **Removed**. In OpenShift Container Platform 4.20 and later, Control Plane Operator v2 started managing this config map, but the Hosted Cluster Config Operator continued modifying it on every reconciliation cycle. As a consequence, the **openshift-controller-manager-config** config map was updated by Hosted Cluster Config Operator every minute, which triggered the **openshift-controller-manager** file observer to detect changes and restart pods. This behavior caused constant **openshift-controller-manager** pod restarts. With this release, the OpenShift Controller Manager config update logic is removed from the Hosted Cluster Config Operator because Control Plane Operator v2 manages the **openshift-controller-manager-config** config map. As a result, the **openshift-controller-manager** pods no longer experience unnecessary restarts. ([OCPBUGS-74931](#))
- Before this update, during the backup and restore process with OADP, the token secret was deleted before the **NodePool** object was restored. Then, the **NodePool** controller created a token secret without the **ignition-reached** annotation. Because nodes were already running, they did not contact the ignition endpoint again, so the annotation was never set back. As a consequence, the **ReachedIgnitionEndpoint** condition stayed **False**, blocking machine health check creation and disabling auto-repair for the restored node pools. With this release, when the **HostedCluster** object has the **hypershift.openshift.io/restored-from-backup** annotation set by the OADP plugin, the token secret is created with the **ignition-reached=True** parameter, preserving the condition across the restore process. As a result, after a backup and restore process, node pools correctly report **ReachedIgnitionEndpoint=True** so that the machine health check and auto-repair work as expected. ([OCPBUGS-77621](#))
- Before this update, when deploying hosted clusters with a 4.21 or later payload, the HyperShift Operator used hard-coded **quay.io** image references for the Cluster API manager and platform-specific Cluster API provider containers. These hard-coded images bypassed the standard release payload image lookup, which respects **ImageContentSourcePolicies** (ICSPs) and **ImageDigestMirrorSets** (IDMSs). As a consequence, in disconnected or mirrored environments, Cluster API images were always pulled directly from **quay.io** even when registry overrides were configured, causing image pull failures and preventing cluster creation. With this update, the backward-compatible Cluster API image references are resolved by looking up the component from a pinned 4.20.10 release payload through the standard release image provider, which correctly follows registry override configuration. As a result, Cluster API images are pulled from the correct mirror registry in disconnected environments. For this fix to work, the 4.20.10 release payload from the **quay.io/openshift-release-dev/ocp-release:4.20.10-multi** image must be mirrored to the target mirror registry. ([OCPBUGS-74247](#))
- Before this update, requests from the Kubernetes API server bootstrap container were denied

by a validating admission policy that restricts feature gate changes to a specific user. As a consequence, the bootstrap container was unable to apply feature gate changes, causing control plane issues. With this release, a dedicated identity is created for the Kubernetes API server bootstrap container and is allow-listed in the policy. As a result, the bootstrap container can apply feature gate changes without being denied by the validating admission policy. ([OCBUGS-50603](#))

- Before this update, when a predicate of a Control Plane Operator v2 component evaluated to **false**, the framework tried to look up and clean up the associated resource by using the cached client. For resource types not installed on the management cluster, such as the **SecretProviderClass** custom resource definition of the Secrets Store CSI driver, this caused the cached client to create an informer that retried list and watch actions indefinitely, blocking all control plane reconciliation. As a consequence, hosted cluster creation failed on management clusters that did not have the Secrets Store CSI driver custom resource definition installed. With this update, the Control Plane Operator probes whether a resource type is accessible on the management cluster before trying to interact with it. If the custom resource definition is not installed or the operator lacks role-based access permission, the operation is skipped gracefully and the result is cached. As a result, hosted cluster creation succeeds even when optional custom resource definitions such as the Secrets Store CSI driver are not present on the management cluster. ([OCBUGS-65687](#))
- Before this update, when using a custom API server DNS name with external DNS, the **kubeconfig** secret contained an incorrect port. As a consequence, connections to the API server failed with reset errors. With this update, the **kubeconfig** uses the correct port for the configured DNS setup. As a result, external DNS connections work as expected. ([OCBUGS-72258](#))
- Before this update, a race condition in **VolumeSnapshot** processing where a snapshot was deleted between listing and retrieving was treated as an unrecoverable error, ending the processing of remaining snapshots. As a consequence, intermittent backup failures (about 25% of scheduled backups) were marked as **PartiallyFailed** with missing etcd PVC data. With this release, deleted snapshots are gracefully skipped instead of treated as unrecoverable errors, allowing the remaining snapshots to be processed normally. As a result, backups are completed successfully even when snapshot cleanup races with plugin processing. ([OCBUGS-75913](#))
- Before this update, when the scale-from-zero feature was enabled in AWS and a node pool used the **InPlace** node upgrade type with autoscaling set to **min=0**, the scale-from-zero implementation did not support the **InPlace** upgrade strategy. The original implementation used a machine deployment controller approach that only worked with the **Replace** upgrade strategy. As a consequence, new workloads did not trigger node pool scale-up from zero when using the **InPlace** upgrade type, preventing nodes from being created even when pods were pending. With this release, the scale-from-zero implementation uses a generic provider pattern that works with all upgrade types. As a result, node pools that use the **InPlace** upgrade type can scale up from zero when workload demands require additional capacity. The autoscaler correctly provisions nodes regardless of the upgrade strategy. ([OCBUGS-70320](#))
- Before this update, a race condition in the **globalps** controller skipped labeling new nodes, causing a delay in **global-pull-secret-syncer** pod scheduling. As a consequence, users experienced image pull failures from private registries on new nodes. With this release, the scheduling delay is resolved, fixing the race condition in the Hosted Cluster Config Operator. As a result, the **global-pull-secret-syncer** pod now schedules immediately on new nodes, ensuring timely access to private images. ([OCBUGS-77254](#))
- Before this update, the **IsCloudAPI** method for the **Konnectivity** proxy did not include Amazon Web Services (AWS) ISO region domain suffixes, such as **.c2s.ic.gov**, **.hci.ic.gov**, or

.sc2s.sgov.gov in its cloud API detection lists. As a consequence, the Ingress Operator could not add AWS ISO domains to the **NO_PROXY** list, blocking direct communication with endpoints. With this release, the AWS ISO suffixes are added to the **IsCloudAPI** detection list. As a result, the **Konnectivity** proxy correctly identifies the AWS ISO region endpoints as cloud APIs, so that the Ingress Operator can route traffic to those domains directly. ([OCPBUGS-85779](#))

- Before this update, when you deployed a hosted cluster on OpenShift Virtualization with external infrastructure, the **virt-launcher** network policy was not created on the infrastructure cluster where the KubeVirt **virt-launcher** pods and virtual machines (VM)s run. As a consequence, the KubeVirt VMs had unrestricted network access to all pods and services on the infrastructure cluster, breaking tenant isolation. With this release, the **virt-launcher** network policy is created with CIDR-based egress restrictions. As a result, multitenant isolation is no longer compromised. ([OCPBUGS-78575](#))

1.3. TECHNOLOGY PREVIEW FEATURES STATUS

Some features in this release are currently in Technology Preview. These experimental features are not intended for production use. Note the following scope of support on the Red Hat Customer Portal for these features:

Technology Preview Features Support Scope

In the following table, features are marked with the following statuses:

- *Not Available*
- *Technology Preview*
- *General Availability*
- *Deprecated*
- *Removed*



IMPORTANT

For IBM Power and IBM Z, the following exceptions apply:

- For version 4.20 and later, you must run the control plane on machine types that are based on 64-bit x86 architecture or s390x architecture, and node pools on IBM Power or IBM Z.
- For version 4.19 and earlier, you must run the control plane on machine types that are based on 64-bit x86 architecture, and node pools on IBM Power or IBM Z.

Table 1.1. Hosted control planes GA and TP tracker

Feature	4.20	4.21	4.22
Hosted control planes for OpenShift Container Platform using non-bare-metal agent machines	Technology Preview	Technology Preview	Technology Preview

Feature	4.20	4.21	4.22
Hosted control planes for OpenShift Container Platform on RHOSP	Technology Preview	Technology Preview	Technology Preview
Custom taints and tolerations	Technology Preview	Technology Preview	Technology Preview
NVIDIA GPU devices on hosted control planes for OpenShift Virtualization	Technology Preview	Technology Preview	Technology Preview
Hosted control planes for OpenShift Virtualization on IBM Z ^[1]	Not Available	Technology Preview	General Availability
Hosted control planes on IBM Z in a disconnected environment	General Availability	General Availability	General Availability
Hosted control planes for OpenShift Container Platform on Microsoft Azure	Not Available	Not Available	Technology Preview
Backup and restore with the etcd snapshot method	Not Available	Not Available	Technology Preview

1. Hosted control planes for OpenShift Virtualization on IBM Z is supported as Technology Preview starting with OpenShift Container Platform 4.21, multicluster engine for Kubernetes Operator 2.11, and Red Hat Advanced Cluster Management (RHACM) 2.16. Creating hosted control planes with external infrastructure is not supported.

1.4. KNOWN ISSUES

This section includes several known issues for hosted control planes for OpenShift Container Platform 4.22.

- If the annotation and the **ManagedCluster** resource name do not match, the multicluster engine for Kubernetes Operator console displays the cluster as **Pending import**. The cluster cannot be used by the multicluster engine Operator. The same issue happens when there is no annotation and the **ManagedCluster** name does not match the **Infra-ID** value of the **HostedCluster** resource.
- When you use the multicluster engine for Kubernetes Operator console to add a new node pool to an existing hosted cluster, the same version of OpenShift Container Platform might appear more than once in the list of options. You can select any instance in the list for the version that you want.
- When a node pool is scaled down to 0 workers, the list of hosts in the console still shows nodes in a **Ready** state. You can verify the number of nodes in two ways:
 - In the console, go to the node pool and verify that it has 0 nodes.
 - On the command-line interface, run the following commands:
 - Verify that 0 nodes are in the node pool by running the following command:

```
$ oc get nodepool -A
```

- Verify that 0 nodes are in the cluster by running the following command:

```
$ oc get nodes --kubeconfig
```

- Verify that 0 agents are reported as bound to the cluster by running the following command:

```
$ oc get agents -A
```

- When you create a hosted cluster in an environment that uses the dual-stack network, you might encounter pods stuck in the **ContainerCreating** state. This issue occurs because the **openshift-service-ca-operator** resource cannot generate the **metrics-tls** secret that the DNS pods need for DNS resolution. As a result, the pods cannot resolve the Kubernetes API server. To resolve this issue, configure the DNS server settings for a dual stack network.
- If you created a hosted cluster in the same namespace as its managed cluster, detaching the managed hosted cluster deletes everything in the managed cluster namespace including the hosted cluster. The following situations can create a hosted cluster in the same namespace as its managed cluster:
 - You created a hosted cluster on the Agent platform through the multicluster engine for Kubernetes Operator console by using the default hosted cluster namespace.
 - You created a hosted cluster through the command-line interface or API by specifying the hosted cluster namespace to be the same as the hosted cluster name.
- When you use the console or API to specify an IPv6 address for the **spec.services.servicePublishingStrategy.nodePort.address** field of a hosted cluster, a full IPv6 address with 8 hextets is required. For example, instead of specifying **2620:52:0:1306::30**, you need to specify **2620:52:0:1306:0:0:0:30**.
- In hosted control planes on OpenShift Virtualization, if you store all hosted cluster information in a shared namespace and then back up and restore a hosted cluster, you might unintentionally change other hosted clusters. To avoid this issue, back up and restore only hosted clusters that use labels, or avoid storing all hosted cluster information in a shared namespace.
- For version 4.21, hosted control planes pins all Cluster API images to the **4.20.10-multi** release image for compatibility reasons. Hosted control planes pins the images when Cluster API deployments are generated. The **4.20.10-multi** image must always be mirrored and available in order for the Cluster API to work with hosted control planes version 4.21.
- Intermittent egress IP outages occur when a hosted cluster uses the following combined settings:
 - The service publishing strategy for the **Konnectivity** service is set to **Route**.
 - The management cluster uses Virtual Router Redundancy Protocol (VRRP) VIP for ingress.

CHAPTER 2. HOSTED CONTROL PLANES OVERVIEW

You can deploy OpenShift Container Platform clusters by using two different control plane configurations: standalone or hosted control planes.

The standalone configuration uses dedicated virtual machines or physical machines to host the control plane. With hosted control planes for OpenShift Container Platform, you create control planes as pods on a management cluster without the need for dedicated virtual or physical machines for each control plane.

2.1. INTRODUCTION TO HOSTED CONTROL PLANES

Hosted control planes is available by using a supported version of multicloud engine for Kubernetes Operator on several platforms.

You can deploy hosted control planes on the following platforms:

- Bare metal by using the Agent provider
- Non-bare-metal Agent machines, as a Technology Preview feature
- OpenShift Virtualization
- Amazon Web Services (AWS)
- IBM Z
- IBM Power
- Red Hat OpenStack Platform (RHOSP) 17.1, as a Technology Preview feature

The hosted control planes feature is enabled by default.



NOTE

The multicloud engine Operator is an integral part of Red Hat Advanced Cluster Management (RHACM) and is enabled by default with RHACM. However, you do not need RHACM to use hosted control planes.

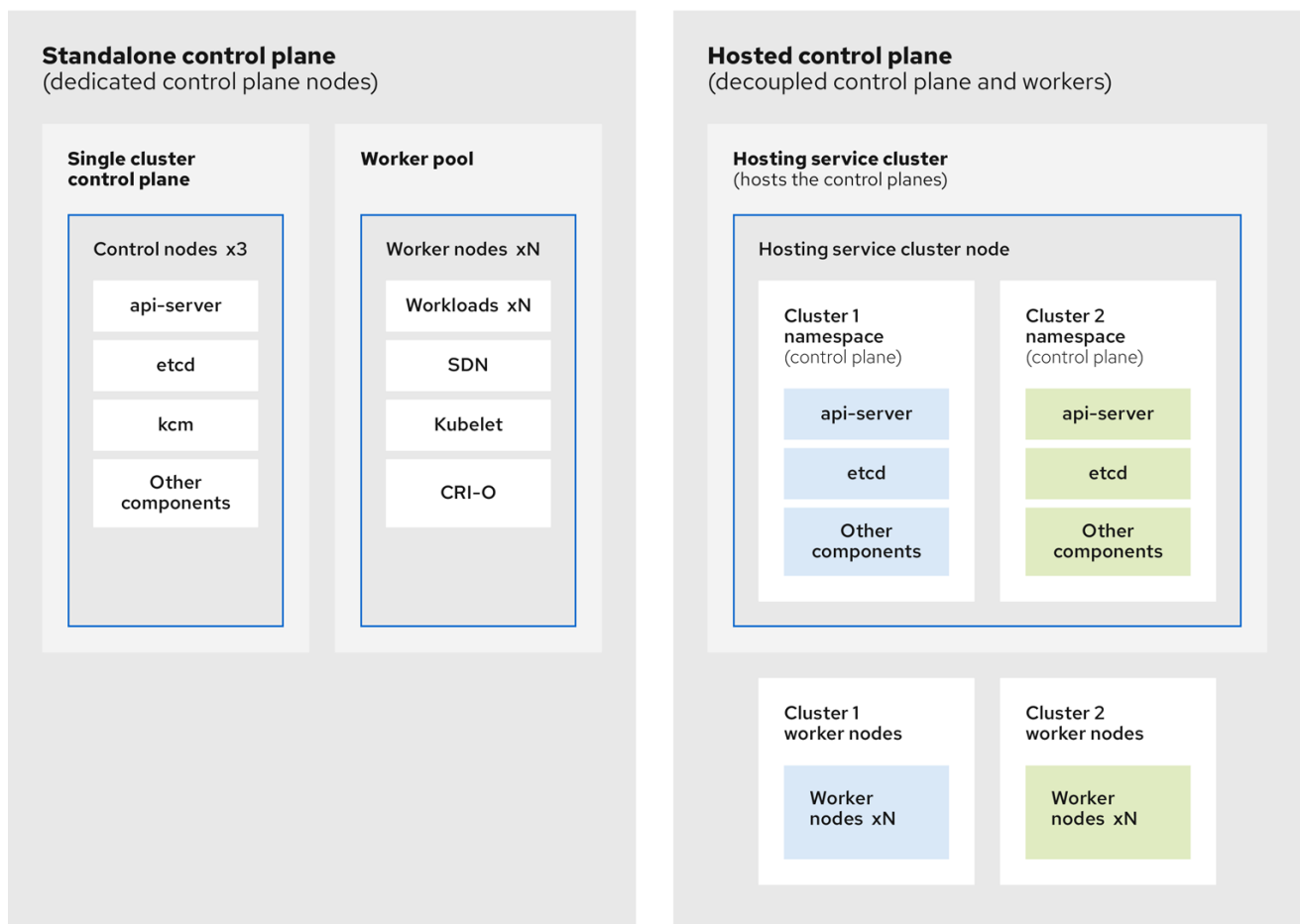
2.1.1. Architecture of hosted control planes

OpenShift Container Platform is often deployed in a coupled, or standalone, model, where a cluster consists of a control plane and a data plane. The control plane includes an API endpoint, a storage endpoint, a workload scheduler, and an actuator that ensures state. The data plane includes compute, storage, and networking where workloads and applications run.

The standalone control plane is hosted by a dedicated group of nodes, which can be physical or virtual, with a minimum number to ensure quorum. The network stack is shared. Administrator access to a cluster offers visibility into the cluster's control plane, machine management APIs, and other components that contribute to the state of a cluster.

Although the standalone model works well, some situations require an architecture where the control plane and data plane are decoupled. In those cases, the data plane is on a separate network domain with a dedicated physical hosting environment. The control plane is hosted by using high-level primitives

such as deployments and stateful sets that are native to Kubernetes. The control plane is treated as any other workload.



272_OpenShift_1122

2.1.2. Benefits of hosted control planes

With hosted control planes, you can pave the way for a true hybrid-cloud approach and enjoy several other benefits.

- The security boundaries between management and workloads are stronger because the control plane is decoupled and hosted on a dedicated hosting service cluster. As a result, you are less likely to leak credentials for clusters to other users. Because infrastructure secret account management is also decoupled, cluster infrastructure administrators cannot accidentally delete control plane infrastructure.
- With hosted control planes, you can run many control planes on fewer nodes. As a result, clusters are more affordable.
- Because the control planes consist of pods that are launched on OpenShift Container Platform, control planes start quickly. The same principles apply to control planes and workloads, such as monitoring, logging, and auto-scaling.
- From an infrastructure perspective, you can push registries, HAProxy, cluster monitoring, storage nodes, and other infrastructure components to the tenant's cloud provider account, isolating usage to the tenant.
- From an operational perspective, multicluster management is more centralized, which results in

fewer external factors that affect the cluster status and consistency. Site reliability engineers have a central place to debug issues and navigate to the cluster data plane, which can lead to shorter Time to Resolution (TTR) and greater productivity.

Additional resources

- [Cluster lifecycle with multicluster engine for Kubernetes Operator overview \(Red Hat Advanced Cluster Management official documentation\)](#)

2.2. DIFFERENCES BETWEEN HOSTED CONTROL PLANES AND OPENSIFT CONTAINER PLATFORM

Hosted control planes is a form factor of OpenShift Container Platform. Hosted clusters and the standalone OpenShift Container Platform clusters are configured and managed differently.

See the following tables to understand the differences between OpenShift Container Platform and hosted control planes:

2.2.1. Cluster creation and lifecycle

OpenShift Container Platform	Hosted control planes
You install a standalone OpenShift Container Platform cluster by using the openshift-install binary or the Assisted Installer.	You install a hosted cluster by using the hypershift.openshift.io API resources such as HostedCluster and NodePool , on an existing OpenShift Container Platform cluster.

2.2.2. Cluster configuration

OpenShift Container Platform	Hosted control planes
You configure cluster-scoped resources such as authentication, API server, and proxy by using the config.openshift.io API group.	You configure resources that impact the control plane in the HostedCluster resource.

2.2.3. etcd encryption

OpenShift Container Platform	Hosted control planes
You configure etcd encryption by using the APIServer resource with AES-GCM or AES-CBC. For more information, see "Enabling etcd encryption".	You configure etcd encryption by using the HostedCluster resource in the SecretEncryption field with AES-CBC or KMS for Amazon Web Services.

2.2.4. Operators and control plane

OpenShift Container Platform	Hosted control planes
A standalone OpenShift Container Platform cluster contains separate Operators for each control plane component.	A hosted cluster contains a single Operator named Control Plane Operator that runs in the hosted control plane namespace on the management cluster.
etcd uses storage that is mounted on the control plane nodes. The etcd cluster Operator manages etcd.	etcd uses a persistent volume claim for storage and is managed by the Control Plane Operator.
The Ingress Operator, network related Operators, and Operator Lifecycle Manager (OLM) run on the cluster.	The Ingress Operator, network related Operators, and Operator Lifecycle Manager (OLM) run in the hosted control plane namespace on the management cluster.
The OAuth server runs inside the cluster and is exposed through a route in the cluster.	The OAuth server runs inside the control plane and is exposed through a route, node port, or load balancer on the management cluster.

2.2.5. Updates

OpenShift Container Platform	Hosted control planes
The Cluster Version Operator (CVO) orchestrates the update process and monitors the ClusterVersion resource. Administrators and OpenShift components can interact with the CVO through the ClusterVersion resource. The oc adm upgrade command results in a change to the ClusterVersion.Spec.DesiredUpdate field in the ClusterVersion resource.	The hosted control planes update results in a change to the .spec.release.image field in the HostedCluster and NodePool resources. Any changes to the ClusterVersion resource are ignored.
After you update an OpenShift Container Platform cluster, both the control plane and compute machines are updated.	After you update the hosted cluster, only the control plane is updated. You perform node pool updates separately.

2.2.6. Machine configuration and management

OpenShift Container Platform	Hosted control planes
The MachineSets resource manages machines in the openshift-machine-api namespace.	The NodePool resource manages machines on the management cluster.
A set of control plane machines are available.	A set of control plane machines do not exist.

OpenShift Container Platform	Hosted control planes
You enable a machine health check by using the MachineHealthCheck resource.	You enable a machine health check through the .spec.management.autoRepair field in the NodePool resource.
You enable autoscaling by using the ClusterAutoscaler and MachineAutoscaler resources.	You enable autoscaling through the spec.autoScaling field in the NodePool resource.
Machines and machine sets are exposed in the cluster.	Machines, machine sets, and machine deployments from upstream Cluster CAPI Operator are used to manage machines but are not exposed to the user.
All machine sets are upgraded automatically when you update the cluster.	You update your node pools independently from the hosted cluster updates.
Only an in-place upgrade is supported in the cluster.	Both replace and in-place upgrades are supported in the hosted cluster.
The Machine Config Operator manages configurations for machines.	The Machine Config Operator does not exist in hosted control planes.
You configure machine Ignition by using the MachineConfig , KubeletConfig , and ContainerRuntimeConfig resources that are selected from a MachineConfigPool selector.	You configure the MachineConfig , KubeletConfig , and ContainerRuntimeConfig resources through the config map referenced in the spec.config field of the NodePool resource.
The Machine Config Daemon (MCD) manages configuration changes and updates on each of the nodes.	For an in-place upgrade, the node pool controller creates a run-once pod that updates a machine based on your configuration.
You can modify the machine configuration resources such as the SR-IOV Operator.	You cannot modify the machine configuration resources.

2.2.7. Networking

OpenShift Container Platform	Hosted control planes
The Kube API server communicates with nodes directly, because the Kube API server and nodes exist in the same Virtual Private Cloud (VPC).	The Kube API server communicates with nodes through Konnectivity. The Kube API server and nodes exist in a different Virtual Private Cloud (VPC).
Nodes communicate with the Kube API server through the internal load balancer.	Nodes communicate with the Kube API server through an external load balancer or a node port.

2.2.8. Web console

OpenShift Container Platform	Hosted control planes
The web console shows the status of a control plane.	The web console does not show the status of a control plane.
You can update your cluster by using the web console.	You cannot update the hosted cluster by using the web console.
The web console displays the infrastructure resources such as machines.	The web console does not display the infrastructure resources.
You can configure machines through the MachineConfig resource by using the web console.	You cannot configure machines by using the web console.

Additional resources

- [Enabling etcd encryption](#)

2.3. RELATIONSHIP BETWEEN HOSTED CONTROL PLANES, MULTICLUSTER ENGINE OPERATOR, AND RHACM

You can configure hosted control planes by using the multicluster engine for Kubernetes Operator. The multicluster engine Operator cluster lifecycle defines the process of creating, importing, managing, and destroying Kubernetes clusters across various infrastructure cloud providers, private clouds, and on-premise data centers.

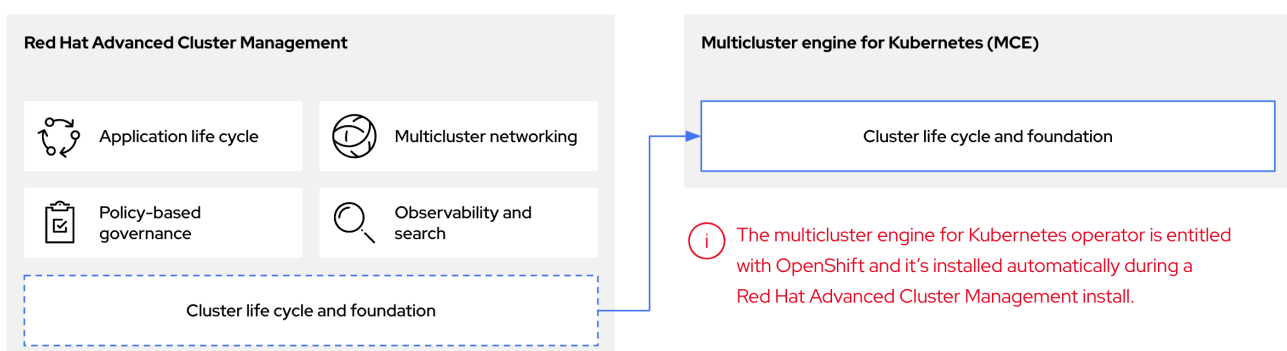


NOTE

The multicluster engine Operator is an integral part of Red Hat Advanced Cluster Management and is enabled by default with RHACM. However, you do not need Red Hat Advanced Cluster Management to use hosted control planes.

The multicluster engine Operator is the cluster lifecycle Operator that provides cluster management capabilities for OpenShift Container Platform and RHACM hub clusters. The multicluster engine Operator enhances cluster fleet management and supports OpenShift Container Platform cluster lifecycle management across clouds and data centers.

Figure 2.1. Cluster life cycle and foundation



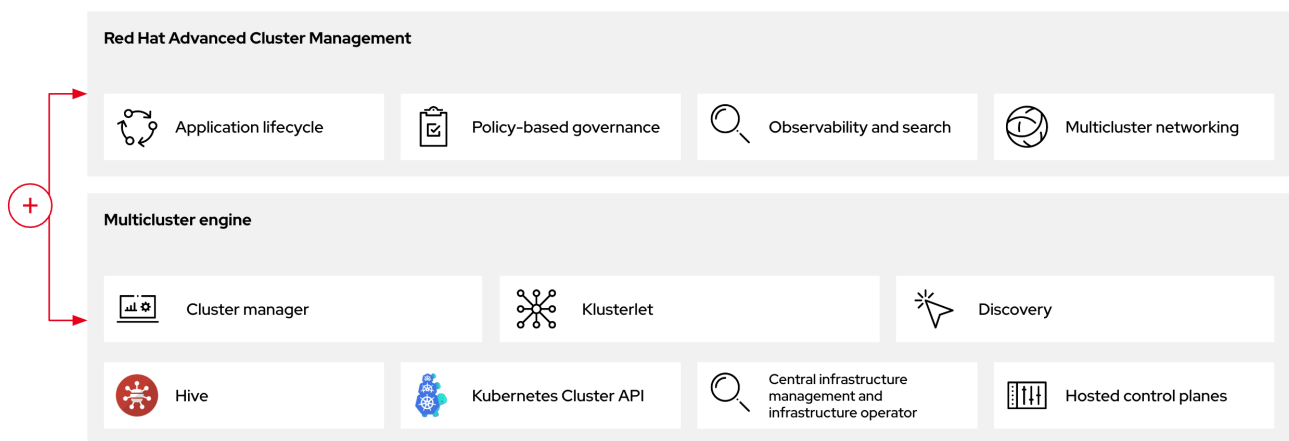
You can use the multicluster engine Operator with OpenShift Container Platform as a standalone cluster manager or as part of a RHACM hub cluster.

TIP

A management cluster is also known as the hosting cluster.

You can deploy OpenShift Container Platform clusters by using two different control plane configurations: standalone or hosted control planes. The standalone configuration uses dedicated virtual machines or physical machines to host the control plane. With hosted control planes for OpenShift Container Platform, you create control planes as pods on a management cluster without the need for dedicated virtual or physical machines for each control plane.

Figure 2.2. RHACM and the multicluster engine Operator introduction diagram



2.3.1. Hosted clusters in Red Hat Advanced Cluster Management

You can bring hosted clusters to a Red Hat Advanced Cluster Management hub cluster to manage them with Red Hat Advanced Cluster Management management components.

For more information, see "Discovering multicluster engine Operator hosted clusters in Red Hat Advanced Cluster Management".

Additional resources

- [Discovering multicluster engine Operator hosted clusters in Red Hat Advanced Cluster Management \(Red Hat Advanced Cluster Management official documentation\)](#)

2.4. VERSIONING FOR HOSTED CONTROL PLANES

The hosted control planes feature includes several components that might require independent versioning and support levels.

Those components are as follows:

- Management cluster
- HyperShift Operator
- Hosted control planes (**hcp**) command-line interface (CLI)

- **hypershift.openshift.io** API
- Control Plane Operator

2.4.1. Management cluster

In management clusters for production use, you need multicluster engine for Kubernetes Operator, which is available through the software catalog. The multicluster engine Operator bundles a supported build of the HyperShift Operator. For your management clusters to remain supported, you must use the version of OpenShift Container Platform that multicluster engine Operator runs on. In general, a new release of multicluster engine Operator runs on the following versions of OpenShift Container Platform:

- The latest General Availability version of OpenShift Container Platform
- Two versions before the latest General Availability version of OpenShift Container Platform

The full list of OpenShift Container Platform versions that you can install through the HyperShift Operator on a management cluster depends on the version of your HyperShift Operator. However, the list always includes at least the same OpenShift Container Platform version as the management cluster and two previous minor versions relative to the management cluster. For example, if the management cluster is running 4.17 and a supported version of multicluster engine Operator, the HyperShift Operator can install 4.17, 4.16, 4.15, and 4.14 hosted clusters.

With each major, minor, or patch version release of OpenShift Container Platform, two components of hosted control planes are released:

- The HyperShift Operator
- The **hcp** command-line interface (CLI)

2.4.2. HyperShift Operator

The HyperShift Operator manages the lifecycle of hosted clusters that are represented by the **HostedCluster** API resources. The HyperShift Operator is released with each OpenShift Container Platform release. The HyperShift Operator creates the **supported-versions** config map in the **hypershift** namespace. The config map contains the supported hosted cluster versions.

You can host different versions of control planes on the same management cluster.

Example supported-versions config map object

```
apiVersion: v1
data:
  supported-versions: '{"versions":["4.22"]}'
kind: ConfigMap
metadata:
  labels:
    hypershift.openshift.io/supported-versions: "true"
  name: supported-versions
  namespace: hypershift
```

2.4.3. hosted control planes CLI

You can use the **hcp** CLI to create hosted clusters. You can download the CLI from multicloud engine Operator. When you run the **hcp version** command, the output shows the latest OpenShift Container Platform that the CLI supports against your **kubeconfig** file.

2.4.4. hypershift.openshift.io API

You can use the **hypershift.openshift.io** API resources, such as, **HostedCluster** and **NodePool**, to create and manage OpenShift Container Platform clusters at scale. A **HostedCluster** resource contains the control plane and common data plane configuration. When you create a **HostedCluster** resource, you have a fully functional control plane with no attached nodes. A **NodePool** resource is a scalable set of worker nodes that is attached to a **HostedCluster** resource.

The API version policy generally aligns with the policy for Kubernetes API versioning.

Updates for hosted control planes involve updating the hosted cluster and the node pools. For more information, see "Updates for hosted control planes".

2.4.5. Control Plane Operator

The Control Plane Operator is released as part of each OpenShift Container Platform payload release image for the following architectures:

- amd64
- arm64
- multi-arch

Additional resources

- [Kubernetes API versioning](#)
- [AMD64 release images](#)
- [ARM64 release images](#)
- [Multi-arch release images](#)

2.5. GLOSSARY OF COMMON CONCEPTS AND PERSONAS FOR HOSTED CONTROL PLANES

When you use hosted control planes for OpenShift Container Platform, it is important to understand its key concepts and the personas that are involved.

2.5.1. Concepts

data plane

The part of the cluster that includes the compute, storage, and networking where workloads and applications run.

hosted cluster

An OpenShift Container Platform cluster with its control plane and API endpoint hosted on a management cluster. The hosted cluster includes the control plane and its corresponding data plane.

hosted cluster infrastructure

Network, compute, and storage resources that exist in the tenant or end-user cloud account.

hosted control plane

An OpenShift Container Platform control plane that runs on the management cluster, which is exposed by the API endpoint of a hosted cluster. The components of a control plane include etcd, the Kubernetes API server, the Kubernetes controller manager, and a VPN.

hosting cluster

See *management cluster*.

managed cluster

A cluster that the hub cluster manages. This term is specific to the cluster lifecycle that the multicluster engine for Kubernetes Operator manages in Red Hat Advanced Cluster Management. A managed cluster is not the same thing as a *management cluster*. For more information, see [Managed cluster](#).

management cluster

An OpenShift Container Platform cluster where the HyperShift Operator is deployed and where the control planes for hosted clusters are hosted. The management cluster is synonymous with the *hosting cluster*.

management cluster infrastructure

Network, compute, and storage resources of the management cluster.

node pool

A resource that manages a set of compute nodes that are associated with a hosted cluster. The compute nodes run applications and workloads within the hosted cluster.

2.5.2. Personas

cluster instance administrator

Users who assume this role are the equivalent of administrators in standalone OpenShift Container Platform. This user has the **cluster-admin** role in the provisioned cluster, but might not have power over when or how the cluster is updated or configured. This user might have read-only access to see some configuration projected into the cluster.

cluster instance user

Users who assume this role are the equivalent of developers in standalone OpenShift Container Platform. This user does not have a view into the software catalog or machines.

cluster service consumer

Users who assume this role can request control planes and worker nodes, drive updates, or modify externalized configurations. Typically, this user does not manage or access cloud credentials or infrastructure encryption keys. The cluster service consumer persona can request hosted clusters and interact with node pools. Users who assume this role have role-based access control (RBAC) to create, read, update, or delete hosted clusters and node pools within a logical boundary.

cluster service provider

Users who assume this role typically have the **cluster-admin** role on the management cluster and have RBAC to monitor and own the availability of the HyperShift Operator and the control planes for the tenant's hosted clusters. The cluster service provider persona is responsible for several activities, including the following examples:

- Owning service-level objects for control plane availability, uptime, and stability
- Configuring the cloud account for the management cluster to host control planes

- Configuring the user-provisioned infrastructure, which includes the host awareness of available compute resources

CHAPTER 3. PREPARING TO DEPLOY HOSTED CONTROL PLANES

3.1. REQUIREMENTS FOR HOSTED CONTROL PLANES

Ensure you are familiar with the general requirements to deploy hosted control planes.

The following requirements apply to hosted control planes:

- In order to run the HyperShift Operator, your management cluster needs at least three worker nodes. In the context of hosted control planes, a *management cluster* is an OpenShift Container Platform cluster where the HyperShift Operator is deployed and where the control planes for hosted clusters are hosted.
The control plane is associated with a hosted cluster and runs as pods in a single namespace. When the cluster service consumer creates a hosted cluster, it creates a worker node that is independent of the control plane.
- You must open the firewall port **53** on Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) to allow the Domain Name Service (DNS) protocol to work as expected.
- You can run both the management cluster and the worker nodes on-premise, such as on a bare-metal platform or on OpenShift Virtualization. In addition, you can run both the management cluster and the worker nodes on cloud infrastructure, such as Amazon Web Services (AWS).
- If you use a mixed infrastructure, such as running the management cluster on AWS and your worker nodes on-premise, or running your worker nodes on AWS and your management cluster on-premise, you must use the **PublicAndPrivate** publishing strategy and follow the latency requirements in the support matrix.
- In Bare Metal Host (BMH) deployments, where the Bare Metal Operator starts machines, the hosted control plane must be able to reach baseboard management controllers (BMCs). If your security profile does not permit the Cluster Baremetal Operator to access the network where the BMHs have their BMCs in order to enable Redfish automation, you can use BYO ISO support. However, in BYO mode, OpenShift Container Platform cannot automate the powering on of BMHs.

3.1.1. Support matrix for hosted control planes

Because multicluster engine for Kubernetes Operator includes the HyperShift Operator, releases of hosted control planes align with releases of multicluster engine Operator. The support matrix includes details about supported clusters, platforms, and architectures, as well as information about updates and technology preview features.

For more information, see "OpenShift Operator Life Cycles".

3.1.1.1. Management cluster support

Any supported OpenShift Container Platform cluster can be a management cluster.

**NOTE**

A single-node OpenShift Container Platform cluster is not supported as a management cluster. If you have resource constraints, you can share infrastructure between a standalone OpenShift Container Platform control plane and hosted control planes. For more information, see "Shared infrastructure between hosted and standalone control planes".

The following table maps multicluster engine Operator versions to the management cluster versions that support them:

Table 3.1. Supported multicluster engine Operator versions for OpenShift Container Platform management clusters

Management cluster version	Supported multicluster engine Operator version
4.14, 4.16	2.6
4.16	2.7
4.16, 4.18	2.8
4.18 - 4.19	2.9
4.18 - 4.20	2.10
4.19 - 4.21	2.11
4.20 - 4.22	2.17

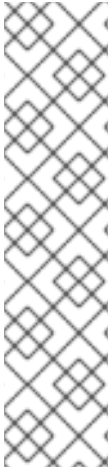
3.1.1.2. Hosted cluster support

For hosted clusters, no direct relationship exists between the management cluster version and the hosted cluster version. The hosted cluster version depends on the HyperShift Operator that is included with your multicluster engine Operator version.

**NOTE**

Ensure a maximum latency of 200 ms between the management cluster and hosted clusters. This requirement is especially important for mixed infrastructure deployments, such as when your management cluster is on AWS and your compute nodes are on-premise.

The following table shows the hosted cluster versions that you can create by using the HyperShift Operator that is associated with a version of multicluster engine Operator:

**NOTE**

Although the HyperShift Operator supports the hosted cluster versions in the following table, multicluster engine Operator supports only as far back as 2 versions earlier than the current version. For example, if the current hosted cluster version is 4.21, multicluster engine Operator supports as far back as version 4.19. If you want to use a hosted cluster version that is earlier than one of the versions that multicluster engine Operator supports, you can detach your hosted clusters from multicluster engine Operator to be unmanaged, or you can use an earlier version of multicluster engine Operator. For instructions to detach your hosted clusters from multicluster engine Operator, see "Removing a cluster from management" (RHACM documentation). For more information about multicluster engine Operator support, see "The multicluster engine for Kubernetes operator 2.17 Support Matrix" (Red Hat Knowledgebase).

Table 3.2. Hosted cluster version mapped to HyperShift Operator associated with multicluster engine Operator version

Hosted cluster version	HyperShift Operator in multicluster engine Operator 2.6	HyperShift Operator in multicluster engine Operator 2.7	HyperShift Operator in multicluster engine Operator 2.8	HyperShift Operator in multicluster engine Operator 2.9	HyperShift Operator in multicluster engine Operator 2.10	HyperShift Operator in multicluster engine Operator 2.11	HyperShift Operator in multicluster engine Operator 2.17
4.14	Yes	Yes	Yes	No	No	No	No
4.16	Yes	Yes	Yes	Yes	No	No	No
4.18	No	No	Yes	Yes	Yes	No	No
4.19	No	No	No	Yes	Yes	Yes	No
4.20	No	No	No	No	Yes	Yes	Yes
4.21	No	No	No	No	No	Yes	Yes
4.22	No	No	No	No	No	No	Yes

3.1.1.3. Hosted cluster platform support

A hosted cluster supports only one infrastructure platform. For example, you cannot create multiple node pools on different infrastructure platforms.

The following table indicates which OpenShift Container Platform versions are supported for each platform of hosted control planes.



IMPORTANT

For IBM Power and IBM Z:

- You must run the control plane on machine types that are based on 64-bit x86 architecture or s390x architecture
- You must run node pools on IBM Power or IBM Z

In the following table, the management cluster version is the OpenShift Container Platform version where the multicluster engine Operator is enabled:

Table 3.3. Required OpenShift Container Platform versions for platforms

Hosted cluster platform	Management cluster version	Hosted cluster version
Amazon Web Services	4.16, 4.18 - 4.22	4.16, 4.18 - 4.22
IBM Power	4.18 - 4.22	4.18 - 4.22
IBM Z	4.18 - 4.22	4.18 - 4.22
OpenShift Virtualization	4.14, 4.16, 4.18 - 4.22	4.14, 4.16, 4.18 - 4.22
Bare metal	4.14, 4.16, 4.18 - 4.22	4.14, 4.16, 4.18 - 4.22
Non-bare-metal agent machines (Technology Preview)	4.16, 4.18 - 4.22	4.16, 4.18 - 4.22
Red Hat OpenStack Platform (RHOSP) (Technology Preview)	4.19 - 4.22	4.19 - 4.22
Microsoft Azure (Technology Preview)	4.22	4.22

3.1.1.4. Multi-architecture support

The following tables indicate the supported architectures for hosted control planes, organized by platform. If an architecture is not listed, it is not yet fully supported.

Table 3.4. Multi-architecture support for hosted control planes

Platform	Control planes	Compute nodes	OpenShift Container Platform version support
AWS	64-bit x86	64-bit x86	4.16, 4.18 - 4.22
AWS	64-bit x86	ARM64	4.18 - 4.22

Platform	Control planes	Compute nodes	OpenShift Container Platform version support
AWS	ARM64	ARM64	4.18 - 4.22
AWS	ARM64	64-bit x86	4.18 - 4.22
Bare metal (Agent platform)	64-bit x86	64-bit x86	4.14, 4.16, 4.18 - 4.22
Bare metal (Agent platform)	64-bit x86	ARM64	4.21 - 4.22
IBM Power	64-bit x86	64-bit x86	4.19 - 4.22
IBM Power	64-bit x86	ppc64le	4.18 - 4.22
IBM Z	64-bit x86	64-bit x86	4.18 - 4.22
IBM Z	64-bit x86	s390x	4.18 - 4.22
IBM Z	s390x	s390x	4.20 - 4.22
Non-bare-metal Agent machines (Technology Preview)	64-bit x86	64-bit x86	4.16, 4.18 - 4.22
OpenShift Virtualization	64-bit x86	64-bit x86	4.14, 4.16, 4.18 - 4.22
OpenShift Virtualization	s390x	s390x	4.22
Red Hat OpenStack Platform (RHOSP) (Technology Preview)	64-bit x86	64-bit x86	4.19 - 4.22

3.1.1.5. Updates of multicluster engine Operator

When you update to another version of the multicluster engine Operator, your hosted cluster can continue to run if the HyperShift Operator that is included in the version of multicluster engine Operator supports the hosted cluster version. The following table shows which hosted cluster versions are supported on which updated multicluster engine Operator versions.

**NOTE**

Although the HyperShift Operator supports the hosted cluster versions in the following table, multicluster engine Operator supports only as far back as 2 versions earlier than the current version. For example, if the current hosted cluster version is 4.21, multicluster engine Operator supports as far back as version 4.19. If you want to use a hosted cluster version that is earlier than one of the versions that multicluster engine Operator supports, you can detach your hosted clusters from multicluster engine Operator to be unmanaged, or you can use an earlier version of multicluster engine Operator. For instructions to detach your hosted clusters from multicluster engine Operator, see "Removing a cluster from management" (RHACM documentation). For more information about multicluster engine Operator support, see "The multicluster engine for Kubernetes operator 2.17 Support Matrix" (Red Hat Knowledgebase).

Table 3.5. Hosted cluster version supported while updating multicluster engine Operator

multicluster engine Operator version	Supported hosted cluster version while updating
Updating from 2.5 to 2.6	OpenShift Container Platform 4.14, 4.16
Updating from 2.6 to 2.7	OpenShift Container Platform 4.14, 4.16
Updating from 2.7 to 2.8	OpenShift Container Platform 4.14, 4.16
Updating from 2.8 to 2.9	OpenShift Container Platform 4.16, 4.18
Updating from 2.9 to 2.10	OpenShift Container Platform 4.18, 4.19
Updating from 2.10 to 2.11	OpenShift Container Platform 4.19, 4.20
Updating from 2.11 to 2.17	OpenShift Container Platform 4.20, 4.21

For example, if you have an OpenShift Container Platform 4.18 hosted cluster on the management cluster and you update from multicluster engine Operator 2.8 to 2.9, the hosted cluster can continue to run.

3.1.1.6. Technology Preview features

For a list of features in this release that have a Technology Preview status, see the "Technology Preview features status" section of the *Hosted control planes release notes*.

3.1.2. Additional resources

- [OpenShift Operator Life Cycles](#)
- [Shared infrastructure between hosted and standalone control planes](#)
- [Technology Preview features status](#)
- [Removing a cluster from management](#)
- [The multicluster engine for Kubernetes operator 2.17 Support Matrix](#)

3.1.3. FIPS-enabled hosted clusters

The binaries for hosted control planes are FIPS-compliant, with the exception of the hosted control planes command-line interface, **hcp**.

If you want to deploy a FIPS-enabled hosted cluster, you must use a FIPS-enabled management cluster. To enable FIPS mode for your management cluster, you must run the installation program from a Red Hat Enterprise Linux (RHEL) computer configured to operate in FIPS mode. For more information about configuring FIPS mode on RHEL, see [Switching RHEL to FIPS mode](#).

When running RHEL or Red Hat Enterprise Linux CoreOS (RHCOS) booted in FIPS mode, OpenShift Container Platform core components use the RHEL cryptographic libraries that have been submitted to NIST for FIPS 140-2/140-3 Validation on only the x86_64, ppc64le, and s390x architectures.

After you set up your management cluster in FIPS mode, the hosted cluster creation process runs on that management cluster.

Additional resources

- [The multicluster engine for Kubernetes operator 2.17 Support Matrix](#)
- [Red Hat OpenShift Container Platform Operator Update Information Checker](#)
- [Shared infrastructure between hosted and standalone control planes](#)

3.1.4. CIDR ranges for hosted control planes

To successfully deploy hosted control planes on OpenShift Container Platform, define the network environment by using specific Classless Inter-Domain Routing (CIDR) subnet ranges.

The following Classless Inter-Domain Routing (CIDR) subnet ranges are the default settings for hosted control planes:

- **v4InternalSubnet:** 100.65.0.0/16 (OVN-Kubernetes)
- **clusterNetwork:** 10.132.0.0/14 (pod network)
- **serviceNetwork:** 172.31.0.0/16

By using one of the default subnet ranges, you can avoid CIDR overlap with the management cluster and avoid connectivity issues. However, you can use other CIDR subnet ranges if they do not overlap with the management cluster.

Additional resources

- [CIDR range definitions](#)

3.2. SIZING GUIDANCE FOR HOSTED CONTROL PLANES

Many factors, including hosted cluster workload and worker node count, affect how many hosted control planes can fit within a certain number of worker nodes.

Use this sizing guide to help with hosted cluster capacity planning.

This guidance assumes a highly available hosted control planes topology. The load-based sizing examples were measured on a bare-metal cluster. Cloud-based instances might have different limiting factors, such as memory size.

You can override the following resource utilization sizing measurements and disable the metric service monitoring.

See the following highly available hosted control planes requirements, which were tested with OpenShift Container Platform version 4.12.9 and later:

- 78 pods
- Three 8 GiB PVs for etcd
- Minimum vCPU: approximately 5.5 cores
- Minimum memory: approximately 19 GiB

Additional resources

- [Overriding resource utilization measurements](#)
- [Distributing hosted cluster workloads](#)

3.2.1. Pod limits

The **maxPods** setting for each node affects how many hosted clusters can fit in a control-plane node. It is important to note the **maxPods** value on all control-plane nodes. Plan for about 75 pods for each highly available hosted control plane.

For bare-metal nodes, the default **maxPods** setting of 250 is likely to be a limiting factor because roughly three hosted control planes fit for each node given the pod requirements, even if the machine has plenty of resources to spare. Setting the **maxPods** value to 500 by configuring the **KubeletConfig** value allows for greater hosted control plane density, which can help you take advantage of additional compute resources.

Additional resources

- [Configuring the maximum number of pods per node](#)

3.2.2. Request-based resource limit

The maximum number of hosted control planes that the cluster can host is calculated based on the hosted control plane CPU and memory requests from the pods.

A highly available hosted control plane consists of 78 pods that request 5 vCPUs and 18 GiB memory. These baseline numbers are compared to the cluster worker node resource capacities to estimate the maximum number of hosted control planes.

3.2.3. Load-based limit

As you plan your deployment, consider the maximum number of hosted control planes that the cluster can host. That number is calculated based on the hosted control plane pods CPU and memory utilizations when some workload is put on the hosted control plane Kubernetes API server.

The following method is used to measure the hosted control plane resource utilizations as the workload increases:

- A hosted cluster with 9 workers that use 8 vCPU and 32 GiB each, while using the KubeVirt platform
- The workload test profile that is configured to focus on API control-plane stress, based on the following definition:
 - Created objects for each namespace, scaling up to 100 namespaces total
 - Additional API stress with continuous object deletion and creation
 - Workload queries-per-second (QPS) and Burst settings set high to remove any client-side throttling

As the load increases by 1000 QPS, the hosted control plane resource utilization increases by 9 vCPUs and 2.5 GB memory.

For general sizing purposes, consider the 1000 QPS API rate that is a *medium* hosted cluster load, and a 2000 QPS API that is a *heavy* hosted cluster load.



NOTE

This test provides an estimation factor to increase the compute resource utilization based on the expected API load. Exact utilization rates can vary based on the type and pace of the cluster workload.

The following example shows hosted control plane resource scaling for the workload and API rate definitions:

Table 3.6. API rate table

QPS (API rate)	vCPU usage	Memory usage (GiB)
Low load (Less than 50 QPS)	2.9	11.1
Medium load (1000 QPS)	11.9	13.6
High load (2000 QPS)	20.9	16.1

The hosted control plane sizing is about control-plane load and workloads that cause heavy API activity, etcd activity, or both. Hosted pod workloads that focus on data-plane loads, such as running a database, might not result in high API rates.

3.2.4. Sizing calculation example

Review an example of how to size a deployment of hosted control planes.

This example provides sizing guidance for the following scenario:

- Three bare-metal workers that are labeled as **hypershift.openshift.io/control-plane** nodes
- **maxPods** value set to 500

- The expected API rate is medium or about 1000, according to the load-based limits

Table 3.7. Limit inputs

Limit description	Server 1	Server 2
Number of vCPUs on worker node	64	128
Memory on worker node (GiB)	128	256
Maximum pods per worker	500	500
Number of workers used to host control planes	3	3
Maximum QPS target rate (API requests per second)	1000	1000

Table 3.8. Sizing calculation example

Calculated values based on worker node size and API rate	Server 1	Server 2	Calculation notes
Maximum hosted control planes per worker based on vCPU requests	12.8	25.6	Number of worker vCPUs ÷ 5 total vCPU requests per hosted control plane
Maximum hosted control planes per worker based on vCPU usage	5.4	10.7	Number of vCPUS ÷ (2.9 measured idle vCPU usage + (QPS target rate ÷ 1000) × 9.0 measured vCPU usage per 1000 QPS increase)
Maximum hosted control planes per worker based on memory requests	7.1	14.2	Worker memory GiB ÷ 18 GiB total memory request per hosted control plane
Maximum hosted control planes per worker based on memory usage	9.4	18.8	Worker memory GiB ÷ (11.1 measured idle memory usage + (QPS target rate ÷ 1000) × 2.5 measured memory usage per 1000 QPS increase)

Maximum hosted control planes per worker based on per node pod limit	6.7	6.7	500 maxPods ÷ 75 pods per hosted control plane
Minimum of previously mentioned maximums	5.4	6.7	
	vCPU limiting factor	maxPods limiting factor	
Maximum number of hosted control planes within a management cluster	16	20	Minimum of previously mentioned maximums × 3 control-plane workers

Table 3.9. Hosted control planes capacity metrics

Name	Description
mce_hs_addon_request_based_hcp_capacity_gauge	Estimated maximum number of hosted control planes the cluster can host based on a highly available hosted control planes resource request.
mce_hs_addon_low_qps_based_hcp_capacity_gauge	Estimated maximum number of hosted control planes the cluster can host if all hosted control planes make around 50 QPS to the clusters Kube API server.
mce_hs_addon_medium_qps_based_hcp_capacity_gauge	Estimated maximum number of hosted control planes the cluster can host if all hosted control planes make around 1000 QPS to the clusters Kube API server.
mce_hs_addon_high_qps_based_hcp_capacity_gauge	Estimated maximum number of hosted control planes the cluster can host if all hosted control planes make around 2000 QPS to the clusters Kube API server.
mce_hs_addon_average_qps_based_hcp_capacity_gauge	Estimated maximum number of hosted control planes the cluster can host based on the existing average QPS of hosted control planes. If you do not have an active hosted control planes, you can expect low QPS.

3.2.5. Shared infrastructure between hosted and standalone control planes

As a service provider, you can more effectively use your resources by sharing infrastructure between a standalone OpenShift Container Platform control plane and hosted control planes. A 3-node OpenShift Container Platform cluster can be a management cluster for a hosted cluster.

Sharing infrastructure can be beneficial in constrained environments, such as in small-scale deployments where you need resource efficiency.

Before you share infrastructure, ensure that your infrastructure has enough resources to support hosted control planes. On the OpenShift Container Platform management cluster, nothing else can be deployed except hosted control planes. Ensure that the management cluster has enough CPU, memory, storage, and network resources to handle the combined load of the hosted clusters. Workload must not be demanding, and it must fall within a low queries-per-second (QPS) profile. For more information about resources and workload, see "Sizing guidance for hosted control planes".

Additional resources

- [Sizing guidance for hosted control planes](#)

3.3. OVERRIDING RESOURCE UTILIZATION MEASUREMENTS

The set of baseline measurements for resource utilization can vary in each hosted cluster. Ensure that you understand the resource utilization needed for your deployment.

3.3.1. Overriding resource utilization measurements for a hosted cluster

You can override resource utilization measurements based on the type and pace of your cluster workload.

Procedure

1. Create the **ConfigMap** resource by running the following command:

```
$ oc create -f <your-config-map-file.yaml>
```

Replace **<your-config-map-file.yaml>** with the name of your YAML file that contains your **hcp-sizing-baseline** config map.

2. Create the **hcp-sizing-baseline** config map in the **local-cluster** namespace to specify the measurements you want to override. Your config map might resemble the following YAML file:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: hcp-sizing-baseline
  namespace: local-cluster
data:
  incrementalCPUUsagePer1KQPS: "9.0"
  memoryRequestPerHCP: "18"
  minimumQPSPerHCP: "50.0"
```

3. Delete the **hypershift-addon-agent** deployment to restart the **hypershift-addon-agent** pod by running the following command:

```
$ oc delete deployment hypershift-addon-agent \
-n open-cluster-management-agent-addon
```

Verification

- Observe the **hypershift-addon-agent** pod logs. Verify that the overridden measurements are updated in the config map by running the following command:

```
$ oc logs hypershift-addon-agent -n open-cluster-management-agent-addon
```

Your logs might resemble the following output:

Example output

```
2024-01-05T19:41:05.392Z INFO agent.agent-reconciler agent/agent.go:793 setting
cpuRequestPerHCP to 5
2024-01-05T19:41:05.392Z INFO agent.agent-reconciler agent/agent.go:802 setting
memoryRequestPerHCP to 18
2024-01-05T19:53:54.070Z INFO agent.agent-reconciler
agent/hcp_capacity_calculation.go:141 The worker nodes have 12.000000 vCPUs
2024-01-05T19:53:54.070Z INFO agent.agent-reconciler
agent/hcp_capacity_calculation.go:142 The worker nodes have 49.173369 GB memory
```

If the overridden measurements are not updated properly in the **hcp-sizing-baseline** config map, you might see the following error message in the **hypershift-addon-agent** pod logs:

Example error

```
2024-01-05T19:53:54.052Z ERROR agent.agent-reconciler agent/agent.go:788 failed to get
configmap from the hub. Setting the HCP sizing baseline with default values. {"error":
"configmaps \"hcp-sizing-baseline\" not found"}
```

3.3.2. Disabling the metric service monitoring

After you enable the **hypershift-addon** managed cluster add-on, metric service monitoring is configured by default so that OpenShift Container Platform monitoring can gather metrics from **hypershift-addon**. If needed, you can disable metric service monitoring.

Procedure

1. Log in to your hub cluster by running the following command:

```
$ oc login
```

2. Edit the **hypershift-addon-deploy-config** add-on deployment configuration specification by running the following command:

```
$ oc edit addondeploymentconfig hypershift-addon-deploy-config \
-n multicluster-engine
```

3. Add the **disableMetrics=true** customized variable to the specification, as shown in the following example:

```

apiVersion: addon.open-cluster-management.io/v1alpha1
kind: AddOnDeploymentConfig
metadata:
  name: hypershift-addon-deploy-config
  namespace: multicloud-engine
spec:
  customizedVariables:
    - name: hcMaxNumber
      value: "80"
    - name: hcThresholdNumber
      value: "60"
    - name: disableMetrics
      value: "true"

```

The **disableMetrics=true** customized variable disables metric service monitoring for both new and existing **hypershift-addon** managed cluster add-ons.

4. Apply the changes to the configuration specification by running the following command:

```
$ oc apply -f <filename>.yaml
```

3.4. INSTALLING THE HOSTED CONTROL PLANES COMMAND-LINE INTERFACE

The hosted control planes command-line interface, **hcp**, is a tool that you can use to get started with hosted control planes. For Day 2 operations, such as management and configuration, use GitOps or your own automation tool.

3.4.1. Installing the hosted control planes command-line interface from the terminal

You can install the hosted control planes command-line interface (CLI), **hcp**, from the terminal.

Prerequisites

- On an OpenShift Container Platform cluster, you have installed multicloud engine for Kubernetes Operator 2.5 or later. The multicloud engine Operator is automatically installed when you install Red Hat Advanced Cluster Management. You can also install multicloud engine Operator without Red Hat Advanced Management as an Operator from the OpenShift Container Platform software catalog.

Procedure

1. Get the URL to download the **hcp** binary by running the following command:

```
$ oc get ConsoleCLIDownload hcp-cli-download -o json | jq -r ".spec"
```

2. Download the **hcp** binary by running the following command:

```
$ wget <hcp_cli_download_url>
```

Replace **hcp_cli_download_url** with the URL that you obtained from the previous step.

3. Unpack the downloaded archive by running the following command:

```
$ tar xvzf hcp.tar.gz
```

4. Make the **hcp** binary file executable by running the following command:

```
$ chmod +x hcp
```

5. Move the **hcp** binary file to a directory in your path by running the following command:

```
$ sudo mv hcp /usr/local/bin/.
```



NOTE

If you download the CLI on a Mac computer, you might see a warning about the **hcp** binary file. You need to adjust your security settings to allow the binary file to be run.

Verification

- Verify that you see the list of available parameters by running the following command:

```
$ hcp create cluster <platform> --help
```

You can use the **hcp create cluster** command to create and manage hosted clusters. The supported platforms are **agent**, **aws**, and **kubevirt**. The **azure** and **openstack** platforms are also available as Technology Preview features.

3.4.2. Installing the hosted control planes command-line interface by using the web console

You can install the hosted control planes command-line interface (CLI), **hcp**, by using the OpenShift Container Platform web console.

Prerequisites

- On an OpenShift Container Platform cluster, you have installed multicluster engine for Kubernetes Operator 2.5 or later. The multicluster engine Operator is automatically installed when you install Red Hat Advanced Cluster Management. You can also install multicluster engine Operator without Red Hat Advanced Management as an Operator from the OpenShift Container Platform software catalog.

Procedure

1. From the OpenShift Container Platform web console, click the **Help icon → Command Line Tools**.
2. Click **Download hcp CLI** for your platform.
3. Unpack the downloaded archive by running the following command:

```
$ tar xvzf hcp.tar.gz
```

4. Run the following command to make the binary file executable:

```
$ chmod +x hcp
```

5. Run the following command to move the binary file to a directory in your path:

```
$ sudo mv hcp /usr/local/bin/.
```



NOTE

If you download the CLI on a Mac computer, you might see a warning about the **hcp** binary file. You need to adjust your security settings to allow the binary file to be run.

Verification

- Verify that you see the list of available parameters by running the following command:

```
$ hcp create cluster <platform> --help
```

You can use the **hcp create cluster** command to create and manage hosted clusters. The supported platforms are **agent**, **aws**, and **kubevirt**. The **azure** and **openstack** platforms are also available as Technology Preview features.

3.4.3. Installing the hosted control planes command-line interface by using the content gateway

You can install the hosted control planes command-line interface (CLI), **hcp**, by using the content gateway.

Prerequisites

- On an OpenShift Container Platform cluster, you have installed multicluster engine for Kubernetes Operator 2.7 or later. The multicluster engine Operator is automatically installed when you install Red Hat Advanced Cluster Management. You can also install multicluster engine Operator without Red Hat Advanced Management as an Operator from OpenShift Container Platform OperatorHub.

Procedure

1. Navigate to the [content gateway](#) and download the **hcp** binary.
2. Unpack the downloaded archive by running the following command:

```
$ tar xvzf hcp.tar.gz
```

3. Make the **hcp** binary file executable by running the following command:

```
$ chmod +x hcp
```

4. Move the **hcp** binary file to a directory in your path by running the following command:

```
$ sudo mv hcp /usr/local/bin/.
```



NOTE

If you download the CLI on a Mac computer, you might see a warning about the **hcp** binary file. You need to adjust your security settings to allow the binary file to be run.

Verification

- Verify that you see the list of available parameters by running the following command:

```
$ hcp create cluster <platform> --help
```

You can use the **hcp create cluster** command to create and manage hosted clusters. The supported platforms are **agent**, **aws**, and **kubevirt**. The **azure** and **openstack** platforms are also available as Technology Preview features.

3.5. DISTRIBUTING HOSTED CLUSTER WORKLOADS

In hosted control planes for OpenShift Container Platform, cluster management is separate from cluster workload. As you prepare your deployment, ensure you know how you want to distribute your hosted cluster workloads.



IMPORTANT

Do not use the management cluster for your workload. Workloads must not run on nodes where control planes run.

3.5.1. Node labeling for hosted control planes

Before you get started with hosted control planes, you must properly label nodes so that the pods of hosted clusters can be scheduled into infrastructure nodes.

Node labeling is also important for the following reasons:

- To ensure high availability and proper workload deployment. For example, to avoid having the control plane workload count toward your OpenShift Container Platform subscription, you can set the **node-role.kubernetes.io/infra** label.
- To ensure that control plane workloads are separate from other workloads in the management cluster.
- To ensure that control plane workloads are configured at the correct multi-tenancy distribution level for your deployment. The distribution levels are as follows:
 - Everything shared: Control planes for hosted clusters can run on any node that is designated for control planes.
 - Nothing shared: Every control plane has its own dedicated nodes.

For more information about dedicating a node to a single hosted cluster, see "Labeling management cluster nodes".

Additional resources

- [Labeling management cluster nodes](#)
- [Network isolation for hosted clusters](#)

3.5.2. Labeling management cluster nodes

Proper node labeling is a prerequisite to deploying hosted control planes.

As a management cluster administrator, you use the following labels and taints in management cluster nodes to schedule a control plane workload:

- **hypershift.openshift.io/control-plane: true**: Use this label and taint to dedicate a node to running hosted control plane workloads. By setting a value of **true**, you avoid sharing the control plane nodes with other components, for example, the infrastructure components of the management cluster or any other mistakenly deployed workload.
- **hypershift.openshift.io/cluster: \${HostedControlPlane Namespace}**: Use this label and taint when you want to dedicate a node to a single hosted cluster.

Apply the following labels on the nodes that host control-plane pods:

- **node-role.kubernetes.io/infra**: Use this label to avoid having the control-plane workload count toward your subscription.
- **topology.kubernetes.io/zone**: Use this label on the management cluster nodes to deploy highly available clusters across failure domains. The zone might be a location, rack name, or the hostname of the node where the zone is set. For example, a management cluster has the following nodes: **worker-1a**, **worker-1b**, **worker-2a**, and **worker-2b**. The **worker-1a** and **worker-1b** nodes are in **rack1**, and the **worker-2a** and **worker-2b** nodes are in **rack2**. To use each rack as an availability zone, enter the following commands:

```
$ oc label node/worker-1a node/worker-1b topology.kubernetes.io/zone=rack1
```

```
$ oc label node/worker-2a node/worker-2b topology.kubernetes.io/zone=rack2
```

Pods for a hosted cluster have tolerations, and the scheduler uses affinity rules to schedule them. Pods tolerate taints for **control-plane** and the **cluster** for the pods. The scheduler prioritizes the scheduling of pods into nodes that are labeled with **hypershift.openshift.io/control-plane** and **hypershift.openshift.io/cluster: \${HostedControlPlane Namespace}**.

For the **ControllerAvailabilityPolicy** option, use **HighlyAvailable**, which is the default value that the hosted control planes command-line interface, **hcp**, deploys. When you use that option, you can schedule pods for each deployment within a hosted cluster across different failure domains by setting **topology.kubernetes.io/zone** as the topology key. Scheduling pods for a deployment within a hosted cluster across different failure domains is available only for highly available control planes.

Procedure

- To enable a hosted cluster to require its pods to be scheduled into infrastructure nodes, set the **HostedCluster.spec.nodeSelector** specification, as shown in the following example:

```
spec:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
```

This way, hosted control planes for each hosted cluster are eligible infrastructure node workloads, and you do not need to entitle the underlying OpenShift Container Platform nodes.

3.5.3. Priority classes

Four built-in priority classes influence the priority and preemption of the hosted cluster pods.

You can create the pods in the management cluster in the following order from highest to lowest:

- **hypershift-operator**: HyperShift Operator pods.
- **hypershift-etcd**: Pods for etcd.
- **hypershift-api-critical**: Pods that are required for API calls and resource admission to succeed. These pods include pods such as **kube-apiserver**, aggregated API servers, and web hooks.
- **hypershift-control-plane**: Pods in the control plane that are not API-critical but still need elevated priority, such as the cluster version Operator.

3.5.4. Custom taints and tolerations

By default, pods for a hosted cluster tolerate the **control-plane** and **cluster** taints. However, you can also use custom taints on nodes so that hosted clusters can tolerate those taints on a per-hosted-cluster basis by setting the **HostedCluster.spec.tolerations** specification.

IMPORTANT

Passing tolerations for a hosted cluster is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Example configuration

```
spec:
  tolerations:
    - effect: NoSchedule
      key: kubernetes.io/custom
      operator: Exists
```

You can also set tolerations on the hosted cluster while you create a cluster by using the **--tolerations** hcp CLI argument.

Example CLI argument

```
--toleration="key=kubernetes.io/custom,operator=Exists,effect=NoSchedule"
```

For fine granular control of hosted cluster pod placement on a per-hosted-cluster basis, use custom tolerations with **nodeSelectors**. You can co-locate groups of hosted clusters and isolate them from other hosted clusters. You can also place hosted clusters in infra and control plane nodes.

Tolerations on the hosted cluster spread only to the pods of the control plane. To configure other pods that run on the management cluster and infrastructure-related pods, such as the pods to run virtual machines, you need to use a different process.

3.5.5. Control plane isolation

You can configure hosted control planes to isolate network traffic or control plane pods.

Each hosted control plane is assigned to run in a dedicated Kubernetes namespace. By default, the Kubernetes namespace denies all network traffic.

The following network traffic is allowed through the network policy that is enforced by the Kubernetes Container Network Interface (CNI):

- Ingress pod-to-pod communication in the same namespace (intra-tenant)
- Ingress on port 6443 to the hosted **kube-apiserver** pod for the tenant
- Metric scraping from the management cluster Kubernetes namespace with the **network.openshift.io/policy-group: monitoring** label is allowed for monitoring

3.5.5.1. Control plane pod isolation

In addition to network policies, each hosted control plane pod is run with the **restricted** security context constraint. This policy denies access to all host features and requires pods to be run with a unique identifier (UID) and with SELinux context that is allocated uniquely to each namespace that hosts a customer control plane.

The policy ensures the following constraints:

- Pods cannot run as privileged.
- Pods cannot mount host directory volumes.
- Pods must run as a user in a pre-allocated range of UIDs.
- Pods must run with a pre-allocated Multi-Category Security (MCS) label.
- Pods cannot access the host network namespace.
- Pods cannot expose host network ports.
- Pods cannot access the host PID namespace.
- By default, pods drop the following Linux capabilities: **KILL**, **MKNOD**, **SETUID**, and **SETGID**.

The management components, such as **kubelet** and **crio**, on each management cluster worker node are protected by an SELinux label that is not accessible to the SELinux context for pods that support hosted control planes.

The following SELinux labels are used for key processes and sockets:

kubelet

system_u:system_r:unconfined_service_t:s0

crio

system_u:system_r:container_runtime_t:s0

crio.sock

system_u:object_r:container_var_run_t:s0

<example user container processes>

system_u:system_r:container_t:s0:c14,c24

To achieve physical or virtual machine (VM) isolation, you need to use a "shared-nothing" approach, where each control plane has its own dedicated node. Nodes for a specific hosted cluster are tainted and labeled with a specific **hypershift.openshift.io/hosted-cluster** value. Security involves the use of a network policy and physical network access control lists (ACLs) across hosted clusters.

3.6. ENABLING OR DISABLING THE HOSTED CONTROL PLANES FEATURE

The hosted control planes feature, as well as the **hypershift-addon** managed cluster add-on, are enabled by default. If needed, you can disable the feature, or if you disabled it, you can manually enable it.

You can uninstall the HyperShift Operator and disable the hosted control planes feature. When you disable the hosted control planes feature, you must destroy the hosted cluster and the managed cluster resource on multicluster engine Operator, as described in the *Destroying a hosted cluster* section.

Additional resources

- [Destroying a hosted cluster](#)

3.6.1. Manually enabling the hosted control planes feature

If the hosted control planes feature is disabled, you can manually enable it.

Procedure

1. Run the following command to enable the feature:

```
$ oc patch mce multiclusterengine --type=merge -p \
'{"spec":{"overrides":{"components":[{"name":"hypershift","enabled": true}]}}}'
```

The default **MultiClusterEngine** resource instance name is **multiclusterengine**, but you can get the **MultiClusterEngine** name from your cluster by running the following command: **\$ oc get mce**.

2. Run the following command to verify that the **hypershift** and **hypershift-local-hosting** features are enabled in the **MultiClusterEngine** custom resource:

```
$ oc get mce multiclusterengine -o yaml
```

The default **MultiClusterEngine** resource instance name is **multiclusterengine**, but you can get the **MultiClusterEngine** name from your cluster by running the following command: **\$ oc get mce**.

Example output

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  overrides:
    components:
      - name: hypershift
        enabled: true
      - name: hypershift-local-hosting
        enabled: true
```

3.6.2. Manually enabling the hypershift-addon managed cluster add-on for local-cluster

Enabling the hosted control planes feature automatically enables the **hypershift-addon** managed cluster add-on. If you need to enable the **hypershift-addon** managed cluster add-on manually, use the **hypershift-addon** to install the HyperShift Operator on **local-cluster**.

Procedure

1. Create the **ManagedClusterAddon** add-on named **hypershift-addon** by creating a file that resembles the following example:

```
apiVersion: addon.open-cluster-management.io/v1alpha1
kind: ManagedClusterAddOn
metadata:
  name: hypershift-addon
  namespace: local-cluster
spec:
  installNamespace: open-cluster-management-agent-addon
```

2. Apply the file by running the following command:

```
$ oc apply -f <filename>
```

Replace **filename** with the name of the file that you created.

3. Confirm that the **hypershift-addon** managed cluster add-on is installed by running the following command:

```
$ oc get managedclusteraddons -n local-cluster hypershift-addon
```

If the add-on is installed, the output resembles the following example:

```
NAME          AVAILABLE DEGRADED PROGRESSING
hypershift-addon True
```


Your **hypershift-addon** managed cluster add-on is installed and the hosting cluster is available to create and manage hosted clusters.

3.6.3. Uninstalling the HyperShift Operator

Before you can disable the hosted control planes feature, you need to uninstall the HyperShift Operator and disable the **hypershift-addon** from the **local-cluster**.

Procedure

1. Run the following command to ensure that there is no hosted cluster running:

```
$ oc get hostedcluster -A
```



IMPORTANT

If a hosted cluster is running, the HyperShift Operator does not uninstall, even if the **hypershift-addon** is disabled.

2. Disable the **hypershift-addon** by running the following command:

```
$ oc patch mce multiclusterengine --type=merge -p \
'{"spec":{"overrides":{"components":[{"name":"hypershift-local-hosting","enabled": false}]}}}'
```

The default **MultiClusterEngine** resource instance name is **multiclusterengine**, but you can get the **MultiClusterEngine** name from your cluster by running the following command: **\$ oc get mce**.



NOTE

You can also disable the **hypershift-addon** for the **local-cluster** from the multicluster engine Operator console after disabling the **hypershift-addon**.

3.6.4. Disabling the hosted control planes feature

If you no longer use the hosted control planes feature, you can disable it.

Prerequisites

- You uninstalled the HyperShift Operator. For more information, see "Uninstalling the HyperShift Operator".

Procedure

1. Run the following command to disable the hosted control planes feature:

```
$ oc patch mce multiclusterengine --type=merge -p \
'{"spec":{"overrides":{"components":[{"name":"hypershift","enabled": false}]}}}'
```

The default **MultiClusterEngine** resource instance name is **multiclusterengine**, but you can get the **MultiClusterEngine** name from your cluster by running the following command: **\$ oc get mce**.

2. You can verify that the **hypershift** and **hypershift-local-hosting** features are disabled in the **MultiClusterEngine** custom resource by running the following command:

```
$ oc get mce multiclusterengine -o yaml
```

The default **MultiClusterEngine** resource instance name is **multiclusterengine**, but you can get the **MultiClusterEngine** name from your cluster by running the following command: **\$ oc get mce**.

See the following example where **hypershift** and **hypershift-local-hosting** have their **enabled:** flags set to **false**:

```
apiVersion: multicluster.openshift.io/v1
kind: MultiClusterEngine
metadata:
  name: multiclusterengine
spec:
  overrides:
    components:
      - name: hypershift
        enabled: false
      - name: hypershift-local-hosting
        enabled: false
```

CHAPTER 4. DEPLOYING HOSTED CONTROL PLANES

4.1. DEPLOYING HOSTED CONTROL PLANES ON AWS

To reduce infrastructure costs and improve cluster management efficiency, you can deploy hosted control planes on AWS. This configuration decouples the control plane from the data plane so that you can manage multiple clusters from a central management service.

A *hosted cluster* is an OpenShift Container Platform cluster with its API endpoint and control plane that are hosted on the management cluster. The hosted cluster includes the control plane and its corresponding data plane. To configure hosted control planes on premises, you must install multicluster engine for Kubernetes Operator in a management cluster. By deploying the HyperShift Operator on an existing managed cluster by using the **hypershift-addon** managed cluster add-on, you can enable that cluster as a management cluster and start to create the hosted cluster. The **hypershift-addon** managed cluster add-on is enabled by default for the **local-cluster** managed cluster.

You can use the multicluster engine Operator console or the hosted control plane command-line interface (CLI), **hcp**, to create a hosted cluster. The hosted cluster is automatically imported as a managed cluster. However, you can [disable this automatic import feature into multicluster engine Operator](#).

4.1.1. Preparing to deploy hosted control planes on AWS

Preparing to deploy hosted control planes on Amazon Web Services (AWS) involves meeting several prerequisites and creating resources, including an S3 bucket, an OIDC secret, a routable public zone, IAM role and STS credentials.

4.1.1.1. Prerequisites to deploy hosted control planes on AWS

To ensure successful deployment of hosted control planes on Amazon Web Services (AWS), your environment must meet the following requirements.

- You installed the multicluster engine for Kubernetes Operator 2.5 and later on an OpenShift Container Platform cluster. The multicluster engine Operator is automatically installed when you install Red Hat Advanced Cluster Management (RHACM). The multicluster engine Operator can also be installed without RHACM as an Operator from the OpenShift Container Platform software catalog.
- You have at least one managed OpenShift Container Platform cluster for the multicluster engine Operator. The **local-cluster** is automatically imported in the multicluster engine Operator version 2.5 and later. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- You installed the [aws command-line interface \(CLI\)](#).
- You installed the hosted control plane CLI, **hcp**.



IMPORTANT

- Run the management cluster and compute nodes on the same platform.
- For each hosted cluster, provide a cluster-wide unique name. A hosted cluster name cannot be the same as any existing managed cluster in order for multicluster engine Operator to manage it.
- Do not use **clusters** as a hosted cluster name.
- Do not create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.

Additional resources

- [Configuring Ansible Automation Platform jobs to run on hosted clusters](#)
- [Advanced configuration](#)
- [Enabling the central infrastructure management service](#)
- [Manually enabling the hosted control planes feature](#)
- [Disabling the hosted control planes feature](#)
- [Deploying the SR-IOV Operator for hosted control planes](#)

4.1.1.2. Creating the Amazon Web Services S3 bucket and S3 OIDC secret

Before you can create and manage a hosted cluster on Amazon Web Services (AWS), you must create the S3 bucket and S3 OIDC secret. These resources provide a place for the cluster to store information about itself and a way for the cluster to prove its identity to AWS.

Procedure

1. Create an S3 bucket that has public access to host OIDC discovery documents for your clusters.
 - a. Enter the following command:

```
$ aws s3api create-bucket --bucket <bucket_name> \
  --create-bucket-configuration LocationConstraint=<region> \
  --region <region> 1
```

where:

<bucket_name>

Specifies the name of the S3 bucket you are creating.

<region>

Specifies that you want to create the bucket in a region other than the **us-east-1** region. Include this line and replace **<region>** with the region you want to use. To create a bucket in the **us-east-1** region, omit this line.

- b. Enter the following command:

```
$ aws s3api delete-public-access-block --bucket <bucket_name>
```



Replace **<bucket_name>** with the name of the S3 bucket you are creating.

- c. Enter the following command:

```
$ echo '{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::<bucket_name>/*"
    }
  ]
}' | envsubst > policy.json
```

Replace **<bucket_name>** with the name of the S3 bucket you are creating.

- d. Enter the following command:

```
$ aws s3api put-bucket-policy --bucket <bucket_name> \
  --policy file://policy.json
```

Replace **<bucket_name>** with the name of the S3 bucket you are creating.



NOTE

If you are using a Mac computer, you must export the bucket name in order for the policy to work.

2. Create an OIDC S3 secret named **hypershift-operator-oidc-provider-s3-credentials** for the HyperShift Operator.
3. Save the secret in the **local-cluster** namespace.
4. See the following table to verify that the secret contains the following fields:

Table 4.1. Required fields for the AWS secret

Field name	Description
bucket	Contains an S3 bucket with public access to host OIDC discovery documents for your hosted clusters.
credentials	A reference to a file that contains the credentials of the default profile that can access the bucket. By default, HyperShift only uses the default profile to operate the bucket .
region	Specifies the region of the S3 bucket.

- To create an AWS secret, run the following command:

```
$ oc create secret generic <secret_name> \
  --from-file=credentials=<path>/.aws/credentials \
  --from-literal=bucket=<s3_bucket> \
  --from-literal=region=<region> \
  -n local-cluster
```



NOTE

Disaster recovery backup for the secret is not automatically enabled. To add the label that enables the **hypershift-operator-oidc-provider-s3-credentials** secret to be backed up for disaster recovery, run the following command:

```
$ oc label secret hypershift-operator-oidc-provider-s3-credentials \
  -n local-cluster cluster.open-cluster-management.io/backup=true
```

4.1.1.3. Creating a routable public zone for hosted clusters

In order to access applications in your hosted clusters, you must configure the routable public zone.

If the public zone exists, skip this step. Otherwise, the public zone affects the existing functions.

Procedure

- To create a routable public zone for DNS records, enter the following command:

```
$ aws route53 create-hosted-zone \
  --name <basedomain> \
  --caller-reference $(whoami)-$(date --rfc-3339=date)
```

Replace **<basedomain>** with your base domain, for example, **www.example.com**.

4.1.1.4. Creating an AWS IAM role and STS credentials

Before you create a hosted cluster on Amazon Web Services (AWS), you must create an AWS IAM role and STS credentials.

Procedure

- Get the Amazon Resource Name (ARN) of your user by running the following command:

```
$ aws sts get-caller-identity --query "Arn" --output text
```

Example output

```
arn:aws:iam::1234567890:user/<aws_username>
```

Use this output as the value for the **<arn>** value in the next step.

- Create a JSON file that contains the trust relationship configuration for your role. See the following example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "<arn>"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Replace **<arn>** with the ARN of your user that you noted in the previous step.

3. Create the Identity and Access Management (IAM) role by running the following command:

```
$ aws iam create-role \
  --role-name <name> \
  --assume-role-policy-document file://<file_name>.json \
  --query "Role.Arn"
```

where:

<name>

Specifies the role name, for example, **hcp-cli-role**.

<file_name>

Specifies the name of the JSON file you created in the previous step.

Example output

```
arn:aws:iam::820196288204:role/myrole
```

4. Create a JSON file named **policy.json** that contains the following permission policies for your role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EC2",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateDhcpOptions",
        "ec2:DeleteSubnet",
        "ec2:ReplaceRouteTableAssociation",
        "ec2:DescribeAddresses",
        "ec2:DescribeInstances",
        "ec2:DeleteVpcEndpoints",
        "ec2:CreateNatGateway",
        "ec2:CreateVpc",
        "ec2:DescribeDhcpOptions",
        "ec2:AttachInternetGateway",
        "ec2:DeleteVpcEndpointServiceConfigurations",

```

```

        "ec2:DeleteRouteTable",
        "ec2:AssociateRouteTable",
        "ec2:DescribeInternetGateways",
        "ec2:DescribeAvailabilityZones",
        "ec2:CreateRoute",
        "ec2:CreateInternetGateway",
        "ec2:RevokeSecurityGroupEgress",
        "ec2:ModifyVpcAttribute",
        "ec2:DeleteInternetGateway",
        "ec2:DescribeVpcEndpointConnections",
        "ec2:RejectVpcEndpointConnections",
        "ec2:DescribeRouteTables",
        "ec2:ReleaseAddress",
        "ec2:AssociateDhcpOptions",
        "ec2:TerminateInstances",
        "ec2:CreateTags",
        "ec2:DeleteRoute",
        "ec2:CreateRouteTable",
        "ec2:DetachInternetGateway",
        "ec2:DescribeVpcEndpointServiceConfigurations",
        "ec2:DescribeNatGateways",
        "ec2:DisassociateRouteTable",
        "ec2:AllocateAddress",
        "ec2:DescribeSecurityGroups",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:CreateVpcEndpoint",
        "ec2:DescribeVpcs",
        "ec2:DeleteSecurityGroup",
        "ec2:DeleteDhcpOptions",
        "ec2:DeleteNatGateway",
        "ec2:DescribeVpcEndpoints",
        "ec2:DeleteVpc",
        "ec2:CreateSubnet",
        "ec2:DescribeSubnets"
    ],
    "Resource": "*"
},
{
    "Sid": "ELB",
    "Effect": "Allow",
    "Action": [
        "elasticloadbalancing:DeleteLoadBalancer",
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeTargetGroups",
        "elasticloadbalancing>DeleteTargetGroup"
    ],
    "Resource": "*"
},
{
    "Sid": "IAMPassRole",
    "Effect": "Allow",
    "Action": "iam:PassRole",
    "Resource": "arn:*.iam:.*:role/*-worker-role",
    "Condition": {
        "ForAnyValue:StringEqualsIfExists": {
            "iam:PassedToService": "ec2.amazonaws.com"
        }
    }
}

```



```

    }
  }
},
{
  "Sid": "IAM",
  "Effect": "Allow",
  "Action": [
    "iam:CreateInstanceProfile",
    "iam:DeleteInstanceProfile",
    "iam:TagInstanceProfile",
    "iam:GetRole",
    "iam:UpdateAssumeRolePolicy",
    "iam:GetInstanceProfile",
    "iam:TagRole",
    "iam:RemoveRoleFromInstanceProfile",
    "iam:CreateRole",
    "iam:DeleteRole",
    "iam:PutRolePolicy",
    "iam:AddRoleToInstanceProfile",
    "iam:CreateOpenIDConnectProvider",
    "iam:TagOpenIDConnectProvider",
    "iam:ListOpenIDConnectProviders",
    "iam:DeleteRolePolicy",
    "iam:UpdateRole",
    "iam:DeleteOpenIDConnectProvider",
    "iam:GetRolePolicy",
    "iam:ListAttachedRolePolicies",
    "iam:ListRolePolicies",
    "iam:DetachRolePolicy"
  ],
  "Resource": "*"
},
{
  "Sid": "Route53",
  "Effect": "Allow",
  "Action": [
    "route53:ListHostedZonesByVPC",
    "route53:CreateHostedZone",
    "route53:ListHostedZones",
    "route53:ChangeResourceRecordSets",
    "route53:ListResourceRecordSets",
    "route53>DeleteHostedZone",
    "route53:AssociateVPCWithHostedZone",
    "route53:ListHostedZonesByName"
  ],
  "Resource": "*"
},
{
  "Sid": "S3",
  "Effect": "Allow",
  "Action": [
    "s3:ListAllMyBuckets",
    "s3:ListBucket",
    "s3>DeleteObject",
    "s3>DeleteBucket"
  ],

```

```

    "Resource": "*"
  }
]
}

```

5. Attach the **policy.json** file that contains the permissions policies for your role by running the following command:

```

$ aws iam put-role-policy \
  --role-name <role_name> \
  --policy-name <policy_name> \
  --policy-document file:///policy.json

```

where:

<role_name>

Specifies the name of your role.

<policy_name>

Specifies your policy name.

6. Retrieve STS credentials in a JSON file named **sts-creds.json** by running the following command:

```

$ aws sts get-session-token --output json > sts-creds.json

```

Example sts-creds.json file

```

{
  "Credentials": {
    "AccessKeyId": "<access_key_id>",
    "SecretAccessKey": "<secret_access_key>",
    "SessionToken": "<session_token>",
    "Expiration": "<time_stamp>"
  }
}

```

4.1.1.5. Enabling AWS PrivateLink for hosted control planes

In order to provision hosted control planes on the Amazon Web Services (AWS) with PrivateLink, you need to enable AWS PrivateLink for hosted control planes.

Procedure

1. Create an AWS credential secret for the HyperShift Operator and name it **hypershift-operator-private-link-credentials**. The secret must reside in the managed cluster namespace that is the namespace of the managed cluster being used as the management cluster. If you used **local-cluster**, create the secret in the **local-cluster** namespace.
2. See the following table to confirm that the secret contains the required fields:

Table 4.2. Required fields for the AWS secret

Field name	Description	Optional or required
region	Region for use with Private Link	Required
aws-access-key-id	The credential access key id.	Required
aws-secret-access-key	The credential access key secret.	Required

3. To create an AWS secret, run the following command:

```
$ oc create secret generic <secret_name> \
  --from-literal=aws-access-key-id=<aws_access_key_id> \
  --from-literal=aws-secret-access-key=<aws_secret_access_key> \
  --from-literal=region=<region> -n local-cluster
```

4. Disaster recovery backup for the secret is not automatically enabled. Run the following command to add the label that enables the **hypershift-operator-private-link-credentials** secret to be backed up for disaster recovery:

```
$ oc label secret hypershift-operator-private-link-credentials \
  -n local-cluster \
  cluster.open-cluster-management.io/backup=""
```

4.1.2. Enabling external DNS for hosted control planes on AWS

To automate the management of DNS records, you can enable external DNS. By configuring this feature, you provide a way for the cluster to update your AWS Route 53 public hosted zones automatically when you create or delete services and ingresses.

The control plane and the data plane are separate in hosted control planes. You can configure DNS in two independent areas:

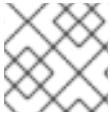
- Ingress for workloads within the hosted cluster, such as the following domain: ***.apps.service-consumer-domain.com**.
- Ingress for service endpoints within the management cluster, such as API or OAuth endpoints through the service provider domain: ***.service-provider-domain.com**.

The input for **hostedCluster.spec.dns** manages the ingress for workloads within the hosted cluster. The input for **hostedCluster.spec.services.servicePublishingStrategy.route.hostname** manages the ingress for service endpoints within the management cluster.

External DNS creates name records for hosted cluster services that specify a publishing type of **LoadBalancer** or **Route** and provide a hostname for that publishing type. For hosted clusters with **Private** or **PublicAndPrivate** endpoint access types, only the **APIServer** and **OAuth** services support hostnames. For **Private** hosted clusters, the DNS record resolves to a private IP address of a Virtual Private Cloud (VPC) endpoint in your VPC.

A hosted control plane exposes the following services:

- **APIServer**
- **OIDC**

**NOTE**

The **NodePort** publishing type is not supported on hosted control planes on AWS.

You can expose these services by using the **servicePublishingStrategy** field in the **HostedCluster** specification. By default, for the **LoadBalancer** and **Route** types of **servicePublishingStrategy**, you can publish the service in one of the following ways:

- By using the hostname of the load balancer that is in the status of the **Service** with the **LoadBalancer** type.
- By using the **status.host** field of the **Route** resource.

However, when you deploy hosted control planes in a managed service context, those methods can expose the ingress subdomain of the underlying management cluster and limit options for the management cluster lifecycle and disaster recovery.

When a DNS indirection is layered on the **LoadBalancer** and **Route** publishing types, a managed service operator can publish all public hosted cluster services by using a service-level domain. This architecture allows remapping on the DNS name to a new **LoadBalancer** or **Route** and does not expose the ingress domain of the management cluster. Hosted control planes uses external DNS to achieve that indirection layer.

You can deploy **external-dns** alongside the HyperShift Operator in the **hypershift** namespace of the management cluster. External DNS watches for **Services** or **Routes** that have the **external-dns.alpha.kubernetes.io/hostname** annotation. That annotation is used to create a DNS record that points to the **Service**, such as an A record, or the **Route**, such as a CNAME record.

You can use external DNS on cloud environments only. For the other environments, you need to manually configure DNS and services.

For more information about external DNS, see [external DNS](#).

4.1.2.1. Setting up external DNS for hosted control planes

To automate the management of DNS records, you can set up external DNS for hosted control planes on AWS. You need this configuration to ensure that when you create or modify services, the corresponding DNS records in your AWS Route 53 public hosted zones update automatically.

You can provision hosted control planes with external DNS or service-level DNS.

Prerequisites

- You created an external public domain.
- You have access to the AWS Route53 Management console.
- You enabled AWS PrivateLink for hosted control planes.

Procedure

1. Create an Amazon Web Services (AWS) credential secret for the HyperShift Operator and name it **hypershift-operator-external-dns-credentials** in the **local-cluster** namespace.
2. Verify that the secret has the required fields. For your reference, the required fields are detailed in the following table.

Table 4.3. Required fields for the AWS secret

Field name	Description	Optional or required
provider	The DNS provider that manages the service-level DNS zone.	Required
domain-filter	The service-level domain.	Required
credentials	The credential file that supports all external DNS types.	Optional when you use AWS keys
aws-access-key-id	The credential access key id.	Optional when you use the AWS DNS service
aws-secret-access-key	The credential access key secret.	Optional when you use the AWS DNS service

3. Create an AWS secret by running the following command:

```
$ oc create secret generic <secret_name> \
  --from-literal=provider=aws \
  --from-literal=domain-filter=<domain_name> \
  --from-file=credentials=<path_to_aws_credentials_file> -n local-cluster
```

NOTE

Disaster recovery backup for the secret is not automatically enabled. To back up the secret for disaster recovery, add the **hypershift-operator-external-dns-credentials** by entering the following command:

```
$ oc label secret hypershift-operator-external-dns-credentials \
  -n local-cluster \
  cluster.open-cluster-management.io/backup=""
```

4.1.2.2. Creating the public DNS hosted zone

You can create the public DNS hosted zone to use as the external DNS domain filter. The External DNS Operator uses the public DNS hosted zone to create your public hosted cluster.

Procedure

1. In the AWS Route 53 management console, click **Create hosted zone**.

2. On the **Hosted zone configuration** page, type a domain name, verify that **Public hosted zone** is selected as the type, and click **Create hosted zone**.
3. After the zone is created, on the **Records** tab, note the values in the **Value/Route traffic to** column.
4. In the main domain, create an NS record to redirect the DNS requests to the delegated zone. In the **Value** field, enter the values that you noted in the previous step.
5. Click **Create records**.
6. Verify that the DNS hosted zone is working by creating a test entry in the new subzone and testing it with a **dig** command, such as in the following example:

```
$ dig +short test.user-dest-public.aws.kerberos.com
```

Example output

```
192.168.1.1
```

7. To create a hosted cluster that sets the hostname for the **LoadBalancer** and **Route** services, enter the following command:

```
$ hcp create cluster aws --name=<hosted_cluster_name> \
  --endpoint-access=PublicAndPrivate \
  --external-dns-domain=<public_hosted_zone> ...
```

Replace **<public_hosted_zone>** with the public hosted zone that you created.

Example services block for the hosted cluster

```
platform:
  aws:
    endpointAccess: PublicAndPrivate
  ...
services:
  - service: APIServer
    servicePublishingStrategy:
      route:
        hostname: api-example.service-provider-domain.com
        type: Route
  - service: OAuthServer
    servicePublishingStrategy:
      route:
        hostname: oauth-example.service-provider-domain.com
        type: Route
  - service: Konnectivity
    servicePublishingStrategy:
      type: Route
  - service: Ignition
    servicePublishingStrategy:
      type: Route
```

The Control Plane Operator creates the **Services** and **Routes** resources and annotates them

with the **external-dns.alpha.kubernetes.io/hostname** annotation. For **Services** and **Routes**, the Control Plane Operator uses a value of the **hostname** parameter in the **servicePublishingStrategy** field for the service endpoints. To create the DNS records, you can use a mechanism, such as the **external-dns** deployment.

You can configure service-level DNS indirection for public services only. You cannot set **hostname** for private services because they use the **hypershift.local** private zone.

The following table shows when it is valid to set **hostname** for a service and endpoint combinations:

Table 4.4. Service and endpoint combinations to set **hostname**

Service	Public	PublicAnd Private	Private
APIServer	Y	Y	N
OAuthServer	Y	Y	N
Konnectivity	Y	N	N
Ignition	Y	N	N

4.1.2.3. Creating a hosted cluster by using the external DNS on AWS

When you create a hosted cluster on AWS, using external DNS provides advantages over standard installation methods. With external DNS, you can automatically synchronize your cluster's service endpoints with AWS Route 53.

Without external DNS, you must manually manage DNS records for every new service or ingress, which increases the risk of configuration errors and downtime.

Prerequisites

- You configured the following artifacts in your management cluster:
 - The public DNS hosted zone
 - The External DNS Operator
 - The HyperShift Operator

Procedure

1. On the **hcp** command-line interface (CLI), enter the following command to access your management cluster:

```
$ export KUBECONFIG=<path_to_management_cluster_kubeconfig>
```

2. Verify that the External DNS Operator is running by entering the following command:

```
$ oc get pod -n hypershift -lapp=external-dns
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
external-dns-7c89788c69-rn8gp	1/1	Running	0	40s

- To create a hosted cluster by using external DNS, enter the following command:

```
$ hcp create cluster aws \
  --role-arn <arn_role> \
  --instance-type <instance_type> \
  --region <region> \
  --auto-repair \
  --generate-ssh \
  --name <hosted_cluster_name> \
  --namespace clusters \
  --base-domain <service_consumer_domain> \
  --node-pool-replicas <node_replica_count> \
  --pull-secret <path_to_your_pull_secret> \
  --release-image quay.io/openshift-release-dev/ocp-release:<ocp_release_image> \
  --external-dns-domain=<service_provider_domain> \
  --endpoint-access=<endpoint_access_configuration> \
  --sts-creds <path_to_sts_credential_file>
```

where:

<arn_role>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.

<instance_types>

Specifies the instance type, for example, **m6i.xlarge**.

<region>

Specifies the AWS region, for example, **us-east-1**.

<hosted_cluster_name>

Specifies your hosted cluster name, for example, **my-external-aws**.

<service_consumer_domain>

Specifies the public hosted zone that the service consumer owns, for example, **service-consumer-domain.com**.

<node_replica_count>

Specifies the node replica count, for example, **2**.

<path_to_your_pull_secret>

Specifies the path to your pull secret file.

<ocp_release_image>

Specifies the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**.

<service_provider_domain>

Specifies the public hosted zone that the service provider owns, for example, **service-provider-domain.com**.

<endpoint_access_configuraton>

Specifies the endpoint access configuration for the external DNS. Set as **PublicAndPrivate**. You can use external DNS with **Public** or **PublicAndPrivate** configurations only.

<path_to_sts_credential_file>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

4.1.2.4. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA
- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```
#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
              - xxx.example.com
              - yyy.example.com
            servingCertificate:
              name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>
```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the **HostedCluster** namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.1.3. Creating a hosted cluster on AWS

On AWS, you can create a hosted cluster by using the command-line interface, **hcp**, or by providing AWS STS credentials. You can also create a hosted cluster in multiple zones on AWS.

A *hosted cluster* is an OpenShift Container Platform cluster with its API endpoint and control plane hosted on a management cluster. The hosted cluster includes the control plane and its corresponding data plane.

The hosted cluster is automatically imported as a managed cluster. If you want to disable this automatic import feature, see "Disabling the automatic import of hosted clusters into multicluster engine Operator".

By default for hosted control planes on AWS, you use an AMD64 hosted cluster. However, you can enable hosted control planes to run on an ARM64 hosted cluster. For more information, see "Running hosted clusters on an ARM64 architecture".

For compatible combinations of node pools and hosted clusters, see the following table:

Table 4.5. Compatible architectures for node pools and hosted clusters

Hosted cluster	Node pools
AMD64	AMD64 or ARM64
ARM64	ARM64 or AMD64

Additional resources

- [Disabling the automatic import of hosted clusters into multicluster engine Operator](#)
- [Running hosted clusters on an ARM64 architecture](#)

4.1.3.1. Creating a hosted cluster on AWS by using the CLI

To create a hosted cluster on Amazon Web Services (AWS), you can use the hosted control plane command-line interface (**hcp**).

Prerequisites

- You have set up the hosted control plane CLI, **hcp**.
- You have enabled the **local-cluster** managed cluster as the management cluster.
- You created an AWS Identity and Access Management (IAM) role and AWS Security Token Service (STS) credentials.

Procedure

1. To create a hosted cluster on AWS, run the following command:

```
$ hcp create cluster aws \
  --name <hosted_cluster_name> \
  --infra-id <infra_id> \
  --base-domain <basedomain> \
  --sts-creds <path_to_sts_credential_file> \
  --pull-secret <path_to_pull_secret> \
  --region <region> \
  --generate-ssh \
  --node-pool-replicas <node_pool_replica_count> \
  --namespace <hosted_cluster_namespace> \
  --role-arn <role_name> \
  --render-into <file_name>.yaml
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster.

<infra_id>

Specifies your infrastructure name. You must provide the same value for **<hosted_cluster_name>** and **<infra_id>**. Otherwise the cluster might not appear correctly in the multicluster engine for Kubernetes Operator console.

<basedomain>

Specifies your base domain, for example, **example.com**.

<path_to_sts_credential_file>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<region>

Specifies the AWS region name, for example, **us-east-1**.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **3**.

<hosted_cluster_namespace>

Specifies that you want to create the **HostedCluster** and **NodePool** custom resource in a specific namespace. Otherwise, by default, all **HostedCluster** and **NodePool** custom resources are created in the **clusters** namespace.

<role_name>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.

<file_name>

Specifies whether the EC2 instance runs on shared or single tenant hardware. The **--render-into** flag renders Kubernetes resources into the YAML file that you specify in this field. Continue to the next step to edit the YAML file.

- If you included the **--render-into** flag in the previous command, edit the specified YAML file. Edit the **NodePool** specification in the YAML file to indicate whether the EC2 instance should run on shared or single-tenant hardware, similar to the following example:

Example YAML file

```
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: <nodepool_name>
spec:
  platform:
    aws:
      placement:
        tenancy: "default"
```

where:

<nodepool_name>

Specifies the name of the **NodePool** resource.

spec.platform.aws.placement.tenancy

Specifies a valid value for tenancy: **"default"**, **"dedicated"**, or **"host"**. Use **"default"** when node pool instances run on shared hardware. Use **"dedicated"** when each node pool instance runs on single-tenant hardware. Use **"host"** when node pool instances run on your

pre-allocated dedicated hosts.

3. If you use external load balancers, configure the ingress endpoint by editing the **HostedCluster** resource as shown in the following example. If you do not configure the endpoint, the default behavior is to randomize the node port that the service exposes the ingress on. To configure how the ingress controller publishes the default ingress route, set the **endpointPublishingStrategy** parameter and its underlying functions:

```
#...
spec:
  operatorConfiguration:
    ingressOperator:
      endpointPublishingStrategy:
        type: LoadBalancerService
      loadBalancer:
        scope: Internal
#...
```

The **spec.operatorConfiguration.ingressOperator.endPointPublishingStrategy.type** parameter specifies the endpoint for the load balancer. For AWS, use the **LoadBalancerService** type.

Verification

1. Verify the status of your hosted cluster to check that the value of **AVAILABLE** is **True**. Run the following command:

```
$ oc get hostedclusters -n <hosted_cluster_namespace>
```

2. Get a list of your node pools by running the following command:

```
$ oc get nodepools --namespace <hosted_cluster_namespace>
```

Additional resources

- [Configuring a custom API server certificate in a hosted cluster](#)

4.1.3.2. Creating a hosted cluster by providing AWS STS credentials

To enhance the security of your hosted control plane deployment, you can create a hosted cluster on AWS by using the AWS Security Token Service (STS).

When you create a hosted cluster by using the **hcp create cluster aws** command, you must provide an Amazon Web Services (AWS) account credentials that have permissions to create infrastructure resources for your hosted cluster.

Infrastructure resources include the following examples:

- Virtual Private Cloud (VPC)
- Subnets
- Network address translation (NAT) gateways

You can provide the AWS credentials by using either of the following ways:

- The AWS Security Token Service (STS) credentials
- The AWS cloud provider secret from multicluster engine Operator

Procedure

- To create a hosted cluster on AWS by providing AWS STS credentials, enter the following command:

```
$ hcp create cluster aws \
  --name <hosted_cluster_name> \
  --node-pool-replicas <node_pool_replica_count> \
  --base-domain <base_domain> \
  --pull-secret <path_to_pull_secret> \
  --sts-creds <path_to_sts_credential_file> \
  --region <region> \
  --role-arn <arn_role>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, for example, **my-hosted-cluster-01**.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **2**.

<base_domain>

Specifies your base domain, for example, **example.com**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<path_to_sts_credentials>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

<region>

Specifies the AWS region name, for example, **us-east-1**.

<arn_role>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.

4.1.3.3. Creating a hosted cluster in multiple zones on AWS

To improve availability and fault tolerance, you can create a hosted cluster across multiple AWS availability zones. Distributing your node pools and compute nodes across several zones protects your workloads against potential outages in a single geographical region.

You can create a hosted cluster in multiple zones on Amazon Web Services (AWS) by using the **hcp** command-line interface (CLI).

Prerequisites

- You created an AWS Identity and Access Management (IAM) role and AWS Security Token Service (STS) credentials.

Procedure

- Create a hosted cluster in multiple zones on AWS by running the following command:

```
$ hcp create cluster aws \
  --name <hosted_cluster_name> \
  --node-pool-replicas=<node_pool_replica_count> \
  --base-domain <base_domain> \
  --pull-secret <path_to_pull_secret> \
  --role-arn <arn_role> \
  --region <region> \
  --zones <zones> \
  --sts-creds <path_to_sts_credential_file>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, such as **my-hosted-cluster-01**.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **2**.

<base_domain>

Specifies your base domain, for example, **example.com**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<arn_role>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.

<region>

Specifies the AWS region name, for example, **us-east-1**.

<zones>

Specifies availability zones within your AWS region, for example, **us-east-1a**, and **us-east-1b**. For each specified zone, the following infrastructure is created: public subnet, private subnet, NAT gateway, and private route table. A public route table is shared across public subnets. One **NodePool** resource is created for each zone. The node pool name is suffixed by the zone name. The private subnet for the zone is set in the **spec.platform.aws.subnet.id** parameter.

<path_to_sts_credential_file>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

4.1.4. Accessing a hosted cluster on AWS

After you create a hosted cluster on AWS, you can access it by using your **kubeconfig** file, access secrets, and **kubeadmin** credentials.

The hosted cluster namespace contains hosted cluster resources and the access secrets. The hosted control plane runs in the hosted control plane namespace.

The secret name formats are shown in the following table:

Table 4.6. Access secrets

Secret	Format	Example
kubeconfig secret	<hosted_cluster_namespace>-<name>-admin-kubeconfig	clusters-hypershift-demo-admin-kubeconfig
kubeadmin password secret	<hosted_cluster_namespace>-<name>-kubeadmin-password	clusters-hypershift-demo-kubeadmin-password



NOTE

The **kubeadmin** password secret is Base64-encoded and the **kubeconfig** secret contains a Base64-encoded **kubeconfig** configuration. You must decode the Base64-encoded **kubeconfig** configuration and save it into a **<hosted_cluster_name>.kubeconfig** file.

Procedure

1. Generate the **kubeconfig** file by entering the following command:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \
--name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

2. Use your **<hosted_cluster_name>.kubeconfig** file that contains the decoded **kubeconfig** configuration to access the hosted cluster. Enter the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

You must decode the **kubeadmin** password secret to log in to the API server or the console of the hosted cluster.

4.1.5. Running hosted clusters on an ARM64 architecture

By default for hosted control planes on Amazon Web Services (AWS), you use an AMD64 hosted cluster. However, you can enable hosted control planes to run on an ARM64 hosted cluster.

For compatible combinations of node pools and hosted clusters, see the following table:

Table 4.7. Compatible architectures for node pools and hosted clusters

Hosted cluster	Node pools
AMD64	AMD64 or ARM64

Hosted cluster	Node pools
ARM64	ARM64 or AMD64

4.1.5.1. Creating a hosted cluster on an ARM64 OpenShift Container Platform cluster

You can run a hosted cluster on an ARM64 OpenShift Container Platform cluster for Amazon Web Services (AWS) by overriding the default release image with a multi-architecture release image.

If you do not use a multi-architecture release image, the compute nodes in the node pool are not created and reconciliation of the node pool stops until you either use a multi-architecture release image in the hosted cluster or update the **NodePool** custom resource based on the release image.

Prerequisites

- You must have an OpenShift Container Platform cluster with a 64-bit ARM infrastructure that is installed on AWS. For more information, see "Create an OpenShift Container Platform Cluster: AWS (ARM)".
- You must create an AWS Identity and Access Management (IAM) role and AWS Security Token Service (STS) credentials. For more information, see "Creating an AWS IAM role and STS credentials".

Procedure

- Create a hosted cluster on an ARM64 OpenShift Container Platform cluster by entering the following command:

```
$ hcp create cluster aws \
  --name <hosted_cluster_name> \
  --node-pool-replicas <node_pool_replica_count> \
  --base-domain <base_domain> \
  --pull-secret <path_to_pull_secret> \
  --sts-creds <path_to_sts_credential_file> \
  --region <region> \
  --release-image quay.io/openshift-release-dev/ocp-release:<ocp_release_image> \
  --role-arn <role_name>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, for example, **my-hosted-cluster-01**.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **3**.

<base_domain>

Specifies your base domain, for example, **example.com**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<path_to_sts_credential_file>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

<region>

Specifies the AWS region name, for example, **us-east-1**.

<ocp_release_image>

Specifies the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**. If you are using a disconnected environment, replace **<ocp_release_image>** with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".

<role_name>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.

Additional resources

- [Extracting the release image digest](#)
- [Create an OpenShift Container Platform Cluster: AWS \(ARM\)](#)
- [Creating an AWS IAM role and STS credentials](#)

4.1.5.2. Creating an ARM or AMD NodePool object on AWS hosted clusters

You can schedule application workloads that are the **NodePool** objects on 64-bit ARM and AMD from the same hosted control plane. To set the required processor architecture for the **NodePool** object, you define the **arch** field in the **NodePool** specification.

The valid values for the **arch** field are as follows:

- **arm64**
- **amd64**

Prerequisites

- You must have a multi-architecture image for the **HostedCluster** custom resource to use. You can access multi-architecture nightly images. For more information, see "Multi-architecture nightly images".

Procedure

- Add an ARM or AMD **NodePool** object to the hosted cluster on AWS by running the following command:

```
$ hcp create nodepool aws \
  --cluster-name <hosted_cluster_name> \
  --name <node_pool_name> \
  --node-count <node_pool_replica_count> \
  --arch <architecture>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, for example, **my-hosted-cluster-01**.

<node_pool_name>

Specifies the node pool name.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **3**.

<architecture>

Specifies the architecture type, such as **arm64** or **amd64**. If you do not specify a value for the **--arch** flag, the **amd64** value is used by default.

Additional resources

- [Multi-architecture nightly images](#)

4.1.6. Creating a private hosted cluster on AWS

After you enable the **local-cluster** as the management cluster, you can deploy a hosted cluster or a private hosted cluster on Amazon Web Services (AWS).

By default, hosted clusters are publicly accessible through public DNS and the default router for the management cluster.

For private clusters on AWS, all communication with the hosted cluster occurs over AWS PrivateLink.

Prerequisites

- You enabled AWS PrivateLink. For more information, see "Enabling AWS PrivateLink".
- You created an AWS Identity and Access Management (IAM) role and AWS Security Token Service (STS) credentials. For more information, see "Creating an AWS IAM role and STS credentials" and "Identity and Access Management (IAM) permissions".
- You configured a bastion instance on AWS. For more information, see "Tutorial: Configuring private network access using a Linux Bastion Host".

Procedure

- Create a private hosted cluster on AWS by entering the following command:

```
$ hcp create cluster aws \
  --name <hosted_cluster_name> \
  --node-pool-replicas=<node_pool_replica_count> \
  --base-domain <basedomain> \
  --pull-secret <path_to_pull_secret> \
  --sts-creds <path_to_sts_credential_file> \
  --region <region> \
  --endpoint-access Private \
  --role-arn <role_name>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, such as, **example**.

<node_pool_replica_count>

Specifies the node pool replica count, for example, **3**.

<basedomain>

Specifies your base domain, for example, **example.com**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<path_to_sts_credential_file>

Specifies the path to your AWS STS credentials file, for example, **/home/user/sts-creds/sts-creds.json**.

<region>

Specifies the AWS region name, for example, **us-east-1**.

Private

Specifies that the cluster is private.

<role_name>

Specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**. For more information about ARN roles, see "Identity and Access Management (IAM) permissions".

The following API endpoints for the hosted cluster are accessible through a private DNS zone:

- **api.<hosted_cluster_name>.hypershift.local**
- ***.apps.<hosted_cluster_name>.hypershift.local**

Additional resources

- [Enabling AWS PrivateLink for hosted control planes](#)
- [Creating an AWS IAM role and STS credentials](#)
- [Identity and Access Management \(IAM\) permissions](#)
- [Tutorial: Configuring private network access using a Linux Bastion Host](#)

4.1.6.1. Accessing a private hosted cluster on AWS

After you create a private hosted cluster, you can access it by using the command-line interface (CLI).

Procedure

1. Find the private IPs of nodes by entering the following command:

```
$ aws ec2 describe-instances \
  --filter="Name=tag:kubernetes.io/cluster/<infra_id>,Values=owned" \
  | jq '.Reservations[] | .Instances[] | select(.PublicDnsName=="") \
  | .PrivateIpAddress'
```

2. Create a **kubeconfig** file for the hosted cluster that you can copy to a node by entering the following command:

```
$ hcp create kubeconfig > <hosted_cluster_kubeconfig>
```

3. To SSH into one of the nodes through the bastion, enter the following command:

```
$ ssh -o ProxyCommand="ssh ec2-user@<bastion_ip> \  
-W %h:%p" core@<node_ip>
```

4. From the SSH shell, copy the **kubeconfig** file contents to a file on the node by entering the following command:

```
$ mv <path_to_kubeconfig_file> <new_file_name>
```

5. Export the **kubeconfig** file by entering the following command:

```
$ export KUBECONFIG=<path_to_kubeconfig_file>
```

6. Observe the hosted cluster status by entering the following command:

```
$ oc get clusteroperators clusterversion
```

4.2. DEPLOYING HOSTED CONTROL PLANES ON AZURE

With hosted control planes on Microsoft Azure, you can reduce the cost associated with dedicated control-plane node VMs for each cluster. You can provision compute nodes as Azure Virtual Machine Scale sets for dynamic scaling, and ensure credential isolation with per-cluster Azure service principals.



IMPORTANT

Hosted control planes on Azure is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

4.2.1. Overview of hosted control planes on Azure

You can deploy and manage hosted clusters on an OpenShift management cluster that runs in Microsoft Azure. With a self-managed deployment model, you manage your own OpenShift cluster.

4.2.1.1. Architecture of hosted control planes on Azure

At a high level, the architecture of hosted control planes on Azure consists of three layers:

Management cluster

An Azure OpenShift cluster that hosts the HyperShift Operator and the control planes for your hosted clusters.

Control plane

Kubernetes control-plane components that run as pods on the management cluster.

Data plane

Compute nodes that run as Azure virtual machines (VMs) in your Azure subscription.

The architecture uses Azure Workload Identity for secure, credential-free authentication between OpenShift Container Platform components and Azure services. As a result, you do not need to manage long-lived service principal credentials, and you get better security through federated identity credentials.

4.2.1.2. Deployment workflow for hosted control planes on Azure

Deploying self-managed hosted control planes on Azure involves a three-phase process in the following order:

Phase 1: Set up Azure Workload Identity

This phase establishes secure authentication infrastructure so that OpenShift Container Platform components can access Azure services. During this phase, the security infrastructure that your hosted clusters require is created:

- Managed identities for each OpenShift Container Platform component, including the image registry, ingress, CSI drivers, cloud provider, network operator, and so on
- OIDC issuer in Azure Blob Storage for service account token validation
- Federated credentials that establish trust relationships between Azure Entra ID and OpenShift Container Platform service accounts

Phase 2: Set up the management cluster

In this phase, you prepare your OpenShift Container Platform management cluster on Azure to host and manage hosted clusters. During this phase, you install the following components:

- Azure DNS zones, for private and public-private topologies
- External DNS, for private and public-private topologies
- HyperShift Operator

Phase 3: Create hosted clusters

In this phase, you create and configure hosted clusters. This phase involves the following steps:

- a. Setting up infrastructure
- b. Creating a hosted cluster
- c. Integrating workload identity
- d. Configuring private endpoint access (optional)

Additional resources

- [Azure Workload Identity documentation](#)

4.2.1.3. Azure infrastructure and credentials

Before you get started with hosted control planes on Microsoft Azure, get familiar with the required Azure resources and the necessary permissions.

4.2.1.3.1. Summary of requirements

You need the following resources, tools, access, and permissions to set up hosted control planes on Azure:

Azure resources

- An OpenShift Container Platform management cluster on Azure.
- An Azure subscription with contributor and user-access administrator permissions.
- (Optional) A parent DNS zone in Azure for delegating cluster DNS records. This DNS zone is required only if you plan to use external DNS.

Tools and access

- Azure command-line interface (CLI), **az**, configured with your subscription
- OpenShift CLI (**oc**)
- hosted control planes CLI, **hcp**
- **jq** command-line JSON processor
- Cloud Credential Operator utility (**ccoctl**)
- A valid OpenShift Container Platform pull secret

Permissions

- Subscription-level contributor and user access administrator roles
- Microsoft Graph API permissions for creating service principals

4.2.1.3.2. DNS management options

You can set up hosted control planes on Azure with or without external DNS. If you plan to use a private or a public-private topology, external DNS is required. See the following table for more information:

Table 4.8. DNS approaches for hosted control planes on Azure

	With external DNS	Without external DNS
Best for	Production, multi-cluster	Development, testing
API server DNS	Custom (api-cluster.example.com)	Azure Load Balancer (abc123.region.cloudapp.azure.com)

	With external DNS	Without external DNS
Setup complexity	Low, requires DNS zones and service principals	None
Management	Fully automatic	Manual or Azure-provided

4.2.1.3.3. Resource group strategy

Cluster-specific resource groups are created and deleted with each hosted cluster. These resources include managed resource groups for cluster infrastructure. If you use custom networking, these resources also include Virtual Network (VNet) resource groups and Network Security Group (NSG) resource groups.

4.2.1.3.4. Security considerations for hosted control planes on Azure

Hosted control planes on Azure employs the following security best practices:

- Workload identity federation, which eliminates long-lived credentials by using OIDC-based authentication.
- Least-privilege access, where each component has its own managed identity with minimal required permissions.
- Network isolation, where you can use custom VNets and NSGs to implement network segmentation and security policies.
- Federated credentials, where trust relationships are scoped to specific service accounts, preventing unauthorized access.
- Private connectivity, available as an option through Azure Private Link, which provides private API server access to ensure that control-plane traffic never traverses the public internet. Private connectivity is available for private and public-private topologies only.

4.2.2. Setting up Azure resources

Before you can create management and hosted clusters for your deployment of hosted control planes on Azure, you need to set up an OIDC issuer, Workload Identities, and your Azure infrastructure.

4.2.2.1. Setting up an OIDC issuer

To prepare to deploy hosted control planes on Azure, you need to set up an OIDC issuer for hosted clusters.

Prerequisites

- The Azure command-line interface (CLI) is installed and configured.
- The **jq** command-line JSON processor is installed.
- The Cloud Credential Operator utility (**ccctl**) is installed. For more information, see "How to obtain the ccctl tool for OpenShift 4".

- The appropriate Azure permissions are set.

Procedure

1. Set your environment variables as shown in the following example:

```
PERSISTENT_RG_NAME="os4-common"
LOCATION="eastus"
AZURE_CREDS="/path/to/azure-creds.json"
SUBSCRIPTION_ID="my-subscription-id"
```

2. Create a persistent resource group by entering the following command:

```
$ az group create --name $PERSISTENT_RG_NAME --location $LOCATION
```

3. Configure an OIDC issuer URL by using the Cloud Credential Operator tool to complete the following steps:

- a. Set the OIDC issuer variables as shown in the following example:

```
OIDC_STORAGE_ACCOUNT_NAME="yourstorageaccount"
TENANT_ID="your-tenant-id"
```

- b. Create an RSA key pair and save the private and public key by entering the following command:

```
$ ccoctl azure create-key-pair
```

- c. Set variables for the token issuer key paths as shown in the following example:

```
SA_TOKEN_ISSUER_PRIVATE_KEY_PATH="/path/to/serviceaccount-signer.private"
SA_TOKEN_ISSUER_PUBLIC_KEY_PATH="/path/to/serviceaccount-signer.public"
```

- d. Create an OIDC issuer by entering the following command:

```
$ ccoctl azure create-oidc-issuer \
  --oidc-resource-group-name ${PERSISTENT_RG_NAME} \
  --tenant-id ${TENANT_ID} \
  --region ${LOCATION} \
  --name ${OIDC_STORAGE_ACCOUNT_NAME} \
  --subscription-id ${SUBSCRIPTION_ID} \
  --public-key-file ${SA_TOKEN_ISSUER_PUBLIC_KEY_PATH}
```

- e. Set the OIDC issuer URL as shown in the following example:

```
OIDC_ISSUER_URL="https://${OIDC_STORAGE_ACCOUNT_NAME}.blob.core.window
s.net/${OIDC_STORAGE_ACCOUNT_NAME}"
```

Verification

- Try to access the OIDC issuer by entering the following command:

```
$ curl -s "${OIDC_ISSUER_URL}/.well-known/openid-configuration" | jq .
```

-

Additional resources

- [How to obtain the ccoctl tool for OpenShift 4 \(Red Hat Knowledgebase article\)](#)

4.2.2.2. Creating Azure Workload Identities

To ensure control over Identity and Access Management (IAM) resources in your Azure deployment, create Workload Identities separately from your infrastructure.

Workload Identities authenticate hosted cluster components to Azure services by using OIDC federation. You must create identities separately and then consume them during infrastructure or cluster creation.

Prerequisites

- You have an Azure credentials file in the following format:

Example file

```
{
  "subscriptionId": "your-subscription-id",
  "tenantId": "your-tenant-id",
  "clientId": "your-client-id",
  "clientSecret": "your-client-secret"
}
```

- You have a resource group to create the managed identities in.
- You have an OIDC issuer URL for Workload Identity federation. For more information, see "Setting up an OIDC issuer".

Procedure

1. Set environment variables as shown in the following example:

```
CLUSTER_NAME="my-self-managed-cluster"
INFRA_ID="${CLUSTER_NAME}-${openssl rand -hex 4}"
```

2. On the hosted control planes command-line interface, **hcp**, enter the following command:

```
$ hcp create iam azure \
  --name <my_cluster_name> \
  --infra-id <infra_id> \
  --azure-creds <azure_credentials_file> \
  --resource-group-name <resource_group> \
  --oidc-issuer-url <oidc_issuer_url> \
  --output-file <workload_identities_file> \
  --location <my_region> \
  --cloud <my_cloud_environment>
```

where:

<my_cluster_name>

Specifies the name of the cluster you intend to create.

<infra_id>

Specifies the unique identifier for naming Azure resources. Typically, this identifier is the cluster name with a suffix.

<azure_credentials_file>

Specifies the Azure credentials file with permission to create managed identities and federated credentials.

<resource_group>

Specifies the name of the resource group where you intend to create identities.

<oidc_issuer_url>

Specifies the URL of the OIDC identity provider for Workload Identity federation.

<workload_identities_file>

Specifies the output file path, such as **my-cluster-name-iam-output.json**.

You can also add these optional flags to the **hcp create iam azure** command:

<my_region>

Specifies the Azure region for the managed identities. The default value is **eastus**.

<my_cloud_environment>

Specifies the Azure cloud environment. The default value is **AzurePublicCloud**.

Verification

- Review the output file, which looks like the following example:

Example output

```
{
  "disk": {
    "tenantID": "...",
    "clientID": "...",
    "resourceID":
"/subscriptions/.../providers/Microsoft.ManagedIdentity/userAssignedIdentities/my-cluster-
abc123-disk"
  },
  "file": {
    "tenantID": "...",
    "clientID": "...",
    "resourceID": "..."
  },
  "imageRegistry": { ... },
  "ingress": { ... },
  "cloudProvider": { ... },
  "nodePoolManagement": { ... },
  "network": { ... },
  "controlPlaneOperator": { ... }
}
```

The output includes 8 user-assigned identities, one per cluster component, along with federated credentials for each identity:

- Disk CSI driver
- File CSI driver
- Image registry
- Ingress Operator
- Cloud provider
- Node pool management
- Network Operator
- Control Plane Operator

4.2.2.3. Creating Azure infrastructure

Create an Azure infrastructure separately so that when you create a hosted cluster on Azure, you can use pre-existing infrastructure.

Prerequisites

- You have an Azure credentials file with the following format:

```
{
  "subscriptionId": "<my_subscription_id>",
  "tenantId": "<my_tenant_id>",
  "clientId": "<my_client_id>",
  "clientSecret": "<my_client_secret>"
}
```

- You have an existing public DNS zone in your Azure subscription for your base domain.
- You created Workload Identities. For more information, see "Creating Azure Workload Identities".

Procedure

- To create the infrastructure with a new virtual network, subnet, and network security group, enter the following command:

```
$ hcp create infra azure \
  --name <my_cluster_name> \
  --infra-id <infra_id> \
  --azure-creds <azure_credentials_file> \
  --base-domain <base_domain> \
  --location <location> \
  --workload-identities-file <workload_identities_file> \
  --assign-identity-roles \
  --dns-zone-rg-name <dns_zone_rg> \
  --output-file <output_infra_file>
```

where:

- **--name** specifies the name of the hosted cluster you intend to create.
- **--infra-id** specifies a unique name that identifies your infrastructure. This value is used to name and tag Azure resources. Typically, it is the name of your cluster with a suffix appended to it.
- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers.
- **--base-domain** specifies the base domain for the ingress of your hosted cluster. The base domain must correspond to an existing public DNS zone in your Azure subscription.
- **--location** specifies the Azure region where you want to create the infrastructure, such as **eastus** or **westus2**.
- **--workload-identities-file** specifies the path to the JSON file that contains the Workload Identity configuration.
- **--assign-identity-roles** specifies that automatic RBAC role assignment is enabled for Workload Identities.
- **--dns-zone-rg-name** specifies the name of the resource group that contains your public DNS zone.
- **--output-file** specifies the file where the details of the infrastructure are stored in YAML format.
- To create the infrastructure with an existing virtual network, subnet, and network security group, enter the following command:

```
$ hcp create infra azure \
  --name <my_cluster_name> \
  --infra-id <infra_id> \
  --azure-creds <azure_credentials_file> \
  --base-domain <base_domain> \
  --location <location> \
  --workload-identities-file <workload_identities_file> \
  --vnet-id
/subscriptions/<subscription_id>/resourceGroups/<resource_group>/providers/Microsoft.Network/virtualNetworks/<vnet_name> \
  --subnet-id
/subscriptions/<subscription_id>/resourceGroups/<resource_group>/providers/Microsoft.Network/virtualNetworks/<vnet_name>/subnets/<subnet_name> \
  --network-security-group-id
/subscriptions/<subscription_id>/resourceGroups/<resource_group>/providers/Microsoft.Network/networkSecurityGroups/<network_security_group_name> \
  --assign-identity-roles \
  --dns-zone-rg-name <dns_zone_rg> \
  --output-file <output_infra_file>
```

where:

- **--name** specifies the name of the hosted cluster you intend to create.

- **--infra-id** specifies a unique name that identifies your infrastructure. This value is used to name and tag Azure resources. Typically, it is the name of your cluster with a suffix appended to it.
- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers.
- **--base-domain** specifies the base domain for the ingress of your hosted cluster. The base domain must correspond to an existing public DNS zone in your Azure subscription.
- **--location** specifies the Azure region where you want to create the infrastructure, such as **eastus** or **westus2**.
- **--workload-identities-file** specifies the path to the JSON file that contains the Workload Identity configuration.
- **--vnet-id** specifies your existing virtual network ID, which includes your subscription ID and your virtual network name.
- **--subnet-id** specifies the ARM resource ID of your subnet, which includes your subscription ID, virtual network name, and subnet name.
- **--network-security-group-id** specifies your existing network security group ID, which includes your subscription ID and your network security group name.
- **--assign-identity-roles** specifies that automatic RBAC role assignment is enabled for Workload Identities.
- **--dns-zone-rg-name** specifies the name of the resource group that contains your public DNS zone.
- **--output-file** specifies the file where the details of the infrastructure are stored in YAML format.

Additional resources

- [Creating Azure Workload Identities](#)

4.2.3. Configuring an Azure management cluster for hosted control planes

To configure the management cluster for hosted control planes on Azure, you need to ensure that external DNS and the HyperShift Operator are set up on the cluster.

The configuration steps include configuring the DNS zone, delegating the DNS records for your cluster, and creating a dedicated service principal for external DNS.

Prerequisites

- You installed the multicluster engine Operator 2.5 or later on an OpenShift Container Platform cluster. You can install multicluster engine Operator as an Operator from the OpenShift Container Platform software catalog. Or, if you already use Red Hat Advanced Cluster Management, the Operator is automatically installed. The HyperShift Operator is enabled by default in the operator package for multicluster engine Operator.
- The Azure command-line interface (CLI) is installed and configured.

- The OpenShift CLI (**oc**) is installed.
- You have an OpenShift Container Platform management cluster on Azure.
- If you are using external DNS, the **jq** command-line JSON processor is installed.

Procedure

1. Set the DNS configuration variables as shown in the following example:

```
PARENT_DNS_RG="<my_parent_dns_resource_group>"
PARENT_DNS_ZONE="<my_parent.dns.zone.com>"
DNS_RECORD_NAME="<my_subdomain>"
RESOURCE_GROUP_NAME="<my_resource_group>"
DNS_ZONE_NAME="<my_subdomain.my_parent.dns.zone.com>"
LOCATION="<my_region>"
```

2. Create the Azure group by entering the following command:

```
$ az group create \
  --name $RESOURCE_GROUP_NAME \
  --location $LOCATION
```

3. Create the Azure DNS zone by entering the following command:

```
$ az network dns zone create \
  --resource-group $RESOURCE_GROUP_NAME \
  --name $DNS_ZONE_NAME
```

4. If an existing name server record exists, delete it by entering the following command:

```
$ az network dns record-set ns delete \
  --resource-group $PARENT_DNS_RG \
  --zone-name $PARENT_DNS_ZONE \
  --name $DNS_RECORD_NAME -y
```

5. Get the name servers from your DNS zone by entering the following command:

```
name_servers=$(az network dns zone show \
  --resource-group $RESOURCE_GROUP_NAME \
  --name $DNS_ZONE_NAME \
  --query nameServers \
  --output tsv)
```

6. Create an array of name servers as shown in the following example:

```
ns_array=()
while IFS= read -r ns; do
  ns_array+=("$ns")
done <<< "$name_servers"
```

7. Add name server records to the parent zone as shown in the following example:

```
for ns in "${ns_array[@]"; do
  az network dns record-set ns add-record \
    --resource-group $PARENT_DNS_RG \
    --zone-name $PARENT_DNS_ZONE \
    --record-set-name $DNS_RECORD_NAME \
    --nsdname "$ns"
done
```

8. Set the external DNS configuration variables as shown in the following example:

```
EXTERNAL_DNS_NEW_SP_NAME=<external_dns_service_principal>
SERVICE_PRINCIPAL_FILEPATH=<path_to_azure_mgmt_json_file>
RESOURCE_GROUP_NAME=<my_resource_group>
```

9. Create the service principal for external DNS by entering the following command:

```
DNS_SP=$(az ad sp create-for-rbac --name ${EXTERNAL_DNS_NEW_SP_NAME})
EXTERNAL_DNS_SP_APP_ID=$(echo "$DNS_SP" | jq -r '.appId')
EXTERNAL_DNS_SP_PASSWORD=$(echo "$DNS_SP" | jq -r '.password')
```

10. Get the DNS zone ID by entering the following command:

```
DNS_ID=$(az network dns zone show \
  --name ${DNS_ZONE_NAME} \
  --resource-group ${RESOURCE_GROUP_NAME} \
  --query "id" \
  --output tsv)
```

11. Assign the **Reader** role to the service principal by entering the following command:

```
$ az role assignment create \
  --role "Reader" \
  --assignee "${EXTERNAL_DNS_SP_APP_ID}" \
  --scope "${DNS_ID}"
```

12. Assign the **Contributor** role to the service principal by entering the following command:

```
$ az role assignment create \
  --role "Contributor" \
  --assignee "${EXTERNAL_DNS_SP_APP_ID}" \
  --scope "${DNS_ID}"
```

13. Save the Azure credentials to a local file by entering the following command:

```
$ cat > ${SERVICE_PRINCIPAL_FILEPATH} <<EOF
{
  "tenantId": "$(az account show --query tenantId -o tsv)",
  "subscriptionId": "$(az account show --query id -o tsv)",
  "resourceGroup": "$RESOURCE_GROUP_NAME",
  "aadClientId": "$EXTERNAL_DNS_SP_APP_ID",
  "aadClientSecret": "$EXTERNAL_DNS_SP_PASSWORD"
}
EOF
```


14. If an existing Kubernetes secret for the Azure credentials exists, delete it by entering the following command:

```
$ oc delete secret/azure-config-file --namespace "hypershift" || true
```

15. Create the Kubernetes secret for the Azure credentials by entering the following command:

```
$ oc create secret generic azure-config-file \
  --namespace "hypershift" \
  --from-file=credentials=${SERVICE_PRINCIPAL_FILEPATH}
```

The secret must be created in the **hypershift** namespace where the external DNS deployment runs. The **credentials** key name is required because the external DNS pod mounts the secret at **/etc/provider/credentials**.

16. Configure the HyperShift Operator to use external DNS by creating the following **ConfigMap** object:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: hypershift-operator-install-flags
  namespace: local-cluster
data:
  installFlagsToAdd: "--external-dns-provider=azure --external-dns-secret=azure-config-file --
external-dns-domain-filter=<dns_zone>"
  installFlagsToRemove: ""
```

The **--external-dns-secret** flag specifies the name of the Kubernetes secret that contains the Azure credentials. The **data.installFlagsToAdd** parameter specifies the flags to pass to the Operator so it detects the DNS.

17. Apply the config map by entering the following command:

```
$ oc apply -f hypershift-operator-install-flags.yaml
```

Verification

- Verify that both the HyperShift Operator and the external DNS are running by entering the following command:

```
$ oc get pods -n hypershift
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
external-dns-xxxxx-xxxxx	1/1	Running	0	1m
operator-xxxxx-xxxxx	1/1	Running	0	1m

4.2.4. Creating a hosted cluster on Azure

Hosted clusters are where your applications run. Each hosted cluster has its own control plane that runs on the management cluster and a set of compute node virtual machines (VMs) in Azure.

The hosted cluster uses Workload Identities to securely access Azure services without storing credentials.

Prerequisites

- You installed the Azure command-line interface (CLI).
- You installed the hosted control planes CLI, **hcp**.
- You installed the OpenShift CLI (**oc**).
- You installed the **jq** command-line JSON processor.
- You have a management cluster where the HyperShift Operator is installed and external DNS is configured.
- You set up Azure resources, including Workload Identities, an OIDC issuer, and infrastructure.
- You have the appropriate Azure permissions.
 - At the subscription level, you must have the **Contributor** role and the **User Access Administrator** role.
 - For Microsoft Graph API, you must have the **Application.ReadWrite.OwnedBy** permission.

Procedure

- Create the **HostedCluster** custom resource by entering the following command:

```
$ hcp create cluster azure \
  --name "$CLUSTER_NAME" \
  --infra-id "$INFRA_ID" \
  --azure-creds $AZURE_CREDS \
  --location ${LOCATION} \
  --node-pool-replicas 2 \
  --base-domain $BASE_DOMAIN \
  --pull-secret $PULL_SECRET \
  --generate-ssh \
  --release-image ${RELEASE_IMAGE} \
  --sa-token-issuer-private-key-path "${SA_TOKEN_ISSUER_PRIVATE_KEY_PATH}" \
  --oidc-issuer-url "${OIDC_ISSUER_URL}" \
  --dns-zone-rg-name ${PERSISTENT_RG_NAME} \
  --assign-service-principal-roles \
  --infra-json <output_infra_file> \
  --diagnostics-storage-account-type Managed \
  --external-dns-domain "${DNS_ZONE_NAME}"
```



NOTE

For non-production environments, you can create a hosted cluster without external DNS by omitting the **--external-dns-domain** and **--assign-service-principal-roles** flags. In that case, the API server is accessible through an Azure load balancer hostname instead of a custom DNS name.

Verification

1. Check the cluster status by entering the following command:

```
$ oc get hostedcluster $CLUSTER_NAME -n clusters
```

2. Wait for the cluster to be complete by entering the following command:

```
$ oc wait \
  --for=jsonpath='{.status.version.history[0].state}'=Completed \
  hostedcluster/$CLUSTER_NAME \
  -n clusters --timeout=20m
```

3. Create the **kubeconfig** file for the cluster by entering the following command:

```
$ hcp create kubeconfig \
  --name $CLUSTER_NAME > $CLUSTER_NAME-kubeconfig
```

4. Get the **kubeconfig** file by entering the following command:

```
$ export KUBECONFIG=$CLUSTER_NAME-kubeconfig
```

5. Access the cluster nodes by entering the following command:

```
$ oc get nodes
```

6. Access the cluster by entering the following command:

```
$ oc get clusterversion
```

4.2.5. Private hosted clusters on Azure

By default, hosted clusters are accessible through public DNS and the default router of the management cluster. If you want communication between your compute nodes and the hosted control plane to be private, you can create hosted clusters that use Azure Private Link for communication.

Private endpoint access uses Azure Private Link to expose the internal load balancer of the hosted control plane to the Azure Virtual Network (VNet) of the hosted cluster. Compute nodes resolve the API server hostname by using private DNS zones that point to the private endpoint IP address.

4.2.5.1. Preparing a subnet for a private hosted cluster on Azure

Azure Private Link requires a dedicated subnet for network address translator (NAT) IP address allocation. You can manually create the subnet, or it can be automatically created during cluster creation.

By manually creating the subnet, you have control over classless inter-domain routing (CIDR) allocation and naming. The following steps apply to manually creating the subnet. If you want the subnet to be automatically created, skip this procedure.



NOTE

Azure Private Link, the NAT subnet, and the internal load balancer of the management cluster must all be in the same Azure region. Private Link is automatically created in the location where the hosted cluster is created. Azure rejects the creation of Private Link if the NAT subnet is in a different region.

Prerequisites

- You have an OpenShift Container Platform management cluster on Azure. For more information, see "Configuring an Azure management cluster for hosted control planes".
- The Azure command-line interface (CLI) is installed and configured.
- The OpenShift CLI (**oc**) is installed.
- If you are using external DNS, the **jq** command-line JSON processor is installed.
- You configured an OIDC issuer. For more information, see "Setting up an OIDC issuer".

Procedure

1. Identify the Azure Virtual Network (VNet) of the management cluster.
 - a. Obtain the infrastructure resource group of the management cluster as shown in the following example:

```
$ MGMT_INFRA_RG=$(oc get infrastructure cluster -o
  jsonpath='{.status.platformStatus.azure.resourceGroupName}')
```

- b. Find the VNet in the infrastructure resource group as shown in the following example:

```
$ MGMT_VNET_NAME=$(az network vnet list --resource-group "${MGMT_INFRA_RG}"
  --query "[0].name" -o tsv)
```

- c. Set the environment variable for the VNet by entering the following command:

```
$ MGMT_VNET_RG="${MGMT_INFRA_RG}"
```

2. Create the NAT subnet.
 - a. Check the existing address space and subnets to ensure that you do not choose an overlapping classless inter-domain routing (CIDR) range. Enter the following command:

```
$ az network vnet show \
  --resource-group "${MGMT_VNET_RG}" \
  --name "${MGMT_VNET_NAME}" \
  --query '{addressSpace: addressSpace.addressPrefixes, subnets: subnets[].{name:
  name, prefix: addressPrefix}}' \
  -o json
```

- b. Create the subnet as shown in the following example:

```
$ az network vnet subnet create \
  --resource-group "${MGMT_VNET_RG}" \
```

```
--vnet-name "${MGMT_VNET_NAME}" \
--name "${NAT_SUBNET_NAME}" \
--address-prefixes 10.1.64.0/24 \
--disable-private-link-service-network-policies true
```

- The NAT subnet must be in the VNet of the management cluster because Private Link is created alongside the internal load balancer of the management cluster.
 - The **10.1.64.0/24** address prefix is an example only. Replace it with a CIDR range that does not overlap with any other subnet in the VNet of the management cluster. If the VNet uses **10.0.0.0/16**, the NAT subnet must fall within that range, or you must expand the address space of the VNet.
 - The **--disable-private-link-service-network-policies** flag is required and must be set to **true**. Otherwise, Azure rejects the creation of Private Link on the subnet.
3. Obtain the NAT subnet resource ID for later use as shown in the following example:

```
$ NAT_SUBNET_ID=$(az network vnet subnet show \
--resource-group "${MGMT_VNET_RG}" \
--vnet-name "${MGMT_VNET_NAME}" \
--name "${NAT_SUBNET_NAME}" \
--query id -o tsv)
```

Additional resources

- [Configuring an Azure management cluster for hosted control planes](#)
- [Setting up an OIDC issuer](#)

4.2.5.2. Installing the HyperShift Operator with private platform support

To set up an environment that supports private clusters, you must install the HyperShift Operator with flags that configure Azure Private Link management.



NOTE

If you already installed the HyperShift Operator but you did not include the **--private-platform Azure** setting, you must run the **hcp install** command again with the private platform flags before you can create private clusters.

Prerequisites

- You have an OpenShift Container Platform management cluster on Azure.
- The Azure command-line interface (CLI) is installed and configured.
- The OpenShift CLI (**oc**) is installed.
- If you are using external DNS, the **jq** command-line JSON processor is installed.
- You configured an OIDC issuer. For more information, see "Setting up an OIDC issuer".

Procedure

1. Obtain credentials so that the Operator can manage Private Link resources.

- a. Obtain the credentials file for Private Link management as shown in the following example:

```
$ AZURE_PRIVATE_CREDS="<path_to_azure_private_credentials_json>"
```

- b. Obtain the infrastructure resource group of the management cluster as shown in the following example:

```
$ MGMT_INFRA_RG=$(oc get infrastructure cluster -o  
jsonpath='{.status.platformStatus.azure.resourceGroupName}')
```

2. Set the external DNS configuration variables as shown in the following examples:

```
$ SERVICE_PRINCIPAL_FILEPATH="<path_to_azure_mgmt_json_file>"
```

```
$ DNS_ZONE_NAME="<my_subdomain_my_parent_dns_zone_com>"
```

3. Install the HyperShift Operator with private platform support as shown in the following example:

```
$ hcp install \  
  --pull-secret ${PULL_SECRET} \  
  --private-platform Azure \  
  --azure-private-creds ${AZURE_PRIVATE_CREDS} \  
  --azure-pls-resource-group ${MGMT_INFRA_RG} \  
  --external-dns-provider=azure \  
  --external-dns-credentials ${SERVICE_PRINCIPAL_FILEPATH} \  
  --external-dns-domain-filter ${DNS_ZONE_NAME}
```

- **--private-platform Azure** specifies that Azure Private Link management is to be enabled in the Operator.
- **--azure-private-creds** specifies the path to the Azure credentials file that is used for Private Link operations.



NOTE

Besides using the **--azure-private-creds** flag, you can use one of the following authentication methods. Be sure to use only one authentication method.

- **--azure-private-secret** specifies an existing Kubernetes secret that has Azure credentials. This flag has a companion flag, **--azure-private-secret-key**. Its default value is **credentials**, but you can customize it for secrets that have non-standard key names.
- **--azure-pls-managed-identity-client-id** specifies the client ID of a managed identity for Private Link operations through Workload Identity federation. If you specify this flag, you must also include the **--azure-pls-subscription-id** flag, which specifies the Azure subscription ID for Private Link operations.

- **--azure-pls-resource-group** specifies the resource group where the Private Link resources

- **--azure-pis-resource-group** specifies the resource group where the Private Link resources are to be created. This resource group is the same as the resource group of the infrastructure for the management cluster.
- **--external-dns-credentials** specifies the path to a file that contains DNS credentials. If preferred, you can use the **--external-dns-secret** flag instead to specify a Kubernetes secret that has DNS credentials.

Additional resources

- [Setting up an OIDC issuer](#)

4.2.5.3. Configuring IAM resources for a private hosted cluster

Create Workload Identities so that your private clusters can manage private endpoints and private DNS zones.

Prerequisites

- You have an OpenShift Container Platform management cluster on Azure that has the HyperShift Operator installed.
- The Azure command-line interface (CLI) is installed and configured.
- The OpenShift CLI (**oc**) is installed.
- If you are using external DNS, the **jq** command-line JSON processor is installed.
- You configured an OIDC issuer. For more information, see "Setting up an OIDC issuer".

Procedure

1. Set your environment variables:

- a. Set your prefix by entering the following command:

```
$ PREFIX="<my_prefix>"
```

- b. Set the cluster name by entering the following command:

```
$ CLUSTER_NAME="${PREFIX}-hc"
```

- c. Set the resource group name by entering the following command:

```
$ RESOURCE_GROUP_NAME="${CLUSTER_NAME}-${PREFIX}"
```

- d. Set the location by entering the following command:

```
$ LOCATION="<my_region>"
```

- e. Set the variable for the path to the Azure credentials file by entering the following command:

```
$ AZURE_CREDS="<path_to_azure_credentials_json>"
```

- f. Set the variable for the OIDC issuer URL by entering the following command:

```
$ OIDC_ISSUER_URL="<my_oidc_url_com>"
```

- g. Set the path to the Workload Identities file by entering the following command:

```
$ WORKLOAD_IDENTITIES_FILE="<path_to_workload_identities_file_json>"
```

2. Create Workload Identities by entering the following command:

```
$ hcp create iam azure \  
  --name "${CLUSTER_NAME}" \  
  --infra-id "${PREFIX}" \  
  --azure-creds "${AZURE_CREDS}" \  
  --location "${LOCATION}" \  
  --resource-group-name "${RESOURCE_GROUP_NAME}" \  
  --oidc-issuer-url "${OIDC_ISSUER_URL}" \  
  --output-file "${WORKLOAD_IDENTITIES_FILE}"
```

The command creates 8 Workload Identities. For **Private** and **PublicAndPrivate** clusters, the Control Plane Operator identity is used to create and manage private endpoints, private DNS zones, Azure Virtual Network (VNet) links, and DNS A records. The Control Plane Identity is assigned the **Contributor** role by default. To use a more restrictive role, use the **--assign-custom-hcp-roles** flag.

Additional resources

- [Setting up an OIDC issuer](#)
- [Creating Azure Workload Identities](#)

4.2.5.4. Creating infrastructure for a private hosted cluster

Set up infrastructure so that you can create private hosted clusters.

Prerequisites

- You have an OpenShift Container Platform management cluster on Azure that has the HyperShift Operator installed.
- The Azure command-line interface (CLI) is installed and configured.
- The OpenShift CLI (**oc**) is installed.
- If you are using external DNS, the **yq** command-line YAML processor is installed.
- You configured an OIDC issuer. For more information, see "Setting up an OIDC issuer".

Procedure

1. Set your environment variables:
 - a. Set the variable for the DNS zone resource group by entering the following command:


```
$ DNS_ZONE_RG_NAME="os4-common"
```

- b. Set the variable for the base domain of your DNS zone by entering the following command:

```
$ PARENT_DNS_ZONE="<your_base_domain_com>"
```

- c. Set the variable that points to the infrastructure output file by entering the following command:

```
$ INFRA_OUTPUT_FILE="${PREFIX}-infra-output.json"
```

2. Create infrastructure by entering the following command:

```
$ hcp create infra azure \
  --azure-creds "${AZURE_CREDS}" \
  --infra-id "${PREFIX}" \
  --name "${CLUSTER_NAME}" \
  --location "${LOCATION}" \
  --base-domain "${PARENT_DNS_ZONE}" \
  --dns-zone-rg-name "${DNS_ZONE_RG_NAME}" \
  --workload-identities-file "${WORKLOAD_IDENTITIES_FILE}" \
  --assign-identity-roles \
  --output-file "${INFRA_OUTPUT_FILE}"
```

3. Read the infrastructure output to get the resource IDs that you created, as shown in the following example:

- a. Read the resource group name by entering the following command:

```
$ MANAGED_RG_NAME=$(yq -r -p yaml '.resourceGroupName'
"${INFRA_OUTPUT_FILE}")
```

- b. Read the VNet ID by entering the following command:

```
$ VNET_ID=$(yq -r -p yaml '.vnetID' "${INFRA_OUTPUT_FILE}")
```

- c. Read the subnet ID by entering the following command:

```
$ SUBNET_ID=$(yq -r -p yaml '.subnetID' "${INFRA_OUTPUT_FILE}")
```

- d. Read the network security group ID by entering the following command:

```
$ NSG_ID=$(yq -r -p yaml '.securityGroupID' "${INFRA_OUTPUT_FILE}")
```

Note the resource IDs because you need them to create the private hosted cluster.

Additional resources

- [Setting up an OIDC issuer](#)

4.2.5.5. Creating a private Azure hosted cluster

Create a private hosted cluster to ensure that the communication between compute nodes and the hosted control plane occurs over Azure Private Link.

Prerequisites

- You configured a subnet, as described in "Preparing a subnet for a private hosted cluster on Azure".
- You installed the HyperShift Operator on an OpenShift Container Platform management cluster on Azure, as described in "Installing the HyperShift Operator with private platform support".
- You created Identity and Access Management (IAM) resources, as described in "Configuring IAM resources for a private hosted cluster".
- You created infrastructure, as described in "Creating infrastructure for a private hosted cluster".
- This procedure relies on environment variables that you created in the prerequisite procedures. Be sure to complete this procedure in the same terminal where you already defined the environment variables.

Procedure

1. Create the private hosted cluster by entering the following command:

```
$ hcp create cluster azure \
  --name "$CLUSTER_NAME" \
  --namespace "clusters" \
  --azure-creds "${AZURE_CREDS}" \
  --location "${LOCATION}" \
  --node-pool-replicas 2 \
  --base-domain "${PARENT_DNS_ZONE}" \
  --pull-secret "${PULL_SECRET}" \
  --generate-ssh \
  --release-image "${RELEASE_IMAGE}" \
  --resource-group-name "${MANAGED_RG_NAME}" \
  --vnet-id "${VNET_ID}" \
  --subnet-id "${SUBNET_ID}" \
  --network-security-group-id "${NSG_ID}" \
  --sa-token-issuer-private-key-path "${SA_TOKEN_ISSUER_PRIVATE_KEY_PATH}" \
  --oidc-issuer-url "${OIDC_ISSUER_URL}" \
  --dns-zone-rg-name "${DNS_ZONE_RG_NAME}" \
  --assign-service-principal-roles \
  --workload-identities-file "${WORKLOAD_IDENTITIES_FILE}" \
  --external-dns-domain "${DNS_ZONE_NAME}" \
  --endpoint-access Private \
  --endpoint-access-private-nat-subnet-id "${NAT_SUBNET_ID}"
```

- When you choose a value for the **--external-dns-domain** flag, ensure that the value does not match **{cluster-name}.{base-domain}**. If you use **{cluster-name}.{base-domain}** for the value, the Control Plane Operator creates a private DNS zone that can shadow the ***.apps** domain, which causes the console and ingress to become unreachable.
- **--endpoint-access** accepts 3 values:
 - **Public**, which is the default value. When you specify **Public**, the API server is accessible

through public endpoint only.

- **PublicAndPrivate**, where the API server is accessible through both public and private endpoints.
- **Private**, where the API server is accessible only through Private Link.
After you create a cluster, you cannot change it between **Public** endpoint and non-public (**PublicAndPrivate** or **Private**) endpoint access.

If you need to allow **Private** endpoint connections from Azure subscriptions other than the hosted cluster's subscription, use the following flag: **--endpoint-access-private-additional-allowed-subscriptions "sub-id-1, sub-id-2"**.

Verification

1. Check the Private Link resources by entering the following command:

```
$ oc get azureprivatelinkservices -n clusters-${CLUSTER_NAME}
```

2. Check the status and connections by entering the following command:

```
$ oc get azureprivatelinkservices -n clusters-${CLUSTER_NAME} -o yaml
```

Table 4.9. Setup progress conditions

Condition	Description
AzureInternalLoadBalancerAvailable	The internal load balancer has a front-end IP address.
AzurePLSCreated	Private Link is created in the management cluster.
AzurePrivateEndpointAvailable	Private endpoint is created in the hosted cluster VNet.
AzurePrivateDNSAvailable	Private DNS zones and A records are created.
AzurePrivateLinkServiceAvailable	All components are ready and private connectivity is available.

3. Check the overall cluster status by entering the following command:

```
$ oc get hostedcluster ${CLUSTER_NAME} -n clusters
```

4. Wait for the status by entering the following command:

```
$ oc wait --for=condition=Available hostedcluster/${CLUSTER_NAME} -n clusters --timeout=30m
```

Additional resources

- [Preparing a subnet for a private hosted cluster on Azure](#)
- [Installing the HyperShift Operator with private platform support](#)
- [Configuring IAM resources for a private hosted cluster](#)
- [Creating infrastructure for a private hosted cluster](#)

4.2.5.6. Accessing a private Azure hosted cluster

After you create a private hosted cluster, you need to take additional steps to access it.

Prerequisites

- You completed the steps in "Creating a private Azure hosted cluster".

Procedure

- To access a private hosted cluster by generating a **kubeconfig** file, enter the following command:

```
$ hcp create kubeconfig \  
  --name ${CLUSTER_NAME} \  
  --port-forward > ${CLUSTER_NAME}-kubeconfig
```

- If you have access to the management cluster, you can port forward to the API server to access the private hosted cluster.

- a. Port forward to the **kube-apiserver** service by entering the following command:

```
$ kubectl port-forward svc/kube-apiserver \  
  -n clusters-${CLUSTER_NAME} 6443:6443 &
```

- b. Access the hosted cluster by using the **kubeconfig** file:

```
$ KUBECONFIG=${CLUSTER_NAME}-kubeconfig oc get nodes
```

- If you have a virtual machine (VM) in an Azure Virtual Network (VNet) that is peered with the hosted cluster's VNet, you can access the API server, but you must first link the private DNS zones to the peered VNet.



NOTE

The Control Plane Operator links private DNS zones only to the hosted cluster's VNet. If you want to resolve the API server hostname from a peered VNet, you must manually link the private DNS zones to that VNet as shown in the following steps. Otherwise, DNS resolution fails from the peered VNet.

- a. Link the private DNS zone to your peered VNet as shown in the following example:

```
$
PEERED_VNET_ID="/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.Network/virtualNetworks/<vnet>"
```

- b. Enter the following command:

```
$ az network private-dns link vnet create \
  --resource-group "${MANAGED_RG_NAME}" \
  --zone-name "${CLUSTER_NAME}.hypershift.local" \
  --name "peered-vnet-link" \
  --virtual-network "${PEERED_VNET_ID}" \
  --registration-enabled false
```

- c. If you also need base-domain resolution, enter the following command:

```
$ az network private-dns link vnet create \
  --resource-group "${MANAGED_RG_NAME}" \
  --zone-name "${PARENT_DNS_ZONE}" \
  --name "peered-vnet-basedomain-link" \
  --virtual-network "${PEERED_VNET_ID}" \
  --registration-enabled false
```

- d. Access the cluster as shown in the following example:

```
$ KUBECONFIG=${CLUSTER_NAME}-kubeconfig oc get nodes
```

Additional resources

- [Creating a private Azure hosted cluster](#)

4.2.5.7. Troubleshooting private hosted clusters on Azure

If your private hosted cluster gets stuck, check the **AzurePrivateLinkService** custom resource conditions.

Procedure

1. Enter the following command to check the Private Link conditions:

```
$ oc get azureprivatelinkservices \
  -n clusters-${CLUSTER_NAME} \
  -o jsonpath='{.items[0].status.conditions}' | jq .
```

2. Review the output and compare it to the following condition table:

Table 4.10. Private cluster stuck conditions

Condition	Possible cause
AzureInternalLoadBalancerAvailable = False	The private-router service has not received an internal load balancer IP address yet. Check the service status and Azure networking.

Condition	Possible cause
AzurePLSCreated = False	Private Link creation failed. Check the NAT subnet policies, credentials, and the HyperShift Operator logs.
AzurePrivateEndpointAvailable = False	Private endpoint creation failed or the connection was not approved. Check the Private Link auto-approval list and the Control Plane Operator logs.
AzurePrivateDNSAvailable = False	The DNS zone or record creation failed. In the Azure subscription that stores the infrastructure resources for the hosted cluster, check the Control Plane Operator identity permissions.

4.2.6. Example: Autoscaling a hosted cluster on Azure

To balance node pools on your Azure hosted cluster, you can configure autoscaling for the cluster.

When you scale up **NodePool** objects, the Cluster API Provider for Azure provisions individual Azure virtual machines.

To configure autoscaling for a hosted cluster on Azure, you edit the **spec.autoscaling** parameters in the **HostedCluster** resource. The following example shows a **HostedCluster** resource with 2 Azure node pools.



NOTE

This example shows only the autoscaling-related fields.

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: my-cluster
  namespace: my-cluster-namespace
spec:
  autoscaling:
    scaling: ScaleUpAndScaleDown
    maxNodesTotal: 12
    expanders:
      - Random
    scaleDown:
      delayAfterAddSeconds: 300
      unneededDurationSeconds: 600
      utilizationThresholdPercent: 40
    balancingIgnoredLabels:
      - "custom.label/environment"
    maxFreeDifferenceRatioPercent: 70
  ---
apiVersion: hypershift.openshift.io/v1beta1
```

```

kind: NodePool
metadata:
  name: my-cluster-nodepool-1
  namespace: my-cluster-namespace
spec:
  clusterName: my-cluster
  autoScaling:
    min: 1
    max: 6
  platform:
    azure:
      vmSize: Standard_D4s_v3
      # ...
---
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: my-cluster-nodepool-2
  namespace: my-cluster-namespace
spec:
  clusterName: my-cluster
  autoScaling:
    min: 1
    max: 6
  platform:
    azure:
      vmSize: Standard_D4s_v3
      # ...

```

- **spec.autoScaling.min** must be greater than or equal to **1**. Scaling from zero (**autoScaling.min: 0**) is not supported on Azure.

4.2.7. Deleting an Azure hosted cluster and its resources

If you no longer need a hosted cluster, you can remove it and its infrastructure. Any related Workload Identities and OIDC issuers that were created during setup can be reused for other clusters or deleted separately if you no longer need them.

4.2.7.1. Deleting a hosted cluster on Azure

If you are no longer using a hosted cluster on Azure, you can delete it.

Procedure

- To delete a hosted cluster, enter the following command:

```

$ hcp destroy cluster azure \
  --name $CLUSTER_NAME \
  --azure-creds $AZURE_CREDS \
  --dns-zone-rg-name $PERSISTENT_RG_NAME \
  --preserve-resource-group

```

- **--name** specifies your hosted cluster name.

- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers.
- **--dns-zone-rg-name** specifies the name of the resource group that contains your DNS zone.
- **--preserve-resource-group** specifies that the infrastructure will be preserved. If you do not want to preserve the infrastructure, do not include this flag.

4.2.7.2. Deleting Azure infrastructure

If you have Azure infrastructure without a hosted cluster, you can remove the infrastructure if you are not using it.

For example, this scenario can happen if you created the infrastructure standalone but never created a hosted cluster. Or, you might have manually deleted the hosted cluster or management cluster, but the infrastructure resources still exist.

You can delete the entire infrastructure, or delete cluster-specific resources but preserve the main resource group. Preserving the main resource group is helpful when you have other resources in the same resource group that you want to keep.

If you have a hosted cluster and want to delete infrastructure while you delete the hosted cluster, follow the steps in "Deleting a hosted cluster on Azure", but omit the **--preserve-resource-group** flag.

Procedure

- To delete the infrastructure, enter one of the following commands:
 - To delete the infrastructure, including the resource group, enter the following command:

```
$ hcp destroy infra azure \  
  --name <my_cluster_name> \  
  --infra-id <infra_id> \  
  --azure-creds <azure_credentials_file>
```

- **--name** specifies your hosted cluster name.
- **--infra-id** specifies a unique name that identifies your infrastructure. This value is used to name and tag Azure resources. Typically, it is the name of your cluster with a suffix appended to it.
- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers.

- To preserve the resource group but delete only cluster-specific resources, enter the following command:

```
$ hcp destroy infra azure \  
  --name <my_cluster_name> \  
  --infra-id <infra_id> \  
  --azure-creds <azure_credentials_file> \  
  --preserve-resource-group
```

- **--name** specifies your hosted cluster name.

- **--infra-id** specifies a unique name that identifies your infrastructure. This value is used to name and tag Azure resources. Typically, it is the name of your cluster with a suffix appended to it.
- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers.
- **--preserve-resource-group** specifies that you want to preserve the resource group.

4.2.7.3. Deleting Azure Workload Identities

As part of the process to delete a hosted cluster on Azure, you need to delete the Workload Identities.

To delete the Workload Identities, you enter a command on the hosted control planes command-line interface, **hcp**. The command uses the file that was generated when you created the Workload Identities to identify the identities to delete. Both the managed identities and their federated credentials are removed.

Prerequisites

- If you created infrastructure by using the Workload Identities, delete the infrastructure before you delete the identities.

Procedure

- Enter the following command:

```
$ hcp destroy iam azure \
  --azure-creds <azure_credentials_file> \
  --workload-identities-file <workload_identities_file> \
  --resource-group-name <resource_group> \
  --name <my_cluster_name> \
  --infra-id <infra_id> \
  --dns-zone-rg-name <dns_zone_rg> \
  --cloud <my_cloud_environment>
```

- **<azure_credentials_file>** specifies the Azure credentials file with permission to create managed identities and federated credentials.
- **<workload_identities_file>** specifies the path to the Workload Identities JSON file, such as **my-cluster-name-iam-output.json**.
- **<resource_group>** specifies the name of the resource group where you created identities.
- **<my_cluster_name>** specifies the name of your hosted cluster.
- **<infra_id>** specifies the unique identifier for naming Azure resources. Typically, this identifier is the cluster name with a suffix.
- **<dns_zone_rg>** specifies the DNS zone resource group.
- **<my_cloud_environment>** specifies the Azure cloud environment. Setting the **--cloud** flag is optional. The default value is **AzurePublicCloud**.

4.2.7.4. Deleting a private hosted cluster on Azure

If you are no longer using a private hosted cluster on Azure, you can delete it.

The deletion process automatically cleans up Private Link resources in the following order:

1. The Control Plane Operator removes the private endpoint, private DNS zones, VNet links, and A records.
2. The **hcp destroy** command removes role-based access control (RBAC) role assignments.
3. The HyperShift Operator removes Private Link.

Procedure

- To delete a private hosted cluster, enter the following command:

```
$ hcp destroy cluster azure \  
  --name ${CLUSTER_NAME} \  
  --azure-creds ${AZURE_CREDS} \  
  --resource-group-name ${MANAGED_RG_NAME} \  
  --dns-zone-rg-name ${DNS_ZONE_RG_NAME}
```

- **--name** specifies your hosted cluster name.
- **--azure-creds** specifies an Azure credentials file that has permission to create infrastructure resources, such as virtual networks, subnets, and load balancers. This flag is required because the **hcp destroy cluster azure** command cleans up RBAC role assignments before it deletes infrastructure.
- **--resource-group-name** specifies the name of the resource group where you created identities.
- **--dns-zone-rg-name** specifies the name of the resource group that contains your DNS zone.

4.3. DEPLOYING HOSTED CONTROL PLANES ON BARE METAL WITH THE AGENT PLATFORM

To maximize hardware performance and maintain control over your physical infrastructure, you can deploy hosted control planes on bare metal by using the Agent platform. This deployment method reduces virtualization overhead and offers low-latency networking for performance-intensive workloads.

4.3.1. About hosted control planes on bare metal with the Agent platform

You can deploy hosted control planes on bare-metal infrastructure with the Agent platform by configuring an OpenShift Container Platform cluster to function as a management cluster.

The management cluster is the OpenShift Container Platform cluster where the control planes are hosted. In some contexts, the management cluster is also known as the *hosting* cluster. The management cluster is not the same thing as the *managed* cluster. A managed cluster is a cluster that the hub cluster manages.

The hosted control planes feature is enabled by default.

The multicluster engine Operator supports only the default **local-cluster**, which is a hub cluster that is managed, and the hub cluster as the management cluster. If you have Red Hat Advanced Cluster Management installed, you can use the managed hub cluster, also known as the **local-cluster**, as the management cluster.

A *hosted cluster* is an OpenShift Container Platform cluster with its API endpoint and control plane that are hosted on the management cluster. The hosted cluster includes the control plane and its corresponding data plane. You can use the multicluster engine Operator console or the hosted control plane command-line interface (**hcp**) to create a hosted cluster.

The hosted cluster is automatically imported as a managed cluster. If you want to disable this automatic import feature, see "Disabling the automatic import of hosted clusters into multicluster engine Operator".

Additional resources

- [Disabling the automatic import of hosted clusters into multicluster engine Operator](#)

4.3.2. Preparing to deploy hosted control planes on bare metal

Before you deploy hosted control planes on bare metal, ensure that you understand the requirements for the deployment.

- Run the management cluster and compute nodes on the same platform.
- All bare-metal hosts require a manual start with a Discovery Image ISO that the central infrastructure management provides. You can start the hosts manually or through automation by using the Cluster Baremetal Operator. After each host starts, it runs an Agent process to discover the host details and complete the installation. An **Agent** custom resource represents each host.
- When you configure storage for hosted control planes, consider the recommended etcd practices. To ensure that you meet the latency requirements, dedicate a fast storage device to all hosted control plane etcd instances that run on each control-plane node. You can use LVM storage to configure a local storage class for hosted etcd pods. For more information, see "Recommended etcd practices" and "Persistent storage using logical volume manager storage".

Additional resources

- [Recommended etcd practices](#)
- [Persistent storage using LVM Storage](#)

4.3.2.1. Bare metal firewall, port, and service requirements

You must meet the firewall, port, and service requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.



NOTE

Services run on their default ports. However, if you use the **NodePort** publishing strategy, services run on the port that is assigned by the **NodePort** service.

Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary. For production deployments, use a load balancer to simplify access through a single IP address.

If your hub cluster has a proxy configuration, ensure that it can reach the hosted cluster API endpoint by adding all hosted cluster API endpoints to the **noProxy** field on the **Proxy** object. For more information, see "Configuring the cluster-wide proxy".

A hosted control plane exposes the following services on bare metal:

- **APIServer**
 - The **APIServer** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- **OAuthServer**
 - The **OAuthServer** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **OAuthServer** service.
- **Konnectivity**
 - The **Konnectivity** service runs on port 443 by default when you use the route and ingress to expose the service.
 - The **Konnectivity** agent establishes a reverse tunnel to allow the control plane to access the network for the hosted cluster. The agent uses egress to connect to the **Konnectivity** server. The server is exposed by using either a route on port 443 or a manually assigned **NodePort**.
 - If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
 - If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.
- **Ignition**
 - The **Ignition** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **Ignition** service.

You do not need the following services on bare metal:

- **OVNSbDb**
- **OIDC**

Additional resources

- [Configuring the cluster-wide proxy](#)

4.3.2.2. Bare metal infrastructure requirements

Although the Agent platform does not create any infrastructure, the Agent platform does have requirements for infrastructure.

- Agents: An *Agent* represents a host that is booted with a discovery image and is ready to be provisioned as an OpenShift Container Platform node.
- DNS: The API and ingress endpoints must be routable.

4.3.3. Configuring a management cluster for bare metal

Before you create a hosted cluster on bare metal with the Agent platform, you need a properly configured OpenShift Container Platform management cluster.

Prerequisites

- The management cluster and compute nodes must be on the same platform.
- You need the multicluster engine for Kubernetes Operator 2.2 and later installed on an OpenShift Container Platform cluster. You can install multicluster engine Operator as an Operator from the OpenShift Container Platform software catalog.

Procedure

1. The multicluster engine Operator must have at least one managed OpenShift Container Platform cluster. The **local-cluster** is automatically imported in multicluster engine Operator 2.2 and later. For more information about the **local-cluster**, see "Advanced configuration" in the Red Hat Advanced Cluster Management documentation. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

2. Add the **topology.kubernetes.io/zone** label to your bare-metal hosts on your management cluster. Ensure that each host has a unique value for **topology.kubernetes.io/zone**. Otherwise, all of the control plane pods are scheduled on a single node, causing a single point of failure.
3. Use the Agent platform to provision hosted control planes on bare metal. The Agent platform uses the central infrastructure management service to add compute nodes to a hosted cluster. For more information, see "Enabling the central infrastructure management service".
4. Install the hosted control plane command-line interface. For more information, see "Installing the hosted control planes command-line interface".

Additional resources

- [Advanced configuration](#)
- [Enabling the central infrastructure management service](#)
- [Installing the hosted control planes command-line interface](#)

4.3.4. DNS configurations on bare metal

The API Server for the hosted cluster is exposed as a **NodePort** service. A DNS entry must exist for **api.<hosted_cluster_name>.<base_domain>** that points to destination where the API Server can be reached.

The DNS entry can be as simple as a record that points to one of the nodes in the management cluster that is running the hosted control plane. The entry can also point to a load balancer that is deployed to redirect incoming traffic to the ingress pods.

Example DNS configuration

```
api.example.krnl.es.  IN A 192.168.122.20
api.example.krnl.es.  IN A 192.168.122.21
api.example.krnl.es.  IN A 192.168.122.22
api-int.example.krnl.es.  IN A 192.168.122.20
api-int.example.krnl.es.  IN A 192.168.122.21
api-int.example.krnl.es.  IN A 192.168.122.22
`*`.apps.example.krnl.es. IN A 192.168.122.23
```



NOTE

In the previous example, ***.apps.example.krnl.es. IN A 192.168.122.23** is either a node in the hosted cluster or a load balancer, if one has been configured.

If you are configuring DNS for a disconnected environment on an IPv6 network, the configuration looks like the following example.

Example DNS configuration for an IPv6 network

```
api.example.krnl.es.  IN A 2620:52:0:1306::5
api.example.krnl.es.  IN A 2620:52:0:1306::6
api.example.krnl.es.  IN A 2620:52:0:1306::7
api-int.example.krnl.es.  IN A 2620:52:0:1306::5
api-int.example.krnl.es.  IN A 2620:52:0:1306::6
api-int.example.krnl.es.  IN A 2620:52:0:1306::7
`*`.apps.example.krnl.es. IN A 2620:52:0:1306::10
```

If you are configuring DNS for a disconnected environment on a dual stack network, be sure to include DNS entries for both IPv4 and IPv6.

Example DNS configuration for a dual stack network

```
host-record=api-int.hub-dual.dns.base.domain.name,192.168.126.10
host-record=api.hub-dual.dns.base.domain.name,192.168.126.10
address=/apps.hub-dual.dns.base.domain.name/192.168.126.11
dhcp-host=aa:aa:aa:aa:10:01,ocp-control-plane-0,192.168.126.20
dhcp-host=aa:aa:aa:aa:10:02,ocp-control-plane-1,192.168.126.21
dhcp-host=aa:aa:aa:aa:10:03,ocp-control-plane-2,192.168.126.22
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,192.168.126.25
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,192.168.126.26

host-record=api-int.hub-dual.dns.base.domain.name,2620:52:0:1306::2
host-record=api.hub-dual.dns.base.domain.name,2620:52:0:1306::2
address=/apps.hub-dual.dns.base.domain.name/2620:52:0:1306::3
dhcp-host=aa:aa:aa:aa:10:01,ocp-control-plane-0,[2620:52:0:1306::5]
```

```

dhcp-host=aa:aa:aa:aa:10:02,ocp-control-plane-1,[2620:52:0:1306::6]
dhcp-host=aa:aa:aa:aa:10:03,ocp-control-plane-2,[2620:52:0:1306::7]
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,[2620:52:0:1306::8]
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,[2620:52:0:1306::9]

```

4.3.4.1. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA
- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```

#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - xxx.example.com
            - yyy.example.com
          servingCertificate:
            name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>

```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation

of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the **HostedCluster** namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.3.5. Creating an InfraEnv resource

Before you can create a hosted cluster on bare metal, you need an **InfraEnv** resource.

On hosted control planes, the control-plane components run as pods on the management cluster while the data plane runs on dedicated nodes. You can use the Assisted Service to boot your hardware with a discovery ISO that adds your hardware to a hardware inventory.

Later, when you create a hosted cluster, the hardware from the inventory is used to provision the data-plane nodes. The object that is used to get the discovery ISO is an **InfraEnv** resource. You need to create a **BareMetalHost** object that configures the cluster to boot the bare-metal node from the discovery ISO.

4.3.5.1. Creating an InfraEnv resource and adding nodes

To ensure that your hardware is provisioned correctly before you create a hosted cluster on bare metal, create an **InfraEnv** resource. You can create the resource and add nodes by using the command-line interface (CLI).

Procedure

1. Create a namespace to store your hardware inventory by entering the following command:

```
$ oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig create \
  namespace <namespace_example>
```


where:

<directory_example>

Is the name of the directory where the **kubeconfig** file for the management cluster is saved.

<namespace_example>

Is the name of the namespace that you are creating; for example, **hardware-inventory**.

Example output

```
namespace/hardware-inventory created
```

2. Copy the pull secret of the management cluster by entering the following command:

```
$ oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig \
-n openshift-config get secret pull-secret -o yaml \
| grep -vE "uid|resourceVersion|creationTimestamp|namespace" \
| sed "s/openshift-config/<namespace_example>/g" \
| oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig \
-n <namespace> apply -f -
```

where:

<directory_example>

Is the name of the directory where the **kubeconfig** file for the management cluster is saved.

<namespace_example>

Is the name of the namespace that you are creating; for example, **hardware-inventory**.

Example output

```
secret/pull-secret created
```

3. Create the **InfraEnv** resource by adding the following content to a YAML file:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: hosted
  namespace: <namespace_example>
spec:
  additionalNTPSources:
  - <ip_address>
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <ssh_public_key>
# ...
```

4. Apply the changes to the YAML file by entering the following command:

```
$ oc apply -f <infraenv_config>.yaml
```

Replace **<infraenv_config>** with the name of your file.

5. Verify that the **InfraEnv** resource was created by entering the following command:

```
$ oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig \
-n <namespace_example> get infraenv hosted
```

6. Add bare-metal hosts by following one of two methods:

- If you do not use the Metal3 Operator, obtain the discovery ISO from the **InfraEnv** resource and boot the hosts manually by completing the following steps:

- a. Download the live ISO by entering the following commands:

```
$ oc get infraenv -A
```

```
$ oc get infraenv <namespace_example> -o jsonpath='{.status.isoDownloadURL}' -n
<namespace_example> <iso_url>
```

- b. Boot the ISO. The node communicates with the Assisted Service and registers as an agent in the same namespace as the **InfraEnv** resource.
- c. For each agent, set the installation disk ID and hostname, and approve it to indicate that the agent is ready for use.

- i. Enter the following command to get the agents for your hosted control plane namespace:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

In this example, two agents are listed.

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
example-agent-1	auto-assign			
example-agent-2	auto-assign			

- ii. Enter the following command to set the installation disk ID and hostname for the first agent:

```
$ oc -n <hosted_control_plane_namespace> \
patch agent example-agent-1 \
-p '{"spec":
{"installation_disk_id":"/dev/sda","approved":true,"hostname":"worker-
0.example.krnl.es"}}' \
--type merge
```

- iii. Enter the following command to set the installation disk ID and hostname for the second agent:

```
$ oc -n <hosted_control_plane_namespace> \
patch agent example-agent-2 -p \
'{"spec":{"installation_disk_id":"/dev/sda","approved":true,"hostname":"worker-
```

```
1.example.krn1.es"}'} \
--type merge
```

- If you use the Metal3 Operator, you can automate the bare-metal host registration by creating the following objects:
 - a. Create a YAML file and add the following content to it:

```
apiVersion: v1
kind: Secret
metadata:
  name: hosted-worker0-bmc-secret
  namespace: <namespace_example>
data:
  password: <password>
  username: <username>
type: Opaque
---
apiVersion: v1
kind: Secret
metadata:
  name: hosted-worker1-bmc-secret
  namespace: <namespace_example>
data:
  password: <password>
  username: <username>
type: Opaque
---
apiVersion: v1
kind: Secret
metadata:
  name: hosted-worker2-bmc-secret
  namespace: <namespace_example>
data:
  password: <password>
  username: <username>
type: Opaque
---
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: hosted-worker0
  namespace: <namespace_example>
labels:
  infraenvs.agent-install.openshift.io: hosted
annotations:
  inspect.metal3.io: disabled
  bmac.agent-install.openshift.io/hostname: hosted-worker0
spec:
  automatedCleaningMode: disabled
  bmc:
    disableCertificateVerification: True
    address: <bmc_address>
    credentialsName: hosted-worker0-bmc-secret
    bootMACAddress: aa:aa:aa:aa:02:01
    online: true
```

```

---
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: hosted-worker1
  namespace: <namespace_example>
  labels:
    infraenvs.agent-install.openshift.io: hosted
  annotations:
    inspect.metal3.io: disabled
    bmac.agent-install.openshift.io/hostname: hosted-worker1
spec:
  automatedCleaningMode: disabled
  bmc:
    disableCertificateVerification: True
    address: <bmc_address>
    credentialsName: hosted-worker1-bmc-secret
  bootMACAddress: aa:aa:aa:aa:02:02
  online: true
---
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: hosted-worker2
  namespace: <namespace_example>
  labels:
    infraenvs.agent-install.openshift.io: hosted
  annotations:
    inspect.metal3.io: disabled
    bmac.agent-install.openshift.io/hostname: hosted-worker2
spec:
  automatedCleaningMode: disabled
  bmc:
    disableCertificateVerification: True
    address: <bmc_address>
    credentialsName: hosted-worker2-bmc-secret
  bootMACAddress: aa:aa:aa:aa:02:03
  online: true
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: capi-provider-role
  namespace: <namespace_example>
rules:
- apiGroups:
  - agent-install.openshift.io
  resources:
  - agents
  verbs:
  - '*'

```

where:

<namespace_example>

Is the your namespace.

<password>

Is the password for your secret.

<username>

Is the user name for your secret.

<bmc_address>

Is the BMC address for the **BareMetalHost** object.



NOTE

When you apply this YAML file, the following objects are created:

- Secrets with credentials for the Baseboard Management Controller (BMCs)
- The **BareMetalHost** objects
- A role for the HyperShift Operator to be able to manage the agents

Notice how the **InfraEnv** resource is referenced in the **BareMetalHost** objects by using the **infraenvs.agent-install.openshift.io: hosted** custom label. This ensures that the nodes are booted with the ISO generated.

- Apply the changes to the YAML file by entering the following command:

```
$ oc apply -f <bare_metal_host_config>.yaml
```

Replace **<bare_metal_host_config>** with the name of your file.

- Enter the following command, and then wait a few minutes for the **BareMetalHost** objects to move to the **Provisioning** state:

```
$ oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig -n <namespace_example> get bmh
```

Example output

NAME	STATE	CONSUMER	ONLINE	ERROR	AGE
hosted-worker0	provisioning	true		106s	
hosted-worker1	provisioning	true		106s	
hosted-worker2	provisioning	true		106s	

- Enter the following command to verify that nodes are booting and showing up as agents. This process can take a few minutes, and you might need to enter the command more than once.

```
$ oc --kubeconfig ~/<directory_example>/mgmt-kubeconfig -n <namespace_example> get agent
```

Example output

-

NAME	CLUSTER	APPROVED	ROLE	STAGE
aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0201		true	auto-assign	
aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0202		true	auto-assign	
aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0203		true	auto-assign	

4.3.5.2. Creating an InfraEnv resource by using the console

If you prefer to use the OpenShift Container Platform web console, you can use it to create an **InfraEnv** resource for a hosted cluster on bare metal.

Procedure

1. Open the OpenShift Container Platform web console and log in by entering your administrator credentials. For instructions to open the console, see "Accessing the web console".
2. In the console header, ensure that **All Clusters** is selected.
3. Click **Infrastructure** → **Host inventory** → **Create infrastructure environment**
4. After you create the **InfraEnv** resource, add bare-metal hosts from within the **InfraEnv** view by clicking **Add hosts** and selecting from the available options.

Additional resources

- [Accessing the web console](#)

4.3.6. Creating a hosted cluster on bare metal

You can create a hosted cluster on bare metal with the Agent platform by using the command-line interface (CLI), the console, or by using a mirror registry.

4.3.6.1. Creating a hosted cluster by using the CLI

On bare-metal infrastructure, you can import a hosted cluster or create one by using the command-line interface (CLI).

After you enable the Assisted Installer as an add-on to multicluster engine Operator and you create a hosted cluster with the Agent platform, the HyperShift Operator installs the Agent Cluster API provider in the hosted control plane namespace. The Agent Cluster API provider connects a management cluster that hosts the control plane and a hosted cluster that consists of only the compute nodes.

Prerequisites

- Each hosted cluster must have a cluster-wide unique name. A hosted cluster name cannot be the same as any existing managed cluster. Otherwise, the multicluster engine Operator cannot manage the hosted cluster.
- Do not use the word **clusters** as a hosted cluster name.
- You cannot create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.
- For best security and management practices, create a hosted cluster separate from other hosted clusters.

- Verify that you have a default storage class configured for your cluster. Otherwise, you might see pending persistent volume claims (PVCs).
- By default when you use the **hcp create cluster agent** command, the command creates a hosted cluster with configured node ports. The preferred publishing strategy for hosted clusters on bare metal exposes services through a load balancer. If you create a hosted cluster by using the web console or by using Red Hat Advanced Cluster Management, to set a publishing strategy for a service besides the Kubernetes API server, you must manually specify the **servicePublishingStrategy** information in the **HostedCluster** custom resource.
- Ensure that you meet the requirements described in "Requirements for hosted control planes on bare metal", which includes requirements related to infrastructure, firewalls, ports, and services. For example, those requirements describe how to add the appropriate zone labels to the bare-metal hosts in your management cluster, as shown in the following example commands:

```
$ oc label node [compute-node-1] topology.kubernetes.io/zone=zone1
```

```
$ oc label node [compute-node-2] topology.kubernetes.io/zone=zone2
```

```
$ oc label node [compute-node-3] topology.kubernetes.io/zone=zone3
```

- Ensure that you have added bare-metal nodes to a hardware inventory.

Procedure

1. Create a namespace by entering the following command:

```
$ oc create ns <hosted_cluster_namespace>
```

Replace **<hosted_cluster_namespace>** with an identifier for your hosted cluster namespace. The HyperShift Operator creates the namespace. During the hosted cluster creation process on bare-metal infrastructure, a generated Cluster API provider role requires that the namespace already exists.

2. Create the configuration file for your hosted cluster by entering the following command:

```
$ hcp create cluster agent \
  --name=<hosted_cluster_name> \
  --pull-secret=<path_to_pull_secret> \
  --agent-namespace=<hosted_control_plane_namespace> \
  --base-domain=<base_domain> \
  --api-server-address=api.<hosted_cluster_name>.<base_domain> \
  --etcd-storage-class=<etcd_storage_class> \
  --ssh-key=<path_to_ssh_key> \
  --namespace=<hosted_cluster_namespace> \
  --control-plane-availability-policy=HighlyAvailable \
  --release-image=quay.io/openshift-release-dev/ocp-release:<ocp_release_image>-multi \
  --node-pool-replicas=<node_pool_replica_count> \
  --render \
  --render-sensitive > hosted-cluster-config.yaml
```

where:

- **--name** specifies the name of your hosted cluster, such as **example**.

- **--pull-secret** specifies the path to your pull secret, such as `/user/name/pullsecret`.
 - **--agent-namespace** specifies your hosted control plane namespace, such as `clusters-example`. Ensure that agents are available in this namespace by using the `oc get agent -n <hosted_control_plane_namespace>` command.
 - **--base-domain** specifies your base domain, such as `krnl.es`.
 - **--api-server-address** specifies the IP address that gets used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.
 - **--etcd-storage-class** specifies the etcd storage class name, such as `lvm-storageclass`.
 - **--ssh-key** specifies the path to your SSH public key. The default file path is `~/.ssh/id_rsa.pub`.
 - **--namespace** specifies your hosted cluster namespace.
 - **--control-plane-availability-policy** specifies the availability policy for the hosted control plane components. Supported options are **SingleReplica** and **HighlyAvailable**. The default value is **HighlyAvailable**.
 - **--release-image** specifies the supported OpenShift Container Platform version that you want to use, such as `4.22.0-multi`. If you are using a disconnected environment, replace `<ocp_release_image>` with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".
 - **--node-pool-replicas** specifies the node pool replica count, such as `3`. You must specify the replica count as `0` or greater to create the same number of replicas. Otherwise, you do not create node pools.
3. Configure the service publishing strategy. By default, hosted clusters use the **NodePort** service publishing strategy because node ports are always available without additional infrastructure. However, you can configure the service publishing strategy to use a load balancer.
- If you are using the default **NodePort** strategy, configure the DNS to point to the hosted cluster compute nodes, not the management cluster nodes. For more information, see "DNS configurations on bare metal".
 - For production environments, use the **LoadBalancer** strategy because this strategy provides certificate handling and automatic DNS resolution. The following example demonstrates changing the service publishing **LoadBalancer** strategy in your hosted cluster configuration file:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  # ...
spec:
  services:
    - service: APIServer
      servicePublishingStrategy:
        type: LoadBalancer
    - service: Ignition
```



```

    servicePublishingStrategy:
      type: Route
  - service: Konnectivity
    servicePublishingStrategy:
      type: Route
  - service: OAuthServer
    servicePublishingStrategy:
      type: Route
  - service: OIDC
    servicePublishingStrategy:
      type: Route
  sshKey:
    name: <ssh_key>
# ...

```

Specify **LoadBalancer** as the API Server type. For all other services, specify **Route** as the type.

4. If you use external load balancers, configure the ingress endpoint as shown in the following example. If you do not configure the endpoint, the default behavior is to randomize the node port that the service exposes the ingress on. To configure how the ingress controller publishes the default ingress route, set the **endpointPublishingStrategy** parameter and its underlying functions by editing the **HostedCluster** resource:

```

apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
#...
spec:
  operatorConfiguration:
    ingressOperator:
      endpointPublishingStrategy:
        hostNetwork:
          httpPort: 80
          httpsPort: 443
          protocol: TCP
          statsPort: 1936
        type: HostNetwork
#...

```

The **spec.operatorConfiguration.ingressOperator.endPointPublishingStrategy.type** parameter specifies the endpoint for the load balancer. For bare-metal installations, use the **HostNetwork** type.

5. Apply the changes to the hosted cluster configuration file by entering the following command:

```
$ oc apply -f hosted_cluster_config.yaml
```

6. Check for the creation of the hosted cluster, node pools, and pods by entering the following commands:

```

$ oc get hostedcluster \
  <hosted_cluster_namespace> -n \
  <hosted_cluster_namespace> -o \
  jsonpath='{.status.conditions[?(@.status=="False")]}'| jq .

```

```
$ oc get nodepool \
  <hosted_cluster_namespace> -n \
  <hosted_cluster_namespace> -o \
  jsonpath='{.status.conditions[?(@.status=="False")]} ' | jq .
```

```
$ oc get pods -n <hosted_cluster_namespace>
```

7. Confirm that the hosted cluster is ready. The status of **Available: True** indicates the readiness of the cluster and the node pool status shows **AllMachinesReady: True**. These statuses indicate the healthiness of all cluster Operators.

8. Install MetalLB in the hosted cluster:

- a. Extract the **kubeconfig** file from the hosted cluster and set the environment variable for hosted cluster access by entering the following commands:

```
$ oc get secret \
  <hosted_cluster_namespace>-admin-kubeconfig \
  -n <hosted_cluster_namespace> \
  -o jsonpath='{.data.kubeconfig}' \
  | base64 -d > \
  kubeconfig-<hosted_cluster_namespace>.yaml
```

```
$ export KUBECONFIG="/path/to/kubeconfig-<hosted_cluster_namespace>.yaml"
```

- b. Install the MetalLB Operator by creating the **install-metallb-operator.yaml** file:

```
apiVersion: v1
kind: Namespace
metadata:
  name: metallb-system
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator
  namespace: metallb-system
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: metallb-operator
  namespace: metallb-system
spec:
  channel: "stable"
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
# ...
```

- c. Apply the file by entering the following command:

```
$ oc apply -f install-metallb-operator.yaml
```

- d. Configure the MetalLB IP address pool by creating the **deploy-metallb-ipaddresspool.yaml** file:

```

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: metallb
  namespace: metallb-system
spec:
  autoAssign: true
  addresses:
  - 10.11.176.71-10.11.176.75
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2advertisement
  namespace: metallb-system
spec:
  ipAddressPools:
  - metallb
# ...

```

- e. Apply the configuration by entering the following command:

```
$ oc apply -f deploy-metallb-ipaddresspool.yaml
```

- f. Verify the installation of MetalLB by checking the Operator status, the IP address pool, and the **L2Advertisement** resource by entering the following commands:

```
$ oc get pods -n metallb-system
```

```
$ oc get ipaddresspool -n metallb-system
```

```
$ oc get l2advertisement -n metallb-system
```

9. Configure the load balancer for ingress:

- a. Create the **ingress-loadbalancer.yaml** file:

```

apiVersion: v1
kind: Service
metadata:
  annotations:
    metallb.universe.tf/address-pool: metallb
  name: metallb-ingress
  namespace: openshift-ingress
spec:
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80

```

```
- name: https
  protocol: TCP
  port: 443
  targetPort: 443
selector:
  ingresscontroller.operator.openshift.io/deployment-ingresscontroller: default
type: LoadBalancer
# ...
```

- b. Apply the configuration by entering the following command:

```
$ oc apply -f ingress-loadbalancer.yaml
```

- c. Verify that the load balancer service works as expected by entering the following command:

```
$ oc get svc metallb-ingress -n openshift-ingress
```

Example output

```
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)
AGE
metallb-ingress LoadBalancer  172.31.127.129 10.11.176.71
80:30961/TCP,443:32090/TCP 16h
```

10. Configure the DNS to work with the load balancer:

- a. Configure the DNS for the **apps** domain by pointing the ***.apps.**
<hosted_cluster_namespace>.<base_domain> wildcard DNS record to the load balancer IP address.
- b. Verify the DNS resolution by entering the following command:

```
$ nslookup console-openshift-console.apps.<hosted_cluster_namespace>.  
<base_domain> <load_balancer_ip_address>
```

Example output

```
Server:      10.11.176.1
Address:     10.11.176.1#53

Name:   console-openshift-console.apps.my-hosted-cluster.sample-base-domain.com
Address: 10.11.176.71
```

Verification

1. Check the cluster Operators by entering the following command:

```
$ oc get clusteroperators
```

Ensure that all Operators show **AVAILABLE: True**, **PROGRESSING: False**, and **DEGRADED: False**.

2. Check the nodes by entering the following command:

```
$ oc get nodes
```

Ensure that each node has the **READY** status.

3. Test access to the console by entering the following URL in a web browser:

```
https://console-openshift-console.apps.<hosted_cluster_namespace>.<base_domain>
```

Additional resources

- [Manually importing a hosted cluster](#)
- [Extracting the release image digest](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.3.6.2. Creating a hosted cluster on bare metal by using the console

Instead of using the command-line interface to create a hosted cluster on bare metal, you can also use the OpenShift Container Platform web console.

Procedure

1. Open the OpenShift Container Platform web console and log in by entering your administrator credentials. For instructions to open the console, see "Accessing the web console".
2. In the console header, ensure that **All Clusters** is selected.
3. Click **Infrastructure → Clusters**.
4. Click **Create cluster → Host inventory → Hosted control plane**
The **Create cluster** page is displayed.
5. On the **Create cluster** page, follow the prompts to enter details about the cluster, node pools, networking, and automation.



NOTE

As you enter details about the cluster, you might find the following tips useful:

- If you want to use predefined values to automatically populate fields in the console, you can create a host inventory credential. For more information, see "Creating a credential for an on-premises environment".
- On the **Cluster details** page, the pull secret is your OpenShift Container Platform pull secret that you use to access OpenShift Container Platform resources. If you selected a host inventory credential, the pull secret is automatically populated.
- On the **Node pools** page, the namespace contains the hosts for the node pool. If you created a host inventory by using the console, the console creates a dedicated namespace.
- On the **Networking** page, you select an API server publishing strategy. The API server for the hosted cluster can be exposed either by using an existing load balancer or as a service of the **NodePort** type. A DNS entry must exist for the **api.<hosted_cluster_name>.<base_domain>** setting that points to the destination where the API server can be reached. This entry can be a record that points to one of the nodes in the management cluster or a record that points to a load balancer that redirects incoming traffic to the Ingress pods.

6. Review your entries and click **Create**.
The **Hosted cluster** view is displayed.
7. Monitor the deployment of the hosted cluster in the **Hosted cluster** view.
8. If you do not see information about the hosted cluster, ensure that **All Clusters** is selected, then click the cluster name.
9. Wait until the control plane components are ready. This process can take a few minutes.
10. To view the node pool status, scroll to the **NodePool** section. The process to install the nodes takes about 10 minutes. You can also click **Nodes** to confirm whether the nodes joined the hosted cluster.

Additional resources

- [Creating a credential for an on-premises environment](#)
- [Accessing the web console](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.3.6.3. Creating a hosted cluster on bare metal by using a mirror registry

You can use a mirror registry to create a hosted cluster on bare metal by specifying the **--image-content-sources** flag in the **hcp create cluster** command.

Procedure

1. Create a YAML file to define Image Content Source Policies (ICSP). See the following example:

```
- mirrors:
  - brew.registry.redhat.io
    source: registry.redhat.io
- mirrors:
  - brew.registry.redhat.io
    source: registry.stage.redhat.io
- mirrors:
  - brew.registry.redhat.io
    source: registry-proxy.engineering.redhat.com
```

2. Save the file as **icsp.yaml**. This file contains your mirror registries.
3. To create a hosted cluster by using your mirror registries, run the following command:

```
$ hcp create cluster agent \
  --name=<hosted_cluster_name> \
  --pull-secret=<path_to_pull_secret> \
  --agent-namespace=<hosted_control_plane_namespace> \
  --base-domain=<base_domain> \
  --api-server-address=api.<hosted_cluster_name>.<base_domain> \
  --image-content-sources <icsp.yaml> \
  --ssh-key <path_to_ssh_key> \
  --namespace <hosted_cluster_namespace> \
  --release-image=quay.io/openshift-release-dev/ocp-release:<ocp_release_image>
```

where:

<hosted_cluster_name>

Specifies the name of your hosted cluster, for example, **my-hosted-cluster**.

<path_to_pull_secret>

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

<hosted_control_plane_namespace>

Specifies your hosted control plane namespace, for example, **clusters-example**. Ensure that agents are available in this namespace by using the **oc get agent -n <hosted-control-plane-namespace>** command.

<base_domain>

Specifies your base domain, for example, **krnl.es**.

<hosted_cluster_name>.<base_domain>

Specifies the IP address that is used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.

<icsp.yaml>

Specifies the **icsp.yaml** file that defines ICSP and your mirror registries.

<path_to_ssh_key>

Specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.

<hosted_cluster_namespace>

Specifies your hosted cluster namespace.

<ocp_release_image>

Specifies the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**. If you are using a disconnected environment, replace **<ocp_release_image>** with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".

Additional resources

- [Extracting the release image digest](#)
- [Accessing the hosted cluster](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.3.7. Verifying hosted cluster creation

After the deployment process is complete, you can verify that the hosted cluster was created successfully.

After you create the hosted cluster, wait a few minutes before you start the steps in the procedure.

Procedure

1. Obtain the kubeconfig for your new hosted cluster by entering the extract command:

```
$ oc extract -n <hosted-control-plane-namespace> secret/admin-kubeconfig \
--to=- > kubeconfig-<hosted-cluster-name>
```

2. Use the kubeconfig to view the cluster Operators of the hosted cluster. Enter the following command:

```
$ oc get co --kubeconfig=kubeconfig-<hosted-cluster-name>
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
console	4.10.26	True	False	False 2m38s
dns	4.10.26	True	False	False 2m52s
image-registry	4.10.26	True	False	False 2m8s
ingress	4.10.26	True	False	False 22m

3. You can also view the running pods on your hosted cluster by entering the following command:

```
$ oc get pods -A --kubeconfig=kubeconfig-<hosted-cluster-name>
```

Example output

NAMESPACE	STATUS	RESTARTS	AGE	NAME	READY
kube-system	Running	0	3m52s	konnektivity-agent-khlqv	0/1
openshift-cluster-node-tuning-operator				tuned-dhw5p	1/1


```

Running      0      109s
openshift-cluster-storage-operator      cluster-storage-operator-5f784969f5-vwzgz
1/1 Running      1 (113s ago) 20m
openshift-cluster-storage-operator      csi-snapshot-controller-6b7687b7d9-7nrfw
1/1 Running      0      3m8s
openshift-console      console-5cbf6c7969-6gk6z      1/1
Running      0      119s
openshift-console      downloads-7bcd756565-6wj5j      1/1
Running      0      4m3s
openshift-dns-operator      dns-operator-77d755cd8c-xjfbn      2/2
Running      0      21m
openshift-dns      dns-default-kfqnh      2/2
Running      0      113s

```

4.4. DEPLOYING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION

With hosted control planes and OpenShift Virtualization, you can create OpenShift Container Platform clusters with worker nodes that are hosted by KubeVirt virtual machines.

Hosted control planes on OpenShift Virtualization provides several benefits:

- Enhances resource usage by packing hosted control planes and hosted clusters in the same underlying bare-metal infrastructure
- Separates hosted control planes and hosted clusters to provide strong isolation
- Reduces cluster provision time by eliminating the bare-metal node bootstrapping process
- Manages many releases under the same base OpenShift Container Platform cluster

The hosted control planes feature is enabled by default.

You can use the hosted control plane command-line interface, **hcp**, to create an OpenShift Container Platform hosted cluster. The hosted cluster is automatically imported as a managed cluster. If you want to disable this automatic import feature, see "Disabling the automatic import of hosted clusters into multicluster engine Operator".

Additional resources

- [Disabling the automatic import of hosted clusters into multicluster engine Operator](#)

4.4.1. Prerequisites to deploy hosted control planes on OpenShift Virtualization

To create an OpenShift Container Platform cluster on OpenShift Virtualization, you must meet several prerequisites.

- You have administrator access to an OpenShift Container Platform cluster, version 4.14 or later, specified in the **KUBECONFIG** environment variable.
- The OpenShift Container Platform management cluster must have wildcard DNS routes enabled, as shown in the following command:

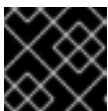
```
$ oc patch ingresscontroller -n openshift-ingress-operator default \
--type=json \
```

```
-p '[{"op": "add", "path": "/spec/routeAdmission", "value": {"wildcardPolicy": "WildcardsAllowed"}}]
```

- The OpenShift Container Platform management cluster has OpenShift Virtualization, version 4.14 or later, installed on it. For more information, see "Installing OpenShift Virtualization using the web console".
- The OpenShift Container Platform management cluster is on-premise bare metal.
- The OpenShift Container Platform management cluster must be configured with **OVNKubernetes** as the default pod network Container Network Interface (CNI). Live migration is supported for nodes only if the CNI is OVN-Kubernetes.
- The OpenShift Container Platform management cluster has a default storage class. For more information, see "Postinstallation storage configuration". The following example shows how to set a default storage class:

```
$ oc patch storageclass ocs-storagecluster-ceph-rbd \
-p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

- When you configure storage for hosted control planes, consider the recommended etcd practices. To ensure that you meet the latency requirements, dedicate a fast storage device to all hosted control plane etcd instances that run on each control-plane node. You can use LVM storage to configure a local storage class for hosted etcd pods. For more information, see "Recommended etcd practices" and "Persistent storage using Logical Volume Manager storage".
- On the OpenShift Container Platform cluster that hosts the OpenShift Virtualization virtual machines, you must use a **ReadWriteMany** (RWX) storage class so that live migration can be enabled.
- You have a valid pull secret file for the **quay.io/openshift-release-dev** repository. For more information, see "Install OpenShift on any x86_64 platform with user-provisioned infrastructure".
- You have installed the hosted control plane command-line interface.
- You have configured a load balancer. For more information, see "Configuring MetalLB".
- For optimal network performance, you are using a network maximum transmission unit (MTU) of 9000 or greater on the OpenShift Container Platform cluster that hosts the KubeVirt virtual machines. If you use a lower MTU setting, network latency and the throughput of the hosted pods are affected. Enable multiqueue on node pools only when the MTU is 9000 or greater.



IMPORTANT

You cannot change the MTU value for your cluster as a postinstallation task.

- The multicluster engine Operator has at least one managed OpenShift Container Platform cluster. The **local-cluster** is automatically imported. For more information about the **local-cluster**, see "Advanced configuration" in the multicluster engine Operator documentation. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- Ensure each hosted cluster has a cluster-wide unique name.
- Do not use **clusters** as a hosted cluster name.
- Do not create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.

Additional resources

- [Installing OpenShift Virtualization using the web console](#)
- [Postinstallation storage configuration](#)
- [Install OpenShift on any x86_64 platform with user-provisioned infrastructure](#)
- [Configuring MetalLB](#)
- [Advanced configuration \(Red Hat Advanced Cluster Management documentation\)](#)
- [Recommended etcd practices](#)
- [Persistent storage using Logical Volume Manager Storage](#)

4.4.1.1. OpenShift Virtualization firewall and port requirements

Ensure that you meet the firewall and port requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.

- The **kube-apiserver** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use the **NodePort** publishing strategy, ensure that the node port that is assigned to the **kube-apiserver** service is exposed.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- If you use the **NodePort** publishing strategy, use a firewall rule for the **ignition-server** and **OAuth-server** settings.
- The **konnnectivity** agent, which establishes a reverse tunnel to allow bi-directional communication on the hosted cluster, requires egress access to the cluster API server address on port 6443. With that egress access, the agent can reach the **kube-apiserver** service.
 - If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
 - If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.
- If you change the default port of 6443, adjust the rules to reflect that change.
- Ensure that you open any ports that are required by the workloads that run in the clusters.
- Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary.

- For production deployments, use a load balancer to simplify access through a single IP address.

4.4.2. Live migration for compute nodes

While the management cluster for hosted cluster virtual machines (VMs) is undergoing updates or maintenance, the hosted cluster VMs can be automatically live migrated to prevent disrupting hosted cluster workloads. As a result, the management cluster can be updated without affecting the availability and operation of the KubeVirt platform hosted clusters.



IMPORTANT

The live migration of KubeVirt VMs is enabled by default provided that the VMs use **ReadWriteMany** (RWX) storage for both the root volume and the storage classes that are mapped to the **kubevirt-csi** CSI provider.

You can verify that the VMs in a node pool are capable of live migration by checking the **KubeVirtNodesLiveMigratable** condition in the **status** section of a **NodePool** object.

In the following example, the VMs cannot be live migrated because RWX storage is not used.

Example configuration where VMs cannot be live migrated

```
- lastTransitionTime: "2024-10-08T15:38:19Z"
  message: |
    3 of 3 machines are not live migratable
    Machine user-np-ngst4-gw2hz: DisksNotLiveMigratable: user-np-ngst4-gw2hz is not a live
    migratable machine: cannot migrate VMI: PVC user-np-ngst4-gw2hz-rhcos is not shared, live
    migration requires that all PVCs must be shared (using ReadWriteMany access mode)
    Machine user-np-ngst4-npq7x: DisksNotLiveMigratable: user-np-ngst4-npq7x is not a live
    migratable machine: cannot migrate VMI: PVC user-np-ngst4-npq7x-rhcos is not shared, live
    migration requires that all PVCs must be shared (using ReadWriteMany access mode)
    Machine user-np-ngst4-q5nkb: DisksNotLiveMigratable: user-np-ngst4-q5nkb is not a live
    migratable machine: cannot migrate VMI: PVC user-np-ngst4-q5nkb-rhcos is not shared, live
    migration requires that all PVCs must be shared (using ReadWriteMany access mode)
  observedGeneration: 1
  reason: DisksNotLiveMigratable
  status: "False"
  type: KubeVirtNodesLiveMigratable
```

In the next example, the VMs meet the requirements to be live migrated.

Example configuration where VMs can be live migrated

```
- lastTransitionTime: "2024-10-08T15:38:19Z"
  message: "All is well"
  observedGeneration: 1
  reason: AsExpected
  status: "True"
  type: KubeVirtNodesLiveMigratable
```

While live migration can protect VMs from disruption in normal circumstances, events such as infrastructure node failure can result in a hard restart of any VMs that are hosted on the failed node. For live migration to be successful, the source node that a VM is hosted on must be working correctly.

When the VMs in a node pool cannot be live migrated, workload disruption might occur on the hosted cluster during maintenance on the management cluster. By default, the hosted control planes controllers try to drain the workloads that are hosted on KubeVirt VMs that cannot be live migrated before the VMs are stopped. Draining the hosted cluster nodes before stopping the VMs allows pod disruption budgets to protect workload availability within the hosted cluster.

4.4.3. Configuring MetalLB

Before you can create a hosted cluster on the KubeVirt platform, you must have the MetalLB load balancer configured.

Prerequisites

- You have installed the MetalLB Operator. For more information, see "Installing the MetalLB Operator".

Procedure

1. Create a **MetalLB** resource by saving the following sample YAML content in the **configure-metallb.yaml** file:

```
apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb-system
```

2. Apply the YAML content by entering the following command:

```
$ oc apply -f configure-metallb.yaml
```

Example output

```
metallb.metallb.io/metallb created
```

3. Create a **IPAddressPool** resource by saving the following sample YAML content in the **create-ip-address-pool.yaml** file:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: metallb
  namespace: metallb-system
spec:
  addresses:
  - 192.168.216.32-192.168.216.122
```

Create an address pool with an available range of IP addresses within the node network. Replace the IP address range with an unused pool of available IP addresses in your network.

4. Apply the YAML content by entering the following command:

```
$ oc apply -f create-ip-address-pool.yaml
```

Example output

```
ipaddresspool.metallb.io/metallb created
```

5. Create a **L2Advertisement** resource by saving the following sample YAML content in the **l2advertisement.yaml** file:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2advertisement
  namespace: metallb-system
spec:
  ipAddressPools:
    - metallb
```

6. Apply the YAML content by entering the following command:

```
$ oc apply -f l2advertisement.yaml
```

Example output

```
l2advertisement.metallb.io/metallb created
```

Additional resources

- [Installing the MetalLB Operator](#)

4.4.4. Hosted clusters with the KubeVirt platform

With OpenShift Container Platform 4.14 or later, you can create a hosted cluster with KubeVirt by using the command-line interface (CLI), the console, or by using external infrastructure.

4.4.4.1. Creating a hosted cluster with the KubeVirt platform by using the CLI

To create a hosted cluster on OpenShift Virtualization, you can use the hosted control plane command-line interface (CLI), **hcp**.



IMPORTANT

Avoid storing all hosted cluster information in a shared namespace. If you create a hosted cluster in a shared namespace and then back up and restore the hosted cluster, you might unintentionally change other hosted clusters. Either store hosted cluster information in a separate namespace or set up your hosted cluster to back up and restore resources based on labels.

Procedure

1. Create a hosted cluster with the KubeVirt platform by entering the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
```

```

--node-pool-replicas <node_pool_replica_count> \
--pull-secret <path_to_pull_secret> \
--memory <value_for_memory> \
--cores <value_for_cpu> \
--etcd-storage-class=<etcd_storage_class> \
--arch <architecture_of_the_nodepool> \
--release-image <ocp_release_image_for_the_cluster> \
--image-content-sources <path_to_image_content_sources_file> \
--additional-trust-bundle <path_to_ca_bundle_file>

```

- **--name** defines the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** defines the node pool replica count, for example, **3**. You must specify the replica count as **0** or greater to create the same number of replicas. Otherwise, no node pools are created.
- **--pull-secret** defines the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** defines a value for memory, for example, **6Gi**.
- **--cores** defines a value for CPU, for example, **2**.
- **--etcd-storage-class** defines the etcd storage class name, for example, **lvm-storageclass**.
- **--arch** defines the architecture of the node pool, for example, **s390x**. The default is **amd64**.
- **--release-image** defines the OpenShift Container Platform release image for the cluster, for example, **quay.io/openshift-release-dev/ocp-release:4.20.14-multi**. You can use the **--release-image** flag to set up the hosted cluster with a specific OpenShift Container Platform release.
- **--image-content-sources** specifies the path to a file with image content sources.
- **--additional-trust-bundle** specifies the path to a file with user CA bundle.

A default node pool is created for the cluster with a specific number of virtual machine worker replicas according to the **--node-pool-replicas** flag.

2. After a few moments, verify that the hosted control plane pods are running by entering the following command:

```
$ oc -n clusters-<hosted-cluster-name> get pods
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
capi-provider-5cc7b74f47-n5gkr	1/1	Running	0	3m
catalog-operator-5f799567b7-fd6jw	2/2	Running	0	69s
certified-operators-catalog-784b9899f9-mrp6p	1/1	Running	0	66s
cluster-api-6bbc867966-l4dwl	1/1	Running	0	66s
.				
.				
.				
redhat-operators-catalog-9d5fd4d44-z8qqk	1/1	Running	0	66s

A hosted cluster that has worker nodes that are backed by KubeVirt virtual machines typically takes 10-15 minutes to be fully provisioned.

Verification

- To check the status of the hosted cluster, see the corresponding **HostedCluster** resource by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

See the following example output, which illustrates a fully provisioned **HostedCluster** object:

NAMESPACE	NAME	VERSION	KUBECONFIG	PROGRESS
AVAILABLE	PROGRESSING	MESSAGE		
clusters	my-hosted-cluster	<4.x.0>	example-admin-kubeconfig	Completed True
False	The hosted control plane is available			

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

Additional resources

- [Labeling management cluster nodes](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.4.4.2. Creating a hosted cluster with the KubeVirt platform by using external infrastructure

By default, the HyperShift Operator hosts both the control plane pods of the hosted cluster and the KubeVirt worker VMs within the same cluster. With the external infrastructure feature, you can place the worker node VMs on a separate cluster from the control plane pods.

- The *management cluster* is the OpenShift Container Platform cluster that runs the HyperShift Operator and hosts the control plane pods for a hosted cluster.
- The *infrastructure cluster* is the OpenShift Container Platform cluster that runs the KubeVirt worker VMs for a hosted cluster.
- By default, the management cluster also acts as the infrastructure cluster that hosts VMs. However, for external infrastructure, the management and infrastructure clusters are different.



IMPORTANT

Avoid storing all hosted cluster information in a shared namespace. If you create a hosted cluster in a shared namespace and then back up and restore the hosted cluster, you might unintentionally change other hosted clusters. Either store hosted cluster information in a separate namespace or set up your hosted cluster to back up and restore resources based on labels.

Prerequisites

- You must have a namespace on the external infrastructure cluster for the KubeVirt nodes to be hosted in.

- You must have a **kubeconfig** file for the external infrastructure cluster.

Procedure

- In the **hcp** command-line interface, place the KubeVirt worker VMs on the infrastructure cluster, use the **--infra-kubeconfig-file** and **--infra-namespace** arguments, as shown in the following example:

```
$ hcp create cluster kubevirt \
  --name <hosted-cluster-name> \
  --node-pool-replicas <worker-count> \
  --pull-secret <path-to-pull-secret> \
  --memory <value-for-memory> \
  --cores <value-for-cpu> \
  --infra-namespace=<hosted-cluster-namespace>-<hosted-cluster-name> \
  --infra-kubeconfig-file=<path-to-external-infra-kubeconfig>
```

- **--name** defines the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** defines the worker count, for example, **2**.
- **--pull-secret** defines the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** defines a value for memory, for example, **6Gi**.
- **--cores** defines a value for CPU, for example, **2**.
- **--infra-namespace** defines the infrastructure namespace, for example, **clusters-example**.
- **--infra-kubeconfig-file** defines the path to your **kubeconfig** file for the infrastructure cluster, for example, **/user/name/external-infra-kubeconfig**.

After you enter the command, the control plane pods are hosted on the management cluster that the HyperShift Operator runs on, and the KubeVirt VMs are hosted on a separate infrastructure cluster.

Additional resources

- [Labeling management cluster nodes](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.4.4.3. Creating a hosted cluster by using the console

If you prefer to work in the OpenShift Container Platform console instead of the CLI, you can create a hosted cluster on the KubeVirt platform by using the console.

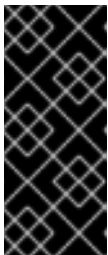


NOTE

If you want to use predefined values to automatically populate fields in the console, you can create a OpenShift Virtualization credential. For more information, see "Creating a credential for an on-premises environment".

Procedure

1. Open the OpenShift Container Platform web console and log in by entering your administrator credentials.
2. In the console header, ensure that **All Clusters** is selected.
3. Click **Infrastructure > Clusters**.
4. Click **Create cluster > Red Hat OpenShift Virtualization > Hosted**.
5. On the **Create cluster** page, follow the prompts to enter details about the cluster and node pools.
On the **Cluster details** page, the pull secret is your OpenShift Container Platform pull secret that you use to access OpenShift Container Platform resources. If you selected a OpenShift Virtualization credential, the pull secret is automatically populated.



IMPORTANT

Avoid storing all hosted cluster information in a shared namespace. If you create a hosted cluster in a shared namespace and then back up and restore the hosted cluster, you might unintentionally change other hosted clusters. Either store hosted cluster information in a separate namespace or set up your hosted cluster to back up and restore resources based on labels.

6. On the **Node pools** page, expand the **Networking options** section and configure the networking options for your node pool:
 - a. In the **Additional networks** field, enter a network name in the format of **<namespace>/<name>**; for example, **my-namespace/network1**. The namespace and the name must be valid DNS labels. Multiple networks are supported.
 - b. By default, the **Attach default pod network** checkbox is selected. You can clear this checkbox only if additional networks exist.
7. Review your entries and click **Create**.
The **Hosted cluster** view is displayed.

Verification

1. Monitor the deployment of the hosted cluster in the **Hosted cluster** view. If you do not see information about the hosted cluster, ensure that **All Clusters** is selected, and click the cluster name.
2. Wait until the control plane components are ready. This process can take a few minutes.
3. To view the node pool status, scroll to the **NodePool** section. The process to install the nodes takes about 10 minutes. You can also click **Nodes** to confirm whether the nodes joined the hosted cluster.

Additional resources

- [Labeling management cluster nodes](#)
- [Configuring a custom API server certificate in a hosted cluster](#)
- [Creating a credential for an on-premises environment \(Red Hat Advanced Cluster Management documentation\)](#)

- [Accessing the hosted cluster](#)

4.4.5. Configuring the default ingress and DNS for hosted control planes on OpenShift Virtualization

Every OpenShift Container Platform cluster includes a default application Ingress Controller, which must have an wildcard DNS record associated with it.

By default, hosted clusters that are created by using OpenShift Virtualization automatically become a subdomain of the OpenShift Container Platform cluster that the virtual machines run on.

For example, your OpenShift Container Platform cluster might have the following default ingress DNS entry:

```
*.apps.mgmt-cluster.example.com
```

As a result, a hosted cluster that is named **guest** and that runs on that underlying OpenShift Container Platform cluster has the following default ingress:

```
*.apps.guest.apps.mgmt-cluster.example.com
```

Procedure

- For the default ingress DNS to work properly, the cluster that hosts the virtual machines must allow wildcard DNS routes. You can configure this behavior by entering the following command:

```
$ oc patch ingresscontroller -n openshift-ingress-operator default \
  --type=json \
  -p '[{"op": "add", "path": "/spec/routeAdmission", "value": {"wildcardPolicy":
    "WildcardsAllowed"}}]'
```



NOTE

When you use the default hosted cluster ingress, connectivity is limited to HTTPS traffic over port 443. Plain HTTP traffic over port 80 is rejected. This limitation applies to only the default ingress behavior.

4.4.5.1. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA
- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```
#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - xxx.example.com
            - yyy.example.com
          servingCertificate:
            name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>
```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the **HostedCluster** namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.4.6. Customized ingress and DNS behavior

If you do not want to use the default ingress and DNS behavior, you can configure a KubeVirt hosted cluster with a unique base domain at creation time.

This option requires manual configuration steps during creation and involves three main steps: cluster creation, load balancer creation, and wildcard DNS configuration.

4.4.6.1. Creating a hosted cluster that specifies the base domain

If you do not want to use the default ingress and DNS behavior, you can configure a KubeVirt hosted cluster with a unique base domain at creation time.

Procedure

1. Create the cluster by entering the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
  --node-pool-replicas <worker_count> \
  --pull-secret <path_to_pull_secret> \
  --memory <value_for_memory> \
  --cores <value_for_cpu> \
  --base-domain <basedomain> \
  --arch <architecture_of_the_nodepool> \
  --release-image <ocp_release_image_for_the_cluster> \
  --image-content-sources <path_to_image_content_sources_file> \
  --additional-trust-bundle <path_to_ca_bundle_file>
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count, for example, **2**.
- **--pull-secret** specifies the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** specifies a value for memory, for example, **6Gi**.
- **--cores** specifies a value for CPU, for example, **2**.
- **--base-domain** specifies the base domain, for example, **hypershift.lab**.
- **--arch** specifies the architecture of the node pool, for example, **s390x**. The default is **amd64**.
- **--release-image** specifies the ocp release image for the cluster, for example, **quay.io/openshift-release-dev/ocp-release:4.20.14-multi**.
- **--image-content-sources** specifies the path to a file with image content sources.
- **--additional-trust-bundle** specifies the path to a file with user CA bundle.

As a result, the hosted cluster has an ingress wildcard that is configured for the cluster name

and the base domain, for example, **.apps.example.hypershift.lab**. The hosted cluster remains in **Partial** status because after you create a hosted cluster with unique base domain, you must configure the required DNS records and load balancer.

Verification

- 1. View the status of your hosted cluster by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

Example output

NAME	VERSION	KUBECONFIG	PROGRESS	AVAILABLE
example	example-admin-kubeconfig	Partial	True	False
hosted control plane is available				

- 2. Access the cluster by entering the following commands:

```
$ hcp create kubeconfig --name <hosted_cluster_name> \
> <hosted_cluster_name>-kubeconfig

$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
console	<4.x.0>	False	False	False 30m
RouteHealthAvailable: failed to GET route (https://console-openshift-console.apps.example.hypershift.lab): Get "https://console-openshift-console.apps.example.hypershift.lab": dial tcp: lookup console-openshift-console.apps.example.hypershift.lab on 172.31.0.10:53: no such host				
ingress	<4.x.0>	True	False	True 28m
The "default" ingress controller reports Degraded=True: DegradedConditions: One or more other status conditions indicate a degraded state: CanaryChecksSucceeding=False (CanaryChecksRepetitiveFailures: Canary route checks for the default ingress controller are failing)				

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

- 3. To fix any errors in the output, complete the steps in "Setting up the load balancer" and "Setting up a wildcard DNS".



NOTE

If your hosted cluster is on bare metal, you might need MetalLB to set up load balancer services. For more information, see "Configuring MetalLB".

4.4.6.2. Setting up the load balancer

Set up the load balancer service that routes ingress traffic to the KubeVirt VMs and assigns a wildcard DNS entry to the load balancer IP address.

Procedure

1. A **NodePort** service that exposes the hosted cluster ingress already exists. You can export the node ports and create the load balancer service that targets those ports.

- a. Get the HTTP node port by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get services \
-n openshift-ingress router-nodeport-default \
-o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}'
```

Note the HTTP node port value to use in the next step.

- b. Get the HTTPS node port by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get services \
-n openshift-ingress router-nodeport-default \
-o jsonpath='{.spec.ports[?(@.name=="https")].nodePort}'
```

Note the HTTPS node port value to use in the next step.

2. Enter the following information in a YAML file:

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: <hosted_cluster_name>
    name: <hosted_cluster_name>-apps
    namespace: clusters-<hosted_cluster_name>
spec:
  ports:
    - name: https-443
      port: 443
      protocol: TCP
      targetPort: <https_node_port>
    - name: http-80
      port: 80
      protocol: TCP
      targetPort: <http_node_port>
  selector:
    kubevirt.io: virt-launcher
  type: LoadBalancer
```

where:

- **<https_node_port>** specifies the HTTPS node port value that you noted in the previous step.
- **<http_node_port>** specifies the HTTP node port value that you noted in the previous step.

3. Create the load balancer service by running the following command:

■

```
$ oc create -f <file_name>.yaml
```

4.4.6.3. Setting up a wildcard DNS

If you are customizing the ingress and DNS for your hosted cluster, you need to set up a wildcard DNS record or CNAME that references the external IP of the load balancer service.

Procedure

1. Get the external IP address by entering the following command:

```
$ oc -n clusters-<hosted_cluster_name> get service <hosted-cluster-name>-apps \
-o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Example output

```
192.168.20.30
```

2. Configure a wildcard DNS entry that references the external IP address. View the following example DNS entry:

```
*.apps.<hosted_cluster_name>.<base_domain>.
```

The DNS entry must be able to route inside and outside of the cluster.

DNS resolutions example

```
dig +short test.apps.example.hypershift.lab
192.168.20.30
```

Verification

- Check that hosted cluster status has moved from **Partial** to **Completed** by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

Example output

NAME	VERSION	KUBECONFIG	PROGRESS	AVAILABLE
example	<4.x.0>	example-admin-kubeconfig	Completed	True
The hosted control plane is available				

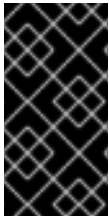
Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

4.4.7. Additional networks, guaranteed CPUs, and VM scheduling for node pools

You can configure additional networks for node pools, request a guaranteed CPU access for Virtual Machines (VMs), or manage scheduling of KubeVirt VMs.

4.4.7.1. Adding multiple networks to a node pool

By default, nodes generated by a node pool are attached to the pod network. You can attach additional networks to the nodes by using **NetworkAttachmentDefinitions**.



IMPORTANT

You can connect compute nodes to multiple networks that have a Linux bridge or a user-defined network (UDN) that uses the localnet topology. Red Hat does not support connecting compute nodes to multiple UDNs that use a layer 2 or a layer 3 overlay topology.

Procedure

- To add multiple networks to nodes, use the **--additional-network** argument by running the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
  --node-pool-replicas <worker_node_count> \
  --pull-secret <path_to_pull_secret> \
  --memory <memory> \
  --cores <cpu> \
  --additional-network name:<namespace/name> \
  --additional-network name:<namespace/name>
```

where:

hosted_cluster_name

Specifies the name of your hosted cluster, for example, **my-hosted-cluster**.

worker_node_count

Specifies your compute node count, for example, **2**.

path_to_pull_secret

Specifies the path to your pull secret, for example, **/user/name/pullsecret**.

memory

Specifies the memory value, for example, **8Gi**.

cpu

Specifies the CPU value, for example, **2**.

namespace/name

Specifies the value of the **--additional-network** argument to **name:<namespace/name>**.

Replace **<namespace/name>** with a namespace and name of your

NetworkAttachmentDefinitions.

4.4.7.1.1. Using an additional network as default

You can add your additional network as a default network for the nodes by disabling the default pod network.

Procedure

- To add an additional network as default to your nodes, run the following command:

```
$ hcp create cluster kubevirt \  
  --name <hosted_cluster_name> \  
  --node-pool-replicas <worker_node_count> \  
  --pull-secret <path_to_pull_secret> \  
  --memory <memory> \  
  --cores <cpu> \  
  --attach-default-network false \  
  --additional-network name:<namespace>/<network_name>
```

- **--name** specifies the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** specifies your worker node count, for example, **2**.
- **--pull-secret** specifies the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** specifies the memory value, for example, **8Gi**.
- **--cores** specifies the CPU value, for example, **2**.
- **--attach-default-network false** disables the default pod network.
- **--additional-network** specifies the additional network that you want to add to your nodes, for example, **name:my-namespace/my-network**.

4.4.7.2. Requesting guaranteed CPU resources

By default, KubeVirt VMs might share their CPUs with other workloads on a node. This might impact performance of a VM. To avoid the performance impact, you can request a guaranteed CPU access for VMs.

Procedure

- To request guaranteed CPU resources, set the **--qos-class** argument to **Guaranteed** by running the following command:

```
$ hcp create cluster kubevirt \  
  --name <hosted_cluster_name> \  
  --node-pool-replicas <worker_node_count> \  
  --pull-secret <path_to_pull_secret> \  
  --memory <memory> \  
  --cores <cpu> \  
  --qos-class Guaranteed
```

- **--name** specifies the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** specifies your worker node count, for example, **2**.
- **--pull-secret** specifies the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** specifies the memory value, for example, **8Gi**.
- **--cores** specifies the CPU value, for example, **2**.

- **--qos-class Guaranteed** guarantees that the specified number of CPU resources are assigned to VMs.

4.4.7.3. Scheduling KubeVirt VMs on a set of nodes

By default, KubeVirt virtual machines (VMs) created by a node pool are scheduled to any available nodes. You can schedule KubeVirt VMs on a specific set of nodes that has enough capacity to run the VM.

Procedure

- To schedule KubeVirt VMs within a node pool on a specific set of nodes, use the **--vm-node-selector** argument by running the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
  --node-pool-replicas <worker_node_count> \
  --pull-secret <path_to_pull_secret> \
  --memory <memory> \
  --cores <cpu> \
  --vm-node-selector <label_key>=<label_value>,<label_key>=<label_value>
```

- **--name** specifies the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** specifies your worker node count, for example, **2**.
- **--pull-secret** specifies the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** specifies the memory value, for example, **8Gi**.
- **--cores** specifies the CPU value, for example, **2**.
- **--vm-node-selector** defines a specific set of nodes that contains the key-value pairs. Replace **<label_key>** with the keys of your labels and replace **<label_value>** with the values of your labels.

4.4.8. Scaling a node pool

You can manually scale a node pool for a hosted cluster on OpenShift Virtualization by using the **oc scale** command.

Procedure

1. Run the following command:

```
NODEPOOL_NAME=${CLUSTER_NAME}-work
NODEPOOL_REPLICAS=5

$ oc scale nodepool/$NODEPOOL_NAME --namespace clusters \
  --replicas=$NODEPOOL_REPLICAS
```

2. After a few moments, enter the following command to see the status of the node pool:

```
$ oc --kubeconfig $CLUSTER_NAME-kubeconfig get nodes
```

-

Example output

NAME	STATUS	ROLES	AGE	VERSION
example-9jvnf	Ready	worker	97s	v1.27.4+18eadca
example-n6prw	Ready	worker	116m	v1.27.4+18eadca
example-nc6g4	Ready	worker	117m	v1.27.4+18eadca
example-thp29	Ready	worker	4m17s	v1.27.4+18eadca
example-twxns	Ready	worker	88s	v1.27.4+18eadca

Additional resources

- [Scaling up and down workloads in a hosted cluster](#)

4.4.8.1. Adding node pools

You can create node pools for a hosted cluster by specifying a name, number of replicas, and any additional information, such as memory and CPU requirements.

Procedure

1. If the management cluster has a cluster-wide proxy configured, you must configure proxy settings in the **HostedCluster** resource by completing the following steps:

- a. Edit the **HostedCluster** resource by entering the following command:

```
$ oc edit hc <hosted_cluster_name> -n <hosted_cluster_namespace>
```

- b. In the **HostedCluster** resource, add the proxy configuration as shown in the following example:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  annotations:
    # ...
    hypershift.openshift.io/HasBeenAvailable: "true"
    hypershift.openshift.io/management-platform: VSphere
  # ...
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
  # ...
spec:
  # ...
  clusterID: fa45babd-40f3-4085-9b30-8bc3b7df1557
  configuration:
    proxy:
      httpProxy: http://web-proxy.example.com:3128
      httpsProxy: http://web-proxy.example.com:3128
      noProxy: .example.com,192.168.10.0/24
```

In the **spec.configuration.proxy** fields, specify the details of the proxy configuration.

- c. Check the status on the management cluster by entering the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace>
```

- d. Check the status on the hosted cluster by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get nodes
```

2. To create a node pool, enter the following information. In this example, the node pool has more CPUs assigned to the VMs:

```
export NODEPOOL_NAME=${CLUSTER_NAME}-extra-cpu
export WORKER_COUNT="2"
export MEM="6Gi"
export CPU="4"
export DISK="16"

$ hcp create nodepool kubevirt \
  --cluster-name $CLUSTER_NAME \
  --name $NODEPOOL_NAME \
  --node-count $WORKER_COUNT \
  --memory $MEM \
  --cores $CPU \
  --root-volume-size $DISK
```

3. Check the status of the node pool by listing **nodepool** resources in the namespace:

```
$ oc get nodepools --namespace <hosted_cluster_namespace>
```

Example output

NAME	CLUSTER	DESIRED NODES	CURRENT NODES	AUTOSCALING	AUTOREPAIR	VERSION	UPDATINGVERSION	UPDATINGCONFIG
example	example	5	5	False	False	<4.x.0>		
example-extra-cpu	example	2		False	False			True
True	Minimum availability requires 2 replicas, current 0 available							

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

Verification

1. After some time, you can check the status of the node pool by entering the following command:

```
$ oc --kubeconfig $CLUSTER_NAME-kubeconfig get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
example-9jvnf	Ready	worker	97s	v1.27.4+18eadca
example-n6prw	Ready	worker	116m	v1.27.4+18eadca
example-nc6g4	Ready	worker	117m	v1.27.4+18eadca
example-thp29	Ready	worker	4m17s	v1.27.4+18eadca
example-twxns	Ready	worker	88s	v1.27.4+18eadca
example-extra-cpu-zh9l5	Ready	worker	2m6s	v1.27.4+18eadca

```
example-extra-cpu-zr8mj Ready worker 102s v1.27.4+18eadca
```

2. Verify that the node pool is in the status that you expect by entering this command:

```
$ oc get nodepools --namespace <hosted_cluster_namespace>
```

Example output

```
NAME          CLUSTER    DESIRED NODES  CURRENT NODES
AUTOSCALING  AUTOREPAIR  VERSION  UPDATINGVERSION  UPDATINGCONFIG
MESSAGE
example       example    5         5               False      False      <4.x.0>
example-extra-cpu  example    2         2               False      False      <4.x.0>
```

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

Additional resources

- [Scaling down the data plane to zero](#)

4.4.9. Verifying hosted cluster creation on OpenShift Virtualization

After you create a hosted cluster on OpenShift Virtualization, verify that it was successfully created.

Procedure

1. Verify that the **HostedCluster** resource transitioned to the **completed** state by entering the following command:

```
$ oc get --namespace clusters hostedclusters <hosted_cluster_name>
```

Example output

```
NAMESPACE NAME    VERSION  KUBECONFIG          PROGRESS  AVAILABLE
PROGRESSING MESSAGE
clusters  example  4.12.2  example-admin-kubeconfig  Completed  True      False
The hosted control plane is available
```

2. Verify that all the cluster operators in the hosted cluster are online by entering the following commands:

```
$ hcp create kubeconfig --name <hosted_cluster_name> \
  > <hosted_cluster_name>-kubeconfig
```

```
$ oc get co --kubeconfig=<hosted_cluster_name>-kubeconfig
```

Example output

```
NAME          VERSION  AVAILABLE  PROGRESSING  DEGRADED
SINCE MESSAGE
console       4.12.2  True      False        False      2m38s
csi-snapshot-controller  4.12.2  True      False        False      4m3s
```

dns	4.12.2	True	False	False	2m52s
image-registry	4.12.2	True	False	False	2m8s
ingress	4.12.2	True	False	False	22m
kube-apiserver	4.12.2	True	False	False	23m
kube-controller-manager	4.12.2	True	False	False	23m
kube-scheduler	4.12.2	True	False	False	23m
kube-storage-version-migrator	4.12.2	True	False	False	4m52s
monitoring	4.12.2	True	False	False	69s
network	4.12.2	True	False	False	4m3s
node-tuning	4.12.2	True	False	False	2m22s
openshift-apiserver	4.12.2	True	False	False	23m
openshift-controller-manager	4.12.2	True	False	False	23m
openshift-samples	4.12.2	True	False	False	2m15s
operator-lifecycle-manager	4.12.2	True	False	False	22m
operator-lifecycle-manager-catalog	4.12.2	True	False	False	23m
operator-lifecycle-manager-packageserver	4.12.2	True	False	False	23m
service-ca	4.12.2	True	False	False	4m41s
storage	4.12.2	True	False	False	4m43s

4.5. DEPLOYING HOSTED CONTROL PLANES ON NON-BARE-METAL AGENT MACHINES

To maintain infrastructure flexibility while using existing virtualization layers, you can deploy hosted control planes on non-bare-metal Agent machines. You can use the management benefits of the Agent platform when running on virtualized environments or other cloud-based virtual machines.



IMPORTANT

Hosted control planes on non-bare-metal agent machines is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can deploy hosted control planes by configuring a cluster to function as a hosting cluster. The hosting cluster is an OpenShift Container Platform cluster where the control planes are hosted. The hosting cluster is also known as the management cluster.



NOTE

The management cluster is not the same thing as the *managed* cluster. A managed cluster is a cluster that the hub cluster manages.

The hosted control planes feature is enabled by default.

The multicluster engine Operator supports only the default **local-cluster** managed hub cluster. On Red Hat Advanced Cluster Management (RHACM) 2.10, you can use the **local-cluster** managed hub cluster as the hosting cluster.

A *hosted cluster* is an OpenShift Container Platform cluster with its API endpoint and control plane that are hosted on the hosting cluster. The hosted cluster includes the control plane and its corresponding data plane. You can use the multicluster engine Operator console or the **hcp** command-line interface (CLI) to create a hosted cluster.

The hosted cluster is automatically imported as a managed cluster. If you want to disable this automatic import feature, see "Disabling the automatic import of hosted clusters into multicluster engine Operator".

Additional resources

- [Disabling the automatic import of hosted clusters into multicluster engine Operator](#)

4.5.1. Preparing to deploy hosted control planes on non-bare-metal agent machines

Before you deploy hosted control planes on non-bare-metal agent machines, ensure that you understand requirements for the deployment.

- You can add agent machines as a worker node to a hosted cluster by using the Agent platform. Agent machine represents a host booted with a Discovery Image and ready to be provisioned as an OpenShift Container Platform node. The Agent platform is part of the central infrastructure management service. For more information, see "Enabling the central infrastructure management service".
- All hosts that are not bare metal require a manual boot with a Discovery Image ISO that the central infrastructure management provides.
- When you scale up the node pool, a machine is created for every replica. For every machine, the Cluster API provider finds and installs an Agent that is approved, is passing validations, is not currently in use, and meets the requirements that are specified in the node pool specification. You can monitor the installation of an Agent by checking its status and conditions.
- When you scale down a node pool, Agents are unbound from the corresponding cluster. Before you can reuse the Agents, you must restart them by using the Discovery image.
- When you configure storage for hosted control planes, consider the recommended etcd practices. To ensure that you meet the latency requirements, dedicate a fast storage device to all hosted control planes etcd instances that run on each control-plane node. You can use LVM storage to configure a local storage class for hosted etcd pods. For more information, see "Recommended etcd practices" and "Persistent storage using logical volume manager storage" in the OpenShift Container Platform documentation.

Additional resources

- [Enabling the central infrastructure management service \(Red Hat Advanced Cluster Management documentation\)](#)
- [Recommended etcd practices](#)
- [Persistent storage using logical volume manager storage](#)

4.5.1.1. Prerequisites for deploying hosted control planes on non-bare-metal agent machines

Before you deploy hosted control planes on non-bare-metal agent machines, ensure you meet the prerequisites.

- You must have multicluster engine for Kubernetes Operator 2.5 or later installed on an OpenShift Container Platform cluster. You can install the multicluster engine Operator as an Operator from the OpenShift Container Platform software catalog.
- You must have at least one managed OpenShift Container Platform cluster for the multicluster engine Operator. The **local-cluster** management cluster is automatically imported. For more information about the **local-cluster**, see "Advanced configuration" in the Red Hat Advanced Cluster Management documentation. You can check the status of your management cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- You have enabled central infrastructure management. For more information, see "Enabling the central infrastructure management service" in the Red Hat Advanced Cluster Management documentation.
- You have installed the **hcp** command-line interface.
- Your hosted cluster has a cluster-wide unique name.
- You are running the management cluster and workers on the same infrastructure.

Additional resources

- [Advanced configuration \(Red Hat Advanced Cluster Management documentation\)](#)
- [Enabling the central infrastructure management service \(Red Hat Advanced Cluster Management documentation\)](#)

4.5.1.2. Firewall, port, and service requirements for non-bare-metal agent machines

You must meet the firewall and port requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.



NOTE

Services run on their default ports. However, if you use the **NodePort** publishing strategy, services run on the port that is assigned by the **NodePort** service.

Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary. For production deployments, use a load balancer to simplify access through a single IP address.

A hosted control plane exposes the following services on non-bare-metal agent machines:

- **APIServer**
 - The **APIServer** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- **OAuthServer**

- The **OAuthServer** service runs on port 443 by default when you use the route and ingress to expose the service.
- If you use the **NodePort** publishing strategy, use a firewall rule for the **OAuthServer** service.
- **Konnectivity**
 - The **Konnectivity** service runs on port 443 by default when you use the route and ingress to expose the service.
 - The **Konnectivity** agent establishes a reverse tunnel to allow the control plane to access the network for the hosted cluster. The agent uses egress to connect to the **Konnectivity** server. The server is exposed by using either a route on port 443 or a manually assigned **NodePort**.
 - If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
 - If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.
- **Ignition**
 - The **Ignition** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **Ignition** service.

You do not need the following services on non-bare-metal agent machines:

- **OVNSbDb**
- **OIDC**

4.5.1.3. Infrastructure requirements for non-bare-metal agent machines

The Agent platform does not create any infrastructure, but it has several requirements.

- **Agents:** An *Agent* represents a host that is booted with a discovery image and is ready to be provisioned as an OpenShift Container Platform node.
- **DNS:** The API and ingress endpoints must be routable.

Additional resources

- [Recommended etcd practices](#)
- [Persistent storage using logical volume manager storage](#)
- [Disabling the automatic import of hosted clusters into multicluster engine Operator](#)
- [Manually enabling the hosted control planes feature](#)
- [Disabling the hosted control planes feature](#)

4.5.2. DNS configuration on non-bare-metal agent machines

The API Server for the hosted cluster is exposed as a **NodePort** service. A DNS entry must exist for **api.<hosted_cluster_name>.<basedomain>** that points to destination where the API Server can be reached.

The DNS entry can be as simple as a record that points to one of the nodes in the managed cluster that is running the hosted control plane. The entry can also point to a load balancer that is deployed to redirect incoming traffic to the ingress pods.

The following examples show how to configure DNS for specific environments.

Example DNS configuration for a connected environment on an IPv4 network

```
api.example.krnl.es.    IN A 192.168.122.20
api.example.krnl.es.    IN A 192.168.122.21
api.example.krnl.es.    IN A 192.168.122.22
api-int.example.krnl.es. IN A 192.168.122.20
api-int.example.krnl.es. IN A 192.168.122.21
api-int.example.krnl.es. IN A 192.168.122.22
`*`.apps.example.krnl.es. IN A 192.168.122.23
```

Example DNS configuration for a disconnected environment on an IPv6 network

```
api.example.krnl.es.    IN A 2620:52:0:1306::5
api.example.krnl.es.    IN A 2620:52:0:1306::6
api.example.krnl.es.    IN A 2620:52:0:1306::7
api-int.example.krnl.es. IN A 2620:52:0:1306::5
api-int.example.krnl.es. IN A 2620:52:0:1306::6
api-int.example.krnl.es. IN A 2620:52:0:1306::7
`*`.apps.example.krnl.es. IN A 2620:52:0:1306::10
```

Example DNS configuration for a disconnected environment on a dual stack network

```
host-record=api-int.hub-dual.dns.base.domain.name,192.168.126.10
host-record=api.hub-dual.dns.base.domain.name,192.168.126.10
address=/apps.hub-dual.dns.base.domain.name/192.168.126.11
dhcp-host=aa:aa:aa:aa:10:01,ocp-master-0,192.168.126.20
dhcp-host=aa:aa:aa:aa:10:02,ocp-master-1,192.168.126.21
dhcp-host=aa:aa:aa:aa:10:03,ocp-master-2,192.168.126.22
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,192.168.126.25
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,192.168.126.26

host-record=api-int.hub-dual.dns.base.domain.name,2620:52:0:1306::2
host-record=api.hub-dual.dns.base.domain.name,2620:52:0:1306::2
address=/apps.hub-dual.dns.base.domain.name/2620:52:0:1306::3
dhcp-host=aa:aa:aa:aa:10:01,ocp-master-0,[2620:52:0:1306::5]
dhcp-host=aa:aa:aa:aa:10:02,ocp-master-1,[2620:52:0:1306::6]
dhcp-host=aa:aa:aa:aa:10:03,ocp-master-2,[2620:52:0:1306::7]
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,[2620:52:0:1306::8]
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,[2620:52:0:1306::9]
```

For this configuration, be sure to include DNS entries for both IPv4 and IPv6.

4.5.3. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA
- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```
#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - xxx.example.com
            - yyy.example.com
          servingCertificate:
            name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>
```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the **HostedCluster** namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.5.4. Creating a hosted cluster on non-bare-metal agent machines by using the CLI

When you create a hosted cluster with the Agent platform, the HyperShift Operator installs the Agent Cluster API provider in the hosted control plane namespace. You can create a hosted cluster on bare metal or import one.



NOTE

- Each hosted cluster must have a cluster-wide unique name. A hosted cluster name cannot be the same as any existing managed cluster in order for multicluster engine Operator to manage it.
- Do not use **clusters** as a hosted cluster name.
- A hosted cluster cannot be created in the namespace of a multicluster engine Operator managed cluster.

Procedure

1. Create the hosted control plane namespace by entering the following command:

```
$ oc create ns <hosted_cluster_namespace>-<hosted_cluster_name>
```

Replace **<hosted_cluster_namespace>** with your hosted cluster namespace name, for example, **my-hosted-cluster-namespace**. Replace **<hosted_cluster_name>** with your hosted cluster name.

2. Create a hosted cluster by entering the following command:

```
$ hcp create cluster agent \
  --name=my-hosted-cluster \
  --pull-secret=/user/name/pullsecret \
  --agent-namespace=clusters-example \
```

```
--base-domain=krnl.es \
--api-server-address=api.my-hosted-cluster.krnl.es \
--etcd-storage-class=lvm-storageclass \
--ssh-key ~/.ssh/id_rsa.pub \
--namespace my-hosted-cluster-namespace \
--control-plane-availability-policy HighlyAvailable \
--release-image=quay.io/openshift-release-dev/ocp-release:4.22.0-multi \
--node-pool-replicas 3
```

- **--name** specifies the name of your hosted cluster.
- **--pull-secret** specifies the path to your pull secret.
- **--agent-namespace** specifies your hosted control plane namespace. Ensure that agents are available in this namespace by using the **oc get agent -n <hosted-control-plane-namespace>** command.
- **--base-domain** specifies your base domain.
- **--api-server-address** The **--api-server-address** flag defines the IP address that is used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.
- **--etcd-storage-class** specifies the etcd storage class name. Verify that you have a default storage class configured for your cluster. Otherwise, you might end up with pending PVCs.
- **--ssh_key** specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.
- **--namespace** specifies your hosted cluster namespace.
- **--control-plane-availability-policy** specifies the availability policy for the hosted control plane components. Supported options are **SingleReplica** and **HighlyAvailable**. The default value is **HighlyAvailable**.
- **--release-image** specifies the supported OpenShift Container Platform version that you want to use.
- **--node-pool-replicas** specifies the node pool replica count. You must specify the replica count as **0** or greater to create the same number of replicas. Otherwise, no node pools are created.

Verification

- After a few moments, verify that your hosted control plane pods are up and running by entering the following command:

```
$ oc -n <hosted_cluster_namespace>-<hosted_cluster_name> get pods
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
catalog-operator-6cd867cc7-phb2q	2/2	Running	0	2m50s
control-plane-operator-f6b4c8465-4k5dh	1/1	Running	0	4m32s

Additional resources

- [Manually importing a hosted cluster](#)
- [Configuring a custom API server certificate in a hosted cluster](#)
- [Extracting the release image digest](#)

4.5.5. Creating a hosted cluster on non-bare-metal agent machines by using the web console

You can create a hosted cluster on non-bare-metal agent machines by using the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Open the OpenShift Container Platform web console and log in by entering your administrator credentials.
2. In the console header, select **All Clusters**.
3. Click **Infrastructure → Clusters**.
4. Click **Create cluster Host inventory → Hosted control plane**
The **Create cluster** page is displayed.
5. On the **Create cluster** page, follow the prompts to enter details about the cluster, node pools, networking, and automation.
As you enter details about the cluster, you might find the following tips useful:
 - If you want to use predefined values to automatically populate fields in the console, you can create a host inventory credential. For more information, see *Creating a credential for an on-premises environment*.
 - On the **Cluster details** page, the pull secret is your OpenShift Container Platform pull secret that you use to access OpenShift Container Platform resources. If you selected a host inventory credential, the pull secret is automatically populated.
 - On the **Node pools** page, the namespace contains the hosts for the node pool. If you created a host inventory by using the console, the console creates a dedicated namespace.
 - On the **Networking** page, you select an API server publishing strategy. The API server for the hosted cluster can be exposed either by using an existing load balancer or as a service of the **NodePort** type. A DNS entry must exist for the **api.<hosted_cluster_name>.<basedomain>** setting that points to the destination where the API server can be reached. This entry can be a record that points to one of the nodes in the management cluster or a record that points to a load balancer that redirects incoming traffic to the Ingress pods.
6. Review your entries and click **Create**.
The **Hosted cluster** view is displayed.

7. Monitor the deployment of the hosted cluster in the **Hosted cluster** view. If you do not see information about the hosted cluster, ensure that **All Clusters** is selected, and click the cluster name. Wait until the control plane components are ready. This process can take a few minutes.
8. To view the node pool status, scroll to the **NodePool** section. The process to install the nodes takes about 10 minutes. You can also click **Nodes** to confirm whether the nodes joined the hosted cluster.

Additional resources

- [Creating a credential for an on-premises environment \(Red Hat Advanced Cluster Management documentation\)](#)
- [Accessing the web console](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.5.6. Creating a hosted cluster on non-bare-metal agent machines by using a mirror registry

You can use a mirror registry to create a hosted cluster on non-bare-metal agent machines by specifying the **--image-content-sources** flag in the **hcp create cluster** command.

Procedure

1. Create a YAML file to define Image Content Source Policies (ICSP). See the following example:

```
- mirrors:
  - brew.registry.redhat.io
    source: registry.redhat.io
- mirrors:
  - brew.registry.redhat.io
    source: registry.stage.redhat.io
- mirrors:
  - brew.registry.redhat.io
    source: registry-proxy.engineering.redhat.com
```

2. Save the file as **icsp.yaml**. This file contains your mirror registries.
3. To create a hosted cluster by using your mirror registries, run the following command:

```
$ hcp create cluster agent \
  --name=my-hosted-cluster \
  --pull-secret=/user/name/pullsecret \
  --agent-namespace=clusters-example \
  --base-domain=krnl.es \
  --api-server-address=api.my-hosted-cluster.krnl.es \
  --image-content-sources icsp.yaml \
  --ssh-key ~/.ssh/id_rsa.pub \
  --namespace my-hosted-cluster-namespace \
  --release-image=quay.io/openshift-release-dev/ocp-release:4.22.0-multi
```

- **--name** specifies the name of your hosted cluster.
- **--pull-secret** specifies the path to your pull secret.

- **--agent-namespace** specifies your hosted control plane namespace. Ensure that agents are available in this namespace by using the **oc get agent -n <hosted-control-plane-namespace>** command.
- **--base-domain** specifies your base domain.
- **--api-server-address** defines the IP address that is used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.
- **--image-content-sources** specifies the **icsp.yaml** file that defines ICSP and your mirror registries.
- **--ssh-key** specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.
- **--namespace** specifies your hosted cluster namespace.
- **--release-image** specifies the supported OpenShift Container Platform version that you want to use. If you are using a disconnected environment, replace the version with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".

Additional resources

- [Accessing the hosted cluster](#)
- [Configuring a custom API server certificate in a hosted cluster](#)

4.5.7. Verifying hosted cluster creation on non-bare-metal agent machines

After the deployment process is complete, you can verify that the hosted cluster was created successfully.

Follow these steps a few minutes after you create the hosted cluster.

Procedure

1. Obtain the **kubeconfig** file for your new hosted cluster by entering the following command:

```
$ oc extract -n <hosted_cluster_namespace> \
  secret/<hosted_cluster_name>-admin-kubeconfig --to=- \
  > kubeconfig-<hosted_cluster_name>
```

2. Use the **kubeconfig** file to view the cluster Operators of the hosted cluster. Enter the following command:

```
$ oc get co --kubeconfig=kubeconfig-<hosted_cluster_name>
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
console	4.10.26	True	False	False 2m38s

csi-snapshot-controller	4.10.26	True	False	False	4m3s
dns	4.10.26	True	False	False	2m52s

- View the running pods on your hosted cluster by entering the following command:

```
$ oc get pods -A --kubeconfig=kubeconfig-<hosted_cluster_name>
```

Example output

NAMESPACE	STATUS	RESTARTS	AGE	NAME	READY
kube-system	Running	0	3m52s	konnnectivity-agent-khlqv	0/1
openshift-cluster-samples-operator	2/2 Running	0	20m	cluster-samples-operator-6b5bcb9dff-kpnbc	
openshift-monitoring	Running	0	100s	alertmanager-main-0	6/6
openshift-monitoring	3/3 Running	0	104s	openshift-state-metrics-677b9fb74f-qqp6g	

4.6. DEPLOYING HOSTED CONTROL PLANES ON IBM Z

You can deploy hosted control planes on IBM Z by configuring a cluster to function as a management cluster. The management cluster is the OpenShift Container Platform cluster where the control planes are hosted. The management cluster is also known as the *hosting* cluster.



NOTE

The *management* cluster is not the *managed* cluster. A managed cluster is a cluster that the hub cluster manages. The *management* cluster can run on either the x86_64 architecture, supported beginning with OpenShift Container Platform 4.17 and multicluster engine for Kubernetes Operator 2.7, or the s390x architecture, supported beginning with OpenShift Container Platform 4.20 and multicluster engine for Kubernetes Operator 2.10.

You can convert a managed cluster to a management cluster by using the **hypershift** add-on to deploy the HyperShift Operator on that cluster. Then, you can start to create the hosted cluster.

The multicluster engine Operator supports only the default **local-cluster**, which is a hub cluster that is managed, and the hub cluster as the management cluster.

To provision hosted control planes on bare metal, you can use the Agent platform. The Agent platform uses the central infrastructure management service to add worker nodes to a hosted cluster. For more information, see "Enabling the central infrastructure management service".

Each IBM Z system host must be started with the PXE or ISO images that are provided by the central infrastructure management. After each host starts, it runs an Agent process to discover the details of the host and completes the installation. An Agent custom resource represents each host.

When you create a hosted cluster with the Agent platform, HyperShift Operator installs the Agent Cluster API provider in the hosted control plane namespace.

4.6.1. Prerequisites to configure hosted control planes on IBM Z

Ensure you meet the prerequisites to configure hosted control planes on IBM Z.

- The multicluster engine for Kubernetes Operator version 2.7 or later must be installed on an OpenShift Container Platform cluster. You can install multicluster engine Operator as an Operator from the OpenShift Container Platform OperatorHub.
- The multicluster engine Operator must have at least one managed OpenShift Container Platform cluster. The **local-cluster** is automatically imported in multicluster engine Operator 2.7 and later. For more information about the **local-cluster**, see *Advanced configuration* in the Red Hat Advanced Cluster Management documentation. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- You need a hosting cluster with at least three worker nodes to run the HyperShift Operator.
- You need to enable the central infrastructure management service. For more information, see "Enabling the central infrastructure management service".
- You need to install the hosted control plane command-line interface. For more information, see "Installing the hosted control plane command-line interface".



NOTE

The *management* cluster can run on either the x86_64 architecture, supported beginning with OpenShift Container Platform 4.17 and multicluster engine for Kubernetes Operator 2.7, or the s390x architecture, supported beginning with OpenShift Container Platform 4.20 and multicluster engine for Kubernetes Operator 2.10.

Additional resources

- [Advanced configuration](#)
- [Enabling the central infrastructure management service](#)
- [Installing the hosted control planes command-line interface](#)

4.6.2. IBM Z infrastructure requirements

The Agent platform does not create any infrastructure, but requires several resources for infrastructure.

- **Agents:** An *Agent* represents a host that is booted with a discovery image, or PXE image and is ready to be provisioned as an OpenShift Container Platform node.
- **DNS:** The API and Ingress endpoints must be routable.

The hosted control planes feature is enabled by default. If you disabled the feature and want to manually enable it, or if you need to disable the feature, see "Enabling or disabling the hosted control planes feature".

4.6.3. DNS configuration for hosted control planes on IBM Z

The API server for the hosted cluster is exposed as a **NodePort** service. A DNS entry must exist for the **api.<hosted_cluster_name>.<base_domain>** that points to the destination where the API server is reachable.

The DNS entry can be as simple as a record that points to one of the nodes in the managed cluster that is running the hosted control plane.

The entry can also point to a load balancer deployed to redirect incoming traffic to the Ingress pods.

See the following example of a DNS configuration:

```
$ cat /var/named/<example.krnl.es.zone>
```

Example output

```
$ TTL 900
@ IN SOA bastion.example.krnl.es.com. hostmaster.example.krnl.es.com. (
    2019062002
    1D 1H 1W 3H )
IN NS bastion.example.krnl.es.com.
;
;
api          IN A 1xx.2x.2xx.1xx
api-int      IN A 1xx.2x.2xx.1xx
;
;
*.apps       IN A 1xx.2x.2xx.1xx
;
;EOF
```

The **api** record refers to the IP address of the API load balancer that handles ingress and egress traffic for hosted control planes.

For IBM z/VM, add IP addresses that correspond to the IP address of the agent.

```
compute-0    IN A 1xx.2x.2xx.1yy
compute-1    IN A 1xx.2x.2xx.1yy
```

4.6.3.1. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA
- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```
#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - xxx.example.com
            - yyy.example.com
          servingCertificate:
            name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>
```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the **HostedCluster** namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.6.4. Creating a hosted cluster on bare metal for IBM Z

On bare-metal infrastructure, you can create or import a hosted cluster. After you enable the Assisted Installer as an add-on to multicluster engine Operator and you create a hosted cluster with the Agent platform, the HyperShift Operator installs the Agent Cluster API provider in the hosted control plane namespace. The Agent Cluster API provider connects a management cluster that hosts the control plane and a hosted cluster that consists of only the compute nodes.

Prerequisites

- Each hosted cluster must have a cluster-wide unique name. A hosted cluster name cannot be the same as any existing managed cluster. Otherwise, the multicluster engine Operator cannot manage the hosted cluster.
- Do not use the word **clusters** as a hosted cluster name.
- You cannot create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.
- For best security and management practices, create a hosted cluster separate from other hosted clusters.
- Verify that you have a default storage class configured for your cluster. Otherwise, you might see pending persistent volume claims (PVCs).

Procedure

1. Create a namespace by entering the following command:

```
$ oc create ns <hosted_cluster_namespace>
```

Replace **<hosted_cluster_namespace>** with an identifier for your hosted cluster namespace. The HyperShift Operator creates the namespace. During the hosted cluster creation process on bare-metal infrastructure, a generated Cluster API provider role requires that the namespace already exists.

2. Create the configuration file for your hosted cluster by entering the following command:

```
$ hcp create cluster agent \
  --name=<hosted_cluster_name> \
  --pull-secret=<path_to_pull_secret> \
  --agent-namespace=<hosted_control_plane_namespace> \
  --base-domain=<base_domain> \
  --api-server-address=api.<hosted_cluster_name>.<base_domain> \
  --etcd-storage-class=<etcd_storage_class> \
  --ssh-key=<path_to_ssh_key> \
  --namespace=<hosted_cluster_namespace> \
  --control-plane-availability-policy=HighlyAvailable \
  --release-image=quay.io/openshift-release-dev/ocp-release:<ocp_release_image>-multi \
  --node-pool-replicas=<node_pool_replica_count> \
```

```
--render \
--render-sensitive \
--ssh-key <home_directory>/<path_to_ssh_key>/<ssh_key> > hosted-cluster-config.yaml
```

where:

- **--name** specifies the name of your hosted cluster, such as **example**.
- **--pull-secret** specifies the path to your pull secret, such as **/user/name/pullsecret**.
- **--agent-namespace** specifies your hosted control plane namespace, such as **clusters-example**. Ensure that agents are available in this namespace by using the **oc get agent -n <hosted_control_plane_namespace>** command.
- **--base-domain** specifies your base domain, such as **krnl.es**.
- **--api-server-address** specifies the IP address that gets used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.
- **--etcd-storage-class** specifies the etcd storage class name, such as **lvm-storageclass**.
- **--ssh-key** specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.
- **--namespace** specifies your hosted cluster namespace.
- **--control-plane-availability-policy** specifies the availability policy for the hosted control plane components. Supported options are **SingleReplica** and **HighlyAvailable**. The default value is **HighlyAvailable**.
- **--release-image** specifies the supported OpenShift Container Platform version that you want to use, such as **4.22.0-multi**. If you are using a disconnected environment, replace **<ocp_release_image>** with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".
- **--node-pool-replicas** specifies the node pool replica count, such as **3**. You must specify the replica count as **0** or greater to create the same number of replicas. Otherwise, you do not create node pools.
- **--ssh-key** specifies the path to the SSH key, such as **user/.ssh/id_rsa**.

3. Apply the changes to the hosted cluster configuration file by entering the following command:

```
$ oc apply -f hosted_cluster_config.yaml
```

4. Check for the creation of the hosted cluster, node pools, and pods by entering the following commands:

```
$ oc get hostedcluster \
  <hosted_cluster_name> -n \
  <hosted_cluster_namespace> -o \
  jsonpath='{.status.conditions[?(@.status=="False")]}'| jq .
```

```
$ oc get hostedcluster \
```

```
<nodepool_name> -n \
<hosted_cluster_namespace> -o \
jsonpath='{.status.conditions[?(@.status=="False")]}'
```

```
$ oc get pods -n <hosted_control_plane_namespace>
```

5. Confirm that the hosted cluster is ready. The status of **Available: True** indicates the readiness of the control plane.

Additional resources

- [Manually importing a hosted cluster](#)
- [Extracting the release image digest](#)
- [Creating a hosted cluster on bare metal by using the console](#)

4.6.5. Creating an InfraEnv resource for hosted control planes on IBM Z

Before you can create a hosted cluster, you need an **InfraEnv** resource, which is the environment where hosts that are booted with PXE images can join as agents. In this case, the agents are created in the same namespace as your hosted control plane.

Procedure

1. Create a YAML file to contain the configuration. See the following example:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_control_plane_namespace>
spec:
  cpuArchitecture: s390x
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <ssh_public_key>
```

2. Save the file as **infraenv-config.yaml**.
3. Apply the configuration by entering the following command:

```
$ oc apply -f infraenv-config.yaml
```

4. To fetch the URL to download the PXE or ISO images, such as, **initrd.img**, **kernel.img**, or **rootfs.img**, which allows IBM Z machines to join as agents, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get InfraEnv <hosted_cluster_name> -o json
```

4.6.6. Adding IBM Z KVM as agents

To attach compute nodes to a hosted control plane, create agents that help you to scale the node pool.

Adding agents in an IBM Z environment requires additional steps, which are described in detail in this section.

Unless stated otherwise, this procedure applies to both z/VM and RHEL KVM installations on IBM Z and IBM LinuxONE.

For IBM Z with KVM, run the following command to start your IBM Z environment with the downloaded PXE images from the **InfraEnv** resource. After the Agents are created, the host communicates with the Assisted Service and registers in the same namespace as the **InfraEnv** resource on the management cluster.

Procedure

1. Run the following command:

```
virt-install \
  --name "<vm_name>" \
  --autostart \
  --ram=16384 \
  --cpu host \
  --vcpus=4 \
  --location "<path_to_kernel_initrd_image>,kernel=kernel.img,initrd=initrd.img" \
  --disk <qcow_image_path> \
  --network network:macvtap-net,mac=<mac_address> \
  --graphics none \
  --noautoconsole \
  --wait=-1
  --extra-args "rd.neednet=1 nameserver=<nameserver>
coreos.live.rootfs_url=http://<http_server>/rootfs.img random.trust_cpu=on
rd.luks.options=discard ignition.firstboot ignition.platform.id=metal console=tty1
console=ttyS1,115200n8 coreos.inst.persistent-kargs=console=tty1
console=ttyS1,115200n8"
```

- **--name** specifies the name of the virtual machine.
 - **--location** specifies the location of the **kernel_initrd_image** file.
 - **--disk** specifies the disk image path.
 - **--network** specifies the Mac address.
 - **--extra-args** specifies the server name of the agents.
2. For ISO boot, download ISO from the **InfraEnv** resource and boot the nodes by running the following command:

```
virt-install \
  --name "<vm_name>" \
  --autostart \
  --memory=16384 \
  --cpu host \
  --vcpus=4 \
  --network network:macvtap-net,mac=<mac_address> \
  --cdrom "<path_to_image.iso>" \
  --disk <qcow_image_path> \
  --graphics none \
```

```
--noautoconsole \
--os-variant <os_version> \
--wait=-1
```

- **--name** specifies the name of the virtual machine.
- **--network** specifies the Mac address.
- **--cdrom** specifies the location of the **image.iso** file.
- **--os-variant** specifies the operating system version that you are using.

4.6.7. Adding IBM Z LPAR as agents

To attach compute nodes to a hosted control plane, create agents that help you to scale the node pool.

Adding agents in an IBM Z environment requires additional steps, which are described in detail in this section.

Unless stated otherwise, this procedure applies to both z/VM and RHEL KVM installations on IBM Z and IBM LinuxONE.

You can add the Logical Partition (LPAR) on IBM Z or IBM LinuxONE as a compute node to a hosted control plane.

Procedure

1. Create a boot parameter file for the agents:

Example parameter file

```
rd.neednet=1 cio_ignore=all,!condev \
console=ttysclp0 \
ignition.firstboot ignition.platform.id=metal
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.<architecture>.img \
coreos.inst.persistent-kargs=console=ttysclp0
ip=<ip>::<gateway>:<netmask>::<interface>:none nameserver=<dns> \
rd.znet=qeth,<network_adaptor_range>,layer2=1
rd.<disk_type>=<adaptor> \
zfcp.allow_lun_scan=0
ai.ip_cfg_override=1 \
random.trust_cpu=on rd.luks.options=discard
```

where:

coreos.live.rootfs_url

For the **coreos.live.rootfs_url** artifact, specify the matching **rootfs** artifact for the **kernel** and **initramfs** that you are starting. Only HTTP and HTTPS protocols are supported.

ip

For the **ip** parameter, manually assign the IP address, as described in "Installing a cluster with z/VM on IBM Z and IBM LinuxONE".

rd

For installations on DASD-type disks, use **rd.dasd** to specify the DASD where Red Hat Enterprise Linux CoreOS (RHCOS) is to be installed. For installations on FCP-type disks, use **rd.zfcp=<adapter>,<wwpn>,<lun>** to specify the FCP disk where RHCOS is to be installed.

ai.ip_cfg_override

Specify this parameter when you use an Open Systems Adapter (OSA) or HiperSockets.

2. Download the **.ins** and **initrd.img.addrsz** files from the **InfraEnv** resource. By default, the URL for the **.ins** and **initrd.img.addrsz** files is not available in the **InfraEnv** resource. You must edit the URL to fetch those artifacts.
 - a. Update the kernel URL endpoint to include **ins-file** by running the followign command:

```
$ curl -k -L -o generic.ins "< url for ins-file >"
```

Example URL

```
https://.../boot-artifacts/ins-file?arch=s390x&version=4.17.0
```

- b. Update the **initrd** URL endpoint to include **s390x-initrd-addrsz**:

Example URL

```
https://.../s390x-initrd-addrsz?api_key=<api-key>&arch=s390x&version=4.17.0
```

3. Transfer the **initrd**, **kernel**, **generic.ins**, and **initrd.img.addrsz** parameter files to the file server. For more information about how to transfer the files with FTP and boot, see "Installing in an LPAR".
4. Start the machine.
5. Repeat the procedure for all other machines in the cluster.

Additional resources

- [Installing in an LPAR](#)

4.6.7.1. Adding IBM z/VM as agents

If you want to use a static IP for z/VM guest, you must configure the **NMStateConfig** attribute for the z/VM agent so that the IP parameter persists in the second start.

Complete the following steps to start your IBM Z environment with the downloaded PXE images from the **InfraEnv** resource. After the Agents are created, the host communicates with the Assisted Service and registers in the same namespace as the **InfraEnv** resource on the management cluster.

Procedure

1. Update the parameter file to add the **rootfs_url**, **network_adaptor** and **disk_type** values.

Example parameter file

```
rd.neednet=1 cio_ignore=all,!condev \
console=ttySCLP0 \
```

```

ignition.firstboot ignition.platform.id=metal \
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.<architecture>.img \
coreos.inst.persistent-kargs=console=ttysclp0
ip=<ip>::<gateway>:<netmask>::<interface>:none nameserver=<dns> \
rd.znet=qeth,<network_adaptor_range>,layer2=1
rd.<disk_type>=<adapter> \
zfcplib.allow_lun_scan=0
ai.ip_cfg_override=1 \

```

where:

coreos.live.rootfs_url

For the **coreos.live.rootfs_url** artifact, specify the matching **rootfs** artifact for the **kernel** and **initramfs** that you are starting. Only HTTP and HTTPS protocols are supported.

ip

For the **ip** parameter, manually assign the IP address, as described in "Installing a cluster with z/VM on IBM Z and IBM LinuxONE".

rd

For installations on DASD-type disks, use **rd.dasd** to specify the DASD where Red Hat Enterprise Linux CoreOS (RHCOS) is to be installed. For installations on FCP-type disks, use **rd.zfcplib=<adapter>,<wwpn>,<lun>** to specify the FCP disk where RHCOS is to be installed.



NOTE

For FCP multipath configurations, provide two disks instead of one.

Example

```

rd.zfcplib=<adapter1>,<wwpn1>,<lun1> \
rd.zfcplib=<adapter2>,<wwpn2>,<lun2>

```

ai.ip_cfg_override

Specify this parameter when you use an Open Systems Adapter (OSA) or HiperSockets.

2. Move **initrd**, kernel images, and the parameter file to the guest VM by running the following commands:

```
vmur pun -r -u -N kernel.img $INSTALLERKERNELLOCATION/<image name>
```

```
vmur pun -r -u -N generic.parm $PARMFILELOCATION/paramfilename
```

```
vmur pun -r -u -N initrd.img $INSTALLERINITRAMFSLOCATION/<image name>
```

3. Run the following command from the guest VM console:

```
cp ipl c
```

4. To list the agents and their properties, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

■

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
50c23cda-cedc-9bbd-bcf1-9b3a5c75804d				auto-assign
5e498cd3-542c-e54f-0c58-ed43e28b568a				auto-assign

- Run the following command to approve the agent.

```
$ oc -n <hosted_control_plane_namespace> patch agent \
  50c23cda-cedc-9bbd-bcf1-9b3a5c75804d -p \
  '{"spec":{"installation_disk_id":"/dev/sda","approved":true,"hostname":"worker-zvm-
  0.hostedn.example.com"}}' \
  --type merge
```

Optionally, you can set the agent ID **<installation_disk_id>** and **<hostname>** in the specification.

- Run the following command to verify that the agents are approved:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
50c23cda-cedc-9bbd-bcf1-9b3a5c75804d		true	auto-assign	
5e498cd3-542c-e54f-0c58-ed43e28b568a		true	auto-assign	

4.6.8. Scaling the NodePool object for a hosted cluster on IBM Z

The **NodePool** object is created when you create a hosted cluster. By scaling the **NodePool** object, you can add more compute nodes to the hosted control plane.

When you scale up a node pool, a machine is created. The Cluster API provider finds an Agent that is approved, is passing validations, is not currently in use, and meets the requirements that are specified in the node pool specification. You can monitor the installation of an Agent by checking its status and conditions.

Procedure

- Run the following command to scale the **NodePool** object to two nodes:

```
$ oc -n <clusters_namespace> scale nodepool <nodepool_name> --replicas 2
```

The Cluster API agent provider randomly picks two agents that are then assigned to the hosted cluster. Those agents go through different states and finally join the hosted cluster as OpenShift Container Platform nodes. The agents pass through the transition phases in the following order:

- **binding**
- **discovering**

- **insufficient**
- **installing**
- **installing-in-progress**
- **added-to-existing-cluster**

2. Run the following command to see the status of a specific scaled agent:

```
$ oc -n <hosted_control_plane_namespace> get agent -o \
  jsonpath='{range .items[*]}BMH: {@.metadata.labels.agent-install.openshift.io/bmh} \
  Agent: {@.metadata.name} State: {@.status.debugInfo.state}{""\n}{end}'
```

Example output

```
BMH: Agent: 50c23cda-cedc-9bbd-bcf1-9b3a5c75804d State: known-unbound
BMH: Agent: 5e498cd3-542c-e54f-0c58-ed43e28b568a State: insufficient
```

3. Run the following command to see the transition phases:

```
$ oc -n <hosted_control_plane_namespace> get agent
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
50c23cda-cedc-9bbd-bcf1-9b3a5c75804d		hosted-forwarder	true	auto-assign
5e498cd3-542c-e54f-0c58-ed43e28b568a		true	auto-assign	
da503cf1-a347-44f2-875c-4960ddb04091		hosted-forwarder	true	auto-assign

4. Run the following command to generate the **kubeconfig** file to access the hosted cluster:

```
$ hcp create kubeconfig \
  --namespace <clusters_namespace> \
  --name <hosted_cluster_namespace> > <hosted_cluster_name>.kubeconfig
```

5. After the agents reach the **added-to-existing-cluster** state, verify that you can see the OpenShift Container Platform nodes by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
worker-zvm-0.hostedn.example.com	Ready	worker	5m41s	v1.24.0+3882f8f
worker-zvm-1.hostedn.example.com	Ready	worker	6m3s	v1.24.0+3882f8f

Cluster Operators start to reconcile by adding workloads to the nodes.

6. Enter the following command to verify that two machines were created when you scaled up the **NodePool** object:

```
$ oc -n <hosted_control_plane_namespace> get machine.cluster.x-k8s.io
```

Example output

NAME	CLUSTER	NODENAME	PROVIDERID	PHASE	AGE
VERSION					
hosted-forwarder-79558597ff-5tbqp	hosted-forwarder-crqq5	worker-zvm-0.hostedn.example.com	agent://50c23cda-cedc-9bbd-bcf1-9b3a5c75804d	Running	41h
4.15.0					
hosted-forwarder-79558597ff-lfjfk	hosted-forwarder-crqq5	worker-zvm-1.hostedn.example.com	agent://5e498cd3-542c-e54f-0c58-ed43e28b568a	Running	41h
4.15.0					

- Run the following command to check the cluster version:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusterversion,co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE
STATUS				
clusterversion.config.openshift.io/version	4.15.0-ec.2	True	False	40h
version is 4.15.0-ec.2				Cluster

- Run the following command to check the cluster operator status:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusteroperators
```

For each component of your cluster, the output shows the following cluster operator statuses: **NAME**, **VERSION**, **AVAILABLE**, **PROGRESSING**, **DEGRADED**, **SINCE**, and **MESSAGE**.

For an output example, see "Initial Operator configuration".

4.7. DEPLOYING HOSTED CONTROL PLANES ON IBM POWER

You can deploy hosted control planes on IBM Power by configuring a cluster to function as a hosting cluster. This configuration provides an efficient and scalable solution for managing many clusters. The hosting cluster is an OpenShift Container Platform cluster that hosts control planes. The hosting cluster is also known as the *management* cluster.



NOTE

The *management* cluster is not the *managed* cluster. A managed cluster is a cluster that the hub cluster manages.

The multicluster engine Operator supports only the default **local-cluster**, which is a managed hub cluster, and the hub cluster as the hosting cluster.

To provision hosted control planes on bare-metal infrastructure, you can use the Agent platform. The Agent platform uses the central infrastructure management service to add compute nodes to a hosted cluster. For more information, see "Enabling the central infrastructure management service".

You must start each IBM Power host with a Discovery image that the central infrastructure management provides. After each host starts, it runs an Agent process to discover the details of the host and completes the installation. An Agent custom resource represents each host.

When you create a hosted cluster with the Agent platform, HyperShift installs the Agent Cluster API provider in the hosted control plane namespace.

4.7.1. Prerequisites to configure hosted control planes on IBM Power

Ensure you meet the prerequisites to configure hosted control planes on IBM Power.

- The multicluster engine for Kubernetes Operator version 2.7 and later installed on an OpenShift Container Platform cluster. The multicluster engine Operator is automatically installed when you install Red Hat Advanced Cluster Management (RHACM). You can also install the multicluster engine Operator without RHACM as an Operator from the OpenShift Container Platform software catalog.
- The multicluster engine Operator must have at least one managed OpenShift Container Platform cluster. The **local-cluster** managed hub cluster is automatically imported in the multicluster engine Operator version 2.7 and later. For more information about **local-cluster**, see *Advanced configuration* in the RHACM documentation. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- You need a hosting cluster with at least 3 compute nodes to run the HyperShift Operator.
- You need to enable the central infrastructure management service. For more information, see "Enabling the central infrastructure management service".
- You need to install the hosted control planes command-line interface. For more information, see "Installing the hosted control plane command-line interface".

The hosted control planes feature is enabled by default. If you disabled the feature and want to manually enable the feature, see "Manually enabling the hosted control planes feature". If you need to disable the feature, see "Disabling the hosted control planes feature".

Additional resources

- [Advanced configuration](#)
- [Enabling the central infrastructure management service](#)
- [Installing the hosted control plane command-line interface](#)
- [Manually enabling the hosted control planes feature](#)
- [Disabling the hosted control planes feature](#)

4.7.2. IBM Power infrastructure requirements

The Agent platform does not create any infrastructure, but requires resources for infrastructure.

- Agents: An *Agent* represents a host that boots with a Discovery image and that you can provision as an OpenShift Container Platform node.

- DNS: The API and Ingress endpoints must be routable.

4.7.3. DNS configuration for hosted control planes on IBM Power

Clients outside the network can access the API server for the hosted cluster. A DNS entry must exist for the **api.<hosted_cluster_name>.<basedomain>** entry that points to the destination where the API server is reachable.

The DNS entry can be as simple as a record that points to one of the nodes in the managed cluster that runs the hosted control plane.

The entry can also point to a deployed load balancer to redirect incoming traffic to the ingress pods.

See the following example of a DNS configuration:

```
$ cat /var/named/<example.krnl.es.zone>
```

Example output

```
$ TTL 900
@ IN SOA bastion.example.krnl.es.com. hostmaster.example.krnl.es.com. (
    2019062002
    1D 1H 1W 3H )
IN NS bastion.example.krnl.es.com.
;
;
;
api IN A 1xx.2x.2xx.1xx
api-int IN A 1xx.2x.2xx.1xx
;
;
;
*.apps.<hosted_cluster_name>.<basedomain> IN A 1xx.2x.2xx.1xx
;
;EOF
```

The **api** record refers to the IP address of the API load balancer that handles ingress and egress traffic for hosted control planes.

For IBM Power, add IP addresses that correspond to the IP address of the agent.

Example configuration

```
compute-0 IN A 1xx.2x.2xx.1yy
compute-1 IN A 1xx.2x.2xx.1yy
```

4.7.3.1. Defining a custom DNS name

As a cluster administrator, you can create a hosted cluster with an external API DNS name that differs from the internal endpoint that gets used for node bootstraps and control plane communication.

You might want to define a different DNS name for the following reasons:

- To replace the user-facing TLS certificate with one from a public CA without breaking the control plane functions that bind to the internal root CA

- To support split-horizon DNS and NAT scenarios
- To ensure a similar experience to standalone control planes, where you can use functions, such as the **Show Login Command** function, with the correct **kubeconfig** and DNS configuration

You can define a DNS name either during your initial setup or during postinstallation operations, by entering a domain name in the **kubeAPIServerDNSName** parameter of a **HostedCluster** object.

Prerequisites

- You have a valid TLS certificate that covers the DNS name that you set in the **kubeAPIServerDNSName** parameter.
- You have a resolvable DNS name URI that can reach and point to the correct address.

Procedure

- In the specification for the **HostedCluster** object, add the **kubeAPIServerDNSName** parameter and the address for the domain and specify which certificate to use, as shown in the following example:

```
#...
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - xxx.example.com
            - yyy.example.com
          servingCertificate:
            name: <my_serving_certificate>
        kubeAPIServerDNSName: <custom_address>
```

The value for the **kubeAPIServerDNSName** parameter must be a valid and addressable domain.

After you define the **kubeAPIServerDNSName** parameter and specify the certificate, the Control Plane Operator controllers create a **kubeconfig** file named **custom-admin-kubeconfig**, where the file gets stored in the **HostedControlPlane** namespace. The generation of certificates happen from the root CA, and the **HostedControlPlane** namespace manages their expiration and renewal.

The Control Plane Operator reports a new **kubeconfig** file named **CustomKubeconfig** in the **HostedControlPlane** namespace. That file uses the defined new server in the **kubeAPIServerDNSName** parameter.

A reference for the custom **kubeconfig** file exists in the **status** parameter as **CustomKubeconfig** of the **HostedCluster** object. The **CustomKubeConfig** parameter is optional, and you can add the parameter only if the **kubeAPIServerDNSName** parameter is not empty. After you set the **CustomKubeConfig** parameter, the parameter triggers the generation of a secret named **<hosted_cluster_name>-custom-admin-kubeconfig** in the

HostedCluster namespace. You can use the secret to access the **HostedCluster** API server. If you remove the **CustomKubeConfig** parameter during postinstallation operations, deletion of all related secrets and status references occur.



NOTE

Defining a custom DNS name does not directly impact the data plane, so no expected rollouts occur. The **HostedControlPlane** namespace receives the changes from the HyperShift Operator and deletes the corresponding parameters.

If you remove the **kubeAPIServerDNSName** parameter from the specification for the **HostedCluster** object, all newly generated secrets and the **CustomKubeconfig** reference are removed from the cluster and from the **status** parameter.

4.7.4. Creating a hosted cluster by using the CLI

On bare-metal infrastructure, you can import a hosted cluster or create one by using the command-line interface (CLI).

After you enable the Assisted Installer as an add-on to multicluster engine Operator and you create a hosted cluster with the Agent platform, the HyperShift Operator installs the Agent Cluster API provider in the hosted control plane namespace. The Agent Cluster API provider connects a management cluster that hosts the control plane and a hosted cluster that consists of only the compute nodes.

Prerequisites

- Each hosted cluster must have a cluster-wide unique name. A hosted cluster name cannot be the same as any existing managed cluster. Otherwise, the multicluster engine Operator cannot manage the hosted cluster.
- Do not use the word **clusters** as a hosted cluster name.
- You cannot create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.
- For best security and management practices, create a hosted cluster separate from other hosted clusters.
- Verify that you have a default storage class configured for your cluster. Otherwise, you might see pending persistent volume claims (PVCs).
- By default when you use the **hcp create cluster agent** command, the command creates a hosted cluster with configured node ports. The preferred publishing strategy for hosted clusters on bare metal exposes services through a load balancer. If you create a hosted cluster by using the web console or by using Red Hat Advanced Cluster Management, to set a publishing strategy for a service besides the Kubernetes API server, you must manually specify the **servicePublishingStrategy** information in the **HostedCluster** custom resource.
- Ensure that you meet the requirements described in "Requirements for hosted control planes on bare metal", which includes requirements related to infrastructure, firewalls, ports, and services. For example, those requirements describe how to add the appropriate zone labels to the bare-metal hosts in your management cluster, as shown in the following example commands:

```
$ oc label node [compute-node-1] topology.kubernetes.io/zone=zone1
```

```
$ oc label node [compute-node-2] topology.kubernetes.io/zone=zone2
```

```
$ oc label node [compute-node-3] topology.kubernetes.io/zone=zone3
```

- Ensure that you have added bare-metal nodes to a hardware inventory.

Procedure

1. Create a namespace by entering the following command:

```
$ oc create ns <hosted_cluster_namespace>
```

Replace **<hosted_cluster_namespace>** with an identifier for your hosted cluster namespace. The HyperShift Operator creates the namespace. During the hosted cluster creation process on bare-metal infrastructure, a generated Cluster API provider role requires that the namespace already exists.

2. Create the configuration file for your hosted cluster by entering the following command:

```
$ hcp create cluster agent \
  --name=<hosted_cluster_name> \
  --pull-secret=<path_to_pull_secret> \
  --agent-namespace=<hosted_control_plane_namespace> \
  --base-domain=<base_domain> \
  --api-server-address=api.<hosted_cluster_name>.<base_domain> \
  --etcd-storage-class=<etcd_storage_class> \
  --ssh-key=<path_to_ssh_key> \
  --namespace=<hosted_cluster_namespace> \
  --control-plane-availability-policy=HighlyAvailable \
  --release-image=quay.io/openshift-release-dev/ocp-release:<ocp_release_image>-multi \
  --node-pool-replicas=<node_pool_replica_count> \
  --render \
  --render-sensitive > hosted-cluster-config.yaml
```

where:

- **--name** specifies the name of your hosted cluster, such as **example**.
- **--pull-secret** specifies the path to your pull secret, such as **/user/name/pullsecret**.
- **--agent-namespace** specifies your hosted control plane namespace, such as **clusters-example**. Ensure that agents are available in this namespace by using the **oc get agent -n <hosted_control_plane_namespace>** command.
- **--base-domain** specifies your base domain, such as **krnl.es**.
- **--api-server-address** specifies the IP address that gets used for the Kubernetes API communication in the hosted cluster. If you do not set the **--api-server-address** flag, you must log in to connect to the management cluster.
- **--etcd-storage-class** specifies the etcd storage class name, such as **lvm-storageclass**.

- **--ssh-key** specifies the path to your SSH public key. The default file path is `~/.ssh/id_rsa.pub`.
 - **--namespace** specifies your hosted cluster namespace.
 - **--control-plane-availability-policy** specifies the availability policy for the hosted control plane components. Supported options are **SingleReplica** and **HighlyAvailable**. The default value is **HighlyAvailable**.
 - **--release-image** specifies the supported OpenShift Container Platform version that you want to use, such as **4.22.0-multi**. If you are using a disconnected environment, replace `<ocp_release_image>` with the digest image. To extract the OpenShift Container Platform release image digest, see "Extracting the release image digest".
 - **--node-pool-replicas** specifies the node pool replica count, such as **3**. You must specify the replica count as **0** or greater to create the same number of replicas. Otherwise, you do not create node pools.
3. Configure the service publishing strategy. By default, hosted clusters use the **NodePort** service publishing strategy because node ports are always available without additional infrastructure. However, you can configure the service publishing strategy to use a load balancer.
- If you are using the default **NodePort** strategy, configure the DNS to point to the hosted cluster compute nodes, not the management cluster nodes. For more information, see "DNS configurations on bare metal".
 - For production environments, use the **LoadBalancer** strategy because this strategy provides certificate handling and automatic DNS resolution. The following example demonstrates changing the service publishing **LoadBalancer** strategy in your hosted cluster configuration file:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  # ...
spec:
  services:
    - service: APIServer
      servicePublishingStrategy:
        type: LoadBalancer
    - service: Ignition
      servicePublishingStrategy:
        type: Route
    - service: Konnectivity
      servicePublishingStrategy:
        type: Route
    - service: OAuthServer
      servicePublishingStrategy:
        type: Route
    - service: OIDC
      servicePublishingStrategy:
        type: Route
  sshKey:
    name: <ssh_key>
  # ...
```

Specify **LoadBalancer** as the API Server type. For all other services, specify **Route** as the type.

4. If you use external load balancers, configure the ingress endpoint as shown in the following example. If you do not configure the endpoint, the default behavior is to randomize the node port that the service exposes the ingress on. To configure how the ingress controller publishes the default ingress route, set the **endpointPublishingStrategy** parameter and its underlying functions by editing the **HostedCluster** resource:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  #...
spec:
  operatorConfiguration:
    ingressOperator:
      endpointPublishingStrategy:
        hostNetwork:
          httpPort: 80
          httpsPort: 443
          protocol: TCP
          statsPort: 1936
        type: HostNetwork
  #...
```

The **spec.operatorConfiguration.ingressOperator.endPointPublishingStrategy.type** parameter specifies the endpoint for the load balancer. For bare-metal installations, use the **HostNetwork** type.

5. Apply the changes to the hosted cluster configuration file by entering the following command:

```
$ oc apply -f hosted_cluster_config.yaml
```

6. Check for the creation of the hosted cluster, node pools, and pods by entering the following commands:

```
$ oc get hostedcluster \
  <hosted_cluster_namespace> -n \
  <hosted_cluster_namespace> -o \
  jsonpath='{.status.conditions[?(@.status=="False")]} ' | jq .
```

```
$ oc get nodepool \
  <hosted_cluster_namespace> -n \
  <hosted_cluster_namespace> -o \
  jsonpath='{.status.conditions[?(@.status=="False")]} ' | jq .
```

```
$ oc get pods -n <hosted_cluster_namespace>
```

7. Confirm that the hosted cluster is ready. The status of **Available: True** indicates the readiness of the cluster and the node pool status shows **AllMachinesReady: True**. These statuses indicate the healthiness of all cluster Operators.
8. Install MetalLB in the hosted cluster:

- a. Extract the **kubeconfig** file from the hosted cluster and set the environment variable for hosted cluster access by entering the following commands:

```
$ oc get secret \
  <hosted_cluster_namespace>-admin-kubeconfig \
  -n <hosted_cluster_namespace> \
  -o jsonpath='{.data.kubeconfig}' \
  | base64 -d > \
  kubeconfig-<hosted_cluster_namespace>.yaml
```

```
$ export KUBECONFIG="/path/to/kubeconfig-<hosted_cluster_namespace>.yaml"
```

- b. Install the MetalLB Operator by creating the **install-metallb-operator.yaml** file:

```
apiVersion: v1
kind: Namespace
metadata:
  name: metallb-system
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator
  namespace: metallb-system
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: metallb-operator
  namespace: metallb-system
spec:
  channel: "stable"
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
# ...
```

- c. Apply the file by entering the following command:

```
$ oc apply -f install-metallb-operator.yaml
```

- d. Configure the MetalLB IP address pool by creating the **deploy-metallb-ipaddresspool.yaml** file:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: metallb
  namespace: metallb-system
spec:
  autoAssign: true
  addresses:
    - 10.11.176.71-10.11.176.75
```

```

---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2advertisement
  namespace: metallb-system
spec:
  ipAddressPools:
  - metallb
# ...

```

- e. Apply the configuration by entering the following command:

```
$ oc apply -f deploy-metallb-ipaddresspool.yaml
```

- f. Verify the installation of MetalLB by checking the Operator status, the IP address pool, and the **L2Advertisement** resource by entering the following commands:

```
$ oc get pods -n metallb-system
```

```
$ oc get ipaddresspool -n metallb-system
```

```
$ oc get l2advertisement -n metallb-system
```

9. Configure the load balancer for ingress:

- a. Create the **ingress-loadbalancer.yaml** file:

```

apiVersion: v1
kind: Service
metadata:
  annotations:
    metallb.universe.tf/address-pool: metallb
  name: metallb-ingress
  namespace: openshift-ingress
spec:
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
  - name: https
    protocol: TCP
    port: 443
    targetPort: 443
  selector:
    ingresscontroller.operator.openshift.io/deployment-ingresscontroller: default
  type: LoadBalancer
# ...

```

- b. Apply the configuration by entering the following command:

```
$ oc apply -f ingress-loadbalancer.yaml
```


- c. Verify that the load balancer service works as expected by entering the following command:

```
$ oc get svc metallb-ingress -n openshift-ingress
```

Example output

```
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)
AGE
metallb-ingress LoadBalancer  172.31.127.129 10.11.176.71
80:30961/TCP,443:32090/TCP 16h
```

10. Configure the DNS to work with the load balancer:

- a. Configure the DNS for the **apps** domain by pointing the ***.apps.**
<hosted_cluster_namespace>.<base_domain> wildcard DNS record to the load balancer IP address.
- b. Verify the DNS resolution by entering the following command:

```
$ nslookup console-openshift-console.apps.<hosted_cluster_namespace>.  
<base_domain> <load_balancer_ip_address>
```

Example output

```
Server:      10.11.176.1
Address:     10.11.176.1#53

Name:   console-openshift-console.apps.my-hosted-cluster.sample-base-domain.com
Address: 10.11.176.71
```

Verification

1. Check the cluster Operators by entering the following command:

```
$ oc get clusteroperators
```

Ensure that all Operators show **AVAILABLE: True**, **PROGRESSING: False**, and **DEGRADED: False**.

2. Check the nodes by entering the following command:

```
$ oc get nodes
```

Ensure that each node has the **READY** status.

3. Test access to the console by entering the following URL in a web browser:

```
https://console-openshift-console.apps.<hosted_cluster_namespace>.<base_domain>
```

Additional resources

- [Requirements for hosted control planes](#)

- [DNS configurations on bare metal](#)
- [Manually importing a hosted cluster](#)
- [Extracting the release image digest](#)

4.7.5. About creating heterogeneous node pools on agent hosted clusters

A node pool is a group of nodes within a cluster that share the same configuration. Heterogeneous node pools have different configurations so that you can create pools and optimize them for various workloads.

You can create heterogeneous node pools on the agent platform. The platform enables clusters to run diverse machine types, such as **x86_64** or **ppc64le**, within a single hosted cluster.

Creating a heterogeneous node pool requires completion of the following general steps:

- Create an **AgentServiceConfig** custom resource (CR) that informs the Operator how much storage it needs for components such as the database and filesystem. The CR also defines which OpenShift Container Platform versions to support.
- Create an agent cluster.
- Create the heterogeneous node pool.
- Configure DNS for hosted control planes
- Create an **InfraEnv** custom resource (CR) for each architecture.
- Add agents to the heterogeneous cluster.

4.7.5.1. Creating the AgentServiceConfig custom resource

To create heterogeneous node pools on an agent hosted cluster, you need to create the **AgentServiceConfig** CR with two heterogeneous architecture operating system (OS) images.

Procedure

- Run the following command:

```
$ envsubst <<"EOF" | oc apply -f -
apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  name: agent
spec:
  databaseStorage:
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: <db_volume_name>
  filesystemStorage:
    accessModes:
      - ReadWriteOnce
  resources:
```

```

    requests:
      storage: <fs_volume_name>
    osImages:
      - openshiftVersion: <ocp_version>
        version: <ocp_release_version_x86>
        url: <iso_url_x86>
        rootFSUrl: <root_fs_url_x86>
        cpuArchitecture: <arch_x86>
      - openshiftVersion: <ocp_version>
        version: <ocp_release_version_ppc64le>
        url: <iso_url_ppc64le>
        rootFSUrl: <root_fs_url_ppc64le>
        cpuArchitecture: <arch_ppc64le>
  EOF

```

where:

<db_volume_name>

Specifies the multicluster engine for Kubernetes Operator **agentserviceconfig** config, database volume name.

<fs_volume_name>

Specifies the multicluster engine Operator **agentserviceconfig** config, filesystem volume name.

<ocp_version>

Specifies the current version of OpenShift Container Platform.

<ocp_release_version_x86>

Specifies the current OpenShift Container Platform release version for x86.

<iso_url_x86>

Specifies the ISO URL for x86.

<root_fs_url_x86>

Specifies the root filesystem URL for x86.

<arch_x86>

Specifies the CPU architecture for x86.

<ocp_release_version_ppc64le>

Specifies the OpenShift Container Platform release version for **ppc64le**.

<iso_url_ppc64le>

Specifies the ISO URL for **ppc64le**.

<root_fs_url_ppc64le>

Specifies the root filesystem URL for **ppc64le**.

<arch_ppc64le>

Specifies the CPU architecture for **ppc64le**.

4.7.5.2. Create an agent cluster

An agent-based approach manages and provisions an agent cluster. An agent cluster can use heterogeneous node pools, allowing the use of different types of compute nodes within the same cluster.

Prerequisites

- You used a multi-architecture release image to enable support for heterogeneous node pools when creating a hosted cluster. Find the latest multi-architecture images on the "Multi-arch release images" page.

Procedure

1. Create an environment variable for the cluster namespace by running the following command:

```
$ export CLUSTERS_NAMESPACE=<hosted_cluster_namespace>
```

2. Create an environment variable for the machine classless inter-domain routing (CIDR) notation by running the following command:

```
$ export MACHINE_CIDR=192.168.122.0/24
```

3. Create the hosted control namespace by running the following command:

```
$ oc create ns <hosted_control_plane_namespace>
```

4. Create the cluster by running the following command:

```
$ hcp create cluster agent \  
  --name=<hosted_cluster_name> \  
  --pull-secret=<pull_secret_file> \  
  --agent-namespace=<hosted_control_plane_namespace> \  
  --base-domain=<base_domain> \  
  --api-server-address=api.<hosted_cluster_name>.<basedomain> \  
  --release-image=quay.io/openshift-release-dev/ocp-release:<ocp_release>
```

where:

<hosted_cluster_name>

Specifies the hosted cluster name.

<pull_secret_file>

Specifies the pull secret file path.

<hosted_control_plane_namespace>

Specifies the namespace for the hosted control plane.

<base_domain>

Specifies the base domain for the hosted cluster.

<ocp_release>

Specifies the current OpenShift Container Platform release version.

Additional resources

- [Multi-arch release images](#)

4.7.5.3. Creating heterogeneous node pools

You create heterogeneous node pools by using the **NodePool** custom resource (CR), so that you can optimize costs and performance by associating different workloads to specific hardware.

Procedure

- To define a **NodePool** CR, create a YAML file similar to the following example:

```
envsubst <<"EOF" | oc apply -f -
apiVersion: apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: <hosted_cluster_name>
  namespace: <clusters_namespace>
spec:
  arch: <arch_ppc64le>
  clusterName: <hosted_cluster_name>
  management:
    autoRepair: false
    upgradeType: InPlace
    nodeDrainTimeout: 0s
    nodeVolumeDetachTimeout: 0s
  platform:
    agent:
      agentLabelSelector:
        matchLabels:
          inventory.agent-install.openshift.io/cpu-architecture: <arch_ppc64le>
      type: Agent
  release:
    image: quay.io/openshift-release-dev/ocp-release:<ocp_release>
  replicas: 0
EOF
```

<arch_ppc64le>: The selector block selects the agents that match the specified label. To create a node pool of architecture **ppc64le** with zero replicas, specify **ppc64le**. This ensures that the selector block selects only agents from **ppc64le** architecture during a scaling operation.

4.7.5.4. DNS configuration for hosted control planes

A Domain Name Service (DNS) configuration for hosted control planes means that external clients can reach ingress controllers, so that the clients can route traffic to internal components. Configuring this setting ensures that traffic gets routed to either a **ppc64le** or an **x86_64** compute node.

You can point an ***.apps.<cluster_name>** record to either of the compute nodes that hosts the ingress application. Or, if you can set up a load balancer on top of the compute nodes, point the record to this load balancer. When you are creating a heterogeneous node pool, make sure the compute nodes can reach each other or keep them in the same network.

4.7.5.5. Creating infrastructure environment resources

For heterogeneous node pools, you must create an **infraEnv** custom resource (CR) for each architecture. This configuration ensures that the correct architecture-specific operating system and boot artifacts get used during the node provisioning process.

For example, for node pools with **x86_64** and **ppc64le** architectures, create an **InfraEnv** CR for **x86_64** and **ppc64le**.



NOTE

Before starting the procedure, ensure that you add the operating system images for both **x86_64** and **ppc64le** architectures to the **AgentServiceConfig** resource. After this, you can use the **InfraEnv** resources to get the minimal ISO image.

Procedure

1. Create the **InfraEnv** resource with **x86_64** architecture for heterogeneous node pools by running the following command:

```
$ envsubst <<"EOF" | oc apply -f -
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: <hosted_cluster_name>-<arch_x86>
  namespace: <hosted_control_plane_namespace>
spec:
  cpuArchitecture: <arch_x86>
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <ssh_pub_key>
EOF
```

where:

<hosted_cluster_name>

Specifies the hosted cluster name.

<arch_x86>

Specifies the **x86_64** architecture.

<hosted_control_plane_namespace>

Specifies the hosted control plane namespace.

<ssh_pub_key>

Specifies the SSH public key.

2. Create the **InfraEnv** resource with **ppc64le** architecture for heterogeneous node pools by running the following command:

```
envsubst <<"EOF" | oc apply -f -
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: <hosted_cluster_name>-<arch_ppc64le>
  namespace: <hosted_control_plane_namespace>
spec:
  cpuArchitecture: <arch_ppc64le>
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <ssh_pub_key>
EOF
```

where:

<hosted_cluster_name>

Specifies the hosted cluster name.

<arch_ppc64le>

Specifies the **ppc64le** architecture.

<hosted_control_plane_namespace>

Specifies the hosted control plane namespace.

<ssh_pub_key>

Specifies the SSH public key.

3. Verify the successful creation of the **InfraEnv** resources by running the following commands:

- Verify the successful creation of the **x86_64 InfraEnv** resource:

```
$ oc describe InfraEnv <hosted_cluster_name>-<arch_x86>
```

- Verify the successful creation of the **ppc64le InfraEnv** resource:

```
$ oc describe InfraEnv <hosted_cluster_name>-<arch_ppc64le>
```

4. Generate a live ISO that allows either a virtual machine or a bare-metal machine to join as agents by running the following commands:

- a. Generate a live ISO for **x86_64**:

```
$ oc -n <hosted_control_plane_namespace> get InfraEnv <hosted_cluster_name>-<arch_x86> -ojsonpath="{.status.isoDownloadURL}"
```

- b. Generate a live ISO for **ppc64le**:

```
$ oc -n <hosted_control_plane_namespace> get InfraEnv <hosted_cluster_name>-<arch_ppc64le> -ojsonpath="{.status.isoDownloadURL}"
```

4.7.5.6. Adding agents to the heterogeneous cluster

You add agents by manually configuring the machine to boot with a live ISO. You can download the live ISO and use it to boot a bare-metal node or a virtual machine.

On boot, the node communicates with the **assisted-service** and registers as an agent in the same namespace as the **InfraEnv** resource. After the creation of each agent, you can optionally set its **installation_disk_id** and **hostname** parameters in the specifications. You can then approve the agent to indicate the agent as ready for use.

Procedure

1. Obtain a list of agents by running the following command:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
86f7ac75-4fc4-4b36-8130-40fa12602218			auto-assign	
e57a637f-745b-496e-971d-1abbf03341ba			auto-assign	

2. Patch an agent by running the following command:

```
$ oc -n <hosted_control_plane_namespace> patch agent 86f7ac75-4fc4-4b36-8130-40fa12602218 -p '{"spec": {"installation_disk_id":"/dev/sda","approved":true,"hostname":"worker-0.example.krnl.es"}}' --type merge
```

3. Patch the second agent by running the following command:

```
$ oc -n <hosted_control_plane_namespace> patch agent 23d0c614-2caa-43f5-b7d3-0b3564688baa -p '{"spec": {"installation_disk_id":"/dev/sda","approved":true,"hostname":"worker-1.example.krnl.es"}}' --type merge
```

4. Check the agent approval status by running the following command:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
86f7ac75-4fc4-4b36-8130-40fa12602218		true	auto-assign	
e57a637f-745b-496e-971d-1abbf03341ba		true	auto-assign	

4.7.5.7. Scaling the node pool

After you approve your agents, you can scale the node pools. The **agentLabelSelector** value that you configured in the node pool ensures that only matching agents get added to the cluster. This also helps scale down the node pool.

To remove specific architecture nodes from the cluster, scale down the corresponding node pool.

Procedure

- Scale the node pool by running the following command:

```
$ oc -n <clusters_namespace> scale nodepool <nodepool_name> --replicas 2
```



NOTE

The Cluster API agent provider picks two agents randomly to assign to the hosted cluster. These agents pass through different states and then join the hosted cluster as OpenShift Container Platform nodes. The various agent states are **binding**, **discovering**, **insufficient**, **installing**, **installing-in-progress**, and **added-to-existing-cluster**.

Verification

1. List the agents by running the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent
```

Example output

```
NAME                                CLUSTER      APPROVED  ROLE    STAGE
4dac1ab2-7dd5-4894-a220-6a3473b67ee6  hypercluster1  true     auto-assign
d9198891-39f4-4930-a679-65fb142b108b           true     auto-assign
da503cf1-a347-44f2-875c-4960ddb04091  hypercluster1  true     auto-assign
```

2. Check the status of a specific scaled agent by running the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent -o jsonpath='{range .items[*]}BMH:
{@.metadata.labels.agent-install.openshift.io/bmh} Agent: {@.metadata.name} State:
{@.status.debugInfo.state}{"\n"}{end}'
```

Example output

```
BMH: ocp-worker-2 Agent: 4dac1ab2-7dd5-4894-a220-6a3473b67ee6 State: binding
BMH: ocp-worker-0 Agent: d9198891-39f4-4930-a679-65fb142b108b State: known-unbound
BMH: ocp-worker-1 Agent: da503cf1-a347-44f2-875c-4960ddb04091 State: insufficient
```

3. After the agents reach the **added-to-existing-cluster** state, verify that the OpenShift Container Platform nodes are ready by running the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

Example output

```
NAME      STATUS  ROLES  AGE   VERSION
ocp-worker-1  Ready   worker  5m41s  v1.24.0+3882f8f
ocp-worker-2  Ready   worker  6m3s   v1.24.0+3882f8f
```

4. Adding workloads to the nodes can reconcile some cluster operators. The following command displays the creation of two machines that happened after scaling up the node pool:

```
$ oc -n <hosted_control_plane_namespace> get machines
```

Example output

```
NAME                                CLUSTER      NODENAME  PROVIDERID
PHASE  AGE  VERSION
hypercluster1-c96b6f675-m5vch  hypercluster1-b2qhl  ocp-worker-1  agent://da503cf1-
a347-44f2-875c-4960ddb04091  Running  15m  4.11.5
hypercluster1-c96b6f675-tl42p  hypercluster1-b2qhl  ocp-worker-2  agent://4dac1ab2-
7dd5-4894-a220-6a3473b67ee6  Running  15m  4.11.5
```

4.8. DEPLOYING HOSTED CONTROL PLANES ON OPENSTACK



IMPORTANT

Deploying hosted control planes clusters on Red Hat OpenStack Platform (RHOSP) is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can deploy hosted control planes with hosted clusters that run on Red Hat OpenStack Platform (RHOSP) 17.1.

A *hosted cluster* is an OpenShift Container Platform cluster with its API endpoint and control plane that are hosted on a management cluster. With hosted control planes, control planes exist as pods on a management cluster without the need for dedicated virtual or physical machines for each control plane.

4.8.1. Prerequisites for OpenStack

Before you create a hosted cluster on Red Hat OpenStack Platform (RHOSP), ensure that you meet the prerequisites.

- You have administrative access to a management OpenShift Container Platform cluster version 4.17 or greater. This cluster can run on bare metal, RHOSP, or a supported public cloud.
- The HyperShift Operator is installed on the management cluster as specified in "Preparing to deploy hosted control planes".
- The management cluster is configured with OVN-Kubernetes as the default pod network CNI.
- The OpenShift CLI (**oc**) and hosted control planes CLI, **hcp** are installed.
- A load-balancer backend, for example, Octavia, is installed on the management OCP cluster. The load balancer is required for the **kube-api** service to be created for each hosted cluster.
 - When ingress is configured with an Octavia load balance, the RHOSP Octavia service is running in the cloud that hosts the guest cluster.
- A valid pull secret file is present for the **quay.io/openshift-release-dev** repository.
- The default external network for the management cluster is reachable from the guest cluster. The **kube-apiserver** load-balancer type service is created on this network.
- If you use a pre-defined floating IP address for ingress, you created a DNS record that points to it for the following wildcard domain: ***.apps.<cluster_name>.<base_domain>**, where:
 - **<cluster_name>** is the name of the management cluster.
 - **<base_domain>** is the parent DNS domain under which your cluster's applications live.

Additional resources

- [Pull secret](#)

4.8.2. Preparing the management cluster for etcd local storage

In a hosted control planes deployment on Red Hat OpenStack Platform (RHOSP), you can improve etcd performance by using local ephemeral storage that is provisioned with the TopoLVM CSI driver instead of relying on the default Cinder-based Persistent Volume Claims (PVCs).

Prerequisites

- You have access to a management cluster with HyperShift installed.
- You can create and manage RHOSP flavors and machine sets.
- You have the **oc** and **openstack** CLI tools installed and configured.
- You are familiar with TopoLVM and Logical Volume Manager (LVM) storage concepts.
- You installed the LVM Storage Operator on the management cluster. For more information, see "Installing LVM Storage by using the CLI" in the Storage section of the OpenShift Container Platform documentation.

Procedure

1. Create a Nova flavor with an additional ephemeral disk by using the **openstack** CLI. For example:

```
$ openstack flavor create \
  --id auto \
  --ram 8192 \
  --disk 0 \
  --ephemeral 100 \
  --vcpus 4 \
  --public \
  hcp-etcd-ephemeral
```



NOTE

Nova automatically attaches the ephemeral disk to the instance and formats it as **vfat** when a server is created with that flavor.

2. Create a compute machine set that uses the new flavor. For more information, see "Creating a compute machine set on OpenStack" in the OpenShift Container Platform documentation.
3. Scale the machine set to meet your requirements. If clusters are deployed for high availability, a minimum of 3 workers must be deployed so the pods can be distributed accordingly.
4. Label the new worker nodes to identify them for etcd use. For example:

```
$ oc label node <node_name> hypershift-capable=true
```

This label is arbitrary; you can update it later.

5. In a file called **lvmcluster.yaml**, create the following **LVMCluster** custom resource to the local storage configuration for etcd:

```

apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: etcd-hcp
  namespace: openshift-storage
spec:
  storage:
    deviceClasses:
      - name: etcd-class
        default: true
        nodeSelector:
          nodeSelectorTerms:
            - matchExpressions:
                - key: hypershift-capable
                  operator: In
                  values:
                    - "true"
        deviceSelector:
          forceWipeDevicesAndDestroyAllData: true
          paths:
            - /dev/vdb

```

In this example resource:

- The ephemeral disk location is **/dev/vdb**, which is the case in most situations. Verify that this location is true in your case, and note that symlinks are not supported.
- The parameter **forceWipeDevicesAndDestroyAllData** is set to a **True** value because the default Nova ephemeral disk comes formatted in VFAT.

6. Apply the **LVMCluster** resource by running the following command:

```
oc apply -f lvmcluster.yaml
```

7. Verify the **LVMCluster** resource by running the following command:

```
$ oc get lvmcluster -A
```

Example output

```

NAMESPACE      NAME      STATUS
openshift-storage  etcd-hcp  Ready

```

8. Verify the **StorageClass** resource by running the following command:

```
$ oc get storageclass
```

Example output

```

NAME              PROVISIONER          RECLAIMPOLICY  VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION  AGE
lvms-etcd-class    topolvm.io           Delete         WaitForFirstConsumer  true

```

23m	standard-csi (default)	cinder.csi.openstack.org	Delete	WaitForFirstConsumer	true
56m					

You can now deploy a hosted cluster with a performant etcd configuration. The deployment process is described in "Creating a hosted cluster on OpenStack".

4.8.3. Creating a floating IP for ingress

If you want to make ingress available in a hosted cluster without manual intervention, you can create a floating IP address for it in advance.

Prerequisites

- You have access to the Red Hat OpenStack Platform (RHOSP) cloud.
- If you use a pre-defined floating IP address for ingress, you created a DNS record that points to it for the following wildcard domain: `*.apps.<cluster_name>.<base_domain>`, where:
 - `<cluster_name>` is the name of the management cluster.
 - `<base_domain>` is the parent DNS domain under which your cluster's applications live.

Procedure

- Create a floating IP address by running the following command:

```
$ openstack floating ip create <external_network_id>
```

where:

<external_network_id>

Specifies the ID of the external network.



NOTE

If you specify a floating IP address by using the `--openstack-ingress-floating-ip` flag without creating it in advance, the **cloud-provider-openstack** component attempts to create it automatically. This process only succeeds if the Neutron API policy permits creating a floating IP address with a specific IP address.

4.8.4. Uploading the RHCOS image to OpenStack

If you want to specify the RHCOS image to use when deploying node pools on hosted control planes and Red Hat OpenStack Platform (RHOSP) deployment, upload the image to the RHOSP cloud.

If you do not upload the image, the OpenStack Resource Controller (ORC) downloads an image from the OpenShift Container Platform mirror and deletes the image after deletion of the hosted cluster.

Prerequisites

- You downloaded the RHCOS image from the OpenShift Container Platform mirror.
- You have access to your RHOSP cloud.

Procedure

- Upload an RHCOS image to RHOSP by running the following command:

```
$ openstack image create --disk-format qcow2 --file <image_file_name> rhcos
```

where:

<image_file_name>

Specifies the file name of the RHCOS image.

4.8.5. Creating a hosted cluster on OpenStack

You can create a hosted cluster on Red Hat OpenStack Platform (RHOSP) by using the **hcp** CLI.

Prerequisites

- You completed all prerequisite steps in "Preparing to deploy hosted control planes".
- You reviewed "Prerequisites for OpenStack".
- You completed all steps in "Preparing the management cluster for etcd local storage".
- You have access to the management cluster.
- You have access to the RHOSP cloud.

Procedure

- Create a hosted cluster by running the **hcp create** command. For example, for a cluster that takes advantage of the performant etcd configuration detailed in "Preparing the management cluster for etcd local storage", enter:

```
$ hcp create cluster openstack \  
  --name my-hcp-cluster \  
  --openstack-node-flavor m1.xlarge \  
  --base-domain example.com \  
  --pull-secret /path/to/pull-secret.json \  
  --release-image quay.io/openshift-release-dev/ocp-release:4.22.0-x86_64 \  
  --node-pool-replicas 3 \  
  --etcd-storage-class lvms-etcd-class
```



NOTE

Many options are available at cluster creation. For RHOSP-specific options, see "Options for creating a Hosted Control Planes cluster on OpenStack". For general options, see the **hcp** documentation.

Verification

1. Verify that the hosted cluster is ready by running the following command on it:

```
$ oc -n clusters-<cluster_name> get pods
```

where:

<cluster_name>

Specifies the name of the cluster.

After several minutes, the output should show that the hosted control plane pods are running.

Example output

```

NAME                                READY  STATUS  RESTARTS  AGE
capi-provider-5cc7b74f47-n5gkr      1/1    Running  0         3m
catalog-operator-5f799567b7-fd6jw   2/2    Running  0         69s
certified-operators-catalog-784b9899f9-mrp6p  1/1    Running  0         66s
cluster-api-6bbc867966-l4dwl        1/1    Running  0         66s
...
...
...
redhat-operators-catalog-9d5fd4d44-z8qqk  1/1    Running  0

```

2. To validate the etcd configuration of the cluster:

a. Validate the etcd persistent volume claim (PVC) by running the following command:

```
$ oc get pvc -A
```

b. Inside the hosted control planes etcd pod, confirm the mount path and device by running the following command:

```
$ df -h /var/lib
```



NOTE

The RHOSP resources that the cluster API provider creates are tagged with the label **openshiftClusterID=<infraID>**.

You can define additional tags for the resources as values in the **HostedCluster.Spec.Platform.OpenStack.Tags** field of a YAML manifest that you use to create the hosted cluster. After you scale up the node pool, the tags apply to resources.

4.8.5.1. Options for creating a Hosted Control Planes cluster on OpenStack

You can supply several options to the **hcp** CLI while deploying a Hosted Control Planes Cluster on Red Hat OpenStack Platform (RHOSP).

Option	Description	Required
--openstack-ca-cert-file	Path to the OpenStack CA certificate file. If not provided, this will be automatically extracted from the cloud entry in clouds.yaml .	No

Option	Description	Required
--openstack-cloud	Name of the cloud entry in clouds.yaml . The default value is openstack .	No
--openstack-credentials-file	Path to the OpenStack credentials file. If not provided, hcp will search the following directories: • The current working directory • \$HOME/.config/openstack • /etc/openstack	No
--openstack-dns-nameservers	List of DNS server addresses that are provided when creating the subnet.	No
--openstack-external-network-id	ID of the OpenStack external network.	No
--openstack-ingress-floating-ip	A floating IP for OpenShift ingress.	No
--openstack-node-additional-port	Additional ports to attach to nodes. Valid values are: network-id , vnic-type , disable-port-security , and address-pairs .	No
--openstack-node-availability-zone	Availability zone for the node pool.	No
--openstack-node-flavor	Flavor for the node pool.	Yes
--openstack-node-image-name	Image name for the node pool.	No

CHAPTER 5. MANAGING HOSTED CONTROL PLANES

5.1. MANAGING HOSTED CONTROL PLANES ON AWS

After you deploy a hosted cluster on Amazon Web Services (AWS), you can manage the cluster.

5.1.1. Prerequisites to manage AWS infrastructure and IAM permissions

To configure hosted control planes for OpenShift Container Platform on Amazon Web Services (AWS), you must meet the infrastructure requirements.

- You must configure hosted control planes before you can create hosted clusters.
- You must create an AWS Identity and Access Management (IAM) role and AWS Security Token Service (STS) credentials.

5.1.1.1. Infrastructure requirements for AWS

When you use hosted control planes on Amazon Web Services (AWS), the infrastructure requirements vary based on your setup.

The infrastructure requirements fit in the following categories:

- Prerequisite and unmanaged infrastructure for the HyperShift Operator in an arbitrary AWS account
- Prerequisite and unmanaged infrastructure in a hosted cluster AWS account
- Hosted control planes-managed infrastructure in a management AWS account
- Hosted control planes-managed infrastructure in a hosted cluster AWS account
- Kubernetes-managed infrastructure in a hosted cluster AWS account

Prerequisite means that hosted control planes requires AWS infrastructure to properly work. *Unmanaged* means that no Operator or controller creates the infrastructure for you.

5.1.1.2. Unmanaged infrastructure for the HyperShift Operator in an AWS account

An arbitrary Amazon Web Services (AWS) account depends on the provider of the hosted control planes service.

In self-managed hosted control planes, the cluster service provider controls the AWS account. The cluster service provider is the administrator who hosts cluster control planes and is responsible for uptime. In managed hosted control planes, the AWS account belongs to Red Hat.

In a prerequisite and unmanaged infrastructure for the HyperShift Operator, the following infrastructure requirements apply for a management cluster AWS account:

- One S3 Bucket
 - OpenID Connect (OIDC)
- Route 53 hosted zones

- A domain to host private and public entries for hosted clusters

5.1.1.3. Unmanaged infrastructure requirements for a management AWS account

When your infrastructure is prerrequired and unmanaged in a hosted cluster Amazon Web Services (AWS) account, ensure that you are familiar with the infrastructure requirements for all access modes.

- One VPC
- One DHCP Option
- Two subnets
 - A private subnet that is an internal data plane subnet
 - A public subnet that enables access to the internet from the data plane
- One internet gateway
- One elastic IP
- One NAT gateway
- One security group (worker nodes)
- Two route tables (one private and one public)
- Two Route 53 hosted zones
- Enough quota for the following items:
 - One Ingress service load balancer for public hosted clusters
 - One private link endpoint for private hosted clusters



NOTE

For private link networking to work, the endpoint zone in the hosted cluster AWS account must match the zone of the instance that is resolved by the service endpoint in the management cluster AWS account. In AWS, the zone names are aliases, such as us-east-2b, which do not necessarily map to the same zone in different accounts. As a result, for private link to work, the management cluster must have subnets or workers in all zones of its region.

5.1.1.4. Infrastructure requirements for a management AWS account

When your infrastructure is managed by hosted control planes in a management AWS account, the infrastructure requirements differ depending on whether your clusters are public, private, or a combination.

For accounts with public clusters, the infrastructure requirements are as follows:

- Network load balancer: a load balancer Kube API server
 - Kubernetes creates a security group
- Volumes

- For etcd (one or three depending on high availability)
- For OVN-Kube

For accounts with private clusters, the infrastructure requirements are as follows:

- Network load balancer: a load balancer private router
- Endpoint service (private link)

For accounts with public and private clusters, the infrastructure requirements are as follows:

- Network load balancer: a load balancer public router
- Network load balancer: a load balancer private router
- Endpoint service (private link)
- Volumes
 - For etcd (one or three depending on high availability)
 - For OVN-Kube

5.1.1.5. Infrastructure requirements for an AWS account in a hosted cluster

When your infrastructure is managed by hosted control planes in a hosted cluster Amazon Web Services (AWS) account, the infrastructure requirements differ depending on whether your clusters are public, private, or a combination.

For accounts with public clusters, the infrastructure requirements are as follows:

- Node pools must have EC2 instances that have **Role** and **RolePolicy** defined.

For accounts with private clusters, the infrastructure requirements are as follows:

- One private link endpoint for each availability zone
- EC2 instances for node pools

For accounts with public and private clusters, the infrastructure requirements are as follows:

- One private link endpoint for each availability zone
- EC2 instances for node pools

5.1.1.6. Kubernetes-managed infrastructure in a hosted cluster AWS account

When Kubernetes manages your infrastructure in a hosted cluster Amazon Web Services (AWS) account, ensure that you meet the infrastructure requirements.

The infrastructure requirements are as follows:

- A network load balancer for default Ingress
- An S3 bucket for registry

5.1.2. Identity and Access Management (IAM) permissions for hosted control planes on AWS

In the context of hosted control planes, the consumer is responsible to create the Amazon Resource Name (ARN) roles. The *consumer* is an automated process to generate the permissions files. It might be the command-line interface (CLI) or OpenShift Cluster Manager.

Hosted control planes can enable granularity to honor the principle of least-privilege components, which means that every component uses its own role to operate or create Amazon Web Services (AWS) objects, and the roles are limited to what is required for the product to function normally.

The hosted cluster receives the ARN roles as input and the consumer creates an AWS permission configuration for each component. As a result, the component can authenticate through STS and preconfigured OIDC IDP.

The following roles are consumed by some of the components from hosted control planes that run on the control plane and operate on the data plane:

- **controlPlaneOperatorARN**
- **imageRegistryARN**
- **ingressARN**
- **kubeCloudControllerARN**
- **nodePoolManagementARN**
- **storageARN**
- **networkARN**

The following example shows a reference to the IAM roles from the hosted cluster:

```
...
endpointAccess: Public
region: us-east-2
resourceTags:
- key: kubernetes.io/cluster/example-cluster-bz4j5
  value: owned
rolesRef:
  controlPlaneOperatorARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-control-plane-operator
  imageRegistryARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-openshift-image-registry
  ingressARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-openshift-ingress
  kubeCloudControllerARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-cloud-controller
  networkARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-cloud-network-config-controller
  nodePoolManagementARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-node-pool
  storageARN: arn:aws:iam::820196288204:role/example-cluster-bz4j5-aws-ebs-csi-driver-controller
type: AWS
...
```

The roles that hosted control planes uses are shown in the following examples:

- **ingressARN**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "elasticloadbalancing:DescribeLoadBalancers",
        "tag:GetResources",
        "route53:ListHostedZones"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "route53:ChangeResourceRecordSets"
      ],
      "Resource": [
        "arn:aws:route53::PUBLIC_ZONE_ID",
        "arn:aws:route53::PRIVATE_ZONE_ID"
      ]
    }
  ]
}
```

- **imageRegistryARN**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:CreateBucket",
        "s3:DeleteBucket",
        "s3:PutBucketTagging",
        "s3:GetBucketTagging",
        "s3:PutBucketPublicAccessBlock",
        "s3:GetBucketPublicAccessBlock",
        "s3:PutEncryptionConfiguration",
        "s3:GetEncryptionConfiguration",
        "s3:PutLifecycleConfiguration",
        "s3:GetLifecycleConfiguration",
        "s3:GetBucketLocation",
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucketMultipartUploads",
        "s3:AbortMultipartUpload",
        "s3:ListMultipartUploadParts"
      ],
      "Resource": "*"
    }
  ]
}
```

```
]
}
```

- **storageARN**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:AttachVolume",
        "ec2:CreateSnapshot",
        "ec2:CreateTags",
        "ec2:CreateVolume",
        "ec2>DeleteSnapshot",
        "ec2>DeleteTags",
        "ec2>DeleteVolume",
        "ec2:DescribeInstances",
        "ec2:DescribeSnapshots",
        "ec2:DescribeTags",
        "ec2:DescribeVolumes",
        "ec2:DescribeVolumesModifications",
        "ec2:DetachVolume",
        "ec2:ModifyVolume"
      ],
      "Resource": "*"
    }
  ]
}
```

- **networkARN**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceStatus",
        "ec2:DescribeInstanceTypes",
        "ec2:UnassignPrivateIpAddresses",
        "ec2:AssignPrivateIpAddresses",
        "ec2:UnassignIpv6Addresses",
        "ec2:AssignIpv6Addresses",
        "ec2:DescribeSubnets",
        "ec2:DescribeNetworkInterfaces"
      ],
      "Resource": "*"
    }
  ]
}
```

- **kubeCloudControllerARN**

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeImages",
        "ec2:DescribeRegions",
        "ec2:DescribeRouteTables",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVolumes",
        "ec2:CreateSecurityGroup",
        "ec2:CreateTags",
        "ec2:CreateVolume",
        "ec2:ModifyInstanceAttribute",
        "ec2:ModifyVolume",
        "ec2:AttachVolume",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:CreateRoute",
        "ec2>DeleteRoute",
        "ec2>DeleteSecurityGroup",
        "ec2>DeleteVolume",
        "ec2:DetachVolume",
        "ec2:RevokeSecurityGroupIngress",
        "ec2:DescribeVpcs",
        "elasticloadbalancing:AddTags",
        "elasticloadbalancing:AttachLoadBalancerToSubnets",
        "elasticloadbalancing:ApplySecurityGroupsToLoadBalancer",
        "elasticloadbalancing:CreateLoadBalancer",
        "elasticloadbalancing:CreateLoadBalancerPolicy",
        "elasticloadbalancing:CreateLoadBalancerListeners",
        "elasticloadbalancing:ConfigureHealthCheck",
        "elasticloadbalancing>DeleteLoadBalancer",
        "elasticloadbalancing>DeleteLoadBalancerListeners",
        "elasticloadbalancing:DescribeLoadBalancers",
        "elasticloadbalancing:DescribeLoadBalancerAttributes",
        "elasticloadbalancing:DetachLoadBalancerFromSubnets",
        "elasticloadbalancing:DeregisterInstancesFromLoadBalancer",
        "elasticloadbalancing:ModifyLoadBalancerAttributes",
        "elasticloadbalancing:RegisterInstancesWithLoadBalancer",
        "elasticloadbalancing:SetLoadBalancerPoliciesForBackendServer",
        "elasticloadbalancing:AddTags",
        "elasticloadbalancing:CreateListener",
        "elasticloadbalancing:CreateTargetGroup",
        "elasticloadbalancing>DeleteListener",
        "elasticloadbalancing>DeleteTargetGroup",
        "elasticloadbalancing:DescribeListeners",
        "elasticloadbalancing:DescribeLoadBalancerPolicies",
        "elasticloadbalancing:DescribeTargetGroups",
        "elasticloadbalancing:DescribeTargetHealth",
        "elasticloadbalancing:ModifyListener",
        "elasticloadbalancing:ModifyTargetGroup",
        "elasticloadbalancing:RegisterTargets",
        "elasticloadbalancing:SetLoadBalancerPoliciesOfListener",
        "iam:CreateServiceLinkedRole",

```

```

        "kms:DescribeKey"
      ],
      "Resource": [
        "*"
      ],
      "Effect": "Allow"
    }
  ]
}

```

- **nodePoolManagementARN**

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "ec2:AllocateAddress",
        "ec2:AssociateRouteTable",
        "ec2:AttachInternetGateway",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:CreateInternetGateway",
        "ec2:CreateNatGateway",
        "ec2:CreateRoute",
        "ec2:CreateRouteTable",
        "ec2:CreateSecurityGroup",
        "ec2:CreateSubnet",
        "ec2:CreateTags",
        "ec2>DeleteInternetGateway",
        "ec2>DeleteNatGateway",
        "ec2>DeleteRouteTable",
        "ec2>DeleteSecurityGroup",
        "ec2>DeleteSubnet",
        "ec2>DeleteTags",
        "ec2:DescribeAccountAttributes",
        "ec2:DescribeAddresses",
        "ec2:DescribeAvailabilityZones",
        "ec2:DescribeImages",
        "ec2:DescribeInstances",
        "ec2:DescribeInternetGateways",
        "ec2:DescribeNatGateways",
        "ec2:DescribeNetworkInterfaces",
        "ec2:DescribeNetworkInterfaceAttribute",
        "ec2:DescribeRouteTables",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:DescribeVpcAttribute",
        "ec2:DescribeVolumes",
        "ec2:DetachInternetGateway",
        "ec2:DisassociateRouteTable",
        "ec2:DisassociateAddress",
        "ec2:ModifyInstanceAttribute",
        "ec2:ModifyNetworkInterfaceAttribute",
        "ec2:ModifySubnetAttribute",
        "ec2:ReleaseAddress",

```



```

        "ec2:RevokeSecurityGroupIngress",
        "ec2:RunInstances",
        "ec2:TerminateInstances",
        "tag:GetResources",
        "ec2:CreateLaunchTemplate",
        "ec2:CreateLaunchTemplateVersion",
        "ec2:DescribeLaunchTemplates",
        "ec2:DescribeLaunchTemplateVersions",
        "ec2>DeleteLaunchTemplate",
        "ec2>DeleteLaunchTemplateVersions"
    ],
    "Resource": [
        "*"
    ],
    "Effect": "Allow"
},
{
    "Condition": {
        "StringLike": {
            "iam:AWSServiceName": "elasticloadbalancing.amazonaws.com"
        }
    },
    "Action": [
        "iam:CreateServiceLinkedRole"
    ],
    "Resource": [
        "arn:*:iam::*:role/aws-service-
role/elasticloadbalancing.amazonaws.com/AWSServiceRoleForElasticLoadBalancing"
    ],
    "Effect": "Allow"
},
{
    "Action": [
        "iam:PassRole"
    ],
    "Resource": [
        "arn:*:iam::*:role/*-worker-role"
    ],
    "Effect": "Allow"
}
]
}

```

- **controlPlaneOperatorARN**

```

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ec2:CreateVpcEndpoint",
                "ec2:DescribeVpcEndpoints",
                "ec2:ModifyVpcEndpoint",
                "ec2>DeleteVpcEndpoints",
                "ec2:CreateTags",

```

```

        "route53:ListHostedZones"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "route53:ChangeResourceRecordSets",
        "route53:ListResourceRecordSets"
      ],
      "Resource": "arn:aws:route53:::%s"
    }
  ]
}

```

5.1.3. Separate creation of the hosted cluster and its resources

By default, the **hcp create cluster aws** command creates the cloud infrastructure with the hosted cluster and applies it. However, you can create the cloud infrastructure separately so that you can use the command to create only the cluster, or render it to modify it before you apply it.

The process to create the hosted cluster and its resources separately involves creating the cloud infrastructure, creating the AWS Identity and Access (IAM) resources, and then creating the cluster.

5.1.3.1. Creating the AWS infrastructure separately

To create the Amazon Web Services (AWS) infrastructure, you need to create a Virtual Private Cloud (VPC) and other resources for your cluster. You can use the AWS console or an infrastructure automation and provisioning tool.

For instructions to use the AWS console, see [Create a VPC plus other VPC resources](#) in the AWS Documentation.

The VPC must include private and public subnets and resources for external access, such as a network address translation (NAT) gateway and an internet gateway. In addition to the VPC, you need a private hosted zone for the ingress of your cluster. If you are creating clusters that use PrivateLink (**Private** or **PublicAndPrivate** access modes), you need an additional hosted zone for PrivateLink.

Procedure

- Create the AWS infrastructure for your hosted cluster by using the following example configuration:

```

---
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: null
  name: clusters
spec: {}
status: {}
---
apiVersion: v1
data:
  .dockerconfigjson: xxxxxxxxxxxx

```

```

kind: Secret
metadata:
  creationTimestamp: null
  labels:
    hypershift.openshift.io/safe-to-delete-with-cluster: "true"
  name: <pull_secret_name>
  namespace: clusters
---
apiVersion: v1
data:
  key: xxxxxxxxxxxxxxxxx
kind: Secret
metadata:
  creationTimestamp: null
  labels:
    hypershift.openshift.io/safe-to-delete-with-cluster: "true"
  name: <etcd_encryption_key_name>
  namespace: clusters
type: Opaque
---
apiVersion: v1
data:
  id_rsa: xxxxxxxxx
  id_rsa.pub: xxxxxxxxx
kind: Secret
metadata:
  creationTimestamp: null
  labels:
    hypershift.openshift.io/safe-to-delete-with-cluster: "true"
  name: <ssh_key_name>
  namespace: clusters
---
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  creationTimestamp: null
  name: <hosted_cluster_name>
  namespace: clusters
spec:
  autoscaling: {}
  configuration: {}
  controllerAvailabilityPolicy: SingleReplica
  dns:
    baseDomain: <dns_domain>
    privateZoneID: xxxxxxxx
    publicZoneID: xxxxxxxx
  etcd:
    managed:
      storage:
        persistentVolume:
          size: 8Gi
          storageClassName: gp3-csi
          type: PersistentVolume
        managementType: Managed
    fips: false
    infraID: <infra_id>

```

```

issuerURL: <issuer_url>
networking:
  clusterNetwork:
    - cidr: 10.132.0.0/14
  machineNetwork:
    - cidr: 10.0.0.0/16
  networkType: OVNKubernetes
  serviceNetwork:
    - cidr: 172.31.0.0/16
olmCatalogPlacement: management
platform:
  aws:
    cloudProviderConfig:
      subnet:
        id: <subnet_xxx>
      vpc: <vpc_xxx>
      zone: us-west-1b
    endpointAccess: Public
    multiArch: false
    region: us-west-1
    rolesRef:
      controlPlaneOperatorARN: arn:aws:iam::820196288204:role/<infra_id>-control-plane-
operator
      imageRegistryARN: arn:aws:iam::820196288204:role/<infra_id>-openshift-image-
registry
      ingressARN: arn:aws:iam::820196288204:role/<infra_id>-openshift-ingress
      kubeCloudControllerARN: arn:aws:iam::820196288204:role/<infra_id>-cloud-controller
      networkARN: arn:aws:iam::820196288204:role/<infra_id>-cloud-network-config-
controller
      nodePoolManagementARN: arn:aws:iam::820196288204:role/<infra_id>-node-pool
      storageARN: arn:aws:iam::820196288204:role/<infra_id>-aws-ebs-csi-driver-controller
    type: AWS
  pullSecret:
    name: <pull_secret_name>
  release:
    image: quay.io/openshift-release-dev/ocp-release:4.16-x86_64
  secretEncryption:
    aescbc:
      activeKey:
        name: <etcd_encryption_key_name>
      type: aescbc
  services:
    - service: APIServer
      servicePublishingStrategy:
        type: LoadBalancer
    - service: OAuthServer
      servicePublishingStrategy:
        type: Route
    - service: Konnectivity
      servicePublishingStrategy:
        type: Route
    - service: Ignition
      servicePublishingStrategy:
        type: Route
    - service: OVNSbDb
      servicePublishingStrategy:

```

```

    type: Route
  sshKey:
    name: <ssh_key_name>
  status:
    controlPlaneEndpoint:
      host: ""
      port: 0
---
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  creationTimestamp: null
  name: <node_pool_name>
  namespace: clusters
spec:
  arch: amd64
  clusterName: <hosted_cluster_name>
  management:
    autoRepair: true
    upgradeType: Replace
  nodeDrainTimeout: 0s
  platform:
    aws:
      instanceProfile: <instance_profile_name>
      instanceType: m6i.xlarge
      rootVolume:
        size: 120
        type: gp3
      subnet:
        id: <subnet_xxx>
        type: AWS
  release:
    image: quay.io/openshift-release-dev/ocp-release:4.16-x86_64
    replicas: 2
  status:
    replicas: 0

```

- **<pull_secret_name>** specifies the name of your pull secret.
- **<etcd_encryption_key_name>** specifies the name of your etcd encryption key.
- **<ssh_key_name>** specifies the name of your SSH key.
- **<hosted_cluster_name>** specifies the name of your hosted cluster.
- **<dns_domain>** specifies your base DNS domain, such as **example.com**.
- **<infra_id>** specifies the value that identifies the IAM resources that are associated with the hosted cluster.
- **<issuer_url>** specifies your issuer URL, which ends with your **infra_id** value. For example, <https://example-hosted-us-west-1.s3.us-west-1.amazonaws.com/example-hosted-infra-id>.

- **<subnet_xxx>** specifies your subnet ID. Both private and public subnets need to be tagged. For public subnets, use **kubernetes.io/role/elb=1**. For private subnets, use **kubernetes.io/role/internal-elb=1**.
- **<vpc_xxx>** specifies your VPC ID.
- **<node_pool_name>** specifies the name of your **NodePool** resource.
- **<instance_profile_name>** specifies the name of your AWS instance.

5.1.3.2. Creating a hosted cluster separately

In hosted control planes on AWS, you can create a hosted cluster separately from creating the infrastructure and Identity and Access Management (IAM) resources.

Prerequisites

- You created infrastructure resources separately. For more information, see "Creating the AWS infrastructure separately".
- You created the following IAM resources:
 - An OpenID Connect (OIDC) identity provider in IAM, which is required to enable STS authentication. For more information, see [Create an OpenID Connect \(OIDC\) identity provider in IAM](#).
 - The seven roles that are listed in "Identity and Access Management (IAM) permissions." The roles are separate for every component that interacts with the provider, such as the Kubernetes controller manager, cluster API provider, and registry. For more information, see [IAM role creation](#).
 - The instance profile, which is the profile that is assigned to all worker instances of the cluster. For more information, see [Use instance profiles](#).

Procedure

1. To create a hosted cluster separately, enter the following command:

```
$ hcp create cluster aws \  
  --infra-id <infra_id> \  
  --name <hosted_cluster_name> \  
  --sts-creds <path_to_sts_credential_file> \  
  --pull-secret <path_to_pull_secret> \  
  --generate-ssh \  
  --node-pool-replicas 3 \  
  --role-arn <role_name> \  
  --render-sensitive \  
  --render > <file_name>.yaml
```

- **--infra-id** specifies the same ID that you specified in the **create infra aws** command. This value identifies the IAM resources that are associated with the hosted cluster.
- **--name** specifies the name of your hosted cluster.
- **--sts-creds** specifies the same name that you specified in the **create infra aws** command.

- **--pull-secret** specifies the name of the file that contains a valid OpenShift Container Platform pull secret.
- **--generate-ssh** is an optional flag, but it is good to include in case you need to SSH to your workers. An SSH key is generated for you and is stored as a secret in the same namespace as the hosted cluster.
- **--role-arn** specifies the Amazon Resource Name (ARN); for example, **arn:aws:iam::820196288204:role/myrole**. For more information about ARN roles, see "Identity and Access Management (IAM) permissions".
- **--render-sensitive** generates secrets that are stored in the YAML file.
- **--render** is an optional flag. You can include this flag to redirect output to a file where you can edit the resources before you apply them to the cluster.

2. Apply the manifests by entering the following command:

```
$ oc apply -f <file_name>.yaml
```

Verification

- After you run the command, you can verify that the following resources are applied to your cluster:
 - A namespace
 - A secret with your pull secret
 - A **HostedCluster**
 - A **NodePool**
 - Three AWS STS secrets for control plane components
 - If you specified the **--generate-ssh** flag, one SSH key secret.

5.1.4. Transitioning a hosted cluster from single-architecture to multi-architecture

You can transition your single-architecture 64-bit AMD hosted cluster to a multi-architecture hosted cluster on Amazon Web Services (AWS) to reduce the cost of running workloads on your cluster.

For example, you can run existing workloads on 64-bit AMD while transitioning to 64-bit ARM and you can manage these workloads from a central Kubernetes cluster.

A single-architecture hosted cluster can manage node pools of only one particular CPU architecture. However, a multi-architecture hosted cluster can manage node pools with different CPU architectures. On AWS, a multi-architecture hosted cluster can manage both 64-bit AMD and 64-bit ARM node pools.

Prerequisites

- You installed an OpenShift Container Platform management cluster for AWS with the multicluster engine for Kubernetes Operator.

- You have an existing single-architecture hosted cluster that uses 64-bit AMD variant of the OpenShift Container Platform release payload.
- You have an existing node pool that uses the same 64-bit AMD variant of the OpenShift Container Platform release payload and is managed by an existing hosted cluster.
- You installed the following command-line tools:
 - **oc**
 - **kubecti**
 - **hcp**
 - **skopeo**

Procedure

1. Review an existing OpenShift Container Platform release image of the single-architecture hosted cluster by running the following command:

```
$ oc get hostedcluster/<hosted_cluster_name> \
-o jsonpath='{.spec.release.image}'
```

Replace **<hosted_cluster_name>** with your hosted cluster name.

Example output

```
quay.io/openshift-release-dev/ocp-release:4.20.0-x86_64
```

2. In your OpenShift Container Platform release image, if you use the digest instead of a tag, find the multi-architecture tag version of your release image:
 - a. Set the **OCP_VERSION** environment variable for the OpenShift Container Platform version by running the following command:

```
$ OCP_VERSION=$(oc image info quay.io/openshift-release-dev/ocp-
release@sha256:ac78ebf77f95ab8ff52847ecd22592b545415e1ff6c7ff7f66bf81f158ae4f5e
\
-o jsonpath='{.config.config.Labels["io.openshift.release"]}') 
```

- b. Set the **MULTI_ARCH_TAG** environment variable for the multi-architecture tag version of your release image by running the following command:

```
$ MULTI_ARCH_TAG=$(skopeo inspect docker://quay.io/openshift-release-dev/ocp-
release@sha256:ac78ebf77f95ab8ff52847ecd22592b545415e1ff6c7ff7f66bf81f158ae4f5e
\
| jq -r '.RepoTags' | sed 's/"//g' | sed 's/,//g' \
| grep -w "$OCP_VERSION-multi$" | xargs)
```

- c. Set the **IMAGE** environment variable for the multi-architecture release image name by running the following command:

```
$ IMAGE=quay.io/openshift-release-dev/ocp-release:$MULTI_ARCH_TAG
```


- d. To see the list of multi-architecture image digests, run the following command:

```
$ oc image info $IMAGE
```

Example output

```
OS      DIGEST
linux/amd64
sha256:b4c7a91802c09a5a748fe19ddd99a8ffab52d8a31db3a081a956a87f22a22ff8
linux/ppc64le
sha256:66fda2ff6bd7704f1ba72be8bfe3e399c323de92262f594f8e482d110ec37388
linux/s390x
sha256:b1c1072dc639aaa2b50ec99b530012e3ceac19ddc28adcbcdc9643f2dfd14f34
linux/arm64
sha256:7b046404572ac96202d82b6cb029b421dddd40e88c73bbf35f602ffc13017f21
```

3. Transition the hosted cluster from single-architecture to multi-architecture:
- Set the multi-architecture OpenShift Container Platform release image for the hosted cluster by ensuring that you use the same OpenShift Container Platform version as the hosted cluster. Run the following command:

```
$ oc patch -n clusters hostedclusters/<hosted_cluster_name> -p \
  '{"spec":{"release":{"image":"quay.io/openshift-release-dev/ocp-release:<4.x.y>-multi"}}}' \
  --type=merge
```

Replace **<4.y.z>** with the supported OpenShift Container Platform version that you use.

- Confirm that the multi-architecture image is set in your hosted cluster by running the following command:

```
$ oc get hostedcluster/<hosted_cluster_name> \
  -o jsonpath='{.spec.release.image}'
```

4. Check that the status of the **HostedControlPlane** resource is **Progressing** by running the following command:

```
$ oc get hostedcontrolplane -n <hosted_control_plane_namespace> -oyaml
```

Example output

```
#...
- lastTransitionTime: "2024-07-28T13:07:18Z"
  message: HostedCluster is deploying, upgrading, or reconfiguring
  observedGeneration: 5
  reason: Progressing
  status: "True"
  type: Progressing
#...
```

5. Check that the status of the **HostedCluster** resource is **Progressing** by running the following command:

—

```
$ oc get hostedcluster <hosted_cluster_name> \
-n <hosted_cluster_namespace> -oyaml
```

Verification

- Verify that a node pool is using the multi-architecture release image in your **HostedControlPlane** resource by running the following command:

```
$ oc get hostedcontrolplane -n clusters-example -oyaml
```

Example output

```
#...
version:
  availableUpdates: null
  desired:
    image: quay.io/openshift-release-dev/ocp-release:4.20.0-multi
    url: https://access.redhat.com/errata/RHBA-2024:4855
    version: 4.20.0
  history:
    - completionTime: "2024-07-28T13:10:58Z"
      image: quay.io/openshift-release-dev/ocp-release:4.20.0-multi
      startedTime: "2024-07-28T13:10:27Z"
      state: Completed
      verified: false
      version: 4.20.0
```



NOTE

The multi-architecture OpenShift Container Platform release image is updated in your **HostedCluster**, **HostedControlPlane** resources, and hosted control plane pods. However, your existing node pools do not transition with the multi-architecture image automatically, because the release image transition is decoupled between the hosted cluster and node pools. You must create new node pools on your new multi-architecture hosted cluster.

Next steps

- Create node pools on the multi-architecture hosted cluster.

5.1.5. Creating node pools on the multi-architecture hosted cluster

After you transition your hosted cluster from single-architecture to multi-architecture, create node pools on compute machines based on 64-bit AMD and 64-bit ARM architectures.

Procedure

- Create node pools based on 64-bit ARM architecture by entering the following command:

```
$ hcp create nodepool aws \
--cluster-name <hosted_cluster_name> \
--name <nodepool_name> \
```

```
--node-count=<node_count> \
--arch arm64
```

- Replace **<hosted_cluster_name>** with your hosted cluster name.
- Replace **<nodepool_name>** with your node pool name.
- Replace **<node_count>** with integer for your node count, for example, **2**.

2. Create node pools based on 64-bit AMD architecture by entering the following command:

```
$ hcp create nodepool aws \
  --cluster-name <hosted_cluster_name> \
  --name <nodepool_name> \
  --node-count=<node_count> \
  --arch amd64
```

- Replace **<hosted_cluster_name>** with your hosted cluster name.
- Replace **<nodepool_name>** with your node pool name.
- Replace **<node_count>** with integer for your node count, for example, **2**.

Verification

- Verify that a node pool is using the multi-architecture release image by entering the following command:

```
$ oc get nodepool/<nodepool_name> -oyaml
```

Example output for 64-bit AMD node pools

```
#...
spec:
  arch: amd64
#...
release:
  image: quay.io/openshift-release-dev/ocp-release:4.20.0-multi
```

Example output for 64-bit ARM node pools

```
#...
spec:
  arch: arm64
#...
release:
  image: quay.io/openshift-release-dev/ocp-release:4.20.0-multi
```

5.1.6. Adding or updating AWS tags for a hosted cluster

As a cluster instance administrator, you can add or update Amazon Web Services (AWS) tags without needing to re-create your hosted cluster. *Tags* are key-value pairs that are attached to AWS resources for management and automation.

You might want to use tags for the following purposes:

- Managing access controls.
- Tracking chargeback or showback.
- Managing cloud Identity and Access Management (IAM) conditional permissions.
- Aggregating resources based on tags. For example, you can query tags to calculate resource usage and billing costs.

You can add or update tags for several different types of resources, including Amazon Elastic File System (EFS) access points, load balancer resources, Amazon Elastic Block Storage (EBS) volumes, IAM users, and AWS S3.



IMPORTANT

On network load balancers, tags cannot be added or updated. The AWS load balancer reconciles whatever tags are in the **HostedCluster** resource. If you try to add or update a tag, the load balancer overwrites the tag.

In addition, tags cannot be updated on the default security group resource that is created directly by hosted control planes.

Prerequisites

- You must have cluster administrator permissions for your hosted cluster on AWS.

Procedure

1. If you want to add or update tags for EFS access points, complete steps 1 and 2. If you are adding or updating tags for other types of resources, complete only step 2.
 - a. In the **aws-efs-csi-driver-operator** service account, add two annotations, as shown in the following example. These annotations are required so that the Amazon Elastic Kubernetes Service (EKS) pod identity webhook that runs on the cluster can correctly assign AWS roles to the pods that the EFS Operator uses.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: <service_account_name>
  namespace: <project_name>
  annotations:
    eks.amazonaws.com/role-arn:<role_arn>
    eks.amazonaws.com/audience:sts.amazonaws.com
```

- b. Delete the Operator pod or roll out a restart of the **aws-efs-csi-driver-operator** deployment.
2. In the **HostedCluster** resource, enter information in the **resourceTags** fields, as shown in the following example:

Example HostedCluster resource

```

apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  #...
spec:
  autoscaling: {}
  clusterID: <cluster_id>
  configuration: {}
  controllerAvailabilityPolicy: SingleReplica
  dns:
    #...
  etcd:
    #...
  fips: false
  infraID: <infra_id>
  infrastructureAvailabilityPolicy: SingleReplica
  issuerURL: https://<issuer_url>.s3.<region>.amazonaws.com
  networking:
    #...
  olmCatalogPlacement: management
  platform:
    aws:
      #...
      resourceTags:
        - key: kubernetes.io/cluster/<tag>
          value: owned
      rolesRef:
        #...
    type: AWS

```

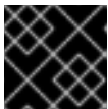
Replace **<tag>** with the tag that you want to add to your resource.

5.1.7. Configuring node pool capacity blocks on AWS

After you create a hosted cluster, you can configure node pool capacity blocks for graphics processing unit (GPU) reservations on Amazon Web Services (AWS).

Procedure

1. Create GPU reservations on AWS by running a command similar to the following example:



IMPORTANT

The zone of the GPU reservation must match your hosted cluster zone.

```

$ aws ec2 describe-capacity-block-offerings \
  --instance-type "p4d.24xlarge" \
  --instance-count "1" \
  --start-date-range "$(date -u +"2025-07-21T10:14:39Z")" \
  --end-date-range "$(date -u -d "2 day" +"2025-07-22T10:16:36Z")" \
  --capacity-duration-hours 24 \
  --output json

```

- **--instance-type** defines the type of your AWS instance.

- **--instance-count** defines your instance purchase quantity. Valid values are integers ranging from **1** to **64**.
- **--start-date-range** defines the start date range.
- **--end-date-range** defines the end date range.
- **--capacity-duration-hours** defines the duration of capacity blocks in hours.

2. Purchase the minimum fee capacity block by running the following command:

```
$ aws ec2 purchase-capacity-block \
  --capacity-block-offering-id "${MIN_FEE_ID}" \
  --instance-platform "Linux/UNIX" \
  --tag-specifications 'ResourceType=capacity-reservation,Tags=[{Key=usage-cluster-
type,Value=hypershift-hosted}]' \
  --output json > "${CR_OUTPUT_FILE}"
```

- **--capacity-block-offering-id** defines the ID of the capacity block offering.
- **--instance-platform** defines the platform of your instance.
- **--tag-specifications** defines the tag for your instance.

3. Create an environment variable to set the capacity reservation ID by running the following command:

```
$ CB_RESERVATION_ID=$(jq -r '.CapacityReservation.CapacityReservationId'
"${CR_OUTPUT_FILE}")
```

Wait for a couple of minutes for the GPU reservation to become available.

4. Add a node pool to use the GPU reservation by running the following command:

```
$ hcp create nodepool aws \
  --cluster-name <hosted_cluster_name> \
  --name <node_pool_name> \
  --node-count <node_pool_count> \
  --instance-type <instance_type> \
  --arch <arch_type> \
  --release-image <release_image> \
  --render > /tmp/np.yaml
```

- **--cluster-name** specifies the name of your hosted cluster.
- **--name** specifies the name of your node pool.
- **--node-count** defines the node pool count, for example, **1**.
- **--instance-type** defines the instance type, for example, **p4d.24xlarge**.
- **--arch** defines an architecture type, for example, **amd64**.
- **--release-image** specifies the release image you want to use.

5. Add the **capacityReservation** setting in your **NodePool** resource by using the following example configuration:

```
# ...
spec:
  arch: amd64
  clusterName: cb-np-hcp
  management:
    autoRepair: false
    upgradeType: Replace
  platform:
    aws:
      instanceProfile: cb-np-hcp-dqppw-worker
      instanceType: p4d.24xlarge
      rootVolume:
        size: 120
        type: gp3
      subnet:
        id: subnet-00000
      placement:
        capacityReservation:
          id: ${CB_RESERVATION_ID}
          marketType: CapacityBlocks
        type: AWS
# ...
```

6. Apply the node pool configuration by running the following command:

```
$ oc apply -f /tmp/np.yaml
```

Verification

1. Verify that your new node pool is created successfully by running the following command:

```
$ oc get np -n clusters
```

Example output

NAMESPACE	NAME	CLUSTER	DESIRED NODES	CURRENT NODES	AUTOSCALING	AUTOREPAIR	VERSION	UPDATINGVERSION	UPDATINGCONFIG	MESSAGE
clusters	cb-np	cb-np-hcp	1	False	False	False	4.22.0-0.nightly-			
	2025-06-05-224220	False	False							

2. Verify that your new compute nodes are created in the hosted cluster by running the following command:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-132-74.ec2.internal	Ready	worker	17m	v1.35.4
ip-10-0-134-183.ec2.internal	Ready	worker	4h5m	v1.35.4

5.1.7.1. Deleting a hosted cluster after configuring node pool capacity blocks

After you configure node pool capacity blocks, you can optionally delete a hosted cluster and uninstall the HyperShift Operator.

Procedure

1. To delete a hosted cluster, run a command similar to the following example:

```
$ hcp destroy cluster aws \
  --name cb-np-hcp \
  --aws-creds $HOME/.aws/credentials \
  --namespace clusters \
  --region us-east-2
```

2. To uninstall the HyperShift Operator, run the following command:

```
$ hcp install render --format=yaml | oc delete -f -
```

5.1.8. Amazon Spot Instance support for node pools

To reduce cloud infrastructure costs for non-critical and fault-tolerant workloads, you can use Amazon Spot Instances for your compute nodes in hosted clusters.

Spot Instances use spare Amazon Elastic Compute Cloud (Amazon EC2) capacity at reduced prices compared to on-demand instances. When Amazon EC2 needs the capacity block, it can interrupt a Spot Instance with a 2-minute warning. You can use Spot Instances for node pools in hosted clusters on AWS. However, you cannot combine Spot Instances with Amazon EC2 Capacity Reservations.



IMPORTANT

Spot Instances are suitable for fault-tolerant, stateless, and flexible workloads. Do not use Spot Instances for workloads that cannot tolerate interruptions. Spot Instances require default tenancy. Dedicated tenancy is not supported.

5.1.8.1. Configuring Amazon SQS and Amazon EventBridge to receive Amazon EC2 interruption events

Before you enable Amazon Spot Instances, you must set up Amazon Simple Queue Service (SQS) and Amazon EventBridge to receive Amazon EC2 interruption events.

For hosted control planes to be able to provide a graceful shut down, it needs to poll the SQS queue and cordon or drain nodes before they are deleted. For more information about Amazon SQS and Amazon EventBridge, see "Getting started with Amazon SQS" and "Getting started: Create an Amazon EventBridge bus rule".

Procedure

1. Create an Amazon SQS queue to receive Spot Instance notifications by entering the following command:

```

CLUSTER_NAME="my-cluster"
AWS_REGION="us-east-1"

$ aws sqs create-queue \
  --queue-name "${CLUSTER_NAME}-spot-interruption-queue" \
  --region "${AWS_REGION}"

```

- Be sure to replace **my-cluster** and **us-east-1** with your cluster name and AWS region.
 - Take note of the URL from the output. You need the URL when you create the hosted cluster.
2. Create Amazon EventBridge rules to route Amazon EC2 Spot interruption warnings and rebalance recommendations to the SQS queue.

- a. Set the attributes by entering the following command:

```

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
QUEUE_URL="https://sqs.${AWS_REGION}.amazonaws.com/${ACCOUNT_ID}/${CLUSTER_NAME}-spot-interruption-queue"

QUEUE_ARN=$(aws sqs get-queue-attributes \
  --queue-url "${QUEUE_URL}" \
  --attribute-names QueueArn \
  --query 'Attributes.QueueArn' --output text)

```

- b. Set the interruption warning by entering the following command:

```

$ aws events put-rule \
  --name "${CLUSTER_NAME}-spot-interruption-warning" \
  --event-pattern '{"source":["aws.ec2"],"detail-type":["EC2 Spot Instance Interruption Warning"]}' \
  --region "${AWS_REGION}"

```

- c. Set the target for the interruption warning by entering the following command:

```

$ aws events put-targets \
  --rule "${CLUSTER_NAME}-spot-interruption-warning" \
  --targets "Id=1,Arn=${QUEUE_ARN}" \
  --region "${AWS_REGION}"

```

- d. Set the recommendations by entering the following command:

```

$ aws events put-rule \
  --name "${CLUSTER_NAME}-rebalance-recommendation" \
  --event-pattern '{"source":["aws.ec2"],"detail-type":["EC2 Instance Rebalance Recommendation"]}' \
  --region "${AWS_REGION}"

```

- e. Set the target for the recommendations by entering the following command:

```

$ aws events put-targets \

```

```
--rule "${CLUSTER_NAME}-rebalance-recommendation" \
--targets "Id=1,Arn=${QUEUE_ARN}" \
--region "${AWS_REGION}"
```

3. Configure the Amazon SQS queue policy to allow EventBridge to send messages to the queue by entering the following command:

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
QUEUE_URL="https://sqs.${AWS_REGION}.amazonaws.com/${ACCOUNT_ID}/${CLUSTER_NAME}-spot-interruption-queue"

$ aws sqs set-queue-attributes \
  --queue-url "${QUEUE_URL}" \
  --attributes '{
    "Policy": "{ \"Version\": \"2012-10-17\", \"Statement\": [{ \"Effect\": \"Allow\", \"Principal\": { \"Service\": \"events.amazonaws.com\" }, \"Action\": \"sqs:SendMessage\", \"Resource\": \"${QUEUE_ARN}\" } ] }'"
  }'
```

Additional resources

- [Getting started with Amazon SQS](#)
- [Getting started: Create an Amazon EventBridge event bus rule](#)

5.1.8.2. Enabling Amazon Spot Instance support for node pools

Enable Amazon Spot Instances for your compute nodes in hosted cluster node pools to reduce cloud infrastructure costs for non-critical and fault-tolerant workloads.

Prerequisites

- You set up Amazon Simple Queue Service (SQS) and Amazon EventBridge to receive Amazon EC2 interruption events.
- You have the URL of your Amazon SQS queue, which you created in "Configuring Amazon SQS and Amazon EventBridge to receive Amazon EC2 interruption events".

Procedure

1. Set the Amazon SQS queue URL on your hosted cluster.
 - If you are setting the SQS queue URL on a new hosted cluster, take the following steps:
 - i. Configure the **HostedCluster** resource as follows:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <my_hosted_cluster>
  namespace: <my_hosted_cluster_namespace>
spec:
  platform:
    type: AWS
    aws:
```

```

    region: us-east-1
    terminationHandlerQueueURL: "https://sqs.us-east-
1.amazonaws.com/123456789012/my-cluster-spot-interruption-queue"
    ...

```

where:

spec.platform.aws.terminationHandlerQueueURL

Specifies the SQS queue URL.

- ii. Apply the configuration by entering the following command:

```
$ oc apply -f <hosted_cluster_config>.yaml
```

- If you are setting the SQS queue URL on an existing hosted cluster, patch the **HostedCluster** resource as follows:

```

$ oc patch hostedcluster my-cluster -n clusters --type merge -p '{
  "spec": {
    "platform": {
      "aws": {
        "terminationHandlerQueueURL": "https://sqs.us-east-
1.amazonaws.com/123456789012/my-cluster-spot-interruption-queue"
      }
    }
  }
}'

```

2. Create a **NodePool** object that has the Spot market type configured, as shown in the following example:

```

apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: spot-workers
  namespace: <my_hosted_cluster_namespace>
spec:
  clusterName: <my_hosted_cluster>
  replicas: 3
  release:
    image: quay.io/openshift-release-dev/ocp-release:4.22.0-x86_64
  management:
    autoRepair: true
    upgradeType: Replace
  platform:
    type: AWS
    aws:
      instanceType: m5.xlarge
      instanceProfile: my-cluster-worker
      rootVolume:
        size: 120
        type: gp3
      placement:
        marketType: Spot

```

marketType: Spot

Specifies that all **Machine** objects and **Node** objects have the **hypershift.openshift.io/interruptible-instance** label and that the EC2 instances have the **aws-node-termination-handler/managed** tag so they can be identified by the termination-handling components. You can specify **spot** options only when the **marketType** field is set to **Spot**.

3. Apply the configuration by entering the following command:

```
$ oc apply -f <node_pool_config>.yaml
```

5.2. MANAGING HOSTED CONTROL PLANES ON BARE METAL

After you deploy hosted control planes on bare metal, you can manage a hosted cluster.

5.2.1. Accessing the hosted cluster

You can access the hosted cluster by either getting the **kubeconfig** file and **kubeadmin** credential directly from resources, or by using the **hcp** command-line interface to generate a **kubeconfig** file.

Prerequisites

To access the hosted cluster by getting the **kubeconfig** file and credentials directly from resources, you must be familiar with the access secrets for hosted clusters. The *hosted cluster (hosting)* namespace has hosted cluster resources and the access secrets. The *hosted control plane* namespace is where the hosted control plane runs.

The secret name formats are as follows:

- **kubeconfig** secret: **<hosted_cluster_namespace>-<name>-admin-kubeconfig**. For example, **clusters-hypershift-demo-admin-kubeconfig**.
- **kubeadmin** password secret: **<hosted_cluster_namespace>-<name>-kubeadmin-password**. For example, **clusters-hypershift-demo-kubeadmin-password**.

The **kubeconfig** secret has a Base64-encoded **kubeconfig** field, which you can decode and save into a file to use with the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

The **kubeadmin** password secret is also Base64-encoded. You can decode it and use the password to log in to the API server or console of the hosted cluster.

Procedure

- To access the hosted cluster by using the **hcp** CLI to generate the **kubeconfig** file, take the following steps:

1. Generate the **kubeconfig** file by entering the following command:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \
  --name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

2. After you save the **kubeconfig** file, you can access the hosted cluster by entering the following example command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

5.2.2. Scaling the NodePool object for a hosted cluster

You can scale up the **NodePool** object by adding nodes to your hosted cluster.

When you scale a node pool, consider the following information:

- When you scale a replica by the node pool, a machine is created. For every machine, the Cluster API provider finds and installs an Agent that meets the requirements that are specified in the node pool specification. You can monitor the installation of an Agent by checking its status and conditions.
- When you scale down a node pool, Agents are unbound from the corresponding cluster. Before you can reuse the Agents, you must restart them by using the Discovery image.

Procedure

1. Scale the **NodePool** object to two nodes:

```
$ oc -n <hosted_cluster_namespace> scale nodepool <nodepool_name> --replicas 2
```

The Cluster API agent provider randomly picks two agents that are then assigned to the hosted cluster. Those agents go through different states and finally join the hosted cluster as OpenShift Container Platform nodes. The agents pass through states in the following order:

- **binding**
- **discovering**
- **insufficient**
- **installing**
- **installing-in-progress**
- **added-to-existing-cluster**

2. Enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
4dac1ab2-7dd5-4894-a220-6a3473b67ee6	hypercluster1	true	auto-assign	
d9198891-39f4-4930-a679-65fb142b108b		true	auto-assign	
da503cf1-a347-44f2-875c-4960ddb04091	hypercluster1	true	auto-assign	

3. Enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent \
-o jsonpath='{range .items[*]}BMH: {@.metadata.labels.agent-install\.openshift\.io/bmh}
Agent: {@.metadata.name} State: {@.status.debugInfo.state}{"\n"}{end}'
```

Example output

```
BMH: ocp-worker-2 Agent: 4dac1ab2-7dd5-4894-a220-6a3473b67ee6 State: binding
BMH: ocp-worker-0 Agent: d9198891-39f4-4930-a679-65fb142b108b State: known-unbound
BMH: ocp-worker-1 Agent: da503cf1-a347-44f2-875c-4960ddb04091 State: insufficient
```

- Obtain the kubeconfig for your new hosted cluster by entering the extract command:

```
$ oc extract -n <hosted_cluster_namespace> \
secret/<hosted_cluster_name>-admin-kubeconfig --to=- \
> kubeconfig-<hosted_cluster_name>
```

- After the agents reach the **added-to-existing-cluster** state, verify that you can see the OpenShift Container Platform nodes in the hosted cluster by entering the following command:

```
$ oc --kubeconfig kubeconfig-<hosted_cluster_name> get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ocp-worker-1	Ready	worker	5m41s	v1.24.0+3882f8f
ocp-worker-2	Ready	worker	6m3s	v1.24.0+3882f8f

Cluster Operators start to reconcile by adding workloads to the nodes.

- Enter the following command to verify that two machines were created when you scaled up the **NodePool** object:

```
$ oc -n <hosted_control_plane_namespace> get machines
```

Example output

NAME	CLUSTER	NODENAME	PROVIDERID
hypercluster1-c96b6f675-m5vch	hypercluster1-b2qhl	ocp-worker-1	agent://da503cf1-a347-44f2-875c-4960ddb04091
hypercluster1-c96b6f675-tl42p	hypercluster1-b2qhl	ocp-worker-2	agent://4dac1ab2-7dd5-4894-a220-6a3473b67ee6

The **clusterversion** reconcile process eventually reaches a point where only Ingress and Console cluster Operators are missing.

- Enter the following command:

```
$ oc --kubeconfig kubeconfig-<hosted_cluster_name> get clusterversion,co
```

Example output

■

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE	STATUS
clusterversion.config.openshift.io/version		False	True	40m	Unable to apply

4.x.z: the cluster operator console has not yet successfully rolled out

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE	MESSAGE
clusteroperator.config.openshift.io/console	4.12z	False	False	False	11m	RouteHealthAvailable: failed to GET route (https://console-openshift-console.apps.hypercluster1.domain.com): Get "https://console-openshift-console.apps.hypercluster1.domain.com": dial tcp 10.19.3.29:443: connect: connection refused
clusteroperator.config.openshift.io/csi-snapshot-controller	4.12z	True	False	False	10m	
clusteroperator.config.openshift.io/dns	4.12z	True	False	False	9m16s	

5.2.2.1. Adding node pools

You can create node pools for a hosted cluster by specifying a name, number of replicas, and any additional information, such as an agent label selector.



NOTE

Only a single agent namespace is supported for each hosted cluster. As a result, when you add a node pool to a hosted cluster, the node pool must be either from a single **InfraEnv** resource or from an **InfraEnv** resource that is in the same agent namespace.

Procedure

1. To create a node pool, enter the following information:

```
$ hcp create nodepool agent \
  --cluster-name <hosted_cluster_name> \
  --name <nodepool_name> \
  --node-count <worker_node_count> \
  --agentLabelSelector size=medium
```

- **--cluster-name** specifies your hosted cluster name, for example, **my-hosted-cluster**.
 - **--name** specifies the name of your node pool, for example, **my-hosted-cluster-extra-cpu**.
 - **--node-count** specifies the worker node count, for example, **2**.
 - **--agentLabelSelector** is optional. The node pool uses agents with the **size=medium** label.
2. Check the status of the node pool by listing **nodepool** resources in the **clusters** namespace:

```
$ oc get nodepools --namespace clusters
```

3. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_control_plane_namespace> secret/admin-kubeconfig --
to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

- After some time, you can check the status of the node pool by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

Verification

- Verify that the number of available node pools match the number of expected node pools by entering this command:

```
$ oc get nodepools --namespace clusters
```

5.2.2.2. Enabling node auto-scaling for the hosted cluster

When you need more capacity in your hosted cluster and spare agents are available, you can enable auto-scaling to install new worker nodes.

Procedure

- To enable auto-scaling, enter the following command:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p ' [{"op": "remove", "path": "/spec/replicas"}, {"op": "add", "path": "/spec/autoScaling",
    "value": { "max": 5, "min": 2 } } ]'
```



NOTE

In the example, the minimum number of nodes is 2, and the maximum is 5. The maximum number of nodes that you can add might be bound by your platform. For example, if you use the Agent platform, the maximum number of nodes is bound by the number of available agents.

- Create a workload that requires a new node.
 - Create a YAML file that has the workload configuration, by using the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: reversewords
  name: reversewords
  namespace: default
spec:
  replicas: 40
  selector:
    matchLabels:
      app: reversewords
```



```

strategy: {}
template:
  metadata:
    creationTimestamp: null
    labels:
      app: reversewords
  spec:
    containers:
      - image: quay.io/mavazque/reversewords:latest
        name: reversewords
    resources:
      requests:
        memory: 2Gi
status: {}

```

- b. Save the file as **workload-config.yaml**.
- c. Apply the YAML by entering the following command:

```
$ oc apply -f workload-config.yaml
```

3. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_cluster_namespace> \
  secret/<hosted_cluster_name>-admin-kubeconfig \
  --to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

4. You can check if new nodes are in the **Ready** status by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

5. To remove the node, delete the workload by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets -n <namespace> \
  delete deployment <deployment_name>
```

6. Wait for several minutes to pass without requiring the additional capacity. On the Agent platform, the agent is decommissioned and can be reused. You can confirm that the node was removed by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```



NOTE

For IBM Z® agents, if you are using an OSA network device in Processor Resource/Systems Manager (PR/SM) mode, auto scaling is not supported. You must delete the old agent manually and scale up the node pool because the new agent joins during the scale down process.

5.2.2.3. Disabling node auto-scaling for the hosted cluster

If needed, you can disable node auto-scaling.

Procedure

- Enter the following command to disable node auto-scaling for the hosted cluster:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p '[{"op": "remove", "path": "/spec/autoScaling"}, {"op": "add", "path": "/spec/replicas",
"value": <specify_value_to_scale_replicas>}]'
```

The command removes **"spec.autoScaling"** from the YAML file, adds **"spec.replicas"**, and sets **"spec.replicas"** to the integer value that you specify.

Additional resources

- [Scaling down the data plane to zero](#)
- [Scaling up and down workloads in a hosted cluster](#)

5.2.3. Handling ingress in a hosted cluster on bare metal

Every OpenShift Container Platform cluster has a default application Ingress Controller that typically has an external DNS record associated with it.

For example, if you create a hosted cluster named **example** with the base domain **krnl.es**, you can expect the wildcard domain ***.apps.example.krnl.es** to be routable.

To set up a load balancer and wildcard DNS record for the ***.apps** domain, perform the following actions on your hosted cluster.

Procedure

- Deploy MetalLB by creating a YAML file that has the configuration for the MetalLB Operator:

```
apiVersion: v1
kind: Namespace
metadata:
  name: metallb
  labels:
    openshift.io/cluster-monitoring: "true"
  annotations:
    workload.openshift.io/allowed: management
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator-operatorgroup
  namespace: metallb
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
```

```

name: metallb-operator
namespace: metallb
spec:
  channel: "stable"
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace

```

2. Save the file as **metallb-operator-config.yaml**.

3. Enter the following command to apply the configuration:

```
$ oc apply -f metallb-operator-config.yaml
```

4. After the Operator is running, create the MetalLB instance:

a. Create a YAML file that has the configuration for the MetalLB instance:

```

apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb

```

b. Save the file as **metallb-instance-config.yaml**.

c. Create the MetalLB instance by entering this command:

```
$ oc apply -f metallb-instance-config.yaml
```

5. Create an **IPAddressPool** resource with a single IP address. This IP address must be on the same subnet as the network that the cluster nodes use.

a. Create a file, such as **ipaddresspool.yaml**, with content similar to the following example:

```

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  namespace: metallb
  name: <ip_address_pool_name>
spec:
  addresses:
    - <ingress_ip>-<ingress_ip>
  autoAssign: false

```

- **metadata.name** specifies the **IPAddressPool** resource name.
- **spec.addresses** specifies the IP address for your environment. For example, **192.168.122.23**.

b. Apply the configuration for the IP address pool by entering the following command:

```
$ oc apply -f ipaddresspool.yaml
```

6. Create a L2 advertisement.

- a. Create a file, such as **l2advertisement.yaml**, with content similar to the following example:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: <l2_advertisement_name>
  namespace: metallb
spec:
  ipAddressPools:
    - <ip_address_pool_name>
```

- **metadata.name** specifies the **L2Advertisement** resource name.
- **spec.ipAddressPools** specifies the **IPAddressPool** resource name.

- b. Apply the configuration by entering the following command:

```
$ oc apply -f l2advertisement.yaml
```

7. After creating a service of the **LoadBalancer** type, MetalLB adds an external IP address for the service.

- a. Configure a new load balancer service that routes ingress traffic to the ingress deployment by creating a YAML file named **metallb-loadbalancer-service.yaml**:

```
kind: Service
apiVersion: v1
metadata:
  annotations:
    metallb.io/address-pool: ingress-public-ip
  name: metallb-ingress
  namespace: openshift-ingress
spec:
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
    - name: https
      protocol: TCP
      port: 443
      targetPort: 443
  selector:
    ingresscontroller.operator.openshift.io/deployment-ingresscontroller: default
  type: LoadBalancer
```

- b. Save the **metallb-loadbalancer-service.yaml** file.
- c. Enter the following command to apply the YAML configuration:

```
$ oc apply -f metallb-loadbalancer-service.yaml
```

- d. Enter the following command to reach the OpenShift Container Platform console:

```
$ curl -kI https://console-openshift-console.apps.example.krn1.es
```

Example output

```
HTTP/1.1 200 OK
```

- e. Check the **clusterversion** and **clusteroperator** values to verify that everything is running. Enter the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusterversion,co
```

Example output

```
NAME                                VERSION AVAILABLE PROGRESSING SINCE
STATUS
clusterversion.config.openshift.io/version 4.x.y   True    False    3m32s Cluster
version is 4.x.y

NAME                                VERSION AVAILABLE
PROGRESSING DEGRADED SINCE MESSAGE
clusteroperator.config.openshift.io/console 4.x.y   True    False
False    3m50s
clusteroperator.config.openshift.io/ingress 4.x.y   True    False
False    53m
```

Replace **<4.x.y>** with the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**.

Additional resources

- [About MetalLB and the MetalLB Operator](#)

5.2.4. Enabling machine health checks on bare metal

You can enable machine health checks on bare metal to repair and replace unhealthy managed cluster nodes automatically. You must have additional agent machines that are ready to install in the managed cluster.

Consider the following limitations before enabling machine health checks:

- You cannot modify the **MachineHealthCheck** object.
- Machine health checks replace nodes only when at least two nodes stay in the **False** or **Unknown** status for more than 8 minutes.

After you enable machine health checks for the managed cluster nodes, the **MachineHealthCheck** object is created in your hosted cluster.

To enable machine health checks in your hosted cluster, modify the **NodePool** resource.

Procedure

1. Verify that the **spec.nodeDrainTimeout** value in your **NodePool** resource is greater than **0s**. Replace **<hosted_cluster_namespace>** with the name of your hosted cluster namespace and **<nodepool_name>** with the node pool name. Run the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep nodeDrainTimeout
```

Example output

```
nodeDrainTimeout: 30s
```

2. If the **spec.nodeDrainTimeout** value is not greater than **0s**, modify the value by running the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec":{"nodeDrainTimeout": "30m"}}' --type=merge
```

3. Enable machine health checks by setting the **spec.management.autoRepair** field to **true** in the **NodePool** resource. Run the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec":{"management":{"autoRepair":true}}}' --type=merge
```

4. Verify that the **NodePool** resource is updated with the **autoRepair: true** value by running the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep autoRepair
```

5.2.5. Disabling machine health checks on bare metal

To disable machine health checks for the managed cluster nodes, modify the **NodePool** resource.

Procedure

1. Disable machine health checks by setting the **spec.management.autoRepair** field to **false** in the **NodePool** resource. Run the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec":{"management":{"autoRepair":false}}}' --type=merge
```

2. Verify that the **NodePool** resource is updated with the **autoRepair: false** value by running the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep autoRepair
```

Additional resources

- [Deploying machine health checks](#)

5.3. MANAGING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION

After you deploy a hosted cluster on OpenShift Virtualization, you can manage the cluster.

5.3.1. Accessing the hosted cluster

You can access the hosted cluster by either getting the **kubeconfig** file and **kubeadmin** credential directly from resources, or by using the **hcp** command-line interface to generate a **kubeconfig** file.

Prerequisites

To access the hosted cluster by getting the **kubeconfig** file and credentials directly from resources, you must be familiar with the access secrets for hosted clusters. The *hosted cluster (hosting)* namespace has hosted cluster resources and the access secrets. The *hosted control plane* namespace is where the hosted control plane runs.

The secret name formats are as follows:

- **kubeconfig** secret: **<hosted_cluster_namespace>-<name>-admin-kubeconfig** (clusters-hypershift-demo-admin-kubeconfig)
- **kubeadmin** password secret: **<hosted_cluster_namespace>-<name>-kubeadmin-password** (clusters-hypershift-demo-kubeadmin-password)

The **kubeconfig** secret has a Base64-encoded **kubeconfig** field, which you can decode and save into a file to use with the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

The **kubeadmin** password secret is also Base64-encoded. You can decode it and use the password to log in to the API server or console of the hosted cluster.

To access the hosted cluster by using the **hcp** CLI to generate the **kubeconfig** file, take the following steps.

Procedure

1. Generate the **kubeconfig** file by entering the following command:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \
  --name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

2. After you save the **kubeconfig** file, you can access the hosted cluster by entering the following example command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

5.3.2. Enabling node auto-scaling for the hosted cluster

When you need more capacity in your hosted cluster and spare agents are available, you can enable auto-scaling to install new worker nodes.

Procedure

Procedure

1. To enable auto-scaling, enter the following command:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p '[{"op": "remove", "path": "/spec/replicas"}, {"op": "add", "path": "/spec/autoScaling",
"value": { "max": 5, "min": 2 }}]'
```

**NOTE**

In the example, the minimum number of nodes is 2, and the maximum is 5. The maximum number of nodes that you can add might be bound by your platform. For example, if you use the Agent platform, the maximum number of nodes is bound by the number of available agents.

2. Create a workload that requires a new node.
 - a. Create a YAML file that has the workload configuration, by using the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: reversewords
  name: reversewords
  namespace: default
spec:
  replicas: 40
  selector:
    matchLabels:
      app: reversewords
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: reversewords
    spec:
      containers:
      - image: quay.io/mavazque/reversewords:latest
        name: reversewords
        resources:
          requests:
            memory: 2Gi
      status: {}
```

- b. Save the file as **workload-config.yaml**.
- c. Apply the YAML by entering the following command:

```
$ oc apply -f workload-config.yaml
```

3. Extract the **admin-kubeconfig** secret by entering the following command:


```
$ oc extract -n <hosted_cluster_namespace> \
secret/<hosted_cluster_name>-admin-kubeconfig \
--to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

4. You can check if new nodes are in the **Ready** status by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

5. To remove the node, delete the workload by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets -n <namespace> \
delete deployment <deployment_name>
```

6. Wait for several minutes to pass without requiring the additional capacity. On the Agent platform, the agent is decommissioned and can be reused. You can confirm that the node was removed by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```



NOTE

For IBM Z® agents, if you are using an OSA network device in Processor Resource/Systems Manager (PR/SM) mode, auto scaling is not supported. You must delete the old agent manually and scale up the node pool because the new agent joins during the scale down process.

5.3.3. Storage for hosted control planes on OpenShift Virtualization

If you do not provide any advanced storage configuration, the default storage class is used for the KubeVirt virtual machine (VM) images, the KubeVirt Container Storage Interface (CSI) mapping, and the etcd volumes.

The following table lists the capabilities that the infrastructure must provide to support persistent storage in a hosted cluster:

Table 5.1. Persistent storage modes in a hosted cluster

Infrastructure CSI provider	Hosted cluster CSI provider	Hosted cluster capabilities
Any RWX Block CSI provider	kubevirt-csi	- Basic RWO Block and File - Basic RWX Block and Snapshot - CSI volume cloning

Infrastructure CSI provider	Hosted cluster CSI provider	Hosted cluster capabilities
Any RWX Block CSI provider	Red Hat OpenShift Data Foundation	Red Hat OpenShift Data Foundation feature set. External mode has a smaller footprint and uses a standalone Red Hat Ceph Storage. Internal mode has a larger footprint, but is self-contained and suitable for use cases that require expanded capabilities such as RWX File .



NOTE

OpenShift Virtualization handles storage on hosted clusters, which especially helps customers whose requirements are limited to block storage.

5.3.3.1. Mapping KubeVirt CSI storage classes

You can map the infrastructure storage class to the hosted storage class during cluster creation.



NOTE

KubeVirt CSI supports mapping only an infrastructure storage class that is capable of **ReadWriteMany** (RWX) access.

The following table shows how volume and access mode capabilities map to KubeVirt CSI storage classes:

Table 5.2. Mapping KubeVirt CSI storage classes to access and volume modes

Infrastructure CSI capability	Hosted cluster CSI capability	VM live migration support	Notes
RWX: Block or Filesystem	ReadWriteOnce (RWO) Block or Filesystem RWX Block only	Supported	Use Block mode because Filesystem volume mode results in degraded hosted Block mode performance. RWX Block volume mode is supported only when the hosted cluster is OpenShift Container Platform 4.16 or later.

Infrastructure CSI capability	Hosted cluster CSI capability	VM live migration support	Notes
RWO Block storage	RWO Block storage or Filesystem	Not supported	Lack of live migration support affects the ability to update the underlying infrastructure cluster that hosts the KubeVirt VMs.
RWO Filesystem	RWO Block or Filesystem	Not supported	Lack of live migration support affects the ability to update the underlying infrastructure cluster that hosts the KubeVirt VMs. Use of the infrastructure Filesystem volume mode results in degraded hosted Block mode performance.

Procedure

- To map the infrastructure storage class to the hosted storage class, use the **--infra-storage-class-mapping** argument as shown in the following example:

```
$ hcp create cluster kubvirt \
  --name my-hosted-cluster \
  --node-pool-replicas 2 \
  --pull-secret /user/name/pullsecret \
  --memory 8Gi \
  --cores 2 \
  --infra-storage-class-mapping=<infrastructure_storage_class>/<hosted_storage_class>
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--infra-storage-class-mapping** specifies the storage class names for the infrastructure and hosted cluster. Replace **<infrastructure_storage_class>** with the infrastructure storage class name and **<hosted_storage_class>** with the hosted cluster storage class name. You can use the **--infra-storage-class-mapping** argument multiple times within the **hcp create cluster** command.

After you create the hosted cluster, the infrastructure storage class is visible within the hosted cluster. When you create a Persistent Volume Claim (PVC) within the hosted cluster that uses one of those storage classes, KubeVirt CSI provisions that volume by using the

infrastructure storage class mapping that you configured during cluster creation.

5.3.3.2. Mapping a single KubeVirt CSI volume snapshot class

You can expose your infrastructure volume snapshot class to the hosted cluster by using KubeVirt CSI.

Procedure

- To map your volume snapshot class to the hosted cluster, use the **--infra-volumesnapshot-class-mapping** argument when creating a hosted cluster as shown in the following example:

```
$ hcp create cluster kubevirt \
  --name my-hosted-cluster \
  --node-pool-replicas 2 \
  --pull-secret /user/name/pullsecret \
  --memory 8Gi \
  --cores 2 \
  --infra-storage-class-mapping=<infrastructure_storage_class>/<hosted_storage_class> \
  --infra-volumesnapshot-class-mapping=
<infrastructure_volume_snapshot_class>/<hosted_volume_snapshot_class>
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--infra-storage-class-mapping** specifies the storage classes. Replace **<infrastructure_storage_class>** with the storage class in the infrastructure cluster, and replace **<hosted_storage_class>** with the storage class in the hosted cluster.
- **--infra-volumesnapshot-class-mapping** specifies the volume snapshot classes. Replace **<infrastructure_volume_snapshot_class>** with the volume snapshot class in the infrastructure cluster, and replace **<hosted_volume_snapshot_class>** with the volume snapshot class in the hosted cluster.



NOTE

If you do not use the **--infra-storage-class-mapping** and **--infra-volumesnapshot-class-mapping** arguments, a hosted cluster is created with the default storage class and the volume snapshot class. Therefore, you must set the default storage class and the volume snapshot class in the infrastructure cluster.

5.3.3.3. Mapping multiple KubeVirt CSI volume snapshot classes

You can map multiple volume snapshot classes to the hosted cluster by assigning them to a specific group. The infrastructure storage class and the volume snapshot class are compatible with each other only if they belong to a same group.

Procedure

- To map multiple volume snapshot classes to the hosted cluster, use the **group** option when creating a hosted cluster, as shown in the following example:

```
$ hcp create cluster kubevirt \
  --name my-hosted-cluster \
  --node-pool-replicas 2 \
  --pull-secret /user/name/pullsecret \
  --memory 8Gi \
  --cores 2 \
  --infra-storage-class-mapping=
<infrastructure_storage_class>/<hosted_storage_class>,group=<group_name> \
  --infra-storage-class-mapping=
<infrastructure_storage_class>/<hosted_storage_class>,group=<group_name> \
  --infra-storage-class-mapping=
<infrastructure_storage_class>/<hosted_storage_class>,group=<group_name> \
  --infra-volumesnapshot-class-mapping=
<infrastructure_volume_snapshot_class>/<hosted_volume_snapshot_class>,group=
<group_name> \
  --infra-volumesnapshot-class-mapping=
<infrastructure_volume_snapshot_class>/<hosted_volume_snapshot_class>,group=
<group_name>
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--infra-storage-class-mapping** specifies the storage classes. Replace **<infrastructure_storage_class>** with the storage class in the infrastructure cluster, **<hosted_storage_class>** with the storage class in the hosted cluster, and **<group_name>** with the group name. For example, **infra-storage-class-mygroup/hosted-storage-class-mygroup,group=mygroup** and **infra-storage-class-mymap/hosted-storage-class-mymap,group=mymap**.
- **--infra-volumesnapshot-class-mapping** specifies the volume snapshot classes. Replace **<infrastructure_volume_snapshot_class>** with the volume snapshot class in the infrastructure cluster and **<hosted_volume_snapshot_class>** with the volume snapshot class in the hosted cluster. For example, **infra-vol-snap-mygroup/hosted-vol-snap-mygroup,group=mygroup** and **infra-vol-snap-mymap/hosted-vol-snap-mymap,group=mymap**.

5.3.3.4. Configuring KubeVirt VM root volume

At cluster creation time, you can configure the storage class that is used to host the KubeVirt virtual machine (VM) root volumes by using the **--root-volume-storage-class** argument.

Procedure

- To set a custom storage class and volume size for KubeVirt VMs, run a command as shown in the following example:

```
$ hcp create cluster kubevirt \  
  --name my-hosted-cluster \  
  --node-pool-replicas 2 \  
  --pull-secret /user/name/pullsecret \  
  --memory 8Gi \  
  --cores 2 \  
  --root-volume-storage-class ocs-storagecluster-ceph-rbd \  
  --root-volume-size 64
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--root-volume-storage-class** specifies a name of the storage class to host the KubeVirt VM root volumes.
- **--root-volume-size** specifies the volume size.
As a result, you get a hosted cluster created with VMs hosted on persistent volume claims (PVCs).

5.3.3.5. Enabling KubeVirt VM image caching

To optimize both cluster startup time and storage usage, you can use KubeVirt virtual machine (VM) image caching.

KubeVirt VM image caching supports the use of a storage class that is capable of smart cloning and the **ReadWriteMany** access mode. For more information about smart cloning, see "Cloning a data volume using smart-cloning".

Image caching works as follows:

1. The VM image is imported to a persistent volume claim (PVC) that is associated with the hosted cluster.
2. A unique clone of that PVC is created for every KubeVirt VM that is added as a worker node to the cluster.

Image caching reduces VM startup time by requiring only a single image import. It can further reduce overall cluster storage usage when the storage class supports copy-on-write cloning.

Procedure

- To enable image caching, during cluster creation, use the **--root-volume-cache-strategy=PVC** argument by running a command as shown in the following example:

```
$ hcp create cluster kubevirt \  
  --root-volume-cache-strategy=PVC
```

```
--name my-hosted-cluster \
--node-pool-replicas 2 \
--pull-secret /user/name/pullsecret \
--memory 8Gi \
--cores 2 \
--root-volume-cache-strategy=PVC
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--root-volume-cache-strategy** specifies a strategy for image caching.

Additional resources

- [Cloning a data volume using smart-cloning](#)

5.3.3.6. KubeVirt CSI storage security and isolation

KubeVirt Container Storage Interface (CSI) extends the storage capabilities of the underlying infrastructure cluster to hosted clusters.

The CSI driver ensures secure and isolated access to the infrastructure storage classes and hosted clusters by using the following security constraints:

- The storage of a hosted cluster is isolated from the other hosted clusters.
- Compute nodes in a hosted cluster do not have a direct API access to the infrastructure cluster. The hosted cluster can provision storage on the infrastructure cluster only through the controlled KubeVirt CSI interface.
- The hosted cluster does not have access to the KubeVirt CSI cluster controller. As a result, the hosted cluster cannot access arbitrary storage volumes on the infrastructure cluster that are not associated with the hosted cluster. The KubeVirt CSI cluster controller runs in a pod in the hosted control plane namespace.
- Role-based access control (RBAC) of the KubeVirt CSI cluster controller limits the persistent volume claim (PVC) access to only the hosted control plane namespace. Therefore, KubeVirt CSI components cannot access storage from the other namespaces.

5.3.3.7. Configuring etcd storage

At cluster creation time, you can configure the storage class that is used to host etcd data by using the **-etcd-storage-class** argument.

Procedure

- To configure a storage class for etcd, run a command similar to the following example:

```
$ hcp create cluster kubevirt \
  --name my-hosted-cluster \
  --node-pool-replicas 2 \
  --pull-secret /user/name/pullsecret \
  --memory 8Gi \
  --cores 2 \
  --etcd-storage-class=lvm-storageclass
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--etcd-storage-class** specifies the etcd storage class name. If you do not provide an **--etcd-storage-class** argument, the default storage class is used.

5.3.4. Attaching NVIDIA GPU devices by using the hcp CLI

You can attach one or more NVIDIA graphics processing unit (GPU) devices to node pools by using the **hcp** command-line interface (CLI) in a hosted cluster on OpenShift Virtualization.



IMPORTANT

Attaching NVIDIA GPU devices to node pools is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have exposed the NVIDIA GPU device as a resource on the node where the GPU device resides. For more information, see [NVIDIA GPU Operator with OpenShift Virtualization](#).
- You have exposed the NVIDIA GPU device as an [extended resource](#) on the node to assign it to node pools.

Procedure

- You can attach the GPU device to node pools during cluster creation by running a command similar to the following example:

```
$ hcp create cluster kubevirt \
  --name my-hosted-cluster \
  --node-pool-replicas 3 \
```



```
--pull-secret /user/name/pullsecret \
--memory 16Gi \
--cores 2 \
--host-device-name="nvidia-a100,count:2"
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count.
- **--pull-secret** specifies the path to your pull secret.
- **--memory** specifies a value for memory.
- **--cores** specifies a value for CPU.
- **--host-device-name** specifies the GPU device name and the count. The **--host-device-name** argument takes the name of the GPU device from the infrastructure node and an optional count that represents the number of GPU devices you want to attach to each virtual machine (VM) in node pools. The default count is **1**. For example, if you attach 2 GPU devices to 3 node pool replicas, all 3 VMs in the node pool are attached to the 2 GPU devices.

TIP

You can use the **--host-device-name** argument multiple times to attach multiple devices of different types.

5.3.5. Attaching NVIDIA GPU devices by using the NodePool resource

You can attach one or more NVIDIA graphics processing unit (GPU) devices to node pools by configuring the **nodepool.spec.platform.kubevirt.hostDevices** field in the **NodePool** resource.

IMPORTANT

Attaching NVIDIA GPU devices to node pools is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Procedure

- To attach a single GPU device, configure the **NodePool** resource by using the following example configuration:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
```

```
spec:
  arch: amd64
  clusterName: <hosted_cluster_name>
  management:
    autoRepair: false
    upgradeType: Replace
  nodeDrainTimeout: 0s
  nodeVolumeDetachTimeout: 0s
  platform:
    kubevirt:
      attachDefaultNetwork: true
    compute:
      cores: <cpu>
      memory: <memory>
    hostDevices:
      - count: <count>
        deviceName: <gpu_device_name>
    networkInterfaceMultiqueue: Enable
    rootVolume:
      persistent:
        size: 32Gi
      type: Persistent
    type: KubeVirt
  replicas: <worker_node_count>
```

- **<hosted_cluster_name>** specifies the name of your hosted cluster; for example, **my-hosted-cluster**.
- **<hosted_cluster_namespace>** specifies the name of the hosted cluster namespace; for example, **my-hc-namespace**.
- **<cpu>** specifies a value for CPU; for example, **2**.
- **<memory>** specifies a value for memory; for example, **16Gi**.
- **<count>** specifies the number of GPU devices you want to attach to each virtual machine (VM) in node pools. For example, if you attach 2 GPU devices to 3 node pool replicas, all 3 VMs in the node pool are attached to the 2 GPU devices. The default count is **1**. The **hostDevices** field defines a list of different types of GPU devices that you can attach to node pools.
- **<gpu_device_name>** specifies the GPU device name; for example, **nvidia-a100**.
- **<worker_node_count>** specifies the worker count; for example, **3**.
- To attach multiple GPU devices, configure the **NodePool** resource by using the following example configuration:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  arch: amd64
  clusterName: <hosted_cluster_name>
```

```

management:
  autoRepair: false
  upgradeType: Replace
nodeDrainTimeout: 0s
nodeVolumeDetachTimeout: 0s
platform:
  kubevirt:
    attachDefaultNetwork: true
    compute:
      cores: <cpu>
      memory: <memory>
    hostDevices:
      - count: <count>
        deviceName: <gpu_device_name>
      - count: <count>
        deviceName: <gpu_device_name>
      - count: <count>
        deviceName: <gpu_device_name>
      - count: <count>
        deviceName: <gpu_device_name>
    networkInterfaceMultiqueue: Enable
    rootVolume:
      persistent:
        size: 32Gi
        type: Persistent
      type: KubeVirt
  replicas: <worker_node_count>

```

- **<hosted_cluster_name>** specifies the name of your hosted cluster; for example, **my-hosted-cluster**.
- **<hosted_cluster_namespace>** specifies the name of the hosted cluster namespace; for example, **my-hc-namespace**.
- **<cpu>** specifies a value for CPU; for example, **2**.
- **<memory>** specifies a value for memory; for example, **16Gi**.
- **<count>** specifies the number of GPU devices you want to attach to each VM in node pools. For example, if you attach 2 GPU devices to 3 node pool replicas, all 3 VMs in the node pool are attached to the 2 GPU devices. The default count is **1**. The **hostDevices** field defines a list of different types of GPU devices that you can attach to node pools.
- **<gpu_device_name>** specifies the GPU device name; for example, **nvidia-a100**.
- **<worker_node_count>** specifies the worker count; for example, **3**.

5.3.6. Evicting KubeVirt virtual machines

In cases where KubeVirt virtual machines (VMs) cannot be live migrated, such as when you use GPU passthrough, the VMs must be evicted at the same time as the **NodePool** resource of the hosted cluster.

Otherwise, the compute nodes might be shut down without being drained from the workload. This might also happen when you are upgrading the OpenShift Virtualization Operator.

To achieve a synchronized restart, you can set the **evictionStrategy** parameter on the **hyperconverged** resource to ensure that only VMs that are drained from workloads are rebooted.

Procedure

1. To learn more about the **hyperconverged** resource and the allowed values for the **evictionStrategy** parameter, enter the following command:

```
$ oc explain --api-version=hco.kubevirt.io/v1beta1 hyperconverged.spec.evictionStrategy
```

2. Patch the **hyperconverged** resource by entering the following command:

```
$ oc patch -n openshift-cnv hyperconvergeds.v1beta1.hco.kubevirt.io kubevirt-  
hyperconverged \  
--type=merge \  
-p '{"spec": {"evictionStrategy": "External"}}'
```

3. Patch the workload update strategy and the workload update methods by entering the following command:

```
$ oc patch -n openshift-cnv hyperconvergeds.v1beta1.hco.kubevirt.io kubevirt-  
hyperconverged \  
--type=merge \  
-p '{"spec": {"workloadUpdateStrategy": {"workloadUpdateMethods":  
["LiveMigrate", "Evict"]}}}'
```

By applying this patch, you specify that VMs should be live-migrated if possible, and that only the VMs that cannot be live-migrated should be evicted.

Verification

- Check whether the patch command was applied properly by entering the following command:

```
$ oc get -n openshift-cnv hyperconvergeds.v1beta1.hco.kubevirt.io kubevirt-hyperconverged  
-ojsonpath='{.spec.evictionStrategy}'
```

Example output

```
External
```

5.3.7. Spreading node pool VMs by using topologySpreadConstraint

In some scenarios, node pool virtual machines (VMs) might run on the same node, which can cause availability issues. To avoid distribution of VMs on a single node, use the descheduler to continuously honor the **topologySpreadConstraint** constraint to spread VMs on multiple nodes.

By default, KubeVirt VMs created by a node pool are scheduled on any available nodes that have the capacity to run the VMs. The **topologySpreadConstraint** constraint is set to schedule VMs on multiple nodes.

Prerequisites

- You installed the Kube Descheduler Operator. For more information, see "Installing the descheduler".

Procedure

- Open the **KubeDescheduler** custom resource (CR) by entering the following command, and then modify the **KubeDescheduler** CR to use the **SoftTopologyAndDuplicates** and **KubeVirtRelieveAndMigrate** profiles so that you maintain the **topologySpreadConstraint** constraint settings.
The **KubeDescheduler** CR named **cluster** runs in the **openshift-kube-descheduler-operator** namespace.

```
$ oc edit kubedescheduler cluster -n openshift-kube-descheduler-operator
```

Example KubeDescheduler configuration

```
apiVersion: operator.openshift.io/v1
kind: KubeDescheduler
metadata:
  name: cluster
  namespace: openshift-kube-descheduler-operator
spec:
  mode: Automatic
  managementState: Managed
  deschedulingIntervalSeconds: 30
  profiles:
    - SoftTopologyAndDuplicates
    - KubeVirtRelieveAndMigrate
  profileCustomizations:
    devDeviationThresholds: AsymmetricLow
    devActualUtilizationProfile: PrometheusCPUCombined
# ...
```

where:

spec.deschedulingIntervalSeconds

Sets the number of seconds between the descheduler running cycles.

spec.profiles

The **SoftTopologyAndDuplicates** profile evicts pods that follow the **whenUnsatisfiable: ScheduleAnyway** soft topology constraint. The **KubeVirtRelieveAndMigrate** profile balances resource usage between nodes and enables strategies, such as **RemovePodsHavingTooManyRestarts** and **LowNodeUtilization**.

Additional resources

- [Installing the descheduler](#)

5.4. MANAGING HOSTED CONTROL PLANES ON NON-BARE-METAL AGENT MACHINES

After you deploy hosted control planes on non-bare-metal agent machines, you can manage a hosted cluster.

5.4.1. Accessing the hosted cluster

You can access the hosted cluster by either getting the **kubeconfig** file and **kubeadmin** credential directly from resources, or by using the **hcp** command-line interface to generate a **kubeconfig** file.

Prerequisites

To access the hosted cluster by getting the **kubeconfig** file and credentials directly from resources, you must be familiar with the access secrets for hosted clusters. The *hosted cluster (hosting)* namespace has hosted cluster resources and the access secrets. The *hosted control plane* namespace is where the hosted control plane runs.

The secret name formats are as follows:

- **kubeconfig** secret: **<hosted_cluster_namespace>-<name>-admin-kubeconfig**. For example, **clusters-hypershift-demo-admin-kubeconfig**.
- **kubeadmin** password secret: **<hosted_cluster_namespace>-<name>-kubeadmin-password**. For example, **clusters-hypershift-demo-kubeadmin-password**.

The **kubeconfig** secret has a Base64-encoded **kubeconfig** field, which you can decode and save into a file to use with the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

The **kubeadmin** password secret is also Base64-encoded. You can decode it and use the password to log in to the API server or console of the hosted cluster.

Procedure

- To access the hosted cluster by using the **hcp** CLI to generate the **kubeconfig** file, take the following steps:

1. Generate the **kubeconfig** file by entering the following command:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \  
--name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

2. After you save the **kubeconfig** file, you can access the hosted cluster by entering the following example command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

5.4.2. Scaling the NodePool object for a hosted cluster

You can scale up the **NodePool** object by adding nodes to your hosted cluster.

When you scale a node pool, consider the following information:

- When you scale a replica by the node pool, a machine is created. For every machine, the Cluster API provider finds and installs an Agent that meets the requirements that are specified in the node pool specification. You can monitor the installation of an Agent by checking its status and conditions.

- When you scale down a node pool, Agents are unbound from the corresponding cluster. Before you can reuse the Agents, you must restart them by using the Discovery image.

Procedure

1. Scale the **NodePool** object to two nodes:

```
$ oc -n <hosted_cluster_namespace> scale nodepool <nodepool_name> --replicas 2
```

The Cluster API agent provider randomly picks two agents that are then assigned to the hosted cluster. Those agents go through different states and finally join the hosted cluster as OpenShift Container Platform nodes. The agents pass through states in the following order:

- **binding**
- **discovering**
- **insufficient**
- **installing**
- **installing-in-progress**
- **added-to-existing-cluster**

2. Enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
4dac1ab2-7dd5-4894-a220-6a3473b67ee6	hypercluster1	true		auto-assign
d9198891-39f4-4930-a679-65fb142b108b		true		auto-assign
da503cf1-a347-44f2-875c-4960ddb04091	hypercluster1	true		auto-assign

3. Enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agent \
-o jsonpath='{range .items[*]}BMH: {@.metadata.labels.agent-install.openshift.io/bmh}
Agent: {@.metadata.name} State: {@.status.debugInfo.state}{"\n"}{end}'
```

Example output

```
BMH: ocp-worker-2 Agent: 4dac1ab2-7dd5-4894-a220-6a3473b67ee6 State: binding
BMH: ocp-worker-0 Agent: d9198891-39f4-4930-a679-65fb142b108b State: known-unbound
BMH: ocp-worker-1 Agent: da503cf1-a347-44f2-875c-4960ddb04091 State: insufficient
```

4. Obtain the kubeconfig for your new hosted cluster by entering the extract command:

```
$ oc extract -n <hosted_cluster_namespace> \
secret/<hosted_cluster_name>-admin-kubeconfig --to=- \
> kubeconfig-<hosted_cluster_name>
```

- After the agents reach the **added-to-existing-cluster** state, verify that you can see the OpenShift Container Platform nodes in the hosted cluster by entering the following command:

```
$ oc --kubeconfig kubeconfig-<hosted_cluster_name> get nodes
```

Example output

```
NAME          STATUS  ROLES  AGE   VERSION
ocp-worker-1  Ready   worker  5m41s v1.24.0+3882f8f
ocp-worker-2  Ready   worker  6m3s  v1.24.0+3882f8f
```

Cluster Operators start to reconcile by adding workloads to the nodes.

- Enter the following command to verify that two machines were created when you scaled up the **NodePool** object:

```
$ oc -n <hosted_control_plane_namespace> get machines
```

Example output

```
NAME          CLUSTER          NODENAME  PROVIDERID
PHASE  AGE  VERSION
hypercluster1-c96b6f675-m5vch hypercluster1-b2qhl ocp-worker-1 agent://da503cf1-
a347-44f2-875c-4960ddb04091 Running 15m 4.x.z
hypercluster1-c96b6f675-tl42p hypercluster1-b2qhl ocp-worker-2 agent://4dac1ab2-
7dd5-4894-a220-6a3473b67ee6 Running 15m 4.x.z
```

The **clusterversion** reconcile process eventually reaches a point where only Ingress and Console cluster Operators are missing.

- Enter the following command:

```
$ oc --kubeconfig kubeconfig-<hosted_cluster_name> get clusterversion,co
```

Example output

```
NAME          VERSION  AVAILABLE  PROGRESSING  SINCE
STATUS
clusterversion.config.openshift.io/version      False      True         40m  Unable to apply
4.x.z: the cluster operator console has not yet successfully rolled out

NAME          VERSION  AVAILABLE
PROGRESSING  DEGRADED  SINCE  MESSAGE
clusteroperator.config.openshift.io/console      4.12z  False  False
False  11m  RouteHealthAvailable: failed to GET route (https://console-openshift-
console.apps.hypercluster1.domain.com): Get "https://console-openshift-
console.apps.hypercluster1.domain.com": dial tcp 10.19.3.29:443: connect: connection
refused
clusteroperator.config.openshift.io/csi-snapshot-controller  4.12z  True
False  False  10m
clusteroperator.config.openshift.io/dns              4.12z  True  False
False  9m16s
```


5.4.2.1. Adding node pools

You can create node pools for a hosted cluster by specifying a name, number of replicas, and any additional information, such as an agent label selector.



NOTE

Only a single agent namespace is supported for each hosted cluster. As a result, when you add a node pool to a hosted cluster, the node pool must be either from a single **InfraEnv** resource or from an **InfraEnv** resource that is in the same agent namespace.

Procedure

1. To create a node pool, enter the following information:

```
$ hcp create nodepool agent \
  --cluster-name <hosted_cluster_name> \
  --name <nodepool_name> \
  --node-count <worker_node_count> \
  --agentLabelSelector size=medium
```

- **--cluster-name** specifies your hosted cluster name, for example, **my-hosted-cluster**.
- **--name** specifies the name of your node pool, for example, **my-hosted-cluster-extra-cpu**.
- **--node-count** specifies the worker node count, for example, **2**.
- **--agentLabelSelector** is optional. The node pool uses agents with the **size=medium** label.

2. Check the status of the node pool by listing **nodepool** resources in the **clusters** namespace:

```
$ oc get nodepools --namespace clusters
```

3. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_control_plane_namespace> secret/admin-kubeconfig --
to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

4. After some time, you can check the status of the node pool by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

Verification

- Verify that the number of available node pools match the number of expected node pools by entering this command:

```
$ oc get nodepools --namespace clusters
```

5.4.2.2. Enabling node auto-scaling for the hosted cluster

When you need more capacity in your hosted cluster and spare agents are available, you can enable auto-scaling to install new worker nodes.

Procedure

1. To enable auto-scaling, enter the following command:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p ' [{"op": "remove", "path": "/spec/replicas"}, {"op": "add", "path": "/spec/autoScaling",
"value": { "max": 5, "min": 2 }} ]'
```



NOTE

In the example, the minimum number of nodes is 2, and the maximum is 5. The maximum number of nodes that you can add might be bound by your platform. For example, if you use the Agent platform, the maximum number of nodes is bound by the number of available agents.

2. Create a workload that requires a new node.
 - a. Create a YAML file that has the workload configuration, by using the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: reversewords
  name: reversewords
  namespace: default
spec:
  replicas: 40
  selector:
    matchLabels:
      app: reversewords
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: reversewords
    spec:
      containers:
      - image: quay.io/mavazque/reversewords:latest
        name: reversewords
        resources:
          requests:
            memory: 2Gi
      status: {}
```

- b. Save the file as **workload-config.yaml**.

- c. Apply the YAML by entering the following command:

```
$ oc apply -f workload-config.yaml
```

3. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_cluster_namespace> \
  secret/<hosted_cluster_name>-admin-kubeconfig \
  --to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

4. You can check if new nodes are in the **Ready** status by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

5. To remove the node, delete the workload by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets -n <namespace> \
  delete deployment <deployment_name>
```

6. Wait for several minutes to pass without requiring the additional capacity. On the Agent platform, the agent is decommissioned and can be reused. You can confirm that the node was removed by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```



NOTE

For IBM Z® agents, if you are using an OSA network device in Processor Resource/Systems Manager (PR/SM) mode, auto scaling is not supported. You must delete the old agent manually and scale up the node pool because the new agent joins during the scale down process.

5.4.2.3. Disabling node auto-scaling for the hosted cluster

If needed, you can disable node auto-scaling.

Procedure

- Enter the following command to disable node auto-scaling for the hosted cluster:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p '[{"op": "remove", "path": "/spec/autoScaling"}, {"op": "add", "path": "/spec/replicas", "value": <specify_value_to_scale_replicas>}]
```

The command removes **"spec.autoScaling"** from the YAML file, adds **"spec.replicas"**, and sets **"spec.replicas"** to the integer value that you specify.

Additional resources

- [Scaling down the data plane to zero](#)
- [Scaling up and down workloads in a hosted cluster](#)

5.4.3. Handling ingress in a hosted cluster on non-bare-metal agent machines

Every OpenShift Container Platform cluster has a default application Ingress Controller that typically has an external DNS record associated with it.

For example, if you create a hosted cluster named **example** with the base domain **krnl.es**, you can expect the wildcard domain ***.apps.example.krnl.es** to be routable.

To set up a load balancer and wildcard DNS record for the ***.apps** domain, perform the following actions on your hosted cluster.

Procedure

1. Deploy MetalLB by creating a YAML file that has the configuration for the MetalLB Operator:

```
apiVersion: v1
kind: Namespace
metadata:
  name: metallb
  labels:
    openshift.io/cluster-monitoring: "true"
  annotations:
    workload.openshift.io/allowed: management
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator-operatorgroup
  namespace: metallb
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: metallb-operator
  namespace: metallb
spec:
  channel: "stable"
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

2. Save the file as **metallb-operator-config.yaml**.
3. Enter the following command to apply the configuration:

```
$ oc apply -f metallb-operator-config.yaml
```

4. After the Operator is running, create the MetalLB instance:

- a. Create a YAML file that has the configuration for the MetalLB instance:

```
apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb
```

- b. Save the file as **metallb-instance-config.yaml**.
- c. Create the MetalLB instance by entering this command:

```
$ oc apply -f metallb-instance-config.yaml
```

5. Create an **IPAddressPool** resource with a single IP address. This IP address must be on the same subnet as the network that the cluster nodes use.

- a. Create a file, such as **ipaddresspool.yaml**, with content similar to the following example:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  namespace: metallb
  name: <ip_address_pool_name>
spec:
  addresses:
    - <ingress_ip>-<ingress_ip>
  autoAssign: false
```

- **metadata.name** specifies the **IPAddressPool** resource name.
- **spec.addresses** specifies the IP address for your environment. For example, **192.168.122.23**.

- b. Apply the configuration for the IP address pool by entering the following command:

```
$ oc apply -f ipaddresspool.yaml
```

6. Create a L2 advertisement.

- a. Create a file, such as **l2advertisement.yaml**, with content similar to the following example:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: <l2_advertisement_name>
  namespace: metallb
spec:
  ipAddressPools:
    - <ip_address_pool_name>
```

- **metadata.name** specifies the **L2Advertisement** resource name.
- **spec.ipAddressPools** specifies the **IPAddressPool** resource name.

- b. Apply the configuration by entering the following command:

```
$ oc apply -f l2advertisement.yaml
```

7. After creating a service of the **LoadBalancer** type, MetalLB adds an external IP address for the service.

- a. Configure a new load balancer service that routes ingress traffic to the ingress deployment by creating a YAML file named **metallb-loadbalancer-service.yaml**:

```
kind: Service
apiVersion: v1
metadata:
  annotations:
    metallb.io/address-pool: ingress-public-ip
  name: metallb-ingress
  namespace: openshift-ingress
spec:
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
    - name: https
      protocol: TCP
      port: 443
      targetPort: 443
  selector:
    ingresscontroller.operator.openshift.io/deployment-ingresscontroller: default
  type: LoadBalancer
```

- b. Save the **metallb-loadbalancer-service.yaml** file.
- c. Enter the following command to apply the YAML configuration:

```
$ oc apply -f metallb-loadbalancer-service.yaml
```

- d. Enter the following command to reach the OpenShift Container Platform console:

```
$ curl -kI https://console-openshift-console.apps.example.krn1.es
```

Example output

```
HTTP/1.1 200 OK
```

- e. Check the **clusterversion** and **clusteroperator** values to verify that everything is running. Enter the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusterversion,co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE
------	---------	-----------	-------------	-------

STATUS

```
clusterversion.config.openshift.io/version 4.x.y True False 3m32s Cluster
version is 4.x.y
```

NAME**VERSION AVAILABLE****PROGRESSING DEGRADED SINCE MESSAGE**

```
clusteroperator.config.openshift.io/console 4.x.y True False
False 3m50s
clusteroperator.config.openshift.io/ingress 4.x.y True False
False 53m
```

Replace **<4.x.y>** with the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**.

Additional resources

- [About MetalLB and the MetalLB Operator](#)

5.4.4. Enabling machine health checks on non-bare-metal agent machines

You can enable machine health checks on bare metal to repair and replace unhealthy managed cluster nodes automatically. You must have additional agent machines that are ready to install in the managed cluster.

Consider the following limitations before enabling machine health checks:

- You cannot modify the **MachineHealthCheck** object.
- Machine health checks replace nodes only when at least two nodes stay in the **False** or **Unknown** status for more than 8 minutes.

After you enable machine health checks for the managed cluster nodes, the **MachineHealthCheck** object is created in your hosted cluster.

To enable machine health checks in your hosted cluster, modify the **NodePool** resource.

Procedure

1. Verify that the **spec.nodeDrainTimeout** value in your **NodePool** resource is greater than **0s**. Replace **<hosted_cluster_namespace>** with the name of your hosted cluster namespace and **<nodepool_name>** with the node pool name. Run the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep
nodeDrainTimeout
```

Example output

```
nodeDrainTimeout: 30s
```

2. If the **spec.nodeDrainTimeout** value is not greater than **0s**, modify the value by running the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec":
{"nodeDrainTimeout": "30m"}}' --type=merge
```

-
- 3. Enable machine health checks by setting the **spec.management.autoRepair** field to **true** in the **NodePool** resource. Run the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec": {"management": {"autoRepair": true}}}' --type=merge
```

- 4. Verify that the **NodePool** resource is updated with the **autoRepair: true** value by running the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep autoRepair
```

5.4.5. Disabling machine health checks on non-bare-metal agent machines

To disable machine health checks for the managed cluster nodes, modify the **NodePool** resource.

Procedure

- 1. Disable machine health checks by setting the **spec.management.autoRepair** field to **false** in the **NodePool** resource. Run the following command:

```
$ oc patch nodepool -n <hosted_cluster_namespace> <nodepool_name> -p '{"spec": {"management": {"autoRepair": false}}}' --type=merge
```

- 2. Verify that the **NodePool** resource is updated with the **autoRepair: false** value by running the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -o yaml | grep autoRepair
```

Additional resources

- [Deploying machine health checks](#)

5.5. MANAGING HOSTED CONTROL PLANES ON IBM POWER

After you deploy hosted control planes on IBM Power, you can manage a hosted cluster.

5.5.1. Creating an InfraEnv resource for hosted control planes on IBM Power

An **InfraEnv** resource is an environment where hosts that are starting the live ISO can join as agents. In this case, the agents are created in the same namespace as your hosted control plane.

You can create an **InfraEnv** resource for hosted control planes on 64-bit x86 bare metal for IBM Power compute nodes.

Procedure

- 1. Create a YAML file to configure an **InfraEnv** resource. See the following example:

```
apiVersion: agent-install.openshift.io/v1beta1
```



```

kind: InfraEnv
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_control_plane_namespace>
spec:
  cpuArchitecture: ppc64le
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <path_to_ssh_public_key>

```

- **metadata.name** specifies the name of your hosted cluster.
- **metadata.namespace** specifies the name of the hosted control plane namespace, for example, **clusters-hosted**.
- **spec.sshAuthorizedKey** specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.

2. Save the file as **infraenv-config.yaml**.

3. Apply the configuration by entering the following command:

```
$ oc apply -f infraenv-config.yaml
```

4. To fetch the URL to download the live ISO, which allows IBM Power machines to join as agents, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get InfraEnv <hosted_cluster_name> \
-o json
```

5.5.2. Adding IBM Power agents to the InfraEnv resource

You can add agents by manually configuring the machine to start with the live ISO.

Procedure

1. Download the live ISO and use it to start a bare metal or a virtual machine (VM) host. You can find the URL for the live ISO in the **status.isoDownloadURL** field in the **InfraEnv** resource. At startup, the host communicates with the Assisted Service and registers as an agent in the same namespace as the **InfraEnv** resource.
2. To list the agents and some of their properties, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
86f7ac75-4fc4-4b36-8130-40fa12602218			auto-assign	
e57a637f-745b-496e-971d-1abbf03341ba			auto-assign	

3. After each agent is created, you can optionally set the **installation_disk_id** and **hostname** for an agent:

- a. To set the **installation_disk_id** field for an agent, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> patch agent <agent_name> -p '{"spec": {"installation_disk_id": "<installation_disk_id>", "approved": true}}' --type merge
```

- b. To set the **hostname** field for an agent, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> patch agent <agent_name> -p '{"spec": {"hostname": "<hostname>", "approved": true}}' --type merge
```

Verification

- To verify that the agents are approved for use, enter the following command:

```
$ oc -n <hosted_control_plane_namespace> get agents
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
86f7ac75-4fc4-4b36-8130-40fa12602218		true	auto-assign	
e57a637f-745b-496e-971d-1abbf03341ba		true	auto-assign	

5.5.3. Scaling the NodePool object for a hosted cluster on IBM Power

The **NodePool** object is created when you create a hosted cluster. By scaling the **NodePool** object, you can add more compute nodes to hosted control planes.

Procedure

1. Run the following command to scale the **NodePool** object to two nodes:

```
$ oc -n <hosted_cluster_namespace> scale nodepool <nodepool_name> --replicas 2
```

The Cluster API agent provider randomly picks two agents that are then assigned to the hosted cluster. Those agents go through different states and finally join the hosted cluster as OpenShift Container Platform nodes. The agents pass through the transition phases in the following order:

- **binding**
- **discovering**
- **insufficient**
- **installing**
- **installing-in-progress**
- **added-to-existing-cluster**

2. Run the following command to see the status of a specific scaled agent:

```
$ oc -n <hosted_control_plane_namespace> get agent \
-o jsonpath='{range .items[*]}BMH: {@.metadata.labels.agent-install\.openshift\.io/bmh}
Agent: {@.metadata.name} State: {@.status.debugInfo.state}{"\n"}{end}'
```

Example output

```
BMH: Agent: 50c23cda-cedc-9bbd-bcf1-9b3a5c75804d State: known-unbound
BMH: Agent: 5e498cd3-542c-e54f-0c58-ed43e28b568a State: insufficient
```

3. Run the following command to see the transition phases:

```
$ oc -n <hosted_control_plane_namespace> get agent
```

Example output

NAME	CLUSTER	APPROVED	ROLE	STAGE
50c23cda-cedc-9bbd-bcf1-9b3a5c75804d	hosted-forwarder	true		auto-assign
5e498cd3-542c-e54f-0c58-ed43e28b568a		true		auto-assign
da503cf1-a347-44f2-875c-4960ddb04091	hosted-forwarder	true		auto-assign

4. Run the following command to generate the **kubeconfig** file to access the hosted cluster:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \
--name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

5. After the agents reach the **added-to-existing-cluster** state, verify that you can see the OpenShift Container Platform nodes by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
worker-zvm-0.hostedn.example.com	Ready	worker	5m41s	v1.24.0+3882f8f
worker-zvm-1.hostedn.example.com	Ready	worker	6m3s	v1.24.0+3882f8f

6. Enter the following command to verify that two machines were created when you scaled up the **NodePool** object:

```
$ oc -n <hosted_control_plane_namespace> get machine.cluster.x-k8s.io
```

Example output

NAME	CLUSTER	NODENAME	PROVIDERID
hosted-forwarder-79558597ff-5tbqp	hosted-forwarder-crqq5	worker-zvm-0.hostedn.example.com	Running 41h
hosted-forwarder-79558597ff-lfjfk	hosted-forwarder-crqq5	worker-zvm-1.hostedn.example.com	Running 41h

- Run the following command to check the cluster version:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusterversion
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE
STATUS				
clusterversion.config.openshift.io/version	4.15.0	True	False	40h
version is 4.15.0				Cluster

- Run the following command to check the Cluster Operator status:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusteroperators
```

For each component of your cluster, the output shows the following Cluster Operator statuses:

- **NAME**
- **VERSION**
- **AVAILABLE**
- **PROGRESSING**
- **DEGRADED**
- **SINCE**
- **MESSAGE**

Additional resources

- [Initial Operator configuration](#)
- [Scaling down the data plane to zero](#)

5.6. MANAGING HOSTED CONTROL PLANES ON RHOSP

After you deploy hosted control planes on Red Hat OpenStack Platform (RHOSP) agent machines, you can manage a hosted cluster.

5.6.1. Accessing the hosted cluster

You can access hosted clusters on Red Hat OpenStack Platform (RHOSP) by extracting the kubeconfig secret directly from resources by using the **oc** CLI.

The *hosted cluster (hosting)* namespace has hosted cluster resources and the access secrets. The *hosted control plane* namespace is where the hosted control plane runs.

The secret name formats are as follows:

- **kubeconfig** secret: **<hosted_cluster_namespace>-<name>-admin-kubeconfig**. For example, **clusters-hypershift-demo-admin-kubeconfig**.

- **kubeadmin** password secret: **<hosted_cluster_namespace>-<name>-kubeadmin-password**. For example, **clusters-hypershift-demo-kubeadmin-password**.

The **kubeconfig** secret has a Base64-encoded **kubeconfig** field. The **kubeadmin** password secret is also Base64-encoded; you can extract it and then use the password to log in to the API server or console of the hosted cluster.

Prerequisites

- The **oc** CLI is installed.

Procedure

1. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_cluster_namespace> \
  secret/<hosted_cluster_name>-admin-kubeconfig \
  --to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

2. View a list of nodes of the hosted cluster to verify your access by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets/kubeconfig get nodes
```

5.6.2. Enabling node auto-scaling for the hosted cluster

When you need more capacity in your hosted cluster on Red Hat OpenStack Platform (RHOSP) and spare agents are available, you can enable auto-scaling to install new worker nodes.

Procedure

1. To enable auto-scaling, enter the following command:

```
$ oc -n <hosted_cluster_namespace> patch nodepool <hosted_cluster_name> \
  --type=json \
  -p '[{"op": "remove", "path": "/spec/replicas"}, {"op": "add", "path": "/spec/autoScaling",
  "value": { "max": 5, "min": 2 } } ]'
```

2. Create a workload that requires a new node.
 - a. Create a YAML file that has the workload configuration, by using the following example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: reversewords
  name: reversewords
  namespace: default
```

```
spec:
  replicas: 40
  selector:
    matchLabels:
      app: reversewords
  template:
    metadata:
      labels:
        app: reversewords
    spec:
      containers:
        - image: quay.io/mavazque/reversewords:latest
          name: reversewords
      resources:
        requests:
          memory: 2Gi
```

- b. Save the file with the name **workload-config.yaml**.
- c. Apply the YAML by entering the following command:

```
$ oc apply -f workload-config.yaml
```

3. Extract the **admin-kubeconfig** secret by entering the following command:

```
$ oc extract -n <hosted_cluster_namespace> \
  secret/<hosted_cluster_name>-admin-kubeconfig \
  --to=./hostedcluster-secrets --confirm
```

Example output

```
hostedcluster-secrets/kubeconfig
```

4. You can check if new nodes are in the **Ready** status by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

5. To remove the node, delete the workload by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets -n <namespace> \
  delete deployment <deployment_name>
```

6. Wait for several minutes to pass without requiring the additional capacity. You can confirm that the node was removed by entering the following command:

```
$ oc --kubeconfig ./hostedcluster-secrets get nodes
```

5.6.3. Configuring node pools for availability zones

To improve the high availability of your hosted cluster, you can distribute node pools across multiple Red Hat OpenStack Platform (RHOSP) Nova availability zones.



NOTE

Availability zones do not necessarily correspond to fault domains and do not inherently provide high availability benefits.

Prerequisites

- You created a hosted cluster.
- You have access to the management cluster.
- The **hcp** and **oc** command-line interfaces (CLIs) are installed.

Procedure

1. Set environment variables that are appropriate for your needs. For example, if you want to create two additional machines in the **az1** availability zone, you could enter:

```
$ export NODEPOOL_NAME="${CLUSTER_NAME}-az1" \
  && export WORKER_COUNT="2" \
  && export FLAVOR="m1.xlarge" \
  && export AZ="az1"
```

2. Create the node pool by using your environment variables by entering the following command:

```
$ hcp create nodepool openstack \
  --cluster-name <cluster_name> \
  --name $NODEPOOL_NAME \
  --replicas $WORKER_COUNT \
  --openstack-node-flavor $FLAVOR \
  --openstack-node-availability-zone $AZ
```

where:

<cluster_name>

Specifies the name of your hosted cluster.

3. Check the status of the node pool by listing **nodepool** resources in the clusters namespace by running the following command:

```
$ oc get nodepools --namespace clusters
```

Example output

NAME	CLUSTER	DESIRED NODES	CURRENT NODES	AUTOSCALING	AUTOREPAIR	VERSION	UPDATINGVERSION	UPDATINGCONFIG	MESSAGE
example	example	5	5	False	False	4.17.0			
example-az1	example	2		False	False			True	
True	Minimum availability requires 2 replicas, current 0 available								

4. Observe the notes as they start on your hosted cluster by running the following command:

```
$ oc --kubeconfig $CLUSTER_NAME-kubeconfig get nodes
```

■

Example output

NAME	STATUS	ROLES	AGE	VERSION
...				
example-extra-az-zh9l5	Ready	worker	2m6s	v1.27.4+18eadca
example-extra-az-zr8mj	Ready	worker	102s	v1.27.4+18eadca
...				

5. Verify that the node pool is created by running the following command:

```
$ oc get nodepools --namespace clusters
```

Example output

NAME	CLUSTER	DESIRED	CURRENT	AVAILABLE	PROGRESSING
MESSAGE					
<node_pool_name>	<cluster_name>	2	2	2	False
available					All replicas are

5.6.4. Additional ports for node pools

You can configure additional ports for node pools to support advanced networking scenarios, such as SR-IOV or multiple networks.

Common reasons to configure additional ports for node pools include the following:

SR-IOV (Single Root I/O Virtualization)

Enables a physical network device to appear as multiple virtual functions (VFs). By attaching additional ports to node pools, workloads can use SR-IOV interfaces to achieve low-latency, high-performance networking.

DPDK (Data Plane Development Kit)

Provides fast packet processing in user space, bypassing the kernel. Node pools with additional ports can expose interfaces for workloads that use DPDK to improve network performance.

Manila RWX volumes on NFS

Supports **ReadWriteMany** (RWX) volumes over NFS, allowing multiple nodes to access shared storage. Attaching additional ports to node pools enables workloads to reach the NFS network used by Manila.

Multus CNI

Enables pods to connect to multiple network interfaces. Node pools with additional ports support use cases that require secondary network interfaces, including dual-stack connectivity and traffic separation.

5.6.4.1. Options for additional ports for node pools

To support advanced networking scenarios, you can use the **--openstack-node-additional-port** flag to attach additional ports to nodes in a hosted cluster on OpenStack.

The flag takes a list of comma-separated parameters that can be used multiple times to attach multiple additional ports to the nodes.

The parameters are as follows:

Parameter	Description	Required	Default
network-id	The ID of the network to attach to the node.	Yes	N/A
vnic-type	The vNIC type to use for the port. If not specified, Neutron uses the default type normal .	No	N/A
disable-port-security	Whether to disable port security for the port. If not specified, Neutron enables port security unless it is explicitly disabled at the network level.	No	N/A
address-pairs	A list of IP address pairs to assign to the port. The format is ip_address=mac_address . Multiple pairs can be provided, separated by a hyphen (-). The mac_address portion is optional.	No	N/A

5.6.4.2. Creating additional ports for node pools

You can configure additional ports for node pools for hosted clusters that run on Red Hat OpenStack Platform (RHOSP).

Prerequisites

- You created a hosted cluster.
- You have access to the management cluster.
- The **hcp** CLI is installed.
- Additional networks are created in RHOSP.
- The project that is used by the hosted cluster must have access to the additional networks.
- You reviewed the options that are listed in "Options for additional ports for node pools".

Procedure

- Create a hosted cluster with additional ports attached to it by running the **hcp create nodepool openstack** command with the **--openstack-node-additional-port** options. For example:

```
$ hcp create nodepool openstack \  
  --cluster-name <cluster_name> \  
  --name <nodepool_name> \  
  --replicas <replica_count> \  
  --openstack-node-flavor <flavor> \  
  --openstack-node-additional-port "network-id=<sriov_net_id>,vnic-type=direct,disable-port-  
security=true" \  
  --openstack-node-additional-port "network-id=<lb_net_id>,address-pairs:192.168.0.1-  
192.168.0.2"
```

where:

<cluster_name>

Specifies the name of the hosted cluster.

<nodepool_name>

Specifies the name of the node pool.

<replica_count>

Specifies the number of replicas that you need.

<flavor>

Specifies the RHOSP flavor to use.

<sriov_net_id>

Specifies a SR-IOV network ID.

<lb_net_id>

Specifies a load balancer network ID.

5.6.5. Node tuning for hosted clusters on RHOSP

You can tune hosted cluster node performance on RHOSP for high-performance workloads, such as cloud-native network functions (CNFs). Performance tuning includes configuring RHOSP resources, creating a performance profile, deploying a tuned **NodePool** resource, and enabling SR-IOV device support.

CNFs are designed to run in cloud-native environments. They can provide network services such as routing, firewalling, and load balancing. You can configure the node pool to use high-performance computing and networking devices to run CNFs.

5.6.5.1. Tuning performance for hosted cluster nodes

To run high-performance workloads on hosted control planes on Red Hat OpenStack Platform (RHOSP), you can create a performance profile and deploy a tuned **NodePool** resource.

Prerequisites

- You have RHOSP flavor that has the necessary resources to run your workload, including dedicated CPU, memory, and host aggregate information.

- You have an RHOSP network that is attached to SR-IOV or DPDK-capable NICs. The network must be available to the project used by hosted clusters.

Procedure

1. Create a performance profile that meets your requirements in a file called **perfprofile.yaml**. For example:

Example performance profile in a config map

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: perfprof-1
  namespace: clusters
data:
  tuning: |
    apiVersion: v1
    kind: ConfigMap
    metadata:
      name: cnf-performanceprofile
      namespace: "${HYPERSHIFT_NAMESPACE}"
    data:
      tuning: |
        apiVersion: performance.openshift.io/v2
        kind: PerformanceProfile
        metadata:
          name: cnf-performanceprofile
        spec:
          additionalKernelArgs:
            - nmi_watchdog=0
            - audit=0
            - mce=off
            - processor.max_cstate=1
            - idle=poll
            - intel_idle.max_cstate=0
            - amd_iommu=on
          cpu:
            isolated: "${CPU_ISOLATED}"
            reserved: "${CPU_RESERVED}"
          hugepages:
            defaultHugepagesSize: "1G"
          pages:
            - count: ${HUGEPAGES}
              node: 0
              size: 1G
          nodeSelector:
            node-role.kubernetes.io/worker: ""
          realTimeKernel:
            enabled: false
          globallyDisableIrqLoadBalancing: true

```



IMPORTANT

If you do not already have environment variables set for the HyperShift Operator namespace, isolated and reserved CPUs, and huge pages count, create them before applying the performance profile.

2. Apply the performance profile configuration by running the following command:

```
$ oc apply -f perfprof.yaml
```

3. If you do not already have a **CLUSTER_NAME** environment variable set for the name of your cluster, define it.
4. Set a node pool name environment variable by running the following command:

```
$ export NODEPOOL_NAME=$CLUSTER_NAME-cnf
```

5. Set a flavor environment variable by running the following command:

```
$ export FLAVOR="m1.xlarge.nfv"
```

6. Create a node pool that uses the performance profile by running the following command:

```
$ hcp create nodepool openstack \
  --cluster-name $CLUSTER_NAME \
  --name $NODEPOOL_NAME \
  --node-count 0 \
  --openstack-node-flavor $FLAVOR
```

7. Patch the node pool to reference the **PerformanceProfile** resource by running the following command:

```
$ oc patch nodepool -n ${HYPERSHIFT_NAMESPACE} ${CLUSTER_NAME} \
  -p '{"spec":{"tuningConfig":{"name":"cnf-performanceprofile"}}}' --type=merge
```

8. Scale the node pool by running the following command:

```
$ oc scale nodepool/$CLUSTER_NAME --namespace ${HYPERSHIFT_NAMESPACE} --
replicas=1
```

9. Wait for the nodes to be ready:

- a. Wait for the nodes to be ready by running the following command:

```
$ oc wait --for=condition=UpdatingConfig=True nodepool \
  -n ${HYPERSHIFT_NAMESPACE} ${CLUSTER_NAME} \
  --timeout=5m
```

- b. Wait for the configuration update to finish by running the following command:

```
$ oc wait --for=condition=UpdatingConfig=False nodepool \
  -n ${HYPERSHIFT_NAMESPACE} ${CLUSTER_NAME} \
  --timeout=30m
```

- c. Wait until all nodes are healthy by running the following command:

```
$ oc wait --for=condition=AllNodesHealthy nodepool \
  -n ${HYPERSHIFT_NAMESPACE} ${CLUSTER_NAME} \
  --timeout=5m
```



NOTE

You can make an SSH connection into the nodes or use the **oc debug** command to verify performance configurations.

5.6.5.2. Enabling the SR-IOV Network Operator in a hosted cluster

To manage the SR-IOV-capable devices on nodes deployed by the **NodePool** resource, you can enable the SR-IOV Network Operator.

The Operator runs in the hosted cluster and requires labeled worker nodes.

Procedure

1. Generate a **kubeconfig** file for the hosted cluster by running the following command:

```
$ hcp create kubeconfig --name $CLUSTER_NAME > $CLUSTER_NAME-kubeconfig
```

2. Create a **kubeconfig** resource environment variable by running the following command:

```
$ export KUBECONFIG=$CLUSTER_NAME-kubeconfig
```

3. Label each worker node to indicate SR-IOV capability by running the following command:

```
$ oc label node <worker_node_name> feature.node.kubernetes.io/network-
sriov.capable=true
```

where:

<worker_node_name>

Specifies the name of a worker node in the hosted cluster.

4. Install the SR-IOV Network Operator in the hosted cluster by following the instructions in "Installing the SR-IOV Network Operator".
5. After installation, configure SR-IOV workloads in the hosted cluster by using the same process as for a standalone cluster.

Additional resources

- [Installing the SR-IOV Network Operator](#)

CHAPTER 6. DEPLOYING HOSTED CONTROL PLANES IN A DISCONNECTED ENVIRONMENT

6.1. INTRODUCTION TO HOSTED CONTROL PLANES IN A DISCONNECTED ENVIRONMENT

In the context of hosted control planes, a disconnected environment is an OpenShift Container Platform deployment that is not connected to the internet and that uses hosted control planes as a base. You can deploy hosted control planes in a disconnected environment on bare metal or OpenShift Virtualization.

Hosted control planes in disconnected environments function differently than in standalone OpenShift Container Platform:

- The control plane is in the management cluster. The control plane is where the pods of the hosted control plane are run and managed by the Control Plane Operator.
- The data plane is in the workers of the hosted cluster. The data plane is where the workloads and other pods run, all managed by the HostedClusterConfig Operator.

Depending on where the pods are running, they are affected by the **ImageDigestMirrorSet** (IDMS) or **ImageContentSourcePolicy** (ICSP) that is created in the management cluster or by the **ImageContentSource** that is set in the **spec** field of the manifest for the hosted cluster. The **spec** field is translated into an IDMS object on the hosted cluster.

You can deploy hosted control planes in a disconnected environment on IPv4, IPv6, and dual-stack networks. IPv4 is one of the simplest network configurations to deploy hosted control planes in a disconnected environment. IPv4 ranges require fewer external components than IPv6 or dual-stack setups. For hosted control planes on OpenShift Virtualization in a disconnected environment, use either an IPv4 or a dual-stack network.

6.2. DEPLOYING HOSTED CONTROL PLANES ON OPENSIFT VIRTUALIZATION IN A DISCONNECTED ENVIRONMENT

When you deploy hosted control planes in a disconnected environment, some of the steps differ depending on whether you use bare metal or OpenShift Virtualization.

To get started, you must meet the following requirements:

- You have a disconnected OpenShift Container Platform environment serving as your management cluster.
- You have an internal registry to mirror images on. For more information, see "About disconnected installation mirroring".



NOTE

A known limitation exists for hosted clusters on an OpenShift Container Platform management cluster that is version 4.21 or later. To avoid issues, you must mirror the 4.20.10 release payload from the **quay.io/openshift-release-dev/ocp-release:4.20.10-multi** image to the target mirror registry. This temporary limitation is expected to be resolved in a later release.

Additional resources

- [About disconnected installation mirroring](#)

6.2.1. Configuring image mirroring for hosted control planes in a disconnected environment

Image mirroring is the process of fetching images from external registries, such as **registry.redhat.com** or **quay.io**, and storing them in your private registry.

In the following procedure, the **oc-mirror** tool is used, which is a binary that uses the **ImageSetConfiguration** object. In the file, you can specify the following information:

- The OpenShift Container Platform versions to mirror. The versions are in **quay.io**.
- The additional Operators to mirror. Select packages individually.
- The extra images that you want to add to the repository.

Prerequisites

- Ensure that the registry server is running before you start the mirroring process.

Procedure

1. Ensure that your **\${HOME}/.docker/config.json** file is updated with the registries that you are going to mirror from and with the private registry that you plan to push the images to.
2. By using the following example, create an **ImageSetConfiguration** object to use for mirroring. Replace values as needed to match your environment:

```
apiVersion: mirror.openshift.io/v2alpha1
kind: ImageSetConfiguration
mirror:
  platform:
    channels:
      - name: candidate-4.22
        minVersion: <4.x.y-build>
        maxVersion: <4.x.y-build>
      type: ocp
    kubeVirtContainer: true
    graph: true
  operators:
    - catalog: registry.redhat.io/redhat/redhat-operator-index:v4.22
      packages:
        - name: lvms-operator
        - name: local-storage-operator
        - name: odf-csi-addons-operator
        - name: odf-operator
        - name: mcg-operator
        - name: ocs-operator
        - name: metallb-operator
        - name: kubevirt-hyperconverged
```

- **mirror.platform.channels.minVersion** specifies the supported OpenShift Container Platform version you want to use.

- **mirror.platform.channels.maxVersion** specifies the supported (product-title) version you want to use.
- **kubeVirtContainer** specifies whether you want to also mirror to the container disk image for the Red Hat Enterprise Linux CoreOS (RHCOS) boot image for the KubeVirt provider. This flag is optional. It is available with oc-mirror v2 only.
- **mirror.operators.packages.name: kubevirt-hyperconverged** must be included for deployments that use the KubeVirt provider.

3. Start the mirroring process by entering the following command:

```
$ oc-mirror --v2 --config imagesetconfig.yaml \
  --workspace file://mirror-file docker://<registry>
```

After the mirroring process is finished, you have a new folder named **mirror-file**, which contains the **ImageDigestMirrorSet** (IDMS), **ImageTagMirrorSet** (ITMS), and the catalog sources to apply on the hosted cluster.

4. Mirror the nightly or CI versions of OpenShift Container Platform by configuring the **imagesetconfig.yaml** file as follows:

```
apiVersion: mirror.openshift.io/v2alpha1
kind: ImageSetConfiguration
mirror:
  platform:
    graph: true
    release: registry.ci.openshift.org/ocp/release:<4.x.y-build>
    kubeVirtContainer: true
# ...
```

- **mirror.platform.release** specifies the supported OpenShift Container Platform version you want to use.
 - **mirror.platform.kubeVirtContainer** specifies that you want to also mirror the container disk image for the Red Hat Enterprise Linux CoreOS (RHCOS) boot image for the KubeVirt provider. This flag is available with oc-mirror v2 only.
5. If you have a partially disconnected environment, mirror the images from the image set configuration to a registry by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml \
  --workspace file://<file_path> docker://<mirror_registry_url> --v2
```

For more information, see "Mirroring an image set in a partially disconnected environment".

6. If you have a fully disconnected environment, complete the following steps:

- Mirror the images from the specified image set configuration to the disk by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml file://<file_path> --v2
```

For more information, see "Mirroring an image set in a fully disconnected environment".

- b. Process the image set file on the disk and mirror the contents to a target mirror registry by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml \
  --from file://<file_path> docker://<mirror_registry_url> --v2
```

7. Mirror the latest multicluster engine Operator images by following the steps in "Install on disconnected networks".

Additional resources

- [Mirroring an image set in a partially disconnected environment](#)
- [Mirroring an image set in a fully disconnected environment](#)
- [Install on disconnected networks](#)

6.2.2. Applying objects in the management cluster

After the mirroring process is complete, you must apply two objects required for mirroring in the management cluster.

You apply the following objects:

- **ImageContentSourcePolicy** (ICSP) or **ImageDigestMirrorSet** (IDMS)
- Catalog sources

When you use the **oc-mirror** tool, the output artifacts are in a folder named **oc-mirror-workspace/results-XXXXXX/**.

The **oc mirror** mirroring file initiates a **MachineConfig** change that does not restart your nodes but restarts the kubelet on each of them. After the nodes are marked as **READY**, you need to apply the newly generated catalog sources.

The catalog sources initiate actions in the **openshift-marketplace** Operator, such as downloading the catalog image and processing it to retrieve all the **PackageManifests** that are included in that image.

Procedure

1. To check the new sources, run the following command by using the new **CatalogSource** as a source:

```
$ oc get packagemanifest
```

2. To apply the artifacts, complete the following steps:

- a. Create the ICSP or IDMS artifacts by entering the following command:

```
$ oc apply -f oc-mirror-workspace/results-XXXXXX/imageContentSourcePolicy.yaml
```

- b. Wait for the nodes to become ready, and then enter the following command:

```
$ oc apply -f catalogSource-XXXXXXXXX-index.yaml
```

3. Mirror the OLM catalogs and configure the hosted cluster to point to the mirror.
When you use the **management** (default) OLMCatalogPlacement mode, the image stream that is used for OLM catalogs is not automatically amended with override information from the ICSP on the management cluster.
 - a. If the OLM catalogs are properly mirrored to an internal registry by using the original name and tag, add the **hypershift.openshift.io/olm-catalogs-is-registry-overrides** annotation to the **HostedCluster** resource. The format is **"sr1=dr1,sr2=dr2"**, where the source registry string is a key and the destination registry is a value.
 - b. To bypass the OLM catalog image stream mechanism, use the following four annotations on the **HostedCluster** resource to directly specify the addresses of the four images to use for OLM Operator catalogs:
 - **hypershift.openshift.io/certified-operators-catalog-image**
 - **hypershift.openshift.io/community-operators-catalog-image**
 - **hypershift.openshift.io/redhat-marketplace-catalog-image**
 - **hypershift.openshift.io/redhat-operators-catalog-image**In this case, the image stream is not created, and you must update the value of the annotations when the internal mirror is refreshed to pull in Operator updates.

Next steps

Deploy the multicluster engine Operator by completing the steps in "Deploying multicluster engine Operator for a disconnected installation of hosted control planes".

Additional resources

- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

6.2.3. Deploying multicluster engine Operator for a disconnected installation of hosted control planes

The multicluster engine for Kubernetes Operator is crucial in deploying clusters across providers. Ensure that you have multicluster engine Operator installed and configured for your deployment.

If you do not have multicluster engine Operator installed, review the following documentation to understand the prerequisites and steps to install it:

- "About cluster lifecycle with multicluster engine operator"
- "Installing and upgrading multicluster engine operator"

Additional resources

- [About cluster lifecycle with multicluster engine operator](#)
- [Installing and upgrading multicluster engine operator](#)

6.2.4. TLS certificates for a disconnected installation of hosted control planes

To ensure proper function in a disconnected deployment, you need to configure the registry CA certificates in the management cluster and the compute nodes for the hosted cluster.

6.2.4.1. Adding the registry CA to the management cluster

To ensure proper function in a disconnected deployment, you need to configure the registry CA certificates in the management cluster.

To add the registry CA to the management cluster, complete the following steps.

Procedure

1. Create a config map that resembles the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: <config_map_name>
  namespace: <config_map_namespace>
data:
  <registry_name>..

-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
<registry_name>..

-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
<registry_name>..

-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----


```

- **metadata.name** specifies the name of the config map.
 - **metadata.namespace** specifies the namespace for the config map.
 - **data** specifies the registry names and the registry certificate content. Replace **<port>** with the port where the registry server is running; for example, **5000**. Ensure that the data in the config map is defined by using **|** only instead of other methods, such as **| -**. If you use other methods, issues can occur when the pod reads the certificates.
2. Patch the cluster-wide object, **image.config.openshift.io** to include the following specification:

```
spec:
  additionalTrustedCA:
    - name: registry-config
```

As a result of this patch, the control plane nodes can retrieve images from the private registry and the HyperShift Operator can extract the OpenShift Container Platform payload for hosted cluster deployments.

The process to patch the object might take several minutes to be completed.

6.2.4.2. Adding the registry CA to the compute nodes for the hosted cluster

To ensure that the data plane compute nodes in the hosted cluster can retrieve images from the private registry, you must add the registry certificate authority (CA) to the compute nodes.

Procedure

1. In the **hc.spec.additionalTrustBundle** file, add the following specification:

```
spec:
  additionalTrustBundle:
    name: user-ca-bundle
```

The **user-ca-bundle** entry is a config map that you create in the next step.

2. In the same namespace where the **HostedCluster** object is created, create the **user-ca-bundle** config map. The config map resembles the following example:

```
apiVersion: v1
data:
  ca-bundle.crt: |
    // Registry1 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

    // Registry2 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

    // Registry3 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

kind: ConfigMap
metadata:
  name: user-ca-bundle
  namespace: <hosted_cluster_namespace>
```

Specify the namespace where the **HostedCluster** object is created.

6.2.5. Creating a hosted cluster on OpenShift Virtualization in a disconnected environment

As part of the process to deploy hosted control planes on OpenShift Virtualization in a disconnected environment, you need to create a hosted cluster. A hosted cluster is an OpenShift Container Platform cluster with its control plane and API endpoint hosted on a management cluster. The hosted cluster includes the control plane and its corresponding data plane.

6.2.5.1. Prerequisites to deploy hosted control planes on OpenShift Virtualization

To create an OpenShift Container Platform cluster on OpenShift Virtualization, you must meet several prerequisites.

- You have administrator access to an OpenShift Container Platform cluster, version 4.14 or later, specified in the **KUBECONFIG** environment variable.
- The OpenShift Container Platform management cluster must have wildcard DNS routes enabled, as shown in the following command:

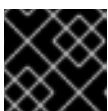
```
$ oc patch ingresscontroller -n openshift-ingress-operator default \
```

```
--type=json \
-p '[{"op": "add", "path": "/spec/routeAdmission", "value": {"wildcardPolicy":
"WildcardsAllowed"}}]'
```

- The OpenShift Container Platform management cluster has OpenShift Virtualization, version 4.14 or later, installed on it. For more information, see "Installing OpenShift Virtualization using the web console".
- The OpenShift Container Platform management cluster is on-premise bare metal.
- The OpenShift Container Platform management cluster must be configured with **OVN-Kubernetes** as the default pod network Container Network Interface (CNI). Live migration is supported for nodes only if the CNI is OVN-Kubernetes.
- The OpenShift Container Platform management cluster has a default storage class. For more information, see "Postinstallation storage configuration". The following example shows how to set a default storage class:

```
$ oc patch storageclass ocs-storagecluster-ceph-rbd \
-p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

- When you configure storage for hosted control planes, consider the recommended etcd practices. To ensure that you meet the latency requirements, dedicate a fast storage device to all hosted control plane etcd instances that run on each control-plane node. You can use LVM storage to configure a local storage class for hosted etcd pods. For more information, see "Recommended etcd practices" and "Persistent storage using Logical Volume Manager storage".
- On the OpenShift Container Platform cluster that hosts the OpenShift Virtualization virtual machines, you must use a **ReadWriteMany** (RWX) storage class so that live migration can be enabled.
- You have a valid pull secret file for the **quay.io/openshift-release-dev** repository. For more information, see "Install OpenShift on any x86_64 platform with user-provisioned infrastructure".
- You have installed the hosted control plane command-line interface.
- You have configured a load balancer. For more information, see "Configuring MetalLB".
- For optimal network performance, you are using a network maximum transmission unit (MTU) of 9000 or greater on the OpenShift Container Platform cluster that hosts the KubeVirt virtual machines. If you use a lower MTU setting, network latency and the throughput of the hosted pods are affected. Enable multiqueue on node pools only when the MTU is 9000 or greater.



IMPORTANT

You cannot change the MTU value for your cluster as a postinstallation task.

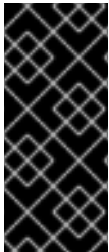
- The multicluster engine Operator has at least one managed OpenShift Container Platform cluster. The **local-cluster** is automatically imported. For more information about the **local-cluster**, see "Advanced configuration" in the multicluster engine Operator documentation. You can check the status of your hub cluster by running the following command:

```
$ oc get managedclusters local-cluster
```

- Ensure each hosted cluster has a cluster-wide unique name.
- Do not use **clusters** as a hosted cluster name.
- Do not create a hosted cluster in the namespace of a multicluster engine Operator managed cluster.

6.2.5.2. Creating a hosted cluster with the KubeVirt platform by using the CLI

To create a hosted cluster on OpenShift Virtualization, you can use the hosted control plane command-line interface (CLI), **hcp**.



IMPORTANT

Avoid storing all hosted cluster information in a shared namespace. If you create a hosted cluster in a shared namespace and then back up and restore the hosted cluster, you might unintentionally change other hosted clusters. Either store hosted cluster information in a separate namespace or set up your hosted cluster to back up and restore resources based on labels.

Procedure

1. Create a hosted cluster with the KubeVirt platform by entering the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
  --node-pool-replicas <node_pool_replica_count> \
  --pull-secret <path_to_pull_secret> \
  --memory <value_for_memory> \
  --cores <value_for_cpu> \
  --etcd-storage-class=<etcd_storage_class> \
  --arch <architecture_of_the_nodepool> \
  --release-image <ocp_release_image_for_the_cluster> \
  --image-content-sources <path_to_image_content_sources_file> \
  --additional-trust-bundle <path_to_ca_bundle_file>
```

- **--name** defines the name of your hosted cluster, for example, **my-hosted-cluster**.
- **--node-pool-replicas** defines the node pool replica count, for example, **3**. You must specify the replica count as **0** or greater to create the same number of replicas. Otherwise, no node pools are created.
- **--pull-secret** defines the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** defines a value for memory, for example, **6Gi**.
- **--cores** defines a value for CPU, for example, **2**.
- **--etcd-storage-class** defines the etcd storage class name, for example, **lvm-storageclass**.
- **--arch** defines the architecture of the node pool, for example, **s390x**. The default is **amd64**.
- **--release-image** defines the OpenShift Container Platform release image for the cluster, for example, **quay.io/openshift-release-dev/ocp-release:4.20.14-multi**. You can use the **--release-image** flag to set up the hosted cluster with a specific OpenShift Container

Platform release.

- **--image-content-sources** specifies the path to a file with image content sources.
- **--additional-trust-bundle** specifies the path to a file with user CA bundle.

A default node pool is created for the cluster with a specific number of virtual machine worker replicas according to the **--node-pool-replicas** flag.

2. After a few moments, verify that the hosted control plane pods are running by entering the following command:

```
$ oc -n clusters-<hosted-cluster-name> get pods
```

Example output

```
NAME                                READY  STATUS  RESTARTS  AGE
capi-provider-5cc7b74f47-n5gkr      1/1    Running  0         3m
catalog-operator-5f799567b7-fd6jw   2/2    Running  0         69s
certified-operators-catalog-784b9899f9-mrp6p  1/1    Running  0         66s
cluster-api-6bbc867966-l4dwl        1/1    Running  0         66s
.
.
.
redhat-operators-catalog-9d5fd4d44-z8qqk  1/1    Running  0         66s
```

A hosted cluster that has worker nodes that are backed by KubeVirt virtual machines typically takes 10-15 minutes to be fully provisioned.

Verification

- To check the status of the hosted cluster, see the corresponding **HostedCluster** resource by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

See the following example output, which illustrates a fully provisioned **HostedCluster** object:

```
NAMESPACE NAME          VERSION  KUBECONFIG          PROGRESS
AVAILABLE PROGRESSING MESSAGE
clusters  my-hosted-cluster  <4.x.0>  example-admin-kubeconfig  Completed  True
False    The hosted control plane is available
```

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

Additional resources

- [Labeling management cluster nodes](#)

6.2.6. Configuring the default ingress and DNS for hosted control planes on OpenShift Virtualization

Every OpenShift Container Platform cluster includes a default application Ingress Controller, which must have an wildcard DNS record associated with it.

By default, hosted clusters that are created by using OpenShift Virtualization automatically become a subdomain of the OpenShift Container Platform cluster that the virtual machines run on.

For example, your OpenShift Container Platform cluster might have the following default ingress DNS entry:

```
*.apps.mgmt-cluster.example.com
```

As a result, a hosted cluster that is named **guest** and that runs on that underlying OpenShift Container Platform cluster has the following default ingress:

```
*.apps.guest.apps.mgmt-cluster.example.com
```

Procedure

- For the default ingress DNS to work properly, the cluster that hosts the virtual machines must allow wildcard DNS routes. You can configure this behavior by entering the following command:

```
$ oc patch ingresscontroller -n openshift-ingress-operator default \
  --type=json \
  -p '[{"op": "add", "path": "/spec/routeAdmission", "value": {"wildcardPolicy":
    "WildcardsAllowed"}}]'
```



NOTE

When you use the default hosted cluster ingress, connectivity is limited to HTTPS traffic over port 443. Plain HTTP traffic over port 80 is rejected. This limitation applies to only the default ingress behavior.

6.2.7. Customize ingress and DNS behavior

If you do not want to use the default ingress and DNS behavior, you can configure a hosted cluster on OpenShift Virtualization with a unique base domain at creation time.

This option requires manual configuration steps during creation and involves three main steps: cluster creation, load balancer creation, and wildcard DNS configuration.

6.2.8. Creating a hosted cluster that specifies the base domain

If you do not want to use the default ingress and DNS behavior, you can configure a KubeVirt hosted cluster with a unique base domain at creation time.

Procedure

- Create the cluster by entering the following command:

```
$ hcp create cluster kubevirt \
  --name <hosted_cluster_name> \
  --node-pool-replicas <worker_count> \
  --pull-secret <path_to_pull_secret> \
```



```
--memory <value_for_memory> \
--cores <value_for_cpu> \
--base-domain <basedomain> \
--arch <architecture_of_the_nodepool> \
--release-image <ocp_release_image_for_the_cluster> \
--image-content-sources <path_to_image_content_sources_file> \
--additional-trust-bundle <path_to_ca_bundle_file>
```

- **--name** specifies the name of your hosted cluster.
- **--node-pool-replicas** specifies the worker count, for example, **2**.
- **--pull-secret** specifies the path to your pull secret, for example, **/user/name/pullsecret**.
- **--memory** specifies a value for memory, for example, **6Gi**.
- **--cores** specifies a value for CPU, for example, **2**.
- **--base-domain** specifies the base domain, for example, **hypershift.lab**.
- **--arch** specifies the architecture of the node pool, for example, **s390x**. The default is **amd64**.
- **--release-image** specifies the ocp release image for the cluster, for example, **quay.io/openshift-release-dev/ocp-release:4.20.14-multi**.
- **--image-content-sources** specifies the path to a file with image content sources.
- **--additional-trust-bundle** specifies the path to a file with user CA bundle.

As a result, the hosted cluster has an ingress wildcard that is configured for the cluster name and the base domain, for example, **.apps.example.hypershift.lab**. The hosted cluster remains in **Partial** status because after you create a hosted cluster with unique base domain, you must configure the required DNS records and load balancer.

Verification

1. View the status of your hosted cluster by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

Example output

NAME	VERSION	KUBECONFIG	PROGRESS	AVAILABLE
example		example-admin-kubeconfig	Partial	True
hosted control plane is available				

2. Access the cluster by entering the following commands:

```
$ hcp create kubeconfig --name <hosted_cluster_name> \
> <hosted_cluster_name>-kubeconfig
```

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
console	<4.x.0>	False	False	False 30m
RouteHealthAvailable: failed to GET route (https://console-openshift-console.apps.example.hypershift.lab): Get "https://console-openshift-console.apps.example.hypershift.lab": dial tcp: lookup console-openshift-console.apps.example.hypershift.lab on 172.31.0.10:53: no such host				
ingress	<4.x.0>	True	False	True 28m The "default" ingress controller reports Degraded=True: DegradedConditions: One or more other status conditions indicate a degraded state: CanaryChecksSucceeding=False (CanaryChecksRepetitiveFailures: Canary route checks for the default ingress controller are failing)

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

- To fix any errors in the output, complete the steps in "Setting up the load balancer" and "Setting up a wildcard DNS".



NOTE

If your hosted cluster is on bare metal, you might need MetalLB to set up load balancer services. For more information, see "Configuring MetalLB".

6.2.9. Setting up the load balancer

Set up the load balancer service that routes ingress traffic to the KubeVirt VMs and assigns a wildcard DNS entry to the load balancer IP address.

Procedure

- A **NodePort** service that exposes the hosted cluster ingress already exists. You can export the node ports and create the load balancer service that targets those ports.
 - Get the HTTP node port by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get services \
-n openshift-ingress router-nodeport-default \
-o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}'
```

Note the HTTP node port value to use in the next step.

- Get the HTTPS node port by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>-kubeconfig get services \
-n openshift-ingress router-nodeport-default \
-o jsonpath='{.spec.ports[?(@.name=="https")].nodePort}'
```

Note the HTTPS node port value to use in the next step.

- Enter the following information in a YAML file:

```
apiVersion: v1
```

```

kind: Service
metadata:
  labels:
    app: <hosted_cluster_name>
    name: <hosted_cluster_name>-apps
    namespace: clusters-<hosted_cluster_name>
spec:
  ports:
    - name: https-443
      port: 443
      protocol: TCP
      targetPort: <https_node_port>
    - name: http-80
      port: 80
      protocol: TCP
      targetPort: <http_node_port>
  selector:
    kubevirt.io: virt-launcher
  type: LoadBalancer

```

where:

- **<https_node_port>** specifies the HTTPS node port value that you noted in the previous step.
- **<http_node_port>** specifies the HTTP node port value that you noted in the previous step.

3. Create the load balancer service by running the following command:

```
$ oc create -f <file_name>.yaml
```

6.2.10. Setting up a wildcard DNS

If you are customizing the ingress and DNS for your hosted cluster, you need to set up a wildcard DNS record or CNAME that references the external IP of the load balancer service.

Procedure

1. Get the external IP address by entering the following command:

```
$ oc -n clusters-<hosted_cluster_name> get service <hosted-cluster-name>-apps \
-o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

Example output

```
192.168.20.30
```

2. Configure a wildcard DNS entry that references the external IP address. View the following example DNS entry:

```
*.apps.<hosted_cluster_name>.<base_domain>.
```

The DNS entry must be able to route inside and outside of the cluster.

DNS resolutions example

```
dig +short test.apps.example.hypershift.lab
192.168.20.30
```

Verification

- Check that hosted cluster status has moved from **Partial** to **Completed** by entering the following command:

```
$ oc get --namespace clusters hostedclusters
```

Example output

```
NAME          VERSION  KUBECONFIG          PROGRESS  AVAILABLE
PROGRESSING MESSAGE
example       <4.x.0>  example-admin-kubeconfig  Completed  True    False
The hosted control plane is available
```

Replace **<4.x.0>** with the supported OpenShift Container Platform version that you want to use.

6.2.11. Deployment finalization

You can monitor the deployment of a hosted cluster from two perspectives: the control plane and the data plane.

6.2.11.1. Monitoring the control plane

While the deployment proceeds, you can monitor the control plane.

You can gather information about the following artifacts:

- The HyperShift Operator
- The **HostedControlPlane** pod
- The bare-metal hosts
- The agents
- The **InfraEnv** resource
- The **HostedCluster** and **NodePool** resources

Procedure

- Enter the following command to export the **kubeconfig** file for the deployment:

```
$ export KUBECONFIG=/root/.kcli/clusters/hub-ipv4/auth/kubeconfig
```

- Enter the following command to monitor the deployment:

```
$ watch "oc get pod -n hypershift;echo;echo;\
oc get pod -n clusters-hosted-ipv4;echo;echo;\
oc get bmh -A;echo;echo;\
oc get agent -A;echo;echo;\
oc get infraenv -A;echo;echo;\
oc get hostedcluster -A;echo;echo;\
oc get nodepool -A;echo;echo;"
```

6.2.11.2. Monitoring the data plane

While the deployment proceeds, you can monitor the data plane.

You can gather information about the following artifacts:

- The cluster version
- The nodes, specifically, about whether the nodes joined the cluster
- The cluster Operators

Procedure

1. Enter the following command to get the **kubeconfig** secret:

```
$ oc get secret -n clusters-hosted-ipv4 admin-kubeconfig \
-o jsonpath='{.data.kubeconfig}' | base64 -d > /root/hc_admin_kubeconfig.yaml
```

2. Enter the following command to export the **kubeconfig** file for the deployment:

```
$ export KUBECONFIG=/root/hc_admin_kubeconfig.yaml
```

3. Enter the following command to monitor the deployment:

```
$ watch "oc get clusterversion,nodes,co"
```

6.3. DEPLOYING HOSTED CONTROL PLANES ON BARE METAL IN A DISCONNECTED ENVIRONMENT

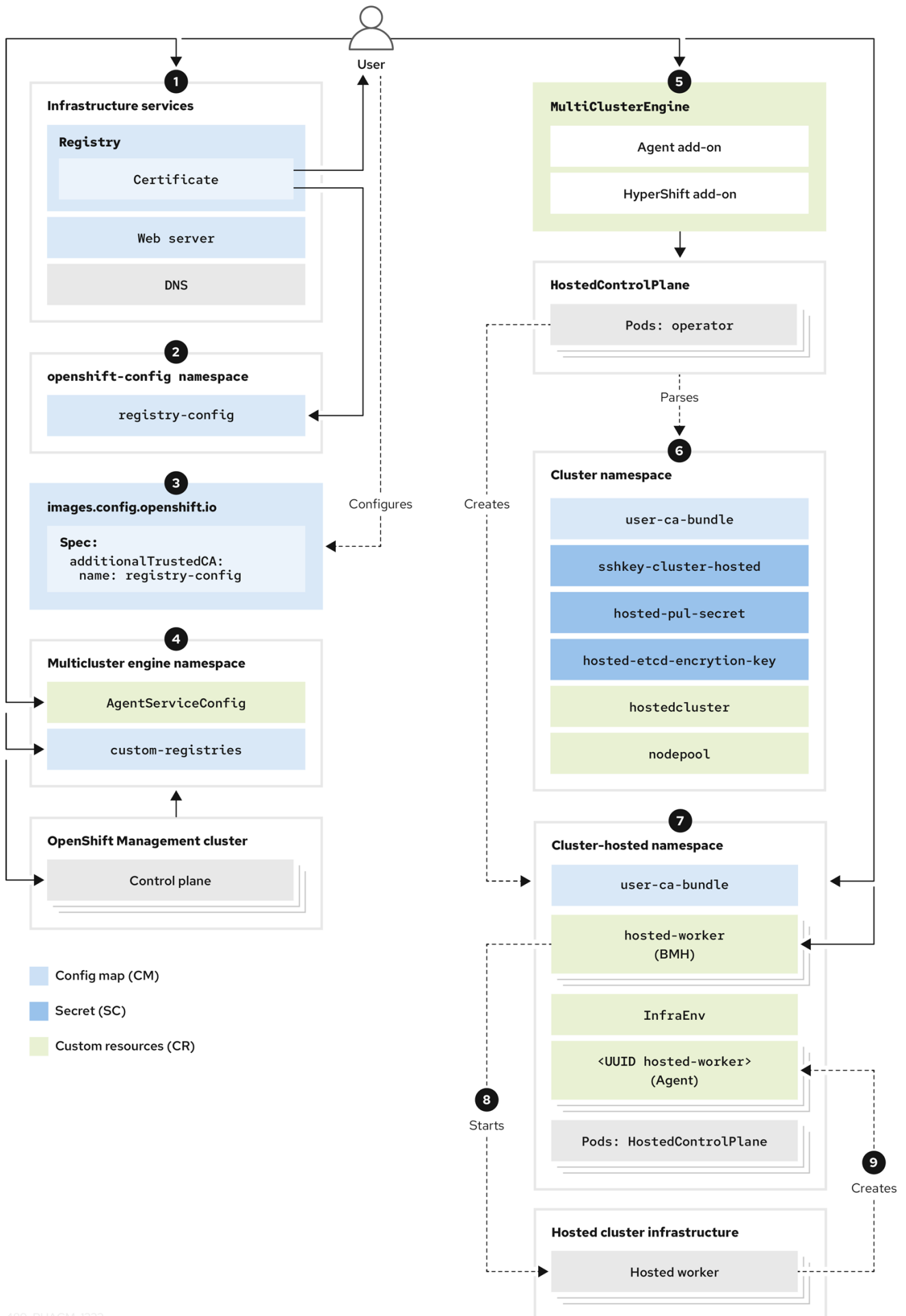
In the context of hosted control planes, a disconnected environment is an OpenShift Container Platform deployment that is not connected to the internet and that uses hosted control planes as a base. You can deploy hosted control planes in a disconnected environment on bare metal.

When you provision hosted control planes on bare metal, you use the Agent platform. The Agent platform and multicluster engine for Kubernetes Operator work together to enable disconnected deployments. The Agent platform uses the central infrastructure management service to add worker nodes to a hosted cluster. For an introduction to the central infrastructure management service, see "Enabling the central infrastructure management service".

6.3.1. Disconnected environment architecture for bare metal

Get familiar with the architecture for a deployment of hosted control planes on bare metal in a disconnected environment.

The following diagram illustrates an example architecture of a disconnected environment:



1. Configure infrastructure services, including the registry certificate deployment with TLS support, web server, and DNS, to ensure that the disconnected deployment works.
2. Create a config map in the **openshift-config** namespace. In this example, the config map is named **registry-config**. The content of the config map is the Registry CA certificate. The data field of the config map must contain the following key/value:
 - Key: **<registry_dns_domain_name>..<port>**, for example, **registry.hypershiftdomain.lab..5000**:. Ensure that you place **..** after the registry DNS domain name when you specify a port.
 - Value: The certificate content
For more information about creating a config map, see "Adding the registry CA to the management cluster" and "Adding the registry CA to the compute nodes for the hosted cluster".
3. Modify the **images.config.openshift.io** custom resource (CR) specification and adds a new field named **additionalTrustedCA** with a value of **name: registry-config**.
4. Create a config map that contains two data fields. One field contains the **registries.conf** file in **RAW** format, and the other field contains the Registry CA and is named **ca-bundle.crt**. The config map belongs to the **multicluster-engine** namespace, and the config map name is referenced in other objects. For an example of a config map, see the following sample configuration:

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: custom-registries
  namespace: multicluster-engine
  labels:
    app: assisted-service
data:
  ca-bundle.crt: |
    -----BEGIN CERTIFICATE-----
    # ...
    -----END CERTIFICATE-----
  registries.conf: |
    unqualified-search-registries = ["registry.access.redhat.com", "docker.io"]

    [[registry]]
    prefix = ""
    location = "registry.redhat.io/openshift4"
    mirror-by-digest-only = true

    [[registry.mirror]]
    location = "registry.ocp-edge-cluster-0.qe.lab.redhat.com:5000/openshift4"

    [[registry]]
    prefix = ""
    location = "registry.redhat.io/rhacm2"
    mirror-by-digest-only = true
  # ...
  # ...

```


5. In the multicluster engine Operator namespace, you create the **multiclusterengine** CR, which enables both the Agent and **hypershift-addon** add-ons. The multicluster engine Operator namespace must contain the config maps to modify behavior in a disconnected deployment. The namespace also contains the **multicluster-engine**, **assisted-service**, and **hypershift-addon-manager** pods.
6. Create the following objects that are necessary to deploy the hosted cluster:
 - **Secrets:** Secrets contain the pull secret, SSH key, and etcd encryption key.
 - **Config map:** The config map contains the CA certificate of the private registry.
 - **HostedCluster:** The **HostedCluster** resource defines the configuration of the cluster that the user intends to create.
 - **NodePool:** The **NodePool** resource identifies the node pool that references the machines to use for the data plane.
7. After you create the hosted cluster objects, the HyperShift Operator establishes the **HostedControlPlane** namespace to accommodate control plane pods. The namespace also hosts components such as Agents, bare metal hosts (BMHs), and the **InfraEnv** resource. Later, you create the **InfraEnv** resource, and after ISO creation, you create the BMHs and their secrets that contain baseboard management controller (BMC) credentials.
8. The Metal3 Operator in the **openshift-machine-api** namespace inspects the new BMHs. Then, the Metal3 Operator tries to connect to the BMCs to start them by using the configured **LiveISO** and **RootFS** values that are specified through the **AgentServiceConfig** CR in the multicluster engine Operator namespace.
9. After the worker nodes of the **HostedCluster** resource are started, an Agent container is started. This agent establishes contact with the Assisted Service, which orchestrates the actions to complete the deployment. Initially, you need to scale the **NodePool** resource to the number of worker nodes for the **HostedCluster** resource. The Assisted Service manages the remaining tasks.
10. At this point, you wait for the deployment process to be completed.

Additional resources

- [Adding the registry CA to the management cluster](#)
- [Adding the registry CA to the compute nodes for the hosted cluster](#)

6.3.2. Requirements to deploy hosted control planes on bare metal in a disconnected environment

To configure hosted control planes in a disconnected environment, you must meet several prerequisites.

- **CPU:** The number of CPUs provided determines how many hosted clusters can run concurrently. In general, use 16 CPUs for each node for 3 nodes. For minimal development, you can use 12 CPUs for each node for 3 nodes.
- **Memory:** The amount of RAM affects how many hosted clusters can be hosted. Use 48 GB of RAM for each node. For minimal development, 18 GB of RAM might be sufficient.

- Storage: Use SSD storage for multicluster engine Operator.
- Management cluster: 250 GB.
- Registry: The storage needed depends on the number of releases, operators, and images that are hosted. An acceptable number might be 500 GB, preferably separated from the disk that hosts the hosted cluster.
- Web server: The storage needed depends on the number of ISOs and images that are hosted. An acceptable number might be 500 GB.
- Production: For a production environment, separate the management cluster, the registry, and the web server on different disks. This example illustrates a possible configuration for production:
 - Registry: 2 TB
 - Management cluster: 500 GB
 - Web server: 2 TB

6.3.3. Extracting the release image digest

To deploy hosted control planes on bare metal in a disconnected environment, you need the OpenShift Container Platform release image. You can extract the release image digest by using the tagged image.

Procedure

- Obtain the image digest by running the following command:

```
$ oc adm release info <tagged_openshift_release_image> | grep "Pull From"
```

Replace **<tagged_openshift_release_image>** with the tagged image for the supported OpenShift Container Platform version, for example, **quay.io/openshift-release-dev/ocp-release:4.14.0-x86_64**.

Example output

```
Pull From: quay.io/openshift-release-dev/ocp-  
release@sha256:69d1292f64a2b67227c5592c1a7d499c7d00376e498634ff8e1946bc9ccdddfc
```

6.3.4. DNS configurations on bare metal

The API Server for the hosted cluster is exposed as a **NodePort** service. A DNS entry must exist for **api.<hosted_cluster_name>.<base_domain>** that points to destination where the API Server can be reached.

The DNS entry can be as simple as a record that points to one of the nodes in the management cluster that is running the hosted control plane. The entry can also point to a load balancer that is deployed to redirect incoming traffic to the ingress pods.

Example DNS configuration

```
api.example.krnl.es.  IN A 192.168.122.20
```

```
api.example.krnl.es. IN A 192.168.122.21
api.example.krnl.es. IN A 192.168.122.22
api-int.example.krnl.es. IN A 192.168.122.20
api-int.example.krnl.es. IN A 192.168.122.21
api-int.example.krnl.es. IN A 192.168.122.22
`*.apps.example.krnl.es. IN A 192.168.122.23
```



NOTE

In the previous example, ***.apps.example.krnl.es. IN A 192.168.122.23** is either a node in the hosted cluster or a load balancer, if one has been configured.

If you are configuring DNS for a disconnected environment on an IPv6 network, the configuration looks like the following example.

Example DNS configuration for an IPv6 network

```
api.example.krnl.es. IN A 2620:52:0:1306::5
api.example.krnl.es. IN A 2620:52:0:1306::6
api.example.krnl.es. IN A 2620:52:0:1306::7
api-int.example.krnl.es. IN A 2620:52:0:1306::5
api-int.example.krnl.es. IN A 2620:52:0:1306::6
api-int.example.krnl.es. IN A 2620:52:0:1306::7
`*.apps.example.krnl.es. IN A 2620:52:0:1306::10
```

If you are configuring DNS for a disconnected environment on a dual stack network, be sure to include DNS entries for both IPv4 and IPv6.

Example DNS configuration for a dual stack network

```
host-record=api-int.hub-dual.dns.base.domain.name,192.168.126.10
host-record=api.hub-dual.dns.base.domain.name,192.168.126.10
address=/apps.hub-dual.dns.base.domain.name/192.168.126.11
dhcp-host=aa:aa:aa:aa:10:01,ocp-control-plane-0,192.168.126.20
dhcp-host=aa:aa:aa:aa:10:02,ocp-control-plane-1,192.168.126.21
dhcp-host=aa:aa:aa:aa:10:03,ocp-control-plane-2,192.168.126.22
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,192.168.126.25
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,192.168.126.26

host-record=api-int.hub-dual.dns.base.domain.name,2620:52:0:1306::2
host-record=api.hub-dual.dns.base.domain.name,2620:52:0:1306::2
address=/apps.hub-dual.dns.base.domain.name/2620:52:0:1306::3
dhcp-host=aa:aa:aa:aa:10:01,ocp-control-plane-0,[2620:52:0:1306::5]
dhcp-host=aa:aa:aa:aa:10:02,ocp-control-plane-1,[2620:52:0:1306::6]
dhcp-host=aa:aa:aa:aa:10:03,ocp-control-plane-2,[2620:52:0:1306::7]
dhcp-host=aa:aa:aa:aa:10:06,ocp-installer,[2620:52:0:1306::8]
dhcp-host=aa:aa:aa:aa:10:07,ocp-bootstrap,[2620:52:0:1306::9]
```

6.3.5. Deploying a registry for hosted control planes in a disconnected environment

For development environments, deploy a small, self-hosted registry by using a Podman container. For production environments, deploy an enterprise-hosted registry, such as Red Hat Quay, Nexus, or Artifactory.

To deploy a small registry by using Podman, complete the following steps.

Procedure

1. As a privileged user, access the **\${HOME}** directory and create the following script:

```
#!/usr/bin/env bash

set -euo pipefail

PRIMARY_NIC=$(ls -l /sys/class/net | grep -v podman | head -1)
export PATH=/root/bin:$PATH
export PULL_SECRET="/root/baremetal/hub/openshift_pull.json"

if [[ ! -f $PULL_SECRET ]];then
    echo "Pull Secret not found, exiting..."
    exit 1
fi

dnf -y install podman httpd httpd-tools jq skopeo libseccomp-devel
export IP=$(ip -o addr show $PRIMARY_NIC | head -1 | awk '{print $4}' | cut -d '/' -f1)
REGISTRY_NAME=registry.$(hostname --long)
REGISTRY_USER=dummy
REGISTRY_PASSWORD=dummy
KEY=$(echo -n $REGISTRY_USER:$REGISTRY_PASSWORD | base64)
echo '{"auths": {"$REGISTRY_NAME:5000": {"auth": \"$KEY\", \"email\": \"sample-email@domain.ltd\"}}}' > /root/disconnected_pull.json
mv $PULL_SECRET /root/openshift_pull.json.old
jq ".auths += {\"$REGISTRY_NAME:5000\": {\"auth\": \"$KEY\", \"email\": \"sample-email@domain.ltd\"}}\" < /root/openshift_pull.json.old > $PULL_SECRET
mkdir -p /opt/registry/{auth,certs,data,conf}
cat <<EOF > /opt/registry/conf/config.yml
version: 0.1
log:
  fields:
    service: registry
storage:
  cache:
    blobdescriptor: inmemory
  filesystem:
    rootdirectory: /var/lib/registry
delete:
  enabled: true
http:
  addr: :5000
  headers:
    X-Content-Type-Options: [nosniff]
health:
  storagedriver:
    enabled: true
    interval: 10s
    threshold: 3
compatibility:
  schema1:
    enabled: true
EOF
```

```

openssl req -newkey rsa:4096 -nodes -sha256 -keyout /opt/registry/certs/domain.key -x509 -
days 3650 -out /opt/registry/certs/domain.crt -subj "/C=US/ST=Madrid/L=San
Bernardo/O=Karmalabs/OU=Guitar/CN=$REGISTRY_NAME" -addext
"subjectAltName=DNS:$REGISTRY_NAME"
cp /opt/registry/certs/domain.crt /etc/pki/ca-trust/source/anchors/
update-ca-trust extract
htpasswd -bBc /opt/registry/auth/htpasswd $REGISTRY_USER $REGISTRY_PASSWORD
podman create --name registry --net host --security-opt label=disable --replace -v
/opt/registry/data:/var/lib/registry:z -v /opt/registry/auth:/auth:z -v
/opt/registry/conf/config.yml:/etc/docker/registry/config.yml -e "REGISTRY_AUTH=htpasswd"
-e "REGISTRY_AUTH_HTPASSWD_REALM=Registry" -e
"REGISTRY_HTTP_SECRET=ALongRandomSecretForRegistry" -e
REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd -v /opt/registry/certs:/certs:z -e
REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt -e
REGISTRY_HTTP_TLS_KEY=/certs/domain.key docker.io/library/registry:latest
[ "$?" == "0" ] || !!
systemctl enable --now registry

```

Replace the location of the **PULL_SECRET** with the appropriate location for your setup.

2. Name the script file **registry.sh** and save it. When you run the script, it pulls in the following information:

- The registry name, based on the hypervisor hostname
- The necessary credentials and user access details

3. Adjust permissions by adding the execution flag as follows:

```
$ chmod u+x ${HOME}/registry.sh
```

4. To run the script without any parameters, enter the following command:

```
$ ${HOME}/registry.sh
```

The script starts the server. The script uses a **systemd** service for management purposes.

5. If you need to manage the script, you can use the following commands.

- a. To view the status, enter the following command:

```
$ systemctl status
```

- b. To start the script, enter the following command:

```
$ systemctl start
```

- c. To stop the script, enter the following command:

```
$ systemctl stop
```

The root folder for the registry is in the **/opt/registry** directory and contains the following subdirectories:

- **certs** contains the TLS certificates.

- **auth** contains the credentials.
- **data** contains the registry images.
- **conf** contains the registry configuration.

6.3.6. Setting up a management cluster for hosted control planes in a disconnected environment

An important part of a hosted control planes deployment is the OpenShift Container Platform management cluster. To set up an management cluster for a disconnected environment, you must install multicluster engine for Kubernetes Operator on it. The multicluster engine Operator plays a crucial role in deploying clusters across providers.

Prerequisites

- There must be bidirectional connectivity between the management cluster and the Baseboard Management Controller (BMC) of the target Bare Metal Host (BMH). As an alternative, you follow a Boot It Yourself approach through the Agent provider.
- The hosted cluster must be able to resolve and reach the API hostname of the management cluster hostname and ***.apps** hostname. Here is an example of the API hostname of the management cluster and ***.apps** hostname:
 - **api.management-cluster.internal.domain.com**
 - **console-openshift-console.apps.management-cluster.internal.domain.com**
- The management cluster must be able to resolve and reach the API and ***.apps** hostname of the hosted cluster. Here is an example of the API hostname of the hosted cluster and ***.apps** hostname:
 - **api.sno-hosted-cluster-1.internal.domain.com**
 - **console-openshift-console.apps.sno-hosted-cluster-1.internal.domain.com**

Procedure

1. Install multicluster engine Operator 2.7 or later on an OpenShift Container Platform cluster. You can install multicluster engine Operator as an Operator from the OpenShift Container Platform software catalog. The HyperShift Operator is included with multicluster engine Operator. For more information about installing multicluster engine Operator, see "Installing and upgrading multicluster engine operator" in the Red Hat Advanced Cluster Management documentation.
2. Ensure that the HyperShift Operator is installed. The HyperShift Operator is automatically included with multicluster engine Operator, but if you need to manually install it, follow the steps in "Manually enabling the hypershift-addon managed cluster add-on for local-cluster".

Next steps

Next, configure the web server.

Additional resources

- [Installing and upgrading multicluster engine operator](#)

- [Manually enabling the hypershift-addon managed cluster add-on for local-cluster](#)
- [Cluster lifecycle with multicluster engine operator overview](#)

6.3.7. Configuring the web server for hosted control planes in a disconnected environment

You must configure an additional web server to host the Red Hat Enterprise Linux CoreOS (RHCOS) images that are associated with the OpenShift Container Platform release that you are deploying as a hosted cluster.

Procedure

1. Extract the **openshift-install** binary from the OpenShift Container Platform release that you want to use by entering the following command:

```
$ oc adm -a ${LOCAL_SECRET_JSON} release extract --command=openshift-install \
  "${LOCAL_REGISTRY}/${LOCAL_REPOSITORY}:${OCP_RELEASE}-
  ${ARCHITECTURE}"
```

2. Run the following script. The script creates a folder in the **/opt/srv** directory. The folder contains the RHCOS images to provision the worker nodes.

```
#!/bin/bash

WEBSRV_FOLDER=/opt/srv
ROOTFS_IMG_URL="$(./openshift-install coreos print-stream-json | jq -r
'.architectures.x86_64.artifacts.metal.formats.pxe.rootfs.location')"
LIVE_ISO_URL="$(./openshift-install coreos print-stream-json | jq -r
'.architectures.x86_64.artifacts.metal.formats.iso.disk.location')"

mkdir -p ${WEBSRV_FOLDER}/images
curl -Lk ${ROOTFS_IMG_URL} -o
${WEBSRV_FOLDER}/images/${ROOTFS_IMG_URL##*/}
curl -Lk ${LIVE_ISO_URL} -o ${WEBSRV_FOLDER}/images/${LIVE_ISO_URL##*/}
chmod -R 755 ${WEBSRV_FOLDER}/*

## Run Webserver
podman ps --noheading | grep -q webserv-ai
if [[ $? == 0 ]];then
  echo "Launching Registry pod..."
  /usr/bin/podman run --name webserv-ai --net host -v /opt/srv:/usr/local/apache2/htdocs:z
  quay.io/alosadag/httpd:p8080
fi
```

- You can find the **ROOTFS_IMG_URL** value on the OpenShift CI Release page.
- You can find the **LIVE_ISO_URL** value on the OpenShift CI Release page.
After the download is completed, a container runs to host the images on a web server. The container uses a variation of the official HTTPd image, which also enables it to work with IPv6 networks.

6.3.8. Configuring image mirroring for hosted control planes in a disconnected environment

Image mirroring is the process of fetching images from external registries, such as **registry.redhat.com** or **quay.io**, and storing them in your private registry.

In the following procedure, the **oc-mirror** tool is used, which is a binary that uses the **ImageSetConfiguration** object. In the file, you can specify the following information:

- The OpenShift Container Platform versions to mirror. The versions are in **quay.io**.
- The additional Operators to mirror. Select packages individually.
- The extra images that you want to add to the repository.

Prerequisites

- Ensure that the registry server is running before you start the mirroring process.

Procedure

1. Ensure that your **\${HOME}/.docker/config.json** file is updated with the registries that you are going to mirror from and with the private registry that you plan to push the images to.
2. By using the following example, create an **ImageSetConfiguration** object to use for mirroring. Replace values as needed to match your environment:

```
apiVersion: mirror.openshift.io/v2alpha1
kind: ImageSetConfiguration
mirror:
  platform:
    channels:
      - name: candidate-4.22
        minVersion: <4.x.y-build>
        maxVersion: <4.x.y-build>
    type: ocp
  kubeVirtContainer: true
  graph: true
operators:
  - catalog: registry.redhat.io/redhat/redhat-operator-index:v4.22
  packages:
    - name: lvms-operator
    - name: local-storage-operator
    - name: odf-csi-addons-operator
    - name: odf-operator
    - name: mcg-operator
    - name: ocs-operator
    - name: metallb-operator
    - name: kubevirt-hyperconverged
```

- **mirror.platform.channels.minVersion** specifies the supported OpenShift Container Platform version you want to use.
- **mirror.platform.channels.maxVersion** specifies the supported (product-title) version you want to use.

- **kubeVirtContainer** specifies whether you want to also mirror to the container disk image for the Red Hat Enterprise Linux CoreOS (RHCOS) boot image for the KubeVirt provider. This flag is optional. It is available with oc-mirror v2 only.
- **mirror.operators.packages.name: kubevirt-hyperconverged** must be included for deployments that use the KubeVirt provider.

3. Start the mirroring process by entering the following command:

```
$ oc-mirror --v2 --config imagesetconfig.yaml \
  --workspace file://mirror-file docker://<registry>
```

After the mirroring process is finished, you have a new folder named **mirror-file**, which contains the **ImageDigestMirrorSet** (IDMS), **ImageTagMirrorSet** (ITMS), and the catalog sources to apply on the hosted cluster.

4. Mirror the nightly or CI versions of OpenShift Container Platform by configuring the **imagesetconfig.yaml** file as follows:

```
apiVersion: mirror.openshift.io/v2alpha1
kind: ImageSetConfiguration
mirror:
  platform:
    graph: true
    release: registry.ci.openshift.org/ocp/release:<4.x.y-build>
    kubeVirtContainer: true
# ...
```

- **mirror.platform.release** specifies the supported OpenShift Container Platform version you want to use.
 - **mirror.platform.kubeVirtContainer** specifies that you want to also mirror the container disk image for the Red Hat Enterprise Linux CoreOS (RHCOS) boot image for the KubeVirt provider. This flag is available with oc-mirror v2 only.
5. If you have a partially disconnected environment, mirror the images from the image set configuration to a registry by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml \
  --workspace file://<file_path> docker://<mirror_registry_url> --v2
```

For more information, see "Mirroring an image set in a partially disconnected environment".

6. If you have a fully disconnected environment, complete the following steps:

- a. Mirror the images from the specified image set configuration to the disk by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml file://<file_path> --v2
```

For more information, see "Mirroring an image set in a fully disconnected environment".

- b. Process the image set file on the disk and mirror the contents to a target mirror registry by entering the following command:

```
$ oc mirror -c imagesetconfig.yaml \
  --from file://<file_path> docker://<mirror_registry_url> --v2
```

7. Mirror the latest multicluster engine Operator images by following the steps in "Install on disconnected networks".

Additional resources

- [Mirroring an image set in a partially disconnected environment](#)
- [Mirroring an image set in a fully disconnected environment](#)
- [Install on disconnected networks](#)

6.3.9. Applying objects in the management cluster

After the mirroring process is complete, you must apply two objects required for mirroring in the management cluster.

You apply the following objects:

- **ImageContentSourcePolicy** (ICSP) or **ImageDigestMirrorSet** (IDMS)
- Catalog sources

When you use the **oc-mirror** tool, the output artifacts are in a folder named **oc-mirror-workspace/results-XXXXXX/**.

The **oc mirror** mirroring file initiates a **MachineConfig** change that does not restart your nodes but restarts the kubelet on each of them. After the nodes are marked as **READY**, you need to apply the newly generated catalog sources.

The catalog sources initiate actions in the **openshift-marketplace** Operator, such as downloading the catalog image and processing it to retrieve all the **PackageManifests** that are included in that image.

Procedure

1. To check the new sources, run the following command by using the new **CatalogSource** as a source:

```
$ oc get packagemanifest
```

2. To apply the artifacts, complete the following steps:

- a. Create the ICSP or IDMS artifacts by entering the following command:

```
$ oc apply -f oc-mirror-workspace/results-XXXXXX/imageContentSourcePolicy.yaml
```

- b. Wait for the nodes to become ready, and then enter the following command:

```
$ oc apply -f catalogSource-XXXXXXXXX-index.yaml
```

3. Mirror the OLM catalogs and configure the hosted cluster to point to the mirror.

When you use the **management** (default) OLMCatalogPlacement mode, the image stream that is used for OLM catalogs is not automatically amended with override information from the ICSP on the management cluster.

- a. If the OLM catalogs are properly mirrored to an internal registry by using the original name and tag, add the **hypershift.openshift.io/olm-catalogs-is-registry-overrides** annotation to the **HostedCluster** resource. The format is "**sr1=dr1,sr2=dr2**", where the source registry string is a key and the destination registry is a value.
- b. To bypass the OLM catalog image stream mechanism, use the following four annotations on the **HostedCluster** resource to directly specify the addresses of the four images to use for OLM Operator catalogs:

- **hypershift.openshift.io/certified-operators-catalog-image**
- **hypershift.openshift.io/community-operators-catalog-image**
- **hypershift.openshift.io/redhat-marketplace-catalog-image**
- **hypershift.openshift.io/redhat-operators-catalog-image**

In this case, the image stream is not created, and you must update the value of the annotations when the internal mirror is refreshed to pull in Operator updates.

Next steps

Deploy the multicluster engine Operator by completing the steps in "Deploying multicluster engine Operator for a disconnected installation of hosted control planes".

Additional resources

- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

6.3.10. Deploying AgentServiceConfig resources

The **AgentServiceConfig** custom resource (CR) is an essential component of the Assisted Service add-on that is part of multicluster engine Operator. The CR is responsible for bare-metal cluster deployment. When the add-on is enabled, you deploy the **AgentServiceConfig** resource to configure the add-on.

In addition to configuring the **AgentServiceConfig** resource, you need to include additional config maps to ensure that multicluster engine Operator functions properly in a disconnected environment.

Procedure

1. Configure the custom registries by adding the following config map, which has the disconnected details to customize the deployment:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: custom-registries
  namespace: multicluster-engine
labels:
  app: assisted-service
data:
  ca-bundle.crt: |
```

```

-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
registries.conf: |
  unqualified-search-registries = ["registry.access.redhat.com", "docker.io"]

  [[registry]]
  prefix = ""
  location = "registry.redhat.io/openshift4"
  mirror-by-digest-only = true

  [[registry.mirror]]
  location = "registry.dns.base.domain.name:5000/openshift4"

  [[registry]]
  prefix = ""
  location = "registry.redhat.io/rhacm2"
  mirror-by-digest-only = true
  # ...
  # ...

```

- The **ca-bundle.crt** field has the Certificate Authorities (CAs) that are loaded into the various processes of the deployment.
 - The **registries.conf** field has information about images and namespaces that need to be consumed from a mirror registry rather than the original source registry.
 - Replace the **dns.base.domain.name** in the **registry.mirror** container reference with the DNS base domain name.
2. Configure the Assisted Service by adding the **AssistedServiceConfig** object, as shown in the following example:

```

apiVersion: agent-install.openshift.io/v1beta1
kind: AgentServiceConfig
metadata:
  annotations:
    unsupported.agent-install.openshift.io/assisted-service-configmap: assisted-service-config
  name: agent
  namespace: multicluster-engine
spec:
  mirrorRegistryRef:
    name: custom-registries
  databaseStorage:
    storageClassName: lvms-vg1
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  filesystemStorage:
    storageClassName: lvms-vg1
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi

```

```

osImages:
  - cpuArchitecture: x86_64
    openshiftVersion: "4.14"
    rootFSUrl: http://registry.dns.base.domain.name:8080/images/rhcos-
414.92.202308281054-0-live-rootfs.x86_64.img
    url: http://registry.dns.base.domain.name:8080/images/rhcos-414.92.202308281054-0-
live.x86_64.iso
    version: 414.92.202308281054-0
  - cpuArchitecture: x86_64
    openshiftVersion: "4.15"
    rootFSUrl: http://registry.dns.base.domain.name:8080/images/rhcos-
415.92.202403270524-0-live-rootfs.x86_64.img
    url: http://registry.dns.base.domain.name:8080/images/rhcos-415.92.202403270524-0-
live.x86_64.iso
    version: 415.92.202403270524-0

```

- The **metadata.annotations["unsupported.agent-install.openshift.io/assisted-service-configmap"]** annotation references the config map name that the Operator consumes to customize behavior.
 - The **spec.mirrorRegistryRef.name** field points to the **ConfigMap** CR that has the disconnected registry information that the Assisted Service Operator consumes. This config map adds those resources during the deployment process.
 - The **spec.osImages** field contains different versions available for deployment by this Operator. This field is mandatory. This example assumes that you already downloaded the **RootFS** and **LiveISO** files.
 - The **cpuArchitecture** field is added for every OpenShift Container Platform release that you want to deploy. In this example, **cpuArchitecture** subsections are included for 4.14 and 4.15.
 - The **osImages.rootFSUrl** field includes **dns.base.domain.name**. Replace that value with the DNS base domain name.
 - The **osImages.url** field includes **dns.base.domain.name**. Replace that value with the DNS base domain name.
3. Deploy all of the objects by concatenating them into a single file and applying them to the management cluster. To do so, enter the following command:

```
$ oc apply -f agentServiceConfig.yaml
```

The command triggers two pods.

Example output

```

assisted-image-service-0          1/1   Running 2      11d
assisted-service-668b49548-9m7xw  2/2   Running 5      11d

```

- The **assisted-image-service** pod is responsible for creating the Red Hat Enterprise Linux CoreOS (RHCOS) boot image template, which is customized for each cluster that you deploy.
- The **assisted-service** refers to the Operator.

Next steps

Configure TLS certificates.

6.3.11. Adding the registry CA to the management cluster

To ensure proper function in a disconnected deployment, you need to configure the registry CA certificates in the management cluster.

To add the registry CA to the management cluster, complete the following steps.

Procedure

1. Create a config map that resembles the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: <config_map_name>
  namespace: <config_map_namespace>
data:
  <registry_name>..

-----BEGIN CERTIFICATE-----
  -----END CERTIFICATE-----
  <registry_name>..

-----BEGIN CERTIFICATE-----
  -----END CERTIFICATE-----
  <registry_name>..

-----BEGIN CERTIFICATE-----
  -----END CERTIFICATE-----


```

- **metadata.name** specifies the name of the config map.
 - **metadata.namespace** specifies the namespace for the config map.
 - **data** specifies the registry names and the registry certificate content. Replace **<port>** with the port where the registry server is running; for example, **5000**. Ensure that the data in the config map is defined by using **|** only instead of other methods, such as **| -**. If you use other methods, issues can occur when the pod reads the certificates.
2. Patch the cluster-wide object, **image.config.openshift.io** to include the following specification:

```
spec:
  additionalTrustedCA:
    - name: registry-config
```

As a result of this patch, the control plane nodes can retrieve images from the private registry and the HyperShift Operator can extract the OpenShift Container Platform payload for hosted cluster deployments.

The process to patch the object might take several minutes to be completed.

6.3.12. Adding the registry CA to the compute nodes for the hosted cluster

To ensure that the data plane compute nodes in the hosted cluster can retrieve images from the private registry, you must add the registry certificate authority (CA) to the compute nodes.

Procedure

1. In the **hc.spec.additionalTrustBundle** file, add the following specification:

```
spec:
  additionalTrustBundle:
    name: user-ca-bundle
```

The **user-ca-bundle** entry is a config map that you create in the next step.

2. In the same namespace where the **HostedCluster** object is created, create the **user-ca-bundle** config map. The config map resembles the following example:

```
apiVersion: v1
data:
  ca-bundle.crt: |
    // Registry1 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

    // Registry2 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

    // Registry3 CA
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----

kind: ConfigMap
metadata:
  name: user-ca-bundle
  namespace: <hosted_cluster_namespace>
```

Specify the namespace where the **HostedCluster** object is created.

6.3.13. Hosted clusters on bare metal in a disconnected environment

In a disconnected environment, creating a hosted cluster involves deploying hosted cluster objects, creating node pools, creating an **InfraEnv** resource, creating bare-metal hosts, and scaling the node pools as needed.

A hosted cluster is an OpenShift Container Platform cluster with its control plane and API endpoint hosted on a management cluster. The hosted cluster includes the corresponding data plane.

6.3.13.1. Deploying hosted cluster objects

Typically, the HyperShift Operator creates the **HostedControlPlane** namespace. However, you might want to include all the objects before the HyperShift Operator begins to reconcile the **HostedCluster** object. Then, when the Operator starts the reconciliation process, it can find all of the objects in place.

Procedure

1. Create a YAML file with the following information about the namespaces:

```
---
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: null
  name: <hosted_cluster_namespace>-<hosted_cluster_name>
spec: {}
status: {}
---
apiVersion: v1
kind: Namespace
metadata:
  creationTimestamp: null
  name: <hosted_cluster_namespace>
spec: {}
status: {}
```

- **<hosted_cluster_name>** is the name of your hosted cluster.
 - **<hosted_cluster_namespace>** is the name of your hosted cluster namespace.
2. Create a YAML file with the following information about the config maps and secrets to include in the **HostedCluster** deployment:

```
---
apiVersion: v1
data:
  ca-bundle.crt: |
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----
kind: ConfigMap
metadata:
  name: user-ca-bundle
  namespace: <hosted_cluster_namespace>
---
apiVersion: v1
data:
  .dockerconfigjson: xxxxxxxxx
kind: Secret
metadata:
  creationTimestamp: null
  name: <hosted_cluster_name>-pull-secret
  namespace: <hosted_cluster_namespace>
---
apiVersion: v1
kind: Secret
metadata:
  name: sshkey-cluster-<hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
stringData:
  id_rsa.pub: ssh-rsa xxxxxxxxx
---
apiVersion: v1
data:
```



```

key: nTPtVBEt03owkrKhldmSW8jrWRxU57KO/fnZa8oaG0Y=
kind: Secret
metadata:
  creationTimestamp: null
  name: <hosted_cluster_name>-etcd-encryption-key
  namespace: <hosted_cluster_namespace>
  type: Opaque

```

- **<hosted_cluster_namespace>** is the name of your hosted cluster namespace.
- **<hosted_cluster_name>** is the name of your hosted cluster.

3. Create a YAML file that contains the RBAC roles so that Assisted Service agents can be in the same **HostedControlPlane** namespace as the hosted control plane and still be managed by the cluster API:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  creationTimestamp: null
  name: capi-provider-role
  namespace: <hosted_cluster_namespace>-<hosted_cluster_name>
rules:
- apiGroups:
  - agent-install.openshift.io
  resources:
  - agents
  verbs:
  - '*'

```

- **<hosted_cluster_namespace>** is the name of your hosted cluster namespace.
- **<hosted_cluster_name>** is the name of your hosted cluster.

4. Create a YAML file with information about the **HostedCluster** object, replacing values as necessary:

```

apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  additionalTrustBundle:
    name: "user-ca-bundle"
  olmCatalogPlacement: guest
  configuration:
    operatorhub:
      disableAllDefaultSources: true
  imageContentSources:
  - source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
    mirrors:
    - registry.<dns.base.domain.name>:5000/openshift/release
  - source: quay.io/openshift-release-dev/ocp-release
    mirrors:
    - registry.<dns.base.domain.name>:5000/openshift/release-images

```

```

- mirrors:
  - registry.<dns.base.domain.name>:5000/openshift/release-images
- source: registry.redhat.io/multicloud-engine
  mirrors:
  - registry.<dns.base.domain.name>:5000/openshift/multicloud-engine
# ...
autoscaling: {}
controllerAvailabilityPolicy: SingleReplica
dns:
  baseDomain: <dns.base.domain.name>
etcd:
  managed:
  storage:
    persistentVolume:
      size: 8Gi
      restoreSnapshotURL: null
      type: PersistentVolume
    managementType: Managed
fips: false
networking:
  clusterNetwork:
  - cidr: 10.132.0.0/14
  - cidr: fd01::/48
  networkType: OVNKubernetes
  serviceNetwork:
  - cidr: 172.31.0.0/16
  - cidr: fd02::/112
platform:
  agent:
    agentNamespace: <bmh_infraenv_namespace>
    type: Agent
pullSecret:
  name: <hosted_cluster_name>-pull-secret
release:
  image: registry.<dns.base.domain.name>:5000/openshift/release-images:<4.x.y>-x86_64
secretEncryption:
  aescbc:
    activeKey:
      name: <hosted_cluster_name>-etcd-encryption-key
      type: aescbc
services:
- service: APIServer
  servicePublishingStrategy:
    type: LoadBalancer
- service: OAuthServer
  servicePublishingStrategy:
    type: Route
- service: OIDC
  servicePublishingStrategy:
    type: Route
- service: Konnectivity
  servicePublishingStrategy:
    type: Route
- service: Ignition
  servicePublishingStrategy:
    type: Route

```

```
sshKey:
  name: sshkey-cluster-<hosted_cluster_name>
status:
controlPlaneEndpoint:
  host: ""
  port: 0
```

- **<hosted_cluster_name>** is the name of your hosted cluster.
 - **<hosted_cluster_namespace>** is the name of your hosted cluster namespace.
 - **disableAllDefaultSources** is **true** if you want to disable all default OLM catalog resources. The default value is **false**, which enables all default OLM catalog resources.
 - **imageContentSources** contains mirror references for user workloads within the hosted cluster.
 - **<dns.base.domain.name>** is the DNS base domain name.
 - **<bhm_infraenv_namespace>** is the namespace where the Bare Metal Host (BMH) and **InfraEnv** resources are created.
 - **<4.x.y>** is the supported OpenShift Container Platform version you want to use.
5. Create all of the objects that you defined in the YAML files by concatenating them into a file and applying them against the management cluster. To do so, enter the following command:

```
$ oc apply -f 01-4.14-hosted_cluster-nodeport.yaml
```

The following example shows the output of the command:

NAME	READY	STATUS	RESTARTS	AGE
capi-provider-5b57dbd6d5-pxlqc	1/1	Running	0	3m57s
catalog-operator-9694884dd-m7zzv	2/2	Running	0	93s
cluster-api-f98b9467c-9hfrq	1/1	Running	0	3m57s
cluster-autoscaler-d7f95dd5-d8m5d	1/1	Running	0	93s
cluster-image-registry-operator-5ff5944b4b-648ht	1/2	Running	0	93s
cluster-network-operator-77b896ddc-wpkq8	1/1	Running	0	94s
cluster-node-tuning-operator-84956cd484-4hfgf	1/1	Running	0	94s
cluster-policy-controller-5fd8595d97-rhbwf	1/1	Running	0	95s
cluster-storage-operator-54dcf584b5-xrnts	1/1	Running	0	93s
cluster-version-operator-9c554b999-l22s7	1/1	Running	0	95s
control-plane-operator-6fdc9c569-t7hr4	1/1	Running	0	3m57s
csi-snapshot-controller-785c6dc77c-8ljmr	1/1	Running	0	77s
csi-snapshot-controller-operator-7c6674bc5b-d9dtp	1/1	Running	0	93s
csi-snapshot-webhook-5b8584875f-2492j	1/1	Running	0	77s
dns-operator-6874b577f-9tc6b	1/1	Running	0	94s
etcd-0	3/3	Running	0	3m39s
hosted-cluster-config-operator-f5cf5c464-4nmbh	1/1	Running	0	93s
ignition-server-6b689748fc-zdqzk	1/1	Running	0	95s
ignition-server-proxy-54d4bb9b9b-6zkg7	1/1	Running	0	95s
ingress-operator-6548dc758b-f9gtg	1/2	Running	0	94s
konnnectivity-agent-7767cdc6f5-tw782	1/1	Running	0	95s
kube-apiserver-7b5799b6c8-9f5bp	4/4	Running	0	3m7s
kube-controller-manager-5465bc4dd6-zpdlk	1/1	Running	0	44s
kube-scheduler-5dd5f78b94-bbbck	1/1	Running	0	2m36s

```

machine-approver-846c69f56-jxvfr      1/1   Running 0    92s
oauth-openshift-79c7bf44bf-j975g     2/2   Running 0    62s
olm-operator-767f9584c-4lcl2         2/2   Running 0    93s
openshift-apiserver-5d469778c6-pl8tj  3/3   Running 0    2m36s
openshift-controller-manager-6475fdff58-hl4f7 1/1   Running 0    95s
openshift-oauth-apiserver-dbbc5cc5f-98574 2/2   Running 0    95s
openshift-route-controller-manager-5f6997b48f-s9vdc 1/1   Running 0    95s
packageserver-67c87d4d4f-kl7qh       2/2   Running 0    93s

```

When the hosted cluster is available, the output looks like the following example:

```

NAMESPACE NAME      VERSION KUBECONFIG      PROGRESS AVAILABLE
PROGRESSING MESSAGE
clusters hosted-dual hosted-admin-kubeconfig Partial True      False      The
hosted control plane is available

```

6.3.13.2. Creating a NodePool object for the hosted cluster

A **NodePool** object is a scalable set of compute nodes that is associated with a hosted cluster.

NodePool machine architectures remain consistent within a specific pool and are independent of the machine architecture of the control plane.

Procedure

1. Create a YAML file with the following information about the **NodePool** object, replacing values as necessary:

```

apiVersion: hypershift.openshift.io/v1beta1
kind: NodePool
metadata:
  creationTimestamp: null
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  arch: amd64
  clusterName: <hosted_cluster_name>
  management:
    autoRepair: false
    upgradeType: InPlace
  nodeDrainTimeout: 0s
  platform:
    type: Agent
  release:
    image: registry.<dns.base.domain.name>:5000/openshift/release-images:4.x.y-x86_64
  replicas: 2
status:
  replicas: 2

```

- **metadata.name** specifies the name of your hosted cluster.
- **metadata.namespace** specifies the name of your hosted cluster namespace.

- **spec.management.autoRepair** specifies whether the node will be re-created if it is removed. In this example, the **autoRepair** field is set to **false** because the node will not be re-created if it is removed.
- **spec.management.upgradeType** specifies the upgrade type. In this example, the **upgradeType** field is set to **InPlace**, which indicates that the same bare-metal node is reused during an upgrade.
- **spec.release.image** specifies which OpenShift Container Platform version the nodes in the **NodePool** object are based on. Replace the **<dns.base.domain.name>** value with your DNS base domain name and the **4.x.y** value with the supported OpenShift Container Platform version you want to use.
- **spec.replicas** specifies the number of node pool replicas to create. In this example, the **replicas** value is **2** to create two node pool replicas in your hosted cluster.

2. Create the **NodePool** object by entering the following command:

```
$ oc apply -f 02-nodepool.yaml
```

Example output

NAMESPACE	NAME	CLUSTER	DESIRED NODES	CURRENT NODES	UPDATINGVERSION
AUTOSCALING	AUTOREPAIR	VERSION			
UPDATINGCONFIG	MESSAGE				
clusters	hosted-dual	hosted	0	False	False
					4.x.y-x86_64

6.3.13.3. Creating an InfraEnv resource for the hosted cluster

The **InfraEnv** resource is an Assisted Service object that includes essential details that are used to create the Red Hat Enterprise Linux CoreOS (RHCOS) boot image that is customized for the hosted cluster.

You can host more than one **InfraEnv** resource, and each one can adopt certain types of hosts. For example, you might want to divide your server farm between a host that has greater RAM capacity.

Procedure

1. Create a YAML file with the following information about the **InfraEnv** resource, replacing values as necessary:

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: ssh-rsa
  AAAAB3NzaC1yc2EAAAADAQABAAQgQDk7ICaUE+/k4zTpxLk4+xFdHi4ZuDi5qjeF52afsNk
  w0w/gllHhwpL5gnp5WkRuL8GwJuZ1VqLC9EKrdmegn4MrmUlq7WTsP0VFOZFBfq2XRUXo
  1wrRdor2z0Bbh93ytR+ZsDbbLIGngXaMa0Vbt+z74FqlcajbHTZ6zBmTpBVq5RHtDPgKITdpE1f
  ongp7+ZXQNBIkaavaqv8bnyrP4BWahLP4iO9/xJF9IQYboYwEEDzmnKLMW1VtCE6nJzEgWC
```

```
ufACTbxpNS7GvKtoHT/OVzw8ArEXhZXQUS1UY8zKsX2iXwmyhw5Sj6YboA8WICs4z+TrFP8
9LmxXY0j6536TQFyRz1iB4WWvCbH5n6W+ABV2e8ssJB1AmEy8QYNwpJQJNpSxzoKBjI73X
vPYYC/ljPFMySwZqrSZCkJYqQ023ySkaQxWZT7in4KeMu7eS2tC+Kn4deJ7KwwUycx8n6RH
MeD8Qg9fITHCv3gmab8JKZJqN3hW1D378JuvmlX4V0=
```

- **metadata.name** specifies the name of your hosted cluster namespace.
- **spec.pullSecretRef** specifies the config map reference in the same namespace as the **InfraEnv**, where the pull secret is used.
- **spec.sshAuthorizedKey** specifies the SSH public key that is placed in the boot image. The SSH key allows access to the worker nodes as the **core** user.

2. Create the **InfraEnv** resource by entering the following command:

```
$ oc apply -f 03-infraenv.yaml
```

Example output

```
NAMESPACE      NAME      ISO CREATED AT
clusters-hosted-dual  hosted  2023-09-11T15:14:10Z
```

6.3.13.4. Creating bare-metal hosts for the hosted cluster

A *bare-metal host* is an **openshift-machine-api** object that encompasses physical and logical details so that it can be identified by a Metal3 Operator. Those details are associated with other Assisted Service objects, known as *agents*.

Prerequisites

- Before you create the bare-metal host and destination nodes, you must have the destination machines ready.
- You have installed a Red Hat Enterprise Linux CoreOS (RHCOS) compute machine on bare-metal infrastructure for use in the cluster.

Procedure

1. Create a YAML file with the following information. For more information about what details to enter for the bare-metal host, see "Provisioning new hosts in a user-provisioned cluster by using the BMO".

Because you have at least one secret that holds the bare-metal host credentials, you need to create at least two objects for each compute node.

```
apiVersion: v1
kind: Secret
metadata:
  name: <hosted_cluster_name>-worker0-bmc-secret
  namespace: <hosted_cluster_namespace>
data:
  password: YWRtaW4=
  username: YWRtaW4=
type: Opaque
# ...
```

```

apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: <hosted_cluster_name>-worker0
  namespace: <hosted_cluster_namespace>
  labels:
    infraenvs.agent-install.openshift.io: <hosted_cluster_name>
  annotations:
    inspect.metal3.io: disabled
    bmac.agent-install.openshift.io/hostname: <hosted_cluster_name>-worker0
spec:
  automatedCleaningMode: disabled
  bmc:
    disableCertificateVerification: true
    address: redfish-
  virtualmedia://[192.168.126.1]:9000/redfish/v1/Systems/local/<hosted_cluster_name>-
  worker0
    credentialsName: <hosted_cluster_name>-worker0-bmc-secret
  bootMACAddress: aa:aa:aa:aa:02:11
  online: true

```

- **metadata.name** specifies the name of your hosted cluster.
- **metadata.namespace** specifies the name of your hosted cluster namespace.
- **data.password** specifies the password of the baseboard management controller (BMC) in Base64 format.
- **data.username** specifies the user name of the BMC in Base64 format.
- **metadata.labels.infraenvs.agent-install.openshift.io** serves as the link between the Assisted Installer and the **BareMetalHost** objects.
- **metadata.annotations.bmac.agent-install.openshift.io/hostname** represents the node name that is adopted during deployment.
- **spec.automatedCleaningMode** prevents the node from being erased by the Metal3 Operator.
- **spec.bmc.disableCertificateVerification** is set to **true** to bypass certificate validation from the client.
- **spec.bmc.address** denotes the BMC address of the compute node.
- **spec.bmc.credentialsName** points to the secret where the user and password credentials are stored.
- **spec.bootMACAddress** indicates the interface MAC address that the node starts from.
- **spec.online** defines the state of the node after the **BareMetalHost** object is created.

2. Deploy the **BareMetalHost** object by entering the following command:

```
$ oc apply -f 04-bmh.yaml
```

During the process, the statuses of the bare-metal hosts change from **Registering** to

Provisioning to **Provisioned**. The nodes start with the **LiveISO** of the agent and a default pod that is named **agent**. That agent is responsible for receiving instructions from the Assisted Service Operator to install the OpenShift Container Platform payload.

- This output indicates that the process is trying to reach the nodes:

Example output

NAMESPACE	NAME	STATE	CONSUMER	ONLINE	ERROR	AGE
clusters-hosted	hosted-worker0	registering	true	2s		
clusters-hosted	hosted-worker1	registering	true	2s		
clusters-hosted	hosted-worker2	registering	true	2s		

- This output indicates that the nodes are starting:

Example output

NAMESPACE	NAME	STATE	CONSUMER	ONLINE	ERROR	AGE
clusters-hosted	hosted-worker0	provisioning	true	16s		
clusters-hosted	hosted-worker1	provisioning	true	16s		
clusters-hosted	hosted-worker2	provisioning	true	16s		

- This output indicates that the nodes started successfully:

Example output

NAMESPACE	NAME	STATE	CONSUMER	ONLINE	ERROR	AGE
clusters-hosted	hosted-worker0	provisioned	true	67s		
clusters-hosted	hosted-worker1	provisioned	true	67s		
clusters-hosted	hosted-worker2	provisioned	true	67s		

3. After the nodes start, notice the agents in the namespace, as shown in this example:

Example output

NAMESPACE	NAME	CLUSTER	APPROVED	ROLE
STAGE				
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0411		true	auto-assign
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0412		true	auto-assign
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0413		true	auto-assign

The agents represent nodes that are available for installation. To assign the nodes to a hosted cluster, scale up the node pool.

Additional resources

- [Provisioning new hosts in a user-provisioned cluster by using the BMO](#)
- [Understanding secrets](#)

6.3.13.5. Scaling up the node pool

After you create bare-metal hosts, agents are added to your namespace. The agents represent nodes that are available for installation. To assign the nodes to a hosted cluster, scale up the node pool.

Prerequisites

- You created bare-metal hosts for your hosted cluster.

Procedure

1. To scale up the node pool, enter the following command:

```
$ oc -n <hosted_cluster_namespace> scale nodepool <hosted_cluster_name> \
--replicas 3
```

- **<hosted_cluster_namespace>** is the name of the hosted cluster namespace.
 - **<hosted_cluster_name>** is the name of the hosted cluster.
2. After the scaling process is complete, notice that the agents are assigned to a hosted cluster:

Example output

NAMESPACE	NAME	CLUSTER	APPROVED	ROLE
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0411	hosted	true	auto-assign
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0412	hosted	true	auto-assign
clusters-hosted	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaa0413	hosted	true	auto-assign

3. Also notice that the node pool replicas are set, as shown in the following example:

Example output

NAMESPACE	NAME	CLUSTER	DESIRED NODES	CURRENT NODES
clusters	hosted	hosted	3	False

Minimum availability requires 3 replicas, current 0 available

In your output, **<4.x.y>** is replaced with the supported OpenShift Container Platform version that you are using.

4. Wait until the nodes join the cluster. During the process, the agents provide updates on their stage and status.

6.3.14. Additional resources

- [Enabling the central infrastructure management service](#)

6.4. DEPLOYING HOSTED CONTROL PLANES ON IBM Z IN A DISCONNECTED ENVIRONMENT

Hosted control planes deployments in disconnected environments function differently than in a standalone OpenShift Container Platform.

Hosted control planes involves two distinct environments:

- Control plane: Located in the management cluster, where the hosted control planes pods are run and managed by the Control Plane Operator.
- Data plane: Located in the workers of the hosted cluster, where the workload and a few other pods run, managed by the Hosted Cluster Config Operator.

The **ImageContentSourcePolicy** (ICSP) custom resource for the data plane is managed through the **ImageContentSources** API in the hosted cluster manifest.

For the control plane, ICSP objects are managed in the management cluster. These objects are parsed by the HyperShift Operator and are shared as **registry-overrides** entries with the Control Plane Operator. These entries are injected into any one of the available deployments in the hosted control planes namespace as an argument.

To work with disconnected registries in the hosted control planes, you must first create the appropriate ICSP in the management cluster. Then, to deploy disconnected workloads in the data plane, you need to add the entries that you want into the **ImageContentSources** field in the hosted cluster manifest.

6.4.1. Prerequisites to deploy hosted control planes on IBM Z in a disconnected environment

To deploy hosted control planes on IBM Z in a disconnected environment, you must meet a few prerequisites.

You need the following resources:

- A mirror registry. For more information, see "Mirror registry for Red Hat OpenShift introduction".
- A mirrored image for a disconnected installation. For more information, see "Mirroring images for a disconnected installation using the oc-mirror plugin".

Additional resources

- [Mirror registry for Red Hat OpenShift introduction](#)
- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

6.4.2. Adding credentials and the registry certificate authority to the management cluster

To pull the mirror registry images from the management cluster, you must first add credentials and the certificate authority of the mirror registry to the management cluster.

Procedure

1. Create a **ConfigMap** with the certificate of the mirror registry by running the following command:

```
$ oc apply -f registry-config.yaml
```

Example output

–

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: registry-config
  namespace: openshift-config
data:
  <mirror_registry>: |
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----
#...

```

2. Patch the **image.config.openshift.io** cluster-wide object to include the following entries:

```

spec:
  additionalTrustedCA:
    - name: registry-config

```

3. Update the management cluster pull secret to add the credentials of the mirror registry.

- a. Fetch the pull secret from the cluster in a JSON format by running the following command:

```

$ oc get secret/pull-secret -n openshift-config -o json \
  | jq -r '.data[".dockerconfigjson"]' \
  | base64 -d > authfile

```

- b. Edit the fetched secret JSON file to include a section with the credentials of the certificate authority:

```

"auths": {
  "<mirror_registry>": {
    "auth": "<credentials>",
    "email": "you@example.com"
  }
},

```

- **<mirror_registry>** specifies the name of the mirror registry.
- **<credentials>** specifies the credentials for the mirror registry to allow fetch of images.

- c. Update the pull secret on the cluster by running the following command:

```

$ oc set data secret/pull-secret -n openshift-config \
  --from-file=.dockerconfigjson=authfile

```

6.4.3. Update the registry certificate authority in the **AgentServiceConfig** resource with the mirror registry

When you use a mirror registry for images, agents need to trust the registry's certificate to securely pull images. You can add the certificate authority of the mirror registry to the **AgentServiceConfig** custom resource by creating a **ConfigMap**.

Prerequisites

- You must have installed multicluster engine for Kubernetes Operator.

Procedure

1. In the same namespace where you installed multicluster engine Operator, create a **ConfigMap** resource with the mirror registry details. This **ConfigMap** resource ensures that you grant the hosted cluster workers the capability to retrieve images from the mirror registry.

Example ConfigMap file

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mirror-config
  namespace: multicluster-engine
  labels:
    app: assisted-service
data:
  ca-bundle.crt: |
    -----BEGIN CERTIFICATE-----
    -----END CERTIFICATE-----
  registries.conf: |

  [[registry]]
    location = "registry.stage.redhat.io"
    insecure = false
    blocked = false
    mirror-by-digest-only = true
    prefix = ""

  [[registry.mirror]]
    location = "<mirror_registry>"
    insecure = false

  [[registry]]
    location = "registry.redhat.io/multicluster-engine"
    insecure = false
    blocked = false
    mirror-by-digest-only = true
    prefix = ""

  [[registry.mirror]]
    location = "<mirror_registry>/multicluster-engine"
    insecure = false
```

Replace **<mirror_registry>** with the name of the mirror registry.

2. Patch the **AgentServiceConfig** resource to include the **ConfigMap** resource that you created. If the **AgentServiceConfig** resource is not present, create the **AgentServiceConfig** resource with the following content embedded into it:

```
spec:
  mirrorRegistryRef:
    name: mirror-config
```

6.4.4. Adding the registry certificate authority to the hosted cluster

When you are deploying hosted control planes on IBM Z in a disconnected environment, include the **additional-trust-bundle** and **image-content-sources** resources. The hosted cluster uses those resources to inject the certificate authority into the data plane compute nodes so that the images are pulled from the registry.

Procedure

1. Create the **icsp.yaml** file with the **image-content-sources** information.
The **image-content-sources** information is available in the **ImageContentSourcePolicy** YAML file that is generated after you mirror the images by using **oc-mirror**.

Example ImageContentSourcePolicy file

```
# cat icsp.yaml
- mirrors:
  - <mirror_registry>/openshift/release
    source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
- mirrors:
  - <mirror_registry>/openshift/release-images
    source: quay.io/openshift-release-dev/ocp-release
```

2. Create a hosted cluster and provide the **additional-trust-bundle** certificate to update the compute nodes with the certificates as in the following example:

```
$ hcp create cluster agent \
  --name=my-hosted-cluster \
  --pull-secret=/user/name/pullsecret \
  --agent-namespace=clusters-hosted \
  --base-domain=example.com \
  --api-server-address=api.my-hosted-cluster.example.com \
  --etcd-storage-class=lvm-storageclass \
  --ssh-key ~/.ssh/id_rsa.pub \
  --namespace <hosted_cluster_namespace> \
  --control-plane-availability-policy SingleReplica \
  --release-image=quay.io/openshift-release-dev/ocp-release:4.22.0-multi \
  --additional-trust-bundle <path for cert> \
  --image-content-sources icsp.yaml
```

- **--name** specifies the name of your hosted cluster.
- **--pull-secret** specifies the path to your pull secret.
- **--agent-namespace** specifies the name of the hosted control plane namespace.
- **--base-domain** specifies the name of your base domain.
- **--etcd-storage-class** specifies the etcd storage class name.
- **--ssh-key** specifies the path to your SSH public key. The default file path is **~/.ssh/id_rsa.pub**.
- **--namespace** specifies the name of the hosted cluster namespace.

- **--release-image** specifies the supported OpenShift Container Platform version that you want to use.
- **--additional-trust-bundle** specifies the path to the Certificate Authority of the mirror registry.

6.5. MONITORING USER WORKLOAD IN A DISCONNECTED ENVIRONMENT

When the **--enable-uwm-telemetry-remote-write** option is enabled, user workload monitoring is enabled and it can remotely write telemetry metrics from control planes.

6.5.1. Resolving user workload monitoring issues

If you installed multicluster engine Operator on OpenShift Container Platform clusters that are not connected to the internet, when you try to run the user workload monitoring feature, it might fail with an error.

For example, when you try to run the user workload monitoring feature by entering the following command, it fails with an error:

```
$ oc get events -n hypershift
```

```
LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
4m46s      Warning   ReconcileError deployment/operator Failed to ensure UWM telemetry remote
write: cannot get telemeter client secret: Secret "telemeter-client" not found
```

To resolve the error, you must disable the user workload monitoring option by creating a config map in the **local-cluster** namespace. You can create the config map either before or after you enable the add-on. The add-on agent reconfigures the HyperShift Operator.

Procedure

1. Create the following config map:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: hypershift-operator-install-flags
  namespace: local-cluster
data:
  installFlagsToAdd: ""
  installFlagsToRemove: "--enable-uwm-telemetry-remote-write"
```

2. Apply the config map by running the following command:

```
$ oc apply -f <filename>.yaml
```

6.5.2. Verifying the status of the hosted control plane feature

The hosted control plane feature is enabled by default. However, if you are not sure that it is enabled, you can run a command to verify its status.

Procedure

1. If the feature is disabled and you want to enable it, enter the following command. Replace **<multiclusterengine>** with the name of your multicluster engine Operator instance:

```
$ oc patch mce <multiclusterengine> --type=merge -p \
  '{"spec":{"overrides":{"components":[{"name":"hypershift","enabled": true}]}}}'
```

When you enable the feature, the **hypershift-addon** managed cluster add-on is installed in the **local-cluster** managed cluster, and the add-on agent installs the HyperShift Operator on the multicluster engine Operator hub cluster.

2. Confirm that the **hypershift-addon** managed cluster add-on is installed by entering the following command:

```
$ oc get managedclusteraddons -n local-cluster hypershift-addon
```

Example output

```
NAME          AVAILABLE DEGRADED PROGRESSING
hypershift-addon True      False
```

3. To avoid a timeout during this process, enter the following commands:
 - a. To avoid a timeout when the condition is **Degraded**, enter the following command:

```
$ oc wait --for=condition=Degraded=True managedclusteraddons/hypershift-addon \
  -n local-cluster --timeout=5m
```

- b. To avoid a timeout when the condition is **Available**, enter the following command:

```
$ oc wait --for=condition=Available=True managedclusteraddons/hypershift-addon \
  -n local-cluster --timeout=5m
```

When the process is complete, the **hypershift-addon** managed cluster add-on and the HyperShift Operator are installed, and the **local-cluster** managed cluster is available to host and manage hosted clusters.

6.5.3. Configuring the hypershift-addon managed cluster add-on to run on an infrastructure node

By default, no node placement preference is specified for the **hypershift-addon** managed cluster add-on. Consider running the add-ons on the infrastructure nodes, because by doing so, you can prevent incurring billing costs against subscription counts and separate maintenance and management tasks.

Procedure

1. Log in to the hub cluster.
2. Open the **hypershift-addon-deploy-config** add-on deployment configuration specification for editing by entering the following command:

```
$ oc edit addondeploymentconfig hypershift-addon-deploy-config \
-n multicluster-engine
```

3. Add the **nodePlacement** field to the specification, as shown in the following example:

```
apiVersion: addon.open-cluster-management.io/v1alpha1
kind: AddOnDeploymentConfig
metadata:
  name: hypershift-addon-deploy-config
  namespace: multicluster-engine
spec:
  nodePlacement:
    nodeSelector:
      node-role.kubernetes.io/infra: ""
  tolerations:
    - effect: NoSchedule
      key: node-role.kubernetes.io/infra
      operator: Exists
```

4. Save the changes. The **hypershift-addon** managed cluster add-on is deployed on an infrastructure node for new and existing managed clusters.

CHAPTER 7. CONFIGURING CERTIFICATES FOR HOSTED CONTROL PLANES

To establish secure and encrypted communication between your clients and the hosted control plane, you must configure a server certificate for your hosted cluster.

With hosted control planes, the steps to configure certificates differ from those of standalone OpenShift Container Platform.

7.1. CONFIGURING A CUSTOM API SERVER CERTIFICATE IN A HOSTED CLUSTER

To configure a custom certificate for the API server, specify the certificate details in the **spec.configuration.apiServer** section of your **HostedCluster** configuration.

You can configure a custom certificate during either Day 1 or Day 2 operations. However, because the service publishing strategy is immutable after you set it during hosted cluster creation, you must know what the hostname is for the Kubernetes API server that you plan to configure.

Prerequisites

- You created a Kubernetes secret that contains your custom certificate in the management cluster. The secret contains the following keys:
 - **tls.crt**: The certificate
 - **tls.key**: The private key
- If your **HostedCluster** configuration includes custom serving certificates via the **spec.configuration.apiServer.servingCerts.namedCertificates** specification, ensure that the Subject Alternative Names (SANs) of the certificate do not conflict with the external API server address. For example, depending on the hostname pattern used in your environment, the address might be in the following format: **api.<cluster_name>.<domain>**.
The **HostedCluster** resource automatically includes the external API address in the default Kubernetes API server certificate SANs. If the same hostname is in both the custom certificate and the automatically generated Kubernetes API server certificate, the configuration is rejected to prevent TLS serving ambiguity.

This validation applies to all service publishing strategies, including **LoadBalancer** and **NodePort**. The only exception is when you use Amazon Web Services (AWS) as the provider with **Private** or **PublicAndPrivate** endpoint access configurations, where the platform manages the SAN conflict.

- The certificate must be valid for the external API endpoint.
- The validity period of the certificate aligns with your cluster's expected life cycle.

Procedure

1. Create a secret with your custom certificate by entering the following command:

```
$ oc create secret tls sample-hosted-kas-custom-cert \
  --cert=path/to/cert.crt \
  --key=path/to/key.key \
```

```
-n <hosted_cluster_namespace>
```

2. Update your **HostedCluster** configuration with the custom certificate details, as shown in the following example:

```
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
              - api-custom-cert-sample-hosted.sample-hosted.example.com
            servingCertificate:
              name: sample-hosted-kas-custom-cert
```

- **spec.configuration.apiServer.servingCerts.namedCertificates.names** specifies the list of DNS names that the certificate is valid for.
 - **spec.configuration.apiServer.servingCerts.namedCertificates.servingCertificate** specifies the name of the secret that contains the custom certificate.
3. Apply the changes to your **HostedCluster** configuration by entering the following command:

```
$ oc apply -f <hosted_cluster_config>.yaml
```

Verification

- Check the API server pods to ensure that the new certificate is mounted.
- Test the connection to the API server by using the custom domain name.
- Verify the certificate details in your browser or by using tools such as **openssl**.

7.2. CONFIGURING THE KUBERNETES API SERVER FOR A HOSTED CLUSTER

You can customize the Kubernetes API server for your hosted cluster.

Prerequisites

- You have a running hosted cluster.
- You have access to modify the **HostedCluster** resource.
- You have a custom DNS domain to use for the Kubernetes API server.
 - The custom DNS domain must be properly configured and resolvable.
 - The DNS domain must have valid TLS certificates configured.
 - Network access to the domain must be properly configured in your environment.
 - The custom DNS domain must be unique across your hosted clusters.

- You have a configured custom certificate. For more information, see "Configuring a custom API server certificate in a hosted cluster".

Procedure

1. In your provider platform, configure the DNS record so that the **kubeAPIServerDNSName** URL points to the IP address that the Kubernetes API server is being exposed to. The DNS record must be properly configured and resolvable from your cluster.

Example command to configure the DNS record

```
$ dig + short kubeAPIServerDNSName
```

2. In your **HostedCluster** specification, modify the **kubeAPIServerDNSName** field, as shown in the following example:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  configuration:
    apiServer:
      servingCerts:
        namedCertificates:
          - names:
            - api-custom-cert-sample-hosted.sample-hosted.example.com
            servingCertificate:
              name: sample-hosted-kas-custom-cert
        kubeAPIServerDNSName: api-custom-cert-sample-hosted.sample-hosted.example.com
# ...
```

- **spec.configuration.apiServer.servingCerts.namedCertificates.names** specifies the list of DNS names that the certificate is valid for. The names listed in this field cannot be the same as the names specified in the **spec.servicePublishingStrategy.*hostname** field.
 - **spec.configuration.apiServer.servingCerts.namedCertificates.servingCertificate** specifies the name of the secret that contains the custom certificate.
 - **spec.kubeAPIServerDNSName** accepts a URI that will be used as the API server endpoint.
3. Apply the configuration by entering the following command:

```
$ oc -f <hosted_cluster_spec>.yaml
```

After the configuration is applied, the HyperShift Operator generates a new **kubeconfig** secret that points to your custom DNS domain.

4. Retrieve the **kubeconfig** secret by using the CLI or the console.
 - a. To retrieve the secret by using the CLI, enter the following command:

```
$ kubectl get secret <hosted_cluster_name>-custom-admin-kubeconfig \
-n <cluster_namespace> \
-o jsonpath='{.data.kubeconfig}' | base64 -d
```

- b. To retrieve the secret by using the console, go to your hosted cluster and click **Download Kubeconfig**.



NOTE

You cannot consume the new **kubeconfig** secret by using the **show login command** option in the console.

7.3. OAUTH SERVER CERTIFICATES FOR HOSTED CONTROL PLANES

In hosted control planes, the OAuth server shares its serving certificate configuration with the Kubernetes API server. To configure a custom serving certificate for the OAuth server, you modify the **spec.configuration.apiServer** section in the **HostedCluster** resource.



IMPORTANT

This configuration method deviates from the standard OpenShift Container Platform behavior. In OpenShift Container Platform, OAuth certificates are configured separately through the **componentRoute** properties of the Ingress Operator. In hosted control planes, the **namedCertificates** configuration in the API server settings applies to both the Kubernetes API server and the OAuth server.

In hosted control planes, the Control Plane Operator reads serving certificates through the shared **GetNamedCertificates()** function. Certificates are not configured in an OAuth-specific section of the **HostedCluster** resource. In addition, OAuth server certificates are not provided through an OAuth custom resource definition (CRD) configuration. Instead, hosted control planes automatically injects the selected certificates into the OAuth server deployment.

Table 7.1. OAuth certificate differences between OpenShift Container Platform and hosted control planes

Area	OpenShift Container Platform	hosted control planes
Certificate source	Ingress Operator generates and maps certificates through component routes	OAuth uses apiServer.servingCerts.namedCertificates settings
Certificate selection	Based on ingress-managed routes	Based on host name match in namedCertificates property
User responsibility	No need to manually provide OAuth certificates	User must supply certificates if custom behavior is needed
Code path	Ingress Operator manages the OAuth route	Control Plane Operator manages the OAuth server container runtime arguments

7.4. CONFIGURING OAUTH SERVER CERTIFICATES FOR A HOSTED CLUSTER

If you want to use certificates from a trusted certificate authority (CA) to access a hosted cluster, you can configure OAuth server certificates.

Prerequisites

- You have a running hosted cluster.
- You have **cluster-admin** access to the management cluster.
- You have access to modify the **HostedCluster** resource.
- You have a TLS secret that contains your signed certificate and private key in the hosted cluster namespace with the following keys:
 - **tls.crt**
 - **tls.key**

Procedure

1. Identify your hosted cluster namespace:
 - a. Export the namespace where your hosted cluster is running by entering the following command:

```
$ export HC_NAMESPACE=<hosted_cluster_namespace>
```

- b. Export the hosted cluster name by entering the following command:

```
$ export CLUSTER_NAME=<hosted_cluster_name>
```

2. Generate a quick test certificate by entering the following command:

```
$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout tls.key \
-out tls.crt \
-subj "/CN=openshift-oauth" \
-addext "subjectAltName=DNS:oauth-${HC_NAMESPACE}-${CLUSTER_NAME}.api-
custom-cert-sample-hosted.sample-hosted.example.com"
```

The **api-custom-cert-sample-hosted.sample-hosted.example.com** value is used in the command and throughout the rest of this procedure as an example.



NOTE

This example uses a placeholder hostname. After you discover your OAuth route later in this procedure, you must regenerate this certificate with the correct hostname before you edit the **HostedCluster** resource.

3. Confirm that the file exists by entering the following command:

■

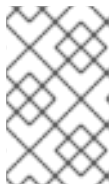
```
$ ls tls.crt tls.key
```

4. If you have not already created the TLS secret in the hosted cluster namespace, create the secret by entering the following command:

```
$ oc create secret tls my-oauth-cert-secret \
  --cert=path/to/tls.crt \
  --key=path/to/tls.key \
  -n ${HC_NAMESPACE}
```

Example output

```
secret/my-oauth-cert-secret created
```



NOTE

Although the OAuth server runs in the hosted control plane namespace, the serving certificate must exist in the hosted cluster namespace. Secrets that are created in the hosted control plane namespace are not picked up.

5. Discover the correct OAuth route:

- a. Use the management cluster **kubconfig** file to enter the following command:

```
$ oc get routes -n ${HC_NAMESPACE}-${CLUSTER_NAME}
```

- b. If the route name is **oauth**, confirm it by entering the following command:

```
$ oc get route oauth -n ${HC_NAMESPACE}-${CLUSTER_NAME} -o yaml
```

- c. Prepare to extract the OAuth route host by entering the following command:

```
OAUTH_HOST=$(oc get route oauth \
  -n ${HC_NAMESPACE}-${CLUSTER_NAME} \
  -o jsonpath='{.spec.host}')
```

- d. Extract the OAuth route host by entering the following command:

```
$ echo "${OAUTH_HOST}"
```

Example output

```
oauth-${HC_NAMESPACE}-${CLUSTER_NAME}.api-custom-cert-sample-
hosted.sample-hosted.example.com
```

6. Edit the **HostedCluster** resource:

- a. Open the **HostedCluster** resource for editing by entering the following command:

```
$ oc edit hostedcluster $CLUSTER_NAME -n ${HC_NAMESPACE}
```

- b. In the resource, configure the named certificates by adding the **servingCerts.namedCertificates** stanza to the **spec.configuration.apiServer** section:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  configuration:
    apiServer:
      audit:
        profile: Default
      servingCerts:
        namedCertificates:
          - names:
            - api-custom-cert-sample-hosted.sample-hosted.example.com
            servingCertificate:
              name: my-oauth-cert-secret
# ...
```

where:

spec.configuration.apiServer.servingCerts.namedCertificates.names

Specifies the actual host name of your OAuth route.

spec.configuration.apiServer.servingCerts.servingCertificate.name

Specifies the name of your TLS secret. This secret must exist in the hosted cluster namespace.

- c. Save and apply the changes. The Control Plane Operator reconciles the changes, the configuration propagates to the control plane, and the OAuth server begins serving the new certificate.



IMPORTANT

No separate OAuth certificate configuration field exists for a hosted cluster.

Verification

- Verify the certificate being served by the route by entering the following command:

```
$ echo | openssl s_client \
  -connect "${OAUTH_HOST}:443" \
  -servername "${OAUTH_HOST}" \
  2>/dev/null \
  | openssl x509 -noout -subject -issuer -ext subjectAltName
```

Example output

```
subject=CN=openshift-oauth
issuer=CN=openshift-oauth
X509v3 Subject Alternative Name:
  DNS:oauth-${HC_NAMESPACE}-${CLUSTER_NAME}.api-custom-cert-sample-
  hosted.sample-hosted.example.com
```



The output shows that the OAuth route is serving the custom certificate and the certificate comes from the **my-oauth-cert-secret** secret.

7.5. TROUBLESHOOTING ACCESSING A HOSTED CLUSTER BY USING A CUSTOM DNS

If you encounter an issue when you access a hosted cluster by using a custom DNS, you can determine the root cause so that you can resolve the issue.

Procedure

1. Verify that the DNS record is properly configured and resolved.
2. Check that the TLS certificates for the custom domain are valid, verifying that the SAN is correct for your domain, by entering the following command:

```
$ oc get secret \  
-n clusters <serving_certificate_name> \  
-o jsonpath='{.data.tls.crt}' | base64 \  
-d | openssl x509 -text -noout -
```

3. Ensure that network connectivity to the custom domain is working.
4. In the **HostedCluster** resource, verify that the status shows the correct custom **kubeconfig** information, as shown in the following example:

Example HostedCluster status

```
status:  
  customKubeconfig:  
    name: sample-hosted-custom-admin-kubeconfig
```

5. Check the **kube-apiserver** logs in the **HostedControlPlane** namespace by entering the following command:

```
$ oc logs -n <hosted_control_plane_namespace> \  
-l app=kube-apiserver -f -c kube-apiserver
```


CHAPTER 8. UPDATING HOSTED CONTROL PLANES

Updates for hosted control planes involve updating the hosted cluster and the node pools.

For a cluster to remain fully operational during an update process, you must meet the requirements of the hosted cluster and node pool version skew policy while completing the control plane and node updates. For more information, see "Hosted cluster and node pool version skew policy".

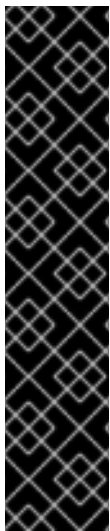
8.1. REQUIREMENTS TO UPGRADE HOSTED CONTROL PLANES

The multicluster engine for Kubernetes Operator can manage one or more OpenShift Container Platform clusters. After you create a hosted cluster on OpenShift Container Platform, you must import your hosted cluster in the multicluster engine Operator as a managed cluster.

You can then use the OpenShift Container Platform cluster as a management cluster.

Consider the following requirements before you start updating hosted control planes:

- You must use the bare metal platform for an OpenShift Container Platform cluster when using OpenShift Virtualization as a provider.
- You must use bare metal or OpenShift Virtualization as the cloud platform for the hosted cluster. You can find the platform type of your hosted cluster in the **spec.Platform.type** specification of the **HostedCluster** custom resource (CR).



IMPORTANT

You must update hosted control planes in the following order:

1. Upgrade an OpenShift Container Platform cluster to the latest version. For more information, see "Updating a cluster using the web console" or "Updating a cluster using the CLI".
2. Upgrade the multicluster engine Operator to the latest version. For more information, see "Updating installed Operators".
3. Upgrade the hosted cluster and node pools from the previous OpenShift Container Platform version to the latest version. For more information, see "Updating a control plane in a hosted cluster" and "Updating node pools in a hosted cluster".

Additional resources

- [Updating a cluster using the web console](#)
- [Updating a cluster using the CLI](#)
- [Updating installed Operators](#)
- [Updating a control plane in a hosted cluster](#)
- [Updating node pools in a hosted cluster](#)

8.2. HOSTED CLUSTER AND NODE POOL VERSION SKEW POLICY

In hosted control planes, ensure that you use a node pool version that is supported with your hosted cluster version.

- Node pool versions up to 3 minor versions behind the hosted cluster version are supported. For example, a 4.21 hosted cluster supports node pool versions 4.21, 4.20, 4.19, and 4.18.
- Node pool versions cannot be higher than the hosted cluster version. This includes patch versions within the same minor version. For example, a 4.20.3 node pool version is *not* supported on a 4.20.2 hosted cluster.

Table 8.1. Example supported node pool versions for your hosted cluster

Hosted cluster version	Supported node pool versions	Example unsupported node pool versions
4.21.0	4.21.0, 4.20.z, 4.19.z, 4.18.z	4.21.1+, 4.17.z, 4.16.z
4.20.2	4.20.0, 4.20.1, 4.20.2, 4.19.z, 4.18.z, 4.17.z	4.21.z, 4.20.3+, 4.16.z

8.2.1. Version compatibility reporting

The node pool controller sets the **SupportedVersionSkew** condition to report one of the following version compatibility statuses:

True

Indicates that the node pool version is compatible with the hosted cluster version.

False

Indicates that the node pool version is incompatible with the hosted cluster version. A detailed error message is provided. You must upgrade or roll back your node pools to a compatible version. If incompatibility is detected, the existing node pools continue to operate, but without stability or support. When creating new node pools, the CLI returns an error if you try to create a node pool with an incompatible version.

8.3. SETTING UPDATE CHANNELS IN A HOSTED CLUSTER

To see available updates for a hosted cluster, you must set the **spec.channel** field on the **HostedCluster** custom resource (CR).

You can see available updates in the **HostedCluster.Status** field of the **HostedCluster** CR.

The available updates are not fetched from the Cluster Version Operator (CVO) of a hosted cluster. The list of the available updates can be different from the available updates from the following fields of the **HostedCluster** custom resource (CR):

- **status.version.availableUpdates**
- **status.version.conditionalUpdates**

The initial **HostedCluster** CR does not have any information in the **status.version.availableUpdates** and **status.version.conditionalUpdates** fields. After you set the **spec.channel** field to the stable

OpenShift Container Platform release version, the HyperShift Operator reconciles the **HostedCluster** CR and updates the **status.version** field with the available and conditional updates.

Procedure

1. In the **HostedCluster** CR, add the channel configuration as shown in the following example:

```
spec:
  autoscaling: {}
  channel: stable-<4.y>
  clusterID: d6d42268-7dff-4d37-92cf-691bd2d42f41
  configuration: {}
  controllerAvailabilityPolicy: SingleReplica
  dns:
    baseDomain: dev11.red-chesterfield.com
    privateZoneID: Z0180092I0DQRKL55LN0
    publicZoneID: Z00206462VG6ZP0H2QLWK
```

Replace **<4.y>** with the OpenShift Container Platform release version you specified in **spec.release**. For example, if you set the **spec.release** to **ocp-release:4.16.4-multi**, you must set **spec.channel** to **stable-4.16**.

2. After you configure the channel in the **HostedCluster** CR, to view the output of the **status.version.availableUpdates** and **status.version.conditionalUpdates** fields, run the following command:

```
$ oc get -n <hosted_cluster_namespace> hostedcluster <hosted_cluster_name> -o yaml
```

Example output

```
version:
  availableUpdates:
    - channels:
        - candidate-4.16
        - candidate-4.17
        - eus-4.16
        - fast-4.16
        - stable-4.16
      image: quay.io/openshift-release-dev/ocp-
release@sha256:b7517d13514c6308ae16c5fd8108133754eb922cd37403ed27c846c129e67a
9a
      url: https://access.redhat.com/errata/RHBA-2024:6401
      version: 4.16.11
    - channels:
        - candidate-4.16
        - candidate-4.17
        - eus-4.16
        - fast-4.16
        - stable-4.16
      image: quay.io/openshift-release-dev/ocp-
release@sha256:d08e7c8374142c239a07d7b27d1170eae2b0d9f00ccf074c3f13228a1761c162

      url: https://access.redhat.com/errata/RHSA-2024:6004
      version: 4.16.10
```

```

- channels:
  - candidate-4.16
  - candidate-4.17
  - eus-4.16
  - fast-4.16
  - stable-4.16
  image: quay.io/openshift-release-dev/ocp-
release@sha256:6a80ac72a60635a313ae511f0959cc267a21a89c7654f1c15ee16657aafa41a
0
  url: https://access.redhat.com/errata/RHBA-2024:5757
  version: 4.16.9
- channels:
  - candidate-4.16
  - candidate-4.17
  - eus-4.16
  - fast-4.16
  - stable-4.16
  image: quay.io/openshift-release-dev/ocp-
release@sha256:ea624ae7d91d3f15094e9e15037244679678bdc89e5a29834b2ddb7e1d9b57
e6
  url: https://access.redhat.com/errata/RHSA-2024:5422
  version: 4.16.8
- channels:
  - candidate-4.16
  - candidate-4.17
  - eus-4.16
  - fast-4.16
  - stable-4.16
  image: quay.io/openshift-release-dev/ocp-
release@sha256:e4102eb226130117a0775a83769fe8edb029f0a17b6cbca98a682e3f1225d6b
7
  url: https://access.redhat.com/errata/RHSA-2024:4965
  version: 4.16.6
- channels:
  - candidate-4.16
  - candidate-4.17
  - eus-4.16
  - fast-4.16
  - stable-4.16
  image: quay.io/openshift-release-dev/ocp-
release@sha256:f828eda3eaac179e9463ec7b1ed6baeba2cd5bd3f1dd56655796c86260db819
b
  url: https://access.redhat.com/errata/RHBA-2024:4855
  version: 4.16.5
conditionalUpdates:
- conditions:
  - lastTransitionTime: "2024-09-23T22:33:38Z"
    message: |-
      Could not evaluate exposure to update risk SRIOVFailedToConfigureVF (creating
      PromQL round-tripper: unable to load specified CA cert /etc/tls/service-ca/service-ca.crt: open
      /etc/tls/service-ca/service-ca.crt: no such file or directory)
      SRIOVFailedToConfigureVF description: OCP Versions 4.14.34, 4.15.25, 4.16.7 and
      ALL subsequent versions include kernel datastructure changes which are not compatible
      with older versions of the SR-IOV operator. Please update SR-IOV operator to versions
      dated 20240826 or newer before updating OCP.
      SRIOVFailedToConfigureVF URL: https://issues.redhat.com/browse/NHE-1171

```

```

reason: EvaluationFailed
status: Unknown
type: Recommended
release:
  channels:
    - candidate-4.16
    - candidate-4.17
    - eus-4.16
    - fast-4.16
    - stable-4.16
  image: quay.io/openshift-release-dev/ocp-
release@sha256:fb321a3f50596b43704dbbed2e51fdefd7a7fd488ee99655d03784d0cd02283f

  url: https://access.redhat.com/errata/RHSA-2024:5107
  version: 4.16.7
risks:
  - matchingRules:
    - promql:
      promql: |
        group(csv_succeeded[_id="d6d42268-7dff-4d37-92cf-691bd2d42f41", name=~"sriov-
network-operator[.].*"])
      or
      0 * group(csv_count[_id="d6d42268-7dff-4d37-92cf-691bd2d42f41"])
    type: PromQL
  message: OCP Versions 4.14.34, 4.15.25, 4.16.7 and ALL subsequent versions
  include kernel datastructure changes which are not compatible with older
  versions of the SR-IOV operator. Please update SR-IOV operator to versions
  dated 20240826 or newer before updating OCP.
  name: SRIOVFailedToConfigureVF
  url: https://issues.redhat.com/browse/NHE-1171

```

8.4. UPDATES OF THE OPENSIFT CONTAINER PLATFORM VERSION IN A HOSTED CLUSTER

Hosted control planes enables the decoupling of updates between the control plane and the data plane.

As a cluster service provider or cluster administrator, you can manage the control plane and the data separately.

You can update a control plane by modifying the **HostedCluster** custom resource (CR) and a node by modifying its **NodePool** CR. Both the **HostedCluster** and **NodePool** CRs specify an OpenShift Container Platform release image in a **.release** field.

To keep your hosted cluster fully operational during an update process, the control plane and the node versions must be compatible. For more information, see "Hosted cluster and node pool version skew policy".

8.4.1. The multicluster engine Operator hub management cluster

The multicluster engine for Kubernetes Operator requires a specific OpenShift Container Platform version for the management cluster to remain in a supported state. You can install the multicluster engine Operator from the software catalog in the OpenShift Container Platform web console.

For more information, see the support matrixes for the multicluster engine Operator versions.

The multicluster engine Operator supports the following OpenShift Container Platform versions:

- The latest unreleased version
- The latest released version
- Two versions before the latest released version

You can also get the multicluster engine Operator version as a part of Red Hat Advanced Cluster Management (RHACM).

8.4.2. Supported OpenShift Container Platform versions in a hosted cluster

When deploying a hosted cluster, the OpenShift Container Platform version of the management cluster does not affect the OpenShift Container Platform version of a hosted cluster.

The HyperShift Operator creates the **supported-versions** ConfigMap in the **hypershift** namespace. The **supported-versions** ConfigMap describes the range of supported OpenShift Container Platform versions that you can deploy.

See the following example of the **supported-versions** ConfigMap:

```
apiVersion: v1
data:
  server-version: 2f6cfe21a0861dea3130f3bed0d3ae5553b8c28b
  supported-versions: '{"versions":["4.17","4.16","4.15","4.14"]}'
kind: ConfigMap
metadata:
  creationTimestamp: "2024-06-20T07:12:31Z"
  labels:
    hypershift.openshift.io/supported-versions: "true"
  name: supported-versions
  namespace: hypershift
  resourceVersion: "927029"
  uid: f6336f91-33d3-472d-b747-94abae725f70
```

IMPORTANT

To create a hosted cluster, you must use the OpenShift Container Platform version from the support version range. However, the multicluster engine Operator can manage only between **n+1** and **n-2** OpenShift Container Platform versions, where **n** defines the current minor version. You can check the multicluster engine Operator support matrix to ensure the hosted clusters managed by the multicluster engine Operator are within the supported OpenShift Container Platform range.

To deploy a higher version of a hosted cluster on OpenShift Container Platform, you must update the multicluster engine Operator to a new minor version release to deploy a new version of the HyperShift Operator. Upgrading the multicluster engine Operator to a new patch, or z-stream, release does not update the HyperShift Operator to the next version.

See the following example output of the **hcp version** command that shows the supported OpenShift Container Platform versions for OpenShift Container Platform 4.16 in the management cluster:

```
Client Version: openshift/hypershift: fe67b47fb60e483fe60e4755a02b3be393256343. Latest
```

supported OCP: 4.17.0
 Server Version: 05864f61f24a8517731664f8091cedcfc5f9b60d
 Server Supports OCP Versions: 4.17, 4.16, 4.15, 4.14

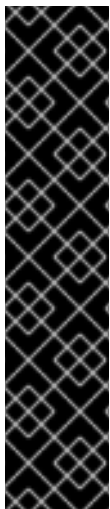
Additional resources

- [Hosted cluster and node pool version skew policy](#)
- [multicluster engine Operator 2.17](#)
- [multicluster engine Operator 2.11](#)
- [multicluster engine Operator 2.10](#)
- [multicluster engine Operator 2.9](#)
- [multicluster engine Operator 2.8](#)
- [multicluster engine Operator 2.7](#)
- [multicluster engine Operator 2.6](#)
- [multicluster engine Operator 2.5](#)
- [multicluster engine Operator 2.4](#)

8.5. UPDATES FOR THE HOSTED CLUSTER

The **spec.release.image** value dictates the version of the control plane. The **HostedCluster** object transmits the intended **spec.release.image** value to the **HostedControlPlane.spec.releaseImage** value and runs the appropriate Control Plane Operator version.

The hosted control plane manages the rollout of the new version of the control plane components along with any OpenShift Container Platform components through the new version of the Cluster Version Operator (CVO).



IMPORTANT

In hosted control planes, the **NodeHealthCheck** resource cannot detect the status of the CVO. A cluster administrator must manually pause the remediation triggered by **NodeHealthCheck**, before performing critical operations, such as updating the cluster, to prevent new remediation actions from interfering with cluster updates.

To pause the remediation, enter the array of strings, for example, **pause-test-cluster**, as a value of the **pauseRequests** field in the **NodeHealthCheck** resource. For more information, see "About the Node Health Check Operator".

After the cluster update is complete, you can edit or delete the remediation. Go to the **Compute → NodeHealthCheck** page, click your node health check, and then click **Actions**, which shows a drop-down list.

Additional resources

- [About the Node Health Check Operator](#)

8.6. UPDATES FOR NODE POOLS

You can update node pools by exposing the **spec.release** and **spec.config** values.

You can start a rolling node pool update in the following ways:

- Changing the **spec.release** or **spec.config** values.
- Changing any platform-specific field, such as the AWS instance type. The result is a set of new instances with the new type.
- Changing the cluster configuration, if the change propagates to the node.

Node pools support replace updates and in-place updates. The **nodepool.spec.release** value dictates the version of any particular node pool. A **NodePool** object completes a replace or an in-place rolling update according to the **.spec.management.upgradeType** value.

After you create a node pool, you cannot change the update type. If you want to change the update type, you must create a node pool and delete the other one.

8.6.1. Replace updates for node pools

A *replace* update creates instances in the new version while it removes old instances from the previous version. This update type is effective in cloud environments where this level of immutability is cost effective.

Replace updates do not preserve any manual changes because the node is entirely re-provisioned.

8.6.2. In place updates for node pools

An *in-place* update directly updates the operating systems of the instances. This type is suitable for environments where the infrastructure constraints are greater, such as bare metal.

In-place updates can preserve manual changes, but reports errors if you manually change any file system or operating system configuration that the cluster directly manages, such as kubelet certificates.

8.7. UPDATING NODE POOLS IN A HOSTED CLUSTER

You can update your version of OpenShift Container Platform by updating the node pools in your hosted cluster. The node pool version must not surpass the hosted control plane version.

The **.spec.release** field in the **NodePool** custom resource (CR) shows the version of a node pool.

Procedure

- Change the **spec.release.image** value in the node pool by entering the following command:

```
$ oc patch nodepool <node_pool_name> -n <hosted_cluster_namespace> \
  --type=merge \
  -p '{"spec":{"nodeDrainTimeout":"60s","release":{"image":"<openshift_release_image>"}}}'
```

- Replace **<node_pool_name>** with your node pool name and **<hosted_cluster_namespace>** with your hosted cluster namespace.

- The `<openshift_release_image>` variable specifies the new OpenShift Container Platform release image that you want to upgrade to, for example, `quay.io/openshift-release-dev/ocp-release:<4.y.z>-x86_64`. Replace `<4.y.z>` with the supported OpenShift Container Platform version.

Verification

- To verify that the new version was rolled out, check the `.status.conditions` value in the node pool by running the following command:

```
$ oc get -n <hosted_cluster_namespace> nodepool <node_pool_name> -o yaml
```

Example output

```
status:
  conditions:
  - lastTransitionTime: "2024-05-20T15:00:40Z"
    message: 'Using release image: quay.io/openshift-release-dev/ocp-release:4.20.0-x86_64'
    reason: AsExpected
    status: "True"
    type: ValidReleaseImage
```

8.8. UPDATING A CONTROL PLANE IN A HOSTED CLUSTER

In hosted control planes, you can upgrade your version of OpenShift Container Platform by updating the hosted cluster.

The `.spec.release` in the **HostedCluster** custom resource (CR) shows the version of the control plane. The **HostedCluster** updates the `.spec.release` field to the **HostedControlPlane.spec.release** and runs the appropriate Control Plane Operator version.

The **HostedControlPlane** resource orchestrates the rollout of the new version of the control plane components along with the OpenShift Container Platform component in the data plane through the new version of the Cluster Version Operator (CVO). The **HostedControlPlane** includes the following artifacts:

- CVO
- Cluster Network Operator (CNO)
- Cluster Ingress Operator
- Manifests for the Kube API server, scheduler, and manager
- Machine approver
- Autoscaler
- Infrastructure resources to enable ingress for control plane endpoints such as the Kube API server, ignition, and connectivity

You can set the `.spec.release` field in the **HostedCluster** CR to update the control plane by using the information from the `status.version.availableUpdates` and `status.version.conditionalUpdates` fields.

Procedure

1. Add the **hypershift.openshift.io/force-upgrade-to=<openshift_release_image>** annotation to the hosted cluster by entering the following command:

```
$ oc annotate hostedcluster \
  -n <hosted_cluster_namespace> <hosted_cluster_name> \
  "hypershift.openshift.io/force-upgrade-to=<openshift_release_image>" \
  --overwrite
```

- Replace **<hosted_cluster_name>** with your hosted cluster name and **<hosted_cluster_namespace>** with your hosted cluster namespace.
 - The **<openshift_release_image>** variable specifies the new OpenShift Container Platform release image that you want to upgrade to, for example, **quay.io/openshift-release-dev/ocp-release:<4.y.z>-x86_64**. Replace **<4.y.z>** with the supported OpenShift Container Platform version.
2. Change the **spec.release.image** value in the hosted cluster by entering the following command:

```
$ oc patch hostedcluster <hosted_cluster_name> -n <hosted_cluster_namespace> \
  --type=merge \
  -p '{"spec":{"release":{"image":"<openshift_release_image>"}}}'
```

Verification

- To verify that the new version was rolled out, check the **.status.conditions** and **.status.version** values in the hosted cluster by running the following command:

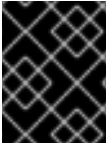
```
$ oc get -n <hosted_cluster_namespace> hostedcluster <hosted_cluster_name> \
  -o yaml
```

Example output

```
status:
  conditions:
  - lastTransitionTime: "2024-05-20T15:01:01Z"
    message: Payload loaded version="4.20.0" image="quay.io/openshift-release-dev/ocp-release:4.20.0-x86_64"
    status: "True"
    type: ClusterVersionReleaseAccepted
  #...
  version:
    availableUpdates: null
    desired:
      image: quay.io/openshift-release-dev/ocp-release:4.20.0-x86_64
      version: 4.20.0
```

8.9. UPDATING A HOSTED CLUSTER BY USING THE MULTICLUSTER ENGINE OPERATOR CONSOLE

You can update your hosted cluster by using the multicluster engine Operator console.



IMPORTANT

Before updating a hosted cluster, you must refer to the available and conditional updates of a hosted cluster. Choosing a wrong release version might break the hosted cluster.

Procedure

1. Select **All clusters**.
2. Navigate to **Infrastructure → Clusters** to view managed hosted clusters.
3. Click the **Upgrade available** link to update the control plane and node pools.

CHAPTER 9. HIGH AVAILABILITY FOR HOSTED CONTROL PLANES

9.1. ABOUT HIGH AVAILABILITY FOR HOSTED CONTROL PLANES

You can maintain high availability (HA) for hosted control planes by recovering etcd members for a hosted cluster, backing up and restoring etcd for a hosted cluster, and completing a disaster recovery process for a hosted cluster.

9.1.1. Impact of the failed management cluster component

If the management cluster component fails, your workload remains unaffected. In the OpenShift Container Platform management cluster, the control plane is decoupled from the data plane to provide resiliency.

The following table covers the impact of a failed management cluster component on the control plane and the data plane. However, the table does not cover all scenarios for the management cluster component failures.

Table 9.1. Impact of the failed component on hosted control planes

Name of the failed component	Hosted control plane API status	Hosted cluster data plane status
Worker node	Available	Available
Availability zone	Available	Available
Management cluster control plane	Available	Available
Management cluster control plane and worker nodes	Not available	Available

9.2. RECOVERING AN UNHEALTHY ETCD CLUSTER FOR HOSTED CONTROL PLANES

In a highly available control plane, three etcd pods run as a part of a stateful set in an etcd cluster. To recover an etcd cluster, identify unhealthy etcd pods by checking the etcd cluster health.

9.2.1. Checking the status of an etcd cluster

You can check the status of the etcd cluster health by logging into any etcd pod.

Procedure

1. Log in to an etcd pod by entering the following command:

```
$ oc rsh -n clusters-<hosted_cluster_name> -c etcd <etcd_pod_name>
```

2. Print the health status of an etcd cluster by entering the following command:

```
sh-4.4# etcdctl endpoint status -w table
```

Example output

```
+-----+-----+-----+-----+-----+-----+
|      ENDPOINT      | ID   | VERSION | DB SIZE | IS LEADER | IS LEARNER |
| RAFT TERM | RAFT INDEX | RAFT APPLIED INDEX | ERRORS |
+-----+-----+-----+-----+-----+-----+
| https://192.168.1xxx.20:2379 | 8fxxxxxxxxxx | 3.5.12 | 123 MB | false | false |
| 10 | 180156 | 180156 | |
| https://192.168.1xxx.21:2379 | a5xxxxxxxxxx | 3.5.12 | 122 MB | false | false |
| 10 | 180156 | 180156 | |
| https://192.168.1xxx.22:2379 | 7cxxxxxxxxxx | 3.5.12 | 124 MB | true | false |
| 10 | 180156 | 180156 | |
+-----+-----+-----+-----+-----+-----+
+-----+-----+
```

9.2.2. Recovering a failing etcd pod

Each etcd pod of a 3-node cluster has its own persistent volume claim (PVC) to store its data. An etcd pod might fail because of corrupted or missing data. You can recover a failing etcd pod and its PVC.

Procedure

1. To confirm that the etcd pod is failing, enter the following command:

```
$ oc get pods -l app=etcd -n clusters-<hosted_cluster_name>
```

Replace **<hosted_cluster_name>** with the name of the hosted cluster of the etcd instance.

Example output

```
NAME    READY  STATUS             RESTARTS  AGE
etcd-0  2/2    Running            0         64m
etcd-1  2/2    Running            0         45m
etcd-2  1/2    CrashLoopBackOff   1 (5s ago) 64m
```

The failing etcd pod might have the **CrashLoopBackOff** or **Error** status.

2. Delete the failing pod and its PVC by entering the following command:

```
$ oc delete pods <etcd_pod_name> -n clusters-<hosted_cluster_name>
```

Replace **<etcd_pod_name>** with the name of the failing pod.

Verification

- Verify that a new etcd pod is up and running by entering the following command:

```
$ oc get pods -l app=etcd -n clusters-<hosted_cluster_name>
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
etcd-0	2/2	Running	0	67m
etcd-1	2/2	Running	0	48m
etcd-2	2/2	Running	0	2m2s

9.3. BACKING UP AND RESTORING ETCD IN AN ON-PREMISE ENVIRONMENT

You can back up and restore etcd for hosted control planes in an on-premise environment to fix failures.

9.3.1. Backing up and restoring etcd on a hosted cluster in an on-premise environment

By backing up and restoring etcd on a hosted cluster, you can fix failures, such as corrupted or missing data in an etcd member of a three-node cluster. If multiple members of the etcd cluster encounter data loss or have a **CrashLoopBackOff** status, this approach helps prevent an etcd quorum loss.

Prerequisites

- The **oc** and **jq** binaries have been installed.
- Management cluster prerequisites:
 - A valid **StorageClass** resource is configured in the management cluster.
 - You have **cluster-admin** access to the management cluster.
 - You have access to online storage that is compatible with OpenShift ADP cloud storage providers, such as Amazon Web Services (AWS) S3, Microsoft Azure, Google Cloud, or MinIO. If you use S3 for backup storage, ensure that IAM roles and policies are configured. For more information, see "Configuring Amazon Web Services".
 - Hosted control plane pods are accessible and functioning properly.
 - You have access to the **openshift-adp** subscription through a **CatalogSource** object.
- Hosted cluster service publishing strategy prerequisites:
 - The **APIServer** service must have a fixed hostname. Otherwise, the restore process fails and nodes cannot rejoin the cluster. For hosted control planes on AWS, the **APIServer** service can also use a **Route** service publishing strategy with a fixed hostname.
 - For production environments, it is strongly recommended to configure all services with fixed hostnames. By having fixed hostnames, you can ensure full service continuity and DNS consistency during the restore process on a different management cluster.
 - When you restore a hosted cluster to a different management cluster, all services in the hosted cluster must be configured with a fixed hostname in its **servicePublishingStrategy** property. This requirement applies to all platforms: AWS, Agent, OpenShift Virtualization, and Red Hat OpenStack Platform (RHOSP).



IMPORTANT

Restoring a hosted cluster to a different management cluster is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

- For hosted control planes on AWS, the OIDC provider configuration must be accessible so that any necessary fixes can be completed after the restore process. See the following procedure for more information about applying any necessary fixes.
- For hosted control planes on bare metal, the **InfraEnv** resource must reside in a different namespace from the hosted control plane namespace. Do not delete the **InfraEnv** resource during the backup or restore process.



IMPORTANT

After you back up the hosted cluster, you must back up workloads in the data cluster and then destroy the original hosted cluster so that the restore process can begin.

Procedure

1. Set up your environment variables:
 - a. Set up environment variables for your hosted cluster by entering the following commands, replacing values as necessary:

```
$ CLUSTER_NAME=my-cluster
```

```
$ HOSTED_CLUSTER_NAMESPACE=clusters
```

```
$ CONTROL_PLANE_NAMESPACE="${HOSTED_CLUSTER_NAMESPACE}-  
${CLUSTER_NAME}"
```

- b. Pause reconciliation of the hosted cluster by entering the following command, replacing values as necessary:

```
$ oc patch -n ${HOSTED_CLUSTER_NAMESPACE}  
hostedclusters/${CLUSTER_NAME} \  
-p '{"spec":{"pausedUntil":"true"}}' --type=merge
```

2. Take a snapshot of etcd by using one of the following methods:
 - a. Use a previously backed-up snapshot of etcd.
 - b. If you have an available etcd pod, take a snapshot from the active etcd pod by completing the following steps:
 - i. List etcd pods by entering the following command:

```
$ oc get -n ${CONTROL_PLANE_NAMESPACE} pods -l app=etcd
```

- ii. Take a snapshot of the pod database and save it locally to your machine by entering the following commands:

```
$ ETCD_POD=etcd-0
```

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} -c etcd -t ${ETCD_POD} -- \
  env ETCDCTL_API=3 /usr/bin/etcdctl \
  --cacert /etc/etcd/tls/etcd-ca/ca.crt \
  --cert /etc/etcd/tls/client/etcd-client.crt \
  --key /etc/etcd/tls/client/etcd-client.key \
  --endpoints=https://localhost:2379 \
  snapshot save /var/lib/snapshot.db
```

- iii. Verify that the snapshot is successful by entering the following command:

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} -c etcd -t ${ETCD_POD} -- \
  env ETCDCTL_API=3 /usr/bin/etcdctl -w table snapshot status \
  /var/lib/snapshot.db
```

- c. Make a local copy of the snapshot by entering the following command:

```
$ oc cp -c etcd
${CONTROL_PLANE_NAMESPACE}/${ETCD_POD}:/var/lib/snapshot.db \
/tmp/etcd.snapshot.db
```

- i. Make a copy of the snapshot database from etcd persistent storage:

- A. List etcd pods by entering the following command:

```
$ oc get -n ${CONTROL_PLANE_NAMESPACE} pods -l app=etcd
```

- B. Find a pod that is running and set its name as the value of **ETCD_POD**:
ETCD_POD=etcd-0, and then copy its snapshot database by entering the following command:

```
$ oc cp -c etcd \
${CONTROL_PLANE_NAMESPACE}/${ETCD_POD}:/var/lib/data/member/snap/
db \
/tmp/etcd.snapshot.db
```

3. Scale down the etcd statefulset by entering the following command:

```
$ oc scale -n ${CONTROL_PLANE_NAMESPACE} statefulset/etcd --replicas=0
```

- a. Delete volumes for second and third members by entering the following command:

```
$ oc delete -n ${CONTROL_PLANE_NAMESPACE} pvc/data-etcd-1 pvc/data-etcd-2
```

- b. Create a pod to access the first etcd member's data:

- i. Get the etcd image by entering the following command:

```
$ ETCD_IMAGE=$(oc get -n ${CONTROL_PLANE_NAMESPACE} statefulset/etcd \
-o jsonpath='{ .spec.template.spec.containers[0].image }')
```

- ii. Create a pod that allows access to etcd data:

```
$ cat << EOF | oc apply -n ${CONTROL_PLANE_NAMESPACE} -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etcd-data
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etcd-data
  template:
    metadata:
      labels:
        app: etcd-data
    spec:
      containers:
        - name: access
          image: $ETCD_IMAGE
          volumeMounts:
            - name: data
              mountPath: /var/lib
          command:
            - /usr/bin/bash
          args:
            - -c
            - |-
              while true; do
                sleep 1000
              done
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: data-etcd-0
EOF
```

- iii. Check the status of the **etcd-data** pod and wait for it to be running by entering the following command:

```
$ oc get -n ${CONTROL_PLANE_NAMESPACE} pods -l app=etcd-data
```

- iv. Get the name of the **etcd-data** pod by entering the following command:

```
$ DATA_POD=$(oc get -n ${CONTROL_PLANE_NAMESPACE} pods --no-headers \
-l app=etcd-data -o name | cut -d/ -f2)
```

- c. Copy an etcd snapshot into the pod by entering the following command:

```
$ oc cp /tmp/etcd.snapshot.db \
  ${CONTROL_PLANE_NAMESPACE}/${DATA_POD}:/var/lib/restored.snap.db
```

- d. Remove old data from the **etcd-data** pod by entering the following commands:

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} ${DATA_POD} -- rm -rf /var/lib/data
```

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} ${DATA_POD} -- mkdir -p
/var/lib/data
```

- e. Restore the etcd snapshot by entering the following command:

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} ${DATA_POD} -- \
  etcdctl snapshot restore /var/lib/restored.snap.db \
  --data-dir=/var/lib/data --skip-hash-check \
  --name etcd-0 \
  --initial-cluster-token=etcd-cluster \
  --initial-cluster etcd-0=https://etcd-0.etcd-
discovery.${CONTROL_PLANE_NAMESPACE}.svc:2380,etcd-1=https://etcd-1.etcd-
discovery.${CONTROL_PLANE_NAMESPACE}.svc:2380,etcd-2=https://etcd-2.etcd-
discovery.${CONTROL_PLANE_NAMESPACE}.svc:2380 \
  --initial-advertise-peer-urls https://etcd-0.etcd-
discovery.${CONTROL_PLANE_NAMESPACE}.svc:2380
```

- f. Remove the temporary etcd snapshot from the pod by entering the following command:

```
$ oc exec -n ${CONTROL_PLANE_NAMESPACE} ${DATA_POD} -- \
  rm /var/lib/restored.snap.db
```

- g. Delete data access deployment by entering the following command:

```
$ oc delete -n ${CONTROL_PLANE_NAMESPACE} deployment/etcd-data
```

- h. Scale up the etcd cluster by entering the following command:

```
$ oc scale -n ${CONTROL_PLANE_NAMESPACE} statefulset/etcd --replicas=3
```

- i. Wait for the etcd member pods to return and report as available by entering the following command:

```
$ oc get -n ${CONTROL_PLANE_NAMESPACE} pods -l app=etcd -w
```

4. Restore reconciliation of the hosted cluster by entering the following command:

```
$ oc patch -n ${HOSTED_CLUSTER_NAMESPACE} hostedclusters/${CLUSTER_NAME} \
  -p '{"spec":{"pausedUntil":"null"}}' --type=merge
```

5. Manually roll out the hosted cluster by entering the following command:

```
$ oc annotate hostedcluster -n \
  <hosted_cluster_namespace> <hosted_cluster_name> \
  hypershift.openshift.io/restart-date=$(date --iso-8601=seconds)
```

The Multus admission controller and network node identity pods do not start yet.

6. Delete the pods for the second and third members of etcd and their PVCs by entering the following commands:

```
$ oc delete -n ${CONTROL_PLANE_NAMESPACE} pvc/data-etcd-1 pod/etcd-1 --wait=false
```

```
$ oc delete -n ${CONTROL_PLANE_NAMESPACE} pvc/data-etcd-2 pod/etcd-2 --wait=false
```

7. Manually roll out the hosted cluster again by entering the following command:

```
$ oc annotate hostedcluster -n \
  <hosted_cluster_namespace> <hosted_cluster_name> \
  hypershift.openshift.io/restart-date=$(date --iso-8601=seconds) \
  --overwrite
```

After a few minutes, the control plane pods start running.

8. If your hosted cluster is on AWS and you need to apply OIDC fixes after the restore process, enter the following command:

```
$ hcp fix dr-oidc-iam --hc-name <hosted_cluster_name> --hc-namespace
  <hosted_cluster_namespace> --aws-creds ~/.aws/credentials[4:48 AM]
```

This command regenerates the OIDC in S3 in case OIDC is deleted.

Additional resources

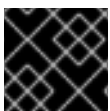
- [Configuring Amazon Web Services](#)

9.4. BACKING UP AND RESTORING ETCD ON THE MANAGEMENT CLUSTER

You can back up and restore etcd on the management cluster to fix failures.

9.4.1. Taking a snapshot of etcd for a hosted cluster

To back up etcd for a hosted cluster, you must take a snapshot of etcd. Later, you can restore etcd by using the snapshot.



IMPORTANT

This procedure requires API downtime.

Procedure

1. Stop all etcd-writer deployments by entering the following command:

```
$ oc scale deployment -n <hosted_cluster_namespace> --replicas=0 \
  kube-apiserver openshift-apiserver openshift-oauth-apiserver
```

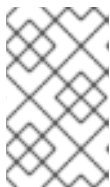
- To take an etcd snapshot, use the **exec** command in each etcd container by entering the following command:

```
$ oc exec -it <etcd_pod_name> -n <hosted_cluster_namespace> -- \
  env ETCDCTL_API=3 /usr/bin/etcdctl \
  --cacert /etc/etcd/tls/etcd-ca/ca.crt \
  --cert /etc/etcd/tls/client/etcd-client.crt \
  --key /etc/etcd/tls/client/etcd-client.key \
  --endpoints=localhost:2379 \
  snapshot save /var/lib/data/snapshot.db
```

- To check the snapshot status, use the **exec** command in each etcd container by running the following command:

```
$ oc exec -it <etcd_pod_name> -n <hosted_cluster_namespace> -- \
  env ETCDCTL_API=3 /usr/bin/etcdctl -w table snapshot status \
  /var/lib/data/snapshot.db
```

- Copy the snapshot data to a location where you can retrieve it later, such as an S3 bucket. See the following example.



NOTE

The following example uses signature version 2. If you are in a region that supports signature version 4, such as the **us-east-2** region, use signature version 4. Otherwise, when copying the snapshot to an S3 bucket, the upload fails.

Example

```
BUCKET_NAME=somebucket
CLUSTER_NAME=cluster_name
FILEPATH="/${BUCKET_NAME}/${CLUSTER_NAME}-snapshot.db"
CONTENT_TYPE="application/x-compressed-tar"
DATE_VALUE=`date -R`
SIGNATURE_STRING="PUT\n\n${CONTENT_TYPE}\n${DATE_VALUE}\n${FILEPATH}"
ACCESS_KEY=<access_key>
SECRET_KEY=<secret>
SIGNATURE_HASH=`echo -en ${SIGNATURE_STRING} | openssl sha1 -hmac
${SECRET_KEY} -binary | base64`
HOSTED_CLUSTER_NAMESPACE=hosted_cluster_namespace

$ oc exec -it etcd-0 -n ${HOSTED_CLUSTER_NAMESPACE} -- curl -X PUT -T
"/var/lib/data/snapshot.db" \
-H "Host: ${BUCKET_NAME}.s3.amazonaws.com" \
-H "Date: ${DATE_VALUE}" \
-H "Content-Type: ${CONTENT_TYPE}" \
-H "Authorization: AWS ${ACCESS_KEY}:${SIGNATURE_HASH}" \
https://${BUCKET_NAME}.s3.amazonaws.com/${CLUSTER_NAME}-snapshot.db
```

- To restore the snapshot on a new cluster later, save the encryption secret that the hosted cluster references.
 - Get the secret encryption key by entering the following command:

```
$ oc get -n <hosted_cluster_namespace> hostedcluster <hosted_cluster_name> \
-o=jsonpath='{.spec.secretEncryption.aescbc}'
```

Example output

```
{"activeKey":{"name":"<hosted_cluster_name>-etcd-encryption-key"}}
```

- b. Save the secret encryption key by entering the following command:

```
$ oc get -n <hosted_cluster_namespace> secret <hosted_cluster_name>-etcd-
encryption-key \
-o=jsonpath='{.data.key}'
```

You can decrypt this key when restoring a snapshot on a new cluster.

6. Restart all etcd-writer deployments by entering the following command:

```
$ oc scale deployment -n <control_plane_namespace> --replicas=3 \
kube-apiserver openshift-apiserver openshift-oauth-apiserver
```

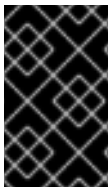
Next steps

Restore the etcd snapshot.

9.4.2. Restoring an etcd snapshot on a hosted cluster

If you have a snapshot of etcd from your hosted cluster, you can restore it. Currently, you can restore an etcd snapshot only during cluster creation.

To restore an etcd snapshot, you change the output from the **create cluster --render** command and define a **restoreSnapshotURL** value in the etcd section of the **HostedCluster** specification.



IMPORTANT

If the snapshot that you are restoring is from a hosted cluster with **LoadBalancer** services, the load balancer IPs might no longer be valid. In that case, you must delete and re-create the **LoadBalancer** services.

Prerequisites

- You took an etcd snapshot on a hosted cluster.

Procedure

1. Delete the hosted cluster that you backed up in "Taking a snapshot of etcd for a hosted cluster" by entering the following command:

```
$ hcp destroy cluster <cluster_infra> \
--name <hosted_cluster_name> \
--namespace <hosted_cluster_namespace>
```

2. On the **aws** command-line interface (CLI), create a pre-signed URL so that you can download your etcd snapshot from S3 without passing credentials to the etcd deployment:

- a. Define the snapshot by entering the following command:

```
$ ETCD_SNAPSHOT=${ETCD_SNAPSHOT:-
"s3://${BUCKET_NAME}/${CLUSTER_NAME}-snapshot.db"}
```

- b. Define the snapshot URL by entering the following command:

```
$ ETCD_SNAPSHOT_URL=$(aws s3 presign ${ETCD_SNAPSHOT})
```

3. Create the new hosted cluster by entering the following command:

```
$ hcp create cluster <platform> \
  --name <hosted_cluster_name> \
  --namespace <hosted_cluster_namespace> \
  --node-pool-replicas=2 \
  --node-upgrade-type=Replace \
  --pull-secret <path_to_pull_secret> \
  --memory <value_for_memory> \
  --cores <value_for_cpu> \
  --etcd-storage-class=gp3-csi \
  --release-image=<release_image_reference> \
  --etcd-storage-size=8Gi \
  --fips=false \
  --render \
  --render-sensitive
```

- **<hosted_cluster_name>** specifies the name of the new hosted cluster. The name of the new cluster must be identical to the name of the cluster from which the etcd backup was taken.
- **<platform>** specifies the platform you are creating the hosted cluster on, such as **kubevirt** or **aws**.
- **<hosted_cluster_namespace>** specifies the namespace where you are creating the hosted cluster.
- **<path_to_pull_secret>** specifies the path to your pull secret; for example, **/user/name/pullsecret**.
- **<value_for_memory>** specifies the memory value, such as **8Gi**.
- **<value_for_cpu>** specifies the CPU value, such as **2**.
- **<release_image_reference>** specifies the OpenShift Container Platform release image for the cluster, for example, **quay.io/openshift-release-dev/ocp-release:4.20.14-multi**. You can use the **--release-image** flag to set up the hosted cluster with a specific OpenShift Container Platform release.



NOTE

The **--render** flag in the **hcp create** command does not render the secrets. To render the secrets, you must use both the **--render** and the **--render-sensitive** flags in the **hcp create** command.

Example output

```

apiVersion: v1
kind: Namespace
metadata:
  name: <hosted_cluster_namespace>
spec: {}
status: {}
---
apiVersion: v1
data:
  .dockerconfigjson: <path_to_pull_secret>
kind: Secret
metadata:
  labels:
    hypershift.openshift.io/safe-to-delete-with-cluster: "true"
  name: <hosted_cluster_name_pull_secret>
  namespace: <hosted_cluster_namespace>
---
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  autoscaling: {}
  capabilities: {}
  configuration: {}
  controllerAvailabilityPolicy: HighlyAvailable
  dns:
    baseDomain: ""
  etcd:
    managed:
      storage:
        persistentVolume:
          size: 8Gi
          storageClassName: gp3-csi
          type: PersistentVolume
        managementType: Managed
  fips: false
  infraID: <hosted_cluster_name>-68tlg
  infrastructureAvailabilityPolicy: HighlyAvailable
  networking:
    clusterNetwork:
      - cidr: 10.132.0.0/14
    networkType: OVNKubernetes
    serviceNetwork:
      - cidr: 172.31.0.0/16
  olmCatalogPlacement: guest
  platform:
    kubevirt:
      baseDomainPassthrough: true
      type: KubeVirt
  pullSecret:
    name: <hosted_cluster_name_pull_secret>
  release:

```

```

    image: quay.io/openshift-release-dev/ocp-release:4.21.12-multi
  services:
    - service: APIServer
      servicePublishingStrategy:
        type: LoadBalancer
    - service: Ignition
      servicePublishingStrategy:
        type: Route
    - service: Konnectivity
      servicePublishingStrategy:
        type: Route
    - service: OAuthServer
      servicePublishingStrategy:
        type: Route
  sshKey: {}
  status:
    controlPlaneEndpoint:
      host: ""
      port: 0
  ---
  apiVersion: hypershift.openshift.io/v1beta1
  kind: NodePool
  metadata:
    name: <hosted_cluster_name>
    namespace: <hosted_cluster_namespace>
  spec:
    arch: amd64
    clusterName: <hosted_cluster_name>
    management:
      autoRepair: false
      upgradeType: Replace
    nodeDrainTimeout: 0s
    nodeVolumeDetachTimeout: 0s
    platform:
      kubevirt:
        attachDefaultNetwork: true
      compute:
        cores: 2
        memory: 8Gi
        networkInterfaceMultiqueue: Enable
        rootVolume:
          persistent:
            size: 32Gi
            type: Persistent
        type: KubeVirt
    release:
      image: quay.io/openshift-release-dev/ocp-release:<4.x.x>-multi
    replicas: 2
  status:
    replicas: 0

```

- **<hosted_cluster_namespace>** specifies the name of the hosted cluster namespace.
- **<path_to_pull_secret>** specifies the path to the pull secret.

- **<hosted_cluster_name_pull_secret>** specifies the name of the restored pull secret for the new hosted cluster.
- **<hosted_cluster_name>** specifies the name of the new hosted cluster. The name of the new cluster must be identical to the name of the cluster from which the etcd backup was taken.
- **<4.x.x>** specifies the version of the release image.

4. Change the **HostedCluster** specification to refer to the URL:

```
spec:
  etcd:
    managed:
      storage:
        persistentVolume:
          size: 8Gi
          type: PersistentVolume
        restoreSnapshotURL:
          - "${ETCD_SNAPSHOT_URL}"
      managementType: Managed
```

5. Change the **HostedCluster** specification to include the new etcd encryption key:

```
apiVersion: v1
data:
  key: <pre_generated_etcd_encryption_key>
kind: Secret
metadata:
  labels:
    hypershift.openshift.io/safe-to-delete-with-cluster: "true"
  name: <new_hc_etcd_encryption_key>
  namespace: <hosted_cluster_namespace>
type: Opaque
---
spec:
  secretEncryption:
    aescbc:
      activeKey:
        name: <new_hc_etcd_encryption_key>
        type: aescbc
```

- **<pre_generated_etcd_encryption_key>** specifies the etcd encryption key of original hosted cluster.
- **<new_hc_etcd_encryption_key>** specifies the etcd encryption key of the new hosted cluster. Ensure that the secret that you referenced from the **spec.secretEncryption.aescbc** value has the same Advanced Encryption Standard (AES) key that you saved earlier.

Verification

- To verify that the snapshot was restored, enter the following command:

```
$ oc logs -n <hosted_control_plane_namespace> etcd-0 -c etcd-init
```

If you use a high availability deployment, you can also check the **etcd-1** and **etcd-2** containers.

9.5. BACKING UP AND RESTORING A HOSTED CLUSTER ON OPENSIFT VIRTUALIZATION

You can back up and restore a hosted cluster on OpenShift Virtualization to fix failures.

9.5.1. Backing up a hosted cluster on OpenShift Virtualization

When you back up a hosted cluster on OpenShift Virtualization, the hosted cluster can remain running. The backup contains the hosted control plane components and the etcd for the hosted cluster.

When the hosted cluster is not running compute nodes on external infrastructure, hosted cluster workload data that is stored in persistent volume claims (PVCs) that are provisioned by KubeVirt CSI are also backed up. The backup does not contain any KubeVirt virtual machines (VMs) that are used as compute nodes. Those VMs are automatically re-created after the restore process is completed.

Procedure

1. Create a Velero backup resource by creating a YAML file that is similar to the following example:

```
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: hc-clusters-hosted-backup
  namespace: openshift-adp
  labels:
    velero.io/storage-location: default
spec:
  includedNamespaces:
    - clusters
    - clusters-hosted
  includedResources:
    - sa
    - role
    - rolebinding
    - deployment
    - statefulset
    - pv
    - pvc
    - bmh
    - configmap
    - infraenv
    - priorityclasses
    - pdb
    - hostedcluster
    - nodepool
    - secrets
    - hostedcontrolplane
    - cluster
    - datavolume
    - service
    - route
  excludedResources: [ ]
  labelSelector:
```

```

matchExpressions:
- key: 'hypershift.openshift.io/is-kubevirt-rhcos'
  operator: 'DoesNotExist'
storageLocation: default
preserveNodePorts: true
ttl: 4h0m0s
snapshotMoveData: true
datamover: "velero"
defaultVolumesToFsBackup: false

```

where:

includedNamespaces

Specifies the namespaces from the objects to back up. Include namespaces from both the hosted cluster and the hosted control plane. In this example, **clusters** is a namespace from the hosted cluster and **clusters-hosted** is a namespace from the hosted control plane.



IMPORTANT

If you store all hosted cluster information in a shared namespace and then back up and restore a hosted cluster, you might unintentionally change other hosted clusters. To avoid this issue, do not store all hosted cluster information in a shared namespace. Alternatively, you can back up and restore resources based on labels.

labelSelector

Specifies the boot image of the VMs that are used as the hosted cluster nodes are stored in large PVCs. To reduce backup time and storage size, you can filter those PVCs out of the backup by adding this label selector.

snapshotMoveData

Along with the **datamover** field, specifies that the CSI **VolumeSnapshots** are automatically uploaded to remote cloud storage.

datamover

Along with the **snapshotMoveData** field, specifies that the CSI **VolumeSnapshots** are automatically uploaded to remote cloud storage.

defaultVolumesToFsBackup

Specifies whether pod volume file system backup is used for all volumes by default. Set this value to **false** to back up the PVCs that you want.

2. Apply the changes to the YAML file by entering the following command:

```
$ oc apply -f <backup_file_name>.yaml
```

Replace **<backup_file_name>** with the name of your file.

3. Monitor the backup process in the backup object status and in the Velero logs.

- To monitor the backup object status, enter the following command:

```
$ watch "oc get backups.velero.io -n openshift-adp <backup_file_name> -o
jsonpath='{.status}' | jq"
```

- To monitor the Velero logs, enter the following command:

```
$ oc logs -n openshift-adp -ldeploy=velero -f
```

Verification

- When the **status.phase** field is **Completed**, the backup process is considered complete.

9.5.2. Restoring a hosted cluster on OpenShift Virtualization

After you back up a hosted cluster on OpenShift Virtualization, you can restore the backup.



NOTE

The restore process can be completed only on the same management cluster where you created the backup.

Procedure

1. Ensure that no pods or persistent volume claims (PVCs) are running in the **HostedControlPlane** namespace.
2. Delete the following objects from the management cluster:
 - **HostedCluster**
 - **NodePool**
 - PVCs
3. Create a restoration manifest YAML file that is similar to the following example:

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: hc-clusters-hosted-restore
  namespace: openshift-adp
spec:
  backupName: hc-clusters-hosted-backup
  restorePVs: true
  existingResourcePolicy: update
  excludedResources:
    - nodes
    - events
    - events.events.k8s.io
    - backups.velero.io
    - restores.velero.io
    - resticrepositories.velero.io
```

- **spec.restorePVs** starts the recovery of pods with the included persistent volumes.
- **spec.existingResourcePolicy: update** ensures that any existing objects are overwritten with backup content. This action can cause issues with objects that contain immutable fields, which is why you deleted the **HostedCluster**, node pools, and PVCs. If you do not set

this policy, the Velero engine skips the restoration of objects that already exist.

4. Apply the changes to the YAML file by entering the following command:

```
$ oc apply -f <restore_resource_file_name>.yaml
```

Replace **<restore_resource_file_name>** with the name of your file.

5. Monitor the restore process by checking the restore status field and the Velero logs.

- To check the restore status field, enter the following command:

```
$ watch "oc get restores.velero.io -n openshift-adp <backup_file_name> -o  
jsonpath='{.status}' | jq"
```

- To check the Velero logs, enter the following command:

```
$ oc logs -n openshift-adp -ldeploy=velero -f
```

Verification

- When the **status.phase** field is **Completed**, the restore process is considered complete.

Next steps

- After some time, the KubeVirt VMs are created and join the hosted cluster as compute nodes. Make sure that the hosted cluster workloads are running again as expected.

9.6. DISASTER RECOVERY FOR A HOSTED CLUSTER IN AWS

You can recover a hosted cluster to the same region within Amazon Web Services (AWS). For example, you need disaster recovery when the upgrade of a management cluster fails and the hosted cluster is in a read-only state.

The disaster recovery process involves the following steps:

1. Backing up the hosted cluster on the source management cluster
2. Restoring the hosted cluster on a destination management cluster
3. Deleting the hosted cluster from the source management cluster

Your workloads remain running during the process. The Cluster API might be unavailable for a period, but that does not affect the services that are running on the worker nodes.

IMPORTANT

Both the source management cluster and the destination management cluster must have the **--external-dns** flags to maintain the API server URL. See the following example:

Example: External DNS flags

```
--external-dns-provider=aws \
--external-dns-credentials=<path_to_aws_credentials_file> \
--external-dns-domain-filter=<basedomain>
```

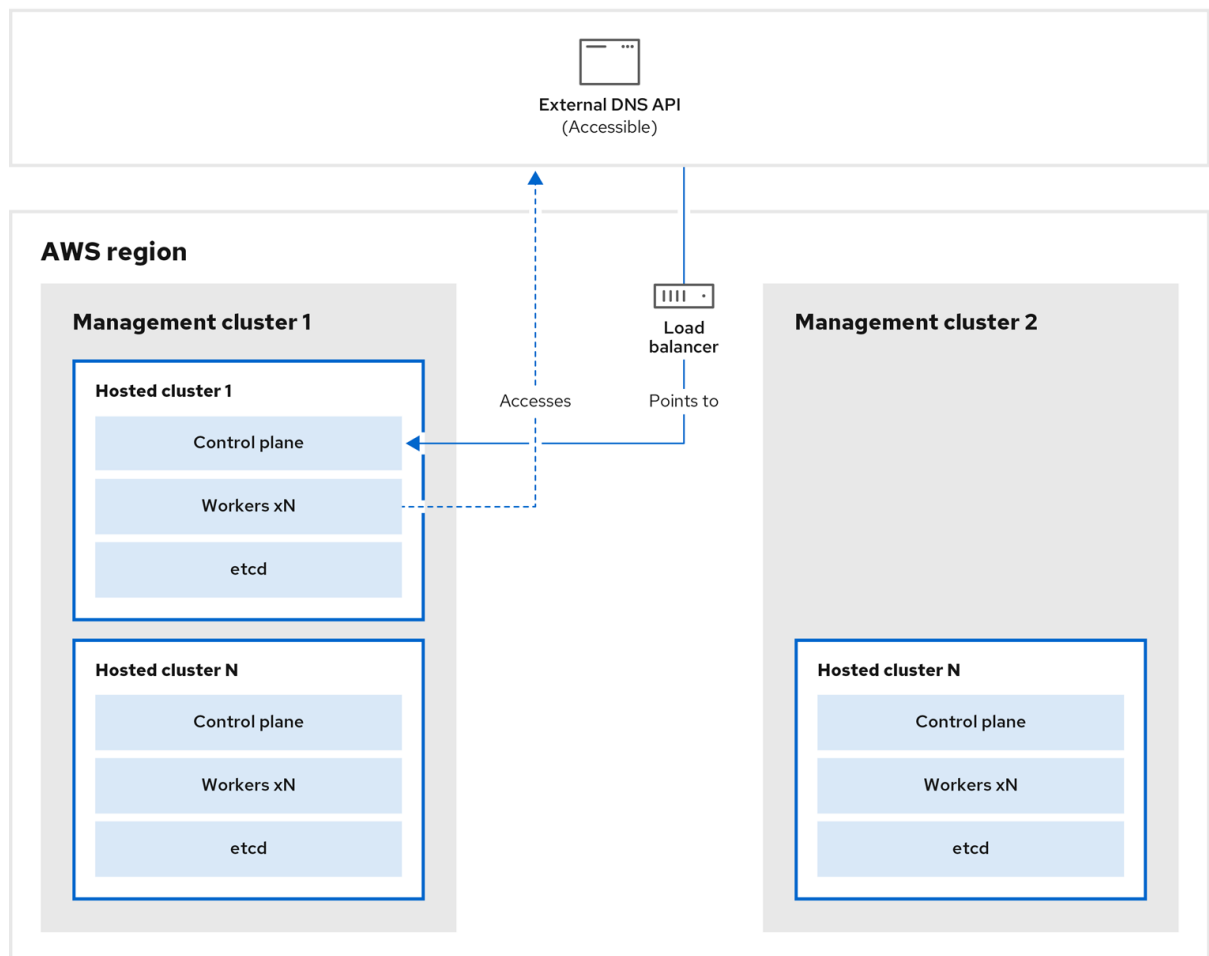
If you do not include the **--external-dns** flags to maintain the API server URL, you cannot migrate the hosted cluster.

9.6.1. Overview of the backup and restore process for hosted control planes on AWS

Get familiar with the backup and restore process for hosted control planes on Amazon Web Services (AWS).

The backup and restore process works as follows:

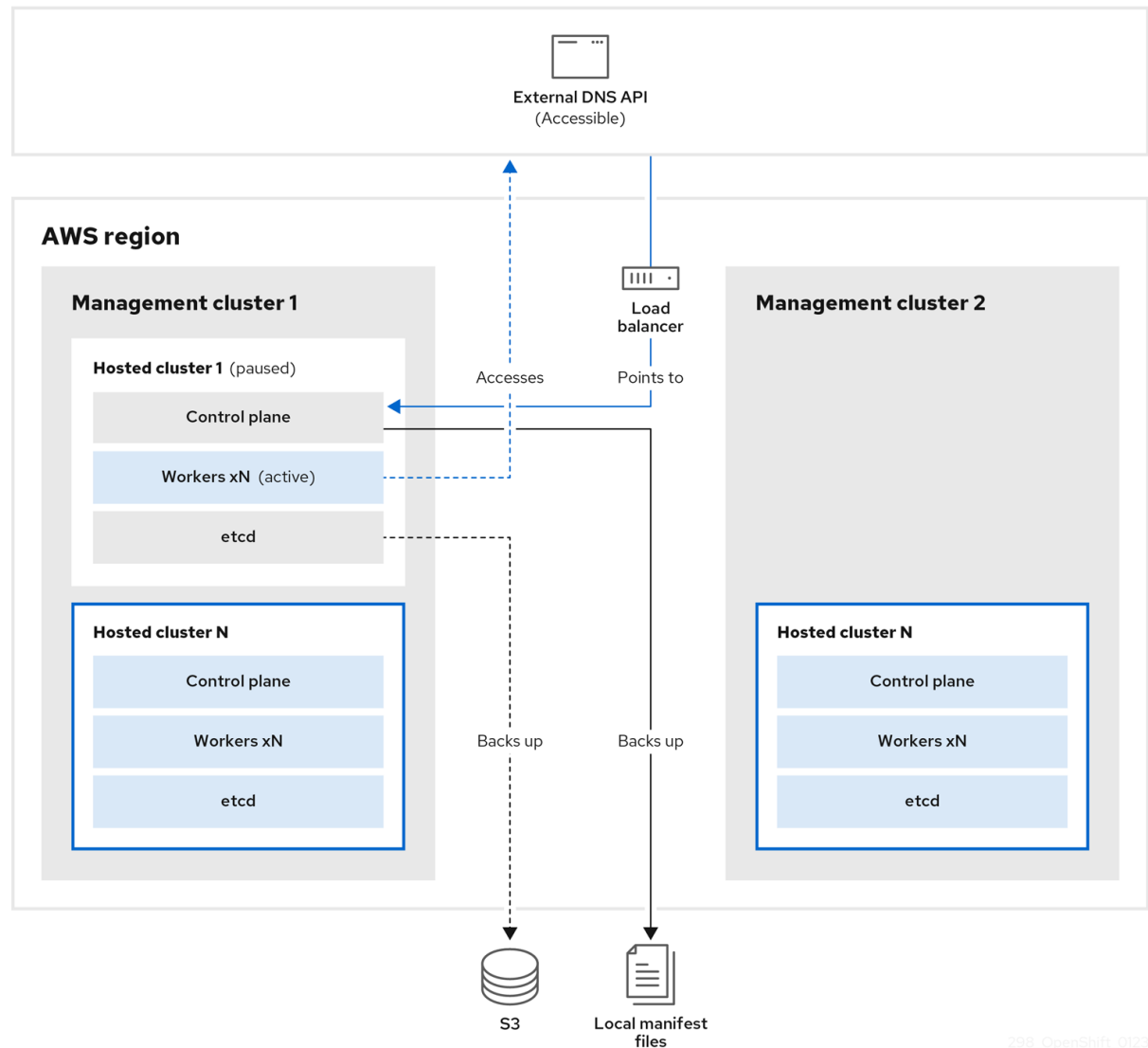
1. On management cluster 1, which you can think of as the source management cluster, the control plane and workers interact by using the external DNS API. The external DNS API is accessible, and a load balancer sits between the management clusters.



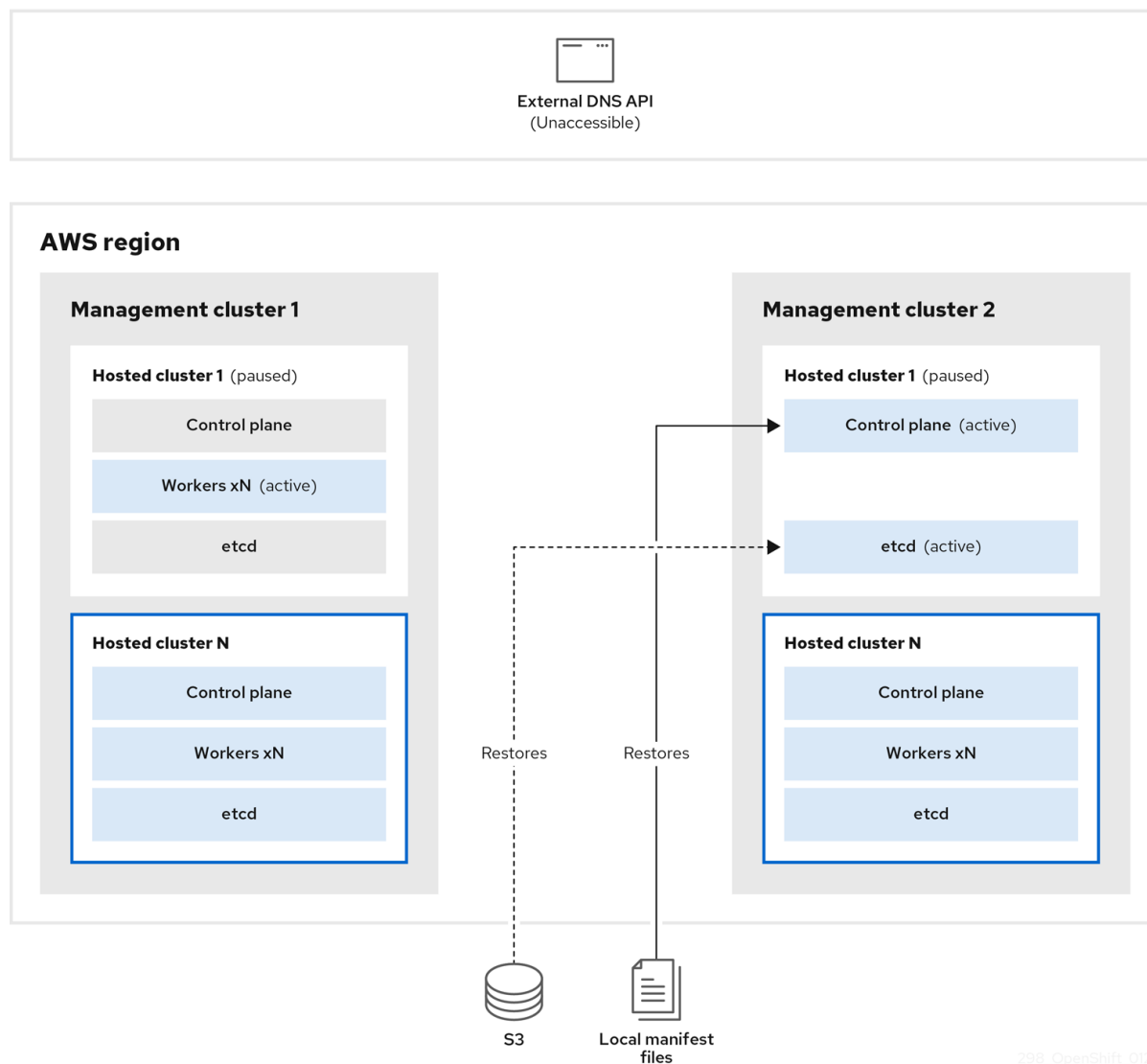
298_OpenShift_0123

2. You take a snapshot of the hosted cluster, which includes etcd, the control plane, and the

worker nodes. During this process, the worker nodes continue to try to access the external DNS API even if it is not accessible, the workloads are running, the control plane is saved in a local manifest file, and etcd is backed up to an S3 bucket. The data plane is active and the control plane is paused.

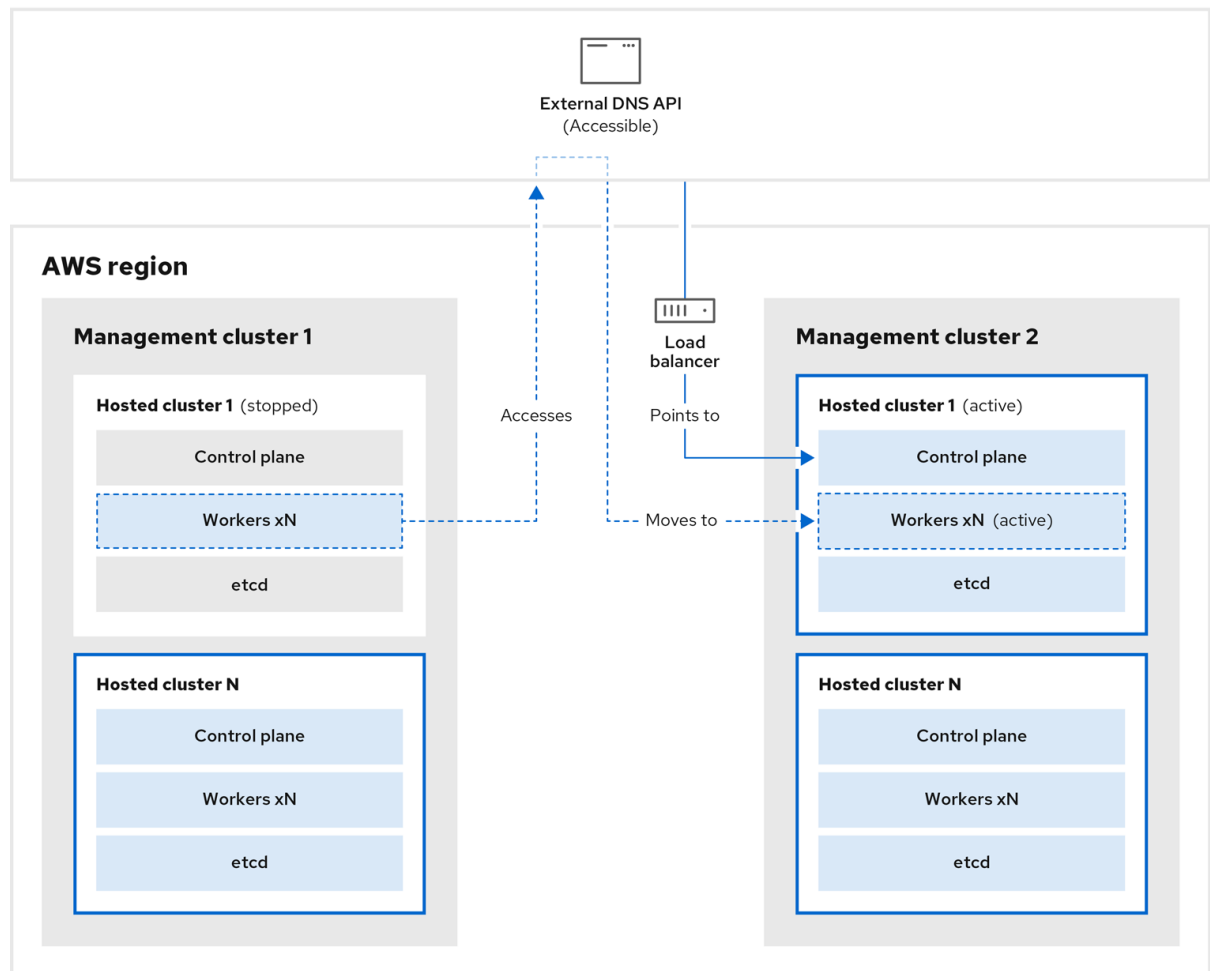


- On management cluster 2, which you can think of as the destination management cluster, you restore etcd from the S3 bucket and restore the control plane from the local manifest file. During this process, the external DNS API is stopped, the hosted cluster API becomes inaccessible, and any workers that use the API are unable to update their manifest files, but the workloads are still running.



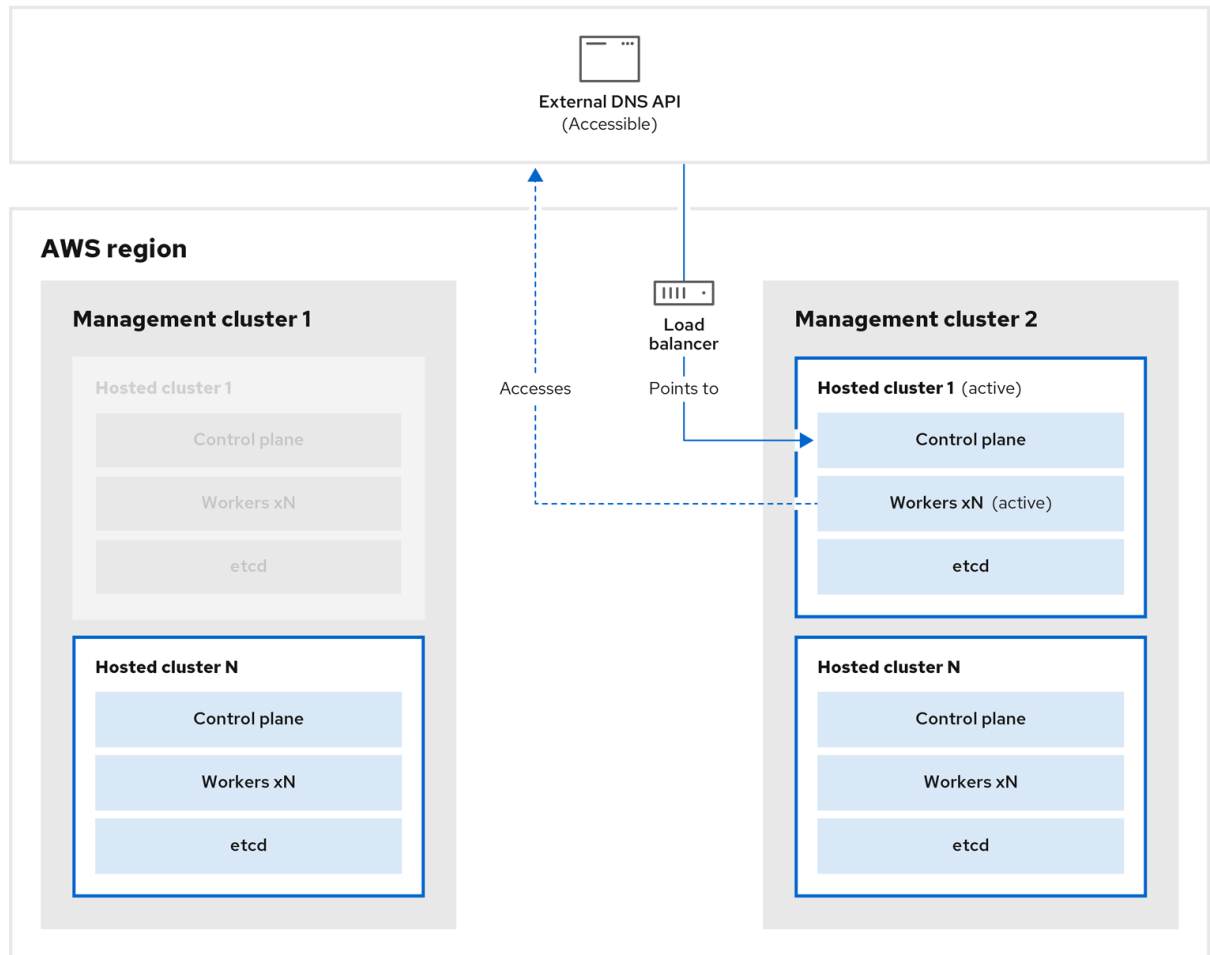
298_OpenShift_0123

4. The external DNS API is accessible again, and the worker nodes use it to move to management cluster 2. The external DNS API can access the load balancer that points to the control plane.



298_OpenShift_0123

- On management cluster 2, the control plane and worker nodes interact by using the external DNS API. The resources are deleted from management cluster 1, except for the S3 backup of etcd. If you try to set up the hosted cluster again on management cluster 1, it will not work.



298_OpenShift_0123

9.6.2. Backing up a hosted cluster on AWS

To recover your hosted cluster in your target management cluster, you first need to back up all of the relevant data.

Procedure

1. Create a config map file to declare the source management cluster by entering the following command:

```
$ oc create configmap mgmt-parent-cluster -n default \
  --from-literal=from=${MGMT_CLUSTER_NAME}
```

2. Shut down the reconciliation in the hosted cluster and in the node pools by entering the following commands:

```
$ PAUSED_UNTIL="true"
```

```
$ oc patch -n ${HC_CLUSTER_NS} hostedclusters/${HC_CLUSTER_NAME} \
  -p '{"spec":{"pausedUntil":"'${PAUSED_UNTIL}'"}}' --type=merge
```

```
$ oc patch -n ${HC_CLUSTER_NS} nodepools/${NODEPOOLS} \
  -p '{"spec":{"pausedUntil":"'${PAUSED_UNTIL}'"}}' --type=merge
```

```
$ oc scale deployment -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --replicas=0 \
kube-apiserver openshift-apiserver openshift-oauth-apiserver control-plane-operator
```

3. Back up etcd and upload the data to an S3 bucket by running the following bash script:

TIP

Wrap this script in a function and call it from the main function.

```
# ETCD Backup
ETCD_PODS="etcd-0"
if [ "${CONTROL_PLANE_AVAILABILITY_POLICY}" = "HighlyAvailable" ]; then
    ETCD_PODS="etcd-0 etcd-1 etcd-2"
fi

for POD in ${ETCD_PODS}; do
    # Create an etcd snapshot
    oc exec -it ${POD} -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -- env
    ETCDCTL_API=3 /usr/bin/etcdctl --cacert /etc/etcd/tls/client/etcd-client-ca.crt --cert
    /etc/etcd/tls/client/etcd-client.crt --key /etc/etcd/tls/client/etcd-client.key --
    endpoints=localhost:2379 snapshot save /var/lib/data/snapshot.db
    oc exec -it ${POD} -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -- env
    ETCDCTL_API=3 /usr/bin/etcdctl -w table snapshot status /var/lib/data/snapshot.db

    FILEPATH="/${BUCKET_NAME}/${HC_CLUSTER_NAME}-${POD}-snapshot.db"
    CONTENT_TYPE="application/x-compressed-tar"
    DATE_VALUE=`date -R`
    SIGNATURE_STRING="PUT\n\n${CONTENT_TYPE}\n${DATE_VALUE}\n${FILEPATH}"

    set +x
    ACCESS_KEY=$(grep aws_access_key_id ${AWS_CREDS} | head -n1 | cut -d= -f2 | sed
    "s/ //g")
    SECRET_KEY=$(grep aws_secret_access_key ${AWS_CREDS} | head -n1 | cut -d= -f2 |
    sed "s/ //g")
    SIGNATURE_HASH=$(echo -en ${SIGNATURE_STRING} | openssl sha1 -hmac
    "${SECRET_KEY}" -binary | base64)
    set -x

    # FIXME: this is pushing to the OIDC bucket
    oc exec -it etcd-0 -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -- curl -X PUT -T
    "/var/lib/data/snapshot.db" \
    -H "Host: ${BUCKET_NAME}.s3.amazonaws.com" \
    -H "Date: ${DATE_VALUE}" \
    -H "Content-Type: ${CONTENT_TYPE}" \
    -H "Authorization: AWS ${ACCESS_KEY}:${SIGNATURE_HASH}" \
    https://${BUCKET_NAME}.s3.amazonaws.com/${HC_CLUSTER_NAME}-${POD}-
    snapshot.db
done
```

For more information about backing up etcd, see "Backing up and restoring etcd on a hosted cluster".

4. Back up Kubernetes and OpenShift Container Platform objects by entering the following commands. You need to back up the following objects:

- **HostedCluster** and **NodePool** objects from the HostedCluster namespace
- **HostedCluster** secrets from the HostedCluster namespace
- **HostedControlPlane** from the Hosted Control Plane namespace
- **Cluster** from the Hosted Control Plane namespace
- **AWSCluster**, **AWSMachineTemplate**, and **AWSMachine** from the Hosted Control Plane namespace
- **MachineDeployments**, **MachineSets**, and **Machines** from the Hosted Control Plane namespace
- **ControlPlane** secrets from the Hosted Control Plane namespace

- a. Enter the following commands:

```
$ mkdir -p ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS} \
    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}
```

```
$ chmod 700 ${BACKUP_DIR}/namespaces/
```

- b. Back up the **HostedCluster** objects from the **HostedCluster** namespace by entering the following commands:

```
$ echo "Backing Up HostedCluster Objects:"
```

```
$ oc get hc ${HC_CLUSTER_NAME} -n ${HC_CLUSTER_NS} -o yaml > \
    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/hc-
    ${HC_CLUSTER_NAME}.yaml
```

```
$ echo "--> HostedCluster"
```

```
$ sed -i " -e '/^status:$/, $ d' \
    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/hc-
    ${HC_CLUSTER_NAME}.yaml
```

- c. Back up the **NodePool** objects from the **HostedCluster** namespace by entering the following commands:

```
$ oc get np ${NODEPOOLS} -n ${HC_CLUSTER_NS} -o yaml > \
    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/np-${NODEPOOLS}.yaml
```

```
$ echo "--> NodePool"
```

```
$ sed -i " -e '/^status:$/, $ d' \
    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/np-${NODEPOOLS}.yaml
```

- d. Back up the secrets in the **HostedCluster** namespace by running the following shell script:

```
$ echo "--> HostedCluster Secrets:"
for s in $(oc get secret -n ${HC_CLUSTER_NS} | grep "^${HC_CLUSTER_NAME}" |
awk '{print $1}'); do
    oc get secret -n ${HC_CLUSTER_NS} $s -o yaml >
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/secret-${s}.yaml
done
```

- e. Back up the secrets in the **HostedCluster** control plane namespace by running the following shell script:

```
$ echo "--> HostedCluster ControlPlane Secrets:"
for s in $(oc get secret -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} | egrep -v
"docker|service-account-token|oauth-openshift|NAME|token-
${HC_CLUSTER_NAME}" | awk '{print $1}'); do
    oc get secret -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} $s -o yaml >
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/secret-${s}.yaml
done
```

- f. Back up the hosted control plane by entering the following commands:

```
$ echo "--> HostedControlPlane:"
```

```
$ oc get hcp ${HC_CLUSTER_NAME} -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o yaml > \
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/hcp-${HC_CLUSTER_NAME}.yaml
```

- g. Back up the cluster by entering the following commands:

```
$ echo "--> Cluster:"
```

```
$ CL_NAME=$(oc get hcp ${HC_CLUSTER_NAME} -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} \
-o jsonpath={.metadata.labels.*} | grep ${HC_CLUSTER_NAME})
```

```
$ oc get cluster ${CL_NAME} -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -o
yaml > \
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}/cl-
${HC_CLUSTER_NAME}.yaml
```

- h. Back up the AWS cluster by entering the following commands:

```
$ echo "--> AWS Cluster:"
```

```
$ oc get awscluster ${HC_CLUSTER_NAME} -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o yaml > \
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awscl-${HC_CLUSTER_NAME}.yaml
```

- i. Back up the AWS **MachineTemplate** objects by entering the following commands:

■

```
$ echo "--> AWS Machine Template:"
```

```
$ oc get awsmachinetemplate ${NODEPOOLS} -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o yaml > \
  ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awsmt-${HC_CLUSTER_NAME}.yaml
```

- j. Back up the AWS **Machines** objects by running the following shell script:

```
$ echo "--> AWS Machine:"
```

```
$ CL_NAME=$(oc get hcp ${HC_CLUSTER_NAME} -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o jsonpath={.metadata.labels.*} | grep
${HC_CLUSTER_NAME})
for s in $(oc get awsmachines -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --
no-headers | grep ${CL_NAME} | cut -f1 -d\ ); do
  oc get -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} awsmachines $s -o
yaml > ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awsm-${s}.yaml
done
```

- k. Back up the **MachineDeployments** objects by running the following shell script:

```
$ echo "--> HostedCluster MachineDeployments:"
for s in $(oc get machinedeployment -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o name); do
  mdp_name=$(echo ${s} | cut -f 2 -d /)
  oc get -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} $s -o yaml >
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/machinedeployment-${mdp_name}.yaml
done
```

- l. Back up the **MachineSets** objects by running the following shell script:

```
$ echo "--> HostedCluster MachineSets:"
for s in $(oc get machineset -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -o
name); do
  ms_name=$(echo ${s} | cut -f 2 -d /)
  oc get -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} $s -o yaml >
${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/machineset-${ms_name}.yaml
done
```

- m. Back up the **Machines** objects from the Hosted Control Plane namespace by running the following shell script:

```
$ echo "--> HostedCluster Machine:"
for s in $(oc get machine -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -o
name); do
  m_name=$(echo ${s} | cut -f 2 -d /)
  oc get -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} $s -o yaml >
```

```

    ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
    ${HC_CLUSTER_NAME}/machine-${m_name}.yaml
done

```

5. Clean up the **ControlPlane** routes by entering the following command:

```
$ oc delete routes -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --all
```

By entering that command, you enable the ExternalDNS Operator to delete the Route53 entries.

6. Verify that the Route53 entries are clean by running the following script:

```

function clean_routes() {

    if [[ -z "${1}" ]];then
        echo "Give me the NS where to clean the routes"
        exit 1
    fi

    # Constants
    if [[ -z "${2}" ]];then
        echo "Give me the Route53 zone ID"
        exit 1
    fi

    ZONE_ID=${2}
    ROUTES=10
    timeout=40
    count=0

    # This allows us to remove the ownership in the AWS for the API route
    oc delete route -n ${1} --all

    while [ ${ROUTES} -gt 2 ]
    do
        echo "Waiting for ExternalDNS Operator to clean the DNS Records in AWS Route53
where the zone id is: ${ZONE_ID}..."
        echo "Try: (${count}/${timeout})"
        sleep 10
        if [[ $count -eq timeout ]];then
            echo "Timeout waiting for cleaning the Route53 DNS records"
            exit 1
        fi
        count=$((count+1))
        ROUTES=$(aws route53 list-resource-record-sets --hosted-zone-id ${ZONE_ID} --max-
items 10000 --output json | grep -c ${EXTERNAL_DNS_DOMAIN})
    done
}

# SAMPLE: clean_routes "<HC ControlPlane Namespace>" "<AWS_ZONE_ID>"
clean_routes "${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}" "${AWS_ZONE_ID}"

```

Verification

Check all of the OpenShift Container Platform objects and the S3 bucket to verify that everything looks as expected.

Next steps

Restore your hosted cluster.

9.6.3. Restoring a hosted cluster

Gather all of the objects that you backed up and restore them in your destination management cluster.

Prerequisites

- You backed up the data from your source management cluster.

TIP

Ensure that the **kubeconfig** file of the destination management cluster is placed as it is set in the **KUBECONFIG** variable or, if you use the script, in the **MGMT2_KUBECONFIG** variable. Use **export KUBECONFIG=<Kubeconfig FilePath>** or, if you use the script, use **export KUBECONFIG=\${MGMT2_KUBECONFIG}**.

Procedure

1. Verify that the new management cluster does not contain any namespaces from the cluster that you are restoring by entering these commands:

```
$ export KUBECONFIG=${MGMT2_KUBECONFIG}
```

```
$ BACKUP_DIR=${HC_CLUSTER_DIR}/backup
```

Namespace deletion in the destination Management cluster

```
$ oc delete ns ${HC_CLUSTER_NS} || true
```

```
$ oc delete ns ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} || true
```

2. Re-create the deleted namespaces by entering these commands:

Namespace creation commands

```
$ oc new-project ${HC_CLUSTER_NS}
```

```
$ oc new-project ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}
```

3. Restore the secrets in the HC namespace by entering this command:

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/secret-*
```

4. Restore the objects in the **HostedCluster** control plane namespace by entering these commands:

Restore secret command

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/secret-*
```

Cluster restore commands

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/hcp-*
```

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/cl-*
```

5. If you are recovering the nodes and the node pool to reuse AWS instances, restore the objects in the HC control plane namespace by entering these commands:

Commands for AWS

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awscl-*
```

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awsmt-*
```

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/awsm-*
```

Commands for machines

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/machinedeployment-*
```

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/machineset-*
```

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME}/machine-*
```

6. Restore the etcd data and the hosted cluster by running this bash script:

```
ETCD_PODS="etcd-0"
if [ "${CONTROL_PLANE_AVAILABILITY_POLICY}" = "HighlyAvailable" ]; then
  ETCD_PODS="etcd-0 etcd-1 etcd-2"
fi

HC_RESTORE_FILE=${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/hc-
${HC_CLUSTER_NAME}-restore.yaml
HC_BACKUP_FILE=${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/hc-
${HC_CLUSTER_NAME}.yaml
HC_NEW_FILE=${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/hc-
${HC_CLUSTER_NAME}-new.yaml
```

```

cat ${HC_BACKUP_FILE} > ${HC_NEW_FILE}
cat > ${HC_RESTORE_FILE} <<EOF
    restoreSnapshotURL:
EOF

for POD in ${ETCD_PODS}; do
    # Create a pre-signed URL for the etcd snapshot
    ETCD_SNAPSHOT="s3://${BUCKET_NAME}/${HC_CLUSTER_NAME}-${POD}-
snapshot.db"
    ETCD_SNAPSHOT_URL=$(AWS_DEFAULT_REGION=${MGMT2_REGION} aws s3
presign ${ETCD_SNAPSHOT})

    # FIXME no CLI support for restoreSnapshotURL yet
    cat >> ${HC_RESTORE_FILE} <<EOF
        - "${ETCD_SNAPSHOT_URL}"
    EOF
done

cat ${HC_RESTORE_FILE}

if ! grep ${HC_CLUSTER_NAME}-snapshot.db ${HC_NEW_FILE}; then
    sed -i " -e '/type: PersistentVolume/r ${HC_RESTORE_FILE}'" ${HC_NEW_FILE}
    sed -i " -e '/pausedUntil:/d'" ${HC_NEW_FILE}
fi

HC=$(oc get hc -n ${HC_CLUSTER_NS} ${HC_CLUSTER_NAME} -o name || true)
if [[ ${HC} == "" ]];then
    echo "Deploying HC Cluster: ${HC_CLUSTER_NAME} in ${HC_CLUSTER_NS}
namespace"
    oc apply -f ${HC_NEW_FILE}
else
    echo "HC Cluster ${HC_CLUSTER_NAME} already exists, avoiding step"
fi

```

7. If you are recovering the nodes and the node pool to reuse AWS instances, restore the node pool by entering this command:

```
$ oc apply -f ${BACKUP_DIR}/namespaces/${HC_CLUSTER_NS}/np-*
```

Verification

- To verify that the nodes are fully restored, use this function:

```

timeout=40
count=0
NODE_STATUS=$(oc get nodes --kubeconfig=${HC_KUBECONFIG} | grep -v NotReady |
grep -c "worker") || NODE_STATUS=0

while [ ${NODE_POOL_REPLICAS} != ${NODE_STATUS} ]
do
    echo "Waiting for Nodes to be Ready in the destination MGMT Cluster:
${MGMT2_CLUSTER_NAME}"
    echo "Try: (${count}/${timeout})"
    sleep 30
    if [[ $count -eq timeout ]];then

```

```

    echo "Timeout waiting for Nodes in the destination MGMT Cluster"
    exit 1
fi
count=$((count+1))
NODE_STATUS=$(oc get nodes --kubeconfig=${HC_KUBECONFIG} | grep -v NotReady |
grep -c "worker") || NODE_STATUS=0
done

```

Next steps

Shut down and delete your cluster.

9.6.4. Deleting a hosted cluster from your source management cluster

After you back up your hosted cluster and restore it to your destination management cluster, you shut down and delete the hosted cluster on your source management cluster.

Prerequisites

- You backed up your data and restored it to your source management cluster.

TIP

Ensure that the **kubeconfig** file of the destination management cluster is placed as it is set in the **KUBECONFIG** variable or, if you use the script, in the **MGMT_KUBECONFIG** variable. Use **export KUBECONFIG=<Kubeconfig FilePath>** or, if you use the script, use **export KUBECONFIG=\${MGMT_KUBECONFIG}**.

Procedure

1. Scale the **deployment** and **statefulset** objects by entering these commands:



IMPORTANT

Do not scale the stateful set if the value of its **spec.persistentVolumeClaimRetentionPolicy.whenScaled** field is set to **Delete**, because this could lead to a loss of data.

As a workaround, update the value of the **spec.persistentVolumeClaimRetentionPolicy.whenScaled** field to **Retain**. Ensure that no controllers exist that reconcile the stateful set and would return the value back to **Delete**, which could lead to a loss of data.

```
$ export KUBECONFIG=${MGMT_KUBECONFIG}
```

Scale down deployment commands

```
$ oc scale deployment -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --replicas=0 --all
```

```
$ oc scale statefulset.apps -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --replicas=0 -all
```

```
$ sleep 15
```

2. Delete the **NodePool** objects by entering these commands:

```
NODEPOOLS=$(oc get nodepools -n ${HC_CLUSTER_NS} -o=jsonpath='{.items[?
(@.spec.clusterName=="${HC_CLUSTER_NAME}").metadata.name]}')
if [[ ! -z "${NODEPOOLS}" ]];then
  oc patch -n "${HC_CLUSTER_NS}" nodepool ${NODEPOOLS} --type=json --patch='[ {
"op":"remove", "path": "/metadata/finalizers" }]'
  oc delete np -n ${HC_CLUSTER_NS} ${NODEPOOLS}
fi
```

3. Delete the **machine** and **machineset** objects by entering these commands:

```
# Machines
for m in $(oc get machines -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -o name); do
  oc patch -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} ${m} --type=json --patch='[ {
"op":"remove", "path": "/metadata/finalizers" }]' || true
  oc delete -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} ${m} || true
done
```

```
$ oc delete machineset -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --all || true
```

4. Delete the cluster object by entering these commands:

```
$ C_NAME=$(oc get cluster -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} -o name)
```

```
$ oc patch -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} ${C_NAME} --type=json --
patch='[ { "op":"remove", "path": "/metadata/finalizers" }]'
```

```
$ oc delete cluster.cluster.x-k8s.io -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} --all
```

5. Delete the AWS machines (Kubernetes objects) by entering these commands. Do not worry about deleting the real AWS machines. The cloud instances will not be affected.

```
for m in $(oc get awsmachine.infrastructure.cluster.x-k8s.io -n ${HC_CLUSTER_NS}-
${HC_CLUSTER_NAME} -o name)
do
  oc patch -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} ${m} --type=json --patch='[ {
"op":"remove", "path": "/metadata/finalizers" }]' || true
  oc delete -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} ${m} || true
done
```

6. Delete the **HostedControlPlane** and **ControlPlane** HC namespace objects by entering these commands:

Delete HostedControlPlane and ControlPlane HC NS commands

```
$ oc patch -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}
hostedcontrolplane.hypershift.openshift.io ${HC_CLUSTER_NAME} --type=json --patch='[ {
"op":"remove", "path": "/metadata/finalizers" }]'
```

```
$ oc delete hostedcontrolplane.hypershift.openshift.io -n ${HC_CLUSTER_NS}-  
${HC_CLUSTER_NAME} --all
```

```
$ oc delete ns ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME} || true
```

7. Delete the **HostedCluster** and HC namespace objects by entering these commands:

Delete HC and HC Namespace commands

```
$ oc -n ${HC_CLUSTER_NS} patch hostedclusters ${HC_CLUSTER_NAME} -p  
'{"metadata":{"finalizers":null}}' --type merge || true
```

```
$ oc delete hc -n ${HC_CLUSTER_NS} ${HC_CLUSTER_NAME} || true
```

```
$ oc delete ns ${HC_CLUSTER_NS} || true
```

Verification

- To verify that everything works, enter these commands:

Validations commands

```
$ export KUBECONFIG=${MGMT2_KUBECONFIG}
```

```
$ oc get hc -n ${HC_CLUSTER_NS}
```

```
$ oc get np -n ${HC_CLUSTER_NS}
```

```
$ oc get pod -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}
```

```
$ oc get machines -n ${HC_CLUSTER_NS}-${HC_CLUSTER_NAME}
```

Commands for inside the HostedCluster

```
$ export KUBECONFIG=${HC_KUBECONFIG}
```

```
$ oc get clusterversion
```

```
$ oc get nodes
```

Next steps

Delete the OVN pods in the hosted cluster so that you can connect to the new OVN control plane that runs in the new management cluster:

1. Load the **KUBECONFIG** environment variable with the hosted cluster's **kubeconfig** path.
2. Enter this command:

```
$ oc delete pod -n openshift-ovn-kubernetes --all
```

9.7. DISASTER RECOVERY FOR A HOSTED CLUSTER BY USING OADP

By using the OpenShift API for Data Protection (OADP) Operator for disaster recovery, you can restore hosted cluster namespaces from object storage instead of manually rebuilding every cluster. In addition, you back up etcd as part of the control plane backup, and you can back up hosted clusters independently.

You can use the OADP Operator to perform disaster recovery for hosted control planes on Amazon Web Services (AWS) and bare metal.

The disaster recovery process with OpenShift API for Data Protection (OADP) involves the following steps:

1. Preparing your platform, such as Amazon Web Services or bare metal, to use OADP
2. Backing up the data plane workload
3. Backing up the control plane workload
4. Restoring a hosted cluster by using OADP

9.7.1. Preparing AWS to use OADP for disaster recovery

Before you can perform disaster recovery for hosted control planes on Amazon Web Services (AWS), you need to meet a few prerequisites and configure OpenShift API for Data Protection (OADP) on AWS S3 compatible storage.

Prerequisites

- You installed the OADP Operator on the management cluster. For more information, see "About installing OADP".
- You created a storage class for the management cluster.
- You have access to the management cluster with **cluster-admin** privileges.
- You have access to the OADP subscription through a catalog source.
- You have access to a cloud storage provider that is compatible with OADP, such as S3, Microsoft Azure, Google Cloud, or MinIO.
- In a disconnected environment, you have access to a self-hosted storage provider that is compatible with OADP, such as [Red Hat OpenShift Data Foundation](#) or [MinIO](#).
- Your hosted control planes pods are up and running.
- You are using a supported version of OADP for your management cluster. For example, if your management cluster is on OpenShift Container Platform 4.20, you must use OADP version 1.5. For more information, see "Support for OpenShift API for Data Protection (OADP)".

Procedure

- To prepare AWS to use OADP, follow the steps in "Configuring the OpenShift API for Data Protection with AWS S3 compatible storage".
After you create the **DataProtectionApplication** object, new **velero** deployment and **node-agent** pods are created in the **openshift-adp** namespace.

Next steps

- Back up the data plane workload and the control plane workload.

Additional resources

- [About installing OADP](#)
- [Support for OpenShift API for Data Protection \(OADP\)](#)
- [Configuring the OpenShift API for Data Protection with AWS S3 compatible storage](#)

9.7.2. Preparing bare metal to use OADP for disaster recovery

Before you can perform disaster recovery for hosted control planes on bare metal, you need to meet a few prerequisites and configure the OpenShift API for Data Protection (OADP) with Multicloud Object Gateway.

Prerequisites

- You installed the OADP Operator on the management cluster. For more information, see "About installing OADP".
- You created a storage class for the management cluster.
- You have access to the management cluster with **cluster-admin** privileges.
- You have access to the OADP subscription through a catalog source.
- You have access to a cloud storage provider that is compatible with OADP, such as S3, Microsoft Azure, Google Cloud, or MinIO.
- In a disconnected environment, you have access to a self-hosted storage provider that is compatible with OADP, such as [Red Hat OpenShift Data Foundation](#) or [MinIO](#).
- Your hosted control planes pods are up and running.
- You are using a supported version of OADP for your management cluster. For example, if your management cluster is on OpenShift Container Platform 4.20, you must use OADP version 1.5. For more information, see "Support for OpenShift API for Data Protection (OADP)".

Procedure

- To prepare bare metal to use OADP, complete the steps in "Configuring the OpenShift API for Data Protection with Multicloud Object Gateway".
After you create the **DataProtectionApplication** object, new **velero** deployment and **node-agent** pods are created in the **openshift-adp** namespace.

Next steps

- Back up the data plane workload and the control plane workload.

Additional resources

- [About installing OADP](#)
- [Support for OpenShift API for Data Protection \(OADP\)](#)
- [Configuring the OpenShift API for Data Protection with Multicloud Object Gateway](#)

9.7.3. Data plane workload backup

As part of the process to back up and restore by using the OADP Operator, you can back up the data plane workload.

If the data plane workload is not important, you can skip this procedure.

To back up the data plane workload by using the OADP Operator, see "Backing up applications". After you complete those steps, you can restore your hosted cluster by using OADP.

Additional resources

- [Backing up applications](#)

9.7.4. Control plane workload backup

You can back up the control plane workload by creating the **Backup** custom resource (CR).

The steps vary depending on whether your platform is AWS or bare metal.

9.7.4.1. Backing up the control plane workload on AWS

You can back up the control plane workload by creating the **Backup** custom resource (CR).

For information about monitoring the backup process, see "Observing the backup and restore process".

Procedure

1. Pause the reconciliation of the **HostedCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  patch hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
  --type json -p '[{"op": "add", "path": "/spec/pausedUntil", "value": "true"}]'
```

2. Get the infrastructure ID of your hosted cluster by running the following command:

```
$ oc get hostedcluster -n local-cluster <hosted_cluster_name> -o=jsonpath="{.spec.infraID}"
```

Note the infrastructure ID to use in the next step.

3. Pause the reconciliation of the **cluster.cluster.x-k8s.io** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
```



```
patch cluster.cluster.x-k8s.io \
-n local-cluster-<hosted_cluster_name> <hosted_cluster_infra_id> \
--type json -p ' [{"op": "add", "path": "/spec/paused", "value": true}]'
```

4. Pause the reconciliation of the **NodePool** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
patch nodepool -n <hosted_cluster_namespace> <node_pool_name> \
--type json -p ' [{"op": "add", "path": "/spec/pausedUntil", "value": "true"}]'
```

5. Pause the reconciliation of the **AgentCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
annotate agentcluster -n <hosted_control_plane_namespace> \
cluster.x-k8s.io/paused=true --all'
```

6. Pause the reconciliation of the **AgentMachine** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
annotate agentmachine -n <hosted_control_plane_namespace> \
cluster.x-k8s.io/paused=true --all'
```

7. Annotate the **HostedCluster** resource to prevent the deletion of the hosted control plane namespace by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
annotate hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
hypershift.openshift.io/skip-delete-hosted-controlplane-namespace=true
```

8. Create a YAML file that defines the **Backup** CR:

Example backup-control-plane.yaml file

```
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: <backup_resource_name>
  namespace: openshift-adp
  labels:
    velero.io/storage-location: default
spec:
  hooks: {}
  includedNamespaces:
    - <hosted_cluster_namespace>
    - <hosted_control_plane_namespace>
  includedResources:
    - sa
    - role
    - rolebinding
    - pod
    - pvc
    - pv
    - bmh
    - configmap
```

```

- infraenv
- priorityclasses
- pdb
- agents
- hostedcluster
- nodepool
- secrets
- hostedcontrolplane
- cluster
- agentcluster
- agentmachinetemplate
- agentmachine
- machinedeployment
- machineset
- machine
excludedResources: []
storageLocation: default
ttl: 2h0m0s
snapshotMoveData: true
datamover: "velero"
defaultVolumesToFsBackup: true

```

- **metadata.name** specifies the name of your **Backup** resource.
- **spec.includedNamespaces** includes specific namespaces to back up objects from them. You must replace **<hosted_cluster_namespace>** with the name of the hosted cluster namespace and replace **<hosted_control_plane_namespace>** with the name of the hosted control plane namespace.
- **spec.includedResources** includes the **infraenv** resource. You must create the **infraenv** resource in a separate namespace. Do not delete the **infraenv** resource during the backup process.
- **spec.snapshotMoveData** and **spec.datamover** enable the CSI volume snapshots and upload the control plane workload automatically to the cloud storage.
- **spec.defaultVolumesToFsBackup** sets the **fs-backup** backing up method for persistent volumes (PVs) as default. This setting is useful when you use a combination of Container Storage Interface (CSI) volume snapshots and the **fs-backup** method.



NOTE

If you want to use CSI volume snapshots, you must add the **backup.velero.io/backup-volumes-excludes=<pv_name>** annotation to your PVs.

9. Apply the **Backup** CR by running the following command:

```
$ oc apply -f backup-control-plane.yaml
```

Verification

- Verify if the value of the **status.phase** is **Completed** by running the following command:

```
$ oc get backups.velero.io <backup_resource_name> -n openshift-adp \
-o jsonpath='{.status.phase}'
```

Next steps

- Restoring a hosted cluster by using OADP

9.7.4.2. Backing up the control plane workload on a bare metal

You can back up the control plane workload by creating the **Backup** custom resource (CR).

For more information about monitoring the backup process, see "Observing the backup and restore process".

Procedure

1. Pause the reconciliation of the **HostedCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  patch hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
  --type json -p '[{"op": "add", "path": "/spec/pausedUntil", "value": "true"}]'
```

2. Get the infrastructure ID of your hosted cluster by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  get hostedcluster -n <hosted_cluster_namespace> \
  <hosted_cluster_name> -o=jsonpath='{.spec.infraID}'
```

3. Note the infrastructure ID to use in the next step.
4. Pause the reconciliation of the **cluster.cluster.x-k8s.io** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  annotate cluster -n <hosted_control_plane_namespace> \
  <hosted_cluster_infra_id> cluster.x-k8s.io/paused=true
```

5. Pause the reconciliation of the **NodePool** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  patch nodepool -n <hosted_cluster_namespace> <node_pool_name> \
  --type json -p '[{"op": "add", "path": "/spec/pausedUntil", "value": "true"}]'
```

6. Pause the reconciliation of the **AgentCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  annotate agentcluster -n <hosted_control_plane_namespace> \
  cluster.x-k8s.io/paused=true --all
```

7. Pause the reconciliation of the **AgentMachine** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  annotate agentmachine -n <hosted_control_plane_namespace> \
  cluster.x-k8s.io/paused=true --all
```

8. If you are backing up and restoring to the same management cluster, annotate the **HostedCluster** resource to prevent the deletion of the hosted control plane namespace by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  annotate hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
  hypershift.openshift.io/skip-delete-hosted-controlplane-namespace=true
```

9. Create a YAML file that defines the **Backup** CR:

Example backup-control-plane.yaml file

```
apiVersion: velero.io/v1
kind: Backup
metadata:
  name: <backup_resource_name>
  namespace: openshift-adp
  labels:
    velero.io/storage-location: default
spec:
  hooks: {}
  includedNamespaces:
    - <hosted_cluster_namespace>
    - <hosted_control_plane_namespace>
    - <agent_namespace>
  includedResources:
    - sa
    - role
    - rolebinding
    - pod
    - pvc
    - pv
    - bmh
    - configmap
    - infraenv
    - priorityclasses
    - pdb
    - agents
    - hostedcluster
    - nodepool
    - secrets
    - services
    - deployments
    - hostedcontrolplane
    - cluster
    - agentcluster
    - agentmachinetemplate
    - agentmachine
    - machinedeployment
    - machineset
    - machine
```

```

excludedResources: []
storageLocation: default
ttl: 2h0m0s
snapshotMoveData: true
datamover: "velero"
defaultVolumesToFsBackup: true

```

- **metadata.name** specifies the name of your **Backup** resource.
- **spec.includedNamespaces** specifies namespaces to back up objects from them. Replace **<hosted_cluster_namespace>** with the name of the hosted cluster namespace, replace **<hosted_control_plane_namespace>** with the name of the hosted control plane namespace, and replace **<agent_namespace>** with the namespace where your **Agent**, **BMH**, and **InfraEnv** CRs are located.
- **spec.snapshotMoveData** enable the CSI volume snapshots and upload the control plane workload automatically to the cloud storage.
- **spec.defaultVolumesToFsBackup** sets the **fs-backup** backing up method for persistent volumes (PVs) as default. This setting is useful when you use a combination of Container Storage Interface (CSI) volume snapshots and the **fs-backup** method.



NOTE

If you want to use CSI volume snapshots, you must add the **backup.velero.io/backup-volumes-excludes=<pv_name>** annotation to your PVs.

10. Apply the **Backup** CR by running the following command:

```
$ oc apply -f backup-control-plane.yaml
```

Verification

- Verify if the value of the **status.phase** is **Completed** by running the following command:

```
$ oc get backups.velero.io <backup_resource_name> -n openshift-adp \
-o jsonpath='{.status.phase}'
```

Next steps

- Restore a hosted cluster by using OADP.

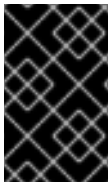
9.7.5. Hosted cluster restoration by using OADP

You can restore a hosted cluster into the same management cluster or into a new management cluster.

9.7.5.1. Restoring a hosted cluster into the same management cluster by using OADP

You can restore the hosted cluster by creating the **Restore** custom resource (CR).

- If you are using an *in-place* update, the **InfraEnv** resource does not need spare nodes. You need to re-provision the worker nodes from the new management cluster.
- If you are using a *replace* update, you need some spare nodes for the **InfraEnv** resource to deploy the worker nodes.



IMPORTANT

After you back up your hosted cluster, you must delete it to start the restoring process. To start node provisioning, you must back up workloads in the data plane before deleting the hosted cluster.

Prerequisites

- You completed the steps in [Removing a cluster by using the console](#) to delete your hosted cluster.
- You completed the steps in [Removing remaining resources after removing a cluster](#).

To monitor and observe the backup process, see "Observing the backup and restore process".

Procedure

1. Verify that no pods and persistent volume claims (PVCs) are present in the hosted control plane namespace by running the following command:

```
$ oc get pod pvc -n <hosted_control_plane_namespace>
```

Expected output

```
No resources found
```

2. Create a YAML file that defines the **Restore** CR:

Example restore-hosted-cluster.yaml file

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: <restore_resource_name>
  namespace: openshift-adp
spec:
  backupName: <backup_resource_name>
  restorePVs: true
  existingResourcePolicy: update
  excludedResources:
    - nodes
    - events
    - events.events.k8s.io
    - backups.velero.io
    - restores.velero.io
    - resticrepositories.velero.io
```

- **metadata.name** specifies the name of your **Restore** resource.

- **spec.backupName** specifies the name of your **Backup** resource.
- **spec.restorePVs: true** starts the recovery of persistent volumes (PVs) and its pods.
- **spec.existingResourcePolicy: update** ensures that the existing objects are overwritten with the backed up content.



IMPORTANT

You must create the **infraenv** resource in a separate namespace. Do not delete the **infraenv** resource during the restore process. The **infraenv** resource is mandatory for the new nodes to be reprovisioned.

3. Apply the **Restore** CR by running the following command:

```
$ oc apply -f restore-hosted-cluster.yaml
```

4. Verify if the value of the **status.phase** is **Completed** by running the following command:

```
$ oc get hostedcluster <hosted_cluster_name> -n <hosted_cluster_namespace> \
-o jsonpath='{.status.phase}'
```

5. After the restore process is complete, start the reconciliation of the **HostedCluster** and **NodePool** resources that you paused during backing up of the control plane workload:

- a. Start the reconciliation of the **HostedCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
patch hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
--type json \
-p ' [{"op": "add", "path": "/spec/pausedUntil", "value": "false"} ]'
```

- b. Start the reconciliation of the **NodePool** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
patch nodepool -n <hosted_cluster_namespace> <node_pool_name> \
--type json \
-p ' [{"op": "add", "path": "/spec/pausedUntil", "value": "false"} ]'
```

6. Start the reconciliation of the Agent provider resources that you paused during backing up of the control plane workload:

- a. Start the reconciliation of the **AgentCluster** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
annotate agentcluster -n <hosted_control_plane_namespace> \
cluster.x-k8s.io/paused- --overwrite=true --all
```

- b. Start the reconciliation of the **AgentMachine** resource by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
annotate agentmachine -n <hosted_control_plane_namespace> \
cluster.x-k8s.io/paused- --overwrite=true --all
```

7. Remove the **hypershift.openshift.io/skip-delete-hosted-controlplane-namespace-** annotation in the **HostedCluster** resource to avoid manually deleting the hosted control plane namespace by running the following command:

```
$ oc --kubeconfig <management_cluster_kubeconfig_file> \
  annotate hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \
  hypershift.openshift.io/skip-delete-hosted-controlplane-namespace- \
  --overwrite=true --all
```

9.7.5.2. Restoring a hosted cluster into a new management cluster by using OADP

You can restore the hosted cluster into a new management cluster by creating the **Restore** custom resource (CR).

- If you are using an in-place update, the **InfraEnv** resource does not need spare nodes. Instead, you need to re-provision the worker nodes from the new management cluster.
- If you are using a replace update, you need some spare nodes for the **InfraEnv** resource to deploy the worker nodes.

Prerequisites

- You configured the new management cluster to use OpenShift API for Data Protection (OADP). The new management cluster must have the same Data Protection Application (DPA) as the management cluster that you backed up from so that the **Restore** CR can access the backup storage.
- You configured the networking settings of the new management cluster to resolve the DNS of the hosted cluster.
 - The DNS of the host must resolve to the IP of both the new management cluster and the hosted cluster.
 - The hosted cluster must resolve to the IP of the new management cluster.

To monitor and observe the backup process, see "Observing the backup and restore process".



IMPORTANT

Complete the following steps on the new management cluster that you are restoring the hosted cluster to, not on the management cluster that you created the backup from.

Procedure

1. Create a YAML file that defines the **Restore** CR:

Example restore-hosted-cluster.yaml file

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: <restore_resource_name>
  namespace: openshift-adp
```



```
spec:
  includedNamespaces:
    - <hosted_cluster_namespace>
    - <hosted_control_plane_namespace>
    - <agent_namespace>
  backupName: <backup_resource_name>
  cleanupBeforeRestore: CleanupRestored
  veleroManagedClustersBackupName: <managed_cluster_name>
  veleroCredentialsBackupName: <credentials_backup_name>
  veleroResourcesBackupName: <resources_backup_name>
  restorePVs: true
  preserveNodePorts: true
  existingResourcePolicy: update
  excludedResources:
    - pod
    - nodes
    - events
    - events.events.k8s.io
    - backups.velero.io
    - restores.velero.io
    - resticrepositories.velero.io
    - pv
    - pvc
```

- **metadata.name** specifies the name of your **Restore** resource.
- **spec.includedNamespaces** specifies namespaces to back up objects from them. Replace **<hosted_cluster_namespace>** with the name of the hosted cluster namespace, replace **<hosted_control_plane_namespace>** with the name of the hosted control plane namespace, and replace **<agent_namespace>** with the namespace where your **Agent**, **BMH**, and **InfraEnv** CRs are located.
- **spec.backupName** specifies the name of your **Backup** resource.
- **spec.veleroManagedClustersBackupName** can be omitted if you are not using Red Hat Advanced Cluster Management.
- **spec.restorePVs: true** starts the recovery of persistent volumes (PVs) and its pods.
- **spec.existingResourcePolicy: update** ensures that the existing objects are overwritten with the backed up content.

2. Apply the **Restore** CR by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> apply -f restore-hosted-cluster.yaml
```

3. Verify that the value of the **status.phase** is **Completed** by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \
  get restore.velero.io <restore_resource_name> \
  -n openshift-adp -o jsonpath='{.status.phase}'
```

4. Verify that all CRs are restored by running the following commands:

```
$ oc --kubeconfig <restore_management_kubeconfig> get infraenv -n <agent_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get agent -n <agent_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get bmh -n <agent_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get hostedcluster -n  
<hosted_cluster_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get nodepool -n  
<hosted_cluster_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get agentmachine -n  
<hosted_controlplane_namespace>
```

```
$ oc --kubeconfig <restore_management_kubeconfig> get agentcluster -n  
<hosted_controlplane_namespace>
```

5. If you plan to use the new management cluster as your main management cluster going forward, complete the following steps. Otherwise, if you plan to use the management cluster that you backed up from as your main management cluster, complete steps 5 - 8 in "Restoring a hosted cluster into the same management cluster by using OADP".

- a. Remove the Cluster API deployment from the management cluster that you backed up from by running the following command:

```
$ oc --kubeconfig <backup_management_kubeconfig> delete deploy cluster-api \  
-n <hosted_control_plane_namespace>
```

Because only one Cluster API can access a cluster at a time, this step ensures that the Cluster API for the new management cluster functions correctly.

- b. After the restore process is complete, start the reconciliation of the **HostedCluster** and **NodePool** resources that you paused during backing up of the control plane workload:

- i. Start the reconciliation of the **HostedCluster** resource by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \  
patch hostedcluster -n <hosted_cluster_namespace> <hosted_cluster_name> \  
--type json \  
-p '[{"op": "replace", "path": "/spec/pausedUntil", "value": "false"}]'
```

- ii. Start the reconciliation of the **NodePool** resource by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \  
patch nodepool -n <hosted_cluster_namespace> <node_pool_name> \  
--type json \  
-p '[{"op": "replace", "path": "/spec/pausedUntil", "value": "false"}]'
```

- iii. Verify that the hosted cluster is reporting that the hosted control plane is available by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> get hostedcluster
```

- iv. Verify that the hosted cluster is reporting that the cluster operators are available by running the following command:

```
$ oc get co --kubeconfig <hosted_cluster_kubeconfig>
```

- c. Start the reconciliation of the Agent provider resources that you paused during backing up of the control plane workload:

- i. Start the reconciliation of the **AgentCluster** resource by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \
  annotate agentcluster -n <hosted_control_plane_namespace> \
  cluster.x-k8s.io/paused- --overwrite=true --all
```

- ii. Start the reconciliation of the **AgentMachine** resource by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \
  annotate agentmachine -n <hosted_control_plane_namespace> \
  cluster.x-k8s.io/paused- --overwrite=true --all
```

- iii. Start the reconciliation of the **Cluster** resource by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \
  annotate cluster -n <hosted_control_plane_namespace> \
  cluster.x-k8s.io/paused- --overwrite=true --all
```

- d. Verify that the node pool is working as expected by running the following command:

```
$ oc --kubeconfig <restore_management_kubeconfig> \
  get nodepool -n <hosted_cluster_namespace>
```

Example output

```
NAME      CLUSTER  DESIRED NODES  CURRENT NODES  AUTOSCALING
AUTOREPAIR  VERSION  UPDATINGVERSION  UPDATINGCONFIG  MESSAGE
hosted-0  hosted-0  3              3              False          False          4.17.11  False
False
```

- e. Optional: To ensure that no conflicts exist and that the new management cluster has continued functionality, remove the **HostedCluster** resources from the backup management cluster by completing the following steps:

- i. In the management cluster that you backed up from, in the **ClusterDeployment** resource, set the **spec.preserveOnDelete** parameter to **true** by running the following command:

```
$ oc --kubeconfig <backup_management_kubeconfig> patch \
  -n <hosted_control_plane_namespace> \
  ClusterDeployment/<hosted_cluster_name> -p \
```

```
'{"spec":{"preserveOnDelete":true}}' \
--type=merge
```

This step ensures that the hosts are not deprovisioned.

- ii. Delete the machines by running the following commands:

```
$ oc --kubeconfig <backup_management_kubeconfig> patch \
<machine_name> -n <hosted_control_plane_namespace> -p \
'{"op":"remove","path":"/metadata/finalizers"}' \
--type=merge
```

```
$ oc --kubeconfig <backup_management_kubeconfig> \
delete machine <machine_name> \
-n <hosted_control_plane_namespace>
```

- iii. Delete the **AgentCluster** and **Cluster** resources by running the following commands:

```
$ oc --kubeconfig <backup_management_kubeconfig> \
delete agentcluster <hosted_cluster_name> \
-n <hosted_control_plane_namespace>
```

```
$ oc --kubeconfig <backup_management_kubeconfig> \
patch cluster <cluster_name> \
-n <hosted_control_plane_namespace> \
-p '{"op":"remove","path":"/metadata/finalizers"}' \
--type=json
```

```
$ oc --kubeconfig <backup_management_kubeconfig> \
delete cluster <cluster_name> \
-n <hosted_control_plane_namespace>
```

- iv. If you use Red Hat Advanced Cluster Management, delete the managed cluster by running the following commands:

```
$ oc --kubeconfig <backup_management_kubeconfig> \
patch managedcluster <hosted_cluster_name> \
-n <hosted_cluster_namespace> \
-p '{"op":"remove","path":"/metadata/finalizers"}' \
--type=json
```

```
$ oc --kubeconfig <backup_management_kubeconfig> \
delete managedcluster <hosted_cluster_name> \
-n <hosted_cluster_namespace>
```

- v. Delete the **HostedCluster** resource by running the following command:

```
$ oc --kubeconfig <backup_management_kubeconfig> \
delete hostedcluster \
-n <hosted_cluster_namespace> <hosted_cluster_name>
```

- [Removing a cluster by using the console](#)
- [Removing remaining resources after removing a cluster](#)

9.7.6. Observing the backup and restore process

When you use OpenShift API for Data Protection (OADP) to back up and restore a hosted cluster, you can monitor and observe the process.

Procedure

1. Observe the backup process by running the following command:

```
$ watch "oc get backups.velero.io -n openshift-adp <backup_resource_name> -o
jsonpath='{.status}'"
```

2. Observe the restore process by running the following command:

```
$ watch "oc get restores.velero.io -n openshift-adp <backup_resource_name> -o
jsonpath='{.status}'"
```

3. Observe the Velero logs by running the following command:

```
$ oc logs -n openshift-adp -ldeploy=velero -f
```

4. Observe the progress of all of the OADP objects by running the following command:

```
$ watch "echo BackupRepositories;;echo;oc get backuprepositories.velero.io -A;echo; echo
BackupStorageLocations: ;echo; oc get backupstoragelocations.velero.io -A;echo;echo
DataUploads: ;echo;oc get datauploads.velero.io -A;echo;echo DataDownloads: ;echo;oc get
datadownloads.velero.io -n openshift-adp; echo;echo VolumeSnapshotLocations: ;echo;oc
get volumesnapshotlocations.velero.io -A;echo;echo Backups;;echo;oc get backup -A;
echo;echo Restores;;echo;oc get restore -A"
```

9.7.7. Using the Velero CLI to describe the Backup and Restore resources

When you use OpenShift API for Data Protection, you can get more details of the **Backup** and **Restore** resources by using the **velero** command-line interface (CLI).

Procedure

1. Create an alias to use the **velero** CLI from a container by running the following command:

```
$ alias velero='oc -n openshift-adp exec deployment/velero -c velero -it -- ./velero'
```

2. Get details of your **Restore** custom resource (CR) by running the following command:

```
$ velero restore describe <restore_resource_name> --details
```

Replace **<restore_resource_name>** with the name of your **Restore** resource.

3. Get details of your **Backup** CR by running the following command:

```
$ velero restore describe <backup_resource_name> --details
```

Replace **<backup_resource_name>** with the name of your **Backup** resource.

9.8. AUTOMATED DISASTER RECOVERY FOR A HOSTED CLUSTER BY USING OADP

In hosted clusters on bare-metal or Amazon Web Services (AWS) platforms, you can automate some backup and restore steps by using the OpenShift API for Data Protection (OADP) Operator.

The process involves the following steps:

1. Configuring OADP
2. Defining a Data Protection Application (DPA)
3. Backing up the data plane workload
4. Backing up the control plane workload
5. Restoring a hosted cluster by using OADP

9.8.1. Prerequisites to automate disaster recovery by using OADP

Ensure that you meet the prerequisites to automate disaster recovery for hosted control planes by using OADP.

The following prerequisites apply to the management cluster:

- You installed the OADP Operator. For more information, see "About installing OADP".
- You created a storage class.
- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OADP subscription through a catalog source.
- You have access to a cloud storage provider that is compatible with OADP, such as S3, Microsoft Azure, Google Cloud, or MinIO.
- In a disconnected environment, you have access to a self-hosted storage provider that is compatible with OADP, for example Red Hat OpenShift Data Foundation or MinIO.
- Your hosted control planes pods are up and running.
- You are using a supported version of OADP for your management cluster. For example, if your management cluster is on OpenShift Container Platform 4.20, you must use OADP version 1.5. For more information, see "Support for OpenShift API for Data Protection (OADP)".

Additional resources

- [About installing OADP](#)
- [Red Hat OpenShift Data Foundation](#)

- [MinIO](#)
- [Support for OpenShift API for Data Protection \(OADP\)](#)

9.8.2. Configuring OADP to automate disaster recovery for hosted control planes

Before you can automate disaster recovery by using OpenShift API for Data Protection (OADP), you need to configure it for your hosted control planes platform.

Procedure

- If your hosted cluster is on AWS, follow the steps in "Configuring the OpenShift API for Data Protection with AWS S3 compatible storage" to configure OADP.
- If your hosted cluster is on a bare metal, follow the steps in "Configuring the OpenShift API for Data Protection with Multicloud Object Gateway" to configure OADP.

Additional resources

- [Configuring the OpenShift API for Data Protection with AWS S3 compatible storage](#)
- [Configuring the OpenShift API for Data Protection with Multicloud Object Gateway](#)

9.8.3. Automation of the backup and restore process with a DPA

You can automate parts of the backup and restore process by using a Data Protection Application (DPA). When you use a DPA, the steps to pause and restart the reconciliation of resources are automated. The DPA defines information including backup locations and Velero pod configurations.

9.8.3.1. Creating a Data Protection Application for bare metal

Automate parts of the backup and restore process on bare metal by creating a Data Protection Application (DPA). A DPA defines information including backup locations and Velero pod configurations.

You can create a DPA by defining a **DataProtectionApplication** object.

Procedure

1. Create a manifest file similar to the following example:

```
apiVersion: oadp.openshift.io/v1alpha1
kind: DataProtectionApplication
metadata:
  name: dpa-sample
  namespace: openshift-adp
spec:
  backupLocations:
  - name: default
  velero:
    provider: aws
    default: true
    objectStorage:
      bucket: <bucket_name>
      prefix: <bucket_prefix>
    config:
```

```

    region: minio
    profile: "default"
    s3ForcePathStyle: "true"
    s3Url: "<bucket_url>"
    insecureSkipTLSVerify: "true"
  credential:
    key: cloud
    name: cloud-credentials
    default: true
  snapshotLocations:
    - velero:
        provider: aws
        config:
          region: minio
          profile: "default"
        credential:
          key: cloud
          name: cloud-credentials
  configuration:
    nodeAgent:
      enable: true
      uploaderType: kopia
  velero:
    defaultPlugins:
      - openshift
      - aws
      - csi
      - hypershift
    resourceTimeout: 2h

```

- **spec.backupLocations.velero.provider** specifies the provider for Velero. If you are using bare metal and MinIO, you can use **aws** as the provider.
- **spec.backupLocations.velero.objectStorage.bucket** specifies the bucket name; for example, **oadp-backup**.
- **spec.backupLocations.velero.objectStorage.prefix** specifies the bucket prefix; for example, **hcp**.
- **spec.backupLocations.velero.config.region** specifies the bucket region. In this example, the region is **minio**, which is a storage provider that is compatible with the S3 API.
- **spec.backupLocations.velero.config.s3Url** specifies the URL of the S3 endpoint.
- **spec.snapshotLocations.velero.provider** specifies the provider for Velero. If you are using bare metal and MinIO, you can use **aws** as the provider.
- **spec.snapshotLocations.velero.config.region** specifies the region. In this example, the region is **minio**, which is a storage provider that is compatible with the S3 API.
- **spec.configuration.nodeAgent.uploaderType** specifies **kopia** as the uploader type. The **restic** uploader type is deprecated for OADP 1.5 and later.

2. Create the DPA object by running the following command:

```
$ oc create -f dpa.yaml
```


■

After you create the **DataProtectionApplication** object, new **velero** deployment and **node-agent** pods are created in the **openshift-adp** namespace.

Next steps

- Back up the data plane workload.

9.8.3.2. Creating a Data Protection Application for AWS

Automate parts of the backup and restore process on AWS by creating a Data Protection Application (DPA). A DPA defines information including backup locations and Velero pod configurations.

You can create a DPA by defining a **DataProtectionApplication** object.

Procedure

1. Create a manifest file similar to the following example:

```
apiVersion: oadp.openshift.io/v1alpha1
kind: DataProtectionApplication
metadata:
  name: dpa-sample
  namespace: openshift-adp
spec:
  backupLocations:
  - name: default
    velero:
      provider: aws
      default: true
      objectStorage:
        bucket: <bucket_name>
        prefix: <bucket_prefix>
      config:
        region: minio
        profile: "backupStorage"
      credential:
        key: cloud
        name: cloud-credentials
  snapshotLocations:
  - velero:
      provider: aws
      config:
        region: minio
        profile: "volumeSnapshot"
      credential:
        key: cloud
        name: cloud-credentials
  configuration:
    nodeAgent:
      enable: true
      uploaderType: kopia
    velero:
      defaultPlugins:
      - openshift
```

```
- aws
- csi
- hypershift
resourceTimeout: 2h
```

- **spec.backupLocations.velero.objectStorage.bucket** specifies the bucket name; for example, **oadp-backup**.
- **spec.backupLocations.velero.objectStorage.prefix** specifies the bucket prefix; for example, **hcp**.
- **spec.backupLocations.velero.config.region** specifies the bucket region. The bucket region in this example is **minio**, which is a storage provider that is compatible with the S3 API.
- **spec.snapshotLocations.velero.config.region** specifies the region. The region in this example is **minio**, which is a storage provider that is compatible with the S3 API.
- **spec.configuration.nodeAgent.uploaderType** specifies **kopia** as the uploader type. The **restic** uploader type is deprecated for OADP 1.5 and later.

2. Create the DPA resource by running the following command:

```
$ oc create -f dpa.yaml
```

After you create the **DataProtectionApplication** object, new **velero** deployment and **node-agent** pods are created in the **openshift-adp** namespace.

Next steps

- Back up the data plane workload.

9.8.4. Backing up the data plane workload by using the OADP Operator

You can back up the data plane workload by using the OADP Operator.

However, if the data plane workload is not important, you can skip this procedure.

Procedure

- To back up the data plane workload, follow the steps in "Backing up applications".

Additional resources

- [Backing up applications](#)

9.8.5. Backing up the control plane workload

You can back up the control plane workload by creating the **Backup** custom resource (CR).

To monitor and observe the backup process, see "Observing the backup and restore process".

Procedure

1. Create a YAML file that defines the **Backup** CR:

```

apiVersion: velero.io/v1
kind: Backup
metadata:
  name: <backup_resource_name>
  namespace: openshift-adp
  labels:
    velero.io/storage-location: default
spec:
  hooks: {}
  includedNamespaces:
    - <hosted_cluster_namespace>
    - <hosted_control_plane_namespace>
  includedResources:
    - sa
    - role
    - rolebinding
    - pod
    - pvc
    - pv
    - bmh
    - configmap
    - infraenv
    - priorityclasses
    - pdb
    - agents
    - hostedcluster
    - nodepool
    - secrets
    - services
    - deployments
    - hostedcontrolplane
    - cluster
    - agentcluster
    - agentmachinetemplate
    - agentmachine
    - machinedeployment
    - machineset
    - machine
    - route
    - clusterdeployment
  excludedResources: []
  storageLocation: default
  ttl: 2h0m0s
  snapshotMoveData: true
  datamover: "velero"
  defaultVolumesToFsBackup: false

```

- **metadata.name** specifies the name for your **Backup** resource.
- **spec.includedNamespaces** specifies namespaces to back up objects from. You must replace **<hosted_cluster_namespace>** with the name of the hosted cluster namespace and replace **<hosted_control_plane_namespace>** with the name of the hosted control plane namespace.

- **spec.includedResources** includes the **infraenv** value. You must create the **infraenv** resource in a separate namespace. Do not delete the **infraenv** resource during the backup process.
- **spec.snapshotMoveData: true** and **spec.datamover: velero** enable the CSI volume snapshots and upload the control plane workload automatically to cloud storage.
- **spec.defaultVolumesToFsBackup** specifies that the **fs-backup** backing up method for persistent volumes (PVs) is not used.



NOTE

If you want to use CSI volume snapshots, you must add the **backup.velero.io/backup-volumes-excludes=<pv_name>** annotation to your PVs.

2. Apply the **Backup** CR by running the following command:

```
$ oc apply -f backup-control-plane.yaml
```

Verification

- Verify that the value of the **status.phase** is **Completed** by running the following command:

```
$ oc get backups.velero.io <backup_resource_name> -n openshift-adp \
-o jsonpath='{.status.phase}'
```

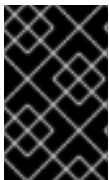
Next steps

- Restore the hosted cluster by using OADP.

9.8.6. Restoring a hosted cluster by using OADP

You can restore the hosted cluster by creating the **Restore** custom resource (CR).

- If you are using an in-place update, the **InfraEnv** resource does not need spare nodes. You need to re-provision the worker nodes from the new management cluster.
- If you are using a replace update, you need some spare nodes for the **InfraEnv** resource to deploy the worker nodes.



IMPORTANT

After you back up your hosted cluster, you must delete it to start the restoring process. To start node provisioning, you must back up workloads in the data plane before deleting the hosted cluster.

Prerequisites

- You completed the steps in [Removing a cluster by using the console](#) (RHACM documentation) to delete your hosted cluster.

- You completed the steps in [Removing remaining resources after removing a cluster](#) (RHACM documentation).

To monitor and observe the backup process, see "Observing the backup and restore process".

Procedure

1. Verify that no pods and persistent volume claims (PVCs) are present in the hosted control plane namespace by running the following command:

```
$ oc get pod pvc -n <hosted_control_plane_namespace>
```

Expected output

```
No resources found
```

2. Create a YAML file that defines the **Restore** CR:

```
apiVersion: velero.io/v1
kind: Restore
metadata:
  name: <restore_resource_name>
  namespace: openshift-adp
spec:
  backupName: <backup_resource_name>
  restorePVs: true
  existingResourcePolicy: update
  excludedResources:
    - nodes
    - events
    - events.events.k8s.io
    - backups.velero.io
    - restores.velero.io
    - resticrepositories.velero.io
```

- **metadata.name** specifies the name for your **Restore** resource.
- **spec.backupName** specifies the name of your **Backup** resource.
- **spec.restorePVs: true** indicates the recovery of persistent volumes (PVs) and their pods.
- **spec.existingResourcePolicy: update** ensures that the existing objects are overwritten with the backed up content.



IMPORTANT

You must create the **InfraEnv** resource in a separate namespace. Do not delete the **InfraEnv** resource during the restore process. The **InfraEnv** resource is mandatory for the new nodes to be reprovisioned.

3. Apply the **Restore** CR by running the following command:

```
$ oc apply -f restore-hosted-cluster.yaml
```

4. Verify if the value of the **status.phase** is **Completed** by running the following command:

```
$ oc get hostedcluster <hosted_cluster_name> -n <hosted_cluster_namespace> \
-o jsonpath='{.status.phase}'
```

9.8.7. Observing the backup and restore process

When you use OpenShift API for Data Protection (OADP) to back up and restore a hosted cluster, you can monitor and observe the process.

Procedure

1. Observe the backup process by running the following command:

```
$ watch "oc get backups.velero.io -n openshift-adp <backup_resource_name> -o
jsonpath='{.status}'"
```

2. Observe the restore process by running the following command:

```
$ watch "oc get restores.velero.io -n openshift-adp <backup_resource_name> -o
jsonpath='{.status}'"
```

3. Observe the Velero logs by running the following command:

```
$ oc logs -n openshift-adp -ldeploy=velero -f
```

4. Observe the progress of all of the OADP objects by running the following command:

```
$ watch "echo BackupRepositories::echo;oc get backuprepositories.velero.io -A;echo; echo
BackupStorageLocations: ;echo; oc get backupstoragelocations.velero.io -A;echo;echo
DataUploads: ;echo;oc get datauploads.velero.io -A;echo;echo DataDownloads: ;echo;oc get
datadownloads.velero.io -n openshift-adp; echo;echo VolumeSnapshotLocations: ;echo;oc
get volumesnapshotlocations.velero.io -A;echo;echo Backups::echo;oc get backup -A;
echo;echo Restores::echo;oc get restore -A"
```

9.8.8. Using the Velero CLI to describe the Backup and Restore resources

When you use OpenShift API for Data Protection, you can get more details of the **Backup** and **Restore** resources by using the **velero** command-line interface (CLI).

Procedure

1. Create an alias to use the **velero** CLI from a container by running the following command:

```
$ alias velero='oc -n openshift-adp exec deployment/velero -c velero -it -- ./velero'
```

2. Get details of your **Restore** custom resource (CR) by running the following command:

```
$ velero restore describe <restore_resource_name> --details
```

Replace **<restore_resource_name>** with the name of your **Restore** resource.

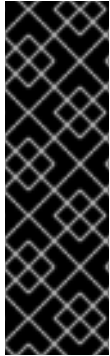
- Get details of your **Backup** CR by running the following command:

```
$ velero restore describe <backup_resource_name> --details
```

Replace **<backup_resource_name>** with the name of your **Backup** resource.

9.9. BACKING UP ETCD DATA FOR HOSTED CONTROL PLANES BY USING THE ETCD SNAPSHOT METHOD

To back up etcd data for hosted control planes, you can use the default volume snapshot approach, or you can take the etcd snapshot approach, which results in smaller backup artifacts.



IMPORTANT

The etcd snapshot method is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

9.9.1. Overview of the etcd snapshot method

You can use the etcd snapshot method as an alternative to the default volume snapshot approach to back up etcd data for hosted control planes. The etcd snapshot method results in smaller backup artifacts.

Instead of capturing persistent volume claim (PVC) content by using container storage interface (CSI) volume snapshots or a filesystem backup, you can take a snapshot of the etcd database and upload it to object storage.

The etcd snapshot approach is driven by the **HCPetcdBackup** custom resource and is orchestrated through the OpenShift API for Data Protection (OADP) HyperShift plugin during Velero backup operations.

For a more detailed comparison of the etcd snapshot method and the default volume snapshot method, see the following table.

Table 9.2. Comparing the etcd snapshot and volume snapshot methods

Aspect	Volume snapshot (default)	etcd snapshot
Backup mechanism	CSI volume snapshots or a Kopia filesystem backup of etcd PVCs. One per replica; typically 3.	The etcdctl snapshot save mechanism produces a single snapshot file and uploads it to object storage.
Portability	Tied to the storage provider and the CSI driver.	Storage-agnostic.

Aspect	Volume snapshot (default)	etcd snapshot
Backup size	Full PVC content. Highly available deployments have 3 PVCs.	A single etcd database snapshot.
Restore mechanism	PVC restore from snapshot.	etcdctl snapshot restore by using the init container.
Requirements	A CSI driver with snapshot support or a Kopia node agent.	The HCPetcdBackup feature gate must be enabled.
Encryption	Depends on the storage provider.	Optional Key Management Service (KMS) encryption for hosted control planes on AWS.

9.9.2. Configuring the etcd snapshot method

Before you can use the etcd snapshot method to back up your etcd data for hosted control planes, you must meet a few prerequisites and configure a plugin.

Prerequisites

- The **HCPetcdBackup** feature gate is enabled in the HyperShift Operator.
- The OpenShift API for Data Protection (OADP) Operator version 1.6 or later with the HyperShift plugin is deployed. For more information, see "Configuring OADP" and "Automating the backup and restore process by using a DPA".
- Object storage is configured. Use a Velero backup storage location that points to AWS S3.
- Service publishing requirements:
 - If you are restoring a hosted cluster to a different management cluster, use a fixed hostname that is configured through DNS so that you can update the DNS record to point to the endpoint of the new management cluster and make the migration transparent for existing nodes.
 - For production environments, all services must have fixed hostnames.
 - On AWS, the API server can also use a **Route** service publishing strategy with a fixed hostname.
- Platform-specific requirements:
 - For hosted control planes on AWS, ensure that the OIDC provider configuration is accessible for any fixes that are needed after the restore process. If you use AWS S3 for backup storage, ensure that IAM roles and policies are configured according to "About installing OADP".
 - For hosted control planes on bare metal with the Agent provider, ensure that the **InfraEnv** resource is in a separate namespace from the hosted control plane namespace. Be careful to not delete the **InfraEnv** resource during the backup or restore processes.

Procedure

1. Configure the OADP HyperShift plugin by creating a config map in the OADP namespace and specifying the etcd backup method, as shown in the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: hypershift-oadp-plugin-config
  namespace: openshift-adp
data:
  etcdBackupMethod: "etcdSnapshot"
```

where:

metadata.name

Specifies the name of the config map. Use **hypershift-oadp-plugin-config** as the name of the config map.

metadata.namespace

Specifies the OADP namespace.

data.etcdBackupMethod

Specifies the etcd backup method. The default is **volumeSnapshot**. Use **etcdSnapshot** to enable the etcd snapshot method. If the **etcdBackupMethod** parameter is set to **etcdSnapshot** but the **HCPetcdBackup** custom resource is not installed, the plugin fails.

2. Apply the configuration by entering the following command:

```
$ oc apply -f hypershift-oadp-plugin-config.yaml
```

Additional resources

- [Configuring OADP](#)
- [Automating the backup and restore process by using DPA](#)
- [About installing OADP](#)

9.9.3. Backing up etcd data by using the etcd snapshot method

You can start the etcd snapshot backup process from the hosted control planes command-line interface (CLI).

Prerequisite

- You completed the steps in "Configuring the etcd snapshot method".

Procedure

- Start the backup process by entering the following command:

```
$ hcp create oadp-backup \
  --hc-name <my_hosted_cluster> \
  --hc-namespace <my_hosted_cluster_namespace> \
```

```
--name <my_backup> \
--storage-location default \
--use-etcd-snapshot
```

The command generates an **oadp-backup** custom resource (CR) that includes the namespaces of the hosted cluster and the hosted control plane, a platform-aware resource list that excludes etcd-related resources, and snapshot settings.

Next, the OADP HyperShift plugin and the **HCPEtcdBackup** CR work to complete the backup process.

Additional resources

- [Configuring the etcd snapshot method](#)

9.9.4. Restoring etcd data by using the etcd snapshot method

You can specify a backup to recover from or set a schedule to run the recovery process on.

The process to recover a hosted control plane from an etcd snapshot backup involves the OADP HyperShift plugin, the Control Plane Operator, and the etcd init container.

Prerequisites

- You completed the steps in "Configuring the etcd snapshot method".
- No running pods or persistent volume claims (PVCs) are in the hosted control plane namespace. If you are restoring on the same management cluster, delete the hosted cluster and node pools first.
- The status of the Velero backup is **status.phase: Completed**.
- The OADP components are running and the **DataProtectionApplication** (DPA) custom resource (CR) is reconciled.
- For hosted control planes on AWS, your backup storage location (BSL) credentials are valid and have permission to read the snapshot from S3.
- For bare metal on the Agent platform, your **InfraEnv** objects are preserved. Do not delete them.

Procedure

- Start the restore process by entering the following command:

```
$ hcp create oadp-restore \
  --hc-name <my_hosted_cluster> \
  --hc-namespace <my_hosted_cluster_namespace> \
  --name <my_restore> \
  --from-backup <my_backup>
```

where:

<my_backup>

Specifies the name of the backup to use. If you run the restore process on a schedule, replace the **--from-backup** flag with the **--from-schedule** flag and specify the name of the schedule to use.

Verification

1. Check that the etcd pods are running and that the cluster is functioning as expected by entering the following command:

```
$ oc get pods -n <my_hosted_cluster_namespace> -l app=etcd
```

2. Check the restore conditions by entering the following command:

```
$ oc get hostedcluster <my_hosted_cluster> -n <my_hosted_cluster_namespace> -o jsonpath='{.status.conditions}' | jq '.[] | select(.type | test("Restore|Etcd"))'
```

3. On AWS deployments, when you restore to a different management cluster, the OIDC provider might need to be updated. Enter the following command:

```
$ hcp fix dr-oidc-iam --hc-name <my_hosted_cluster> --hc-namespace <my_hosted_cluster_namespace>
```

4. Confirm that the API server of the hosted cluster is accessible by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_kubeconfig> get nodes
```

5. Confirm that workloads are running by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_kubeconfig> get clusteroperators
```

Additional resources

- [Configuring the etcd snapshot method](#)

9.9.5. Condition codes for the etcd snapshot method

During the backup and restore process with the etcd snapshot method, you might see messages about the condition of hosted control planes resources.

Table 9.3. Explanations of conditions during etcd snapshot backup and restore processes

Resource	Condition, field, or annotation	Meaning
HCPetcdBackup	BackupCompleted	Tracks the backup lifecycle: InProgress , Succeeded , Failed , Rejected , or EtcdUnhealthy .
HostedControlPlane	EtcdSnapshotRestored	This value is set to True after etcd is restored from the snapshot.

Resource	Condition, field, or annotation	Meaning
HostedCluster	Status.LastSuccessfulEtcdBackupURL	Persists the last snapshot URL. After a successful backup, the HyperShift Operator sets this condition.
HostedCluster	etcd-snapshot-url	The OADP plugin inserts this annotation during the backup process. During the restore process, the plugin reads this annotation to set the RestoreSnapshotURL value.
HostedCluster	restored-from-backup	This annotation is set during the restore process. It is removed after the HostedClusterRestoredFromBackup condition is True .

CHAPTER 10. AUTHENTICATION AND AUTHORIZATION FOR HOSTED CONTROL PLANES

The OpenShift Container Platform control plane includes a built-in OAuth server. You can obtain OAuth access tokens to authenticate to the OpenShift Container Platform API. After you create your hosted cluster, you can configure OAuth by specifying an identity provider.

10.1. CONFIGURING THE OAUTH SERVER FOR A HOSTED CLUSTER BY USING THE CLI

You can configure the internal OAuth server for your hosted cluster by using the command-line interface (CLI).

You can configure OAuth for the following supported identity providers:

- **oidc**
- **htpasswd**
- **keystone**
- **ldap**
- **basic-authentication**
- **request-header**
- **github**
- **gitlab**
- **google**

Adding any identity provider in the OAuth configuration removes the default **kubeadmin** user provider.



NOTE

When you configure identity providers, you must configure at least one **NodePool** replica in your hosted cluster in advance. Traffic for DNS resolution is sent through the worker nodes. You do not need to configure the **NodePool** replicas in advance for the **htpasswd** and **request-header** identity providers.

Prerequisites

- You created your hosted cluster.

Procedure

1. Edit the **HostedCluster** custom resource (CR) on the management cluster by running the following command:

```
$ oc edit hostedcluster <hosted_cluster_name> -n <hosted_cluster_namespace>
```

2. Add the OAuth configuration in the **HostedCluster** CR by using the following example:

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  configuration:
    oauth:
      identityProviders:
        - openID:
            claims:
              email:
                - <email_address>
              name:
                - <display_name>
              preferredUsername:
                - <preferred_username>
            clientID: <client_id>
            clientSecret:
              name: <client_id_secret_name>
            issuer: https://example.com/identity
            mappingMethod: lookup
            name: IAM
            type: OpenID
```

- **metadata.name** specifies your hosted cluster name.
- **metadata.namespace** specifies your hosted cluster namespace.
- **spec.configuration.oauth.identityProviders.openID** is a provider name that is prefixed to the value of the identity claim to form an identity name. The provider name is also used to build the redirect URL.
- **spec.configuration.oauth.identityProviders.openID.claims.email** defines a list of attributes to use as the email address.
- **spec.configuration.oauth.identityProviders.openID.claims.name** defines a list of attributes to use as a display name.
- **spec.configuration.oauth.identityProviders.openID.claims.preferredUsername** defines a list of attributes to use as a preferred user name.
- **spec.configuration.oauth.identityProviders.openID.clientID** defines the ID of a client registered with the OpenID provider. You must allow the client to redirect to the **https://oauth-openshift.apps.<cluster_name>.<cluster_domain>/oauth2callback/<idp_provider_name>** URL.
- **spec.configuration.oauth.identityProviders.openID.clientSecret.name** defines a secret of a client registered with the OpenID provider.
- **spec.configuration.oauth.identityProviders.openID.issuer** specifies the Issuer Identifier described in the OpenID spec. You must use **https** without query or fragment component. For more information about Issuer Identifiers, see "Issuer Identifier".

- **spec.configuration.oauth.identityProviders.mappingMethod** defines a mapping method that controls how mappings are established between identities of this provider and **User** objects.

3. Save the file to apply the changes.

10.2. CONFIGURING THE OAUTH SERVER FOR A HOSTED CLUSTER BY USING THE WEB CONSOLE

You can configure the internal OAuth server for your hosted cluster by using the OpenShift Container Platform web console.

You can configure OAuth for the following supported identity providers:

- **oidc**
- **htpasswd**
- **keystone**
- **ldap**
- **basic-authentication**
- **request-header**
- **github**
- **gitlab**
- **google**

Adding any identity provider in the OAuth configuration removes the default **kubeadmin** user provider.



NOTE

When you configure identity providers, you must configure at least one **NodePool** replica in your hosted cluster in advance. Traffic for DNS resolution is sent through the worker nodes. You do not need to configure the **NodePool** replicas in advance for the **htpasswd** and **request-header** identity providers.

Prerequisites

- You logged in as a user with **cluster-admin** privileges.
- You created your hosted cluster.

Procedure

1. Go to **Home → API Explorer**.
2. Use the **Filter by kind** box to search for your **HostedCluster** resource.
3. Click the **HostedCluster** resource that you want to edit.

4. Click the **Instances** tab.

5. Click the Options menu  next to your hosted cluster name entry and click **Edit HostedCluster**.

6. Add the OAuth configuration in the YAML file:

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: HostedCluster
metadata:
  #...
spec:
  configuration:
    oauth:
      identityProviders:
        - openID:
            claims:
              email:
                - <email_address>
              name:
                - <display_name>
              preferredUsername:
                - <preferred_username>
            clientID: <client_id>
            clientSecret:
              name: <client_id_secret_name>
            issuer: https://example.com/identity
            mappingMethod: lookup
            name: IAM
            type: OpenID
```

- **spec.configuration.oauth.identityProviders.openID** specifies the provider name that is prefixed to the value of the identity claim to form an identity name. The provider name is also used to build the redirect URL.
- **spec.configuration.oauth.identityProviders.openID.claims.email** defines a list of attributes to use as the email address.
- **spec.configuration.oauth.identityProviders.openID.claims.name** defines a list of attributes to use as a display name.
- **spec.configuration.oauth.identityProviders.openID.claims.preferredUsername** defines a list of attributes to use as a preferred user name.
- **spec.configuration.oauth.identityProviders.openID.clientID** defines the ID of a client registered with the OpenID provider. You must allow the client to redirect to the **https://oauth-openshift.apps.<cluster_name>.<cluster_domain>/oauth2callback/<idp_provider_name>** URL.
- **spec.configuration.oauth.identityProviders.openID.clientSecret.name** defines a secret of a client registered with the OpenID provider.

- **spec.configuration.oauth.identityProviders.openID.issuer** specifies the Issuer Identifier described in the OpenID spec. You must use **https** without query or fragment component. For more information, see "Issuer Identifier".
- **spec.configuration.oauth.identityProviders.mappingMethod** defines a mapping method that controls how mappings are established between identities of this provider and **User** objects.

7. Click **Save**.

10.3. IAM ROLES ASSIGNED BY USING THE CCO IN A HOSTED CLUSTER ON AWS

You can assign components Identity and Access Management (IAM) roles that provide short-term, limited-privilege security credentials by using the Cloud Credential Operator (CCO) in hosted clusters on Amazon Web Services (AWS).

By default, the CCO runs in a hosted control plane.



NOTE

The CCO supports a manual mode only for hosted clusters on AWS. By default, hosted clusters are configured in a manual mode. The management cluster might use modes other than manual.

10.3.1. Enabling Operators to support CCO-based workflows with AWS STS

As an Operator author designing your project to run on Operator Lifecycle Manager (OLM), you can enable your Operator to authenticate against AWS on STS-enabled OpenShift Container Platform clusters by customizing your project to support the Cloud Credential Operator (CCO).

With this method, the Operator is responsible for and requires RBAC permissions for creating the **CredentialsRequest** object and reading the resulting **Secret** object.



NOTE

By default, pods related to the Operator deployment mount a **serviceAccountToken** volume so that the service account token can be referenced in the resulting **Secret** object.

Prerequisites

- OpenShift Container Platform 4.14 or later
- Cluster in STS mode
- OLM-based Operator project

Procedure

1. Update your Operator project's **ClusterServiceVersion** (CSV) object:
 - a. Ensure your Operator has RBAC permission to create **CredentialsRequests** objects:

Example clusterPermissions list

```
# ...
install:
  spec:
    clusterPermissions:
      - rules:
        - apiGroups:
          - "cloudcredential.openshift.io"
          resources:
            - credentialsrequests
          verbs:
            - create
            - delete
            - get
            - list
            - patch
            - update
            - watch
```

- b. Add the following annotation to claim support for this method of CCO-based workflow with AWS STS:

```
# ...
metadata:
  annotations:
    features.operators.openshift.io/token-auth-aws: "true"
```

2. Update your Operator project code:

- a. Get the role ARN from the environment variable set on the pod by the **Subscription** object. For example:

```
// Get ENV var
roleARN := os.Getenv("ROLEARN")
setupLog.Infof("getting role ARN", "role ARN = ", roleARN)
webIdentityTokenPath := "/var/run/secrets/openshift/serviceaccount/token"
```

- b. Ensure you have a **CredentialsRequest** object ready to be patched and applied. For example:

Example CredentialsRequest object creation

```
import (
  minterv1 "github.com/openshift/cloud-credential-operator/pkg/apis/cloudcredential/v1"
  corev1 "k8s.io/api/core/v1"
  metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

var in = minterv1.AWSPProviderSpec{
  StatementEntries: []minterv1.StatementEntry{
    {
      Action: []string{
        "s3:*",
```

```

    },
    Effect: "Allow",
    Resource: "arn:aws:s3:*:*:*:*",
  },
},
STSIAMRoleARN: "<role_arn>",
}

var codec = minterv1.Codec
var ProviderSpec, _ = codec.EncodeProviderSpec(in.DeepCopyObject())

const (
    name      = "<credential_request_name>"
    namespace = "<namespace_name>"
)

var CredentialsRequestTemplate = &minterv1.CredentialsRequest{
    ObjectMeta: metav1.ObjectMeta{
        Name:      name,
        Namespace: "openshift-cloud-credential-operator",
    },
    Spec: minterv1.CredentialsRequestSpec{
        ProviderSpec: ProviderSpec,
        SecretRef: corev1.ObjectReference{
            Name:      "<secret_name>",
            Namespace: namespace,
        },
        ServiceAccountNames: []string{
            "<service_account_name>",
        },
        CloudTokenPath: "",
    },
}

```

Alternatively, if you are starting from a **CredentialsRequest** object in YAML form (for example, as part of your Operator project code), you can handle it differently:

Example **CredentialsRequest** object creation in YAML form

```

// CredentialsRequest is a struct that represents a request for credentials
type CredentialsRequest struct {
    APIVersion string `yaml:"apiVersion"`
    Kind       string `yaml:"kind"`
    Metadata   struct {
        Name      string `yaml:"name"`
        Namespace string `yaml:"namespace"`
    } `yaml:"metadata"`
    Spec struct {
        SecretRef struct {
            Name      string `yaml:"name"`
            Namespace string `yaml:"namespace"`
        } `yaml:"secretRef"`
        ProviderSpec struct {
            APIVersion string `yaml:"apiVersion"`
            Kind       string `yaml:"kind"`
            StatementEntries []struct {

```

```

        Effect string `yaml:"effect"`
        Action []string `yaml:"action"`
        Resource string `yaml:"resource"`
    } `yaml:"statementEntries"`
    STSIAMRoleARN string `yaml:"stsIAMRoleARN"`
} `yaml:"providerSpec"`

// added new field
CloudTokenPath string `yaml:"cloudTokenPath"`
} `yaml:"spec"`
}

// ConsumeCredsRequestAddingTokenInfo is a function that takes a YAML filename and
// two strings as arguments
// It unmarshals the YAML file to a CredentialsRequest object and adds the token
// information.
func ConsumeCredsRequestAddingTokenInfo(fileName, tokenString, tokenPath string)
(*CredentialsRequest, error) {
    // open a file containing YAML form of a CredentialsRequest
    file, err := os.Open(fileName)
    if err != nil {
        return nil, err
    }
    defer file.Close()

    // create a new CredentialsRequest object
    cr := &CredentialsRequest{}

    // decode the yaml file to the object
    decoder := yaml.NewDecoder(file)
    err = decoder.Decode(cr)
    if err != nil {
        return nil, err
    }

    // assign the string to the existing field in the object
    cr.Spec.CloudTokenPath = tokenPath

    // return the modified object
    return cr, nil
}

```



NOTE

Adding a **CredentialsRequest** object to the Operator bundle is not currently supported.

- c. Add the role ARN and web identity token path to the credentials request and apply it during Operator initialization:

Example applying CredentialsRequest object during Operator initialization

```

// apply CredentialsRequest on install
credReq := credreq.CredentialsRequestTemplate
credReq.Spec.CloudTokenPath = webIdentityTokenPath

```

```

c := mgr.GetClient()
if err := c.Create(context.TODO(), credReq); err != nil {
    if errors.IsAlreadyExists(err) {
        setupLog.Error(err, "unable to create CredRequest")
        os.Exit(1)
    }
}

```

- d. Ensure your Operator can wait for a **Secret** object to show up from the CCO, as shown in the following example, which is called along with the other items you are reconciling in your Operator:

Example wait for Secret object

```

// WaitForSecret is a function that takes a Kubernetes client, a namespace, and a v1
"k8s.io/api/core/v1" name as arguments
// It waits until the secret object with the given name exists in the given namespace
// It returns the secret object or an error if the timeout is exceeded
func WaitForSecret(client kubernetes.Interface, namespace, name string) (*v1.Secret,
error) {
    // set a timeout of 10 minutes
    timeout := time.After(10 * time.Minute)

    // set a polling interval of 10 seconds
    ticker := time.NewTicker(10 * time.Second)

    // loop until the timeout or the secret is found
    for {
        select {
        case <-timeout:
            // timeout is exceeded, return an error
            return nil, fmt.Errorf("timed out waiting for secret %s in namespace %s", name,
namespace)
            // add to this error with a pointer to instructions for following a manual path to a
            Secret that will work on STS
        case <-ticker.C:
            // polling interval is reached, try to get the secret
            secret, err := client.CoreV1().Secrets(namespace).Get(context.Background(), name,
metav1.GetOptions{})
            if err != nil {
                if errors.IsNotFound(err) {
                    // secret does not exist yet, continue waiting
                    continue
                } else {
                    // some other error occurred, return it
                    return nil, err
                }
            } else {
                // secret is found, return it
                return secret, nil
            }
        }
    }
}

```

The **timeout** value is based on an estimate of how fast the CCO might detect an added **CredentialsRequest** object and generate a **Secret** object. You might consider lowering the time or creating custom feedback for cluster administrators that could be wondering why the Operator is not yet accessing the cloud resources.

- e. Set up the AWS configuration by reading the secret created by the CCO from the credentials request and creating the AWS config file containing the data from that secret:

Example AWS configuration creation

```
func SharedCredentialsFileFromSecret(secret *corev1.Secret) (string, error) {
    var data []byte
    switch {
    case len(secret.Data["credentials"]) > 0:
        data = secret.Data["credentials"]
    default:
        return "", errors.New("invalid secret for aws credentials")
    }

    f, err := ioutil.TempFile("", "aws-shared-credentials")
    if err != nil {
        return "", errors.Wrap(err, "failed to create file for shared credentials")
    }
    defer f.Close()
    if _, err := f.Write(data); err != nil {
        return "", errors.Wrapf(err, "failed to write credentials to %s", f.Name())
    }
    return f.Name(), nil
}
```



IMPORTANT

The secret is assumed to exist, but your Operator code should wait and retry when using this secret to give time to the CCO to create the secret.

Additionally, the wait period should eventually time out and warn users that the OpenShift Container Platform cluster version, and therefore the CCO, might be an earlier version that does not support the **CredentialsRequest** object workflow with STS detection. In such cases, instruct users that they must add a secret by using another method.

- f. Configure the AWS SDK session, for example:

Example AWS SDK session configuration

```
sharedCredentialsFile, err := SharedCredentialsFileFromSecret(secret)
if err != nil {
    // handle error
}
options := session.Options{
    SharedConfigState: session.SharedConfigEnable,
    SharedConfigFiles: []string{sharedCredentialsFile},
}
```

10.3.2. Verifying the CCO installation in a hosted cluster on AWS

You can verify that the Cloud Credential Operator (CCO) is running correctly in your hosted control plane.

Prerequisites

- You configured the hosted cluster on Amazon Web Services (AWS).

Procedure

1. Verify that the CCO is configured in a manual mode in your hosted cluster by running the following command:

```
$ oc get cloudcredentials <hosted_cluster_name> \
  -n <hosted_cluster_namespace> \
  -o=jsonpath={.spec.credentialsMode}
```

Expected output

```
Manual
```

2. Verify that the value for the **serviceAccountIssuer** resource is not empty by running the following command:

```
$ oc get authentication cluster --kubeconfig <hosted_cluster_name>.kubeconfig \
  -o jsonpath --template '{.spec.serviceAccountIssuer }'
```

Example output

```
https://aos-hypershift-ci-oidc-29999.s3.us-east-2.amazonaws.com/hypershift-ci-29999
```

10.4. ADDITIONAL RESOURCES

- [Issuer Identifier](#)
- [Understanding identity provider configuration](#)
- [Cluster Operators reference page for the Cloud Credential Operator](#)

CHAPTER 11. HANDLING MACHINE CONFIGURATION FOR HOSTED CONTROL PLANES

In a standalone OpenShift Container Platform cluster, a machine config pool manages a set of nodes. You can handle a machine configuration by using the **MachineConfigPool** custom resource (CR).

TIP

You can reference any **machineconfiguration.openshift.io** resources in the **nodepool.spec.config** field of the **NodePool** CR.

In hosted control planes, the **MachineConfigPool** CR does not exist. A node pool contains a set of compute nodes. You can handle a machine configuration by using node pools.

You can manage your workloads in your hosted cluster by using the cluster autoscaler.



NOTE

In OpenShift Container Platform 4.18 or later, the default container runtime for worker nodes is changed from runC to crun.

11.1. CONFIGURING NODE POOLS FOR HOSTED CONTROL PLANES

In hosted control planes, you can configure node pools by creating a **MachineConfig** object inside of a config map in the management cluster.

Procedure

1. To create a **MachineConfig** object inside of a config map in the management cluster, enter the following information:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: <configmap_name>
  namespace: clusters
data:
  config: |
    apiVersion: machineconfiguration.openshift.io/v1
    kind: MachineConfig
    metadata:
      labels:
        machineconfiguration.openshift.io/role: worker
      name: <machineconfig_name>
    spec:
      config:
        ignition:
          version: 3.2.0
      storage:
        files:
          - contents:
              source: data:...
```



```
mode: 420
overwrite: true
path: ${PATH}
```

The **path** field specifies the path on the node where the **MachineConfig** object is stored.

2. After you add the object to the config map, you can apply the config map to the node pool as follows:

```
$ oc edit nodepool <nodepool_name> --namespace <hosted_cluster_namespace>
```

3. Edit the **NodePool** resource to include the config map:

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
# ...
  name: nodepool-1
  namespace: clusters
# ...
spec:
  config:
    - name: <configmap_name>
# ...
```

Replace **<configmap_name>** with the name of your config map.

11.2. REFERENCING THE KUBELET CONFIGURATION IN NODE POOLS

To reference your kubelet configuration in node pools, you add the kubelet configuration in a config map and then apply the config map in the **NodePool** resource.

Procedure

1. Add the kubelet configuration inside of a config map in the management cluster by entering the following information:

Example ConfigMap object with the kubelet configuration

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: <configmap_name>
  namespace: clusters
data:
  config: |
    apiVersion: machineconfiguration.openshift.io/v1
    kind: KubeletConfig
    metadata:
      name: <kubeletconfig_name>
    spec:
      kubeletConfig:
        registerWithTaints:
```

```
- key: "example.sh/unregistered"
  value: "true"
  effect: "NoExecute"
```

- **<configmap_name>** specifies the name of your config map.
- **<kubeletconfig_name>** specifies the name of the **KubeletConfig** resource.

2. Apply the config map to the node pool by entering the following command:

```
$ oc edit nodepool <nodepool_name> --namespace clusters
```

Replace **<nodepool_name>** with the name of your node pool.

Example NodePool resource configuration

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
  # ...
  name: nodepool-1
  namespace: clusters
  # ...
spec:
  config:
    - name: example-configmap-1
  # ...
```

11.3. CONFIGURING NODE TUNING IN A HOSTED CLUSTER

To set node-level tuning on the nodes in your hosted cluster, you can use the Node Tuning Operator. In hosted control planes, you can configure node tuning by creating config maps that contain **Tuned** objects and referencing those config maps in your node pools.

Procedure

1. Create a config map that contains a valid tuned manifest, and reference the manifest in a node pool. In the following example, a **Tuned** manifest defines a profile that sets **vm.dirty_ratio** to 55 on nodes that contain the **tuned-1-node-label** node label with any value. Save the following **ConfigMap** manifest in a file named **tuned-1.yaml**:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: tuned-1
  namespace: clusters
data:
  tuning: |
    apiVersion: tuned.openshift.io/v1
    kind: Tuned
    metadata:
      name: tuned-1
      namespace: openshift-cluster-node-tuning-operator
    spec:
```

```

profile:
- data: |
    [main]
    summary=Custom OpenShift profile
    include=openshift-node
    [sysctl]
    vm.dirty_ratio="55"
    name: tuned-1-profile
recommend:
- priority: 20
  profile: tuned-1-profile

```

NOTE

If you do not add any labels to an entry in the **spec.recommend** section of the Tuned spec, node-pool-based matching is assumed, so the highest priority profile in the **spec.recommend** section is applied to nodes in the pool. Although you can achieve more fine-grained node-label-based matching by setting a label value in the Tuned **.spec.recommend.match** section, node labels will not persist during an upgrade unless you set the **.spec.management.upgradeType** value of the node pool to **InPlace**.

2. Create the **ConfigMap** object in the management cluster:

```
$ oc --kubeconfig="$MGMT_KUBECONFIG" create -f tuned-1.yaml
```

3. Reference the **ConfigMap** object in the **spec.tuningConfig** field of the node pool, either by editing a node pool or creating one. In this example, assume that you have only one **NodePool**, named **nodepool-1**, which contains 2 nodes.

```

apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
  ...
  name: nodepool-1
  namespace: clusters
  ...
spec:
  ...
  tuningConfig:
    - name: tuned-1
status:
  ...

```

NOTE

You can reference the same config map in multiple node pools. In hosted control planes, the Node Tuning Operator appends a hash of the node pool name and namespace to the name of the Tuned CRs to distinguish them. Outside of this case, do not create multiple TuneD profiles of the same name in different Tuned CRs for the same hosted cluster.

Verification

Now that you have created the **ConfigMap** object that contains a **Tuned** manifest and referenced it in a **NodePool**, the Node Tuning Operator syncs the **Tuned** objects into the hosted cluster. You can verify which **Tuned** objects are defined and which TuneD profiles are applied to each node.

1. List the **Tuned** objects in the hosted cluster:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get tuned.tuned.openshift.io \
-n openshift-cluster-node-tuning-operator
```

Example output

```
NAME      AGE
default   7m36s
rendered  7m36s
tuned-1    65s
```

2. List the **Profile** objects in the hosted cluster:

```
$ oc --kubeconfig="$HC_KUBECONFIG" get profile.tuned.openshift.io \
-n openshift-cluster-node-tuning-operator
```

Example output

NAME	TUNED	APPLIED	DEGRADED	AGE
nodepool-1-worker-1	tuned-1-profile	True	False	7m43s
nodepool-1-worker-2	tuned-1-profile	True	False	7m14s



NOTE

If no custom profiles are created, the **openshift-node** profile is applied by default.

3. To confirm that the tuning was applied correctly, start a debug shell on a node and check the sysctl values:

```
$ oc --kubeconfig="$HC_KUBECONFIG" \
debug node/nodepool-1-worker-1 -- chroot /host sysctl vm.dirty_ratio
```

Example output

```
vm.dirty_ratio = 55
```

11.4. DEPLOYING THE SR-IOV OPERATOR FOR HOSTED CONTROL PLANES

After you configure and deploy your hosting service cluster, you can create a subscription to the SR-IOV Operator on a hosted cluster. The SR-IOV pod runs on worker machines rather than the control plane.

Prerequisites

You must configure and deploy the hosted cluster on AWS.

Procedure

1. Create a namespace and an Operator group:

```

apiVersion: v1
kind: Namespace
metadata:
  name: openshift-sriov-network-operator
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: sriov-network-operators
  namespace: openshift-sriov-network-operator
spec:
  targetNamespaces:
    - openshift-sriov-network-operator

```

2. Create a subscription to the SR-IOV Operator:

```

apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: sriov-network-operator-subscription
  namespace: openshift-sriov-network-operator
spec:
  channel: stable
  name: sriov-network-operator
  config:
    nodeSelector:
      node-role.kubernetes.io/worker: ""
  source: redhat-operators
  sourceNamespace: openshift-marketplace

```

Verification

1. To verify that the SR-IOV Operator is ready, run the following command and view the resulting output:

```
$ oc get csv -n openshift-sriov-network-operator
```

Example output

NAME	DISPLAY	VERSION	REPLACES
sriov-network-operator.4.22.0-202211021237	SR-IOV Network Operator	4.22.0-	
202211021237	sriov-network-operator.4.22.0-202210290517	Succeeded	

2. To verify that the SR-IOV pods are deployed, run the following command:

```
$ oc get pods -n openshift-sriov-network-operator
```

11.5. CONFIGURING THE NTP SERVER FOR HOSTED CLUSTERS

You can configure the Network Time Protocol (NTP) server for your hosted clusters by using Butane.

Procedure

1. Create a Butane config file, **99-worker-chrony.bu**, that includes the contents of the **chrony.conf** file. For more information about Butane, see "Creating machine configs with Butane".

Example 99-worker-chrony.bu configuration

```
# ...
variant: openshift
version: 4.22.0
metadata:
  name: 99-worker-chrony
  labels:
    machineconfiguration.openshift.io/role: worker
storage:
  files:
    - path: /etc/chrony.conf
      mode: 0644
      overwrite: true
      contents:
        inline: |
          pool 0.rhel.pool.ntp.org iburst
          driftfile /var/lib/chrony/drift
          makestep 1.0 3
          rtcsync
          logdir /var/log/chrony
# ...
```

- **storage.files.mode** specifies an octal value mode for the **mode** field in the machine config file. After you create the file and apply the changes, the **mode** field is converted to a decimal value.
- **storage.files.contents.inline** specifies any valid, reachable time source, such as the one provided by your Dynamic Host Configuration Protocol (DHCP) server.



NOTE

For machine-to-machine communication, the NTP on the User Datagram Protocol (UDP) port is **123**. If you configured an external NTP time server, you must open UDP port **123**.

2. Use Butane to generate a **MachineConfig** object file, **99-worker-chrony.yaml**, that contains a configuration that Butane sends to the nodes. Run the following command:

```
$ butane 99-worker-chrony.bu -o 99-worker-chrony.yaml
```

Example 99-worker-chrony.yaml configuration

```
# Generated by Butane; do not edit
apiVersion: machineconfiguration.openshift.io/v1
```

```

kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: <machineconfig_name>
spec:
  config:
    ignition:
      version: 3.2.0
  storage:
    files:
      - contents:
          source: data:...
          mode: 420
          overwrite: true
          path: /example/path

```

3. Add the contents of the **99-worker-chrony.yaml** file inside of a config map in the management cluster:

Example config map

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: <configmap_name>
  namespace: <namespace>
data:
  config: |
    apiVersion: machineconfiguration.openshift.io/v1
    kind: MachineConfig
    metadata:
      labels:
        machineconfiguration.openshift.io/role: worker
      name: <machineconfig_name>
    spec:
      config:
        ignition:
          version: 3.2.0
      storage:
        files:
          - contents:
              source: data:...
              mode: 420
              overwrite: true
              path: /example/path
# ...

```

Replace **<namespace>** with the name of your namespace where you created the node pool, such as **clusters**.

4. Apply the config map to your node pool by running the following command:

```
$ oc edit nodepool <nodepool_name> --namespace <hosted_cluster_namespace>
```

Example NodePool configuration

```

apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
  # ...
  name: nodepool-1
  namespace: clusters
  # ...
spec:
  config:
    - name: example-config-map
  # ...

```

5. Add the list of your NTP servers in the **infra-env.yaml** file, which defines the **InfraEnv** custom resource (CR):

Example infra-env.yaml file

```

apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
# ...
spec:
  additionalNTPSources:
    - <ntp_server>
    - <ntp_server1>
    - <ntp_server2>
  # ...

```

Replace **<ntp_server>** with the name of your NTP server. For more details about creating a host inventory and the **InfraEnv** CR, see "Creating a host inventory".

6. Apply the **InfraEnv** CR by running the following command:

```
$ oc apply -f infra-env.yaml
```

Verification

- Check the following fields to know the status of your host inventory:
 - **conditions:** The standard Kubernetes conditions indicating if the image was created successfully.
 - **isoDownloadURL:** The URL to download the Discovery Image.
 - **createdTime:** The time at which the image was last created. If you modify the **InfraEnv** CR, ensure that you have updated the timestamp before downloading a new image. Verify that your host inventory is created by running the following command:

```
$ oc describe infraenv <infraenv_resource_name> -n <infraenv_namespace>
```




NOTE

If you modify the **InfraEnv** CR, confirm that the **InfraEnv** CR has created a new Discovery Image by looking at the **createdTime** field. If you already booted hosts, boot them again with the latest Discovery Image.

Additional resources

- [Creating machine configs with Butane](#)
- [Creating a host inventory by using the command line interface](#)

11.6. SCALING UP AND DOWN WORKLOADS IN A HOSTED CLUSTER

To scale up and down the workloads in your hosted cluster, you can use the **ScaleUpAndScaleDown** behavior. The compute nodes scale up when you add workloads and scale down when you delete workloads.

Prerequisites

- You have created the **HostedCluster** and **NodePool** resources.

Procedure

1. Enable cluster autoscaling for your hosted cluster by setting the scaling behavior to **ScaleUpAndScaleDown**. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedcluster <hosted_cluster_name> \
  --type=merge \
  --patch='{"spec": {"autoscaling": {"scaling": "ScaleUpAndScaleDown",
"maxPodGracePeriod": 60, "scaleDown": {"utilizationThresholdPercent": 50}}}}'
```

2. Remove the **spec.replicas** field from the **NodePool** resource to allow cluster autoscaler to manage the node count. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <node_pool_name> \
  --type=json \
  --patch='[{"op": "remove", "path": "/spec/replicas"}]'
```

3. Enable cluster autoscaling to configure the minimum and maximum node counts for your node pools. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <nodepool_name> \
  --type=merge --patch='{"spec": {"autoScaling": {"max": 3, "min": 1}}}'
```

Verification

- To verify that all compute nodes are in the **Ready** status, run the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

Additional resources

- [Scaling the NodePool object for a hosted cluster \(bare-metal platforms\)](#)
- [Scaling the NodePool object for a hosted cluster \(non-bare metal agent machines\)](#)
- [Scaling a node pool \(OpenShift Virtualization\)](#)

11.7. SCALING UP WORKLOADS IN A HOSTED CLUSTER

To scale up the workloads in your hosted cluster, you can use the **ScaleUpOnly** behavior.

Prerequisites

- You have created the **HostedCluster** and **NodePool** resources.

Procedure

1. Enable cluster autoscaling for your hosted cluster by setting the scaling behavior to **ScaleUpOnly**. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> hostedcluster <hosted_cluster_name> --  
type=merge --patch='{ "spec": { "autoscaling": { "scaling": "ScaleUpOnly",  
"maxPodGracePeriod": 60}}}'
```

2. Remove the **spec.replicas** field from the **NodePool** resource to allow the cluster autoscaler to manage the node count. Run the following command:

```
$ oc patch -n clusters nodepool <node_pool_name> --type=json --patch='[{"op": "remove",  
"path": "/spec/replicas"}]'
```

3. Enable cluster autoscaling to configure the minimum and maximum node counts for your node pools. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> nodepool <nodepool_name> \  
--type=merge --patch='{ "spec": { "autoScaling": { "max": 3, "min": 1}}}'
```

Verification

1. Verify that all compute nodes are in the **Ready** status by running the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

2. Verify that the compute nodes are scaled up successfully by checking the node count for your node pools. Run the following command:

```
$ oc --kubeconfig nested.config get nodes -l 'hypershift.openshift.io/nodePool=  
<node_pool_name>'
```

11.8. SETTING THE PRIORITY EXPANDER IN A HOSTED CLUSTER

You can define the priority for your node pools and create high priority machines before low priority machines by using the priority expander in your hosted cluster.

Prerequisites

- You have created the **HostedCluster** and **NodePool** resources.

Procedure

1. To define the priority for your node pools, create a config map named **priority-expander-configmap.yaml** in your hosted cluster. Node pools with low numbers receive high priority. See the following example configuration:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-autoscaler-priority-expander
  namespace: kube-system
# ...
data:
  priorities: |-
    10:
      - ".*<node_pool_name1>.*"
    100:
      - ".*<node_pool_name2>.*"
# ...
```

2. Generate the **kubeconfig** file by running the following command:

```
$ hcp create kubeconfig --name <hosted_cluster_name> --namespace
<hosted_cluster_namespace> > nested.config
```

3. Create the **ConfigMap** object by running the following command:

```
$ oc --kubeconfig nested.config create -f priority-expander-configmap.yaml
```

4. Enable cluster autoscaling by setting the priority expander for your hosted cluster. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedcluster <hosted_cluster_name> \
  --type=merge \
  --patch='{"spec": {"autoscaling": {"scaling": "ScaleUpOnly", "maxPodGracePeriod": 60,
"expanders": ["Priority"]}}}'
```

5. Remove the **spec.replicas** field from the **NodePool** resource to allow the cluster autoscaler to manage the node count. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <node_pool_name> \
  --type=json
  --patch='[{"op": "remove", "path": "/spec/replicas"}]'
```

6. Enable cluster autoscaling to configure the minimum and maximum node counts for your node pools. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <nodepool_name> \
  --type=merge --patch='{ "spec": { "autoScaling": { "max": 3, "min": 1 } } }'
```

Verification

- After you apply new workloads, verify that the compute nodes associated with the priority node pool are scaled up first. Run the following command to check the status of the compute node:

```
$ oc --kubeconfig nested.config get nodes -l 'hypershift.openshift.io/nodePool=
<node_pool_name>'
```

11.9. BALANCING IGNORED LABELS IN A HOSTED CLUSTER

After you scale up your node pools, you can use **balancingIgnoredLabels** to evenly distribute the machines across node pools.

Prerequisites

- You have created the **HostedCluster** and **NodePool** resources.

Procedure

1. Add the **node.group.balancing.ignored** label to each of the relevant node pool by using the same label value. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <node_pool_name> \
  --type=merge \
  --patch='{ "spec": { "nodeLabels": { "node.group.balancing.ignored": "<label_name>" } } }'
```

2. Enable cluster autoscaling for your hosted cluster by running the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedcluster <hosted_cluster_name> \
  --type=merge \
  --patch='{ "spec": { "autoscaling": { "balancingIgnoredLabels":
[ "node.group.balancing.ignored" ] } } }'
```

3. Remove the **spec.replicas** field from the **NodePool** resource to allow the cluster autoscaler to manage the node count. Run the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <node_pool_name> \
  --type=json \
  --patch='[{"op": "remove", "path": "/spec/replicas"}]'
```

4. Enable cluster autoscaling to configure the minimum and maximum node counts for your node pools. Run the following command:

—

```
$ oc patch -n <hosted_cluster_namespace> \
  nodepool <nodepool_name> \
  --type=merge --patch='{"spec": {"autoScaling": {"max": 3, "min": 1}}}'
```

5. Generate the **kubeconfig** file by running the following command:

```
$ hcp create kubeconfig \
  --name <hosted_cluster_name> \
  --namespace <hosted_cluster_namespace> > nested.config
```

6. After you scale up the node pools, check that all compute nodes are in the **Ready** status by running the following command:

```
$ oc --kubeconfig nested.config get nodes -l 'hypershift.openshift.io/nodePool=
<node_pool_name>'
```

7. Confirm that the new nodes contain the **node.group.balancing.ignored** label by running the following command:

```
$ oc --kubeconfig nested.config get nodes \
  -l 'hypershift.openshift.io/nodePool=<node_pool_name>' \
  -o jsonpath='{.items[*].metadata.labels}' | grep "node.group.balancing.ignored"
```

8. Enable cluster autoscaling for your hosted cluster by running the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedcluster <hosted_cluster_name> \
  --type=merge \
  --patch='{"spec": {"autoscaling": {"balancingIgnoredLabels":
["node.group.balancing.ignored"]}}}'
```

Verification

- Verify that the number of nodes provisioned by each node pool is evenly distributed. For example, if you created three node pools with the same label value, the node counts might be 3, 2, and 3. Run the following command:

```
$ oc --kubeconfig nested.config get nodes -l 'hypershift.openshift.io/nodePool=
<node_pool_name>'
```

CHAPTER 12. FEATURE GATES IN A HOSTED CLUSTER

You can use feature gates in a hosted cluster to enable features that are not part of the default set of features. You can enable the **TechPreviewNoUpgrade** feature set by using feature gates in your hosted cluster.

12.1. ENABLING FEATURE SETS BY USING FEATURE GATES

You can enable the **TechPreviewNoUpgrade** feature set in a hosted cluster by editing the **HostedCluster** custom resource (CR) with the OpenShift CLI.

Prerequisites

- You installed the OpenShift CLI (**oc**).

Procedure

1. Open the **HostedCluster** CR for editing on the hosting cluster by running the following command:

```
$ oc edit hostedcluster <hosted_cluster_name> -n <hosted_cluster_namespace>
```

2. Define the feature set by entering a value in the **featureSet** field. For example:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  configuration:
    featureGate:
      featureSet: TechPreviewNoUpgrade
```

- **metadata.name** specifies your hosted cluster name.
- **metadata.namespace** specifies your hosted cluster namespace.
- **spec.configuration.featureGate.featureSet** is a subset of the current Technology Preview features.



WARNING

Enabling the **TechPreviewNoUpgrade** feature set on your cluster cannot be undone and prevents minor version updates. With this feature set, you can enable these Technology Preview features on test clusters, where you can fully test them. Do not enable this feature set on production clusters.

3. Save the file to apply the changes.

Verification

- Verify that the **TechPreviewNoUpgrade** feature gate is enabled in your hosted cluster by running the following command:

```
$ oc get featuregate cluster -o yaml
```

Additional resources

- [FeatureGate \[config.openshift.io/v1\]](https://config.openshift.io/v1)

CHAPTER 13. OBSERVABILITY FOR HOSTED CONTROL PLANES

You can gather metrics for hosted control planes by configuring metrics sets. Monitoring dashboards are created in the management cluster for each hosted cluster that it manages.

13.1. CONFIGURING METRICS SETS FOR HOSTED CONTROL PLANES

Hosted control planes creates **ServiceMonitor** resources in each control plane namespace that allow a Prometheus stack to gather metrics from the control planes.

The **ServiceMonitor** resources use metrics relabelings to define which metrics are included or excluded from a particular component, such as etcd or the Kubernetes API server. The number of metrics that are produced by control planes directly impacts the resource requirements of the monitoring stack that gathers them.

Instead of producing a fixed number of metrics that apply to all situations, you can configure a metrics set that identifies a set of metrics to produce for each control plane. The following metrics sets are supported:

- **Telemetry**: These metrics are needed for telemetry. This set is the default set and is the smallest set of metrics.
- **SRE**: This set includes the necessary metrics to produce alerts and allow the troubleshooting of control plane components.
- **All**: This set includes all of the metrics that are produced by standalone OpenShift Container Platform control plane components.

Procedure

- To configure a metrics set, set the **METRICS_SET** environment variable in the HyperShift Operator deployment by entering the following command:

```
$ oc set env -n hypershift deployment/operator METRICS_SET=All
```

13.1.1. SRE metrics set example

When you specify the **SRE** metrics set, the HyperShift Operator looks for a config map named **sre-metric-set** with a single key: **config**. The value of the **config** key must contain a set of **RelabelConfigs** that are organized by control plane component.

You can specify the following components:

- **etcd**
- **kubeAPIServer**
- **kubeControllerManager**
- **openshiftAPIServer**
- **openshiftControllerManager**

- **openshiftRouteControllerManager**
- **cvo**
- **olm**
- **catalogOperator**
- **registryOperator**
- **nodeTuningOperator**
- **controlPlaneOperator**
- **hostedClusterConfigOperator**

A configuration of the **SRE** metrics set is illustrated in the following example:

```
kubeAPIServer:
- action: "drop"
  regex: "etcd_(debugging|disk|server).*"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "apiserver_admission_controller_admission_latencies_seconds.*"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "apiserver_admission_step_admission_latencies_seconds.*"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "scheduler_(e2e_scheduling_latency_microseconds|scheduling_algorithm_predicate_evaluation|scheduling_algorithm_priority_evaluation|scheduling_algorithm_preemption_evaluation|scheduling_algorithm_latency_microseconds|binding_latency_microseconds|scheduling_latency_seconds)"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "apiserver_(request_count|request_latencies|request_latencies_summary|dropped_requests|storage_data_key_generation_latencies_microseconds|storage_transformation_failures_total|storage_transformation_latencies_microseconds|proxy_tunnel_sync_latency_secs)"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "docker_(operations|operations_latency_microseconds|operations_errors|operations_timeout)"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "reflector_(items_per_list|items_per_watch|list_duration_seconds|lists_total|short_watches_total|watch_duration_seconds|watches_total)"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "etcd_(helper_cache_hit_count|helper_cache_miss_count|helper_cache_entry_count|request_cache_get_latencies_summary|request_cache_add_latencies_summary|request_latencies_summary)"
  sourceLabels: ["__name__"]
- action: "drop"
  regex: "transformation_(transformation_latencies_microseconds|failures_total)"
```

```

    sourceLabels: ["__name__"]
  - action: "drop"
    regex:
"network_plugin_operations_latency_microseconds|sync_proxy_rules_latency_microseconds|rest_client
_request_latency_seconds"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "apiserver_request_duration_seconds_bucket;
(0.15|0.25|0.3|0.35|0.4|0.45|0.6|0.7|0.8|0.9|1.25|1.5|1.75|2.5|3|3.5|4.5|6|7|8|9|15|25|30|50)"
    sourceLabels: ["__name__", "le"]
kubeControllerManager:
  - action: "drop"
    regex: "etcd_(debugging|disk|request|server).*"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "rest_client_request_latency_seconds_(bucket|count|sum)"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "root_ca_cert_publisher_sync_duration_seconds_(bucket|count|sum)"
    sourceLabels: ["__name__"]
openshiftAPIServer:
  - action: "drop"
    regex: "etcd_(debugging|disk|server).*"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "apiserver_admission_controller_admission_latencies_seconds_.*"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "apiserver_admission_step_admission_latencies_seconds_.*"
    sourceLabels: ["__name__"]
  - action: "drop"
    regex: "apiserver_request_duration_seconds_bucket;
(0.15|0.25|0.3|0.35|0.4|0.45|0.6|0.7|0.8|0.9|1.25|1.5|1.75|2.5|3|3.5|4.5|6|7|8|9|15|25|30|50)"
    sourceLabels: ["__name__", "le"]
openshiftControllerManager:
  - action: "drop"
    regex: "etcd_(debugging|disk|request|server).*"
    sourceLabels: ["__name__"]
openshiftRouteControllerManager:
  - action: "drop"
    regex: "etcd_(debugging|disk|request|server).*"
    sourceLabels: ["__name__"]
olm:
  - action: "drop"
    regex: "etcd_(debugging|disk|server).*"
    sourceLabels: ["__name__"]
catalogOperator:
  - action: "drop"
    regex: "etcd_(debugging|disk|server).*"
    sourceLabels: ["__name__"]
cvo:
  - action: drop
    regex: "etcd_(debugging|disk|server).*"
    sourceLabels: ["__name__"]

```

13.2. CUSTOMIZED HOSTED CLUSTER IDENTIFIERS

When you enable observability for hosted control planes, control plane metrics include an `_id` label that identifies the hosted cluster. You can set **`spec.clusterID`** in the **HostedCluster** custom resource (CR) at creation time to use a stable identifier instead of a randomly assigned UUID.

When you forward hosted cluster metrics to an external monitoring system, the `_id` label is commonly used to identify the cluster. If you reinstall a hosted cluster, specifying the same **`clusterID`** value preserves your external monitoring configuration.

Each hosted cluster has a unique cluster identifier. The HyperShift Operator uses this identifier in telemetry and in metrics that the control plane operators produce. The identifier is exposed on time series as the `_id` label.

If you do not specify **`spec.clusterID`** when you create a **HostedCluster** CR, the HyperShift controller generates a random RFC4122 UUID and sets the field for you.



NOTE

The **`spec.clusterID`** specification is not the same as the **`spec.infraID`** specification. The **`infraID`** value identifies cloud infrastructure resources.

13.2.1. Example: Setting a custom cluster identifier

You can set the **`spec.clusterID`** value only when you create a **HostedCluster** custom resource (CR).



IMPORTANT

After you set **`spec.clusterID`** is set, you cannot change it. Plan the identifier before you create the hosted cluster.

The following example shows a **HostedCluster** CR with a custom cluster identifier set:

Example HostedCluster CR with a custom cluster identifier

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  clusterID: fa45babd-40f3-4085-9b30-8bc3b7df1557
  controllerAvailabilityPolicy: SingleReplica
  dns:
    baseDomain: example.com
  platform:
    type: AWS
  release:
    image: <ocp_release_image>
  pullSecret:
    name: <pull_secret_name>
```

The **spec.clusterID** value is the UUID that you want to use as the stable cluster identifier in metrics. The value must be a valid RFC4122 UUID: **xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx** in hexadecimal digits.

The value of **spec.clusterID** is added as the **_id** label on control plane metrics through Prometheus relabeling rules on **ServiceMonitor** and **PodMonitor** resources. HyperShift Operator metrics for the hosted cluster also use the same **_id** label, so you can correlate metrics from the management cluster and the hosted control plane in one query.

For example, to filter metrics for a specific hosted cluster, use the **_id** label in a PromQL expression:

```
{__name__=~"hypershift_.*", _id="fa45babd-40f3-4085-9b30-8bc3b7df1557"}
```

When you enable monitoring dashboards, the **CLUSTER_ID** placeholder in the dashboard template is replaced with the same UUID. For more information, see "Dashboard customization".

13.2.1.1. Cluster identifier reuse after a reinstall

If you delete and re-create a hosted cluster, a new random **clusterID** is assigned unless you specify one. To keep the same identifier in external monitoring systems, set **spec.clusterID** in the new **HostedCluster** CR to the UUID that you used before.

Additional resources

- [Dashboard customization](#)
- [Configuring metrics sets for hosted control planes](#)
- [Enabling monitoring dashboards in a hosted cluster](#)

13.3. ENABLING MONITORING DASHBOARDS IN A HOSTED CLUSTER

You can enable monitoring dashboards in a hosted cluster by creating a config map.

Procedure

1. Create the **hypershift-operator-install-flags** config map in the **local-cluster** namespace. See the following example configuration:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: hypershift-operator-install-flags
  namespace: local-cluster
data:
  installFlagsToAdd: "--monitoring-dashboards --metrics-set=All"
  installFlagsToRemove: ""
```

The **--monitoring-dashboards --metrics-set=All** flag adds the monitoring dashboard for all metrics.

2. Wait a couple of minutes for the HyperShift Operator deployment in the **hypershift** namespace to be updated to include the following environment variable:

```
- name: MONITORING_DASHBOARDS
```

value: "1"

When monitoring dashboards are enabled, for each hosted cluster that the HyperShift Operator manages, the Operator creates a config map named **hc-<hosted_cluster_namespace>-<hosted_cluster_name>** in the **openshift-config-managed** namespace, where **<hosted_cluster_namespace>** is the namespace of the hosted cluster and **<hosted_cluster_name>** is the name of the hosted cluster. As a result, a new dashboard is added in the administrative console of the management cluster.

3. To view the dashboard, log in to the management cluster's console and go to the dashboard for the hosted cluster by clicking **Observe → Dashboards**.
4. Optional: To disable monitoring dashboards in a hosted cluster, remove the **--monitoring-dashboards --metrics-set=All** flag from the **hypershift-operator-install-flags** config map. When you delete a hosted cluster, its corresponding dashboard is also deleted.

Additional resources

- [Customized hosted cluster identifiers](#)

13.3.1. Dashboard customization

To generate dashboards for each hosted cluster, the HyperShift Operator uses a template that is stored in the **monitoring-dashboard-template** config map in the Operator namespace (**hypershift**). This template contains a set of Grafana panels that contain the metrics for the dashboard.

You can edit the content of the config map to customize the dashboards.

When a dashboard is generated, the following strings are replaced with values that correspond to a specific hosted cluster:

Name	Description
__NAME__	The name of the hosted cluster
__NAMESPACE__	The namespace of the hosted cluster
__CONTROL_PLANE_NAMESPACE__	The namespace where the control plane pods of the hosted cluster are placed
__CLUSTER_ID__	The UUID of the hosted cluster, which matches the _id label of the hosted cluster metrics

To set a custom cluster identifier when you create the hosted cluster, see "Customized hosted cluster identifiers".

13.4. CONNECTIVITY MONITORING FOR HOSTED CONTROL PLANES

Cluster service providers can monitor connectivity metrics to ensure proper function during an update. They can also use the metrics to find connectivity issues between the control plane and the data plane, or vice versa.

Studying these metrics over time can inform decisions about capacity planning and scaling.

13.4.1. Connectivity monitoring from the control plane to the data plane

Cluster administrators can monitor network activity between a hosted control plane and the compute nodes in a data plane by using the **DataPlaneConnectionAvailable** condition. This condition is useful for identifying and troubleshooting network connectivity issues in hosted clusters.

The **DataPlaneConnectionAvailable** condition is available by default starting with version 4.21.

The **DataPlaneConnectionAvailable** condition monitors the connectivity from the control plane to the data plane by taking the following steps:

1. Counts available compute nodes in the hosted cluster.
2. Lists the **konnnectivity-agent** pods that are running in the **kube-system** namespace on the data plane.
3. Reads the logs from the running **konnnectivity-agent** pod to verify that it can communicate with the data plane.

The **hosted-cluster-config-operator** component that runs in the control plane namespace evaluates the condition and provides status and reason information.

The following table details the status and reason values that can be displayed for the condition:

Status	Reason value	Description
True	AsExpected	The control plane can reach the data plane nodes through the konnnectivity-agent pods.
False	KonnnectivityAgentPodsNotFound	No konnnectivity-agent pods are running, or none are found.
False	ReconciliationError	An error occurred while listing the konnnectivity-agent pods.
Unknown	NoWorkerNodesAvailable	No compute nodes are available in the cluster. No errors occurred, but no compute nodes were found.
Unknown	ReconcileError	Unable to count compute nodes because an error occurred.

For information about how to troubleshoot connectivity issues, see "Troubleshooting connectivity for hosted control planes".

Additional resources

- [Troubleshooting connectivity for hosted control planes](#)

13.4.2. Connectivity monitoring from the data plane to the control plane

Cluster administrators can monitor network activity between the compute nodes in a data plane and a hosted control plane by using the **ControlPlaneConnectionAvailable** condition. This condition is useful for identifying and troubleshooting network connectivity issues in hosted clusters.

The **ControlPlaneConnectionAvailable** condition detects whether data plane nodes can reach control plane components. The **hosted-cluster-config-operator** component evaluates the condition, and a deployment with 3 replicas checks connectivity.

The condition monitors the connectivity between the data plane and the control plane by taking the following steps:

1. Deploys a **kas-connection-checker** deployment to the **kube-system** namespace on the data plane.
2. Each pod runs a shell script in an infinite loop that transfers data to and from the Kubernetes API server endpoint every 60 seconds. On success, the script patches the **control-plane-connectivity-check** config map with a **lastSucceeded** timestamp.
3. The **hosted-cluster-config-operator** component checks whether the **control-plane-connectivity-check** config map exists and whether the **lastSucceeded** timestamp is within the last 5 minutes. It does not check pod readiness counts.

The following table details the status and reason values that can be displayed for the condition:

Status	Reason value	Description
True	AsExpected	All data plane nodes can reach the control plane (NumberReady = DesiredNumberScheduled).
False	KASAccessFailed	At least one data plane node cannot reach the control plane. The message shows the ratio of pods that are ready; for example, 1/3 pods ready .
Unknown	NoWorkerNodesAvailable	No compute nodes are available to check connectivity (DesiredNumberScheduled = 0).
Unknown	StatusUnknown	The Kubernetes API server connection checker DaemonSet was not found.
Unknown	ReconcileError	An API error blocked the retrieval of the DaemonSet status.



IMPORTANT

This condition has a known limitation with HTTPS proxy environments. In HTTPS proxy environments, the condition might incorrectly report **False** because of probe limitations.

CHAPTER 14. NETWORKING FOR HOSTED CONTROL PLANES

Ensure optimal performance with hosted control planes by configuring network settings. Those settings include internal subnets and proxy support for control-plane workloads, compute nodes, management clusters, and hosted clusters.

14.1. NETWORKING OVERVIEW FOR HOSTED CONTROL PLANES

Proper network design can ensure the performance, automation, and security of hosted control planes. Your network configuration depends on the segmentation that you need and the provider that you use, such as a cloud provider, bare metal on the Agent platform, or OpenShift Virtualization.

For example, for hosted control planes on OpenShift Virtualization, you can create **NodePool** virtual machines (VMs) with different network configurations, including using the pod network and bridging an external network into the VMs. Network latency and throughput are important factors to ensure control-plane communication, workload traffic, and storage access. The use of dedicated physical network adapters can ensure adequate throughput and low latency for all components.

14.1.1. Control plane and data plane networking considerations

When you consider your networking configuration for hosted control planes, remember the structure of the control plane and data plane:

Control plane

The control plane includes pods that run on the management cluster in a dedicated namespace. The pods expose the hosted control plane **kube-apiserver** component, OAuth, Ignition server, and Konnectivity service. Hosted cluster nodes access those services through the data plane network, so the services must be reachable from the data plane network, even if a firewall is in place.

The management cluster hosts different control planes, and their services are exposed through its network. Network segmentation by API is not currently supported.

Data plane

The data plane includes the compute, storage, and networking where workloads and applications run. It has a minimum of 2 compute nodes.

For example, the data plane includes compute nodes that are represented by bare-metal servers or VMs that are hosted on OpenShift Virtualization. For OpenShift Virtualization, you can host different data planes on the management cluster. For bare-metal servers or an external OpenShift Virtualization cluster, you can host data planes by external infrastructure.

Some VLAN-based network segmentation is possible if the following conditions are true:

- All of the configured VLANs can route traffic to the endpoints of the API, OAuth, Ignition server, and Konnectivity service.
- Hosted cluster users can access the API and OAuth.

The node pools in hosted clusters on the data plane depend on the components that run on the control plane, such as Konnectivity, Ignition, and OAuth. By default, you install those components by using a **Route** endpoint publishing strategy, and admit them on the default ingress controller, which is often installed on primary networks from the management cluster. To segment this implementation on a secondary network, you must change several network flows between the control plane and the data plane.

14.1.2. DHCP requirements for hosted control planes

The requirements for Dynamic Host Configuration Protocol (DHCP) differ depending on your platform provider:

- For bare metal providers, you can use dynamic or manual IP segmentation, so a DHCP server is not mandatory. Ensure that the same node always has the same IP address.
- For OpenShift Virtualization, dynamic IP assignment is the only option, so a DHCP server is mandatory.

14.2. NETWORK ISOLATION FOR HOSTED CLUSTERS

To provide distinct, secure environments for multiple hosted clusters that share the same management cluster, you can configure hosted clusters with specific network and virtual machine (VM) isolation levels.

For hosted clusters, network isolation levels include container-based isolation, VM-based isolation, and physical isolation:

Container-based isolation

With container-based isolation, you use shared compute nodes, and isolation is enforced by policy, namespaces, and control group or SELinux boundaries. Control-plane components run as pods on the management cluster, isolated mainly by Kubernetes or Linux mechanisms, not by giving each hosted cluster its own VM or server. Each control plane runs in a dedicated namespace, with default-deny networking and a network policy that allows only defined traffic. Pods use the restricted security context constraint, with unique identifiers (UIDs) scoped for each namespace.

VM-based isolation

With VM-based isolation, you run hosted control plane pods inside VMs; for example, on OpenShift Virtualization. This type of isolation is useful if you require VM-based isolation over container-based isolation for a multitenant management cluster. You add a hypervisor boundary between hosted clusters that share a management cluster. Control plane pods are pinned to dedicated VMs. To achieve VM-based isolation, you follow a "shared-nothing" approach, where each control plane has its own dedicated node. The management cluster must be an OpenShift Container Platform cluster on a VM so that the VM compute nodes can be used with the "shared-nothing" annotations to dedicate them to specific hosted control planes. Those hosted control planes will not share kernel space with control planes from other hosted clusters that are on the same management cluster.

Physical isolation

With physical isolation, you also follow a "shared-nothing" approach. Nodes for a specific hosted cluster are tainted and labeled with a specific **hypershift.openshift.io/hosted-cluster** value. Security involves the use of a network policy and physical network access control lists (ACLs) across hosted clusters.

For more information, see "Control plane isolation" and "Distributing hosted cluster workloads".

Additional resources

- [Control plane isolation](#)
- [Distributing hosted cluster workloads](#)

14.2.1. Control plane isolation

You can configure hosted control planes to isolate network traffic or control plane pods.

Each hosted control plane is assigned to run in a dedicated Kubernetes namespace. By default, the Kubernetes namespace denies all network traffic.

The following network traffic is allowed through the network policy that is enforced by the Kubernetes Container Network Interface (CNI):

- Ingress pod-to-pod communication in the same namespace (intra-tenant)
- Ingress on port 6443 to the hosted **kube-apiserver** pod for the tenant
- Metric scraping from the management cluster Kubernetes namespace with the **network.openshift.io/policy-group: monitoring** label is allowed for monitoring

14.2.1.1. Control plane pod isolation

In addition to network policies, each hosted control plane pod is run with the **restricted** security context constraint. This policy denies access to all host features and requires pods to be run with a unique identifier (UID) and with SELinux context that is allocated uniquely to each namespace that hosts a customer control plane.

The policy ensures the following constraints:

- Pods cannot run as privileged.
- Pods cannot mount host directory volumes.
- Pods must run as a user in a pre-allocated range of UIDs.
- Pods must run with a pre-allocated Multi-Category Security (MCS) label.
- Pods cannot access the host network namespace.
- Pods cannot expose host network ports.
- Pods cannot access the host PID namespace.
- By default, pods drop the following Linux capabilities: **KILL**, **MKNOD**, **SETUID**, and **SETGID**.

The management components, such as **kubelet** and **crio**, on each management cluster worker node are protected by an SELinux label that is not accessible to the SELinux context for pods that support hosted control planes.

The following SELinux labels are used for key processes and sockets:

kubelet

system_u:system_r:unconfined_service_t:s0

crio

system_u:system_r:container_runtime_t:s0

crio.sock

system_u:object_r:container_var_run_t:s0

<example user container processes>

system_u:system_r:container_t:s0:c14,c24

To achieve physical or virtual machine (VM) isolation, you need to use a "shared-nothing" approach, where each control plane has its own dedicated node. Nodes for a specific hosted cluster are tainted and labeled with a specific **hypershift.openshift.io/hosted-cluster** value. Security involves the use of a network policy and physical network access control lists (ACLs) across hosted clusters.

14.3. FIREWALL, PORT, AND SERVICE REQUIREMENTS FOR HOSTED CONTROL PLANES

Ensure that you meet the firewall, port, and service requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.

14.3.1. Bare metal firewall, port, and service requirements

You must meet the firewall, port, and service requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.



NOTE

Services run on their default ports. However, if you use the **NodePort** publishing strategy, services run on the port that is assigned by the **NodePort** service.

Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary. For production deployments, use a load balancer to simplify access through a single IP address.

If your hub cluster has a proxy configuration, ensure that it can reach the hosted cluster API endpoint by adding all hosted cluster API endpoints to the **noProxy** field on the **Proxy** object. For more information, see "Configuring the cluster-wide proxy".

A hosted control plane exposes the following services on bare metal:

- **APIServer**
 - The **APIServer** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- **OAuthServer**
 - The **OAuthServer** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **OAuthServer** service.
- **Konnectivity**
 - The **Konnectivity** service runs on port 443 by default when you use the route and ingress to expose the service.
 - The **Konnectivity** agent establishes a reverse tunnel to allow the control plane to access the network for the hosted cluster. The agent uses egress to connect to the **Konnectivity** server. The server is exposed by using either a route on port 443 or a manually assigned

NodePort.

- If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
- If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.

- **Ignition**

- The **Ignition** service runs on port 443 by default when you use the route and ingress to expose the service.
- If you use the **NodePort** publishing strategy, use a firewall rule for the **Ignition** service.

You do not need the following services on bare metal:

- **OVNSbDb**
- **OIDC**

14.3.2. OpenShift Virtualization firewall and port requirements

Ensure that you meet the firewall and port requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.

- The **kube-apiserver** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use the **NodePort** publishing strategy, ensure that the node port that is assigned to the **kube-apiserver** service is exposed.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- If you use the **NodePort** publishing strategy, use a firewall rule for the **ignition-server** and **OAuth-server** settings.
- The **konnnectivity** agent, which establishes a reverse tunnel to allow bi-directional communication on the hosted cluster, requires egress access to the cluster API server address on port 6443. With that egress access, the agent can reach the **kube-apiserver** service.
 - If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
 - If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.
- If you change the default port of 6443, adjust the rules to reflect that change.
- Ensure that you open any ports that are required by the workloads that run in the clusters.
- Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary.
- For production deployments, use a load balancer to simplify access through a single IP address.

14.3.3. Firewall, port, and service requirements for non-bare-metal agent machines

You must meet the firewall and port requirements so that ports can communicate between the management cluster, the control plane, and hosted clusters.



NOTE

Services run on their default ports. However, if you use the **NodePort** publishing strategy, services run on the port that is assigned by the **NodePort** service.

Use firewall rules, security groups, or other access controls to restrict access to only required sources. Avoid exposing ports publicly unless necessary. For production deployments, use a load balancer to simplify access through a single IP address.

A hosted control plane exposes the following services on non-bare-metal agent machines:

- **APIServer**
 - The **APIServer** service runs on port 6443 by default and requires ingress access for communication between the control plane components.
 - If you use MetalLB load balancing, allow ingress access to the IP range that is used for load balancer IP addresses.
- **OAuthServer**
 - The **OAuthServer** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **OAuthServer** service.
- **Konnectivity**
 - The **Konnectivity** service runs on port 443 by default when you use the route and ingress to expose the service.
 - The **Konnectivity** agent establishes a reverse tunnel to allow the control plane to access the network for the hosted cluster. The agent uses egress to connect to the **Konnectivity** server. The server is exposed by using either a route on port 443 or a manually assigned **NodePort**.
 - If the cluster API server address is an internal IP address, allow access from the workload subnets to the IP address on port 6443.
 - If the address is an external IP address, allow egress on port 6443 to that external IP address from the nodes.
- **Ignition**
 - The **Ignition** service runs on port 443 by default when you use the route and ingress to expose the service.
 - If you use the **NodePort** publishing strategy, use a firewall rule for the **Ignition** service.

You do not need the following services on non-bare-metal agent machines:

- **OVNSbDb**
- **OIDC**

14.3.4. Example firewall configuration

Review an example of what the firewall configuration looks like for a typical hosted control planes on AWS deployment that uses **Route** service publishing.

Ingress rules

- Port **6443**/TCP: Kubernetes API server, from compute nodes and external clients
- Port **443**/TCP: OpenShift Router for Ignition or Konnectivity routes, from compute nodes

Egress rules

- Port **443**/TCP: HTTPS, to container registries, routes, and external services
- Port **6443**/TCP: Management cluster API, to management cluster
- Port **53**/TCP and UDP: DNS, to DNS servers

If you use **NodePort** or **LoadBalancer** service publishing instead of **Route** service publishing, the following rules apply:

- Port **8091**/TCP: Konnectivity server, from compute nodes
- Port **8443**/TCP: Ignition Proxy, from compute nodes during the bootstrap process, **NodePort** publishing strategy only
- Port **9090**/TCP: Ignition server, from compute nodes during the bootstrap process, **NodePort** publishing strategy only

14.4. INGRESS AND EGRESS REQUIREMENTS FOR HOSTED CONTROL PLANES

Specific network ports must be open for communication between the management cluster, the hosted control planes components, and the compute nodes. The ports are categorized into ingress ports, which involve incoming traffic to hosted control planes and egress ports, which involve outgoing traffic from hosted control planes.

14.4.1. Ingress requirements for hosted control planes

Ingress ports involve incoming traffic to hosted control planes. Ensure the correct ports are open for communication between the management cluster, the hosted control planes components, and the compute nodes.

The following table details the ports for incoming traffic to hosted control planes across all platforms:

Table 14.1. Common ingress ports

Port	Protocol	Service	Description	Code reference
6443	TCP	Kubernetes API server	Primary API server port for kubectl and cluster communication	support/config/constants.go:35 - KASSVCPort = 6443
9090	TCP	Ignition server	Port from compute nodes during the bootstrap process, NodePort or Route service publishing strategy	-

The service publishing strategy determines additional ports. The **Ignition Proxy** and **Konnectivity** services are exposed through one of the following service publishing strategies:

Route

This setting is the default on OpenShift Container Platform. Traffic flows through the OpenShift router on port 443. No additional firewall rules are needed beyond standard HTTPS.

NodePort

Direct access is required to port 8091 (Konnectivity) and port 8443 (Ignition Proxy).

LoadBalancer

Direct access is required to port 8091 (Konnectivity) through the cloud load balancer.

The following table details the ingress port configurations that are specific to each platform:

Table 14.2. Platform-specific ingress port configurations

Platform	Port	Service	Description	Code reference
Agent	8443	Ignition Proxy	HTTPS proxy for ignition content delivery (NodePort publishing)	hypershift-operator/controllers/hostedcluster/network_policies.go:390
Agent	8081	Agent CAPI health probe	Health check endpoint for Agent platform CAPI provider	hypershift-operator/controllers/hostedcluster/internal/platform/agent.go:96,105,115

Platform	Port	Service	Description	Code reference
Agent	8080	Agent CAPI metrics	Metrics endpoint for Agent platform CAPI provider (binds to localhost only)	hypershift-operator/controllers/hostedcluster/internal/platform/agent/agent.go:97
AWS	9440	CAPI health check	Health and readiness probe endpoint for AWS CAPI provider	hypershift-operator/controllers/hostedcluster/internal/platform/aws/aws.go:222-223
Bare metal without the Agent platform	8443	Ignition Proxy	HTTPS proxy for ignition content delivery (NodePort publishing)	-
KubeVirt	9440	CAPI health check	Health and readiness probe endpoint	hypershift-operator/controllers/hostedcluster/internal/platform/kubevirt/kubevirt.go:140
RHOSP (Technology Preview)	9440	CAPI health check	Health and readiness probe endpoint	hypershift-operator/controllers/hostedcluster/internal/platform/openstack/openstack.go:238
RHOSP (Technology Preview)	8081	ORC health check	Health and readiness probe endpoint for OpenStack Resource Controller	hypershift-operator/controllers/hostedcluster/internal/platform/openstack/openstack.go:294,311

The following table details the ingress port configurations for private clusters, such as those on AWS:

Table 14.3. Ingress port configurations for private clusters

Port	Service	Description	Code reference
8080	Private router HTTP	HTTP traffic through the private router	hypershift-operator/controllers/hostedcluster/network_policies.go:244
8443	Private router HTTPS	HTTPS traffic through the private router	hypershift-operator/controllers/hostedcluster/network_policies.go:245

14.4.2. Egress requirements for hosted control planes

Egress ports involve outgoing traffic from hosted control planes. Ensure the correct ports are open for communication between the management cluster, the hosted control planes components, and the compute nodes.

The following table details the ports that must be accessible for outgoing traffic from hosted control planes, across all platforms.

Table 14.4. Common egress ports

Port	Protocol	Service	Purpose
443	TCP	HTTPS	OLM images, Ignition content, external HTTPS services
6443	TCP	Kubernetes API server	Communication with management cluster API
53	TCP and UDP	DNS	Standard DNS queries

Compute nodes require outbound network access to several hosted control planes services. The following table details the egress requirements for compute nodes.

Table 14.5. Compute node egress requirements

Port	Protocol	Service	Purpose	When required
443	TCP	HTTPS	Container registries, Ignition or Konnectivity service via Route service publishing strategy, external HTTPS services	Always

Port	Protocol	Service	Purpose	When required
6443	TCP	Kubernetes API server	Cluster management and kubelet communication	Always
8091	TCP	Konnectivity server	Establishes a reverse tunnel for control plane access	NodePort or LoadBalancer publishing only
8443	TCP	Ignition Proxy	Retrieves bootstrap configuration	NodePort publishing only for Agent platform or bare metal
53	TCP and UDP	DNS	Name resolution	Always

14.4.3. Handling ingress in a hosted cluster on bare metal

Every OpenShift Container Platform cluster has a default application Ingress Controller that typically has an external DNS record associated with it.

For example, if you create a hosted cluster named **example** with the base domain **krnl.es**, you can expect the wildcard domain ***.apps.example.krnl.es** to be routable.

To set up a load balancer and wildcard DNS record for the ***.apps** domain, perform the following actions on your hosted cluster.

Procedure

1. Deploy MetalLB by creating a YAML file that has the configuration for the MetalLB Operator:

```

apiVersion: v1
kind: Namespace
metadata:
  name: metallb
  labels:
    openshift.io/cluster-monitoring: "true"
  annotations:
    workload.openshift.io/allowed: management
---
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: metallb-operator-operatorgroup
  namespace: metallb
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
```

```
name: metallb-operator
namespace: metallb
spec:
  channel: "stable"
  name: metallb-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

2. Save the file as **metallb-operator-config.yaml**.

3. Enter the following command to apply the configuration:

```
$ oc apply -f metallb-operator-config.yaml
```

4. After the Operator is running, create the MetalLB instance:

a. Create a YAML file that has the configuration for the MetalLB instance:

```
apiVersion: metallb.io/v1beta1
kind: MetalLB
metadata:
  name: metallb
  namespace: metallb
```

b. Save the file as **metallb-instance-config.yaml**.

c. Create the MetalLB instance by entering this command:

```
$ oc apply -f metallb-instance-config.yaml
```

5. Create an **IPAddressPool** resource with a single IP address. This IP address must be on the same subnet as the network that the cluster nodes use.

a. Create a file, such as **ipaddresspool.yaml**, with content similar to the following example:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  namespace: metallb
  name: <ip_address_pool_name>
spec:
  addresses:
    - <ingress_ip>-<ingress_ip>
  autoAssign: false
```

- **metadata.name** specifies the **IPAddressPool** resource name.
- **spec.addresses** specifies the IP address for your environment. For example, **192.168.122.23**.

b. Apply the configuration for the IP address pool by entering the following command:

```
$ oc apply -f ipaddresspool.yaml
```

6. Create a L2 advertisement.

- a. Create a file, such as **l2advertisement.yaml**, with content similar to the following example:

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: <l2_advertisement_name>
  namespace: metallb
spec:
  ipAddressPools:
    - <ip_address_pool_name>
```

- **metadata.name** specifies the **L2Advertisement** resource name.
- **spec.ipAddressPools** specifies the **IPAddressPool** resource name.

- b. Apply the configuration by entering the following command:

```
$ oc apply -f l2advertisement.yaml
```

7. After creating a service of the **LoadBalancer** type, MetalLB adds an external IP address for the service.

- a. Configure a new load balancer service that routes ingress traffic to the ingress deployment by creating a YAML file named **metallb-loadbalancer-service.yaml**:

```
kind: Service
apiVersion: v1
metadata:
  annotations:
    metallb.io/address-pool: ingress-public-ip
  name: metallb-ingress
  namespace: openshift-ingress
spec:
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
    - name: https
      protocol: TCP
      port: 443
      targetPort: 443
  selector:
    ingresscontroller.operator.openshift.io/deployment-ingresscontroller: default
  type: LoadBalancer
```

- b. Save the **metallb-loadbalancer-service.yaml** file.
- c. Enter the following command to apply the YAML configuration:

```
$ oc apply -f metallb-loadbalancer-service.yaml
```

- d. Enter the following command to reach the OpenShift Container Platform console:

```
$ curl -kI https://console-openshift-console.apps.example.krn1.es
```

Example output

```
HTTP/1.1 200 OK
```

- e. Check the **clusterversion** and **clusteroperator** values to verify that everything is running. Enter the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get clusterversion,co
```

Example output

```
NAME                                VERSION AVAILABLE PROGRESSING SINCE
STATUS
clusterversion.config.openshift.io/version 4.x.y   True    False    3m32s Cluster
version is 4.x.y

NAME                                VERSION AVAILABLE
PROGRESSING DEGRADED SINCE MESSAGE
clusteroperator.config.openshift.io/console 4.x.y   True    False
False    3m50s
clusteroperator.config.openshift.io/ingress 4.x.y   True    False
False    53m
```

Replace **<4.x.y>** with the supported OpenShift Container Platform version that you want to use, for example, **4.22.0-multi**.

14.5. CONFIGURING INTERNAL OVN IPV4 SUBNETS FOR HOSTED CLUSTERS

In hosted clusters, you can configure internal OVN subnets to avoid routing conflicts, customize network architecture, or enable virtual private cloud (VPC) peering.

Avoid CIDR conflicts

Connect VPCs that host Red Hat OpenShift Service on AWS clusters with other VPCs that use the default OVN internal subnets of 100.88.0.0/16 and 100.64.0.0/16. To avoid a conflict on a **kubevirt** hosted cluster, the HyperShift Operator sets the OVN join subnet to **100.66.0.0/16** instead of **100.64.0.0/16** or **100.65.0.0/16**. The default classless inter-domain routing (CIDR) for the OVN gateway router is **100.64.0.0/16**. The default CIDR for the user-defined network (UDN) join subnet is **100.65.0.0/16**.

Customize network architecture

Configure internal OVN subnets to align with your corporate network policies.

Enable VPC peering

Deploy hosted clusters in environments where default subnets conflict with peered networks.

To configure OVN-internal subnets, you expose two OVN-Kubernetes internal subnet configuration options:

internalJoinSubnet

Internal subnet used by OVN-Kubernetes for the join network (default: **100.64.0.0/16**)

internalTransitSwitchSubnet

Internal subnet used for the distributed transit switch in OVN Interconnect architecture (default: **100.88.0.0/16**)

You can configure internal OVN subnets in an existing hosted cluster or configure the subnets while you create a hosted cluster.

Prerequisites

- Your hosted cluster version must be OpenShift Container Platform 4.20 or later.
- For the network type, your hosted cluster must use **networkType: OVNKubernetes**.
- Custom subnets must not overlap with the following subnets:
 - Machine CIDRs
 - Service CIDRs
 - Cluster network CIDRs
 - Any other networks in your infrastructure

Procedure

- To configure internal OVN subnets while you create a hosted cluster, in the configuration file for the hosted cluster, include the following section:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_control_plane_namespace>
spec:
  networking:
    networkType: OVNKubernetes
    machineCIDR: 10.0.0.0/16
    serviceCIDR: 172.30.0.0/16
    clusterNetwork:
      - cidr: 10.128.0.0/14
  operatorConfiguration:
    clusterNetworkOperator:
      ovnKubernetesConfig:
        ipv4:
          internalJoinSubnet: "100.99.0.0/16"
          internalTransitSwitchSubnet: "100.69.0.0/16"
```

where:

metadata

Specifies the name of the hosted cluster and the name of the hosted control plane namespace.

spec.operatorConfiguration.clusterNetworkOperator.ovnKubernetesConfig.ipv4

Specifies the subnets to use. Both subnet fields in this section must be in a valid IPv4 CIDR notation, such as **192.168.1.0/24**. The prefix range is **/0** to **/30**, inclusive. The first octet cannot be 0, and the string length must be 9-18 characters. The subnet fields cannot use the same value. The subnet must be large enough to accommodate one IP address per node in the cluster. When you plan subnet size, consider future cluster growth. If you omit these fields, the default value for the **internalJoinSubnet** field is **100.64.0.0/16**, and the default value for the **internalTransitSwitchSubnet** field is **100.88.0.0/16**.

For full details about creating a hosted cluster, see "Creating a hosted cluster by using the CLI".

- To configure internal OVN subnets in an existing hosted cluster, enter the following command:

```
$ oc patch hostedcluster <hosted_cluster_name> \
  -n <hosted_control_plane_namespace> \
  --type=merge \
  -p '{
    "spec": {
      "operatorConfiguration": {
        "clusterNetworkOperator": {
          "ovnKubernetesConfig": {
            "ipv4": {
              "internalJoinSubnet": "100.99.0.0/16",
              "internalTransitSwitchSubnet": "100.69.0.0/16"
            }
          }
        }
      }
    }
  }'
```

where:

spec.operatorConfiguration.clusterNetworkOperator.ovnKubernetesConfig.ipv4

Specifies the subnets to use. Both subnet fields in this section must be in a valid IPv4 CIDR notation, such as **192.168.1.0/24**. The prefix range is **/0** to **/30**, inclusive. The first octet cannot be 0, and the string length must be 9-18 characters. The subnet fields cannot use the same value. The subnet must be large enough to accommodate one IP address per node in the cluster. When you plan subnet size, consider future cluster growth. If you omit these fields, the default value for the **internalJoinSubnet** field is **100.64.0.0/16**, and the default value for the **internalTransitSwitchSubnet** field is **100.88.0.0/16**.



IMPORTANT

When you make this change to an existing hosted cluster, the **ovnkube-node** DaemonSet is rolled out and the OVN components on compute nodes are restarted. During this process, you might experience brief network disruptions.

Verification

1. Verify that the hosted configuration is correct by entering the following command:


```
$ oc get hostedcluster <hosted_cluster_name> -n <hosted_control_plane_namespace> \
-o jsonpath='{.spec.operatorConfiguration.clusterNetworkOperator.ovnKubernetesConfig}' |
jq .
```

Example output

```
{
  "ipv4": {
    "internalJoinSubnet": "100.99.0.0/16",
    "internalTransitSwitchSubnet": "100.69.0.0/16"
  }
}
```

2. Check the Network Operator configuration in the hosted cluster:

- a. Extract the hosted cluster kubeconfig file by entering the following command:

```
$ oc extract secret/<hosted_cluster_name>-admin-kubeconfig \
-n <hosted_control_plane_namespace> --to=- > <hosted_cluster_kubeconfig_file>
```

- b. Verify the Network Operator configuration by entering the following command:

```
$ oc get network.operator.openshift.io cluster \
--kubeconfig=<hosted_cluster_kubeconfig_file> \
-o jsonpath='{.spec.defaultNetwork.ovnKubernetesConfig.ipv4}' | jq .
```

Example output

```
{
  "internalJoinSubnet": "100.99.0.0/16",
  "internalTransitSwitchSubnet": "100.69.0.0/16"
}
```

3. Create 2 test pods by completing the following steps:

- a. In node 1, create pod 1, as shown in the following example:

```
kind: Pod
apiVersion: v1
metadata:
  name: "<pod_1>"
  namespace: "<hosted_control_plane_namespace>"
  labels:
    name: <pod_name>
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - image: "<image_url>"
      name: <pod_name>
      securityContext:
        allowPrivilegeEscalation: false
```

```
capabilities:
  drop: ["ALL"]
nodeName: "${NODE1}"
```

- b. In node 2, create pod 2, as shown in the following example:

```
kind: Pod
apiVersion: v1
metadata:
  name: "<pod_2>"
  namespace: "<hosted_control_plane_namespace>"
  labels:
    name: <pod_name>
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - image: "<image_url>"
    name: <pod_name>
    securityContext:
      allowPrivilegeEscalation: false
      capabilities:
        drop: ["ALL"]
    nodeName: "${NODE2}"
```

4. Create a test service that backs up both pods, as shown in the following example:

```
kind: Service
apiVersion: v1
metadata:
  name: "<test_service_name>"
  namespace: "<hosted_control_plane_namespace>"
  labels:
    name: test-service
spec:
  internalTrafficPolicy: "Cluster"
  externalTrafficPolicy: ""
  ipFamilyPolicy: "SingleStack"
  ports:
  - name: http
    port: <service_test_port_number>
    protocol: "TCP"
    targetPort: 8080
  selector:
    name: "<pod_name>"
  type: "ClusterIP"
```

5. Verify that the OVN pods are running:

- a. Enter the following command:

```
$ oc rollout status daemonset/ovnkube-node \
-n openshift-ovn-kubernetes \
```

```
--kubeconfig=<hosted_cluster_kubeconfig_file> \
--timeout=5m
```

- b. Enter the following command:

```
$ oc get pods -n openshift-ovn-kubernetes --kubeconfig=
<hosted_cluster_kubeconfig_file>
```

All **ovnkube-node** pods should be in **Running** state with all containers ready.

6. Make sure that the changes synchronized to the Network Operator by entering the following command:

```
$ oc get network.operator.openshift.io/cluster \
-ojsonpath='{.spec.defaultNetwork.ovnKubernetesConfig.ipv4}' \
--kubeconfig=<clusters-hostedclustername> | jq .
```

7. Get the IP address of pod 2 and transfer it from pod 1:

- a. Enter the following command:

```
$ pod2_ip=oc get pod -n e2e-test-networking-ovnkubernetes-xt8s <pod_2> -
o=jsonpath={.status.podIPs[0].ip}
```

- b. Enter the following command:

```
$ oc exec <pod_1> -- /bin/sh -x -c curl --connect-timeout 5 -s <pod2_ip>:8080
```

8. Get the service IP address and verify that the pod can be visited externally from a service:

- a. Enter the following command:

```
$ SERVICE_IP=oc get service test-service-o=jsonpath={.spec.clusterIPs[0]}
```

- b. Enter the following command:

```
$ oc exec <pod_1> -- /bin/sh -x -c curl --connect-timeout 5 -s $SERVICE_IP:
<service_test_port_number>
```

Additional resources

- [Troubleshooting internal subnets for hosted clusters](#)
- [Creating a hosted cluster by using the CLI](#)

14.6. PROXY SUPPORT FOR HOSTED CONTROL PLANES

To ensure that control-plane workloads, compute nodes, management clusters, and hosted clusters have the access they need for optimal performance, you can configure proxy support.

In standalone OpenShift Container Platform, the primary purposes of proxy support are ensuring that workloads in the cluster are configured to use the HTTP or HTTPS proxy to access external services, honoring the **NO_PROXY** setting if one is configured, and accepting any trust bundle that is configured

for the proxy.

In hosted control planes, proxy support includes use cases beyond those in standalone OpenShift Container Platform.

14.6.1. Control plane workloads that need to access external services

Operators that run in the control plane need to access external services through the proxy that is configured for the hosted cluster. The proxy is usually accessible only through the data plane.

The control plane workloads are as follows:

- The Control Plane Operator needs to validate and obtain endpoints from certain identity providers when it creates the OAuth server configuration.
- The OAuth server needs non-LDAP identity provider access.
- The OpenShift API server handles image registry metadata import.
- The Ingress Operator needs access to validate external canary routes.
- You must open the firewall port **53** on Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) to allow the Domain Name Service (DNS) protocol to work as expected.

In a hosted cluster, you must send traffic that originates from the Control Plane Operator, Ingress Operator, OAuth server, and OpenShift API server pods through the data plane to the configured proxy and then to its final destination.



NOTE

Some operations are not possible when a hosted cluster is reduced to zero compute nodes; for example, when you import OpenShift image streams from a registry that requires proxy access.

14.6.2. Compute nodes that need to access an ignition endpoint

When compute nodes need a proxy to access the ignition endpoint, you must configure the proxy in the user-data stub that is configured on the compute node when it is created.

For cases where machines need a proxy to access the ignition URL, the proxy configuration is included in the stub.

The stub resembles the following example:

```
{
  "ignition": {
    "config": {
      "merge": [
        {
          "name": "Authorization",
          "value": "Bearer ...",
          "source": "https://ignition.controlplanehost.example.com/ignition",
          "verification": {}
        },
        {
          "name": "TargetConfigVersionHash",
          "value": "a4c1b0dd",
          "source": "https://ignition.controlplanehost.example.com/ignition",
          "verification": {}
        }
      ],
      "replace": {
        "verification": {}
      },
      "proxy": {
        "httpProxy": "http://proxy.example.org:3128",
        "httpsProxy": "https://proxy.example.org:3129",
        "noProxy": "host.example.org",
        "security": {
          "tls": {
            "certificateAuthorities": [
              {
                "source": "...",
                "verification": {}
              }
            ]
          }
        },
        "timeouts": {},
        "version": "3.2.0",
        "passwd": {},
        "storage": {},
        "systemd": {}
      }
    }
  }
}
```

14.6.3. Compute nodes that need to access the API server

For communication with the control plane, hosted control planes uses a local proxy in every compute node that listens on IP address 172.20.0.1 and forwards traffic to the API server. If an external proxy is required to access the API server, that local proxy needs to use the external proxy to send traffic out.

When a proxy is not needed, hosted control planes uses **haproxy** for the local proxy, which only forwards packets via TCP. When a proxy is needed, hosted control planes uses a custom proxy, **control-plane-operator-kubernetes-default-proxy**, to send traffic through the external proxy.



NOTE

This use case is relevant to self-managed hosted control planes, not to Red Hat OpenShift Service on AWS.

14.6.4. Management clusters that need external access

The HyperShift Operator has a controller that monitors the OpenShift global proxy configuration of the management cluster and sets the proxy environment variables on its own deployment.

Control plane deployments that need external access are configured with the proxy environment variables of the management cluster.

14.6.5. Management cluster with a proxy and a hosted cluster with a secondary network and no default pod network

If a management cluster uses a proxy configuration and you are configuring a hosted cluster with a secondary network but are not attaching the default pod network, add the CIDR of the secondary network to the proxy configuration.

Specifically, you need to add the CIDR of the secondary network to the **noProxy** section of the proxy configuration for the management cluster. Otherwise, the Kubernetes API server will route some API requests through the proxy. In the hosted cluster configuration, the CIDR of the secondary network is automatically added to the **noProxy** section.

Additional resources

- [Troubleshooting internal subnets for hosted clusters](#)
- [Creating a hosted cluster by using the CLI](#)
- [About the OVN-Kubernetes network plugin](#)
- [Configuring the cluster-wide proxy](#)

CHAPTER 15. TROUBLESHOOTING HOSTED CONTROL PLANES

If you encounter an issue with hosted control planes, you can gather information about the hosted cluster, OpenShift Container Platform, or other components so that you can determine the root cause and take steps to resolve it.

15.1. GATHERING INFORMATION TO TROUBLESHOOT HOSTED CONTROL PLANES

When you need to troubleshoot an issue with hosted clusters, you can gather information by running the **must-gather** command. The command generates output for the management cluster and the hosted cluster.

The output for the management cluster contains the following content:

- **Cluster-scoped resources:** These resources are node definitions of the management cluster.
- **The hypershift-dump compressed file:** This file is useful if you need to share the content with other people.
- **Namespaced resources:** These resources include all of the objects from the relevant namespaces, such as config maps, services, events, and logs.
- **Network logs:** These logs include the OVN northbound and southbound databases and the status for each one.
- **Hosted clusters:** This level of output involves all of the resources inside of the hosted cluster.

The output for the hosted cluster contains the following content:

- **Cluster-scoped resources:** These resources include all of the cluster-wide objects, such as nodes and CRDs.
- **Namespaced resources:** These resources include all of the objects from the relevant namespaces, such as config maps, services, events, and logs.

Although the output does not contain any secret objects from the cluster, it can contain references to the names of secrets.

Prerequisites

- You must have **cluster-admin** access to the management cluster.
- You need the **name** value for the **HostedCluster** resource and the namespace where the CR is deployed.
- You must have the **hcp** command-line interface installed. For more information, see "Installing the hosted control planes command-line interface".
- You must have the OpenShift CLI (**oc**) installed.
- You must ensure that the **kubeconfig** file is loaded and is pointing to the management cluster.

Procedure

- To gather the output for troubleshooting, enter the following command:

```
$ oc adm must-gather \
  --image=registry.redhat.io/rhacm2/acm-must-gather-rhel9:v2.17 \
  /usr/bin/gather hosted-cluster-namespace=HOSTEDCLUSTERNAMESPACE \
  hosted-cluster-name=HOSTEDCLUSTERNAME \
  --dest-dir=NAME ; tar -cvzf NAME.tgz NAME
```

where:

- The **hosted-cluster-namespace=HOSTEDCLUSTERNAMESPACE** parameter is optional. If you do not include it, the command runs as though the hosted cluster is in the default namespace, which is **clusters**.
- If you want to save the results of the command to a compressed file, specify the **--dest-dir=NAME** parameter and replace **NAME** with the name of the directory where you want to save the results.

Additional resources

- [Installing the hosted control planes command-line interface](#)

15.2. GATHERING OPENSIFT CONTAINER PLATFORM DATA FOR A HOSTED CLUSTER

You can gather OpenShift Container Platform debugging information for a hosted cluster by using the multicluster engine Operator web console or by using the CLI.

15.2.1. Gathering data for a hosted cluster by using the CLI

You can gather OpenShift Container Platform debugging information for a hosted cluster by using the command-line interface (CLI).

Prerequisites

- You must have **cluster-admin** access to the management cluster.
- You need the **name** value for the **HostedCluster** resource and the namespace where the CR is deployed.
- You must have the **hcp** command-line interface installed. For more information, see "Installing the hosted control planes command-line interface".
- You must have the OpenShift CLI (**oc**) installed.
- You must ensure that the **kubeconfig** file is loaded and is pointing to the management cluster.

Procedure

1. Generate the **kubeconfig** file by entering the following command:

```
$ hcp create kubeconfig --namespace <hosted_cluster_namespace> \
--name <hosted_cluster_name> > <hosted_cluster_name>.kubeconfig
```

2. After you save the **kubeconfig** file, you can access the hosted cluster by entering the following example command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

3. Collect the must-gather information by entering the following command:

```
$ oc adm must-gather
```

15.2.2. Gathering data for a hosted cluster by using the web console

You can gather OpenShift Container Platform debugging information for a hosted cluster by using the multicluster engine Operator web console.

Prerequisites

- You must have **cluster-admin** access to the management cluster.
- You need the **name** value for the **HostedCluster** resource and the namespace where the CR is deployed.
- You must have the **hcp** command-line interface installed. For more information, see "Installing the hosted control planes command-line interface".
- You must have the OpenShift CLI (**oc**) installed.
- You must ensure that the **kubeconfig** file is loaded and is pointing to the management cluster.

Procedure

1. In the web console, select **All Clusters** and select the cluster you want to troubleshoot.
2. In the upper-right corner, select **Download kubeconfig**.
3. Export the downloaded **kubeconfig** file.
4. Collect the must-gather information by entering the following command:

```
$ oc adm must-gather
```

15.3. ENTERING THE MUST-GATHER COMMAND IN A DISCONNECTED ENVIRONMENT

When you need to troubleshoot an issue in a disconnected environment, you can gather information by running the **must-gather** command. The command generates output for the management cluster and the hosted cluster.

Procedure

1. In a disconnected environment, mirror the Red Hat Operator catalog images into their mirror registry. For more information, see "Install on disconnected networks".
2. Run the following command to extract logs that reference the image from their mirror registry:

```

REGISTRY=registry.example.com:5000
IMAGE=$REGISTRY/rhacm2/acm-must-gather-rhel9:v2.17

$ oc adm must-gather \
  --image=$IMAGE /usr/bin/gather \
  hosted-cluster-namespace=HOSTEDCLUSTERNAMESPACE \
  hosted-cluster-name=HOSTEDCLUSTERNAME \
  --dest-dir=./data

```

Additional resources

- [Install on disconnected networks](#)

15.4. TROUBLESHOOTING HOSTED CLUSTERS ON OPENSIFT VIRTUALIZATION

When you troubleshoot a hosted cluster on OpenShift Virtualization, start with the top-level **HostedCluster** and **NodePool** resources and then work down the stack until you find the root cause. The following steps can help you discover the root cause of common issues.

15.4.1. Troubleshooting HostedCluster resource stuck in a partial state

If a hosted control plane is not coming fully online because a **HostedCluster** resource is pending, identify the problem by checking prerequisites, resource conditions, and node and Operator status.

Procedure

- Ensure that you meet all of the prerequisites for a hosted cluster on OpenShift Virtualization.
- View the conditions on the **HostedCluster** and **NodePool** resources for validation errors that prevent progress.
- By using the **kubeconfig** file of the hosted cluster, inspect the status of the hosted cluster:
 - View the output of the **oc get clusteroperators** command to see which cluster Operators are pending.
 - View the output of the **oc get nodes** command to ensure that worker nodes are ready.

15.4.2. Identifying why no compute nodes are registered

If a hosted control plane is not coming fully online because the hosted control plane has no compute nodes registered, identify the problem by checking the status of various parts of the hosted control plane.

Procedure

- View the **HostedCluster** and **NodePool** conditions for failures that indicate what the problem might be.
- Enter the following command to view the KubeVirt compute node virtual machine (VM) status for the **NodePool** resource:

```
$ oc get vm -n <namespace>
```

- If the VMs are stuck in the provisioning state, enter the following command to view the CDI import pods within the VM namespace for clues about why the importer pods have not completed:

```
$ oc get pods -n <namespace> | grep "import"
```

- If the VMs are stuck in the starting state, enter the following command to view the status of the virt-launcher pods:

```
$ oc get pods -n <namespace> -l kubevirt.io=virt-launcher
```

If the virt-launcher pods are in a pending state, investigate why the pods are not being scheduled. For example, not enough resources might exist to run the virt-launcher pods.

- If the VMs are running but they are not registered as compute nodes, use the web console to gain VNC access to one of the affected VMs. The VNC output indicates whether the ignition configuration was applied. If a VM cannot access the hosted control plane ignition server on startup, the VM cannot be provisioned correctly.
- If the ignition configuration was applied but the VM is still not registering as a node, see "Identifying the problem: Access the VM console logs" to learn how to access the VM console logs during startup.

Additional resources

- [Identifying the problem: Access the VM console logs](#)

15.4.3. Identifying why compute nodes are not ready

During cluster creation, nodes enter the **NotReady** state temporarily while the networking stack is rolled out. This part of the process is normal. However, if this part of the process takes longer than 15 minutes, identify the problem by investigating the node object and pods.

Procedure

1. Enter the following command to view the conditions on the node object and determine why the node is not ready:

```
$ oc get nodes -o yaml
```

2. Enter the following command to look for failing pods within the cluster:

```
$ oc get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded
```

15.4.4. Identifying why ingress and console cluster Operators are not coming online

If a hosted control plane is not coming fully online because the Ingress and console cluster Operators are not online, check the wildcard DNS routes and load balancer.

Procedure

- If the cluster uses the default Ingress behavior, enter the following command to ensure that wildcard DNS routes are enabled on the OpenShift Container Platform cluster that the virtual machines (VMs) are hosted on:

```
$ oc patch ingresscontroller -n openshift-ingress-operator \
  default --type=json -p \
  ' [{ "op": "add", "path": "/spec/routeAdmission", "value": {wildcardPolicy:
  "WildcardsAllowed"}} ]'
```

- If you use a custom base domain for the hosted control plane, complete the following steps:
 - Ensure that the load balancer is targeting the VM pods correctly.
 - Ensure that the wildcard DNS entry is targeting the load balancer IP address.

15.4.5. Identifying why load balancer services for the hosted cluster are unavailable

If a hosted control plane is not coming fully online because the load balancer services are not becoming available, check events, details, and the Kubernetes Cluster Configuration Manager (KCCM) pod.

Procedure

- Look for events and details that are associated with the load balancer service within the hosted cluster.
- By default, load balancers for the hosted cluster are handled by the `kubevirt-cloud-controller-manager` within the hosted control plane namespace. Ensure that the KCCM pod is online and view its logs for errors or warnings. To identify the KCCM pod in the hosted control plane namespace, enter the following command:

```
$ oc get pods -n <hosted_control_plane_namespace> \
  -l app=cloud-controller-manager
```

15.4.6. Identifying why hosted cluster PVCs are not available

If a hosted control plane is not coming fully online because the persistent volume claims (PVCs) for a hosted cluster are not available, check the PVC events and details, and component logs.

Procedure

- Look for events and details that are associated with the PVC to understand which errors are occurring.
- If a PVC is failing to attach to a pod, view the logs for the `kubevirt-csi-node` **daemonset** component within the hosted cluster to further investigate the problem. To identify the `kubevirt-csi-node` pods for each node, enter the following command:

```
$ oc get pods -n openshift-cluster-csi-drivers -o wide \
  -l app=kubevirt-csi-driver
```

- If a PVC cannot bind to a persistent volume (PV), view the logs of the kubevirt-csi-controller component within the hosted control plane namespace. To identify the kubevirt-csi-controller pod within the hosted control plane namespace, enter the following command:

```
$ oc get pods -n <hcp namespace> -l app=kubevirt-csi-driver
```

15.4.7. Identifying why VM nodes are not joining the cluster

If a hosted control plane is not coming fully online because the virtual machine (VM) nodes are not correctly joining the cluster, access the VM console logs.

Procedure

- To access the VM console logs, complete the steps in "How to get serial console logs for VMs part of OpenShift Virtualization Hosted Control Plane clusters".

Additional resources

- [How to get serial console logs for VMs part of OpenShift Virtualization Hosted Control Plane clusters \(Red Hat Knowledgebase\)](#)

15.4.8. Resolving RHCOS image mirroring failures

For hosted control planes on OpenShift Virtualization in a disconnected environment, if **oc-mirror** fails to automatically mirror the Red Hat Enterprise Linux CoreOS (RHCOS) image to the internal registry, you can manually mirror the RHCOS image to the internal registry.

When you create your first hosted cluster, the Kubevirt virtual machine does not boot, because the boot image is not available in the internal registry.

To resolve this issue, manually mirror the RHCOS image to the internal registry.

Procedure

1. Get the internal registry name by running the following command:

```
$ oc get imagecontentsourcepolicy -o json \
  | jq -r '.items[].spec.repositoryDigestMirrors[0].mirrors[0]'
```

2. Get a payload image by running the following command:

```
$ oc get clusterversion version -ojsonpath='{.status.desired.image}'
```

3. Extract the **0000_50_installer_coreos-bootimages.yaml** file that contains boot images from your payload image on the hosted cluster. Replace **<payload_image>** with the name of your payload image. Run the following command:

```
$ oc image extract \
  --file /release-manifests/0000_50_installer_coreos-bootimages.yaml \
  <payload_image> --confirm
```

4. Get the RHCOS image by running the following command:

```
$ cat 0000_50_installer_coreos-bootimages.yaml | yq -r .data.stream \
| jq -r '.architectures.x86_64.images.kubvirt."digest-ref"'
```

5. Mirror the RHCOS image to your internal registry by running the following command:

```
$ oc image mirror <rhcos_image> <internal_registry>
```

- Replace **<rhcos_image>** with your RHCOS image; for example, **quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:d9643ead36b1c026be664c9c65c11433c6cdf71bfd93ba229141d134a4a6dd94**.
 - Replace **<internal_registry>** with the name of your internal registry; for example, **virthost.ostest.test.metakube.org:5000/localimages/ocp-v4.0-art-dev**.
6. Create a YAML file named **rhcos-boot-kubvirt.yaml** that defines the **ImageDigestMirrorSet** object. See the following example configuration:

```
apiVersion: config.openshift.io/v1
kind: ImageDigestMirrorSet
metadata:
  name: rhcos-boot-kubvirt
spec:
  repositoryDigestMirrors:
  - mirrors:
    - virthost.ostest.test.metakube.org:5000/localimages/ocp-v4.0-art-dev
    source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
```

- **spec.repositoryDigestMirrors.mirrors** specifies the name of your internal registry.
 - **spec.repositoryDigestMirrors.source** specifies your RHCOS image without its digest.
7. Apply the **rhcos-boot-kubvirt.yaml** file to create the **ImageDigestMirrorSet** object by running the following command:

```
$ oc apply -f rhcos-boot-kubvirt.yaml
```

15.4.9. Returning non-bare-metal clusters to the late binding pool

If you are using late binding managed clusters without **BareMetalHosts**, you must complete additional manual steps to delete a late binding cluster and return the nodes back to the Discovery ISO.

For late binding managed clusters without **BareMetalHosts**, removing cluster information does not automatically return all nodes to the Discovery ISO.

To unbind the non-bare-metal nodes with late binding, complete the following steps.

Procedure

1. Remove the cluster information. For more information, see "Removing a cluster from management".
2. Clean the root disks.

3. Reboot manually with the Discovery ISO.

Additional resources

- [Removing a cluster from management](#)

15.5. TROUBLESHOOTING HOSTED CLUSTERS ON BARE METAL

If you encounter issues with hosted control planes on bare metal, review the troubleshooting procedures to diagnose and resolve them.

15.5.1. Determining why nodes are not added to a hosted cluster on bare metal

When you scale up a hosted cluster with nodes that were provisioned by using Assisted Installer, the host fails to pull the ignition with a URL that contains port **22642**. That URL is invalid for hosted control planes and indicates that an issue exists with the cluster.

Procedure

1. To determine the issue, review the assisted-service logs by entering the following command:

```
$ oc logs -n multicluster-engine <assisted_service_pod_name>
```

Replace **<assisted_service_pod_name>** with the Assisted Service pod name.

2. In the logs, find errors that resemble these examples:

```
error="failed to get pull secret for update: invalid pull secret data in secret pull-secret"
```

```
pull secret must contain auth for \"registry.redhat.io\"
```

3. To fix this issue, see "Add the pull secret to the namespace" in the multicluster engine for Kubernetes Operator documentation.



NOTE

To use hosted control planes, you must have multicluster engine Operator installed, either as a standalone Operator or as part of Red Hat Advanced Cluster Management. Because the Operator has a close association with Red Hat Advanced Cluster Management, the documentation for the Operator is published within that product's documentation. Even if you do not use Red Hat Advanced Cluster Management, the parts of its documentation that cover multicluster engine Operator are relevant to hosted control planes.

Additional resources

- [Add the pull secret to the namespace](#)

15.6. RESTARTING HOSTED CONTROL PLANE COMPONENTS

If you are an administrator for hosted control planes, you can use the **hypershift.openshift.io/restart-date** annotation to restart all control plane components for a particular **HostedCluster** resource.

For example, you might need to restart control plane components for certificate rotation.

Procedure

- To restart a control plane, annotate the **HostedCluster** resource by entering the following command:

```
$ oc annotate hostedcluster \
  -n <hosted_cluster_namespace> \
  <hosted_cluster_name> \
  hypershift.openshift.io/restart-date=$(date --iso-8601=seconds)
```

The control plane is restarted whenever the value of the annotation changes. The **date** command serves as the source of a unique string. The annotation is treated as a string, not a timestamp.

Verification

After you restart a control plane, the following hosted control planes components are typically restarted:



NOTE

You might see some additional components restarting as a side effect of changes implemented by the other components.

- catalog-operator
- certified-operators-catalog
- cluster-api
- cluster-autoscaler
- cluster-policy-controller
- cluster-version-operator
- community-operators-catalog
- control-plane-operator
- hosted-cluster-config-operator
- ignition-server
- ingress-operator
- konnectivity-agent
- konnectivity-server
- kube-apiserver
- kube-controller-manager
- kube-scheduler

- machine-approver
- oauth-openshift
- olm-operator
- openshift-apiserver
- openshift-controller-manager
- openshift-oauth-apiserver
- packageserver
- redhat-marketplace-catalog
- redhat-operators-catalog

15.7. PAUSING THE RECONCILIATION OF A HOSTED CLUSTER AND HOSTED CONTROL PLANE

If you are a cluster instance administrator, you can pause the reconciliation of a hosted cluster and hosted control plane. You might want to pause reconciliation when you back up and restore an etcd database or when you need to debug problems with a hosted cluster or hosted control plane.

Procedure

1. To pause reconciliation for a hosted cluster and hosted control plane, populate the **pausedUntil** field of the **HostedCluster** resource.

- To pause the reconciliation until a specific time, enter the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedclusters/<hosted_cluster_name> \
  -p '{"spec":{"pausedUntil":"<timestamp>"}}' \
  --type=merge
```

Replace **<timestamp>** with a timestamp in the RFC339 format; for example, **2024-03-03T03:28:48Z**. The reconciliation is paused until the specified time is passed.

- To pause the reconciliation indefinitely, enter the following command:

```
$ oc patch -n <hosted_cluster_namespace> \
  hostedclusters/<hosted_cluster_name> \
  -p '{"spec":{"pausedUntil":"true"}}' \
  --type=merge
```

The reconciliation is paused until you remove the field from the **HostedCluster** resource.

When the pause reconciliation field is populated for the **HostedCluster** resource, the field is automatically added to the associated **HostedControlPlane** resource.

2. To remove the **pausedUntil** field, enter the following patch command:

```
$ oc patch -n <hosted_cluster_namespace> \
```



```
hostedclusters/<hosted_cluster_name> \
-p '{"spec":{"pausedUntil":null}}' \
--type=merge
```

15.8. SCALING DOWN THE DATA PLANE TO ZERO

If you are not using the hosted control plane, to save the resources and cost you can scale down a data plane to zero.



NOTE

Ensure you are prepared to scale down the data plane to zero. Because the workload from the worker nodes disappears after scaling down.

Procedure

1. Set the **kubeconfig** file to access the hosted cluster by running the following command:

```
$ export KUBECONFIG=<install_directory>/auth/kubeconfig
```

2. Get the name of the **NodePool** resource associated to your hosted cluster by running the following command:

```
$ oc get nodepool --namespace <hosted_cluster_namespace>
```

3. Optional: To prevent the pods from draining, add the **nodeDrainTimeout** field in the **NodePool** resource by running the following command:

```
$ oc edit nodepool <nodepool_name> --namespace <hosted_cluster_namespace>
```

Example output

```
apiVersion: hypershift.openshift.io/v1alpha1
kind: NodePool
metadata:
# ...
  name: nodepool-1
  namespace: clusters
# ...
spec:
  arch: amd64
  clusterName: clustername
  management:
    autoRepair: false
    replace:
      rollingUpdate:
        maxSurge: 1
        maxUnavailable: 0
      strategy: RollingUpdate
    upgradeType: Replace
  nodeDrainTimeout: 0s
  nodeVolumeDetachTimeout: 0
# ...
```

spec.arch.clusterName

Defines the name of your hosted cluster.

spec.nodeDrainTimeout

Specifies the total amount of time that the controller spends to drain a node. By default, the **nodeDrainTimeout: 0s** setting blocks the node draining process. To allow the node draining process to continue for a certain period of time, you can set the value of the **nodeDrainTimeout** field; for example, **nodeDrainTimeout: 1m**.

spec.nodeVolumeDetachTimeout

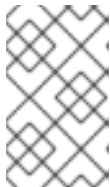
Specifies the total amount of time that the controller spends detaching volumes from a node. By default, the **0** setting blocks the volume detachment process.

**NOTE**

To prevent nodes from getting stuck when scaling down, set the **.spec.nodeDrainTimeout** and **.spec.nodeVolumeDetachTimeout** in the **NodePool** resource to a value greater than **0**. This setting forces nodes to be removed after the timeout specified in the field is reached, regardless of whether the node can be drained or the volumes can be detached.

4. Scale down the **NodePool** resource associated to your hosted cluster by running the following command:

```
$ oc scale nodepool/<nodepool_name> --namespace <hosted_cluster_namespace> \
--replicas=0
```

**NOTE**

After scaling down the data plan to zero, some pods in the control plane stay in the **Pending** status and the hosted control plane stays up and running. If necessary, you can scale up the **NodePool** resource.

5. Optional: Scale up the **NodePool** resource associated to your hosted cluster by running the following command:

```
$ oc scale nodepool/<nodepool_name> --namespace <hosted_cluster_namespace> --
replicas=1
```

After rescaling the **NodePool** resource, wait for couple of minutes for the **NodePool** resource to become available in a **Ready** state.

Verification

- Verify that the value for the **nodeDrainTimeout** field is greater than **0s** by running the following command:

```
$ oc get nodepool -n <hosted_cluster_namespace> <nodepool_name> -
ojsonpath='{.spec.nodeDrainTimeout}'
```

15.9. RESOLVING AGENT SERVICE FAILURES FOR HOSTED CONTROL PLANES ON IBM Z

In some cases, agents might fail to join the cluster after booting the machines with the boot artifacts.

You can confirm this issue by checking the **agent.service** logs for the following error:

Error: copying system image from manifest list: Source image rejected: A signature was required, but no signature exists

This issue occurs because image signature verification fails when no signature is present. As a workaround, you can disable signature verification by modifying the container policy.

Procedure

1. Add the **ignitionConfigOverride** field in the **InfraEnv** manifest to override the **/etc/containers/policy.json** file. This disables signature verification for container images.
2. Replace the base64-encoded content in the **ignitionConfigOverride** with the required **/etc/containers/policy.json** configuration according to your image registries. See the following example:

Example

```
{
  "default": [
    {
      "type": "insecureAcceptAnything"
    }
  ],
  "transports": {
    "docker": {
      "<REGISTRY1>": [
        {
          "type": "insecureAcceptAnything"
        }
      ],
      "REGISTRY2": [
        {
          "type": "insecureAcceptAnything"
        }
      ]
    },
    "docker-daemon": {
      "": [
        {
          "type": "insecureAcceptAnything"
        }
      ]
    }
  ]
}
```

Example **InfraEnv** manifest with **ignitionConfigOverride**

```
apiVersion: agent-install.openshift.io/v1beta1
kind: InfraEnv
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_control_plane_namespace>
spec:
  cpuArchitecture: s390x
  pullSecretRef:
    name: pull-secret
  sshAuthorizedKey: <ssh_public_key>
  ignitionConfigOverride: '{"ignition":{"version":"3.2.0"},"storage":{"files":
[{"path":"/etc/containers/policy.json","mode":420,"overwrite":true,"contents":
{"source":"data:text/plain;charset=utf-
8;base64,ewogICAgImRlZmF1bHQiOiBbCiAgICAglCAgIAGewogICAgICAglCAglCAidHlwZSI6ICJ
pbNlYy3VyZUFjY2VwdEFueXRoaWw5nlhogICAgICAglIH0KICAgIF0sCiAgICAidHJhbnNwb3J0cy
l6CiAgICAglCAgewogICAgICAglCAglCAiZG9ja2VyLWRhZW1vbil6CiAgICAglCAglCAglCAglC
B7CiAgICAglCAglCAglCAglCAglCAglil6lFt7lnR5cGU OiJpbNlYy3VyZUFjY2VwdEFueXRoaW
5nIn1dCiAgICAglCAglCAglCAglCB9CiAgICAglCAglCAgfQp9"}]]}}'
```

15.10. KNOWN LIMITATIONS FOR INTERNAL SUBNETS FOR HOSTED CLUSTERS

Several known limitations exist for internal subnets on hosted clusters.

- IPv6 subnets are not supported.
- The hosted control planes command-line interface, **hcp**, might not have native flags for the subnet fields. Manual YAML editing or **oc patch** is required.
- Modifying OVN subnets after you create a cluster triggers a rollout of OVN components, which might cause brief network disruptions.
- You cannot modify OVN subnet configuration while a cluster update is in progress or scheduled.

15.10.1. Configuring the ovnKubernetesConfig object fails with an error

When you try to configure the **ovnKubernetesConfig** object on a hosted cluster by using a different network type, such as **OpenShiftSDN**, an error occurs because hosted control planes works only with the **OVNKubernetes** network type.

Procedure

- Verify the network type of your hosted cluster by entering the following command:

```
$ oc get hostedcluster <hosted_cluster_name> -n <hosted_control_plane_namespace> \
-o jsonpath='{.spec.networking.networkType}'
```

15.10.2. Setting CIDR values in internal subnet fields

If the **internalJoinSubnet** field and the **internalTransitSwitchSubnet** field are set to the same classless inter-domain routing (CIDR) values, an error occurs.

Procedure

- Use different subnets for each field, as shown in the following example:

```
apiVersion: hypershift.openshift.io/v1beta1
kind: HostedCluster
metadata:
  # ...
spec:
  #...
  operatorConfiguration:
    clusterNetworkOperator:
      ovnKubernetesConfig:
        ipv4:
          internalJoinSubnet: "100.99.0.0/16"
          internalTransitSwitchSubnet: "100.69.0.0/16"
  # ...
```

15.10.3. Ensuring a valid IPv4 CIDR format

If you do not specify subnets in a valid classless inter-domain range (CIDR) format, an error occurs.

Procedure

- Ensure that the CIDR format follows the following format:

```
X.X.X.X/Y
```

where:

X

is a value from **0** to **255**. The first octet must not be **0**.

Y

is a value from **0** to **30**.

Valid examples

```
100.99.0.0/16
192.168.1.0/24
```

Invalid examples

```
100.99.0.0
256.1.1.0/16
0.99.0.0/16
```

15.10.4. Avoiding an overlap between OVN subnets and CIDR values

If the configured OVN subnets overlap with the machine classless inter-domain routing (CIDR), service CIDR, cluster network CIDR, or with each other, an error occurs.

Procedure

- Use subnets that do not overlap with any network CIDR. You can use a CIDR calculator to verify that no overlaps exist.

Example of configuration with no overlaps

```
spec:
  networking:
    machineCIDR: 10.0.0.0/16
    serviceCIDR: 172.30.0.0/16
    clusterNetwork:
      - cidr: 10.128.0.0/14

  operatorConfiguration:
    clusterNetworkOperator:
      ovnKubernetesConfig:
        ipv4:
          internalJoinSubnet: "100.99.0.0/16"
          internalTransitSwitchSubnet: "100.69.0.0/16"
```

15.10.5. Resolving a stuck OVN rollout

After you change an existing configuration, the OVN component rollout might take a long time or encounter issues.

Procedure

1. Check the status of the **ovnkube-node** DaemonSet rollout by entering the following command:

```
$ oc rollout status daemonset/ovnkube-node \
  -n openshift-ovn-kubernetes \
  --kubeconfig=hosted-kubeconfig
```

2. Check the pod logs for errors by entering the following command:

```
$ oc logs -n openshift-ovn-kubernetes \
  -l app=ovnkube-node \
  --kubeconfig=hosted-kubeconfig
```

If the rollout is stuck, you might need to revert the configuration change.

15.11. TROUBLESHOOTING CONNECTIVITY FOR HOSTED CONTROL PLANES

By using connectivity metrics, you can diagnose whether any issues are related to connectivity from a hosted control plane to a data plane.

15.11.1. Troubleshooting connectivity from the control plane to the data plane

To diagnose connectivity issues from a hosted control plane to the compute nodes in a data plane, check the status of the **DataPlaneConnectionAvailable** condition.

If the status of the **DataPlaneConnectionAvailable** condition is **True**, the control plane can successfully reach the data plane nodes through the **konnektivity-agent** pods. If the status is **False**, take the following steps to determine why the control plane cannot reach the data plane.

Procedure

1. Check the network policies that might block the **Konnektivity** service traffic.
2. Review the firewall rules between the control plane and the data plane.
3. View the status of the **konnektivity-agent** pods in the data plane.
4. In the control plane, review the **Konnektivity** server deployment.

Additional resources

- [Connectivity monitoring for hosted control planes](#)

CHAPTER 16. DESTROYING A HOSTED CLUSTER

16.1. DESTROYING A HOSTED CLUSTER ON AWS

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.1.1. Destroying a hosted cluster on AWS by using the CLI

To delete a hosted cluster and its managed cluster resource from hosted control planes on Amazon Web Services (AWS), use the command-line interface (CLI).

Procedure

1. Delete the managed cluster resource on multicluster engine Operator by running the following command:

```
$ oc delete managedcluster <hosted_cluster_name>
```

Replace **<hosted_cluster_name>** with the name of your cluster.

2. Delete the hosted cluster and its backend resources by running the following command:

```
$ hcp destroy cluster aws \
  --name <hosted_cluster_name> \
  --infra-id <infra_id> \
  --role-arn <arn_role> \
  --sts-creds <path_to_sts_credential_file> \
  --base-domain <base_domain>
```

where:

- **<hosted_cluster_name>** specifies the name of your hosted cluster, for example, **my-hosted-cluster**.
- **<infra_id>** specifies the infrastructure name for your hosted cluster.
- **<arn_role>** specifies the Amazon Resource Name (ARN), for example, **arn:aws:iam::820196288204:role/myrole**.
- **<path_to_sts_credential_file>** specifies the path to your AWS Security Token Service (STS) credentials file, for example, **/home/user/sts-creds/sts-creds.json**.
- **<base_domain>** specifies your base domain, for example, **example.com**.



IMPORTANT

If your session token for AWS Security Token Service (STS) is expired, retrieve the STS credentials in a JSON file named **sts-creds.json** by running the following command:

```
$ aws sts get-session-token --output json > sts-creds.json
```


16.2. DESTROYING A HOSTED CLUSTER ON BARE METAL

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.2.1. Destroying a hosted cluster on bare metal by using the CLI

If you created a hosted cluster by using the command-line interface (CLI), you can destroy that hosted cluster and its back-end resources by running a command.

Procedure

- Delete the hosted cluster and its back-end resources by running the following command:

```
$ oc delete -f <hosted_cluster_config>.yaml
```

Specify the name of the configuration YAML file that was rendered when you created the hosted cluster.



NOTE

If you created the hosted cluster without specifying the **--render** and **--render-sensitive** flags in its configuration file, you must remove its back-end resources manually.

16.2.2. Destroying a hosted cluster on bare metal by using the web console

You can use the multicluster engine Operator web console to destroy a hosted cluster on bare metal.

Procedure

1. In the console, click **Infrastructure** → **Clusters**.
2. On the **Clusters** page, select the cluster that you want to destroy.
3. In the **Actions** menu, select **Destroy clusters** to remove the cluster.

16.3. DESTROYING A HOSTED CLUSTER ON OPENSIFT VIRTUALIZATION

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.3.1. Destroying a hosted cluster on OpenShift Virtualization by using the CLI

You can use the command-line interface (CLI) to destroy a hosted cluster and its managed cluster resource on OpenShift Virtualization.

Procedure

1. Delete the managed cluster resource on multicluster engine Operator by running the following command:



```
$ oc delete managedcluster <hosted_cluster_name>
```

2. Delete the hosted cluster and its backend resources by running the following command:

```
$ hcp destroy cluster kubevirt --name <hosted_cluster_name>
```

16.4. DESTROYING A HOSTED CLUSTER ON IBM Z

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.4.1. Destroying a hosted cluster on x86 bare metal with IBM Z compute nodes

To destroy a hosted cluster and its managed cluster on **x86** bare metal with IBM Z® compute nodes, you can use the command-line interface (CLI).

Procedure

1. Scale the **NodePool** object to **0** nodes by running the following command:

```
$ oc -n <hosted_cluster_namespace> scale nodepool <nodepool_name> \
  --replicas 0
```

After the **NodePool** object is scaled to **0**, the compute nodes are detached from the hosted cluster. In OpenShift Container Platform version 4.17, this function is applicable only for IBM Z with KVM. For z/VM and LPAR, you must delete the compute nodes manually.

If you want to re-attach compute nodes to the cluster, you can scale up the **NodePool** object with the number of compute nodes that you want. For z/VM and LPAR to reuse the agents, you must re-create them by using the **Discovery** image.



IMPORTANT

If the compute nodes are not detached from the hosted cluster or are stuck in the **Notready** state, delete the compute nodes manually by running the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig delete \
  node <compute_node_name>
```

If you are using an OSA network device in Processor Resource/Systems Manager (PR/SM) mode, auto scaling is not supported. You must delete the old agent manually and scale up the node pool because the new agent joins during the scale down process.

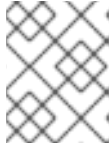
2. Verify the status of the compute nodes by entering the following command:

```
$ oc --kubeconfig <hosted_cluster_name>.kubeconfig get nodes
```

After the compute nodes are detached from the hosted cluster, the status of the agents is changed to **auto-assign**.

3. Delete the agents from the cluster by running the following command:

```
$ oc -n <hosted_control_plane_namespace> delete agent <agent_name>
```



NOTE

You can delete the virtual machines that you created as agents after you delete the agents from the cluster.

4. Destroy the hosted cluster by running the following command:

```
$ hcp destroy cluster agent --name <hosted_cluster_name> \
  --namespace <hosted_cluster_namespace>
```

16.5. DESTROYING A HOSTED CLUSTER ON IBM POWER

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.5.1. Destroying a hosted cluster on IBM Power by using the CLI

To destroy a hosted cluster on IBM Power, you can use the `hcp` command-line interface (CLI).

Procedure

- Delete the hosted cluster by running the following command:

```
$ hcp destroy cluster agent
  --name=<hosted_cluster_name> \
  --namespace=<hosted_cluster_namespace> \
  --cluster-grace-period <duration>
```

- **<hosted_cluster_name>** specifies the name of your hosted cluster.
- **<hosted_cluster_namespace>** specifies the name of your hosted cluster namespace.
- **<duration>** specifies the duration to destroy the hosted cluster completely, for example, **20m0s**.

16.6. DESTROYING A HOSTED CONTROL PLANE ON OPENSTACK

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.6.1. Destroying a hosted cluster by using the CLI

You can destroy a hosted cluster and its associated resources on Red Hat OpenStack Platform (RHOSP) by using the **hcp** CLI tool.

Prerequisites

- You installed the hosted control planes CLI, **hcp**.

Procedure

- To destroy the cluster and its associated resources, run the following command:

```
$ hcp destroy cluster openstack --name=<cluster_name>
```

Replace **<cluster_name>** with the name of the hosted cluster.

After the process completes, your cluster and all resources that are associated with it are destroyed.

16.7. DESTROYING A HOSTED CLUSTER ON NON-BARE-METAL AGENT MACHINES

You might want to remove a hosted cluster if you are no longer using it, you are trying to reduce resources, or the hosted cluster is experiencing issues that are difficult to resolve.

16.7.1. Destroying a hosted cluster on non-bare-metal agent machines

You can use the **hcp** command-line interface (CLI) to destroy a hosted cluster on non-bare-metal agent machines.

Procedure

- Delete the hosted cluster and its backend resources by running the following command:

```
$ hcp destroy cluster agent --name <hosted_cluster_name>
```

Replace **<hosted_cluster_name>** with the name of your hosted cluster.

16.7.2. Destroying a hosted cluster on non-bare-metal agent machines by using the web console

You can use the multicluster engine Operator web console to destroy a hosted cluster on non-bare-metal agent machines.

Procedure

1. In the console, click **Infrastructure** → **Clusters**.
2. On the **Clusters** page, select the cluster that you want to destroy.
3. In the **Actions** menu, select **Destroy clusters** to remove the cluster.

CHAPTER 17. MANUALLY IMPORTING A HOSTED CLUSTER

Hosted clusters are automatically imported into multicluster engine Operator after the hosted control plane becomes available.

17.1. LIMITATIONS OF MANAGING IMPORTED HOSTED CLUSTERS

Hosted clusters are automatically imported into the local multicluster engine for Kubernetes Operator, unlike a standalone OpenShift Container Platform or third-party clusters. Hosted clusters run some of their agents in the *hosted mode* so that the agents do not use the resources of your cluster.

If you choose to automatically import hosted clusters, you can update node pools and the control plane in hosted clusters by using the **HostedCluster** resource on the management cluster. To update node pools and a control plane, see "Updating node pools in a hosted cluster" and "Updating a control plane in a hosted cluster".

You can import hosted clusters into a location other than the local multicluster engine Operator by using the Red Hat Advanced Cluster Management (RHACM). For more information, see "Discovering multicluster engine for Kubernetes Operator hosted clusters in Red Hat Advanced Cluster Management".

In this topology, you must update your hosted clusters by using the command-line interface or the console of the local multicluster engine for Kubernetes Operator where the cluster is hosted. You cannot update the hosted clusters through the RHACM hub cluster.

Additional resources

- [Updating node pools in a hosted cluster](#)
- [Updating a control plane in a hosted cluster](#)
- [Discovering multicluster engine for Kubernetes Operator hosted clusters in Red Hat Advanced Cluster Management](#)

17.2. MANUALLY IMPORTING HOSTED CLUSTERS

Typically, hosted clusters are automatically imported to multicluster engine Operator after the hosted control plane becomes available. However, you can import hosted clusters manually as required.

Procedure

1. In the console, click **Infrastructure** → **Clusters** and select the hosted cluster that you want to import.
2. Click **Import hosted cluster**.



NOTE

For your *discovered* hosted cluster, you can also import from the console, but the cluster must be in an upgradeable state. Import on your cluster is disabled if the hosted cluster is not in an upgradeable state because the hosted control plane is not available. Click **Import** to begin the process. The status is **Importing** while the cluster receives updates and then changes to **Ready**.

17.3. MANUALLY IMPORTING A HOSTED CLUSTER ON AWS

You can import a hosted cluster on Amazon Web Services (AWS) with the command-line interface.

Procedure

1. Create your **ManagedCluster** resource by using the following sample YAML file:

```
apiVersion: cluster.open-cluster-management.io/v1
kind: ManagedCluster
metadata:
  annotations:
    import.open-cluster-management.io/hosting-cluster-name: local-cluster
    import.open-cluster-management.io/klusterlet-deploy-mode: Hosted
    open-cluster-management/created-via: hypershift
  labels:
    cloud: auto-detect
    cluster.open-cluster-management.io/clusterset: default
    name: <hosted_cluster_name>
    vendor: OpenShift
    name: <hosted_cluster_name>
spec:
  hubAcceptsClient: true
  leaseDurationSeconds: 60
```

Replace **<hosted_cluster_name>** with the name of your hosted cluster.

2. Run the following command to apply the resource:

```
$ oc apply -f <file_name>
```

Replace **<file_name>** with the YAML file name you created in the previous step.

3. If you have Red Hat Advanced Cluster Management installed, create your **KlusterletAddonConfig** resource by using the following sample YAML file. If you have installed multicluster engine Operator only, skip this step:

```
apiVersion: agent.open-cluster-management.io/v1
kind: KlusterletAddonConfig
metadata:
  name: <hosted_cluster_name>
  namespace: <hosted_cluster_namespace>
spec:
  clusterName: <hosted_cluster_name>
  clusterNamespace: <hosted_cluster_namespace>
  clusterLabels:
    cloud: auto-detect
    vendor: auto-detect
  applicationManager:
    enabled: true
  certPolicyController:
    enabled: true
  iamPolicyController:
    enabled: true
  policyController:
```

```

enabled: true
searchCollector:
  enabled: false

```

- Replace **<hosted_cluster_name>** with the name of your hosted cluster.
- Replace **<hosted_cluster_namespace>** with the name of your hosted cluster namespace.

4. Run the following command to apply the resource:

```
$ oc apply -f <file_name>
```

Replace **<file_name>** with the name of the YAML that you created in the previous step.

5. After the import process is complete, your hosted cluster becomes visible in the console. You can also check the status of your hosted cluster by running the following command:

```
$ oc get managedcluster <hosted_cluster_name>
```

17.4. DISABLING THE AUTOMATIC IMPORT OF HOSTED CLUSTERS INTO MULTICLUSTER ENGINE OPERATOR

Hosted clusters are automatically imported into multicluster engine Operator after the control plane becomes available. If needed, you can disable the automatic import of hosted clusters.

Any hosted clusters that were previously imported are not affected, even if you disable automatic import. When you upgrade to multicluster engine Operator 2.5 and automatic import is enabled, all hosted clusters that are not imported are automatically imported if their control planes are available.



NOTE

If Red Hat Advanced Cluster Management is installed, all Red Hat Advanced Cluster Management add-ons are also enabled.

When automatic import is disabled, only newly created hosted clusters are not automatically imported. Hosted clusters that were already imported are not affected. You can still manually import clusters by using the console or by creating the **ManagedCluster** and **KlusterletAddonConfig** custom resources.

Procedure

1. On the hub cluster, open the **hypershift-addon-deploy-config** specification that is in the **AddonDeploymentConfig** resource in the namespace where multicluster engine Operator is installed by entering the following command:

```
$ oc edit addondeploymentconfig hypershift-addon-deploy-config \
-n multicluster-engine
```

2. In the **spec.customizedVariables** section, add the **autoImportDisabled** variable with value of **"true"**, as shown in the following example:

```

apiVersion: addon.open-cluster-management.io/v1alpha1
kind: AddOnDeploymentConfig
metadata:

```

```
name: hypershift-addon-deploy-config
namespace: multicluster-engine
spec:
  customizedVariables:
    - name: hcMaxNumber
      value: "80"
    - name: hcThresholdNumber
      value: "60"
    - name: autoImportDisabled
      value: "true"
```

3. To re-enable automatic import, set the value of the **autoImportDisabled** variable to **"false"** or remove the variable from the **AddonDeploymentConfig** resource.